DICE 汤森教仪

DICE 白熊

| 白熊人工智能实训平台用户手册

启东汤森教学仪器有限公司

www.dice.com.cn

目录

1、了解白熊人工智能实验平台.....11
1.1、人工智能背景与发展.....11
1.1.1 人工智能发展背景.....11
1.1.2 人工智能的定义.....12
1.1.3 人工智能的发展.....13
1.1.4 人工智能的依托.....14
1.1.5 人工智能的应用.....15
1.1.6 人工智能存在的问题.....17
1.1.7 人工智能的前景及展望.....17
1.2、白熊人工智能实验平台介绍.....18
1.2.1、白熊人工智能实验平台功能介绍.....18
1.2.2、白熊硬件资源介绍.....18
1.2.3、Jetson Nano 主板功能介绍.....20
2、开发环境搭建.....26
2.1、白熊 AI 平台系统.....26
2.1.1、Jetson Nano 系统安装.....26
2.1.2、Jetson Nano 的无头模式.....28
2.1.3、查看 IP 地址.....29
2.1.4、结束出厂大程序.....30
2.2、常用工具的使用.....31
2.2.1、XShell SSH 工具使用.....31
2.2.2、FileZilla 文件传输软件使用.....35
2.2.3、远程 NoMachine 桌面服务访问.....40
2.3、JupyterLab 软件开发平台使用.....44
2.3.1、JupyterLab 简介.....44
2.3.2、启动 Jupyter Lab 编程环境.....46
2.3.3、Jupyter Lab 编程常见操作.....47
3、AI 人工智能机械臂开发知识准备.....58
3.1、串口总线介绍.....58
3.1.1、传统 PWM 舵机存在的主要问题如下所示：.....59
3.1.2、串行总线舵机有如下优势：.....61
3.2、串口总线舵机上位机软件使用.....63
3.2.1、串口连接.....64
3.2.2、串口连接.....65
3.2.3、修改舵机 ID.....66
3.2.4、舵机控制模式选择.....67

3.2.5、修改自定义参数.....68
3.2.6、上位机参数说明.....69
3.2.7、角度控制模式.....73
3.2.8、多圈角度模式.....74
3.2.9、轮式模式控制.....74
3.2.10、阻尼模式.....75
3.3、UART 串行总线舵机通信协议.....75
3.3.1、串口通信配置.....75
3.3.2、数据类型说明.....76
3.3.3、舵机控制指令.....76
3.3.4、数据包.....77
3.4、Jetson Nano 的 AI 计算优势.....78
3.4.1、深度学习推理基准.....79
3.4.2、多流视频分析.....81
3.4.3、Hello AI World.....82
3.5、OpenCV 计算机视觉库介绍.....84
3.5.1、OpenCV 的起源背景.....84
3.5.2、OpenCV 简介.....85
3.6、TensorFlow AI 框架介绍.....88
3.6.1、TensorFlow 简介.....88
3.6.2、TensorFlow 的特征.....89
3.3.3、TensorFlow 中的概念.....90
3.3.4、TensorflowSharp 中的几类主要变量.....94
3.7、PyTorch AI 框架介绍.....95
3.7.1、什么是 PyTorch.....95
3.7.2、PyTorch 的优点.....96
3.4.3、PyTorch 的架构.....96
3.8、Jetson Nano 使用 Docker 容器技术.....98
3.8.1、Docker 容器技术简介.....98
3.8.2、Jetson Nano Docker 测试.....98
3.8.3、Jetson Nano Docker 环境搭建及使用.....99
4、传感器项目实践.....102
传感器实验操作说明：.....102
4.1、双色 LED 实验.....102
4.1.1、硬件介绍：.....102
4.1.2 实验原理：.....102
4.1.3 实验步骤：.....103
4.1.4、python 代码：.....105
4.2、轻触开关按键实验.....107

4.2.1、硬件介绍.....107
4.2.2、实验原理.....107
4.2.3、实验步骤.....107
4.2.4、python 代码.....109
4.3、继电器实验.....111
4.3.1、硬件介绍.....111
4.3.2、实验原理.....111
4.3.3、实验步骤.....113
4.3.4、python 代码.....114
4.4、U 型光电传感器实验.....116
4.4.1、硬件介绍.....116
4.4.2、实验原理.....116
4.4.3、实验步骤.....117
4.4.4、python 代码.....118
4.5、 PCF8591 模数转换器实验.....121
4.5.1、硬件介绍.....121
4.5.2、实验原理.....121
4.5.3、实验步骤.....124
4.5.4、python 代码.....127
4.6、 PS2 操作杆实验.....129
4.7、 霍尔传感器实验.....134
4.7.1、硬件介绍.....134
4.7.2、实验原理.....135
4.7.3、实验步骤.....135
4.7.4、python 代码.....137
4.8、 光敏传感器实验.....139
4.8.1、硬件介绍.....139
4.8.2、实验原理.....139
4.8.3、实验步骤.....139
4.8.4、python 代码.....141
4.9、 火焰报警实验.....143
4.9.1、硬件介绍.....143
4.9.2、实验原理.....143
4.9.3、实验步骤.....143
4.9.4、python 代码.....145
4.10、 气体传感器检测.....147
4.10.1、硬件介绍.....147
4.10.2、实验原理.....147
4.10.3、实验步骤.....147

4.10.4、python 代码.....149
4.11、 触摸开关传感器实验.....152
4.11.1、硬件介绍.....152
4.11.2、实验原理.....152
4.11.3、实验步骤.....152
4.11.4、python 代码.....154
4.12、 超声波传感器距离检测.....157
4.12.1 硬件介绍.....157
4.12.2、实验原理.....157
4.12.3、实验步骤.....158
4.12.4、python 代码.....160
4.13、 旋转编码器实验.....162
4.13.1、硬件介绍.....162
4.13.2、实验原理.....162
4.13.3、实验步骤.....164
4.13.4、python 代码.....166
4.14、 红外避障传感器实验.....169
4.14.1、硬件介绍.....169
4.14.2、实验原理.....169
4.14.3、实验步骤.....170
4.14.4、python 代码.....171
4.15、 BMP180 气压传感器实验.....173
4.15.1 硬件介绍.....173
4.15.2、实验原理.....173
4.15.3、实验步骤.....176
4.15.4、python 代码.....177
4.16、 直流电机模块实验.....179
4.16.1、硬件介绍.....179
4.16.2、实验原理.....179
4.16.3、实验步骤.....180
4.16.4、python 代码.....181
4.17、 步进电机驱动模块实验.....184
4.17.1、硬件介绍.....184
4.17.2、实验原理.....184
4.17.3、实验步骤.....186
4.17.4、python 代码.....187
4.18、 循迹传感器实验.....190
4.18.1、硬件介绍.....190
4.18.2、实验原理.....190

4.18.3、实验步骤.....190
4.18.4、python 代码.....192
4.19、 MPU6050 陀螺仪加速度传感器.....194
4.19.1、硬件介绍.....194
4.19.2 实验原理.....194
4.19.3、实验步骤.....196
4.19.4、python 代码.....198
5、机械臂基础控制和实践.....201
机械臂基础控制实验操作说明：.....201
5.1、蜂鸣器控制实验.....201
5.1.1、硬件说明.....201
5.1.2、实验原理.....201
5.1.3、实验步骤.....202
5.1.4、python 代码.....204
5.2、用户按键实验.....205
5.2.1、硬件说明.....205
5.1.2、实验原理.....205
5.1.3、实验步骤.....205
5.2.4、python 代码.....207
5.3、OLED 实验.....208
5.3.1、硬件说明.....208
5.3.2、实验原理.....208
5.3.3、实验步骤.....210
5.3.4、python 代码.....211
5.4、控制单个舵机.....213
5.4.1 总线舵机介绍.....213
5.4.2 总线舵机驱动库说明.....213
5.4.3 实验步骤.....216
5.4.4、python 代码.....217
5.5、同时控制 6 个总线舵机动作.....219
5.5.1、机械臂总线舵机介绍.....219
5.5.2、实验步骤.....219
5.5.2、python 代码.....220
5.6、读取舵机当前的位置.....222
5.6.1、机械臂舵机位置反馈介绍.....222
5.6.2、实验步骤.....222
5.6.2、python 代码.....222
5.7、机械臂关节标定实践.....225
5.7.1、机械臂关节标定介绍.....225

5.7.2、机械臂关节标定计算表达.....225
5.7.3、实验步骤.....226
5.7.4、python 代码.....228
5.8、机械臂运动学正逆解算.....230
5.8.1、机械臂运动学正逆解算介绍.....230
5.8.2、机械臂的运动学分析.....230
5.8.3、python 代码.....232
5.9、机械臂关节弧度及末端姿态控制.....241
5.9.1、机械臂关节弧度及末端姿态控制介绍.....241
5.9.2、实验步骤.....241
5.9.3、python 代码.....242
5.10、大自然搬运工（需更新）.....246
5.10.1、实验简介.....246
5.10.2、python 代码.....247
5.11、方块堆叠（需更新）.....251
5.11.1、实验简介.....251
5.11.2、实验步骤.....251
5.11.3、python 代码.....252
5.12、两台机械臂同步运作（需更新）.....256
5.12.1、通讯原理.....256
5.12.2、实验前准备.....256
5.12.3、实验步骤.....256
5.12.4、python 代码.....257
6、AI 基础图像处理.....262
6.1、安装和使用 Matplotlib、Pyplot 和 Numpy.....262
6.1.1、介绍.....262
6.1.2、安装 Matplotlib、Pyplot 和 Numpy 库.....262
6.1.3 使用 Matplotlib、Pyplot 和 Numpy 库.....263
6.1.4、python 代码.....264
6.2、在 OpenCV 中运行摄像头.....265
6.2.1、介绍.....265
6.2.2、摄像头的使用.....265
6.2.3、python 代码.....267
6.3、OpenCV 读取、写入和显示图像.....269
6.3.1、介绍.....269
6.3.2、程序位置.....269
6.3.3、读入图像.....269
6.3.4、显示图像.....269
6.3.5、保存图像.....270

6.3.6、使用 Matplotlib 显示图像.....270
6.4、OpenCV 读取、显示和保存视频.....272
6.4.1、介绍.....272
6.4.2、程序位置.....272
6.4.3、用摄像头捕获视频.....272
6.4.4、从文件播放视频.....274
6.4.5、保存视频.....275
6.5、OpenCV 图像质量和像素操作.....278
6.5.1、OpenCV 压缩图像.....278
6.5.2、OpenCV 像素操作.....279
6.6、OpenCV 几何变换基础.....282
6.6.1、OpenCV 图片缩放.....282
6.6.2、OpenCV 图片剪切.....285
6.6.3、OpenCV 图片平移.....286
6.6.4、OpenCV 图片镜像.....289
6.7、OpenCV 几何变换进阶.....293
6.7.1、OpenCV 仿射变换.....293
6.7.2、OpenCV 图片旋转.....295
6.7.3、OpenCV 透视变换.....299
6.8、OpenCV 图片处理.....301
6.8.1、OpenCV 灰度处理.....301
6.8.2、OpenCV 图像二值化.....307
6.8.3、OpenCV 边缘检测.....309
6.9、OpenCV 绘制文字线段.....314
6.9.1、OpenCV 线段绘制.....314
6.9.2、OpenCV 矩形圆形绘制.....315
6.9.3、OpenCV 文字图片绘制.....318
6.10、OpenCV 图像美化 1.....321
6.10.1、OpenCV 彩色图片直方图.....321
6.10.2、OpenCV 直方图均衡化.....323
6.11、OpenCV 图像美化 2.....328
6.11.1、OpenCV 图片修补.....328
6.11.2、OpenCV 亮度增强.....330
6.12、OpenCV 图像美化 3.....333
6.12.1、OpenCV 磨皮美白与双边滤波.....333
6.12.2、OpenCV 高斯均值滤波.....335
7、AI 视觉综合项目（部分实验需更新）.....340
7.1、使用 OpenCV 进行颜色检测.....340
7.1.1、原理介绍.....340

7.1.2、OpenCV 颜色识别 343
7.1.3、OpenCV 颜色物体追踪 346
7.2、OpenCV 中使用 Haar 分类器进行特征检测 351
7.2.1、OpenCV 脸部和眼睛检测 351
7.2.2、OpenCV 实现行人检测 356
7.2.3、OpenCV 实现汽车检测 358
7.3、OpenCV 进行车牌检测 362
7.3.1、OpenCV 进行车牌检测 362
7.4、OpenCV 定位物体的实时位置 366
7.4.1、OpenCV 定位物体的实时位置 366
7.5、OpenCV 运动检测与追踪 371
7.5.1、OpenCV 运动检测与追踪 371
7.6、OpenCV 二维码识别 375
7.6.1、OpenCV 二维码识别 375
7.7、基于 Dlib 玩转人脸身份识别 380
7.7.1、OpenCV 二维码识别 380
7.8、安装 NVDIA 对象检测和推理工具 393
7.8.1、OpenCV 二维码识别 393
7.9、初识深度神经网络及卷积神经网络图像识别 396
7.9.1、人工智能、机器学习与深度学习 396
7.9.2、卷积神经网络（CNN） 399
7.9.3、卷积神经网络经典模型 AlexNet 及 GoogleNet 403
7.9.4、基于 GoogleNet 模型图像识别实践 405
7.10、基于 Jetson Nano 识别和定位图像中感兴趣的对象（需更新） 408
7.10.1、基于 Jetson Nano 识别和定位图像中感兴趣的对象 408
7.11、基于 Pytorch 的手势识别、表情识别及脸部特征识别（需更新） 412
7.11.1、ResNet-18 网络 412
7.12、手势识别固定动作 413
7.12.1、手势识别说明 413
7.12.2、手势识别固定动作 420
7.12.3、手势识别固定动作 428
7.13、颜色识别抓取积木 438
7.13.1、颜色校准 438
7.13.2、颜色识别抓取积木 444
7.14、你放我抓（需更新） 454
7.14.1、你放我抓 454
7.15、垃圾分拣（需更新） 457
7.15.1、单个垃圾分拣（需更新） 457
7.16、目标追踪（需更新） 461

7.16.1、颜色追踪与学习追踪.....461
8.AI 听觉综合项目.....467
8.1、声卡设置.....467
8.2、在线语音合成.....470
8.3、语音听写流式.....475
8.4、图灵机器人.....481
8.5、AIUI.....483
8.6、VAD 语音活动检测.....486
8.7、小薇机器人语音对话.....491
8.8、Snowboy 语音唤醒.....496
9、ROS 基础和运动学.....497
9.1、ROS 基础课程.....497
9.1.1、ROS 简介.....497
9.1.2、ROS 小海龟介绍.....497
9.1.3、【rosnode】节点信息.....498
9.1.4、ROS-rqt(QT 工具).....501
9.1.5、【rostopic】介绍.....505
9.1.6、【rostopic list】列出所有当前订阅和发布的话题.....506
9.1.7、rosmsg 消息介绍.....507
9.2、ROS 创建工程项目.....510
9.2.1、创建工作空间.....510
9.2.2、创建 package 功能包.....510
9.2.3、【CMakeLists.txt】介绍.....511
9.3、自定义消息.....514
9.3.1、新建自定义消息.....514
9.3.2、环境搭建.....515
9.3.3、编译.....516
9.4、Server 通讯.....518
9.5、机械臂 URDF 模型.....520
9.5.1、概述.....520
9.5.2、Links.....520
9.5.3、Joints.....521
9.5.4、launch 文件介绍.....523
9.5.5、URDF 可视化.....523
9.6、机械臂运动学正反解.....525
9.6.1、概述.....525
9.6.2、三维空间的位姿描述.....525
9.6.3、映射变换.....528
9.6.4、正解反解.....530

9.7、MoveIt 配置 537
9.7.1、配置流程 537
9.7.2、MoveIt 配置包 544
10、物联网综合实验 547

1、了解白熊人工智能实验平台

AI 人工智能涉及到知识比较广阔，需要有较强的数学基础、编程基础及相关的嵌入式开发能力。目前已有的书籍或者产品，要么过于理论，很容易让我们从入门到放弃，要么开发过于复杂，没有基础的人却而止步，综上所诉我们开发了全新的白熊人工智能实验平台

我们基于多维度学习实践平台，置身初学者角度，从基础单独的GPIO 扩展开始学习过渡到传感器实验项目再进入OpenCV、PyTorch、ROS 机器人系统、机器运动学、AI 视觉、AI 听觉、物联网应用等学习，从而学会AI 人工智能开发。

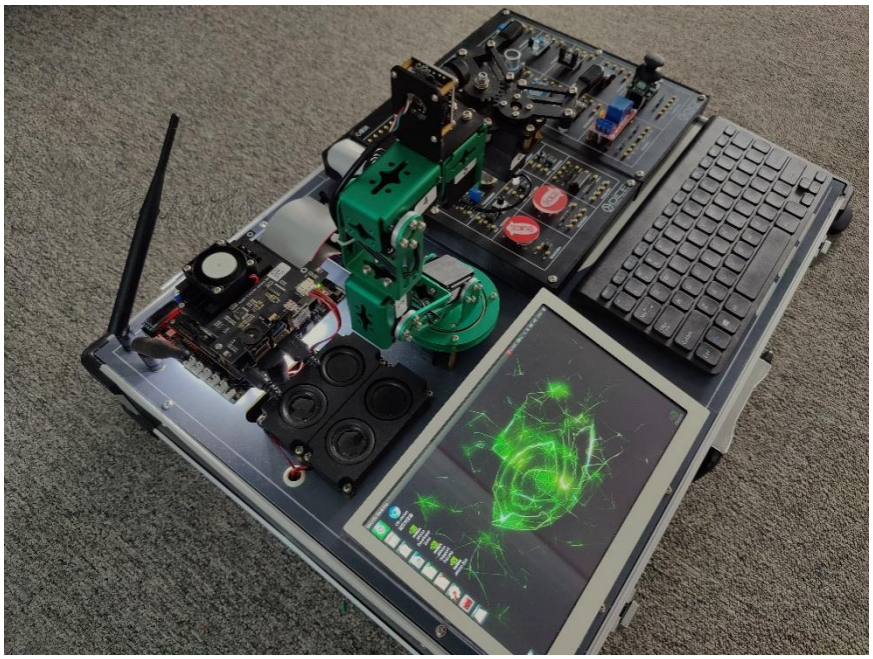


图1.1 白熊人工智能实验平台

1.1、人工智能背景与发展

1.1.1 人工智能发展背景

人工智能（Artificial Intelligence，简称AI）诞生于1956 年的美国，至今已接近七十年。它属于自然科学和社会科学的交叉。人工智能基于认知科学、哲学、不定性论的边缘科学研究项目，在上世纪80 年代中期人工神经网络取得重大进展，其成果大量应用于系统的感知，模型建立和控制。神经网络通过对输入输出样本的学习，不断地调整网络的权值和阈值，使网络实现给定的输入输出映射关系，并具有一定的学

习和自适应运用能力。智能技术是当前新技术、新产品、新产业的重要发展方向、开发策略和显著标志，借助大数据技术，尤其它在解决远程控制、故障诊断、非线性等问题上的优势，给机械系统、符号计算、模式识别的发展指明了方向。随着时代的发展及信息革命的到来，人工智能的研究领域日益拓宽，其内容逐步丰富，对人类发展有划时代的意义。

人工智能的迅速发展将深刻改变人类社会生活、改变世界。经过60多年的演进，特别是在移动互联网、大数据、超级计算、传感网、脑科学等新理论新技术以及经济社会发展强烈需求的共同驱动下，人工智能加速发展，进入新阶段，呈现出深度学习、跨界融合、人机协同、群智开放、自主操控等新特征。大数据驱动知识学习、跨媒体协同处理、人机协同增强智能、群体集成智能、自主智能系统成为人工智能的发展重点，受脑科学研究成果启发的类脑智能蓄势待发，芯片化硬件化平台化趋势更加明显，人工智能发展进入新阶段。当前，新一代人工智能相关学科发展、理论建模、技术创新、软硬件升级等整体推进，正在引发链式突破，推动经济社会各领域从数字化、网络化向智能化加速跃升。

人工智能成为国际竞争的新焦点。人工智能是引领未来的战略性技术，世界主要发达国家把发展人工智能作为提升国家竞争力、维护国家安全的重大战略，加紧出台规划和政策，围绕核心技术、顶尖人才、标准规范等强化部署，力图在新一轮国际科技竞争中掌握主导权。当前，我国国家安全和国际竞争形势更加复杂，必须放眼全球，把人工智能发展放在国家战略层面系统布局、主动谋划，牢牢把握人工智能发展新阶段国际竞争的战略主动，打造竞争新优势、开拓发展新空间，有效保障国家安全。

1.1.2 人工智能的定义

人工智能(Artificial Intelligence，简称AI) 是一门研究如何构造智能机器或智能系统，主要研究方向是模拟、延伸、扩展人类智能的。人们通过对外界事物进行长期实践做出一些归纳并对其用数值量化， 用传感设备和用严密的逻辑思维进行推理并干预其结果的差错， 通过逻辑计算然后再用于控制终端设备服务于受众，智能包含能力包括感知能力、记忆和思维能力、学习和自适应的能力，行为能力/ 涉及哲学、认知科学、数学、神经生理学、不定性论、计算机科学、控制论、信息论、仿生学、心理学等；人工智能产品也逐渐融入人类的生活中密不可分。

在人类与人工智能系统之间， 人类始终是主体， 人工智能系统则是人类求解问题的聪明工具。由于人工智能系统接受了人类所预设的求解目标和提供的专门知识， 就保证了人类主体与客体之间实现主客双赢的策略： 主体赢， 因为实现了主体的求解目标； 客体赢，因为遵守体现在知识中的客观规律。

人工智能系统工作的基本过程:

(1) 人类主体（隐性智慧）首先设定工作框架，即定义要解决的实际问题，提供已有知识，预设求解目标。

(2) 人工智能系统收集工作框架的信息，包括模型中的本体论信息。

(3) 通过信息获取（感知）单元，把本体论信息转换为认识论信息。

(4) 通过信息传递（通信）单元，把认识论信传送到信息处理单元。

(5) 通过信息处理（计算）单元，使认识论信息成为适用信息。

(6) 通过知识生成（认知）单元，根据求解问题的需要生成专门知识。

(7) 通过策略创建（决策）单元，把认识论信息、专门知识、求解目标转换为求解问题的智能策略。

(8) 通过策略传递（通信）单元，把智能策略传递到策略执行单元。

(9) 通过策略执行（控制）单元，把智能策略转换成为智能行为，反作用于实际问题。

(10) 如果智能行为反作用于实际问题的结果没有满意地实现预期的求解目标，就把误差作为新的信息反馈到信息获取（感知）单元，通过学习，补充新的知识，优化智能策略，改善执行效果，减小求解误差。

1.1.3 人工智能的发展

人工智能的传说可以追溯到古埃及， 但在历史的长河中大部分的传说都基于人们的假想，随着近代特别是二战之后世界格局的发展， 1946 年世界第一台计算机的诞生以来， 计算机在欧美国家得以迅速发展， 人工智能终可以辅以计算机系统来实现，技术已最终可以创造出机器智能， 1956 年在达特茅斯大学召开的会议上正式使用了“人工智能(Artificial Intelligence,AI) ”这个术语，人工智能领域的研究也从此正式开始， 从那以后，研究者们发展了众多理论和原理，人工智能的概念也随之扩展。随后的几十年中， 人们从问题求解、逻辑推理与定理证明、自然语言理解、博弈、自动程序设计、专家系统、学习以及机器人学等多个角度展开了研究， 已经建立了一些智能程度不同的人工智能系统， 例如能够求集成设计分析电路、字符计算、求解积分方程、合成人类自然语言，而进行基于字符的情报检索， 提供语音识别、触控识别的多模式接口输入模式， 应用于疾病诊断的专家系统以及控制太空飞行器和水下机器人。我们熟知的年初的谷歌"Alphago" 计算机在棋盘上击败了韩国国际象棋大师李世石就是比较突出的例子。

1.1.4 人工智能的依托

神经网络

神经网络（ artificial neural network. ANN）是设置输出的是一个灵感来自系统的模拟生物激励的输出。这里输出、输入都是标准化的量，输出是输入的非线性函数，其值可由连接各神经元的权重改变，以获得期望的输出值， 即所谓的训练过程。基于数值计算方法的神经网络， 将已有数据和已知系统模式作样本，存储在系统的缓存区域， 通过学习获得两者的映射关系， 实现了对人类经验思维的模拟。由于神经网络具有原则上容错、结构拓扑、联想、推测、记忆、自学习适应、并行上传下载和处理复杂模式的功能， 使其在工程实际存在着大量的多故障、多过程、突发性故障； 这里就能体现出一个庞大复杂机器和系统的监测及诊断中的作用了， 他们需要不断查询系统中故障的位置并向中央处理器报告突发错误的原因， 并能够提供简单算法的纠错功能， 当差错过大的时候这需要建立丢弃重传的机制，以保证系统的正常运行。在机械系统的应用方式有： 从模式识别角度应用神经网络作为分类器进行故障诊断； 从预测角度应用神经网络作为动态预测模型进行故障预测；利用神经网络极强的非线性特性。

不定性论

不定性论也称作模糊集理论（ Fuzzy Sets Theory ， FSN），人的认知世界包含大量的不确定元素，但是我们所接收的信息需要对所获信息进行一定的模糊化处理，以减少问题的复杂度。1965 年Zadeh 创立了模糊集理论，模糊逻辑可认为是多值逻辑的扩展， 人们利用在实验中发现的或者通过大量的独立重复试验得出的某个典型值作为一类事物的一个最优的或者折中的方案值来量化复杂的事物，这种模型能够完成传统数学方法难以做到的近似推理。是处理不确定性的一种很好的方法。目前基于多类电量测试信息模糊融合的模拟电路故障诊断方法已经提出。基于K 故障节点诊断法和最小标准差法的元件故障隶属函数构造方法,以及基于可测点电压与不同测试频率下电路增益的模糊信息融合诊断算法， 分别利用此两类测试信息及K 故障诊断法和最小标准差法，对电路进行初步诊断，再运用模糊变换及故障定位规则，得到融合的故障诊断结果。模拟实验结果表明,所提方法大大提高了机械系统故障定位的准确率。

大数据

大数据技术是继移动互联网和云计算技术的一个颠覆性的信息技术， 它可以让我们有一些数量巨大，种类繁多，高价值，低密度，快速变化的有效数据，低成本的存款取，检索，分类，流量统计。但是，这并不认为今天我们能够有效，经济地理解这些数据的巨大价值是隐藏的， 特别是如果这些数据是科学和经验的的社会法律代表了巨大的价值的意义。幸运的是，一些人工智能的理论，更实用的方法，已开始对大数据分析，并且做出了一些令人振奋的结果。

启发式搜索

遗传算法（ Genetic Algorithms ,GA） 和模拟退火算法（ SimulatedAnnealing ,SA）是近年来逐渐上升的两个启发式搜索，通过随机生成新的解决方案，同时保留良好的结果，避免陷入局部最小值，以达到全局最优解或最优解附近。遗传算法是解决优化问题的数值字符串集合通过遗传算子，即选择、交叉和变异操作优化数字字符串。SA邻近区域产生已知的解决方案的新的解决方案，并逐步减少邻近区域的大小，直到接近全局最优的解决方案。这两种方法可以用来解决任何的目标函数和约束优化问题。模糊神经网络在交流伺服系统控制中使用遗传算法比传统的PID 控制和快速响应，错误小，没有冲击和强烈的伺服性能等。仿真结果表明，遗传算法的模糊神经网络控制器的交流伺服控制系统大大减少响应超调系统，具有良好的抗干扰，伺服性。

深度学习

机器学习就是机器自生从周围环境中获取经验， 表现为在自身已有的知识上增强自身知识和能力，与人一样能够学习未知知识。机器学习的研究，主要是探究人类的学习理论、人脑的思维方式， 建立针对具体任务的学习方式。它在开发高智能机器上是一个研究点。人类有条件研究复杂信息的识别和处理过程。用计算机实现环境与空间的自动识别，包括文字、声音、人物、物体等等；开发智能机器的突破口也是， 也能为认识自身智能提供信息和依据。环境与空间的识别是信息的识别和处理过程的一个重要技术支持。

1.1.5 人工智能的应用

符号计算

符号计算是指解码数学表达式、方程时，而不是数值点的离散化，而是通过一系列的身份和数学定理的， 由推理和计算的分析结果。此计算是基于数值完全准确的表达及严格的扣除的基础上，将得到的准确的结果。当然，也有不足，产生符号没有额外的属性的变量x 的纯的形式（例如，以创建符号变量x，但没有指定其为正，或者它是一个实数）。为了与MATLAB版本兼容，x=sym('x','unreal')的功能和x=sym('x','clear') 一样。

符号变量符号变量计算和分析解决方案优势不一而足，例如，符号变量计算过程用π 就用π 的代表，而不是一个特定的近似值或。同理计算过程中自然不对数e 也就用e 表示，而不用其某个近似值代替，使用符号变量可以最大限度减少运营商由于舍入造成误差的计算过程。符号变量也便于操作的演示过程。

模式识别

模式识别是研究如何使机器感知， 视觉和听觉模型识别的主要研究。如目标识别，地形，影像，字体等。目前模式识别主要集中在图形识别和语音识别。图形识别，如识别打印和一些手写文字，虹膜识别技术，如指纹。其中检查白血细胞和癌细胞已进入实用阶段。各种语音信号的主要的语音识别分类。

专家系统

专家系统（ Expert System ， ES）是人工智能的主要分支之一。一个典型的专家系统由四部分组成：知识库，推理机，知识获取机制和人机界面。根据不同的专家系统的知识表示方法，可以分为基于规则和基于专家系统的框架；根据不同的推理方法，可分为正向推理和向后推理。在由产生式规则知识表示， 知识表示方面，一方面，它有利于现有的人工智能语言，而另一方面，它的表达使得人的心理逻辑，有利于知识的获取， 为人们接受， 使用知识表达应用框架已经越来越多。在诊断推理，主要表现在逻辑和推理模型的研究，在人工智能领域方面，有很多逻辑推理， 广泛应用于模糊逻辑推理的专家系统， 以减少对系统和复杂性故障机械系统的诊断可以产生很好的效果。研究和专家系统技术的应用是在机械工程的许多领域的发展，故障诊断，仿真，自动控制，工艺设计，生产计划，产品设计等前所未有的速度。随着研究工作的不断深入， 技术和先进制造技术的一些新的方法集成到机械工程的研究和专家系统技术的应用，不仅打造知识表示，知识库，知识获取和推理模型的关键技术等方面的研究取得了一定的成就， 也出现了一些新的集成专家系统， 如神经网络专家系统， 模糊专家系统，基于Internet的专家系统， CAD专家系统，CAPP专家系统等燃灯的综合利用率的专家系统的透明度、柔韧性；并具有处理不确定知识的能力，机械工程专家系统的更广泛的应用领域的特性。

机器翻译

协同翻译方面，翻译是机器翻译，顾名思义，这里的对象一般是指英文的中文自然语言。通常，未能翻译分为两个阶段一是“语言”的理解，第二种是“生成翻译”。成功的翻译家，当然需要善于原文译文：语言与语言，更重要的是理解内容。第一次实质性的机会，因此研究协同翻译方面远远超过了计算机处理语言的范围,而是人工智能研究的中介目标。事实上，也可以说是协同翻译方面50 年研究中唯一的自然语言处理的一个领域的实用化。协同翻译方面一般分为三种方式：转换方式，中间语言方式和直接的方式。所谓转换方式，首先，分析原文，原文的语法构造乃至语义结构。然后，事先准备的转换规则将原文的结构重组的成长目标的原因已经，最后生成。所谓中间语言方式，是原语言文章转化为了分析，任何语言的概念结构，然后直接特别是概念结构生成目标语言，所谓直接方式，分析前的语言一起去生成目标语言，这种方式结构的中心。

1.1.6 人工智能存在的问题

早在计算机诞生的初期，就提出了用计算机完成智能翻译的想法，由此可知任内对智能的研究具有悠远的历史。但是人类在智能的研究方面遇到诸多的瓶颈，目前所面临的问题主要有机器翻译、人工智能的安全问题等。

机器翻译所面临的问题

机器翻译所面临的问题仍然是翻译准确率不高。歧义性问题一直存在， 是自然语言理解中的一大难关。就以汉语为例， 同一个意思可一用几种甚至几十种不同的语句表达， 同样句子在不同的情况下使用，也拥有不同的含义。所以，要消除歧义就要对句子的构成进行科学规划，对每一个句子进行科学的分析，找出造成歧义的词语，并且分析这些词语在句中导的准确含义。另外，即便对原文有了一定的理解，机器理解语句的含义也不一定准确，而且理解的意义在机器上变现出来也存在问题。目前的机器不能像人一样在时间上增加理解力，没有学习，没有记忆，没有归纳。造成这种结果的原因是计算机本身硬件和软件问题。现在自然语言理解的研究技术很不完整，大多数研究被关在语言这一领域里，并没有对人类的理解问题做深入研究。

人工智能的安全问题

人工智能任然在探究的初期阶段， 有人认为如果让机器像人一样智能， 是很危险的。这样机器可能反抗人类， 原因有很多可能是机器会像人一样犯罪， 对人的统治发起反抗， 而且有可能会统治人类， 当然人类是不会允许这种时间发生的。这种人工智能的安全问题在很多部科幻电影中都展现过， 最主要的是同不同意让人工智能拥有自主意识， 如果使人工智能拥有自主意识， 也就意味着机器具有与人同等的创造性、自我保护思想、情感和自发行为。

1.1.7 人工智能的前景及展望

从智能手机、自动驾驶汽车到医疗机器人，人工智能革命已经到来。人工智能让互联网搜索更加灵敏； 将文本从一种语言翻译成另一种语言； 在拥挤的交通中推荐最畅通的线路； 帮助识别信用卡诈骗等。虽然很多时候我们甚至没有意识到它的存在，但我们的生活却因它悄悄改变。人们总是趋向于安逸的生活， 人工智能的出现满足了人们许多的需求， 这会导致人们满足于享受当前的生活而忘记许多自己的本能。但人们还得需要用自己的双手去创造物质文明， 过度的依赖人工智能设备会让我们意志力淡泊，丧失生活的乐趣， 如果能把智能设备用到改善人们生活方式，提高生活质量的领域去， 不管是社会资源的提供者、生产者还是消费者都会从中受益。

1.2、白熊人工智能实验平台介绍

1.2.1、白熊人工智能实验平台功能介绍

白熊人工智能实验平台是由杭电人工智能研究所和启东汤森教学仪器有限公司联合研制的一款适合高校人工智能教学的全功能平台。白熊AI平台以Jetson Nano为主控，以主流的Jupyter Lab为开发工具，使用Python3为主要编程语言，配合Ubuntu18.04系统、OpenCV图像处理库、TensorFlow AI框架、PyTorch AI框架，将强大的AI力量带给了白熊AI平台。

白熊AI平台教学功能包括：基础的GPIO 扩展、传感器、OpenCV、PyTorch、ROS 机器人系统、机器运动学、AI 视觉、AI 听觉、物联网等。软/硬件一体的面向AI人工智能方向，由浅入深学习人工智能课程，为教师和学生带来更好的教学学习体验。开放的GPIO和模块化设计可以帮助客户在需要的时候开发自己的功能，是AI功能开发或完成毕业设计的好帮手。

图1.2 白熊学习进程

1.2.2、白熊硬件资源介绍

白熊AI平台的硬件主要分为6部分：主控Nvidia Jetson Nano、机械臂及控制主板（安装有USB摄像头）、AI听觉主板、传感器实验模块、10.1英寸显示屏以及无线键鼠。主控将放到下个章节单独介绍。

机械臂

机械臂为六自由度机械臂，采用智能串行总线舵机，采用阳极氧化处理铝合金，更耐磨。通过ROS机器人操控系统，简化了6自由度串行总线舵机复杂运动控制。可以完成Jetson Nano对机械臂的控制的实验，配合AI视觉完成颜色分拣、垃圾分拣等实验。

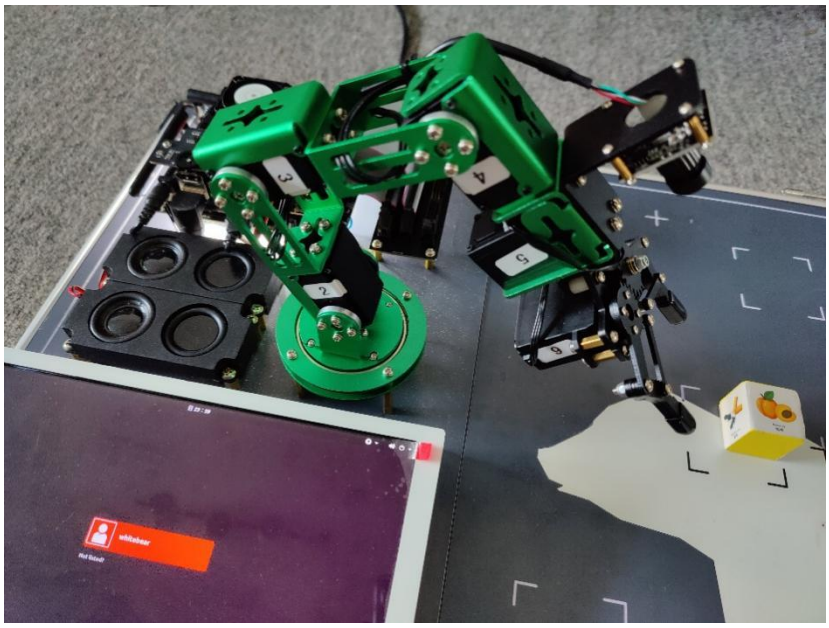


图1.3 机械臂实物照片

AI听觉主板

AI听觉主板通过USB接口与Jeson Nano主板相连，支持播放和录音，板载麦克风和耳机喇叭接口。采用 SSS1629 音频芯片，板载两个高质量 MEMS 硅麦克风，可左右声道录音，音质更佳，板载3.5mm耳机接口，板载双通道喇叭接口，可以直接驱动扬声器。搭载Snowboy唤醒词检测引擎，中文采用科大讯飞语音服务，可以完成在线语音合成、语音听写、VAD语音检测、图灵机器人、语音对话、语音唤醒等AI听觉实验。

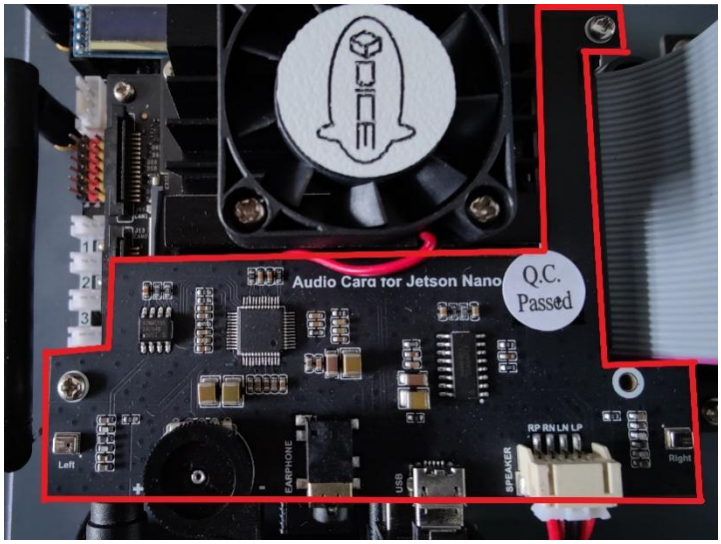


图1.4 AI听觉主板实物照片

传感器模块

使用传感器模块可以使学习者更加快速的入门Jetson Nano的GPIO控制，从基础入手，完成实验项目。上面布局的传感器模块基本都是项目常用的传感器，为后续参加比赛或完成毕业设计打下坚固的基础。同时引出的GPIO和可移动的模块也是我们后续的使用和开发更加方便。

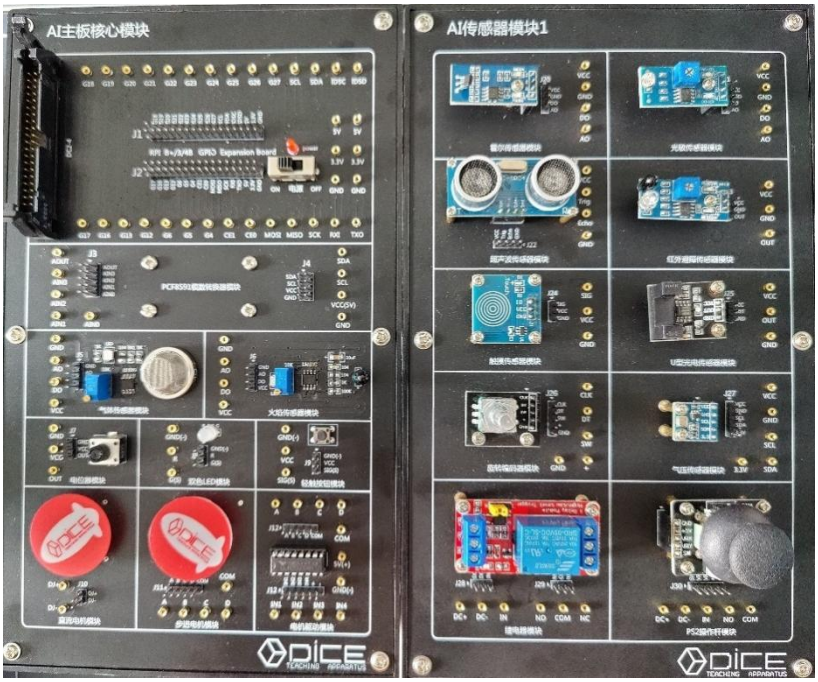


图1.5 传感器模块实物照片

AI主板转接卡

AI主板转接卡是为了将Jetson Nano的GPIO引出。我们可以通过这块转接卡和40芯的排线，将GPIO输出至机械臂主板来控制机械臂，或将GPIO引至“AI主板核心模块”来把Jetson Nano的GPIO开放出来，以便我们完成传感器实验或自由搭建电路。

虽然AI主板转接卡的3组40芯排线座是相通的，但为了能区分模块，我们将DC2-1定义为机械臂主板的排线座，将DC2-2定义为Jetson Nano引出的排线座，将DC2-3定义为AI主板核心模块的排线座。

1.2.3、Jetson Nano主板功能介绍

图 1.7 Nvidia Jetson Nano 图片

Jetson Nano 是英伟达为了借鉴 Raspberry Pi(树莓派）的开源硬件，甚至可以直接调用树莓派的 GPIO 控制库，而无需修改，和树莓派有一个较好的兼容性，英伟达也利用了自身强大的 GPU 优势，从而可以用来开发的用于深度学习或者计算机视觉开发的开发板，开发板集成了 128 核 Maxerll GPU,四核 ARM A57 64bit CPU 和 4GLPDDR4 内存，支持 MIPI CSI-2 接口的摄像头和 PCIE GEN2 高速 1/O 口的无线网卡，Jetson Nano 运行的系统采用的是 Ubuntu 系统，可以处理 472GFLOPS 的 FP16(半浮点数），功耗可以选择 5W 或者 10W 之间。拥有完整的 GPU 让它成为构建人工智能和机器学习项目最理想的选择。

核心配置

GPU:	128 核 Maxwell
CPU:	四核 ARM®A57@1.43Ghz
显存:	4GB 64 位 LPDDR4 内存
存储:	Micro SD 卡槽（支持至少 16GB 内存 TF 卡插入使用）
视频编码:	4K @ 30 | 4x 1080p @ 30 | 9x720p @ 30(H.264/H.265)
视频解码:	4K @ 60 | 2x 4K @ 30 | 8x 1080p @30 | 18x 720p @ 30|(H.264/H.265)
摄像头接口:	2 个 MPI CSI-2 DPHY 通道
网络连接:	千兆以太网，M.2 key E
视频:	HDMI 和 DP
USB:	4 个 USB3.0、USB2.0 Micro-B
其他:	GPIO、I2C、I2S、SPI、UART
尺寸:	100mm x 80 mm x 29 mm

新发布的 JetPack 4.4 SDK 为基于 Ubuntu 18.04 的 Jetson Nano 提供了完整的桌面 Linux 环境，NVIDIA L4T 提供引导加载程序、Linux 内核、必要固件、NVIDIA 驱动、示例文件系统等内容。

JetPack 4.4 Highlights 包含具有以下亮点的 L4T 32.4.3：

- 支持使用 Debian 包管理工具升级 JetPack 和 L4T
- 支持用于 Jetson AGX Xavier 和 Jetson Xavier NX 的通用时间戳引擎（GTE）
- 通过 actmon 提供用于视频图像合成器（VIC）的动态频率调整（DFS）支持
- 包含 SE（安全引擎）示例，可用于展示 Jetson TX2 系列、Jetson AGX Xavier 和 Jetson Xavier NX 模块上由硬件支持的身份验证和加密功能
- 包含同时融合多个 Jetson 模块的实用程序
- 在初次启动 Jetson Xavier NX 和 Jetson Nano 开发者套件并进行初始配置期间，提供在 microSD 卡上指定应用分区大小的选项
- 支持 Vulkan 1.2

该 SDK 还包括本机安装流行的功能开源机器学习（ML）框架，如 TensorFlow，PyTorch，Caffe，Keras 和 MXNet，以及计算机视觉和机器人开发的框架，如 OpenCV 和 ROS。

完全兼容这些框架和 NVIDIA 领先的 AI 平台，可以比以往更轻松地将基于 AI 的推理工作负载部署到 Jetson。Jetson Nano 为各种复杂的深度神经网络（DNN）模型提供实时计算机视觉和推理。这些功能支

持多传感器自主机器人，具有智能边缘分析的物联网设备和先进的 AI 系统。甚至转移学习也可以使用 ML 框架在 Jetson Nano 上本地重新训练网络。

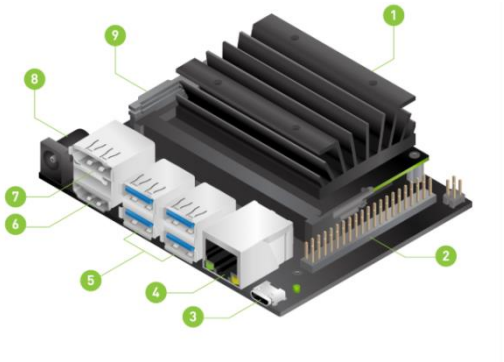


图 1.8 Jeson Nano 硬件外设

① microSD 卡插槽用于系统存储卡
② 40 针扩展接头（兼容 Raspberry 生态）
③ microUSB 端口，用于 5V 电源输入或用于数据
④ 千兆以太网端口
⑤ USB 3.0 端口（x4）
⑥ HDMI 输出端口
⑦ DisplayPort 连接器
⑧ DC Barrel 插孔，用于 5V 电源输入 3A
⑨ MIPI CSI 摄像机连接器（x2）

Jetson Nano 的 GPIO 说明

Jetson Nano 的 GPIO 引脚为 40 脚，GPIO 脚位说明可参考下图，基本上和 Raspberry Pi 的脚位差不多。

Sysfs GPIO	Name	Pin	Pin	Name	Sysfs GPIO
	3.3 VDC *Power*	1	2	5.0 VDC *Power*	
	I2C_2_SDA *I2C Bus 1*	3	4	5.0 VDC *Power*	
	I2C_2_SCL *I2C Bus 1*	5	6	GND	
gpio216	AUDIO_MCLK	7	8	UART_2_TX */dev/ttyTHS1*	
	GND	9	10	UART_2_RX */dev/ttyTHS1*	
gpio50	UART_2_RTS	11	12	I2S_4_SCLK	gpio79
gpio14	SPI_2_SCK	13	14	GND	
gpio194	LCD_TE	15	16	SPI_2_CS1	gpio232
	3.3 VDC *Power*	17	18	SPI_2_CS0	gpio15
gpio16	SPI_1_MOSI	19	20	GND	
gpio17	SPI_1_MISO	21	22	SPI_2_MISO	gpio13
gpio18	SPI_1_SCK	23	24	SPI_1_CS0	gpio19
	GND	25	26	SPI_1_CS1	gpio20
	I2C_1_SDA *I2C Bus 0*	27	28	I2C_1_SCL *I2C Bus 0*	
gpio149	CAM_AF_EN	29	30	GND	
gpio200	GPIO_PZ0	31	32	LCD_BL_PWM	gpio168
gpio38	GPIO_PE6	33	34	GND	
gpio76	I2S_4_LRCK	35	36	UART_2_CTS	gpio51
gpio12	SPI_2_MOSI	37	38	I2S_4_SDIN	gpio77
	GND	39	40	I2S_4_SDOUT	gpio78

图 1.9 GPIO 脚位说明

Jetson Nano 预设提供四种模式：BOARD、BCM、CVM 和 TEGRA_SOC，其中 BOARD 和 BCM 常用在 Python 中，BOARD 代表板子上显示的编号，Jetson Nano 背部则印有对应的 BCM 编号，左下为 pin1，右下为 pin2，以此类推。

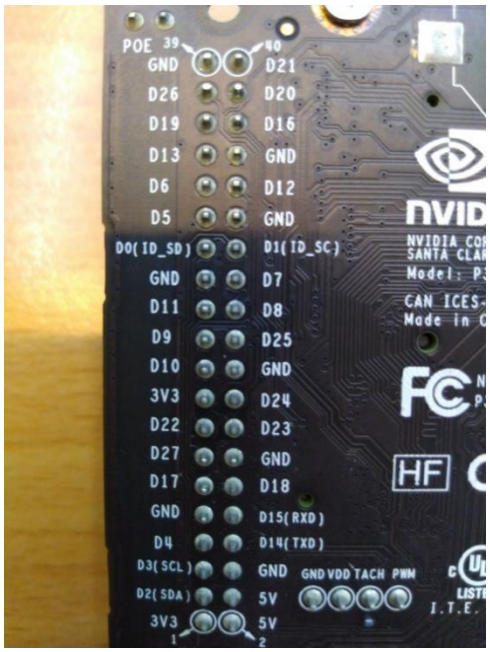


图 1.10 背部 BCM 编码

为求方便，您也可使用下图对照。

BCM	Name	Pin	Pin	Name	BCM
3V3	**3.3 VDC** *Power*	1	2	**5.0 VDC** *Power*	5V
2	**I2C_2_SDA** *I2C Bus 1*	3	4	**5.0 VDC** *Power*	5V
3	**I2C_2_SCL** *I2C Bus 1*	5	6	**GND**	GND
4	**AUDIO_MCLK**	7	8	**UART_2_TX** */dev/ttyTHS1*	14
GND	**GND**	9	10	**UART_2_RX** */dev/ttyTHS1*	15
17	**UART_2_RTS**	11	12	**I2S_4_SCLK**	18
27	**SPI_2_SCK**	13	14	**GND**	GND
22	**LCD_TE**	15	16	**SPI_2_CS1**	23
3V3	**3.3 VDC** *Power*	17	18	**SPI_2_CS0**	24
10	**SPI_1_MOSI**	19	20	**GND**	GND
9	**SPI_1_MISO**	21	22	**SPI_2_MISO**	25
11	**SPI_1_SCK**	23	24	**SPI_1_CS0**	8
GND	**GND**	25	26	**SPI_1_CS1**	7
0	**I2C_1_SDA** *I2C Bus 0*	27	28	**I2C_1_SCL** *I2C Bus 0*	1
5	**CAM_AF_EN**	29	30	**GND**	GND
6	**GPIO_PZ0**	31	32	**LCD_BL_PWM**	12
13	**GPIO_PE6**	33	34	**GND**	GND
19	**I2S_4_LRCK**	35	36	**UART_2_CTS**	16
26	**SPI_2_MOSI**	37	38	**I2S_4_SDIN**	20
GND	**GND**	39	40	**I2S_4_SDOUT**	21

图 1.11 BCM 对照表

对于我们给大家提供的镜像是默认安装好 GPIO 函数控制库的，大家可以不需要再进行安装，如果您想了解安装步骤，您可以按照以下步骤操作:

(1)在终端中输入以下指令，安装 Jetson Nano 官方提供的 GPIO 函数库。

```
sudo apt-get install python-pip
sudo pip install Jetson.GPIO
```

(2)设置对用户和用户组的权限

```
sudo groupadd -f -r <gpio_group_name>
sudo usermod -a -G <gpio_group_name> <user_name>
```

注：gpio_group_name：是自己定义的用户组名，user_name，为自定义的用户名，根据自己的情况来配置。

2、开发环境搭建

2.1、白熊AI平台系统

2.1.1、Jetson Nano 系统安装

镜像的烧入首先需要准备一个microSD卡，microSD卡推荐使用（32G或者64G）的10 class卡，还需要一个USB读卡器和一台计算机。

使用我们资料盘中的镜像文件进行安装（白熊人工智能实验平台资料->开发环境->系统镜像->whitebear_nano_xxxxxx.img）

下面的操作步骤将在windows系统中完成，其他系统中如何烧录镜像将另作介绍。

第1步：安装烧录工具：安装并启动balenaEtcher（白熊人工智能实验平台资料->开发环境->镜像烧录工具->镜像烧录工具balena-etcher->balenaEtcher-Setup-xxxx.exe）

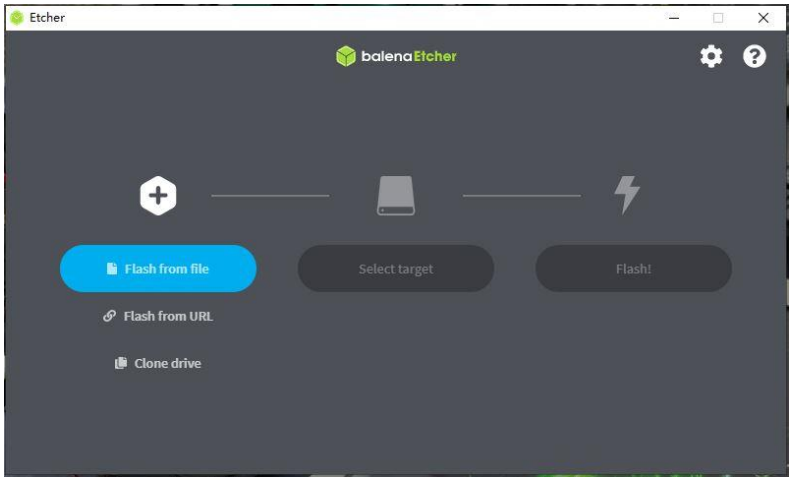


图2.1 软件界面

第2步：点击<Flash from file>，选择镜像文件

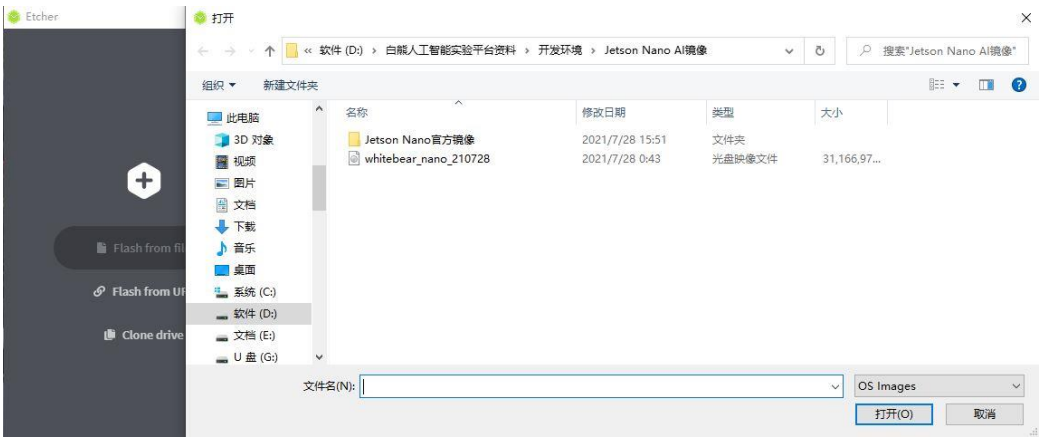


图2.2 选择镜像文件

第3步：点击<Select target>选择microSD卡

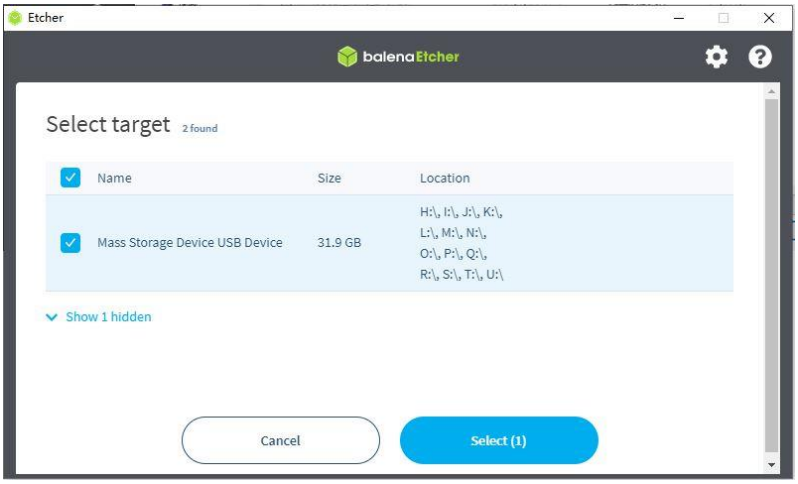


图2.3 选择microSD卡

第4步：点击<Flash>烧录镜像

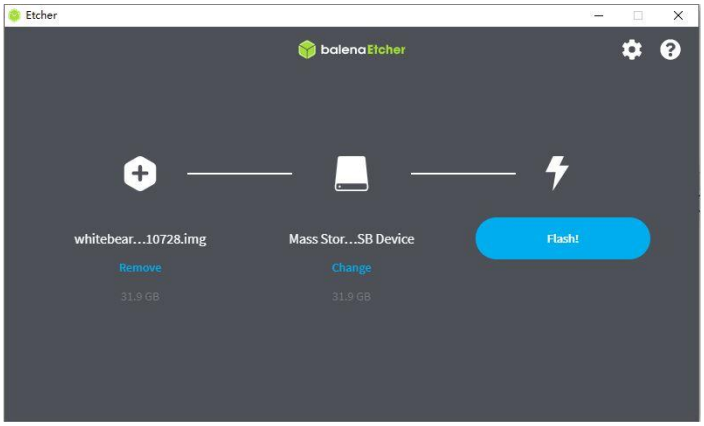


图2.4 烧录镜像

注：在烧录镜像的过程中，因为windows系统无法读取其内容，系统会跳出弹框询问是否格式化驱动器，直接点击'**取消**'即可。

1、 系统烧录完成之后，我们只需将microSD卡从windows端物理卸下，便可使用。

接着我们将已经烧录完系统的microSD卡插到白熊AI实验平台的Jetson Nano主板的卡槽内，再打开电源即可使用。卡槽位于实验箱右上角主板位置的上方，在散热器下方那块主板的背部。

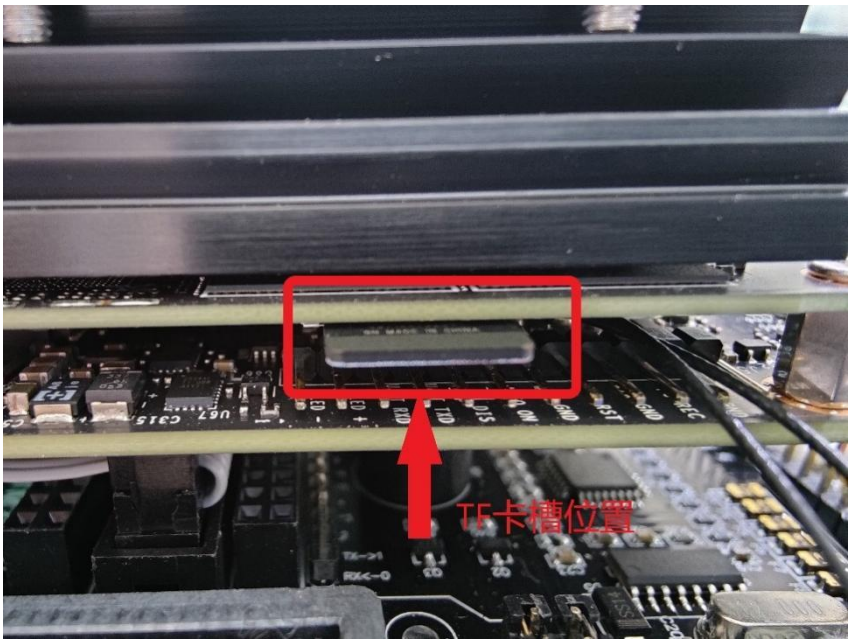


图2.5 TF卡槽位置

2.1.2、Jetson Nano 的无头模式

在白熊人工智能实验箱上我们可以通过显示屏显示 Jetson Nano 和 Ubuntu 桌面，从而可以直接控制进行一些设置。在缺少显示器情况下我们可以采用一种 NVIDIA 独有的连接方式，也是我认为最为方便的连接方式就是无头模式连接，在这种模式下我们只需要一条 MicroUSB 线和电脑的 USB 接口相连接等待电脑自动安装好驱动后即可进行无头连接（Gadget USB Device)。

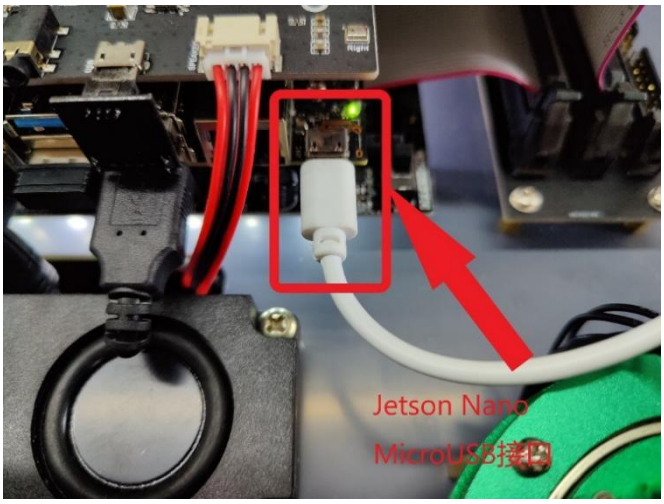


图 2.6 Jetson Nano MicroUSB 接口

连接好之后，查看我们的设备管理器，查看是否安装好驱动程序：

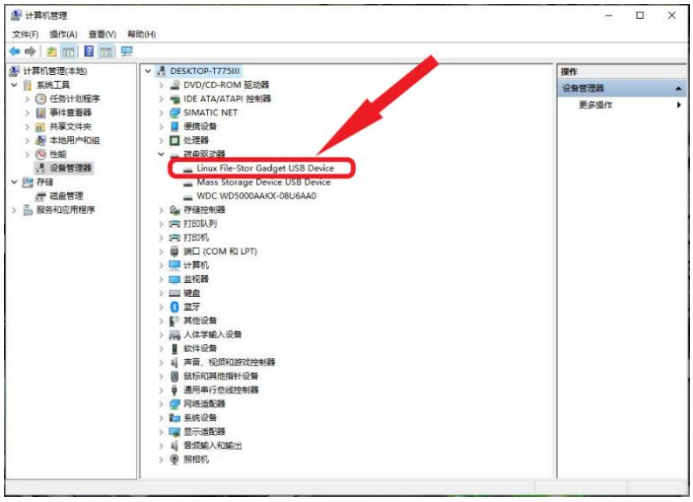


图 2.7 查看 Jetson Nano 驱动

当驱动安装好以后，我们就可以通过XShell SSH、FileZilla、JupyterLab等工具进行登录到Jetson Nano进行操作（地址始终为：192.168.55.1:8888）。

2.1.3、查看 IP 地址

IP地址通俗上讲，就是电脑在网络世界的‘身份证号码’，它由32位的二进制数组成，也就是4个字节。常见的ip地址由十进制数，三位一隔共四段的数字组成。由二进制转化而来。

当我们需要远程登录到我们的Jetson Nano进行互传文件或运行程序等操作时，我们就需要知道主机的IP地址。

首先我们可以通过主板上的有线网卡口或是wlan连接网络，因为我们的实验箱标配了显示屏和键盘鼠标，所以在做一些基础的系统操作时会很方便，我们可以找到桌面右上方的WiFi标志，然后点击，找到想要连接的信号，输入密码即可。

连接上网络后，我们可以打开终端，输入

```
sudo ifconfig
```

我们可以在桌面上点击终端图标或ctr+Alt+T打开终端。

图2.8 系统桌面

输入ifconfig后，如果是有线连接查看对应有线网卡eth0的IP地址，如果是WiFi就查看wlan0下的IP地址。

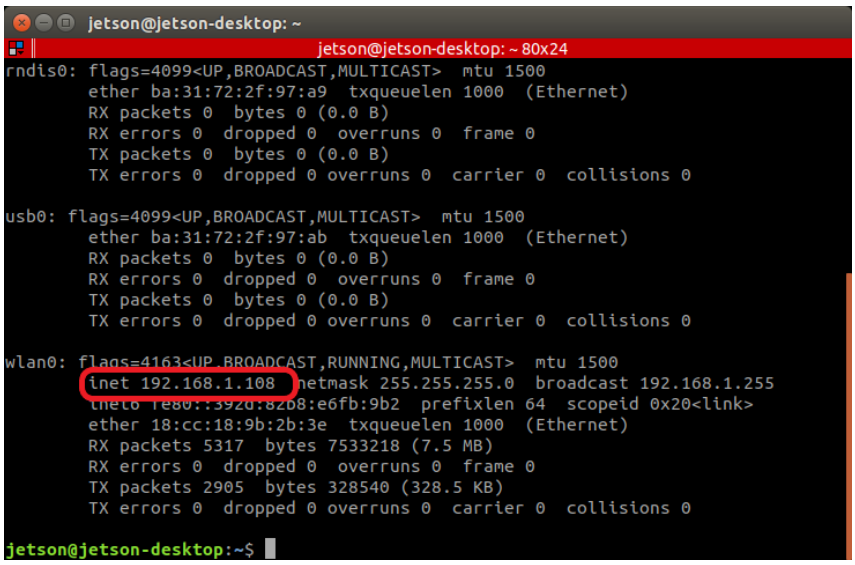


图2.9 IP地址

2.1.4、结束出厂大程序

白熊平台出厂自带的镜像系统，开机会自动运行大程序，如果是需要二次开发则可能因为占用摄像头而无法运行其他程序，所以在进行二次开发之前需要结束掉出厂大程序的进程。

结束大程序方法有两种，第一种是临时结束，第二种是永久结束。临时结束表示立即结束大程序，但是下次开机依然会自动启动大程序。永久结束表示立即结束大程序，下次开机也不会启动大程序。

2.2、常用工具的使用

2.2.1、XShell SSH 工具使用

XShell 是一个强大的安全终端模拟软件，它支持 SSH1,SSH2,以及 Microsoft Windows 平台的 TELNET 协议。XShell 可以在 Windows 界面下用来访问远端不同系统下的服务器，从而比较好的达到远程控制终端的目的。

XShell 软件需要良好的操作界面，多种主题设置，可以让我们的界面变得更加的优质和便捷，连接 SSH 的工具有很多，我们选择 XShell 工具是较为理想的工具。

XShell 软件安装

双击安装软件（白熊人工智能实验平台资料->常用工具->XShell6.exe）

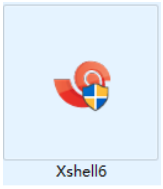


图 2.10 XShell 软件图标

图 2.11 XShell 软件

选择接受许可证协议中的条款后点击下一步，按照软件提示的步骤一直往下操作就可以完成软件的安装过程。

使用 XShell 软件连接 Jetson Nano 主控

第 1 步：新建会话属性，具体设置方式如下图所示：

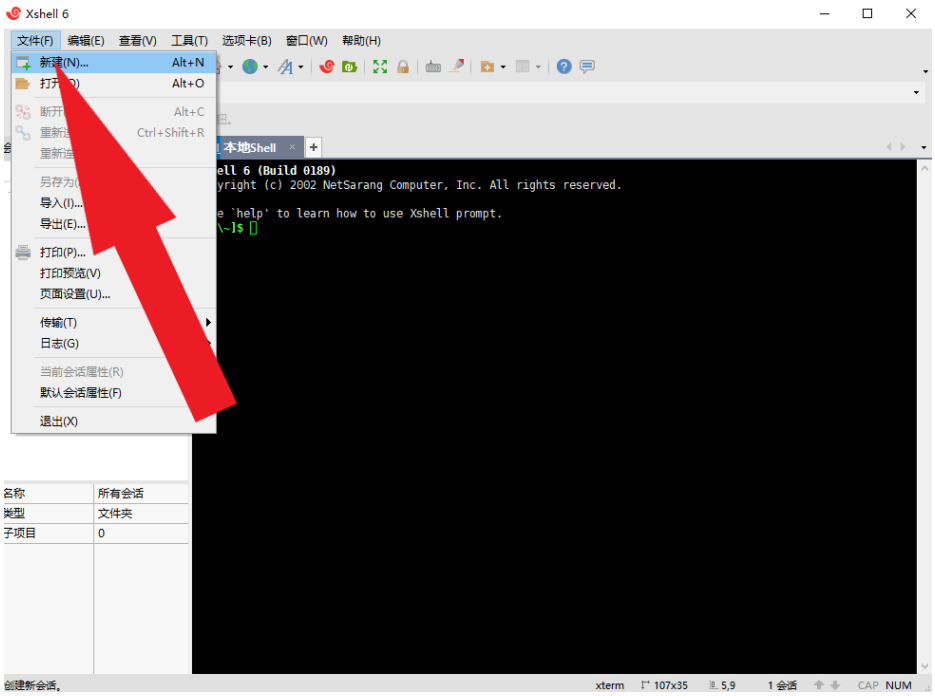


图 2.12 新建会话

第 2 步：自定义名称并填入 IP 地址和端口号，这里的 IP 地址使 Jetson Nano 的 IP 地址，可以通过章节‘2.1.3’的方法获取 IP 地址，端口号选择 22。

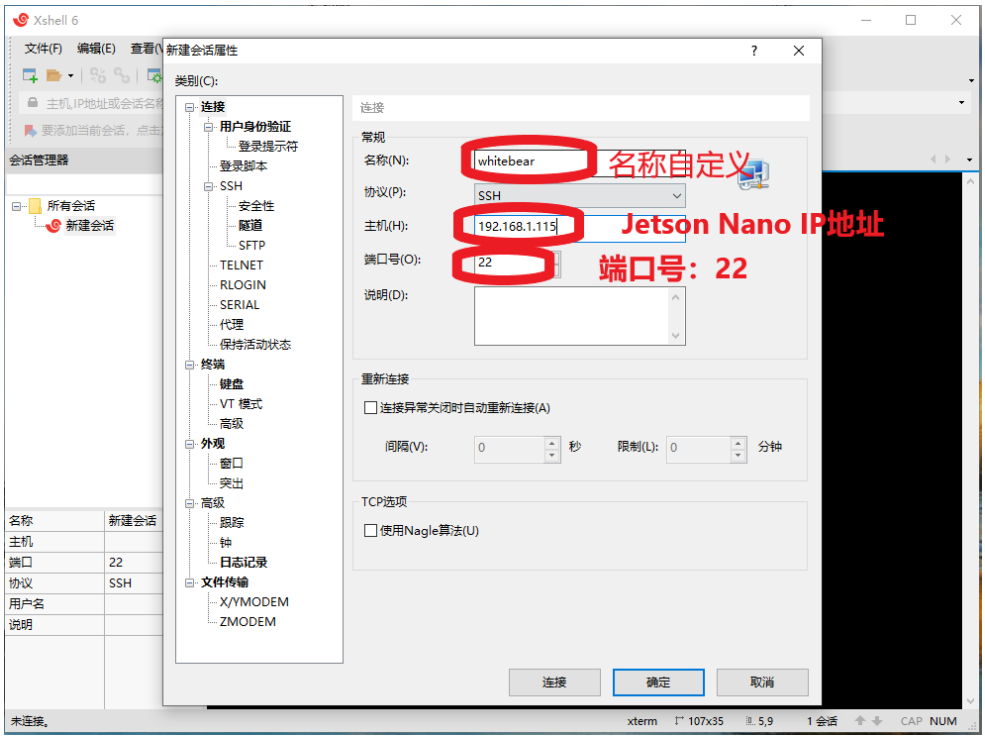


图 2.13 名称 IP 以及端口号

第 3 步：双击刚才新建的会话

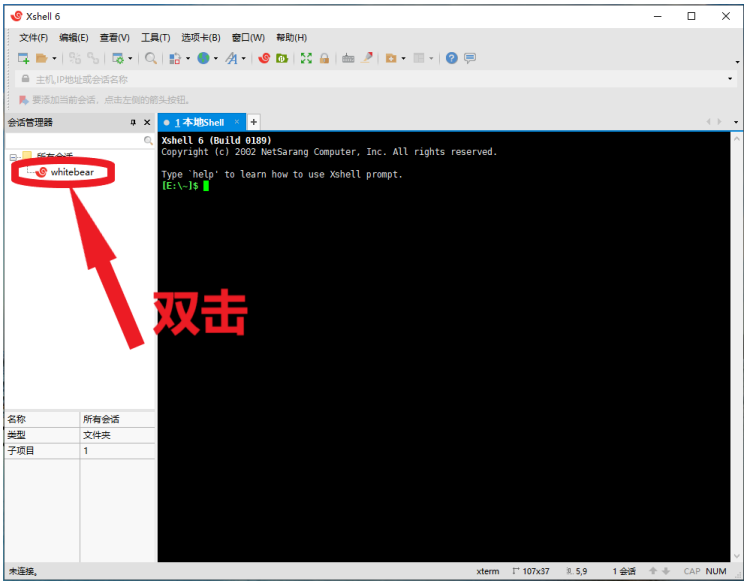


图 2.14 双击会话

第 4 步：输入 Jetson Nano 的用户名

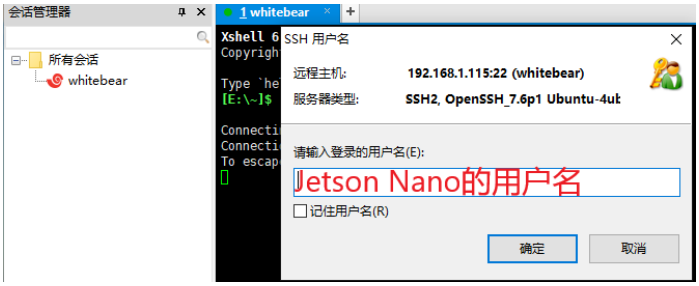


图 2.15 输入用户名

第 5 步：输入 Jetson Nano 的密码

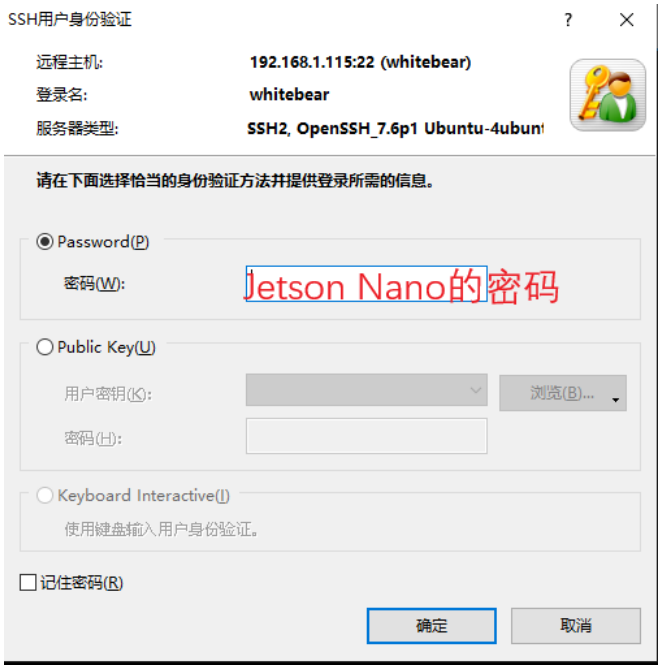


图 2.16 输入密码

点击确定即可连接 Jetson Nano 主板。如果显示如下信息则证明 XShell 已经成功连接 Jetson Nano 主板。

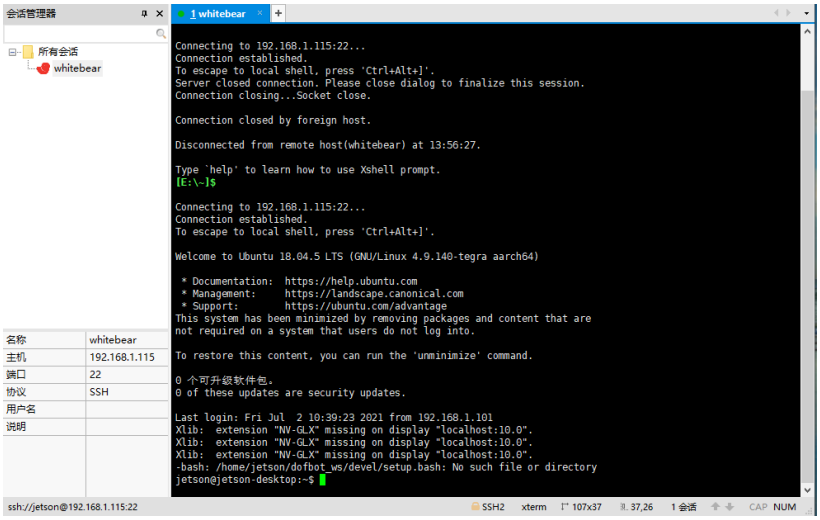


图 2.17 成功连接后显示的信息

此时我们就可以使用 XShell 输入命令控制 Jetson Nano 主板了。

使用 XShell 软件无头模式连接 Jetson Nano 主控

操作方式和上一节的方式相同，固定 IP 地址为 192.168.55.1，端口号为 22。

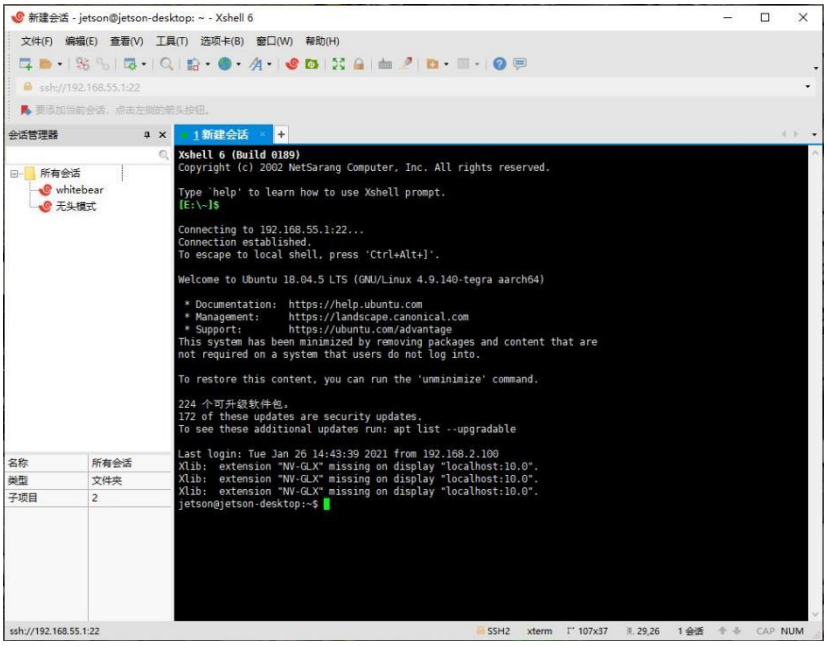


图 2.18 无头模式连接 Jetson Nano

2.2.2、FileZilla 文件传输软件使用

FileZilla 客户端是一个快速可靠的、跨平台的FTP,FTPS和SFTP客户端。具有图形用户界面（GUI)和很多有用的特性，官方网站（https://www.filezilla.cn/),我们可以下载对应的windows、Mac或者Linux版本。

相比较于其它FTP客户端，FileZilla 包含如下特性：

- 易于使用
- 支持FTP,FTP并支持SSL/TLS(FTPS)协议，支持SSH文件传输协议（SFTP)
- 跨平台。在Windows,Linux,*BSD,Mac OSX和其它平台下运行
- 支持IPv6协议
- 多种可用的语言（包含中文）
- 断点续传且支持容量大于4GB的文件
- 多标签用户界面
- 功能强大的站点管理器（Site Manager)和传输队列管理
- 书签功能
- 拖拽功能支持
- 支持传输限速功能
- 文件名过滤器
- 文件夹比较功能
- 网络设置向导
- 远程文件编辑功能
- 保持链接功能
- 支持HTTP/1.1,SOCKS5 和 FTP代理（FTP-Proxy)
- 登录到文件功能

- 同步文件夹浏览
- 远程查找文件

FileZila软件安装

双击安装软件（白熊人工智能实验平台资料->常用工具->FileZilla_3.550.exe）

图2.19 FileZilla图标

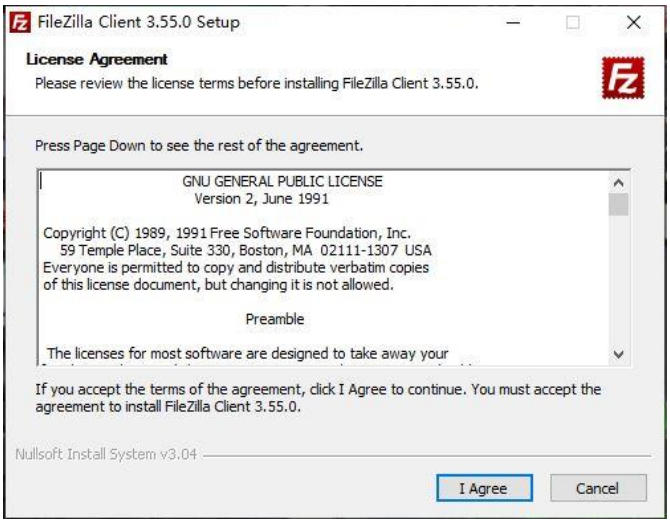


图2.20 FileZilla安装图

选择接受许可证协议中的条款后点击下一步，按照软件提示的步骤一直往下操作就可以完成软件的安装过程。

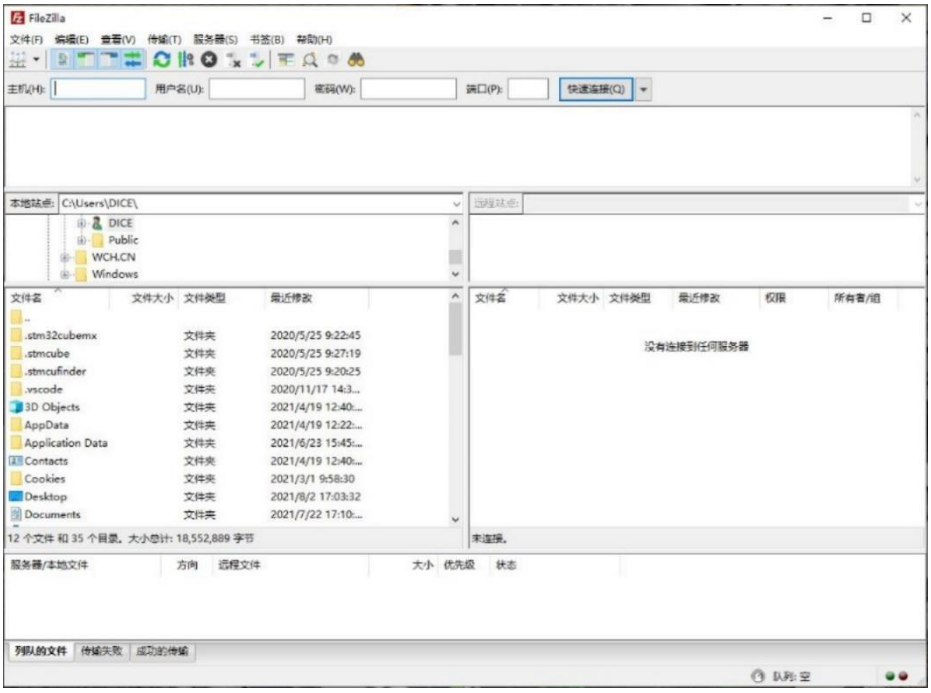


图2.21 FileZilla软件界面

FileZilla软件和Jetson Nano互传文件

FileZilla 软件主要是让我们和电脑直接传递文件，如果有文件在电脑端我们想把文件传递到jetson Nano主板上，我们最好的办法就是通过FileZilla软件传输，要比用U盘等载体传输更加方便。具体操作如下所示：

比如我们通过可以通过章节‘2.1.3’的方法获取IP地址为192.168.1.108,我们就可以进行如下设置：主机：192.168.1.108 用户名：jetson 密码：dicewb 端口号：22

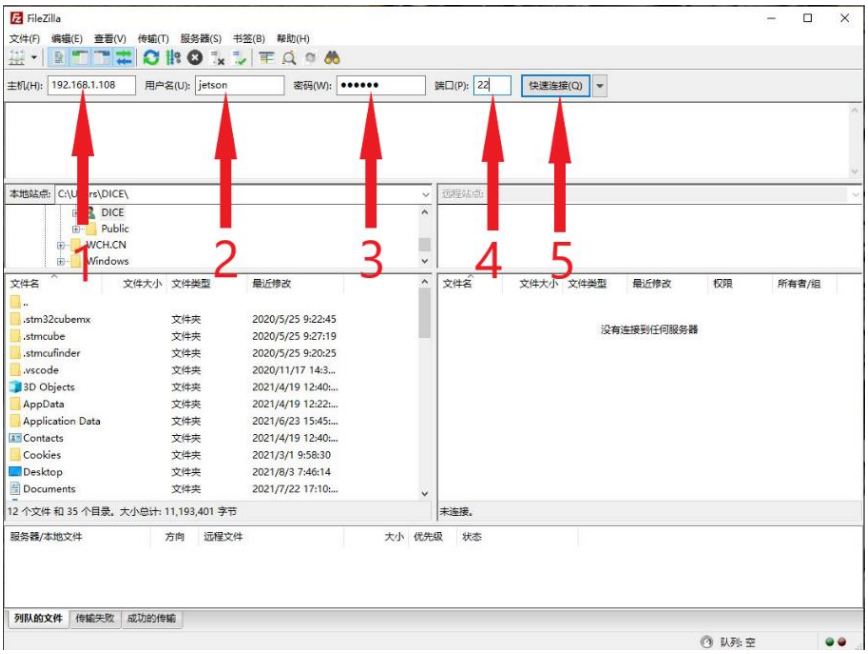


图2.22 FileZilla连接步骤

第1步：输入IP地址

第2步：输入用户名（白熊提供镜像用户名为'jetson'）

第3步：输入密码（白熊提供镜像密码为'dicewb'）

第4步：输入端口号22

第5步：点击快速连接

点击快速连接后，会弹出是否信任主机，我们选择总是信任该主机后，点击确定。

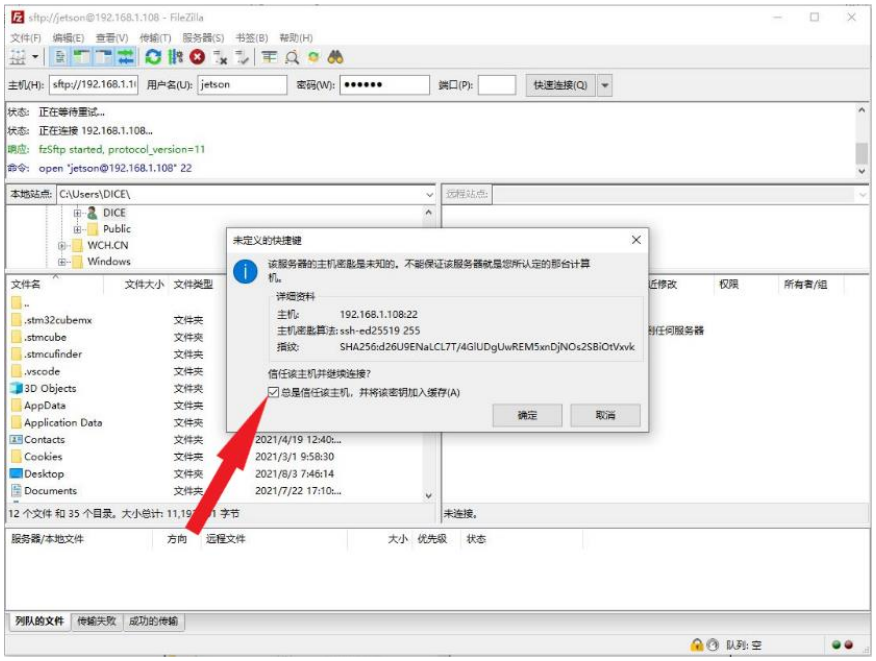


图2.23 信任该主机

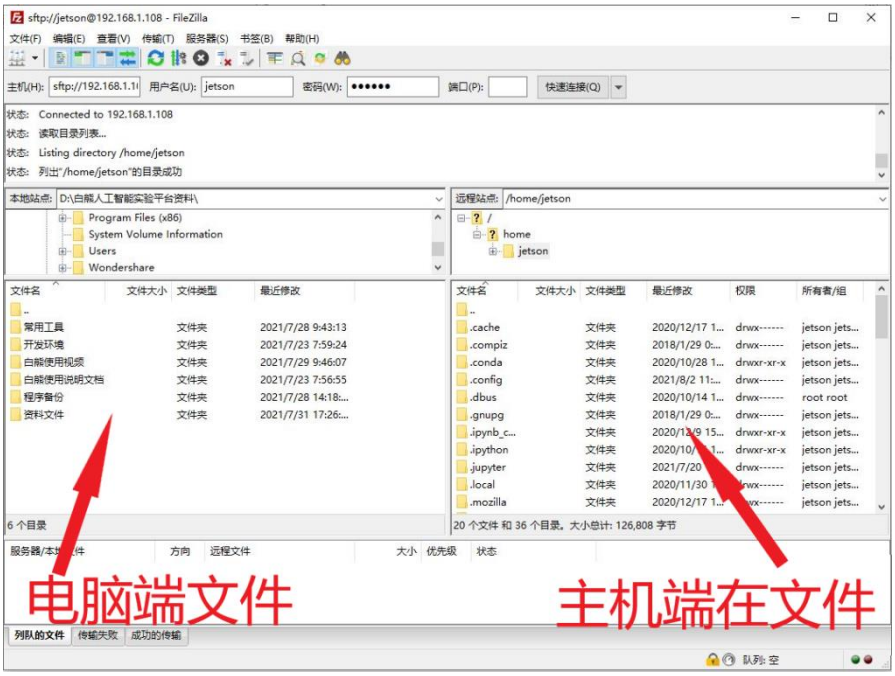


图2.24 成功连接后的文件目录

我们就可以选中电脑端的文件或者主机端的文件，一直按住鼠标左键就可以相互之间拖动进行文件传输。

2.2.3、远程 NoMachine 桌面服务访问

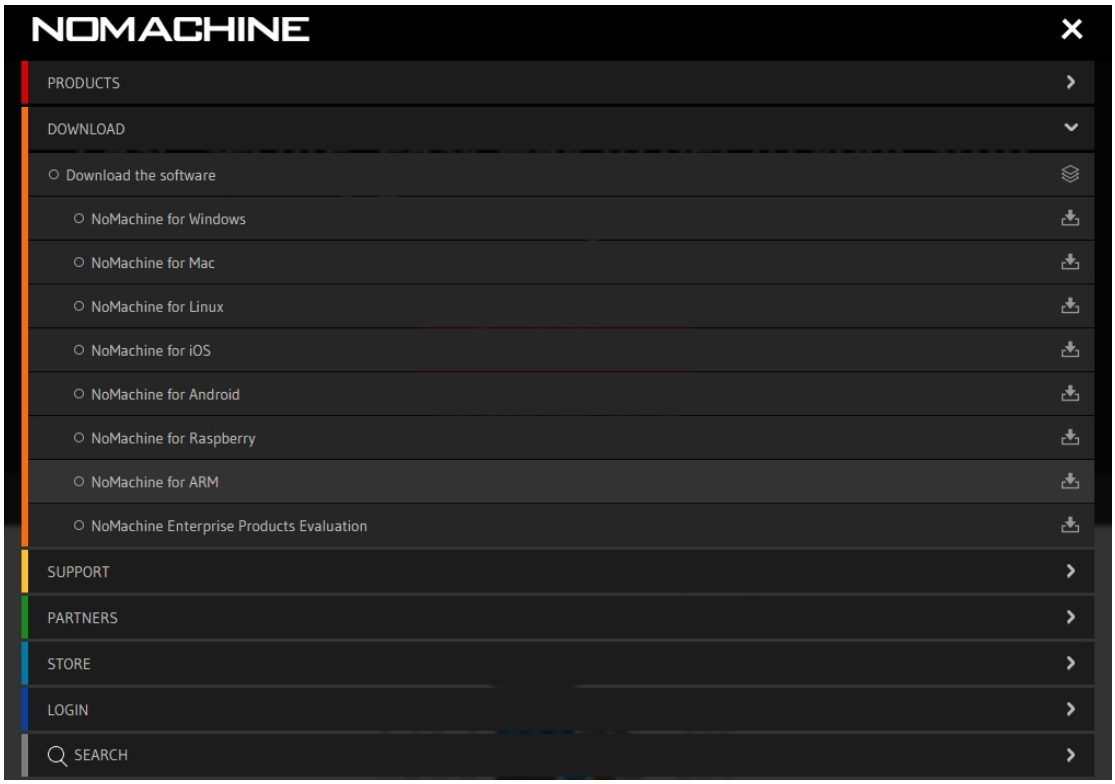
NoMachine 是一款远程桌面工具，就跟VNC一样，但是，我们这里推荐使用NoMachine的主要原因是NoMachine软件的NX协议在高延迟低带宽的链路上提供了近乎本地速度的响应能力。所以，给人的感觉是好像你就在你的电脑跟前。

对应我们提供的镜像文件，是安装了NoMachine远程访问软件的，我们可以跳过这个设置过程，直接用NoMachine软件访问即可。

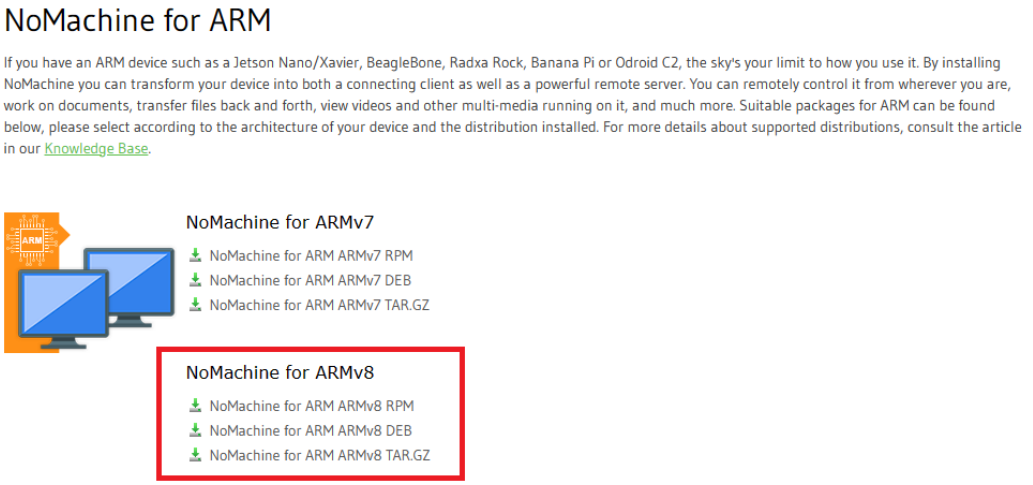
第1步：NoMachine软件下载（官方镜像）

我们直接在Jetson Nano上通过访问该网页地址进行下载，下载对应的软件包即可：

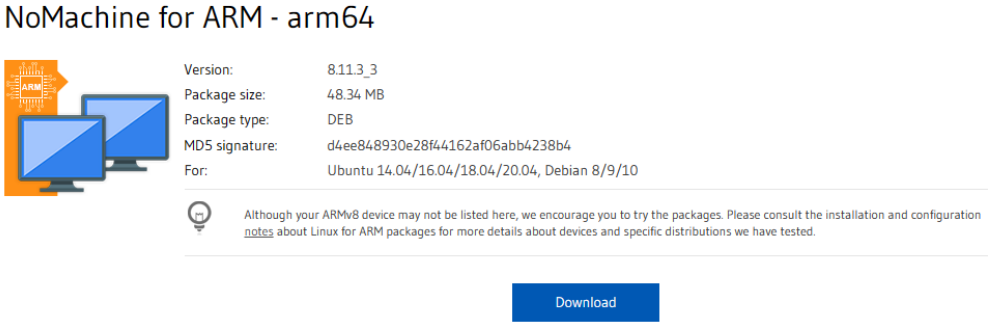
https://www.nomachine.com/



点击菜单按键，选择“DOWNLOAD”,点击“NoMachien for ARM”。



选择“NoMachien for ARMv8 DEB”,因为Jetson Nano基于V8架构，在其他一些嵌入式设备上安装时一定要选择正确的版本。

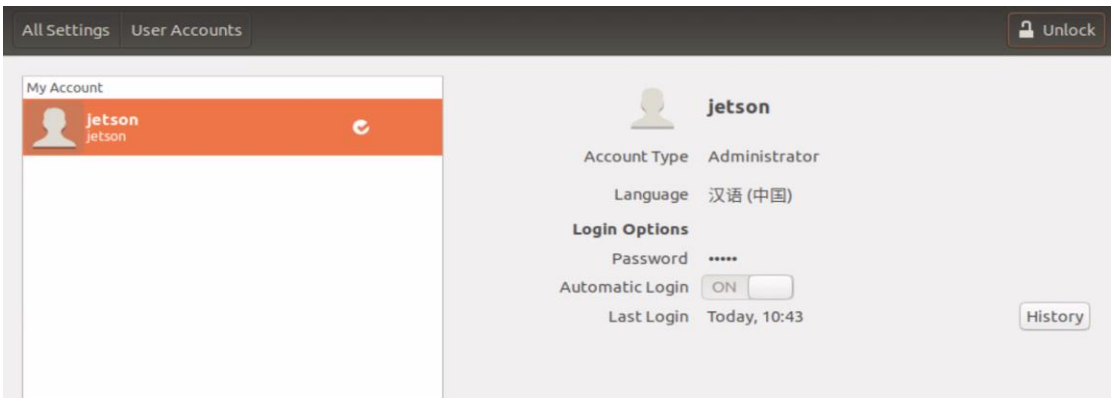


下载完成后，我们找到对应的下载目录进行安装：

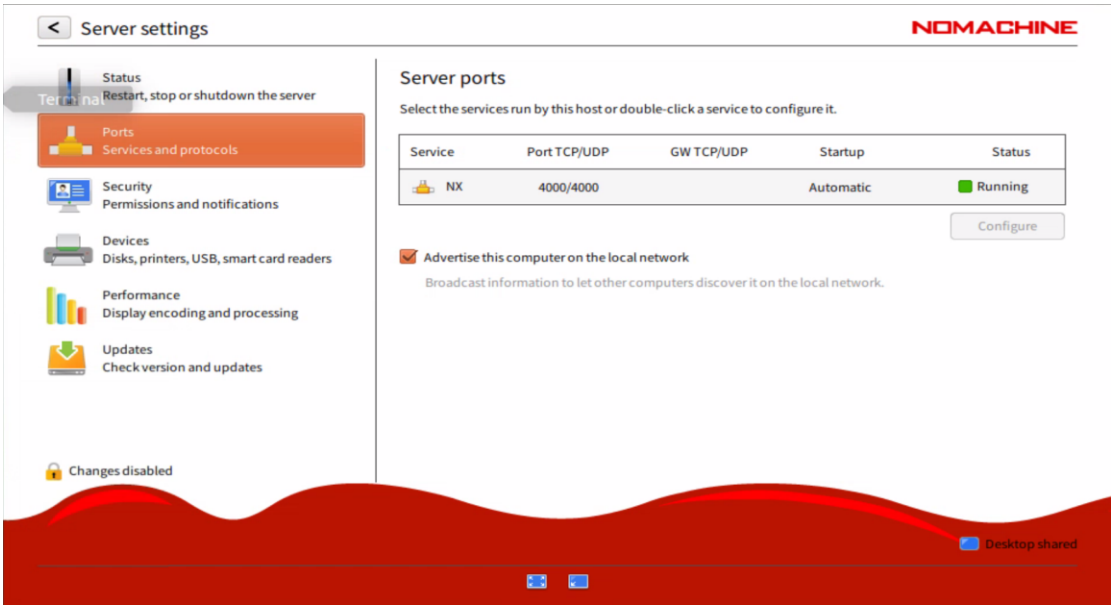
```
sudo dpkg -I nomachine_7.9.2_1arm64.deb
```

安装名称为下载的版本名称，随着版本的迭代升级会有所变化。

安装完毕后要对Jetson Nano进行一下配置。



首先设置开机自动登录，在“System Setting”—“User Accounts”中，点击“UNLOCK“解锁。



打开“NoMachien“的界面，勾选上图选项，默认情况下也都勾选的。这样在Jetson Nano端的配置就完

成了。

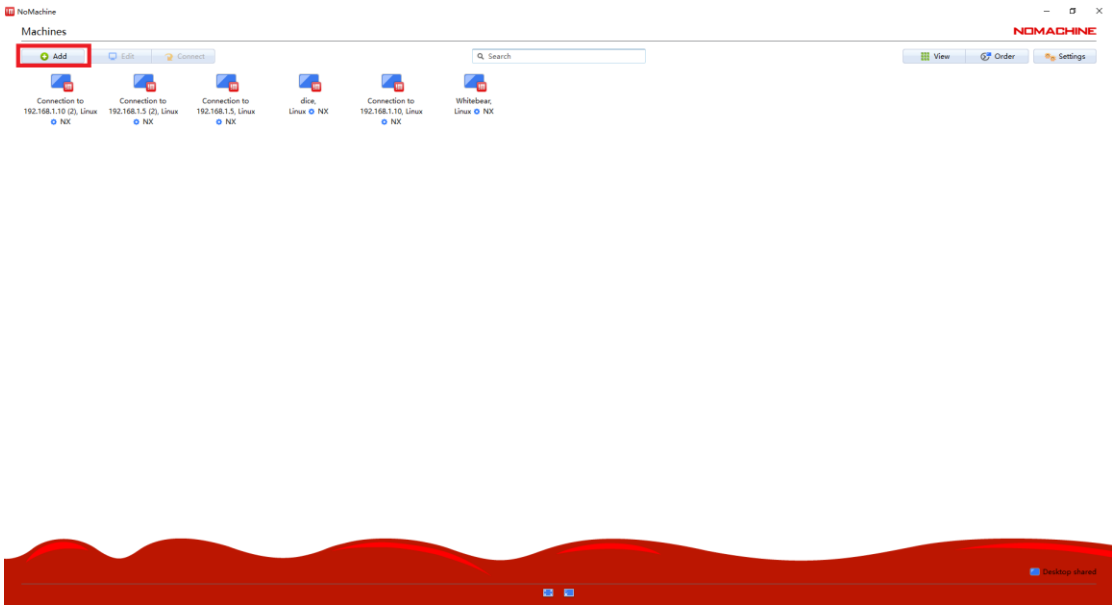
第2步：电脑端安装NoMachien

下面简单说一下Win10上的安装。

同样在官网上下载对应的安装包，双击运行一路“Next“,选择一下合适的安装位置，很快就安装完成了。

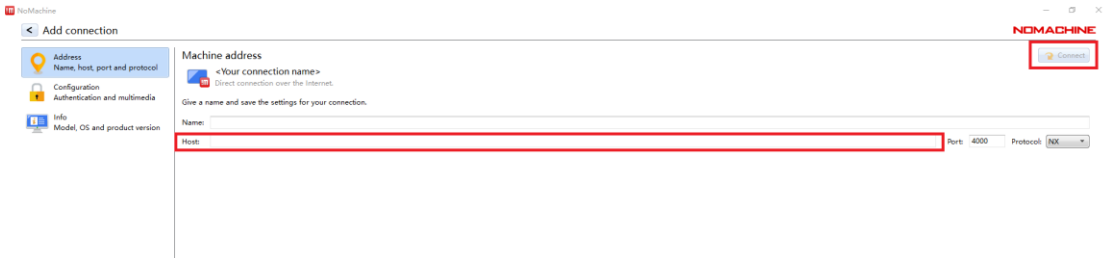
实验箱配套的资料中也包含了该软件，可以直接安装使用。

软件安装完成后，打开的界面如下图所示：



点击Add添加。

在“Host“中填写对应设备的IP，然后点击”Connect“。



输入“Username“和”Password“，Username为jetson，Password为jetson。最好勾选一下记住密码。然后选择”Login“进入桌面。

2.3、JupyterLab 软件开发平台使用

2.3.1、JupyterLab 简介

jupyterLab 其实就像是你的python笔记本一样，不仅可以运行书写的python代码，同时还支持markdown 格式的文本显示。

在jupyterLab 中不仅可以运行python,它还支持R、Julia 和 JavaScript等其他40余种语言。

jupyterLab 是 Jupyter 主打的最新数据科学生产工具，某种意义上，它的出现是为了取代Jupyter Notebook.不过不用担心Jupyter Notebook会消失，JupyterLab 包含了JupyterNotebook 所有功能。

图2.25 JupyterLab宣传图1

JupyterLab作为一种基于web的集成开发环境，你可以使用它编写notebook、操作终端、编辑markdown文本、打开交互模式、查看csv文件及图片等功能。

你可以把JupyterLab当作一种究极进化版的Jupyter Notebook。原来的单兵作战，现在是空陆空联合协作。

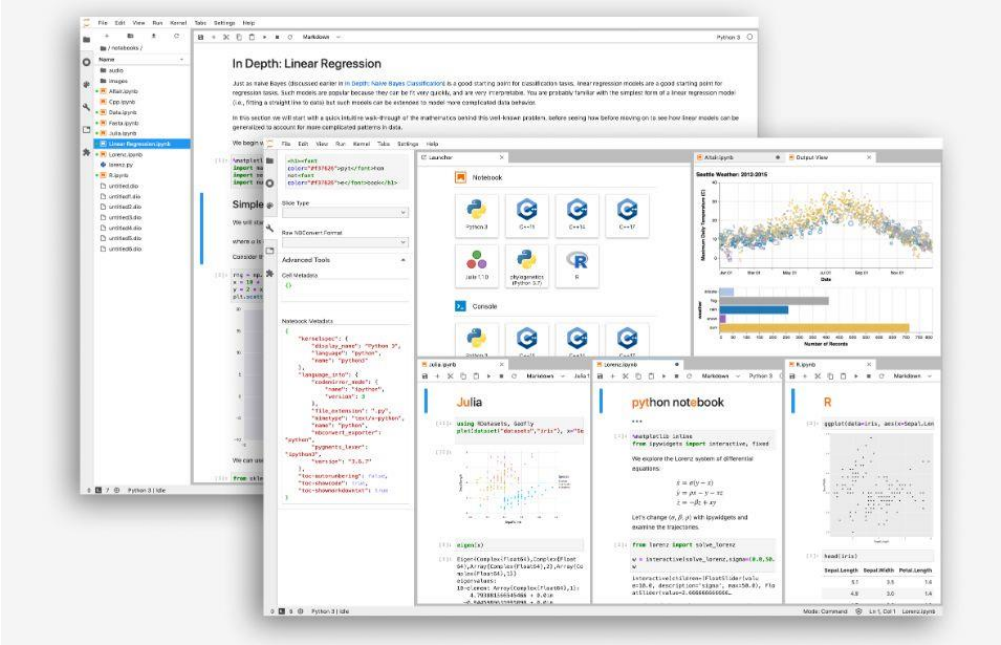


图2.26 JupyterLab宣传图2

总之，JupyterLab有以下特点：

- 交互模式：Python交互式模式可以直接输入代码，然后执行，并立刻得到结果，因此Python交互模式主要是为了调试Python代码用的
- 内核支持的文档：使你可以在可以在Jupyter内核中运行的任何文本文件（Markdown，Python，R等）中启用代码
- 模块化界面：可以在同一个窗口同时打开好几个notebook或文件（HTML, TXT, Markdown等等），都以标签的形式展示，更像是一个IDE
- 镜像notebook输出：让你可以轻易地创建仪表板
- 同一文档多视图：使你能够实时同步编辑文档并查看结果
- 支持多种数据格式：你可以查看并处理多种数据格式，也能进行丰富的可视化输出或者Markdown形式输出
- 云服务：使用Jupyter Lab连接Google Drive等服务，极大得提升生产力

2.3.2、启动 Jupyter Lab 编程环境

对于初学阶段的人工智能程序开发或者教学工作者我们强烈推荐您使用 Jupyter Lab 交互式编程环境，所以我们的教学中将一直采用Jupyter Lab 进行Python 程序开发，我们在后续使用中，我相信大家也一定会像我们一样爱上Jupyter Lab环境来开发程序，Jetson Nano 镜像中已经安装好了 Jupyter Lab 开发环境，我们可以在白熊实验箱上打开'chrome'浏览器访问'**localhost:8888**'来打开Jupyter Lab 开发界面。我们也可以在电脑上通过 web 浏览器来访问Jupyter Lab 开发界面，但是我们的电脑必须要和 白熊AI平台处于同一网络下，否则无法访问，访问方式为在浏览器上输入'**Jetson Nano IP:8888**'来进行登录，如果还没有连接WIFI,采用的是无头模式连接的话在浏览器上输入'**192.168.55.1:8888**' 进行登录访问，第一次登录的时候可能需要我们输入密码，白熊AI平台的密码为 'dicewb'输入后点击登录进行登录。为方便截图后续演示将在PC机web访问完成。

图2.27 Jupyter Lab密码界面

成功登录之后，会打开我们上一次没有关闭的程序界面，如下图所示：

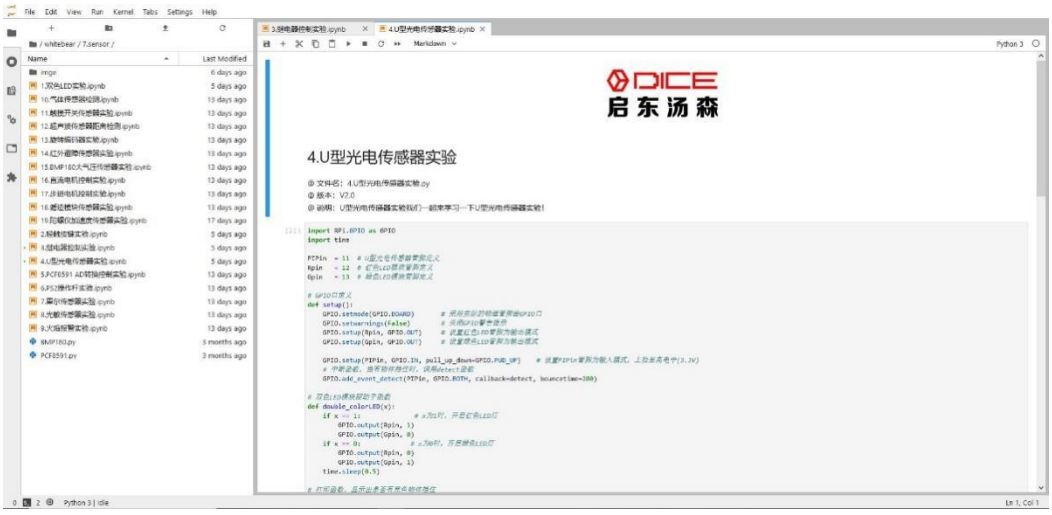


图2.28 Jupyter Lab登录之后的界面

2.3.3、Jupyter Lab 编程常见操作

Jupyter Lab 提供了用于交互式探索性计算的灵活构建基块。尽管 JupyterLab 具有在传统集成开发环境（IDE)中大部分功能，但它仍专注于交互式探索性计算。

Jupyter Lab 界面包括一个主要工作区，该工作区包含文档和活动的选项卡，一个可折叠的左侧边栏以及一个菜单栏。左侧栏包含文件浏览器，正在运行的内核和终端的列表，命令面板，笔记本单元工具检查器以及选项卡列表。

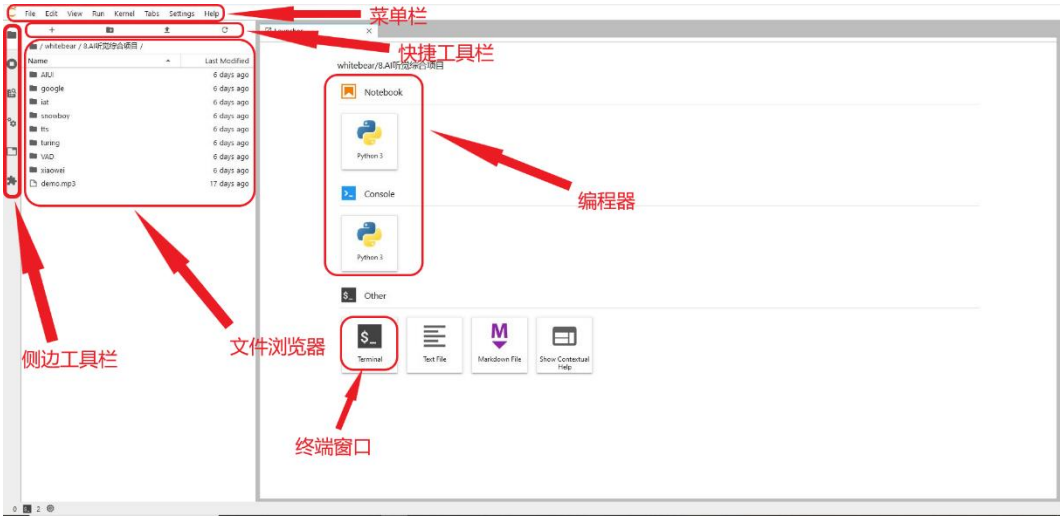


图2.29 Jupyter Lab界面

JupyterLab会话始终驻留在工作空间中。工作区包含JupyterLab的状态：当前打开的文件，应用程序区域和选项卡的布局等。工作区可以使用命名工作区URL保存在服务器上。

（1）菜单栏

Jupyter Lab顶部的菜单栏具有顶级菜单，这些菜单通过其键盘快捷键公开了Jupyter Lab中可用的操作。默认菜单为：

- File:与文件和目录有关操作
- Edit:与编辑文档和其他活动有关的操作
- View:更改 JupyterLab 外观的操作

- Run:在不同活动（例如笔记本和代码控制台）中运行代码的操作
- Kernel:用于管理内核的操作，它们是运行代码的独立过程
- Tabs:停靠面板中打开的文档和活动的列表
- Settings:通用设置和高级设置编辑器
- Help:Jupyter Lab和内核帮助链接列表

（2）左侧工具栏

左侧工具栏包含许多常用的选项卡，例如文件浏览器，正在运行的内核和终端的列表，命令面板以及主工作区中的选项卡列表。

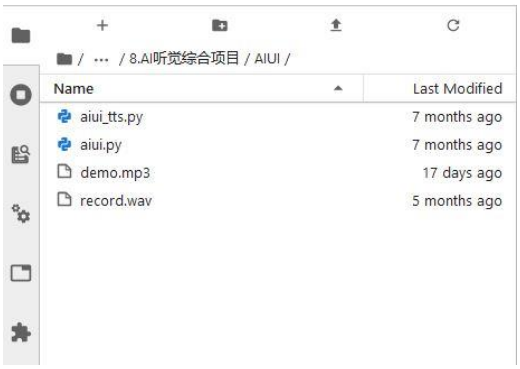


图2.30 左侧工具栏

（3）主要工作区域

Jupyter Lab中的主要工作区使您可以将文档（笔记本，文本文件等）和其他活动（终端，代码控制台等）排列到可以调整大小或细分的选项卡面板中。将选项卡拖动到选项卡面板的中心可将选项卡移动到面板。通过将选项卡拖动到面板的左侧，右侧，顶部或底部来细分选项卡面板。

图2.31 主要工作区

（4）标签和单文档模式

左侧栏中的“选项卡”面板列出了主要工作区中的打开文档或活动：

图2.32 工作区域中使用的内容

（5）键盘快捷键

与传统的Notebook中一样，您可以通过键盘快捷键浏览用户界面。通过选择“设置”（Settings）菜单中的“高级设置编辑器”（Advanced Settings Editor）项，然后在“设置”选项卡中选择“键盘快捷键”(keyboard shortcuts)，可以找到并自定义当前的键盘快捷键列表。

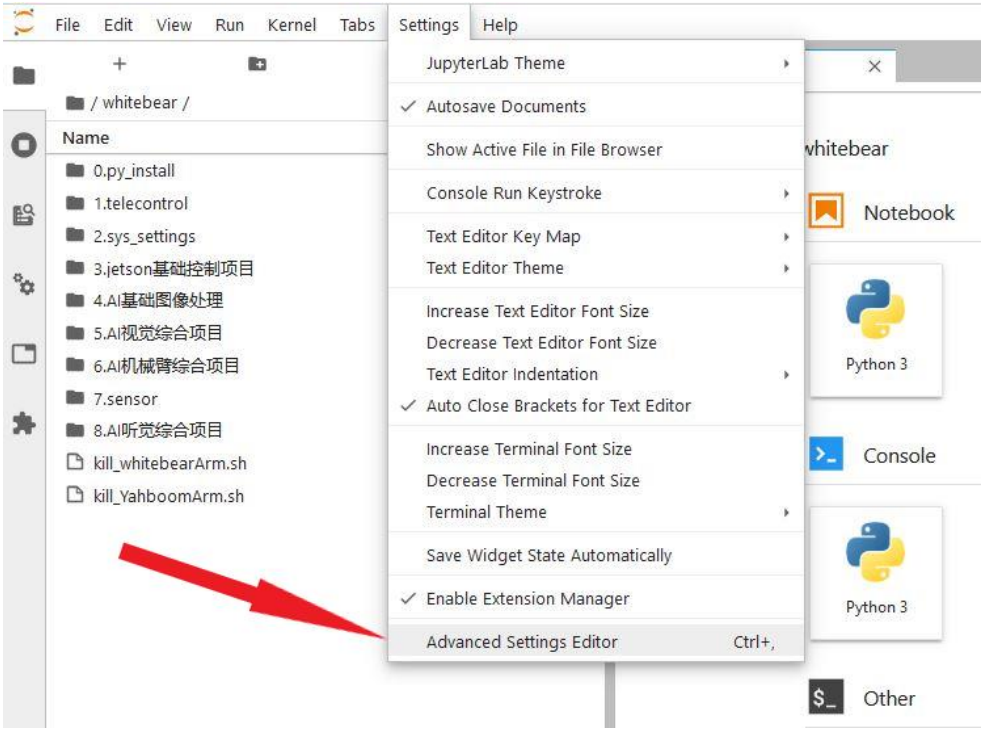


图2.33 设置键盘快捷键步骤1

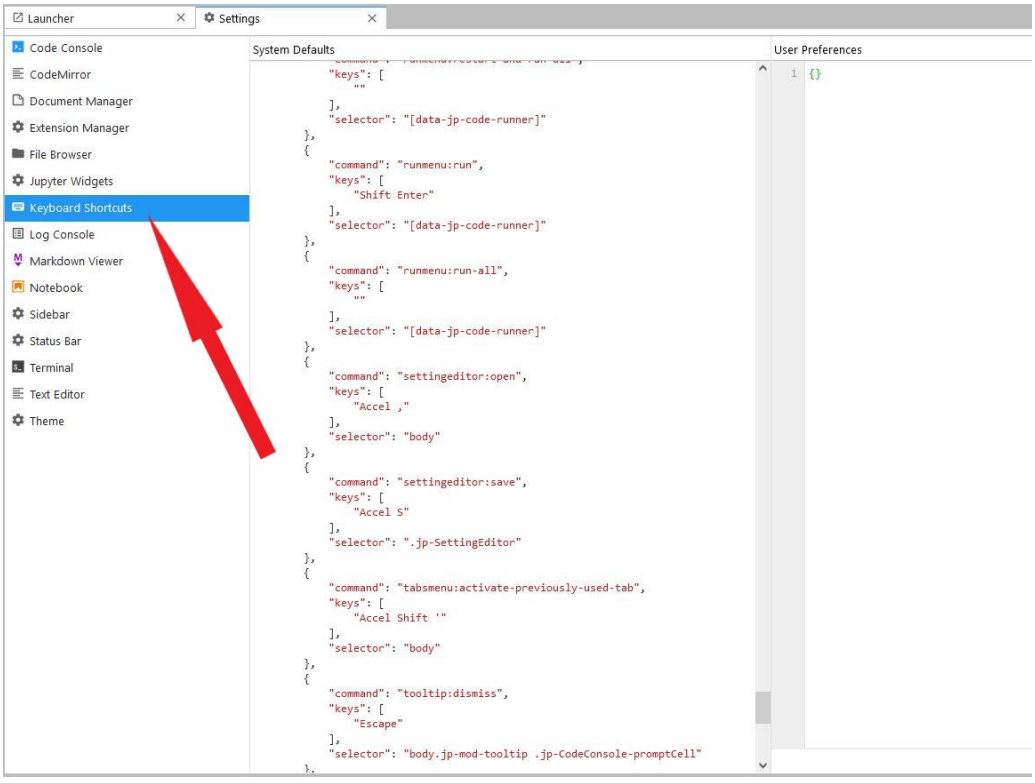


图2.34 设置键盘快捷键步骤2

（6）Notebooks 笔记本

Jupyter Notebooks 是将实时可运行代码与叙述文本（Markdown),方程式（LaTex)图像，交互式可视化以及其他丰富输出结合在一起的文档:

图2.35 Notebook

（7）内核操作

左侧工具栏中的“正在运行”面板显示了当前所有笔记本，代码控制台和目录中正在运行的所有内核和终端的列表。

与经典的 Jupyter Notebook一样，当您关闭Notebook文档，代码控制台或终端时，服务器上运行的基础内核或终端将继续运行。这使您可以执行长时间运行的操作并稍后返回。通过“正在运行”面板，您可以重新打开或聚焦链接到给定内核或终端的文档。

您可以通过单击X按钮关闭所有内核和终端。

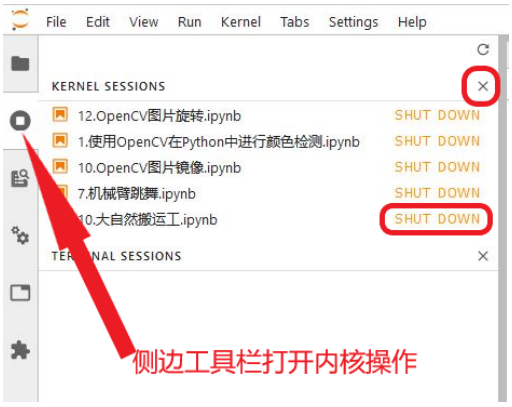


图2.36 侧边工具栏打开内核操作

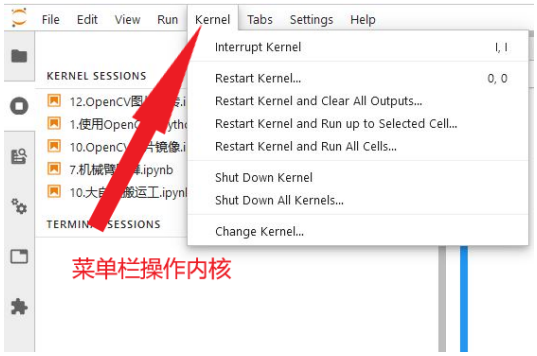


图2.37 菜单栏打开操作内核

（8）为单元格“为输出创建新视图”

选中一个单元格，可以为输出视图创建一个独立的视图窗口便于观察，适用于多组数据查看或者调试程序。

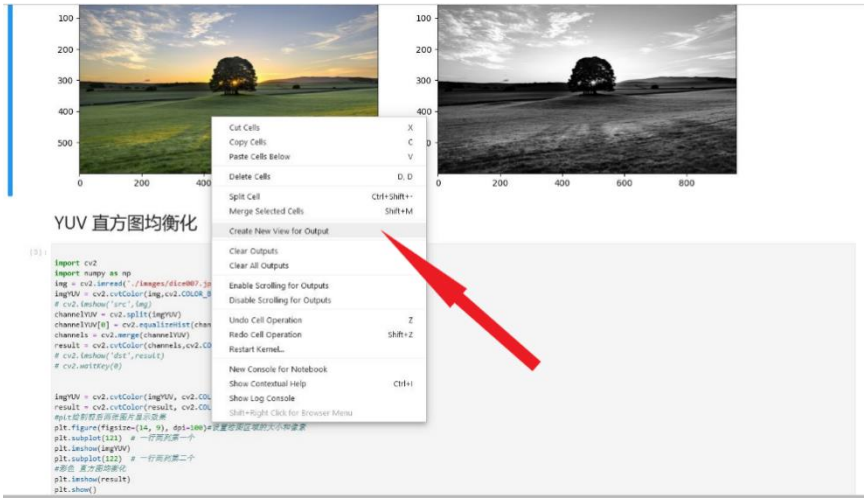


图2.38 创建新视图

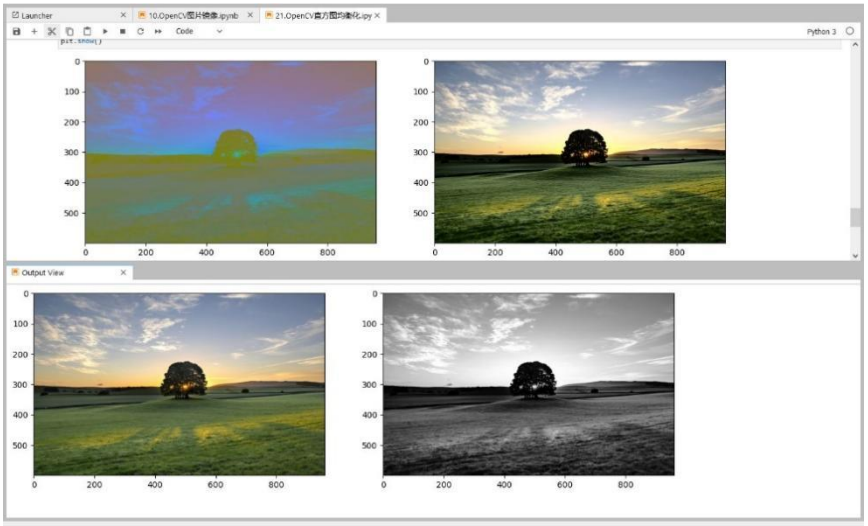


图2.39 新视图

（9）终端窗口

我们可以直接通过Jupyter Lab来登录到Jetson Nano系统中的终端窗口来直接控制系统命令，这是一个非常方便的一个操作，我们甚至可以代替掉 XSHELL软件的 SSH 操作过程。

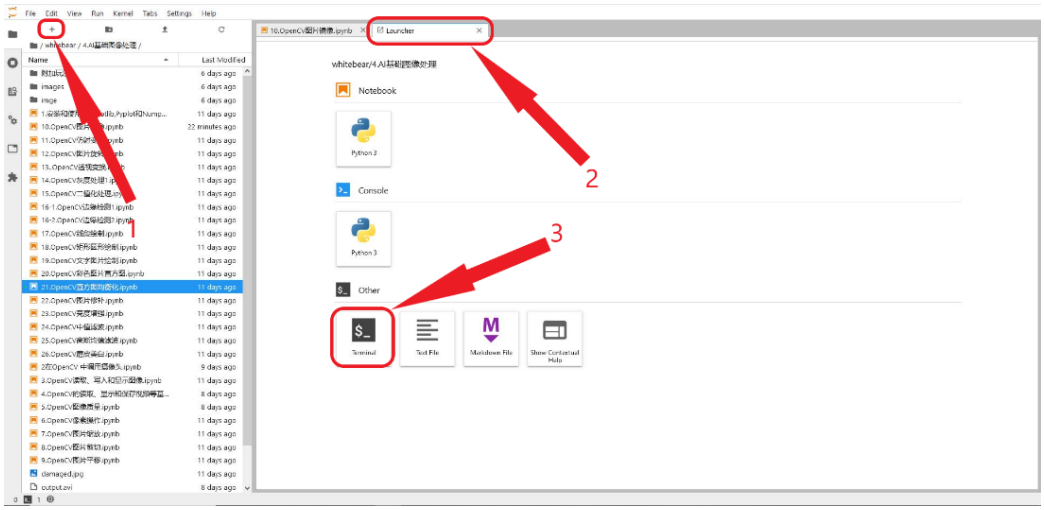


图2.40 Jupyter Lab打开终端步骤

第1步：点击菜单栏下方的“+”号

第2步：弹出“Launcher”窗口

第3步：选择终端窗口打开

图2.41 终端界面

（10）Jupyter Lab 常用快捷键操作

JupyterLab 有两种键盘输入模式。编辑模式，允许你往单元中键入代码或文本，这时的单元框线是蓝色的。命令模式，键盘输入运行程序命令；这时的单元框线是灰色。命令模式（按键 Esc 开启）编辑模式（按键 Enter 切换）。

- 编辑模式：

图2.42 编辑模式截图

- ·命令模式

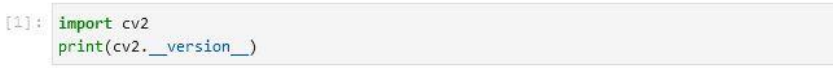


图2.43 命令模式截图

命令模式下快捷键：

命令模式（按键 Esc 开启）

Enter:转入编辑模式

Shift-Enter:运行本单元，选中下个单元

Ctrl-Enter:运行本单元

Alt-Enter:运行本单元，在其下插入新单元

Y:单元转入代码状态

M:单元转入markdown 状态

R:单元转入raw 状态

1:设定1级标题

2:设定2级标题

3:设定3级标题

4:设定 4级标题

5:设定5级标题

6:设定6级标题

Up:选中上方单元

K:选中上方单元

Down:选中下方单元

J:选中下方单元

Shift-K:扩大选中上方单元

Shift-J:扩大选中下方单元

A:在上方插入新单元

B:在下方插入新单元

X:剪切选中的单元

C:复制选中的单元

Shift-V:粘贴到上方单元

V:粘贴到下方单元

Z:恢复删除的最后一个单元

D,D:删除选中的单元

Shift-M:合并选中的单元

Ctrl-S:文件存盘

S:文件存盘

L:转换行号

O:转换输出

Shift-O:转换输出滚动

Esc:关闭页面

Q:关闭页面

H:显示快捷键帮助

1:中断Notebook 内核

0,0:重启Notebook 内核

Shift:忽略

Shift-Space:向上滚动

Space:向下滚动

编辑模式下快捷键：

Tab:代码补全或缩进

Shift-Tab:提示

Ctrl-]:缩进

Ctrl-[:解除缩进

Ctrl-A:全选

Ctrl-Z:复原

Ctrl-Shift-Z:再做

Ctrl-Y:再做

Ctrl-Home:跳到单元开头

Ctrl-Up:跳到单元开头

Ctrl-End:跳到单元末尾

Ctrl-Down:跳到单元末尾

Ctrl-Left:跳到左边一个字首

Ctrl-Right:跳到右边一个字首

Ctrl-Backspace:删除前面一个字

Ctrl-Delete:删除后面一个字

Esc:进入命令模式

Ctrl-M:进入命令模式

Shift-Enter;运行本单元，选中下一单元

Ctrl-Enter:运行本单元

Alt-Enter:运行本单元，在下面插入一单元

Ctrl-Shift--:分割单元

Ctrl-Shift-Subtract:分割单元

Ctrl-S:文件存盘

Shift:忽略

Up:光标上移或转入上一单元

Down:光标下移或转入下一单元

（11）使用频率最高的几个快捷键

插入cell:a/b (a在上方插入新单元，b在下方插入新单元)

删除：x

执行：shift+enter

自动补全：tab

cell模式切换：y(m->code) m(code->m)

shift+tab:打开帮助文档

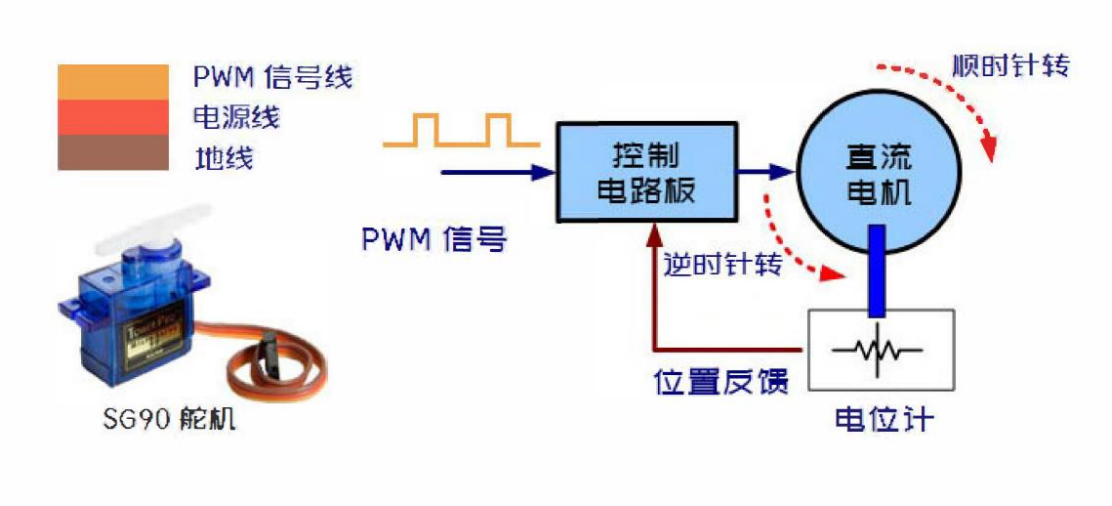
3、AI人工智能机械臂开发知识准备

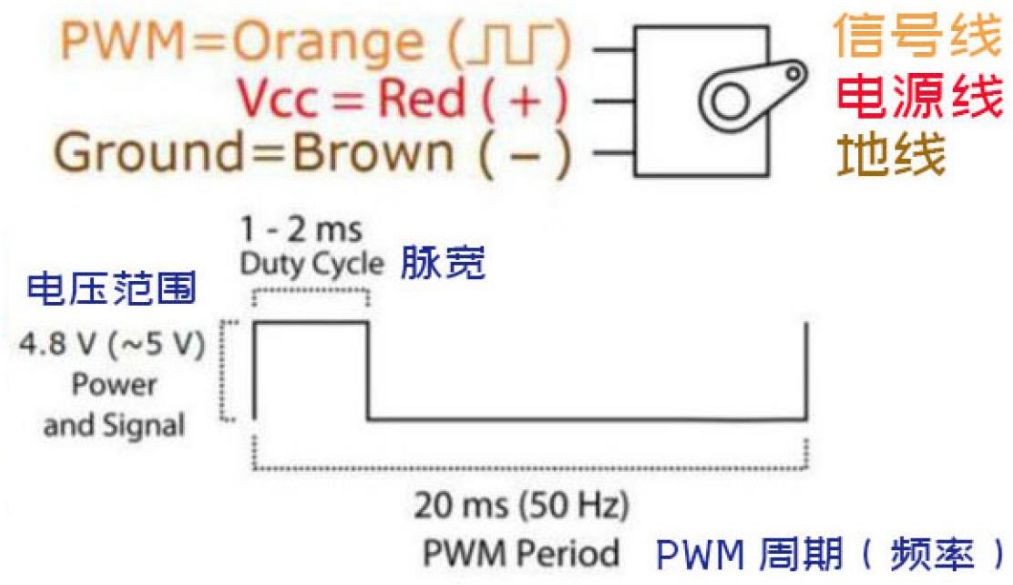
在学习白熊人工智能实验箱之前，我们首先来了解一下二自由度的总线舵机、JupyterLab软件开台、Docker容器技术、OpenCV计算机视觉库、以及TensorFlow AI框架。

3.1、串口总线介绍

在介绍串行总线智能舵机之前，需要先了解传统的PWM舵机存在哪些不足。

传统的PWM舵机也叫数字比例舵机（例如SG90舵机），属于模拟舵机中的一种。单片机需要生成PWM（脉冲方波调制）信号发送给传统PWM舵机，通过调节PWM信号的脉宽来控制传统PWM舵机的角度。如下图：





3.1.1、传统 PWM 舵机存在的主要问题如下所示：

1.占用硬件资源，接线杂乱

一个舵机需要占用一个GPIO，而且这个GPIO接口需要支持PWM信号输出。另外还会占用单片机的定时器资源。当机器人自由度很多的时候（例如六自由度机器人，多足机器人）单片机的IO资源就不够了。

所有的舵机线都直接与单片机相连，当舵机比较多时，接线会显得非常杂乱，另外这种接线方式本身也会影响到机器人关节的旋转。

2.精度差

PWM的分辨率是同时受限于单片机（发送端）与舵机（接受端）双方的。举例来讲，舵机的角度是0-180度，Arduino Uno 的PWM 输出信号的分辨率是255，0-255 对应脉宽是0-20ms，但是PWM舵机只响应0.5-2.5ms 的脉宽，所以实际上Arduino控制舵机的理论精度是7.05°。

$$\frac{180}{255 * \frac{2.5-0.5}{20}} = 7.05$$

另外这里舵机角度的控制精度的计算还没有考虑单片机定时器的精度，舵机接收PWM信号的分辨率，舵机电位计分辨率，还有舵机死 区等诸多因素，都会影响到舵机的精度。

3. 开环控制，无角度反馈

PWM信号是从单片机单方向发送给PWM舵机的，电位计虽然可以采集舵机的角度，但是采集的信号并不会反馈给单片机。所以单片机控制舵机属于开环控制，单片机并不能知道舵机当前是否旋转到了目标位置。另外，当机器人初始化的时候，因为不知道当前的位置，舵机偏移初始位置（bias）较大的时候，会出现舵机旋转过快，误伤到机器或者人。

4. 无轨迹规划，抖动不连贯

PWM 舵机没有轨迹规划，当需要控制舵机关节从一个角度缓慢的旋转到另外一个角度的时候需要开发者自己在单片机内部实现舵机的关节角度的插值算法与轨迹规划。PWM舵机用在机械臂上，受限于控制周期以及角度分辨率，当关节慢速旋转的时候，会出现动作不连贯，抖动的问题。

5. 舵机无保护，容易烧毁

舵机构造比较简单，没有舵机保护的功能。当出现舵机卡齿或者关节卡出等情况，舵机容易烧毁。虽然单个 PWM舵机比较便宜，但是算上舵机烧毁的损耗以及开发者的时间成本，PWM舵机并不“便宜”。

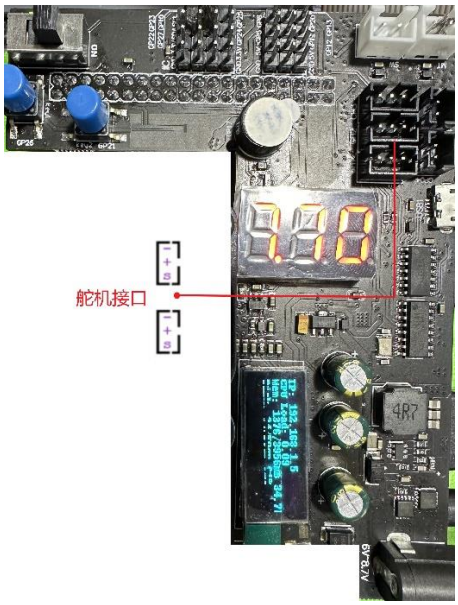
所以传统的PWM舵机比较适合做玩具，不适合用在对精度还有动作流畅度有要求的机器人上。

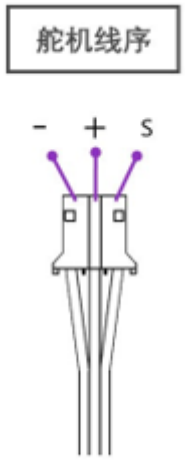
串行总线舵机的优势

串行总线舵机是对一大类舵机的统称，并没有一套统一的标准协议，每个舵机厂商都有自己的通信协议，功能以及硬件配置也各不相同。这里以 CLB-S25 舵机为例，讲解串行总线舵机的相关概念。

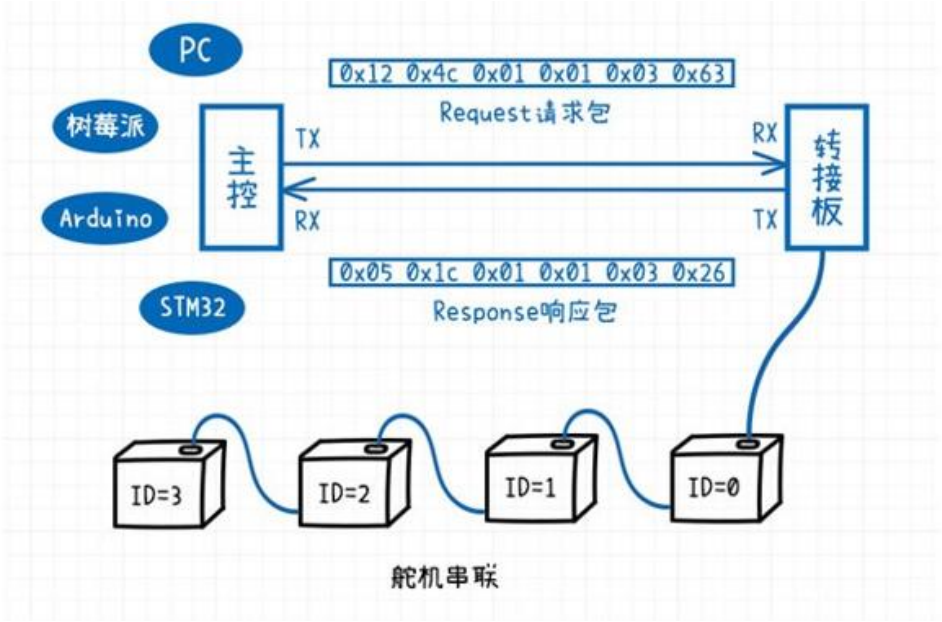
舵机使用的通信协议为单线半双工异步串行通信 （Half duplex Asynchronous Serial Communication），使用的电平标准为 TTL 电平（TTL Level），舵机两侧各有一个接口，通过转接线舵机就可以串联在一条串行总线上。

串行总线舵机需要配合串行总线舵机转接板使用，它的作用是将串行总线舵机的单线转换为双线TTL接日（Rx接收端，Tx发送端），通过双线TTL 串口接口跟单片机进行通信。或者通过转接板上的USB转TTL芯片，通过USB 端口与PC进行通信。





串行总线舵机与PC/单片机之间按照特定的串口通讯协议进行指令传递与反馈数据的解析。



3.1.2、串行总线舵机有如下优势:

1. 硬件资源占用少

通过单片机控制串行总线舵机，只需要占用两个 GPIO 管脚（串口的接收端与发送端）就可以控制最多254个舵机。

2.PC可以直接控制舵机

电脑如果想控制传统的PWM舵机的话，中间必须要通过一个单片机，并且要开发PC与单片机的串口通信协议。对于串口总线舵机来讲，借由转接板，PC可以直接给每个串行总线舵机发送控制指令，对于产品开发或者算法验证就显得非常高效。另外通过PC控制串行总线舵机，只需要占用一个 USB 口。

3.布线美观

串口总线舵机串联的方式，极大的简化了接线的复杂度。

4.舵机精度高

串行总线舵机是通过串口通信的方式，将舵机角度按照通信协议写入到字节流数据中，角度传递更可靠且准确。CLB-S25 这款舵机是4096阶的，假设舵机角度范围是0-270°，那么理论上它的控制精度就是0.066°

$$\frac{270}{4096} = 0.066$$

5.数据反馈，舵机保护

串行总线舵机的角度、电压、电流、温度均可以通过串口通信协议发送给单片机/PC。通过反馈得到的舵机角度，可以知道舵机是否到达目标角度。在舵机的阻尼模式下，还可以周期性的对舵机角度进行采样，用于机械臂示教。另外当检测到负载过大的时候，可以卸力，不会烧坏舵机。

6.内置轨迹规划，平滑流畅

内置轨迹规划算法，操作简单。只需要指定起始位置还有终止位置以及中间的时间间隔，舵机会自动规划轨迹，控制舵机平滑的旋转到目标角度。

7. 功能丰富，一机多用

串行总线舵机除了角度模式外，还具有如下模式：

轮式控制模式：舵机可以用于小车，舵机支持360度旋转，支持定速与定圈控制。

阻尼控制模式：阻尼模式下，对舵机角度周期性轮巡，通过手持机械臂旋转关节的方式，完成示教。

智能抓取模式：舵机可以用于机械臂的夹具，根据舵机的电流，智能调整抓取的力度，防止损坏物体。

8.支持用户自定义参数设置

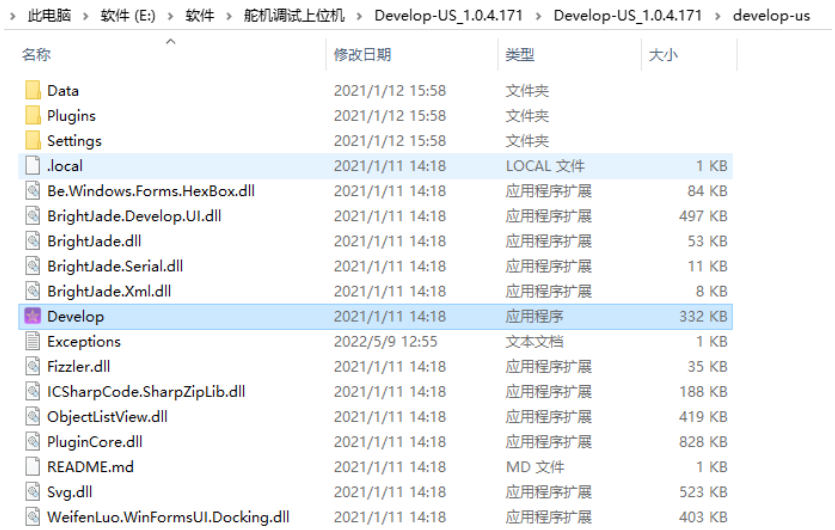
串行总线舵机有一个用户自定义参数列表，用户可以根据自己的需求对舵机进行个性化设置，部分舵机自定义参数如下图所示：

data_id	参数名称(en)	参数名称(cn)	字节类型	字节长度	单位	备注
32		<预留>	uint8_t	1		
33	response_switch	响应开关	uint8_t	1		具体介绍
34	servo_id	舵机ID	uint8_t	1		具体介绍
35		<预留>	uint8_t	1		
36	baudrate	波特率选项	uint8_t	1		具体介绍
37	stall_protect_mode	舵机堵转保护模式	uint8_t	1		具体介绍
38	stall_power_limit	舵机堵转功率上限	uint16_t	2	mW	
39	over_volt_low	舵机电压下	uint16_t	2	mV	

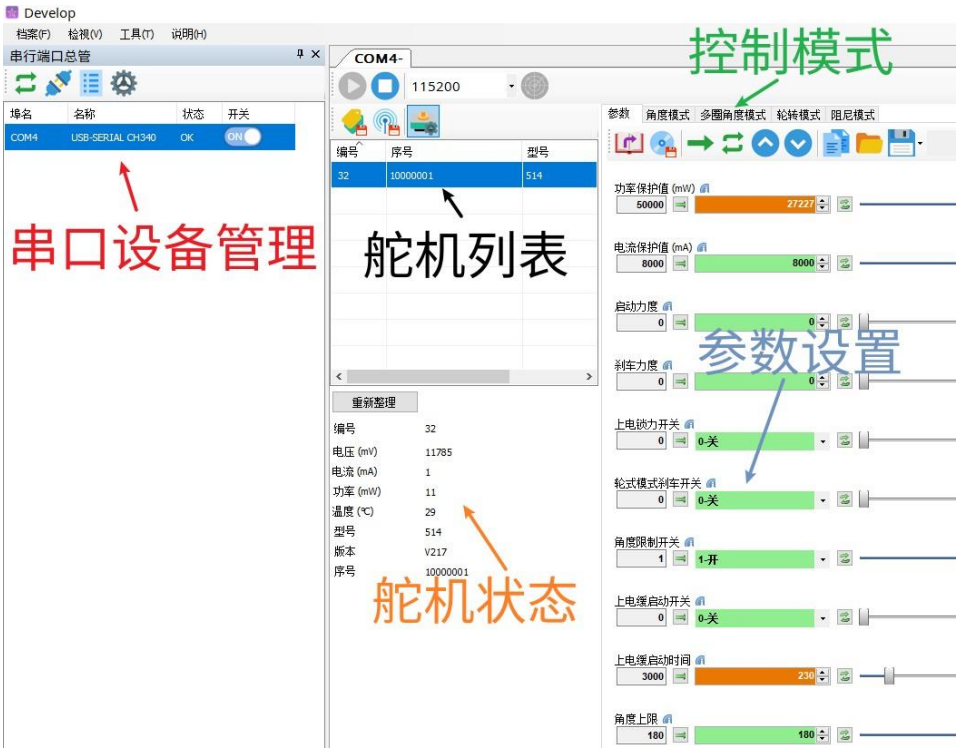
3.2、串口总线舵机上位机软件使用

我们使用的总线舵机在使用的时候，我们可以使用上位机进行调试使用，这里我们来介绍一下上位机软件的主要操作功能及操作步骤。

软件解压完成后，进入到解压软件的文件夹，点击“Develop.exe”运行上位机软件。



上位机软件功能布局：



物理接线，在初次使用的时候，我们可以连接一个舵机，不要连接两个舵机，因为有可能我们舵机的ID号是同一个，这样子冲突了，导致我们无法找到在线的舵机设备：

注意事项：

- 转接板上的舵机接口任选一个。
- 初次使用时，一次只插单个舵机，舵机的默认ID 都是 0x00，依次分配舵机ID。一般按照习惯，最下面的舵机ID设置为 0x00，从下到上依次递增，舵机 ID 分配完成之后，再将舵机串联在一起。
- 尽量不要带电拔插舵机。

3.2.1、串口连接

如果舵机转接板的串口号没有显示在串口列表，点击左上角的刷新Refresh按钮。因为转接板使用的是CH340，列表的第一个 cow就是转接板的设备号，如果没有显示端口，我们可能需要在电脑上安装GH840的驱动，在我们的资料中有提供 CH340 的驱动，我们直接点击安装即可。

按照下图所示创建串口连接。

故障排查

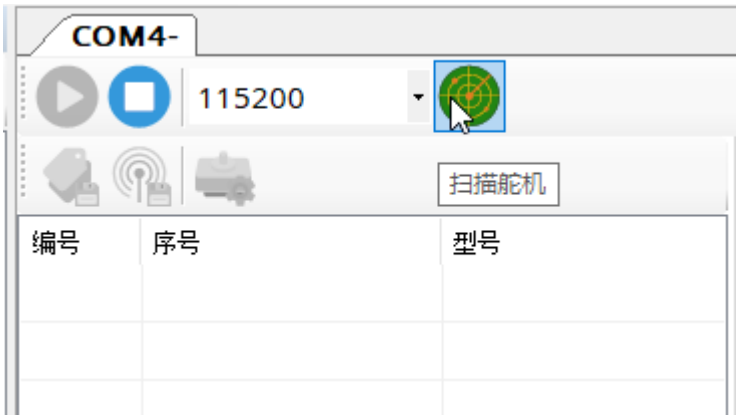
如未识别到舵机转接板：

- 可能是因为当前的操作系统并没有安装CH340驱动。
- 可能是USB端口松动，请尝试更换USB口/USB线。

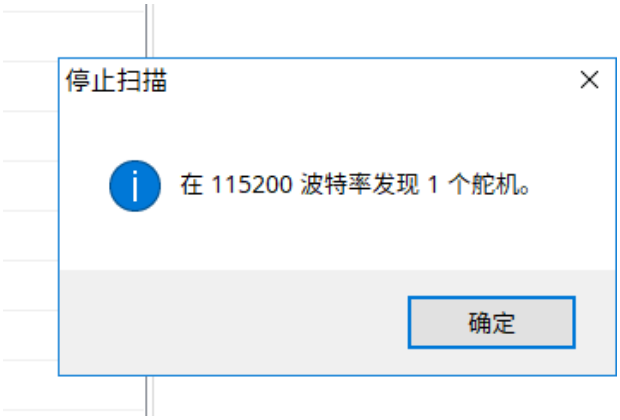
3.2.2、串口连接

如需提高扫描速度，可以进入工具--程序设定--ServoPanelInternal--最大扫描编号，进行设置，将数值调小（默认 254）。

设定舵机的波特率，并进行舵机扫描。也可选择自动扫描进行舵机扫描。

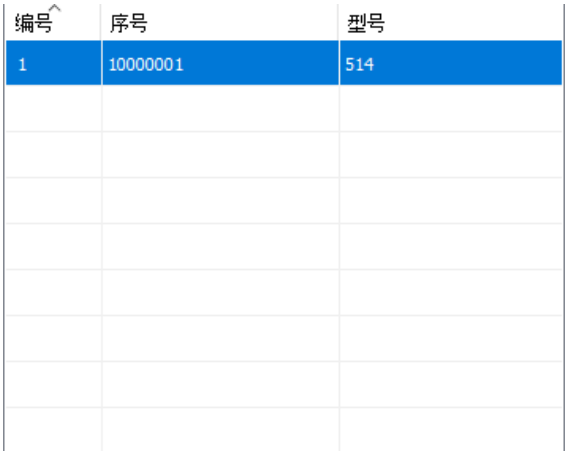


扫描完毕提示舵机个数。



点击确定，在舵机列表栏即会显示当前舵机。

舵机列表个字段的含义：



- 舵机 ID 号，是用户可以设定的舵机编号。
- 舵机序列号，是舵机的唯一标识符，可以理解为舵机的身份证，由舵机厂家出厂时设置，不可修改。
- 舵机型号，是当前舵机的型号名称，同样也属于只读字段。

故障排查

上位机卡死：

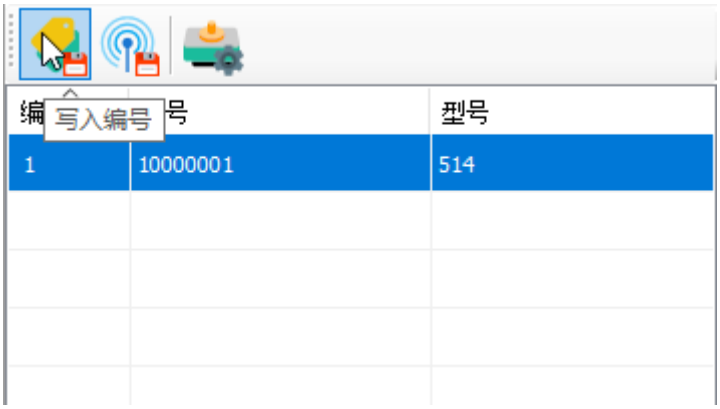
- 可能是由于当前的舵机总线上连接了多个ID号相同的舵机；初次使用的时候，一次只插单个舵机，舵机的默认ID都是1，依次分配舵机不同ID之后再将舵机串联在一起。

获取不到舵机ID：

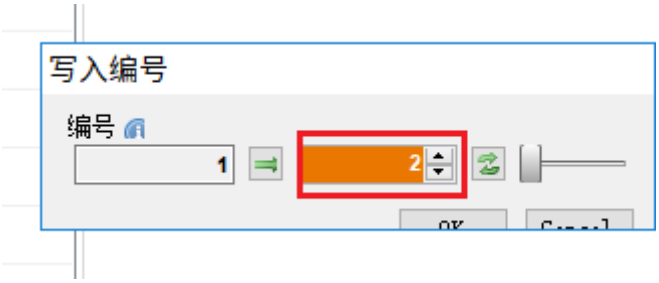
- 请检查舵机外接电源是否接上，电压是否满足条件（6.8-8.4V/高压舵机10-12.4V）：如果使用航模电池，请检查电池是否过放。

3.2.3、修改舵机 ID

在舵机列表中选中要修改ID的舵机，点击第一个图标写入编号。

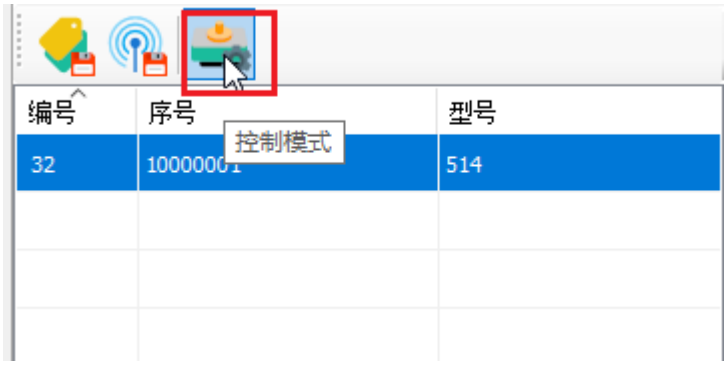


出现写入编号框，在中间框点击箭头修改或者输入编号进行修改。修改完毕，点击OK，舵机 ID 会实时更新。



3.2.4、舵机控制模式选择

选中舵机，然后点击舵机控制模式按钮。



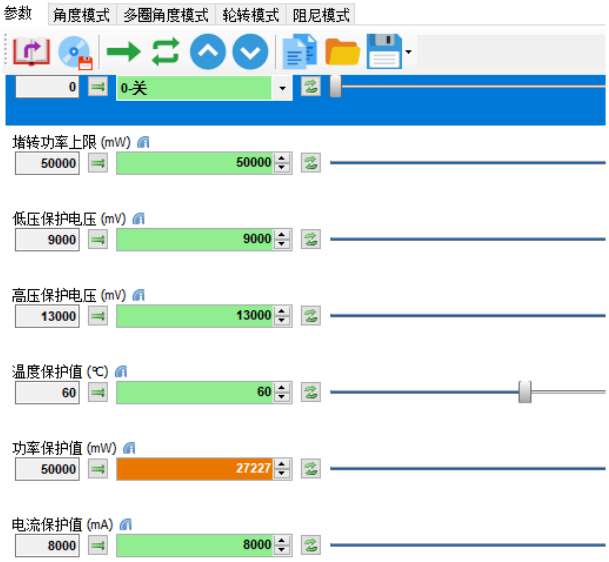
在上位机的操作界面上，一次只能控制单个舵机。

可以对舵机做如下操作

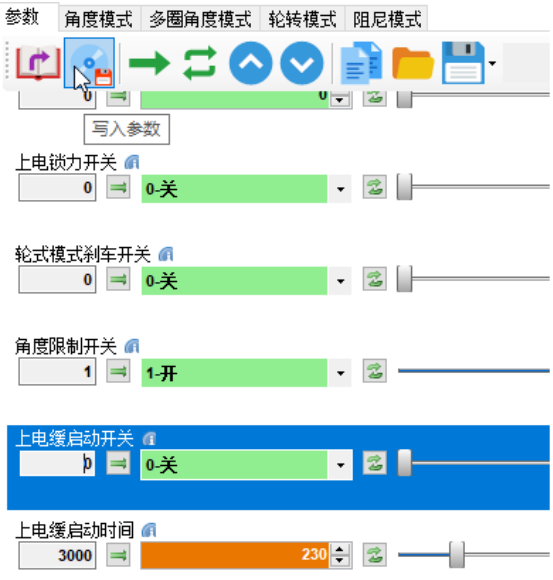
- Parameters 舵机参数设置
- Angle Mode 舵机角度模式
- Wheel Mode 舵机轮式模式
- Damping Mode 舵机阻尼模式

3.2.5、修改自定义参数

选中参数这个子窗口，你可以通过下拉框或者滑动条的方式修改舵机的参数。

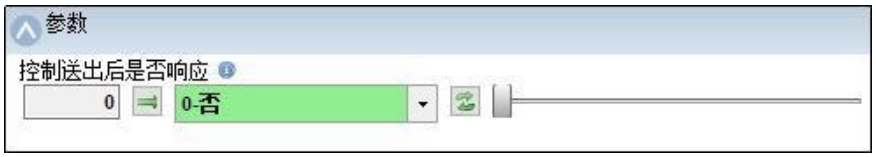


左侧是舵机原来的参数，右侧是修改之后的参数。参数修改之后，绿色会变成橙色。修改到需要的参数数值后，需要点击“写入参数”按钮。



3.2.6、上位机参数说明

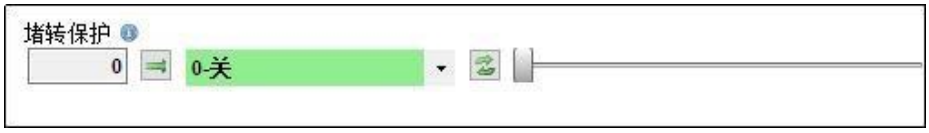
3.2.6.1 控制送出后是否是否响应



控制送出：

- 开启后，在没回应之前不接受新的控制指令；
- 关闭后，新的控制指令被立即执行；
- “0-否”：第一个命令尚未完成立即运行第二个命令；
- “1-是”：第一个命令完成后运行第二个命令；

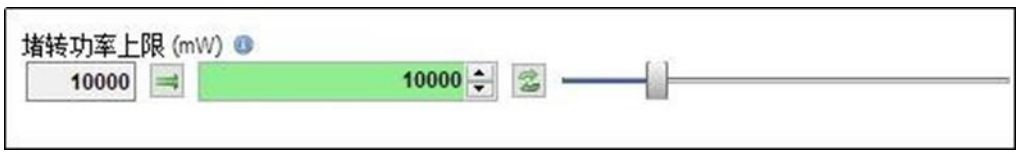
3.2.6.2 堵转保护



堵转保护：

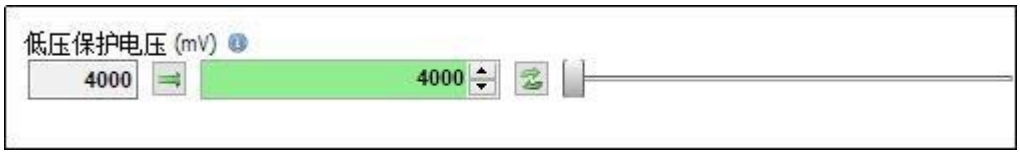
- “0-关”：降低功率到功率上限；
- “1-开”：释放舵机锁力（舵机卸力）；

3.2.6.3 堵转功率上限



- 堵转功率上限（mW）:0-50000;

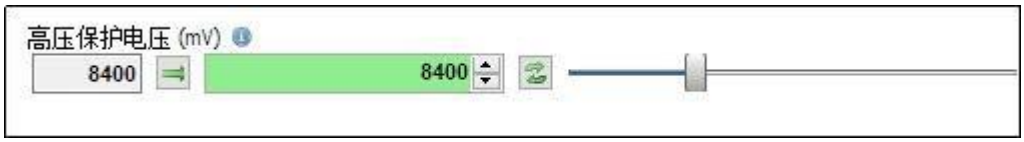
3.2.6.4 低压保护电压



低压保护电压：

- 当舵机电压低于设定的“低压保护电压值（mV）”时，舵机进入保护状态；
- 低压保护电压（mV）范围：4000-20000；

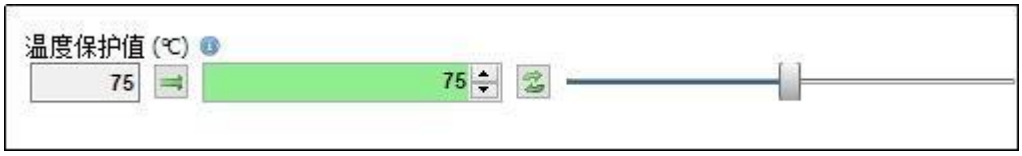
3.2.6.5 高压保护电压



高压保护电压：

- 当舵机电压超过设定的“低压保护电压值（mV）”时，舵机进入保护状态；
- 高压保护电压（mV）范围：4000-20000；

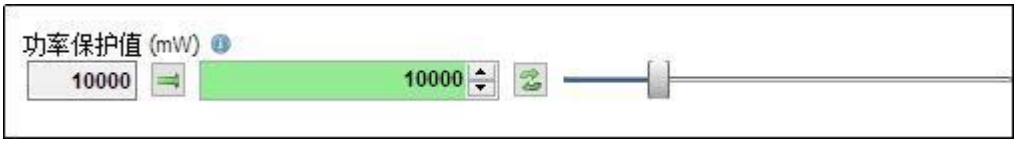
3.2.6.6 温度保护值



温度保护值：

- 当温度达到温度保护值（℃）时，舵机进入过温保护，降低输出功率；
- 温度保护值范围：50-100；

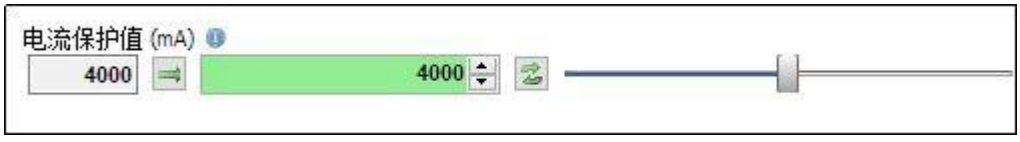
3.2.6.7 功率保护值



功率保护值：

- 当功率达到功率保护值（mW）时，舵机进入功率保护；
- 功率保护值范围：0-50000；

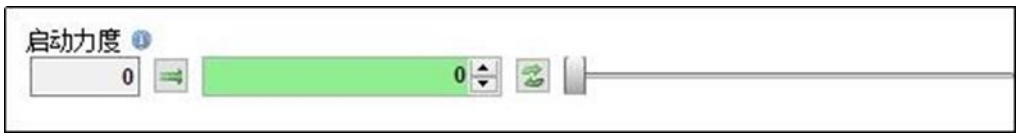
3.2.6.8 电流保护值



电流保护值：

- 当电流达到电流保护值（mA）时，舵机进入电流保护；
- 电流保护值范围：0-8000；

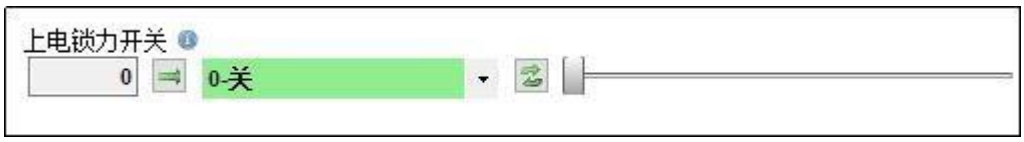
3.2.6.9 启动力度



启动力度：

- 启动力度范围：0-100;

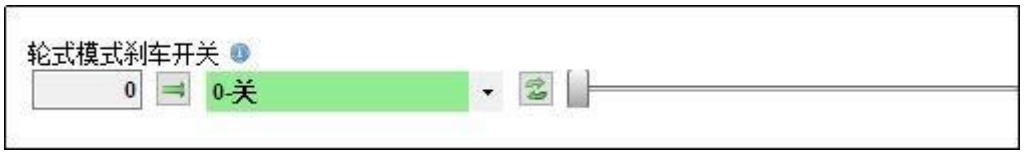
3.2.6.10 上电锁力开关



上电锁力开关：

- 上电舵机保持锁力：舵机上电时，读取舵机当前所在的角度，并让舵机维持在这一角度;
- 释放舵机锁力：不再驱动舵机维持在一个特定的角度，此时可以自由转动舵机的主轴;
- “0-关”：上电舵机释放锁力;
- “1-开”：上电舵机保持锁力;

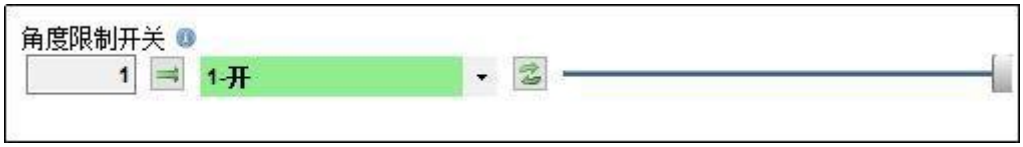
3.2.6.11 轮式模式刹车开关



轮式模式刹车开关：

- 轮式模式下，舵机停止时，设置是否释放舵机锁力;
- “0-关”：停止时释放舵机锁力;
- “1-开”：停止时保持舵机锁力;

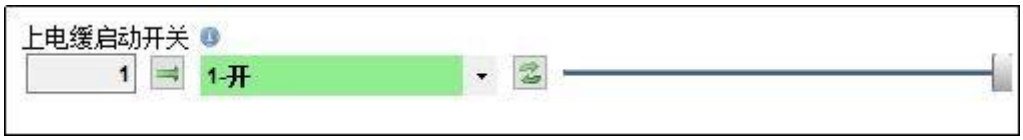
3.2.6.12 角度限制开关



角度限制开关：

- 开启角度限制后，超过限制角度范围的角度控制指令不执行;
- “0-关”：关闭角度限制;
- “1-开”：开启角度限制;

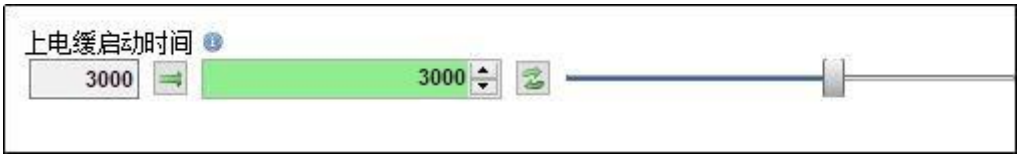
3.2.6.13 上电缓启动开关



上电缓启动开关：

- 舵机在刚开始上电时，执行第一个角度控制指令时，选择是否开启缓慢执行模式；
- “0-关”：表示取消上电缓启动开关；
- “1-开”：表示开启上电缓启动开关；

3.2.6.14 上电缓启动时间



上电缓启动时间：

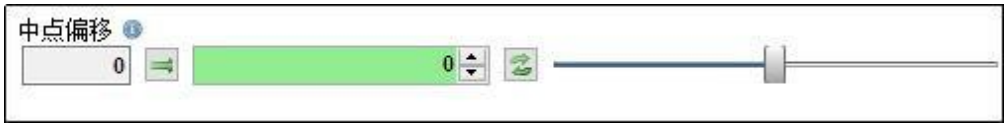
- 舵机上电时以设定的缓启动时间到达目标位置；
- 上电缓启动时间设置（ms）设定范围：0-5000；

3.2.6.15 角度上、下限

角度上、下限：

- 可设定舵机角度的最大值。
- 角度上限范围（°）10—180；
- 角度下限范围（°）-180—-10；

3.2.6.16 中点偏移



中点偏移：

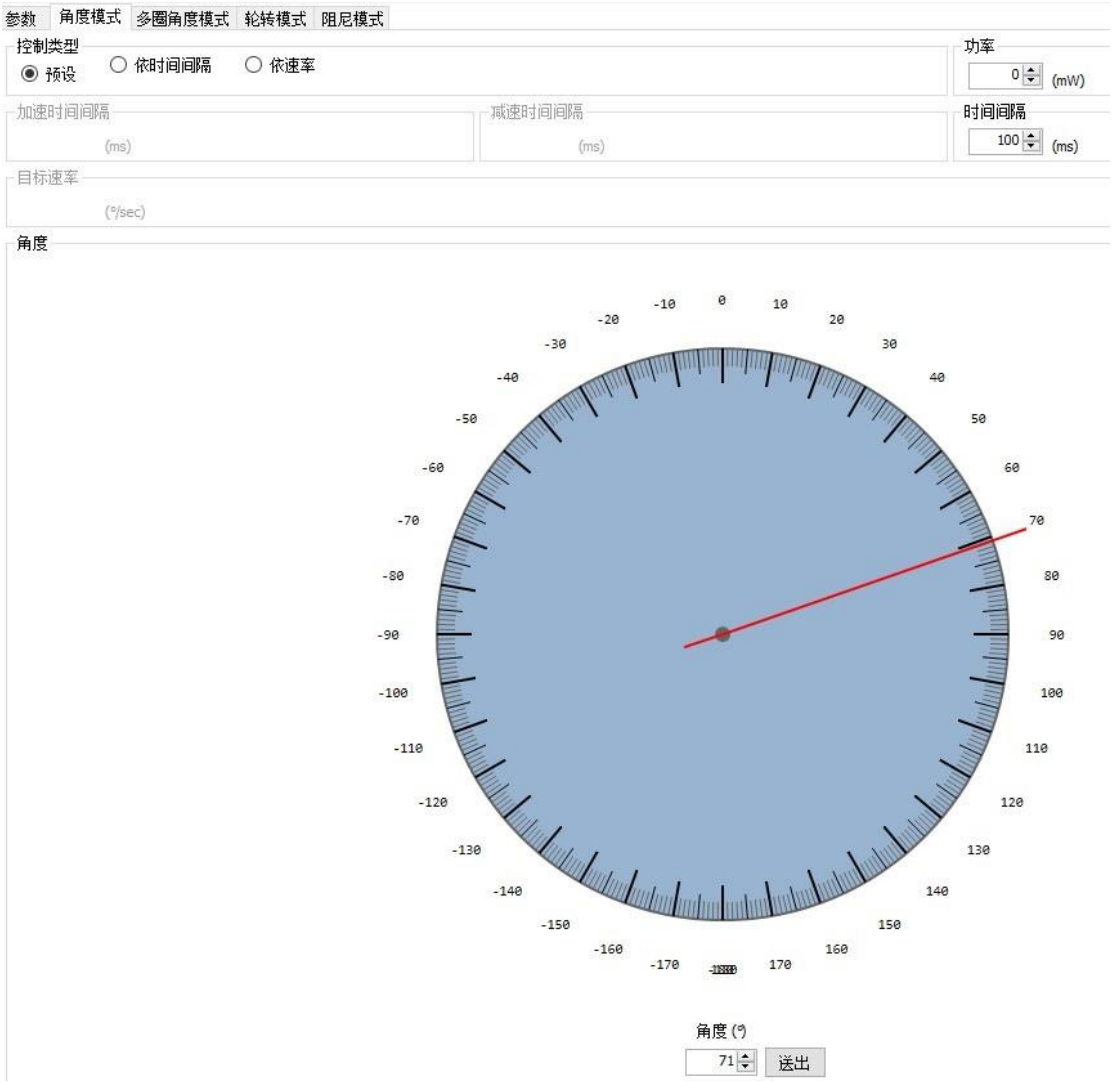
- 设置后按照偏移后的角度执行，例如中点偏移5°，角度控制指令为90°，则舵机执行到未偏移前

的 95°的位置;

- 偏移50对应5度;
- 中点偏移范围（°）-6—6;

3.2.7、角度控制模式

3.2.7.1 角度控制



输入角度，点击送出，舵机即会转到指定角度，右上方还能设置对应的时间间隔（运动时间）和功率值。

3.2.7.2 开启实时控制模式

字段详细含义，请参考<指令>设置舵机角度。

注意事项：

1. 舵机角度在上位机的输入框里的数值单位是°，实际在发送的时候，发送的数据单位是 0.1°。
2. 注意设定的舵机角度不要超出范围，默认舵机的角度范围是［-135%，135°］。也可以设置舵机的角度范围，详情参见舵机用户自定义参数列表中的舵机角度上限和舵机角度下限。
3. 在设置舵机角度时也要考虑到机械结构的角度限制（例如云台）。
4. 若无特殊要求，请设置 power=0。

3.2.8、多圈角度模式

注：多圈角度模式仅适用于磁编码舵机。

在这一模式下，是对角度模式角度值的扩展；可让舵机运动到正负 1024 圈内的任意角度。同时也提供读取当前圈数，读取当前角度，对圈数进行重置的功能。

3.2.9、轮式模式控制

轮转模式有四种控制方式。

1. 舵机停止；
2. 舵机一直旋转；
3. 定圈（旋转特定的圈数）；
4. 定时（旋转特定的时间）；

选择对应的方式，修改对应的参数，点击送出舵机开始旋转。

3.2.10、阻尼模式

阻尼模式多用于机械臂示教以及给舵机泄力。

在功率栏内填写舵机保持功率。可以修改参数为 500，在旋转舵机时可以感受到阻力增大。

3.3、UART串行总线舵机通信协议

3.3.1、串口通信配置

参数名称(en)	参数名称(cn)	参数数值	备注
baudrate	波特率	115200	串口舵机支持的多种波特率，参见<参数>波特率选项 [https://wiki.fashionrobo.com/uartbasic/uart-protocol/#634]
parity	奇偶校验	无	
stopbits	停止位	1	
bytesize	字节长度	8	

3.3.2、数据类型说明

串口通讯协议里用到的数据类型（字节大小、取值范围）

数据类型	数据名称(en)	数据名称(cn)	字节长度	最小值	最大值
uint8_t	byte	字节()	1	0	255
uint16_t	unsigned short	无符号短整型	2	0	65535
int16_t	signed short	有符号短整数	2	-32768	32767
uint32_t	unsigned long	无符号长整数	4	0	429496729
int32_t	signed long	有符号长整数	4	-2147483648	214748364
uint8_t[n]	byte array	变长字节数组	n		

注意事项：

舵机通讯协议里面使用的字节序为小端字节序，例如一个 uint16_t 的数值 4660 ，它的十六进制表示为 0x1234 。在发送/接收时，低位字节在前，高位字节在后，即以 0x34 0x12 形式发送。

3.3.3、舵机控制指令

串口舵机共支持 17 种舵机控制指令。其中 12 种为通用指令，余下 5 种指令仅限于磁编码系列舵机。

通用指令：

指令ID	指令名称(EN)	指令名称(CN)	备注
1	PING	舵机通讯检测	检查舵机是否在线
2	RESET_USER_DATA	用户自定义数据重置	重新将用户自定义数据设置为默认值
3	READ_DATA	舵机数据读取	读取单个用户自定义参数，用户自定义参数的字节长度为可变长

4	WRITE_DATA	用户自定义数据写	写入单个用户自定义数据，用户自定义参数的字节长度为可变长
5	READ_BATCH_DATA	用户自定义数据批量读取	读取所有的用户自定义数据，所有的用户自定义数据的字节长度为 32 字节
6	WRITE_BATCH_DATA	用户自定义数据批量写入	写入所有的用户自定义数据，所有的用户自定义数据的字节长度为 32 字节
7	MOVEO(EN_N)WHEEL_MODE(SPIN)	轮式模式控制	设置旋转方向、定速、定圈、定时
8	MOVE_ON_ANGLE_MODE(ROTATE)	角度模式控制-设置舵机角度	设置舵机旋转的目标角度，时间间隔，执行功率
9	MOVE_ON_DAMPING_MODE(DAMPING)	阻尼模式控制	阻尼模式，设置舵机保持的功率
10	READ_ANGLE	读取舵机角度	
11	MOVE_ON_ANGLE_MODE_EX_BY_INTERVAL	角度模式控制-指定时间	带加减速控制的角度模式，设置加减速时间，时间间隔，目标角度，执行功率
12	MOVE_ON_ANGLE_MODE_EX_BY_VELOCITY	角度模式控制-指定速度	带加减速控制的角度模式，设置加减速时间，时间间隔，目标角度，执行功率

注意事项：

1. 所有的舵机指令都是针对单个舵机的。
2. 批量读/写针对的都是单个舵机的所有的用户自定义数据，而不是所有的舵机。
3. 发送控制指令之前，请确保串联在一起的舵机每个舵机的 ID 号都不一样。
4. 批量读取跟批量写入针对的都是用户自定义参数区块。

3.3.4、数据包

3.3.4.1 请求数据包

字节序号	数据类型	字段名称	字段功能描述
0	uint8_t	0x12	请求头 标识
1	uint8_t	0x4C	请求头 标识
2	uint8_t	cmd_id	舵机控制指令 ID
3	uint8_t	n	数据包内容的字节长度
[4,4+n-1]	uint8_t[n]	content	据包的内容部分，变长

4+n	uint8_t	checksum	数据包的校验和。计算方法：对之前所有的 字节数据求和(0 号到 4+N-1 号字节)，然后跟 256 取余数

3.3.4.2 响应数据包

请求数据包跟响应数据包数据结构的唯一区别就在于前两个字节的标识符。

字节序号	数据类型	字段名称	字段功能描述
0	uint8_t	0x05	响应头标识
1	uint8_t	0x1C	响应头标识
2	uint8_t	cmd_id	舵机控制指令 ID
3	uint8_t	n	数据包内容的字节长度
[4,4+n-1]	uint8_t[n]	content	数据包的内容部分
4+n	uint8_t	checksum	数据包的校验和。计算方法：对之前所有的 字节数据求和(0 号到 4+N-1 号字节)，然后跟 256 取余数

注意事项：

1. 封包内容的第一个字节，往往是舵机的 ID 号，舵机的 ID 号取值范围为[0, 254] ，255 是舵机广播地址。
2. 字节序号是从 0 开始算的。
3. 请求包的 cmd_id 跟响应包的 cmd_id 一致。
4. 响应数据的接收流程。

具体的通讯方式我们提供了一个手册，对于编程来说，我们直接提供了对于操作总线舵机的 python 库简化了整个操作过程。

3.4、Jetson Nano的AI计算优势

Jetson Nano采用四核64位ARMCPU和128核集成NVIDIA GPU,可提供472 GFLOPS的计算性能。它还包括4GB LPDDR4存储器，采用高效、低功耗封装，具有5W/10W功率模式和5VDC输入。

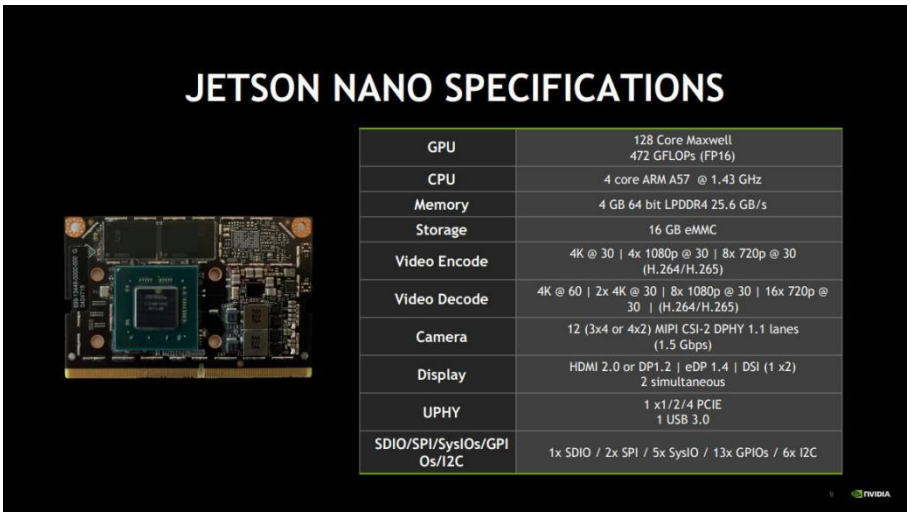


图3.1 Jetson Nano参数

Jetson Nano 基于配备了图形加速的Ubuntu18.04操作系统，全新发布的JetPack SDK为其提供了完整的桌面Linux环境支持，NVIDIA CUDA 工具包，以及cuDNN 和 TensorRT等库。该SDK还包括本机安装的常用开源机器学习（ML)框架，如TensorFlow、PyTorch、Caffe、Keras和MXNet,以及计算机视觉和机器人开发的框架，如OpenCV和ROS.它与这些框架和NVIDIA领先的AI平台完全兼容，可以轻松地将基于AI的推理工作负载部署到Jetson.Jetson Nano能为各种复杂的深度神经网络（DNN)模型提供实时计算机视觉和推理。这些功能支持多传感器自主机器人，以及具有智能边缘分析的物联网设备和先进的AI系统。开发人员甚至可以通过迁移学习，使用机器学习框架在Jetson Nano 本地重新训练网络。

3.4.1、深度学习推理基准

NVIDIA Jetson Nano是个通用性极高的嵌入式高性能设备，适合绝大多数应用场景，并且基于NVIDIA的CUDA统一架构，整个开发生态已积累10年以上的资源与经验，您可以按照最后面提供的范例在Jetson Nano上轻松执行，体验一下NVIDIA开发资源的便利性，以及Jetson Nano的优异性能。当然，开发人员也能改写这些范例代码，轻松开发出合适的应用，立即展现成果。

从Jetson TX2到Jetson AGX Xavier，再到Jetson Nano，Jetson 解决方案均由同一软件堆栈提供支持，可让公司进行一次开发，即可在任意位置部署。Jetson平台由JetPack SDK 提供支持，包括板级支持包 (BSP)、Linux 操作系统、NVIDIA CUDA，并且兼容第三方平台。JetPack 可提供用于深度学习、计算机视觉、加速计算和多媒体的库，而且支持各类传感器的驱动程序。 NVIDIA 还为 JetPack 提供了可以加快开发过程的大量开发者工具，而且提供了有关应用和系统功率和性能的详细信息，因此，有助于开发者快速优化和调

整代码。

JetPack还包括加速库和API :

- 深度学习:TensorRT和cuDNN
- 计算机视觉:OpenCV和VisionWorks
- 加速计算:GPU库，如cuBLAS(用于线性代数)和cuFFT(用于傅里叶变换)等
- 计算机图形:完整的桌面OpenGL 4.6、OpenGL ES 3.2和Vulkan 1.1
- 多媒体:相机开发用libargus，视频管理用视频API

NVIDIA花了十年时间一直在建立和推动人工智能平台旨在帮助工程师和数据科学家训练、优化和部署神经网络。

工程师和数据科学家喜欢为不同的应用程序使用不同的框架。目前我们测试的结果来看，Jetson Nano支持目前很流行的深度学习框架，包括TensorFlow、PyTorch、MxNet、Keras、affe/ Caffe2等。通过实现图像识别，对象检测和定位，姿势估计，语义分割，视频增强和智能分析等强大功能，这些网络可用于构建自动机器和复杂AI系统。注意NVIDIA开源机器人JetBot项目使用的就是TensorFlow。

在Jetson Nano上，用户可以搭配NVIDIA TensorRT推理引擎、混合精度的优化，轻易地把推理性能再往上推升数倍。搭配NVIDIA支持Caffe、Tensorflow、Torch的DIGITS深度学习服务器，用户可以非常轻松地将自己（或别人）训练好的特定模型，轻松移植到Jetson Nano去执行专属的应用，这有机会让开发人员减少70%的研发时间。

图3.2显示了在线提供的流行模型的推理基准测试结果。有关在 Jetson Nano上运行这些基准测试的说明，请参见 此处。推理使用批量1和FP16精度，采用 JetPack 4.2附带的NVIDIA TensorRT加速器库。Jetson Nano在许多场景中都具有实时性能，能够处理多个高清视频流。

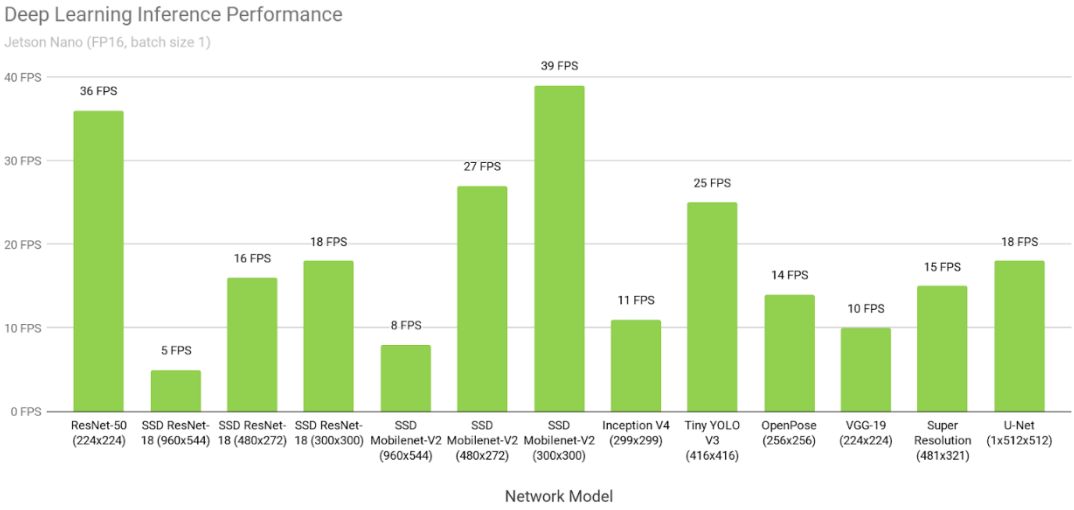


图3.2 常见模型推理测试结果

由于内存容量有限，网络层不受支持或硬件/软件限制，DNR（未运行）结果频繁发生。固定功能神经网络加速器通常支持相对较窄的一组用例，硬件支持专用层操作，需要网络权重和激活以适应有限的片上高速缓存，以避免重大的数据传输损失。它们可能会回退到主机CPU上以运行硬件中不支持的层，并且可能依赖于支持减少的框架子集的模型编译器（例如，TFLite）。

Jetson Nano灵活的软件和完整的框架支持，内存容量和统一内存子系统使其能够运行多种不同的网络，达到全高清分辨率，包括同时在多个传感器流上的可变批量大小。这些基准测试代表了流行网络的一些示例，但用户可以通过加速性能为Jetson Nano部署各种模型和定制架构。而Jetson Nano不仅限于DNN推理。其CUDA架构可用于计算机视觉和数字信号处理（DSP），使用包括FFT，BLAS和LAPACK操作在内的算法，以及用户定义的CUDA内核。

3.4.2、多流视频分析

Jetson Nano可实时处理多达8个高清全动态视频流，并可部署为网络视频录像机（NVR），智能相机和物联网网关的低功耗边缘智能视频分析平台。NVIDIA的DeepStream SDK 使用ZeroCopy和TensorRT优化端到端推理管道，以在边缘和本地服务器上实现最佳性能。如Jetson Nano可以在8个1080p30流上同时执行物体检测，基于ResNet的模型以全分辨率运行，吞吐量为每秒500万像素（MP / s）。

在Jetson Nano上运行的DeepStream应用程序，基于ResNet的对象检测器同时在八个独立的1080p30

视频流上运行。

图3.3框图显示了使用Jetson Nano通过深度学习分析在千兆以太网上摄取和处理多达八个数字流的示例NVR架构。该系统可解码500 MP / s的H.264 / H.265，并编码250 MP / s的H.264 / H.265视频。

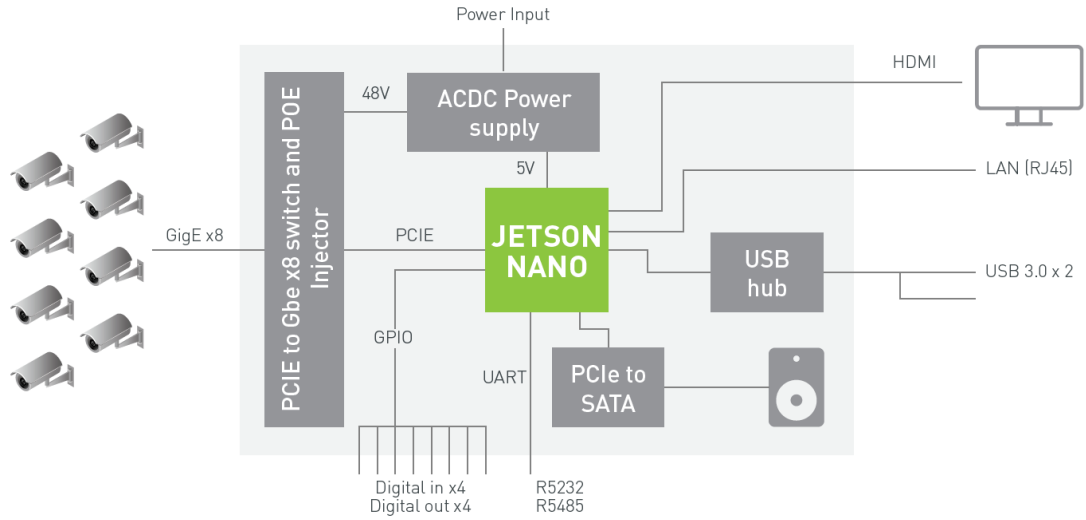


图3.3 带有Jetson Nano和8x HD摄像头输入的参考NVR系统架构

3.4.3、Hello AI World

Hello AI World 提供了一个很好的方式来开始使用Jetson并体验AI的强大功能。在短短几个小时内，您就可以使用JetPack SDK和NVIDIA TensorRT在Jetson Nano Developer Kit上进行一系列深度学习推理演示，并进行实时图像分类和对象检测（使用预训练模型）。本教程重点介绍与计算机视觉相关的网络，并包括使用实时摄像头。您还可以使用C ++编写自己易于理解的识别程序。可用的深度学习ROS节点 将这些识别，检测和分段推理功能与ROS集成在一起 适用于先进的机器人系统和平台。这些实时推理节点可以轻松地放入现有的ROS应用程序中。

图3.4 图像检测识别示例

想要尝试培训他们自己的模型的开发人员可以遵循完整的“ 两天演示 ”教程，该教程涵盖了图像分类，对象检测和带有转移学习的语义分割模型的重新训练和定制。传递学习可以精确调整特定数据集的模型权重，并避免必须从头开始训练模型。传输学习最有效地在连接了NVIDIA独立GPU的PC或云实例上执行，因为培训需要比推理更多的计算资源和时间。

然而，由于Jetson Nano可以运行TensorFlow，PyTorch和Caffe等完整的培训框架，因此它还能够为那些可能无法访问另一台专用培训机并且愿意等待更长时间才能获得结果的人进行转学。下面表格突出了PyTorch使用Jetson Nano在200天图像，22.5GB ImageNet子集上训练Alexnet和ResNet-18的两天到演示教程的转移学习的初步结果：

网络	批量大小	每个时期的时间	张/秒
AlexNet	64	1.16 小时	45
RESNET-18	64	3.22 小时	16

每个时期的时间是完全通过200K图像的训练数据集所需的时间。对于可用结果，分类网络可能需要2-5个时期，并且应该在离散GPU系统上训练生产模型以获得更多时期，直到它们达到最大准确度。但是，Jetson Nano让您可以通过让网络在一夜之间重新培训，在低成本平台上体验深度学习和人工智能。并非所有自定义数据集都可能与此处使用的22.5GB示例一样大。因此，图像/秒表示Jetson Nano的训练性能，每个时期的时间缩放与数据集的大小，训练批量大小和网络复杂性。其他型号也可以在Jetson Nano上重新训练，同时增加训练时间。

3.5、OpenCV 计算机视觉库介绍

当下人工智能的发展是前所未有的迅猛，而计算机视觉(Computer Vision)无疑是人工智能一个非常重要的分支，计算机视觉的发展也成为了很多科研人员和业界开发人员聚焦的热点。我们常说的无人驾驶、智能安防、人脸识别、文字识别、以图搜图、VR/AR、3D重构等等都是以计算机视觉做为技术基础的应用实践。

在计算机视觉项目的开发中，OpenCV作为最大众的开源库，拥有了丰富的常用图像处理函数库，采用C/C++语言编写，可以运行在Linux/Windows/Mac等操作系统上，能够快速的实现一些图像处理和识别的任务。此外，OpenCV还提供了java、python、cuda等的使用接口、机器学习的基础算法调用，从而使得图像处理和图像分析变得更加易于上手，让开发人员更多的精力花在算法的设计上。如果你对OpenCV计算机视觉库开发很感兴趣想深入了解的话下面提供了几个网站可供大家参考研究学习：

Open CV 官方网站： https://opencv.org/

Open CV 官方文档 ： https://docs.opencv.org/master/

Open CV 官方教程： https://docs.opencv.org/master/d9/df8/tutorial_root.html

Open CV 中文站： http://www.opencv.org.cn/forum.php?mod=viewthread&tid=33549

3.5.1、OpenCV 的起源背景

OpenCV诞生于Intel研究中心，其目的是为了促进CPU密集型应用。为了达到这一目的，Intel启动了多个项目，包括实时光线追踪和三维显示墙。一个在Intel工作的OpenCV作者在访问一些大学时，注意到许多顶尖大学中的研究组（如MIT媒体实验室）拥有很好的内部使用的开放计算机视觉库（在学生们间互相传播的代码），这会帮助一个新生从一个很高的起点开始ta的计算机视觉研究。这样一个新生可以在以前的基础上继续开始研究，而不用去从底层编写基本函数。

因此，OpenCV的目的是开发一个普遍可用的计算机视觉库。在Intel的性能库团队的帮助下，OpenCV实现了一些核心代码以及算法，并发给Intel俄罗斯的库团队。这就是OpenCV的诞生之地：在与软件性能库团队的合作下，它开始于Intel的研究中心，并在俄罗斯得到实现和优化。

俄罗斯团队的主要负责人是俄罗斯人Vadim Pisarevsky，他负责管理项目、写代码并优化OpenCV的大部分代码，在OpenCV中很大一部分功劳都属于他。跟他一起，Victor Eruhimov帮助开发了早期的架构，Valery Kuriakin管理俄罗斯实验室并提供了很大的支持。在开始时，OpenCV就定下了以下三大目标：

（1）为基本的视觉应用提供开放且优化的源代码，以促进视觉研究的发展。能有效地避免“闭门造车”。

（2）通过提供一个通用的架构来传播视觉知识，开发者可以在这个架构上继续开展工作，所以代码应该是非常易读的且可改写。

（3）本库采用的协议不要求商业产品继续开放代码，这使得可移植的、性能被优化的代码可以自由获取，可以促进基于视觉的商业应用的发展。

这些目标说明了OpenCV的缘起。计算机视觉应用的发展会增加对快速处理器的需求。与单独销售软件相比，促进处理器的升级会为Intel带来更多收入。这也许是为什么这个开放且免费的库出现在一家硬件生产企业中，而不是在一家软件公司中。从某种程度上说，在一家硬件公司里，在软件方面会有更多创新的空间。就像华为这样的网络通讯设备公司，与硬件相配套的软件也是非常的强大！

3.5.2、OpenCV 简介

OpenCV的全称是：Open Source Computer Vision Library。OpenCV是一个基于BSD许可（开源）发行的跨平台计算机视觉库，可以运行在Linux、Windows、Android和Mac OS操作系统上。它是轻量级的，而且是高效的，由一系列C函数和少量C++类构成，同时提供了Python、Ruby、MATLAB等语言的接口，实现了图像处理和计算机视觉方面的很多通用算法。

OpenCV用C++语言编写，它的主要接口也是C++语言，但是依然保留了大量的C语言接口。该库也有大量的Python、Java and MATLAB/OCTAVE的接口。这些语言的API接口函数可以通过在线文档获得。如今也提供对于C#、Ruby等的支持。

OpenCV是一个用于图像处理、分析、机器视觉方面的开源函数库。无论你是做科学研究，还是商业

应用，OpenCV都可以作为你理想的工具库，因为对于这两者，它完全是免费的。

同时，由于计算机视觉与机器学习密不可分，该库也包含了比较常用的一些机器学习算法，比如图像识别、机器视觉在安防领域有广泛的应用。但很少有人知道，在航拍图片、街道图片（例如google street view）中，要严重依赖于机器视觉的摄像头标定、图像融合等技术。

近年来，在入侵检测、特定目标跟踪、目标检测、人脸检测、人脸识别、人脸跟踪等领域，OpenCV可谓大显身手，而这些，仅仅是其应用的冰山一角。

如今，来自世界各地的各大公司、科研机构的研究人员，共同维护支持着OpenCV的开源库开发。这些公司和机构包括：微软，IBM，索尼、西门子、google、intel、斯坦福、MIT、CMU、剑桥。

计算机视觉市场巨大而且持续增长，且这方面没有标准API，如今的计算机视觉软件大概有以下三种：

（1）研究代码（慢，不稳定，独立并与其他库不兼容）

（2）耗费很高的商业化工具（比如Halcon, MATLAB+Simulink）

（3）依赖硬件的一些特别的解决方案（比如视频监控，制造控制系统，医疗设备）这是如今的现状。

如果能有一个统一的标准API，将简化计算机视觉程序和解决方案的开发，而OpenCV致力于成为这样的标准API。

OpenCV 致力于真实世界的实时应用，通过优化的 C 代码的编写对其执行速度带来了可观的提升，并且可以通过购买 Intel 的 IPP 高性能多媒体函数库(Integrated Performance Primitives)得到更快的处理速度（注：OpenCV 2.0 版的代码已显著优化，无需 IPP 来提升性能，故 2.0 版不再提供 IPP 接口）。下图为 OpenCV 与当前其他主流视觉函数库的性能比较。

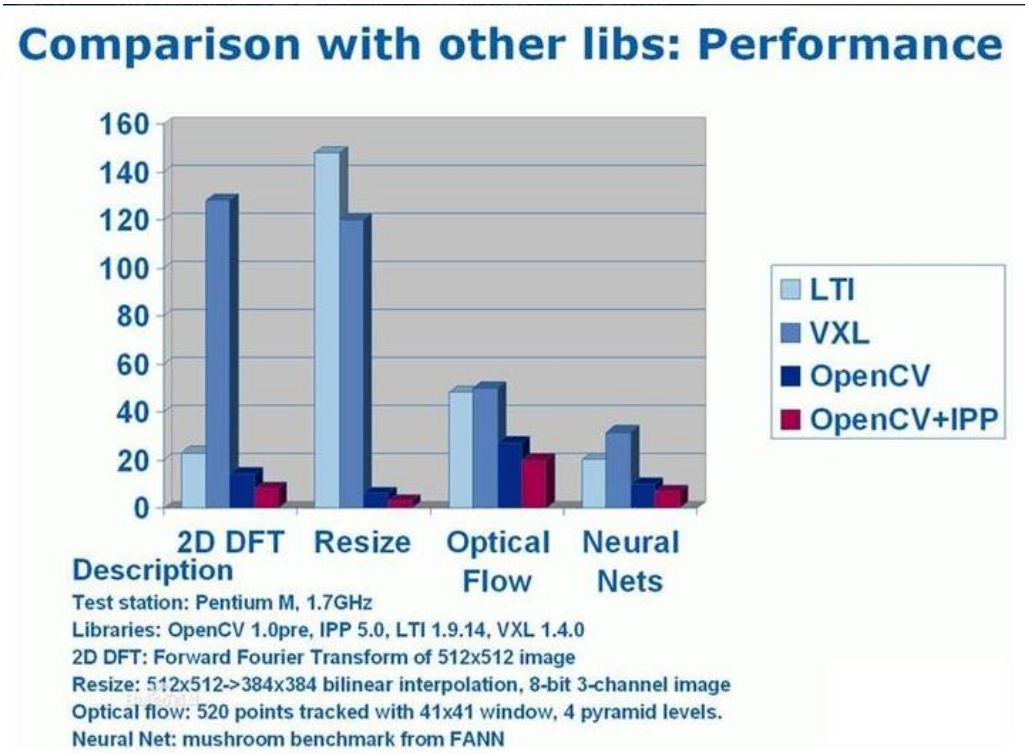


图 3.5 视觉函数库性能比较

3.6、TensorFlow AI 框架介绍

3.6.1、TensorFlow 简介

TensorFlow™ 是一个采用数据流图（data flow graphs），用于数值计算的开源软件库。节点（Nodes）在图中表示数学操作，图中的线（edges）则表示在节点间相互联系的多维数据数组，即张量（tensor）。它灵活的架构让你可以在多种平台上展开计算，例如台式计算机中的一个或多个CPU（或GPU），服务器，移动设备等等。TensorFlow 最初由Google大脑小组（隶属于Google机器智能研究机构）的研究员和工程师们开发出来，用于机器学习和深度神经网络方面的研究，但这个系统的通用性使其也可广泛用于其他计算领域。Tensorflow是谷歌公司在2015年9月开源的一个深度学习框架。

数据流图用“结点”（nodes）和“线”(edges)的有向图来描述数学计算。“节点” 一般用来表示施加的数学操作，诸如加、减、乘等运算，但也可以表示数据输入（feed in）的起点/输出（push out）的终点，或者是读取/写入持久变量（persistent variable）的终点。“线”表示“节点”之间的输入/输出关系。这些数据“线”可以输运“size可动态调整”的多维数据数组，即“张量”（tensor）。张量从图中流过的直观图像是这个工具取名为“TensorFlow”的原因。一旦输入端的所有张量准备好，节点将被分配到各种计算设备完成异步并行地执行运算。

TensorFlow 相关内容比较多，如果需要详细了解要做一个系统性的学习，如果大家比较感兴趣的话可以查询相关网站进行详细学习了解！

NVIDIA 官方网站TensorFlow 学习指南：

https://docs.nvidia.com/deeplearning/frameworks/tensorflow-user-guide/

TensorFlow 官方网站：

https://tensorflow.google.cn/learn?hl=zh-cn

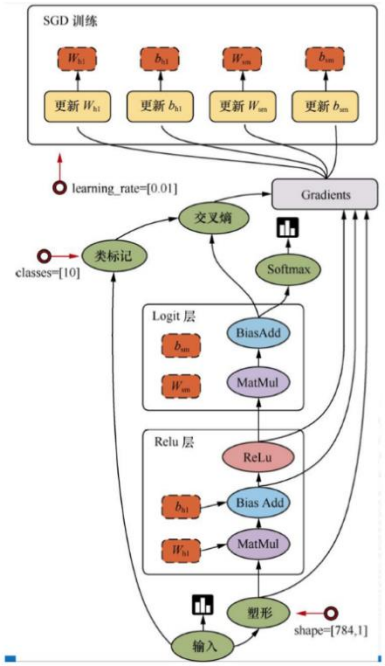


图3.6 数据流图

3.6.2、TensorFlow 的特征

高度的灵活性

TensorFlow 不是一个严格的“神经网络”库。只要你可以将你的计算表示为一个数据流图，你就可以使用TensorFlow。你来构建图，描写驱动计算的内部循环。我们提供了有用的工具来帮助你组装“子图”（常用于神经网络），当然用户也可以自己在TensorFlow基础上写自己的“上层库”。定义顺手好用的新复合操作和写一个python函数一样容易，而且也不用担心性能损耗。当然万一你发现找不到想要的底层数据操作，你也可以自己写一点c++代码来丰富底层的操作。

真正的可移植性（Portability）

TensorFlow 在CPU和GPU上运行，比如说可以运行在台式机、服务器、手机移动设备等等。想要在没有特殊硬件的前提下，在你的笔记本上跑一下机器学习的新想法？TensorFlow可以办到这点。准备将你的训练模型在多个CPU上规模化运算，又不想修改代码？TensorFlow可以办到这点。想要将你的训练好的模型作为产品的一部分用到手机app里？TensorFlow可以办到这点。你改变主意了，想要将你的模型作为云端服务运行在自己的服务器上，或者运行在Docker容器里？TensorFlow也能办到。

将科研和产品联系在一起

过去如果要将科研中的机器学习想法用到产品中，需要大量的代码重写工作。那样的日子一去不复返了！在Google，科学家用TensorFlow尝试新的算法，产品团队则用TensorFlow来训练和使用计算模型，并直接提供给在线用户。使用TensorFlow可以让应用型研究者将想法迅速运用到产品中，也可以让学术性研究者更直接地彼此分享代码，从而提高科研产出率。

自动求微分

基于梯度的机器学习算法会受益于TensorFlow自动求微分的能力。作为TensorFlow用户，你只需要定义预测模型的结构，将这个结构和目标函数（objective function）结合在一起，并添加数据，TensorFlow将自动为你计算相关的微分导数。计算某个变量相对于其他变量的导数仅仅是通过扩展你的图来完成的，所以你能一直清楚看到究竟在发生什么。

多语言支持

TensorFlow 有一个合理的c++使用界面，也有一个易用的python使用界面来构建和执行你的graphs。你可以直接写python/c++程序，也可以用交互式的ipython界面来用TensorFlow尝试些想法，它可以帮你将笔记、代码、可视化等有条理地归置好。当然这仅仅是个起点——我们希望能鼓励你创造自己最喜欢的语言界面，比如Go，Java，Lua，Javascript，或者是R。

性能最优化

比如说你又一个32个CPU内核、4个GPU显卡的工作站，想要将你工作站的计算潜能全发挥出来？由于TensorFlow 给予了线程、队列、异步操作等以最佳的支持，TensorFlow 让你可以将你手边硬件的计算潜能全部发挥出来。你可以自由地将TensorFlow图中的计算元素分配到不同设备上，TensorFlow可以帮你管理好这些不同副本。

3.3.3、TensorFlow 中的概念

Tensorflow中最重要的几个概念：图（Graph）、会话（session）、张量（tensor）、计算节点（Operation）

图（Graph）：它包含了一个计算任务中的所有变量和计算方式。可以将它和C#中的表达式树进行类比。例如，一个1+2可以被看作为两个常量表达式，以一个二元运算表达式连接起来。在Tensorflow的世界中，则可以看成是两个tensor和一个op（operation的缩写，即操作）。简单来说，做一个机器学习的任务就是计算一张图。

在计算图之前，当然要把图建立好。例如，计算（1+2）*3再开根号，是一个包括了3个tensor和3个Op的图。

不过，Tensorflow的图和常规的表达式还有所不同，Tensorflow中的节点变量是可以被递归的更新的。我们所说的"训练"，也就是不停的计算一个图，获得图的计算结果，再根据结果的值调整节点变量的值，然后根据新的变量的值再重新计算图，如此重复，直到结果令人满意（小于某个阈值），或跑到了一个无穷大/小（这说明图的变量初始值设置的有问题），或者结果基本不变了为止。

前面说过，从内部机制上来说，TF就是建立数据流图来进行数值计算。所以，当你使用TF来搭建模型时，其实主要涉及两个方面：根据模型建立计算图，然后送入数据运行计算图得到结果。计算图computational graph是TF中很重要的一个概念，其是由一系列节点（nodes）组成的图模型，每个节点对应的是TF的一个算子（operation）。每个算子会有输入与输出，并且输入和输出都是张量。所以我们使用TF的算子可以构建自己的深度学习模型，其背后的就是一个计算图。还有一点这个计算图是静态的，意思是这个计算图每个节点接收什么样的张量和输出什么样的张量已经固定下来。要运行这个计算图，你需要开启一个会话（session），在session中这个计算图才可以真正运行。

我们用一个简单的例子来说明图模型的机理，首先定义一个计算图：

```
public static void BasicOperation2()
{
    using (var s = new TFSession())
    {
        var g = s.Graph;

        var a = g.Const(5);
        var b = g.Const(3);

        var c = g.Mul(a, b);
        var d = g.Add(a, b);
        var e = g.Add(c, d);

        var runner = s.GetRunner();

        var result = runner.Run(e);

        Console.WriteLine($"相加的结果:{result.GetValue()}");
```

```
        // 相加的结果:23
    }
}
```

代码很简单，我们引入TF库，然后利用TF的基本算术算子（和Numpy接口一致）创建了一个简单的计算图如下图所示。其中每个圆圈代表的是每个算子定义的节点。节点之间的边就是张量，这里其实都是标量（scalers）。

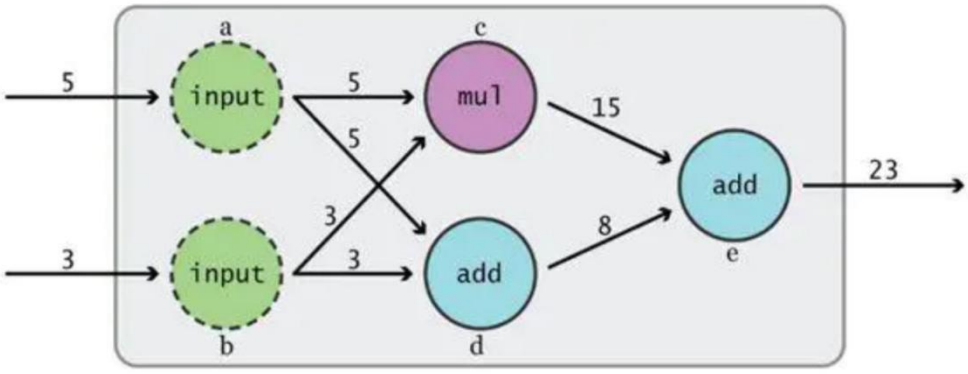


图3.7 计算图实例

当我们想得到e的值时，TF会根据创建好的计算图查找e节点所有的依赖节点，这可以看成一个抽取子计算图的过程。得到这个子计算图就可以从a和b节点开始计算最终得到e的值。还有一点TF会对这个计算过程做并行化优化，不过这都是底层的事了。TF更灵活的一点你可以指定各个节点在哪个具体的设备上运行，如CPU和GPU。理解好计算图，就掌握了TF的基本运作原理。

会话（Session）：为了获得图的计算结果，图必须在会话中被启动。图是会话类型的一个成员，会话类型还包括一个runner，负责执行这张图。会话的主要任务是在图运算时分配CPU或GPU。

张量（tensor）：TensorFlow中所有的输入输出变量都是张量，而不是基本的int，double这样的类型，即使是一个整数1，也必须被包装成一个0维的，长度为1的张量【1】。一个张量和一个矩阵差不多，可以被看成是一个多维的数组，从最基本的一维到N维都可以。张量拥有阶（rank），形状（shape），和数据类型。其中，形状可以被理解为长度，例如，一个形状为2的张量就是一个长度为2的一维数组。而阶可以被理解为维数。

阶	数学实例	Python 例子
0	纯量（只有大小）	s = 483
1	向量(大小和方向)	v = [1.1, 2.2, 3.3]
2	矩阵(数据表)	m = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
3	3阶张量（数据立体）	t = [[[2], [4], [6]], [[8], [10], [12]], [[14], [16], [18]]]

对于任何深度学习框架，你都要先了解张量（Tensor）的概念，张量可以看成是向量和矩阵的衍生。向量是一维的，而矩阵是二维的，对于张量其可以是任何维度的。一般情况下，你要懂得张量的两个属性：形状（shape）和秩（rank）。秩很好理解，就是有多少个维度；而形状是指的每个维度的大小。图3.8是常见的张量的形象图表示：

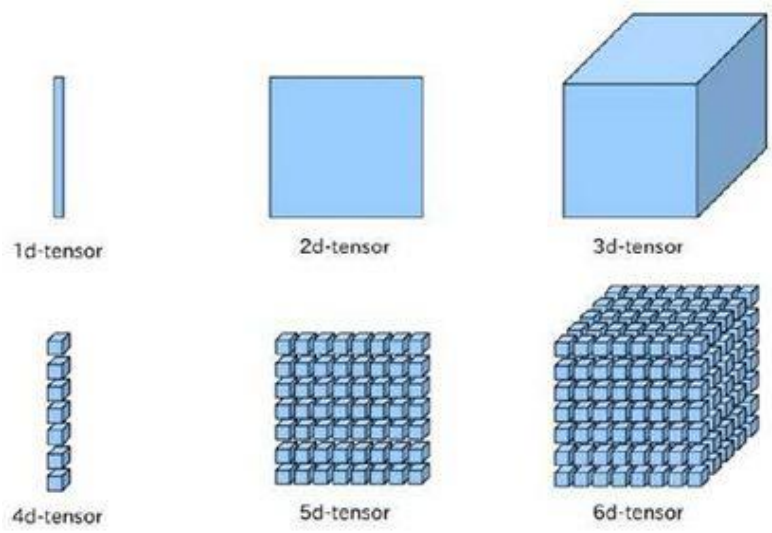


图3.8 张量的形象图表示

名字就是TensorFlow，直观来看，就是张量的流动。张量(tensor)，即任意维度的数据，一维、二维、三维、四维等数据统称为张量。而张量的流动则是指保持计算节点不变，让数据进行流动。这样的设计是针对连接式的机器学习算法，比如逻辑斯底回归，神经网络等。连接式的机器学习算法可以把算法表达成一张图，张量从图中从前到后走一遍就完成了前向运算；而残差从后往前走一遍，就完成了后向传播。

计算节点（Operation）：将多个Tensor连接在一起，形成新的Tensor。例如tf.add、tf.mul等操作。tensor在graph上也是一个Operation（简称op）。但凡是op，都需要通过session运行之后，才能得到结果。Tensorflow中的运算（op）有很多很多种，最简单的当然就是加减乘除，它们的输入和输出都是tensor。

Runner：在建立图之后，必须使用会话中的Runner来运行图，才能得到结果。在运行图时，需要为所

有的变量和占位符赋值，否则就会报错。

3.3.4、TensorflowSharp 中的几类主要变量

- Const：常量，这很好理解。它们在定义时就必须被赋值，而且值永远无法被改变。

- Placeholder：占位符。这是一个在定义时不需要赋值，但在使用之前必须赋值（feed）的变量，通常用作训练数据。

- Variable：变量，它和占位符的不同是它在定义时需要赋值，而且它的数值是可以在图的计算过程中随时改变的。因此，占位符通常用作图的输入（即训练数据），而变量用作图中可以被“训练”或“学习”的那些tensor，例如y=ax+b中的a和b。

3.7、PyTorch AI框架介绍

3.7.1、什么是 PyTorch

PyTorch是一个基于Torch的Python开源机器学习库，用于自然语言处理等应用程序。是由Facebook开源的神经网络框架，专门针对 GPU 加速的深度神经网络（DNN）编程。Torch 是一个经典的对多维矩阵数据进行操作的张量（tensor ）库，在机器学习和其他数学密集型应用有广泛应用。不仅能够实现强大的GPU加速，同时还支持动态神经网络，这一点是现在很多主流框架如TensorFlow都不支持的。与Tensorflow的静态计算图不同，PyTorch的计算图是动态的，可以根据计算需要实时改变计算图。但由于Torch语言采用 Lua，导致在国内一直很小众，并逐渐被支持 Python 的 Tensorflow 抢走用户。作为经典机器学习库 Torch 的端口，PyTorch 为 Python 语言使用者提供了舒适的写代码选择。

PyTorch提供了两个高级功能：

- 具有强大的GPU加速的张量计算（如Numpy)
- 包含自动求导系统的深度神经网络

除了Facebook之外，Twitter、GMU和Salesforce等机构都采用了PyTorch.

TensorFlow和Caffe都是命令式的编程语言，而且是静态的，首先必须构建一个神经网络，然后一次又一次使用相同的结构，如果想要改变网络的结构，就必须从头开始。但是对于PyTorch,通过反向求导技术，可以让你零延迟地任意改变神经网络的行为，而且其实现速度快。正是这一灵活性是PyTorch对比TensorFlow的最大优势。

另外，PyTorch的代码对比TensorFlow而言，更加简洁直观，底层代码也更容易看懂，这对于使用它的人来说理解底层肯定是一件令人激动的事。

3.7.2、PyTorch 的优点

简洁：

PyTorch的设计追求最少的封装，尽量避免重复造轮子。不像 TensorFlow 中充斥着session、graph、operation 、 name_scope 、 variable 、 tensor 、 layer 等全新的概念， PyTorch 的设计遵循tensor→variable(autograd)→nn.Module 三个由低到高的抽象层次，分别代表高维数组（张量）、自动求导（变量）和神经网络（层/模块），而且这三个抽象之间联系紧密，可以同时进行修改和操作。 简洁的设计带来的另外一个好处就是代码易于理解。PyTorch的源码只有TensorFlow的十分之一左右，更少的抽象、更直观的设计使得PyTorch的源码十分易于阅读。

2.速度：

PyTorch 的灵活性不以速度为代价，在许多评测中，PyTorch 的速度表现胜过 TensorFlow和Keras 等框架。框架的运行速度和程序员的编码水平有极大关系，但同样的算法，使用PyTorch实现的那个更有可能快过用其他框架实现的。

3.易用：

PyTorch 是所有的框架中面向对象设计的最优雅的一个。PyTorch的面向对象的接口设计来源于Torch，而Torch的接口设计以灵活易用而著称，Keras作者最初就是受Torch的启发才开发了Keras。PyTorch继承了Torch的衣钵，尤其是API的设计和模块的接口都与Torch高度一致。PyTorch的设计最符合人们的思维，它让用户尽可能地专注于实现自己的想法，即所思即所得，不需要考虑太多关于框架本身的束缚。

4.活跃的社区：

PyTorch 提供了完整的文档，循序渐进的指南，作者亲自维护的论坛 供用户交流和求教问题。Facebook 人工智能研究院对 PyTorch 提供了强力支持，作为当今排名前三的深度学习研究机构，FAIR的支持足以确保PyTorch获得持续的开发更新，不至于像许多由个人开发的框架那样昙花一现。

3.4.3、PyTorch 的架构

PyTorch 通过混合前端，分布式训练以及工具和库生态系统实现快速，灵活的实验和高效生产。PyTorch 和 TensorFlow 具有不同计算图实现形式，TensorFlow 采用静态图机制(预定义后再使用)，PyTorch采用动态图机制(运行时动态定义)。PyTorch 具有以下高级特征:

- 混合前端:新的混合前端在急切模式下提供易用性和灵活性，同时无缝转换到图形模式，以便在C ++

运行时环境中实现速度，优化和功能。

- 分布式训练:通过利用本地支持集合操作的异步执行和可从Python和C ++访问的对等通信，优化了性能。

- Python优先: PyTorch为了深入集成到Python中而构建的，因此它可以与流行的库和Cython和Numba等软件包一起使用。

- 丰富的工具和库:活跃的研究人员和开发人员社区建立了丰富的工具和库生态系统，用于扩展PyTorch并支持从计算机视觉到强化学习等领域的开发。

- 本机ONNX支持:以标准ONNX（开放式神经网络交换）格式导出模型，以便直接访问与ONNX兼容的平台，运行时，可视化工具等。

- C++前端：C++前端是PyTorch的纯C++接口，它遵循已建立的Python前端的设计和体系结构。它旨在实现高性能，低延迟和裸机C++应用程序的研究。 使用GPU和CPU优化的深度学习张量库。

PyTorch的常用工具包

- torch ： 类似 NumPy 的张量库，强 GPU 支持 ；

- torch.autograd ： 基于 tape 的自动区别库，支持 torch 之中的所有可区分张量运行;

- torch.nn ： 为最大化灵活性未涉及、与 autograd 深度整合的神经网络库;

- torch.optim：与 torch.nn 一起使用的优化包，包含 SGD、RMSProp、LBFGS、Adam等标准优化方式;

- torch.multiprocessing： python 多进程并发，进程之间 torch Tensors 的内存共享;

- torch.utils：数据载入器。具有训练器和其他便利功能;

- torch.legacy(.nn/.optim) ： 处于向后兼容性考虑，从 Torch 移植来的 legacy 代码;

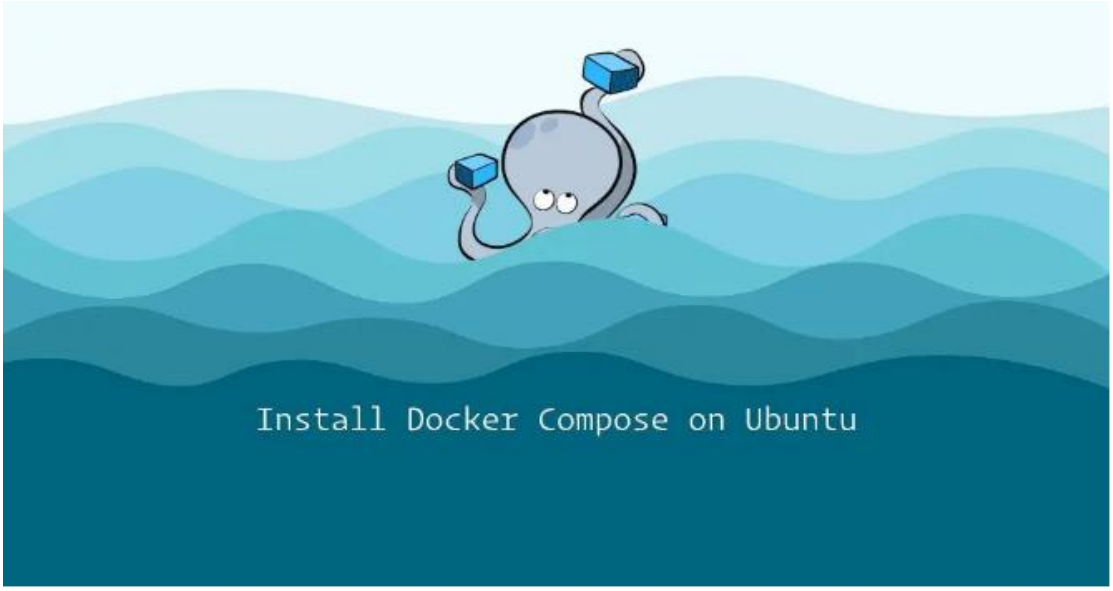
3.8、Jetson Nano使用Docker容器技术

3.8.1、Docker 容器技术简介

Docker 是一个开源的应用容器引擎，让开发者可以打包他们的应用以及依赖包到一个可移植的容器中，然后发布到任何流行的 Linux 或 Windows 操作系统的机器上，也可以实现虚拟化，容器是完全使用沙箱机制，相互之间不会有任何接口。

3.8.2、Jetson Nano Docker 测试

NVIDIA JetPack 包含 NVIDIA Container Runtime，该版本与 Jetson 平台的 Docker 集成。这是必不可少的，因为它使用户能够在 Jetson 设备上运行 GPU 加速深度学习和 HPC 容器。这听起来不错，但要构建、部署和扩展微服务，您需要 Docker Compose。 Docker Compose 是用于定义和运行多容器 Docker 应用程序的工具。使用 Compose，您可以使用 YAML 文件来配置应用程序的服务。然后，使用单个命令，从配置中创建并启动所有服务。



Docker 引擎是在设置期间安装的。通过运行以下命令检查它是否已安装在 Jetson 中。如果看到以下内容，则可以跳过安装部分并转到运行 docker 容器。

```
sudo docker version
```

```
jetson@nano:~$ sudo docker version
Client:
 Version:           24.0.5
 API version:       1.43
 Go version:        go1.20.3
 Git commit:        24.0.5-0ubuntu1~20.04.1
 Built:             Mon Aug 21 19:50:14 2023
 OS/Arch:           linux/arm64
 Context:           default

Server:
 Engine:
  Version:          24.0.5
  API version:      1.43 (minimum version 1.12)
  Go version:       go1.20.3
  Git commit:       24.0.5-0ubuntu1~20.04.1
  Built:            Mon Aug 21 19:50:14 2023
  OS/Arch:          linux/arm64
  Experimental:     false
 containerd:
  Version:          1.7.2
  GitCommit:
 runc:
  Version:          1.1.7-0ubuntu1~20.04.2
  GitCommit:
 docker-init:
  Version:          0.19.0
  GitCommit:
```

3.8.3、Jetson Nano Docker 环境搭建及使用

可以从以下 Github 存储库中提供的脚本安装 Docker

https://github. com/docker/docker-install

运行以下命令下载脚本。

```
curl -fsSL https://get. docker. com -o get-docker. sh
```

生成并运行脚本

```
Sudo sh get-docker.sh
```

如何在容器中运行 Docker 映像：

若要测试 Docker 是否已正确安装，可以使用以下命令在容器中运行名为 Hello-World 的简单映像。如果之前未下载映像，则“pull”命令也将在后面工作以下载映像。

```
sudo docker container run hello-world
```

```
jetson@nano:~$ sudo docker container run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
478afc919002: Pull complete
Digest: sha256:d000bc569937abbe195e20322a0bde6b2922d805332fd6d8a68b19f524b7d21d
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (arm64v8)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

您可以使用手动拉取操作从 Docker 中心下载映像。

```
docker pull [OPTIONS] NAME[:TAG|@DIGEST]
```

若要在容器内运行映像，可以使用以下命令。

```
sudo docker container run [OPTIONS] IMAGE [COMMAND] [ARG…]
```

现在，让我们尝试从 Ubuntu 映像运行容器。首先，我们需要使用“pull”命令下载 docker 镜像。

```
sudo docker pull ubuntu
```

然后，我们可以在具有不同操作的容器中运行 ubuntu 映像。我们将展示一些简单的命令，但您可以使用 --help 检查所有这些命令。

您还可以通过在末尾添加路径来指定将创建 docker 的目录。

运行 docker 容器的示例命令：

```
sudo docker container run ubuntu ls

sudo docker container run ubuntu echo nvidia
```

```
jetson@nano:~$ sudo docker container run ubuntu ls
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
a4a2c7a57ed8: Pull complete
Digest: sha256:f9d633ff6640178c2d0525017174a688e2c1aef28f0a0130b26bd5554491f0da
Status: Downloaded newer image for ubuntu:latest
bin
boot
dev
etc
home
lib
media
mnt
opt
proc
root
run
sbin
srv
sys
tmp
usr
var
```

若要以交互方式运行 Docker 容器，请使用-it 操作。

--rm: 完成后自动删除容器。

--network: 将容器连接到网络，在本例中，它连接到主机。

您可以在容器内执行各种任务，并通过键入“exit”离开

```
sudo docker container run -it --rm --network host ubuntu

echo "hello world" >forecr.txt

cat forecr.txt

ls

exit
```

```
jetson@nano:~$ sudo docker container run -it --rm --network host ubuntu
root@nano:/# echo "hello world" >forecr.txt
root@nano:/# cat forecr.txt
hello world
root@nano:/# ls
bin  boot  dev  etc  forecr.txt  home  lib  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@nano:/# exit
exit
jetson@nano:~$
```

-- volume:访问容器外部文件

```
sudo docker container run -it --rm --network host --volume /home/nvidia ubuntu
```

```
jetson@nano:~$ sudo docker container run -it --rm --network host --volume /home/nvidia ubuntu
root@nano:/# cd home/nvidia/
root@nano:/home/nvidia# ls
root@nano:/home/nvidia#
```

4、传感器项目实践

传感器实验操作说明：

本章节的传感器实验由 Jetson Nano 的 GPIO 直接控制，进行实验前需断开‘机械臂扩展板’的电源，并把‘Jetson Nano 主机’上的 40 针排线连接到‘AI 主板核心卡‘上。

4.1、双色LED实验

4.1.1、硬件介绍：

双色 LED 灯准确来说叫双基色 LED 灯，是指模块只能显示 2 种颜色，一般是红色和绿色，可以有三种状态：灭，颜色 1 亮，颜色 2 亮，根据颜色组合的不同，分为红蓝双色，黄蓝双色，红绿双色等等。

双色 LED 是一种常用的 LED.常于指示状态，比如：红色表示忙碌，绿色表示 Ready。

图 3.1.1 双色 LED

4.1.2 实验原理：

将引脚 S(绿色）和中间管脚（红色）连接到 Jetson Nano 的 GPIO 接口上，对 JetsonNano 进行编程控制，将 LED 的颜色从红色变为黄色，然后使用 GPIO 数字信号控制。

该模块的原理图如下所示：

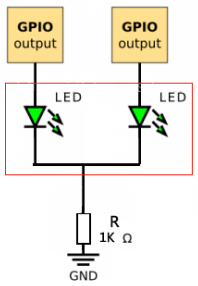


图 4.1.2 双色 LED 原理图

4.1.3 实验步骤:

第 1 步：搭建实验电路

GPIO Expansion Board	双色 LED 模块
GPIO6	G(S)
GPIO5	R
GND	GND

第 2 步：按照前面所学 JupyterLab 的内容，在电脑端通过网页登陆 Jetson Nano 的 IP 地址加上 8888(IP:8888)或者在本地登录 localhost:8888,接下来演示中我们会在电脑端通过网页登陆，如下图所示:

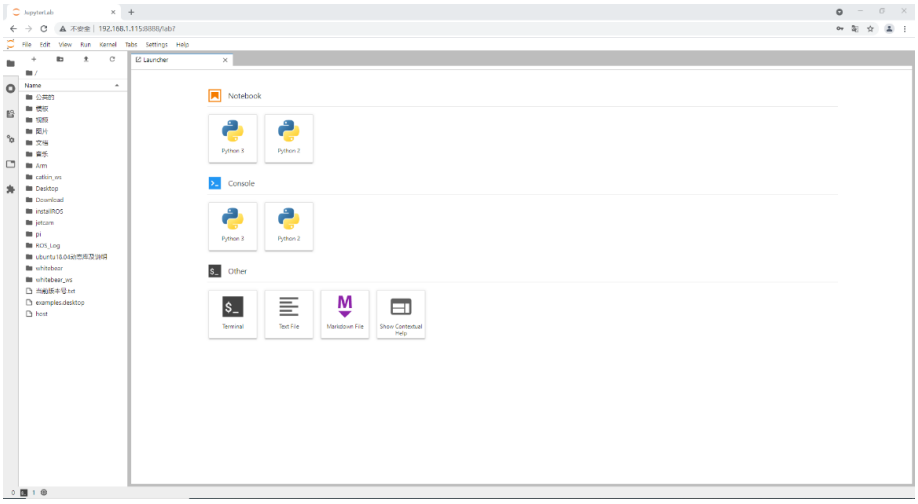


图 4.1.3 JupyterLab 界面

进入‘Notebook’文件夹里的‘sensor 传感器模块’文件夹，找到‘1.双色 LED 实验.ipynb’程序，双击用 JupyterLab 打开程序:

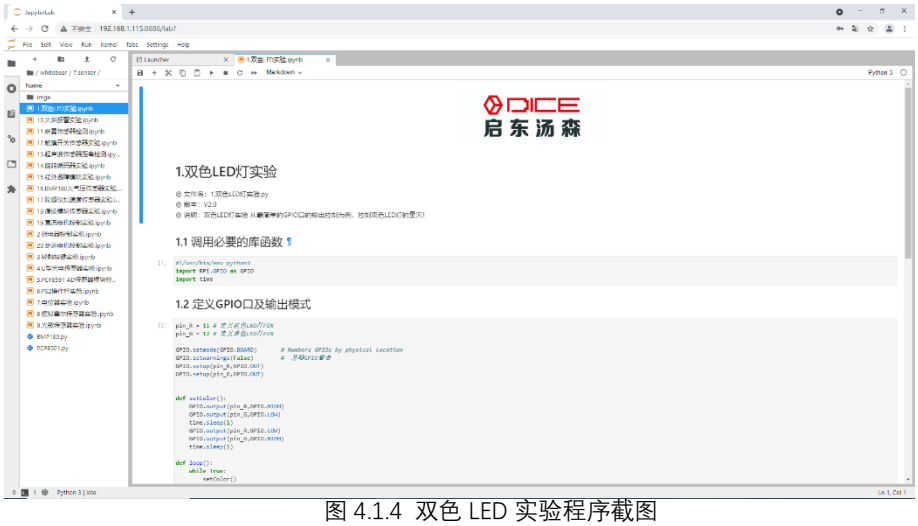


图 4.1.4 双色 LED 实验程序截图

第 3 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 4 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行!

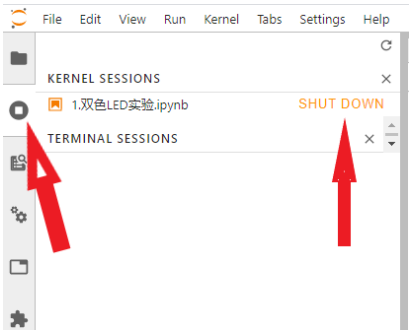


图 4.1.5 清除内核运行

实验结果：您可以看到双色 LED 呈现绿色、红色交替闪烁。

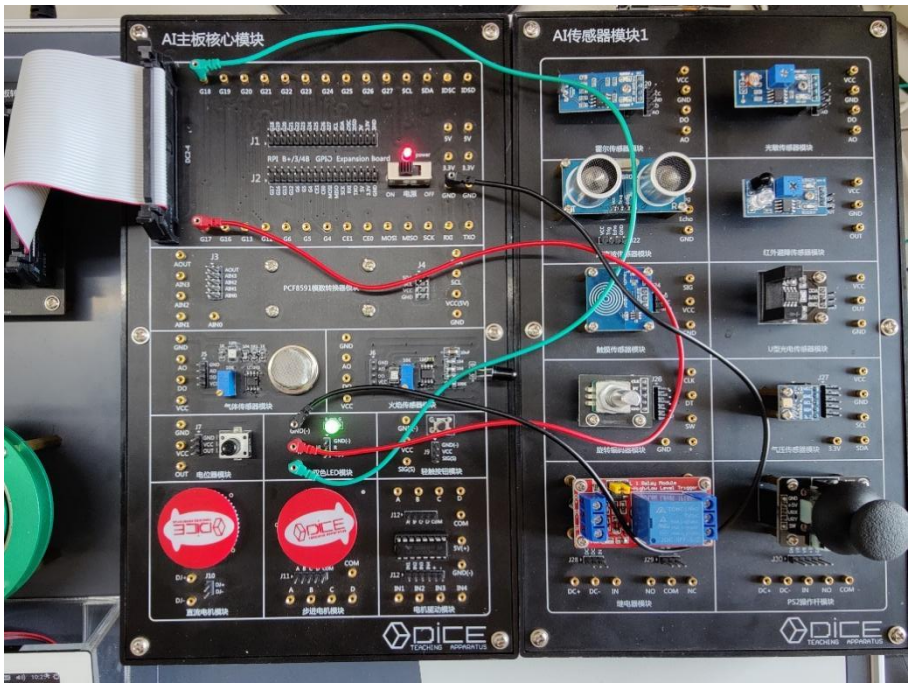


图 4.1.6 实验效果图

4.1.4、python 代码:

1.双色 LED 灯实验

@ 文件名：1.双色 LED 灯实验.py

@ 版本：V2.0

@ 说明：双色 LED 灯实验 从最简单的 GPIO 口的输出控制为例，控制双色 LED 灯的亮灭！

1.1 调用必要的函数库

```
#!/usr/bin/env python3
import RPi.GPIO as GPIO
import time
```

1.2 定义 GPIO 口及输出模式

```
pin_R = 11 #  定义红色 LED 灯 PIN
pin_G = 12 #  定义黄色 LED 灯 PIN

GPIO.setmode(GPIO.BOARD)          # Numbers GPIOs by physical location
GPIO.setwarnings(False)           #    忽略 GPIO 警告
GPIO.setup(pin_R,GPIO.OUT)
GPIO.setup(pin_G,GPIO.OUT)


def setColor():
    GPIO.output(pin_R,GPIO.HIGH)
    GPIO.output(pin_G,GPIO.LOW)
    time.sleep(1)
```

```
    GPIO.output(pin_R,GPIO.LOW)
    GPIO.output(pin_G,GPIO.HIGH)
    time.sleep(1)

def loop():
    while True:
        setColor()

def destroy():
    GPIO.output(pin_R,GPIO.LOW)
    GPIO.output(pin_G,GPIO.LOW)
    GPIO.cleanup()

# 程序入口
if __name__ == "__main__":
    try:
        loop()
    except KeyboardInterrupt:
        destroy()
```

4.2、轻触开关按键实验

4.2.1、硬件介绍

轻触按键模块是最为常见的开关模块，内部有一个轻触拨盘，当按下输出低电平时，松开以保持高电平。

图 4.2.1 按键开关

4.2.2、实验原理

使用按键开关作为 Jetson Nano 的输入设备，当按下开关时，通过树莓派的 Jetson Nano 口设置为输入模式检测到低电平（0V),表示按键被按键，当松开按键时，Jetson Nano 检测到高电平（5V),在本项目实验中我们将通过命令行提示 LED 是否被按键，该模块的原理图如下所示：

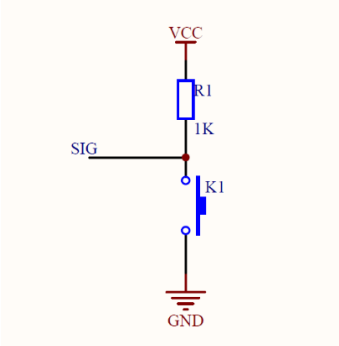


图 4.2.2 轻触按键模块原理图

4.2.3、实验步骤

第 1 步：搭建实验电路

GPIO Expansion Board	轻触按键模块	双色 LED 模块
GPIO5	SIG(S)	
GPIO18		R
GPIO27		G(S)
5V	VCC	
GND	GND	GND

第 2 步：进入‘Notebook’文件夹里的‘sensor 传感器模块’文件夹，找到‘2.轻触按键实验.ipynb’程序，双击用 JupyterLab 打开程序:

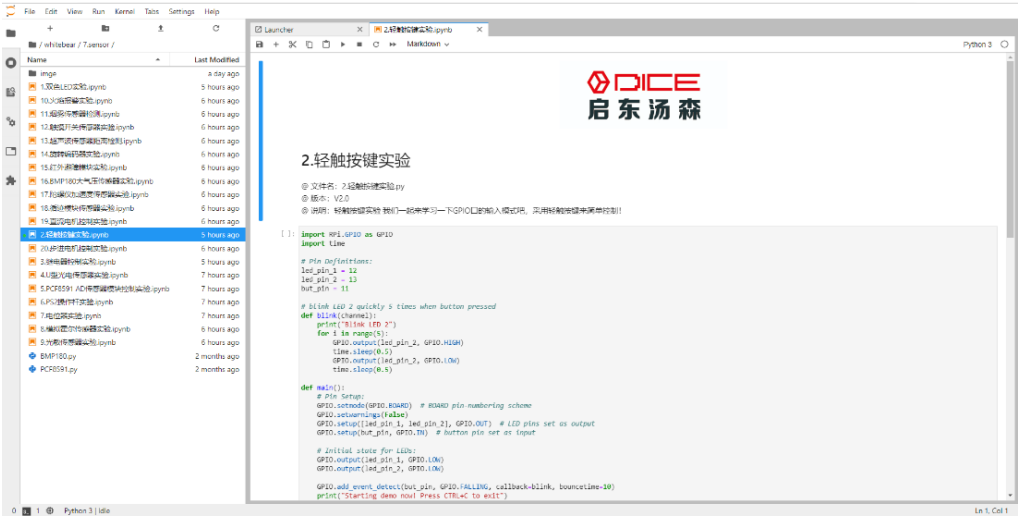


图 4.2.3 轻触按键实验程序截图

第 3 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 4 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：双色 LED 模块上的 LED 红光闪烁。如果按下按钮，屏幕上将显示“Blink LED 2",LED 将发出绿光闪烁。如果松开按钮，重新 LED 将再次闪烁红光。

图 4.2.4 实验效果图 1

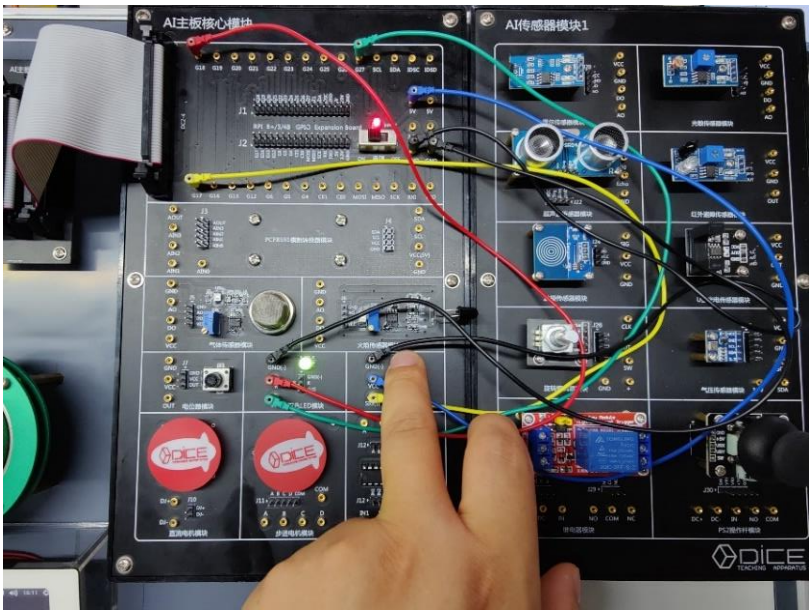


图 4.2.5 实验效果图 2

4.2.4、python 代码

2.轻触按键实验

```
@ 文件名：2.轻触按键实验.py
@ 版本：V2.0
@ 说明：轻触按键实验 我们一起来学习一下 GPIO 口的输入模式吧，采用轻触按键来简单控制！

import RPi.GPIO as GPIO
import time

# Pin Definitions:
led_pin_1 = 12
led_pin_2 = 13
but_pin = 11

# blink LED 2 quickly 5 times when button pressed
def blink(channel):
    print("Blink LED 2")
    for i in range(5):
        GPIO.output(led_pin_2, GPIO.HIGH)
        time.sleep(0.5)
        GPIO.output(led_pin_2, GPIO.LOW)
        time.sleep(0.5)

def main():
```

```
    # Pin Setup:
    GPIO.setmode(GPIO.BOARD)    # BOARD pin-numbering scheme
    GPIO.setwarnings(False)
    GPIO.setup([led_pin_1, led_pin_2], GPIO.OUT)    # LED pins set as output
    GPIO.setup(but_pin, GPIO.IN)    # button pin set as input

    # Initial state for LEDs:
    GPIO.output(led_pin_1, GPIO.LOW)
    GPIO.output(led_pin_2, GPIO.LOW)

    GPIO.add_event_detect(but_pin, GPIO.FALLING, callback=blink, bouncetime=10)
    print("Starting demo now! Press CTRL+C to exit")
    try:
        while True:
            # blink LED 1 slowly
            GPIO.output(led_pin_1, GPIO.HIGH)
            time.sleep(2)
            GPIO.output(led_pin_1, GPIO.LOW)
            time.sleep(2)
    finally:
        GPIO.cleanup()    # cleanup all GPIOs

if __name__ == '__main__':
    main()
```

4.3、继电器实验

4.3.1、硬件介绍

继电器模块是一种电控制器件，是当输入量（激励量）的变化达到规定要求时，在电气输出电路中使被控量发生预定的阶跃变化的一种电器。它具有控制系统（又称输入回路）和被控制系统（又称输出回路）之间的互动关系。通常应用于自动化的控制电路中，它实际上是用小电流去控制大电流运作的一种“自动开关”。故在电路中起着自动调节、安全保护、转换电路等作用。

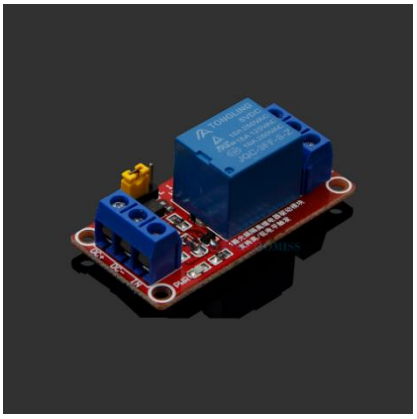


图 4.3.1 继电器模块

4.3.2、实验原理

继电器的工作原理如图 3.3.2 所示：在继电器内部有一个电磁线圈，线圈上端有一个衔铁。在不通电时，在左侧弹簧拉力的作用下，动触点和静触点 B 连通，动触点和静触点 A 不通。当给电磁线圈两端 13、14 通入额定电压以后，在电磁线圈磁力的作用下，衔铁带动连杆向下运动，动触点和静触点 B 断开，与静触点 A 连通。静触点 B 和动触点不通电时连通、通电时断开，把这样一对触点叫常闭；同理，把静触点 A 和动触点不通电时不连通、通电时连通，把这样一对触点叫常开。

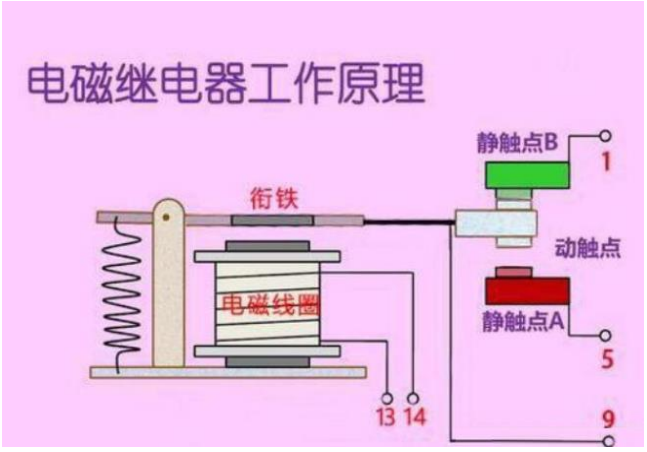


图 4.3.2 继电器工作原理

简而言之，继电器的原理就是：利用控制电磁线圈通电与断电，来达到控制常开与常闭的通断。从而达到通过小电流控制大电流，小电压控制大电压。因为控制电磁线圈只需要很小的电流或者电压即可，而常开和常闭却允许通过比较高的电流及电压。

有些继电器还会在电磁线圈两端并联一个发光二极管和一个电阻，用来显示线圈工作状态。

继电器接线：

把控制开关和继电器线圈串联，然后接到控制电源上，注意控制电源要和继电器线圈电压相符。再把需要控制的用电设备和继电器的常开或者常闭串联，再接上用电设备的电源。如图 4.3.3:

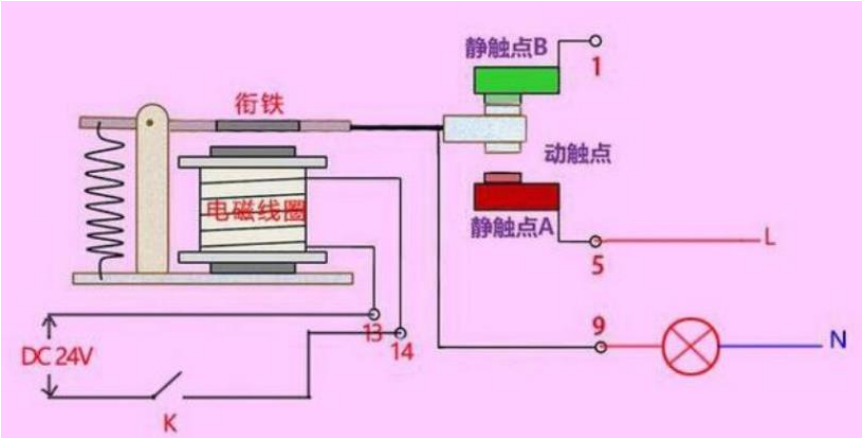


图 4.3.3 继电器接线图

工作流程：

开关 K 闭合，24V 电压加在继电器线圈两端，衔铁被电磁线圈磁力作用而向下运动，从而带着动触点向下运动，动触点和静触点 A 连通。火线电流通过静触点 A--动触点--灯---N 线，形成了一个回路，此时灯开始工作。

所以在这个实验中，将 SIG 连接到 Jetson Nano 板。发送一个低电平给 SIG;PNP 晶体管通电并且继电器的线圈通电。因此，继电器的常开触点闭合，而继电器的常闭触点将脱离公共端口。向 SIG 发送高电平的信息；晶体管将断电，继电器将恢复到初始状态。

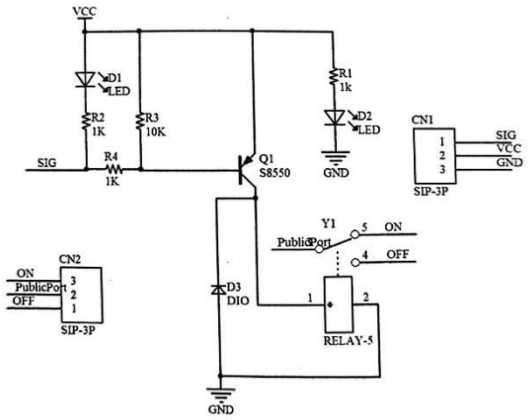


图 4.3.4 继电器模块原理图

4.3.3、实验步骤

第 1 步：搭建实验电路

GPIO Expansion Board	继电器模块	双色 LED 模块
GPIO5	IN	
5V	DC+&COM	
GND	DC-	GND
	NC	R
	NO	G(S)

第 2 步：进入‘Notebook’文件夹里的‘sensor 传感器模块’文件夹，找到‘3.继电器控制实验.ipynb’程序，双击用 JupyterLab 打开程序:

图 4.3.5 继电器控制实验程序截图

第 3 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 4 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：现在，您可能会听到嘀嗒声。这是常闭触点打开，常开触点闭合。您可以将要控制的较高电压设备（如 220V 灯泡）连接到继电器的输出端口。然后继电器将充当自动开关。

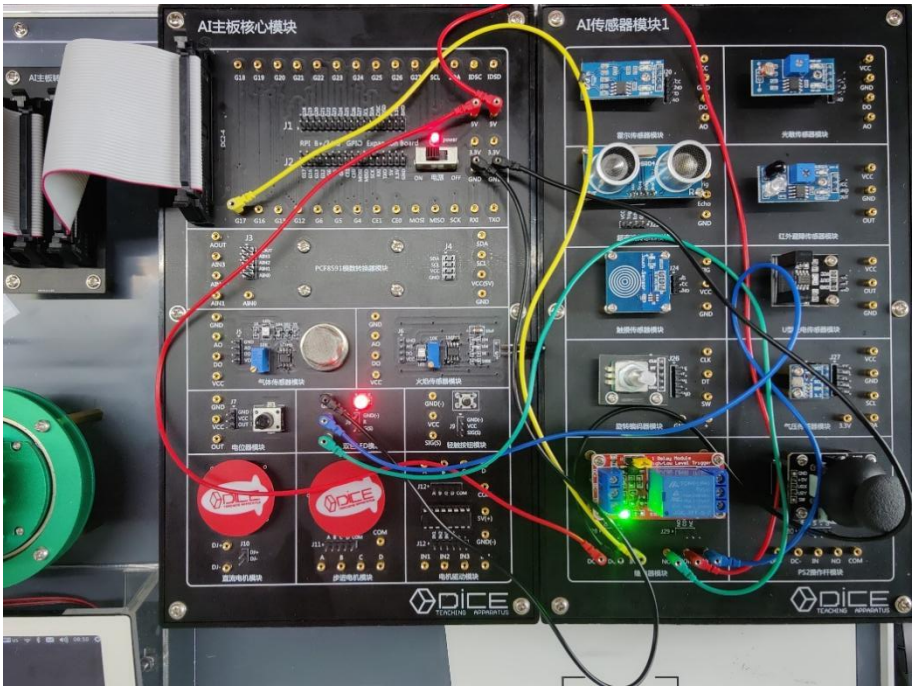


图 4.3.6 继电器控制实验效果图

4.3.4、python 代码

3.继电器控制实验

@ 文件名：3.继电器控制实验.py

@ 版本：V2.0

@ 说明：继电器控制实验 从前面的 LED 灯的控制，我们来学习比较常用的继电器控制实验，可以通过小电流控制大电流设备！

3.1 导入必要的库文件

```
import RPi.GPIO as GPIO
import time
```

3.2 定义继电器的 GPIO 口

```
makerobo_RelayPin = 11      # 定义继电器管脚为 Pin11
```

3.3 初始化工作

```
# 初始化工作
def makerobo_setup():
    GPIO.setmode(GPIO.BOARD)          # 采用实际的物理管脚给 GPIO 口
    GPIO.setwarnings(False)           #  忽略 GPIO 警告
    GPIO.setup(makerobo_RelayPin, GPIO.OUT)   # 设置 Pin 模式为输出模式
     GPIO.output(makerobo_RelayPin, GPIO.LOW) # 关闭继电器
```

3.4 循环执行

```
# 循环函数
def makerobo_loop():
    while True:
        # 如果是低电平有效，则为关闭继电器
        # 继电器打开
        GPIO.output(makerobo_RelayPin, GPIO.HIGH)
        time.sleep(0.5)                    # 延时 500ms
        # 如果是低电平有效，则为使能继电器
        # 继电器关闭
        GPIO.output(makerobo_RelayPin, GPIO.LOW)
        time.sleep(0.5)
```

3.5 程序入口执行

```
# 释放资源
def makerobo_destroy():
    GPIO.output(makerobo_RelayPin, GPIO.LOW) # 关闭继电器
    GPIO.cleanup()                     # 释放资源


# 程序入口
if __name__ == '__main__':
    makerobo_setup()          #  初始化
    try:
        makerobo_loop()       #  调用循环函数
    except KeyboardInterrupt:  #  当按下 Ctrl+C 时，将执行 destroy()子程序。
        makerobo_destroy()    #  释放资源
```

4.4、U 型光电传感器实验

4.4.1、硬件介绍

U 型光电开关其实对射式光电开关的一种又被叫做槽型光电开关，是一款红外线感应光电产品，由红外线发射管和红外线接收管组合而成，而槽型光电开关槽宽就决定了感应接收型号的强弱与接收信号的距离，以光为媒体，由发光体与受光体间的红外光进行接收与转换，检测物体的位置。U 型光电开关于接近开关同样是无接触式的，受检测体的制约少，且检测距离长，可进行长距离的检测（几十米）检测精度高能检测小物体。应用非常的广泛。

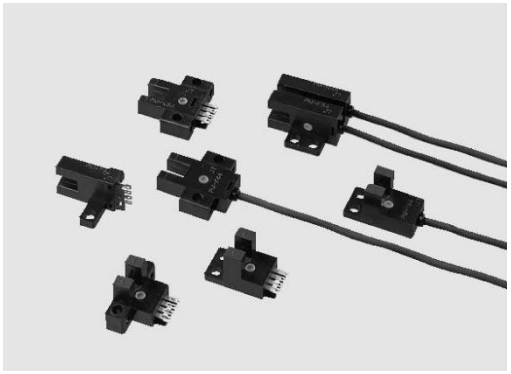


图 4.4.1 U 型光电传感器

4.4.2、实验原理

发射器和接收器于一体的光电传感器，其发射器和接收器分别位于 U 型槽的两边，并形成一光轴，当被检测物体经过 U 型槽且阻断光轴时，光电开关就产生了检测到的开关信号。槽式光电开关比较安全可靠，适合检测高速变化，分辨透明与半透明物体，并且可以调节灵敏度。当有被检测物体经过时，将光 U 型电开关红外线发射器发射的足够量的光线反射到红外线接收器接收器，于是光电开关就产生了开关信号。而槽宽则就决定了感应接收型号的强弱与接收信号的距离，以光为媒体，由发光体与受光体间的红外光进行接收与转换。在这个实验中，我们将通过使用此更改来打开或关闭 LED 的亮灭。

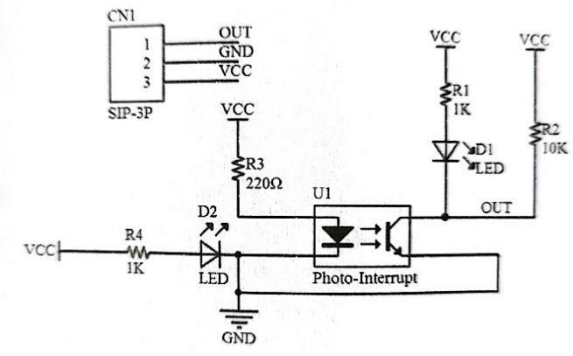


图 4.4.2 U 型光电传感器模块原理图

4.4.3、实验步骤

第 1 步：搭建实验电路

GPIO Expansion Board	U 型光电传感器模块	双色 LED 模块
GPIO23	OUT	
5V	VCC	
GND	GND	GND
GPIO18		R
GPIO27		G(S)

第 2 步：进入‘Notebook’文件夹里的‘sensor 传感器模块’文件夹，找到‘4.U 型光电传感器实验.ipynb’程序，双击用 JupyterLab 打开程序:

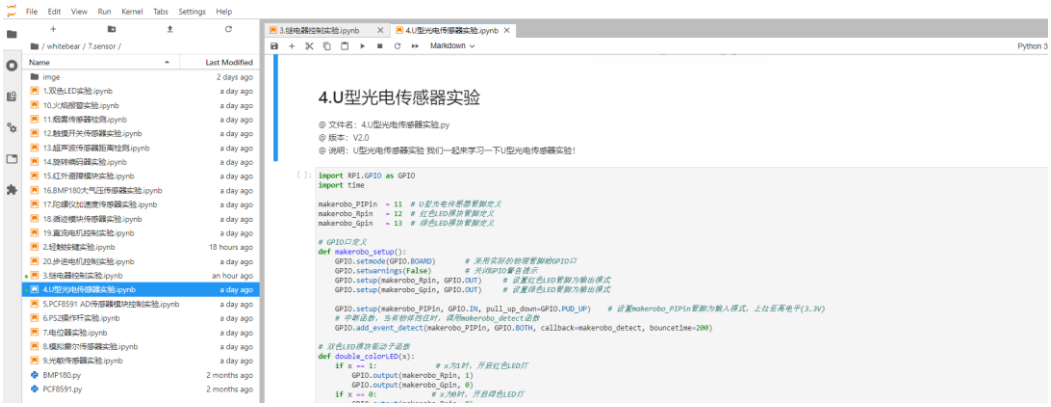


图 3.4.3 U 型光电传感器实验程序截图

第 3 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 4 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：现在 LED 将点亮绿色。如果您在光电断路器的间隙中粘贴一张纸，则“光被阻挡”将在屏幕上打印，LED 将发出红光。如果取出纸张，LED 将再次发出绿光。

图 4.4.4 U 型光电传感器实验效果图 1

```
# 程序入口
if __name__ == '__main__':
    makerobo_setup() # GPIO 初始化
    try:
        loop()
    except KeyboardInterrupt:  # 当按下Ctrl+C时，将执行destroy()子程序。
        destroy()
```

```
**************************************
***** makerobo Light was blocked!   *
**************************************
**************************************
***** makerobo Light was blocked!   *
**************************************
**************************************
***** makerobo Light was blocked!   *
**************************************
**************************************
***** makerobo Light was blocked!   *
**************************************
**************************************
***** makerobo Light was blocked!   *
**************************************
**************************************
***** makerobo Light was blocked!   *
**************************************
```

图 4.4.5 U 型光电传感器实验效果图 2

4.4.4、python 代码

4.U 型光电传感器实验

@ 文件名：4.U 型光电传感器实验.py

@ 版本：V2.0

@ 说明：U 型光电传感器实验 我们一起来学习一下 U 型光电传感器实验！

```
import RPi.GPIO as GPIO
import time

makerobo_PIPin  = 11    # U 型光电传感器管脚定义
makerobo_Rpin   = 12    # 红色 LED 模块管脚定义
makerobo_Gpin   = 13    # 绿色 LED 模块管脚定义

# GPIO 口定义
def makerobo_setup():
    GPIO.setmode(GPIO.BOARD)        # 采用实际的物理管脚给 GPIO 口
```

```
    GPIO.setwarnings(False)            # 关闭 GPIO 警告提示
    GPIO.setup(makerobo_Rpin, GPIO.OUT)        # 设置红色 LED 管脚为输出模式
    GPIO.setup(makerobo_Gpin, GPIO.OUT)        # 设置绿色 LED 管脚为输出模式

    GPIO.setup(makerobo_PIPin, GPIO.IN, pull_up_down=GPIO.PUD_UP)      # 设置 makerobo_PIPin 管脚为输入模式，上拉至高电平(3.3V)
    # 中断函数，当有物体挡住时，调用 makerobo_detect 函数
    GPIO.add_event_detect(makerobo_PIPin, GPIO.BOTH, callback=makerobo_detect, bouncetime=200)

# 双色 LED 模块驱动子函数
def double_colorLED(x):
    if x == 1:                    # x 为 1 时，开启红色 LED 灯
        GPIO.output(makerobo_Rpin, 1)
        GPIO.output(makerobo_Gpin, 0)
    if x == 0:                    # x 为 0 时，开启绿色 LED 灯
        GPIO.output(makerobo_Rpin, 0)
        GPIO.output(makerobo_Gpin, 1)
    time.sleep(0.5)

# 打印函数，显示出是否有黑色物体挡住
def makerobo_Print(x):
    if x == 1:
        print('**************************************')
        print('***** makerobo Light was blocked!     *')
        print('**************************************')
    time.sleep(0.5)

# 中断函数，检测到有物体挡住时，响应中断函数
def makerobo_detect(chn):
    double_colorLED(GPIO.input(makerobo_PIPin))   # 调用双色 LED 驱动函数
    makerobo_Print(GPIO.input(makerobo_PIPin))    # 打印出检测到有物体挡住！

# 循环函数
def loop():
    while True:
        pass

def destroy():
    GPIO.output(makerobo_Rpin, GPIO.HIGH)          # 关闭红色 LED
```

```
    GPIO.output(makerobo_Gpin, GPIO.HIGH)       # 关闭绿色 LED
    GPIO.cleanup()                              # 释放资源


# 程序入口
if __name__ == '__main__':
    makerobo_setup() # GPIO 初始化
    try:
        loop()
    except KeyboardInterrupt:  # 当按下 Ctrl+C 时，将执行 destroy()子程序。
        destroy()
```

4.5、 PCF8591 模数转换器实验

4.5.1、硬件介绍

PCF8591 是一款单芯片，单电源，低功耗 8 位 CMOS 数据采集设备，具有四个模拟输入，一个模拟输出和一个串行 i2c 总线接口。三个地址引脚 A0,A1 和 A2 用于对硬件地址进行编程，从而允许使用多达 8 个连接到 i2c 总线的设备，而无需额外的硬件。通过两行双向 i2c 总线串行传输与设备之间的地址，控制和数据。该设备的功能包括模拟输入多路复用，片上跟踪和保持功能，8 位模数转换和 8 位数模转换。最大转换率由 i2c 总线的最大速度决定。

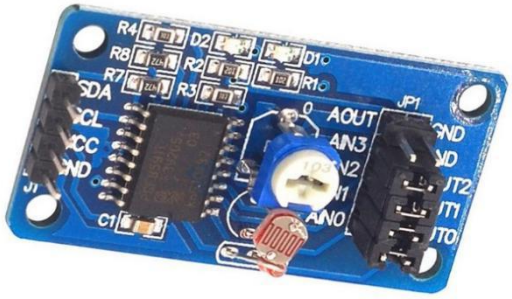


图 4.5.1 PCF8591 模数转换模块

4.5.2、实验原理

PCF8591 芯片特性：

1. 单独供电
2. PCF8591 的操作电压范围 2.5V-6V
3. 低待机电流
4. 通过 I2C 总线串行输入/输出
5. PCF8591 通过 3 个硬件地址引脚寻址
6. PCF8591 的采样率由 i2c 总线速率决定
7. 4 个模拟输入可编程为单端型或差分输入
8. 自动增量频道选择
9. PCF8591 的模拟电压范围从 VSS 到 VDD
10. PCF8591 内置跟踪保持电路
11. 8-bit 逐次逼近 A/D 转换器
12. 通过 1 路模拟输出实现 DAC 增益

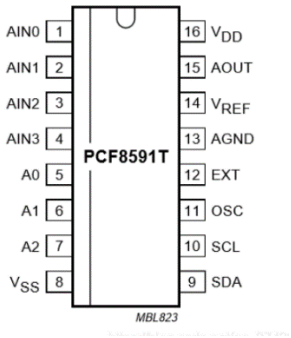


图 4.5.2 PCF8591 芯片引脚图

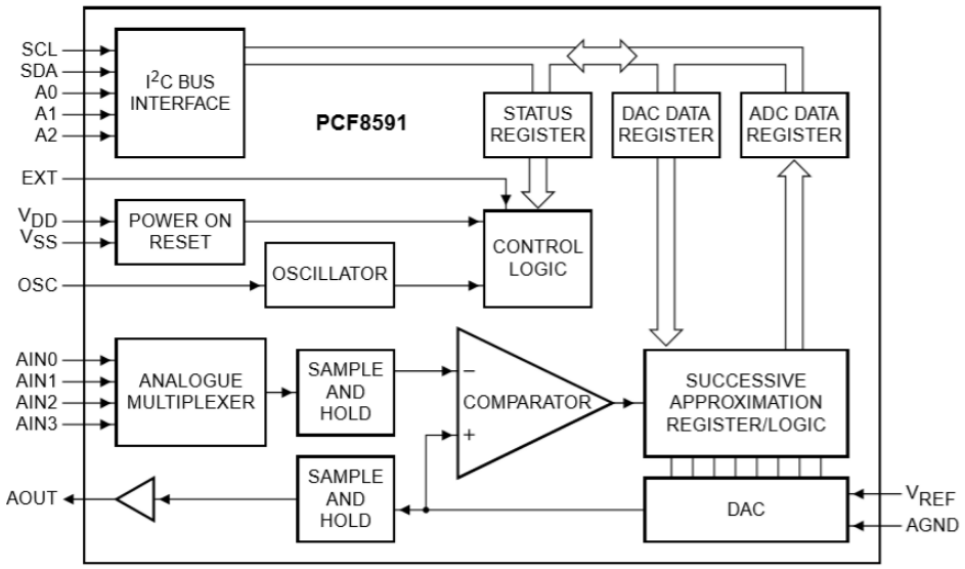


图 4.5.3 PCF8591 芯片原理图

寻址：

i2c 总线系统中的每个 PCF8591 设备通过向该设备发送有效地址而被激活。地址由固定部分和可编程部分组成。可编程部分必须按照地址引脚 A0,A1,A2 进行设置。地址必须以邮寄方式发送。

在 i2c 总线协议中，在启动条件之后的第一个字节。地址字节的最后一个位是读 / 写位，它设置了下列数据传输的方向。

图 4.5.4 PCF8591 器件地址

注：最后一位为 0 是写，最后一位为 1 是读

控制字格式：

发送到 PCF8591 器件的第二个字节将被存储在其控制寄存器中，并且需要控制器件功能。控制寄存器的高半字节用于使能模拟输出，并将模拟输入编程为单端或差分输入。下半字节选择由上半字节定义的一个模拟输入通道。如果设置了自动增加标志，则在每次 A/D 转换后，通道编号会自动递增。如果在使用内部振荡器的应用中需要自动增量模式，则应该设置控制字节(第 6 位)中的模拟输出使能标志。这允许内部振荡器连续运行，从而防止由振荡器启动延迟导致的转换错误。上电复位后，控制寄存器的所有位都复位为逻辑 0。为了省电，D/A 转换器和振荡器被禁用。模拟输出被切换到高阻抗状态。

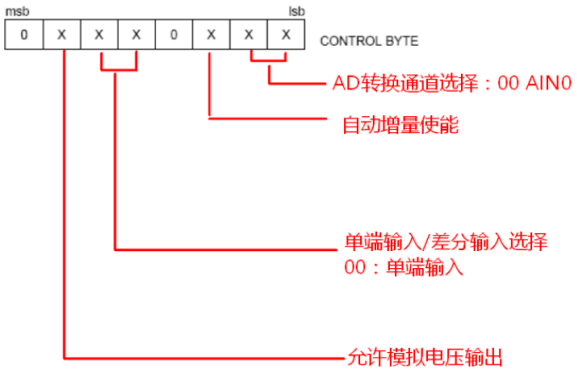


图 4.5.5 PCF8591 控制字格式

最高位默认为 0，第 6 位是选择是否允许模拟电压输出，在 D A 转换时设置为 1,AD 转换时设置为 0 或 1 均可，第 5/4 位是选择模拟电压输出方式，一般选择 00 单端输入方式，其他的几种方式如下图所示：

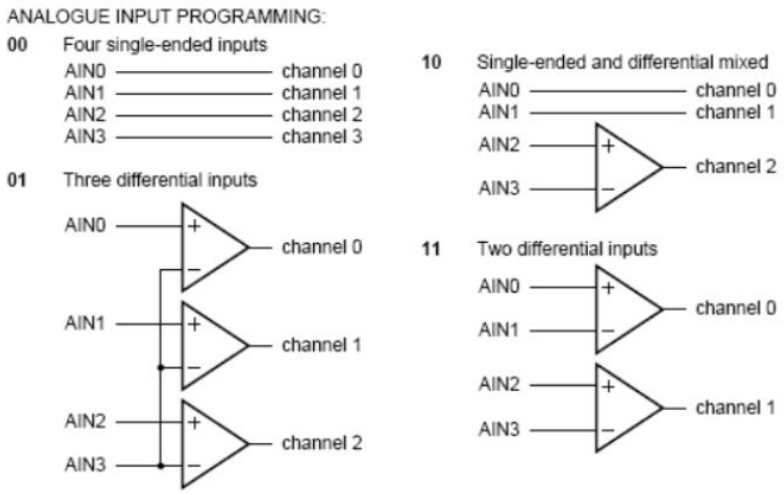


图 4.5.6 PCF8591 输出方式

第 3 位默认为 0，第 2 位是自动增量使能位，如果自动增量（auto-increment）标志置 1，每次 A/D 转换后通道号将自动增加。第 1 / 0 为是在 A D 转换时选择哪一个通道输入的电压转换为数字量。

在本实验中，AINO(模拟输入 0)端口用于接收来自电位计模块的模拟信号，AOUT（模拟输出）用于将模拟信号输出到双色 LED 模块，以便改变 LED 的亮度。该模块的原理图如下所示：

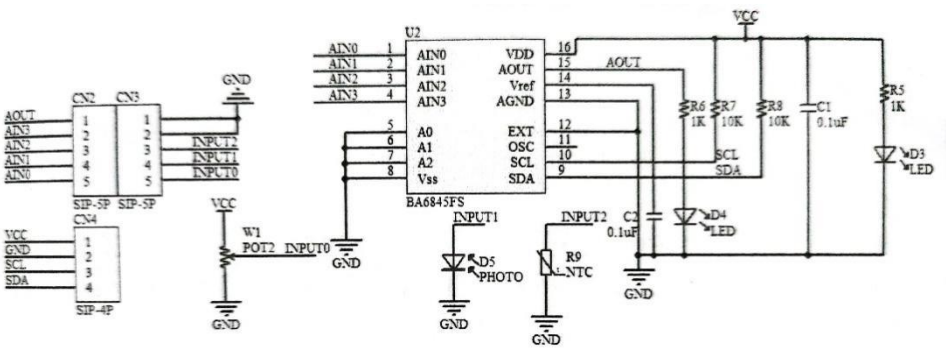


图 4.5.7 PCF8591 模块原理图

4.5.3、实验步骤

第 1 步：搭建实验电路

GPIO Expansion Board	PCF8591 模块	电位器模块	双色 LED 模块
SDA	SDA		
SCL	SCL		
5V	VCC	VCC	
GND	GND	GND	GND
	AIN0	OUT	
	AOUT		R

第 2 步：安装 python3 的 i2c 库

打开 Ubuntu 系统的终端，也可以在 JupyterLab 网页上操作：打开 JupyterLab 网页，点击主页上的终端图标（Teminal）。演示操作会在 JupyterLab 网页上进行。

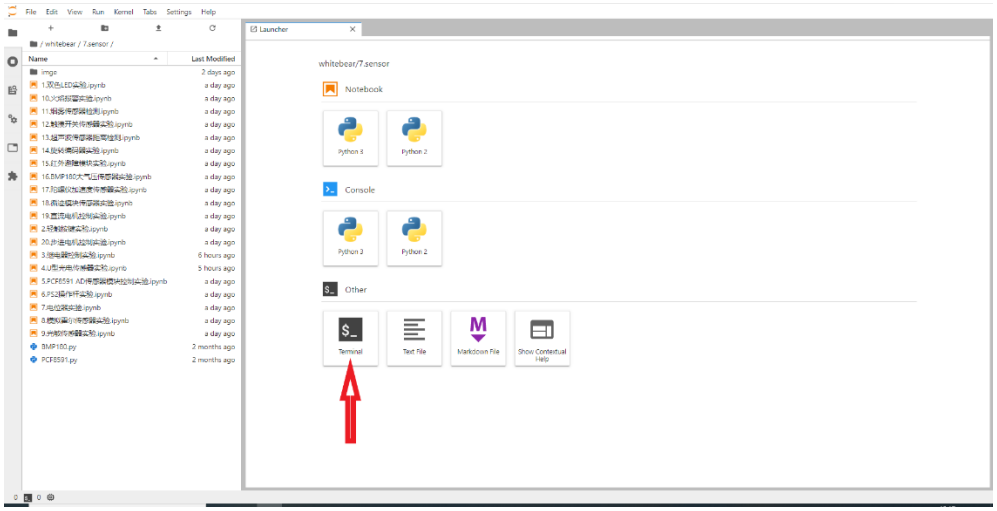


图 4.5.8 JupyterLab 网页打开终端

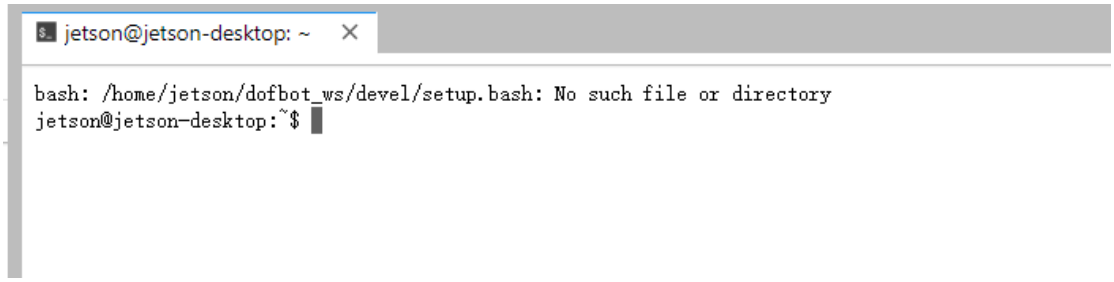


图 4.5.9 打开后的终端

首先我们需要打开 i2c 的用户权限，输入如下命令：

```
sudo usermod -a -G i2c $USER
```

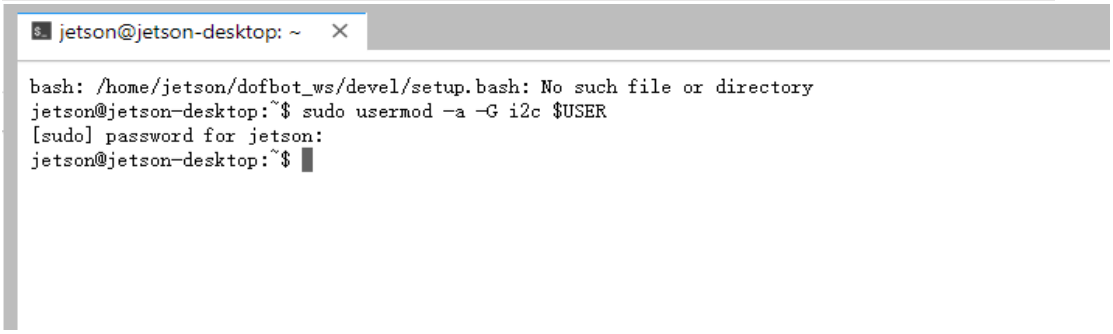


图 4.5.10 打开 i2c 的用户权限

输入如下命令查看 i2c 设备：

```
sudo i2cdetect -y -r 1
```

回车后，如果可以看到 48 地址，说明 PCF8591 模块已经连接到了 Jetson Nano 主板上。

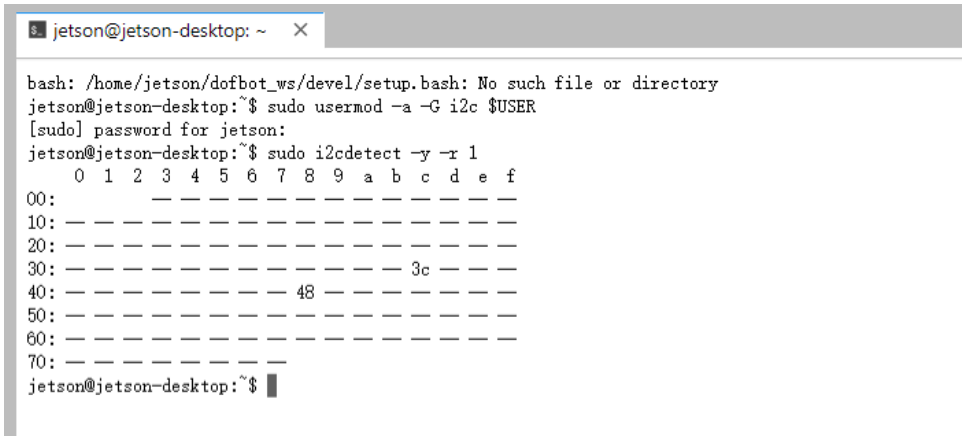


图 4.5.11 PCF8591 成功连接到 Jetson Nano

接下来，安装 python3 的 i2c 库，确保 Jetson Nano 可以连接互联网具有良好的网速。

```
sudo apt-get install python3-smbus
```

```
jetson@jetson-desktop:~$ sudo apt-get install python3-smbus
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
下列软件包是自动安装的并且现在不需要了：
  apt-clone archdetect-deb bogl-bterm busybox-static cryptsetup-bin dpkg-repack gir1.2-timezonemap-1.0 gir1.2-zkl-1.0 grub-common kde-window-manager kinit kio kpackagetool5 kwayland-data kwin-common
  kwin-data kwin-x11 libdebian-installer4 libhttp-parser2.7.1 libkdecorations2-5v5 libkdecorations2private5v5 libkf5activities5 libkf5attica5 libkf5completion-data libkf5completion5
  libkf5declarative-data libkf5declarative5 libkf5doctools5 libkf5globalaccel-data libkf5globalaccel5 libkf5globalaccelprivate5 libkf5idletime5 libkf5jobwidgets-data libkf5jobwidgets5
  libkf5kcmutils-data libkf5kcmutils5 libkf5kiocore5 libkf5kiontlm5 libkf5kiowidgets5 libkf5newstuff-data libkf5newstuff5 libkf5newstuffcore5 libkf5package-data libkf5package5 libkf5plasma5
  libkf5quickaddons5 libkf5solid5 libkf5solid5-data libkf5sonnet5-data libkf5sonnetcore5 libkf5sonnetui5 libkf5textwidgets-data libkf5textwidgets5 libkf5waylandclient5 libkf5waylandserver5
  libkf5xmlgui-bin libkf5xmlgui-data libkf5xmlgui5 libkscreenlocker5 libkwin4-effect-builtins1 libkwineffects11 libkwinglutils11 libkwinxrenderutils11 libllvm9 libqzsttools-p1 libqt5multimedia5
  libqt5multimedia5-plugins libqt5multimediaquick-p5 libqt5multimediawidgets5 libqt5quickwidgets5 libxcb-composite0 libxcb-cursor0 libxcb-damage0 nodejs-doc os-prober python3-click python3-colorama
  python3-dbus.mainloop.pyqt5 python3-icu python3-pam python3-pyqt5 python3-pyqt5.qtsvg python3-pyqt5.qtwebkit qml-module-org-kde-kquickcontrolsaddons qml-module-qtmultimedia qml-module-qtquick2
  rdate tasksel tasksel-data
使用'sudo apt autoremove'来卸载它(它们)。
下列【新】软件包将被安装：
  python3-smbus
升级了 0 个软件包，新安装了 1 个软件包，要卸载 0 个软件包，有 0 个软件包未被升级。
需要下载 6,832 B 的归档。
解压缩后会消耗 32.8 kB 的额外空间。
获取:1 http://ports.ubuntu.com/ubuntu-ports bionic/universe arm64 python3-smbus arm64 4.0-2 [6,832 B]
已下载 6,832 B，耗时 1秒 (11.4 kB/s)
debconf: delaying package configuration, since apt-utils is not installed
正在选中未选择的软件包 python3-smbus:arm64。
(正在读取数据库 ... 系统当前共安装有 200538 个文件和目录。)
正准备解包 .../python3-smbus_4.0-2_arm64.deb  ...
正在解包 python3-smbus:arm64 (4.0-2) ...
正在设置 python3-smbus:arm64 (4.0-2) ...
```

图 4.5.12 安装 python3 的 i2c 库

第 3 步：进入‘Notebook’文件夹里的‘sensor 传感器模块’文件夹，找到‘5.PCF8591 AD 转换控制实验.ipynb’程序，双击用 jupyterlab 打开程序:

第 4 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 jupyterlab 操作。

第 5 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：调节电位器，可以看到在程序中实时打印出 AIN0 模拟量的数值，即可以看到双色 LED 灯模块亮度会发生变化。

```
#程序入口
if __name__ == "__main__":
    try:
        makerobo_setup() #地址设置
        loop()           # 调用无限循环
    except KeyboardInterrupt:
        destroy() #释放AOUT端口
```

```
AIN0=255
AIN0=255
AIN0=255
AIN0=255
AIN0=213
AIN0=143
AIN0=91
AIN0=52
AIN0=20
AIN0=16
AIN0=16
```

图 4.5.13 PCF8591 模数转换器实验效果图 1

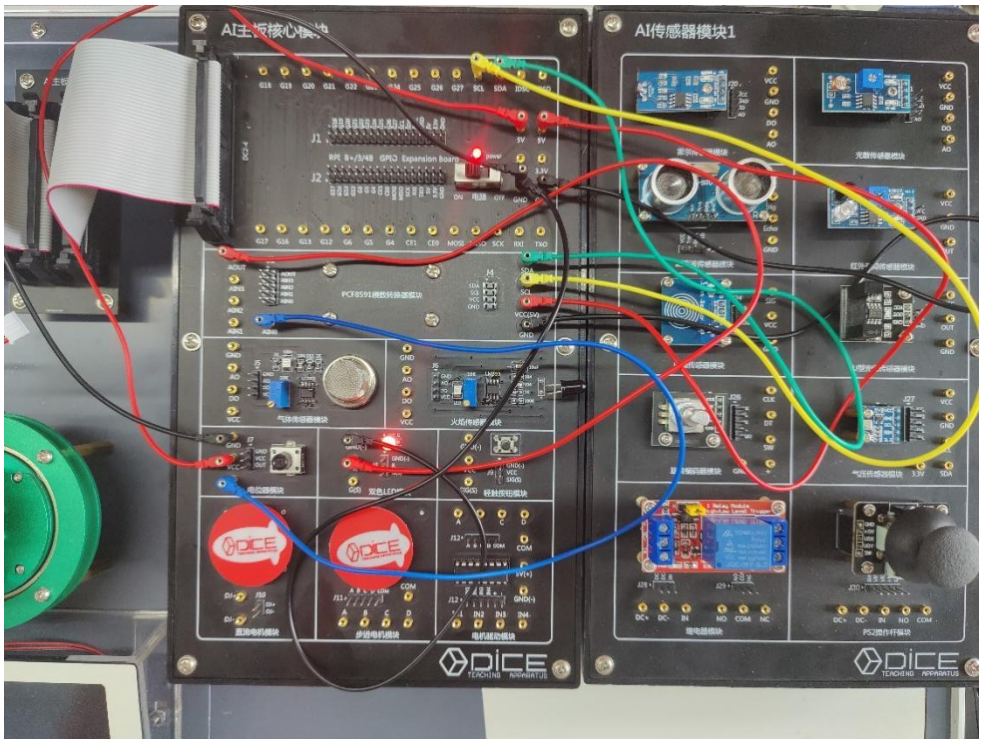


图 4.5.14 PCF8591 模数转换器实验效果图 2

4.5.4、python 代码

5. PCF8591AD 转换控制实验

@ 文件名：5.PCF8591 AD 转换控制实验.py

@ 版本：V2.0

@ 说明：AD/DA 传感器模块实验 由于 Jetson Nano GPIO 口不带模拟量输出，所以可以扩展出模拟量输入模块！

```
import PCF8591 as ADC
import time

# 模块地址设置
def makerobo_setup():
    ADC.setup(0x48)   # 设置 PCF8591 模块地址

# 无限循环
def loop():
    while True:
        print ("AIN0=%d"%ADC.read(0)) #读取 AIN0 的数值，插上跳线帽之后，采用的是内部的电位器；
        time.sleep(0.5)
        ADC.write(ADC.read(0)) # 控制 AOUT 输出电平控制 LED 灯
# 异常处理函数
```

```
def destroy():
    ADC.write(0) #AOUT 输出为 0
#程序入口
if __name__ == "__main__":
    try:
        makerobo_setup() #地址设置
        loop()              #  调用无限循环
    except KeyboardInterrupt:
        destroy() #释放 AOUT 端口
```

4.6、 PS2 操作杆实验

4.6.1、硬件介绍

许多机器人项目需要操作杆。该模块为此提供了负担得起的解决方案。游戏杆模块类似于游戏手柄中的模拟游戏杆。它是通过以 90 度角安装两个电位计来制成的。电位计连接到以弹簧为中心的短杆上。

图 4.6.1 PS2 操作杆

4.6.2、实验原理

处于静止位置时, 此模块从 X 和 Y 产生约 2.5V 的输出。移动操作杆将导致输出在 0V 到 5V 之间变化, 具体取决于其方向。如果将此模块连接到微控制器，则可能会在其静止位置读取大约 512 的值（由于弹簧和机构的微小误差而引起的细微变化）。移动操作杆时，应该看到该值从 0 变为 到 1023,具体取决于其位置。

该模块有两个模拟输出（对应 X 和 Y 坐标）和一个数字输出，表示是否在 Z 轴上按下。

在本实验中，我们将引脚 X 和 Y 连接到 A/D 转换器的模拟输入端口，以便将模拟量转换为数字量。然后在 Jetson Nano 上编程以检测操作杆的移动方向。

该模块的原理图如下所示:

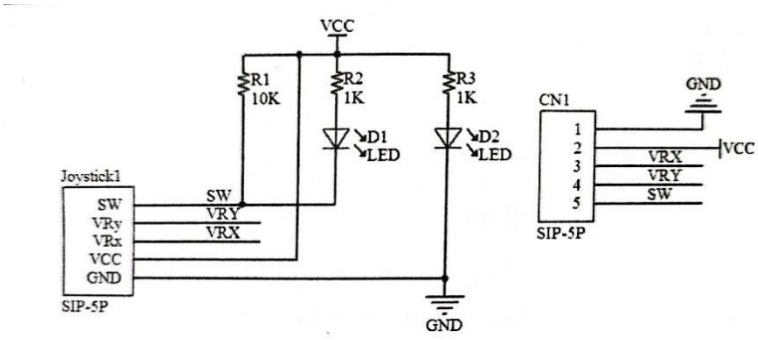


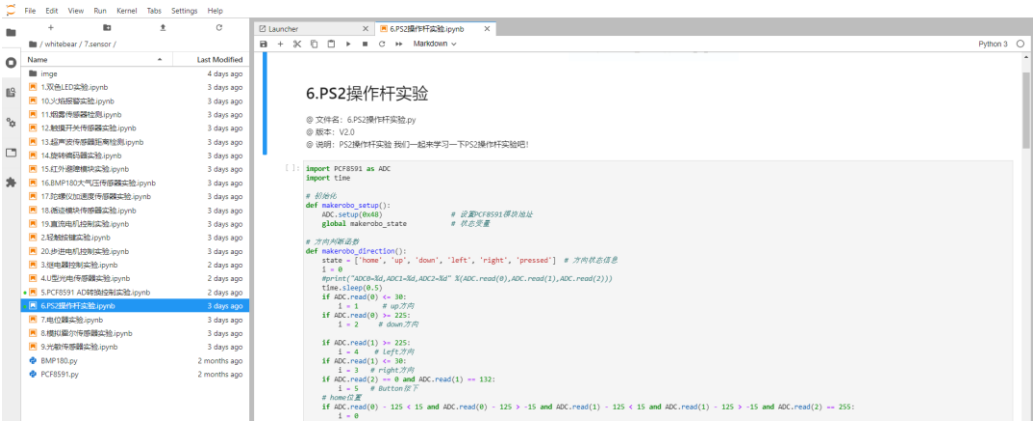
图 4.6.2 PS2 操作杆模块原理图

4.6.3、实验步骤

第 1 步：搭建实验电路

GPIO Expansion Board	PCF8591 模数转换器模块	PS2 操作杆模块
SDA	SDA	
SCL	SCL	
5V	VCC	+5V
GND	GND	GND
	AIN0	VRY
	AIN1	VRX
	AIN2	SW

第 2 步：进入‘Notebook’文件夹里的‘sensor 传感器模块’文件夹，找到‘6.PS2 操作杆实验.ipynb’程序，双击用 jupyterlab 打开程序:



4.6.3 PS2 操作杆实验程序截图

第 3 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 jupyterlab 操作。

第 4 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：现在将摇杆向上推，屏幕上会印上一条“up”字样；向下推，然后打印“down”；如果你向左推，“Left”将被打印；如果你向右推，就会打印出“Right”；如果按下盖子，屏幕上将打印“Button Pressed”。

```
        makerobo_loop() # 调用循环函数
    except KeyboardInterrupt:  # 当按下Ctrl+C时，将执行destroy()子程序。
        destroy()   # 调用释放函数
```

```
home
Button pressed
right
home
Button pressed
right
left
home
Button pressed
up
home
left
down
left
down
home
down
home
Button pressed
home
Button pressed
```

图 4.6.4 PS2 操作杆实验效果图 1

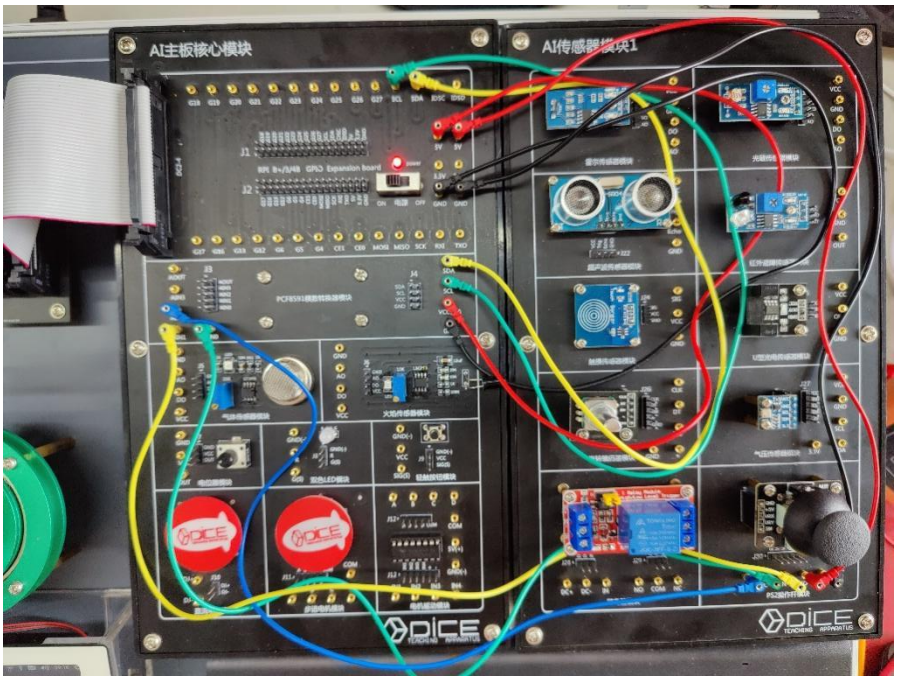


图 4.6.5 PS2 操作杆实验效果图 2

4.6.4、python 代码

6.PS2 操作杆实验

@ 文件名：6.PS2 操作杆实验.py

@ 版本：V2.0

@ 说明：PS2 操作杆实验 我们一起来学习一下 PS2 操作杆实验吧！

```
import PCF8591 as ADC
import time

# 初始化
def makerobo_setup():
    ADC.setup(0x48)                    # 设置 PCF8591 模块地址
    global makerobo_state              # 状态变量
```

```
# 方向判断函数
def makerobo_direction():
    state = ['home', 'up', 'down', 'left', 'right', 'Button pressed']   # 方向状态信息
    i = 0
    #print("ADC0=%d,ADC1=%d,ADC2=%d" %(ADC.read(0),ADC.read(1),ADC.read(2)))
    time.sleep(0.5)
    if ADC.read(0) <= 30:
        i = 1          # up 方向
    if ADC.read(0) >= 225:
        i = 2         # down 方向

    if ADC.read(1) >= 225:
        i = 4       # left 方向
    if ADC.read(1) <= 30:
        i = 3     # right 方向
    if ADC.read(2) == 0 and ADC.read(1) == 126:
        i = 5     # Button 按下
    # home 位置
    if ADC.read(0) - 125 < 15 and ADC.read(0) - 125 > -15 and ADC.read(1) - 125 < 15 and ADC.read(1) -
125 > -15 and ADC.read(2) == 255:
        i = 0
    return state[i]     # 返回状态

# 循环函数
def makerobo_loop():
    makerobo_status = ''       # 状态值赋空值
    while True:
        makerobo_tmp = makerobo_direction()     # 调用方向判断函数
        if makerobo_tmp != None and makerobo_tmp != makerobo_status:   # 判断状态是否发生改变
            print (makerobo_tmp) # 打印出方向位
            makerobo_status = makerobo_tmp # 把当前状态赋给状态值，以防止同一状态多次打印

# 异常处理函数
def destroy():
    pass

# 程序入口
if __name__ == '__main__':
```

```
makerobo_setup()    # 初始化
try:
    makerobo_loop() # 调用循环函数
except KeyboardInterrupt:    # 当按下 Ctrl+C 时，将执行 destroy()子程序。
    destroy()    # 调用释放函数
```

4.7、 霍尔传感器实验

4.7.1、硬件介绍

霍尔传感器是根据霍尔效应制作的一种磁场传感器。霍尔效应是磁电效应的一种，这一现象是霍尔（A.H.Hall，1855—1938）于 1879 年在研究金属的导电机构时发现的。后来发现半导体、导电流体等也有这种效应，而半导体的霍尔效应比金属强得多，利用这现象制成的各种霍尔元件，广泛地应用于工业自动化技术、检测技术及信息处理等方面。霍尔效应是研究半导体材料性能的基本方法。通过霍尔效应实验测定的霍尔系数，能够判断半导体材料的导电类型、载流子浓度及载流子迁移率等重要参数。

霍尔器件具有许多优点，它们的结构牢固，体积小，重量轻，寿命长，安装方便，功耗小，频率高（可达 1MHZ），耐震动，不怕灰尘、油污、水汽及盐雾等的污染或腐蚀。

霍尔线性器件的精度高、线性度好；霍尔开关器件无触点、无磨损、输出波形清晰、无抖动、无回跳、位置重复精度高（可达 μm 级）。取用了各种补偿和保护措施的霍尔器件的工作温度范围宽，可达－55℃～150℃。

按被检测的对象的性质可将它们的应用分为：直接应用和间接应用。前者是直接检测出受检测对象本身的磁场或磁特性，后者是检测受检对象上人为设置的磁场，用这个磁场来作被检测的信息的载体，通过它，将许多非电、非磁的物理量例如力、力矩、压力、应力、位置、位移、速度、加速度、角度、角速度、转数、转速以及工作状态发生变化的时间等，转变成电量来进行检测和控制。

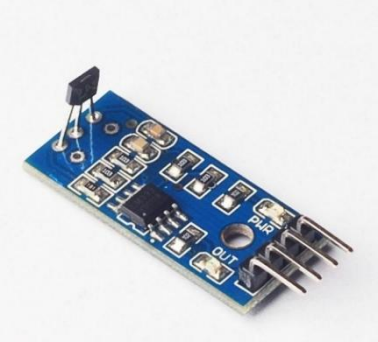


图 4.7.1 霍尔传感器模块

4.7.2、实验原理

霍尔效应：

霍尔效应传感器的工作原理上的原理霍尔效应，这指出，无论何时一个磁场在垂直的方向上施加到的流动电流在导体中，一个电势差引起的。该电压可用于检测传感器是否在磁体附近。Jetson Nano 可以通过其中断引脚检测此电压变化，并确定磁体是否靠近传感器。

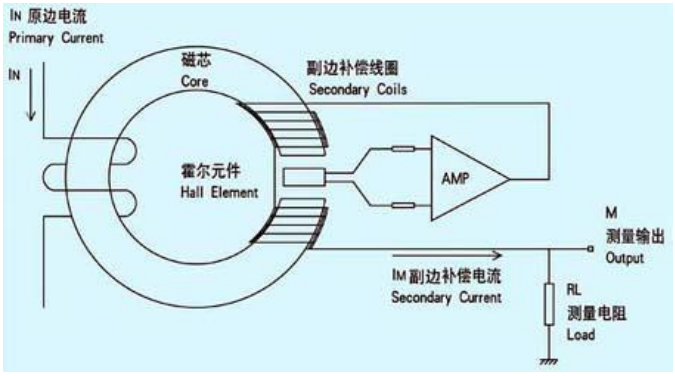


图 4.7.2 霍尔传感器基本原理

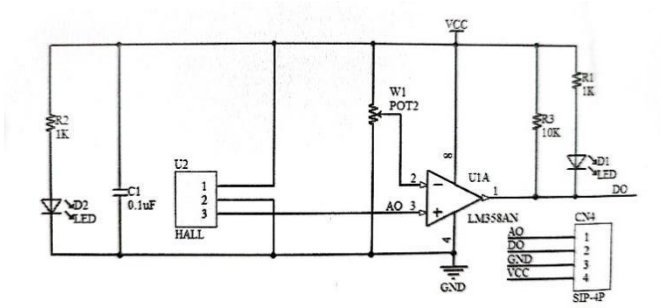


图 4.7.3 霍尔传感器模块原理图

4.7.3、实验步骤

第一步：搭建实验电路

GPIO Expansion Board	PCF8591 模数转换器模块	霍尔传感器
SDA	SDA	
SCL	SCL	
5V	VCC	
GND	GND	GND
	AIN0	A0
3.3V		VCC

第 2 步：进入‘Notebook’文件夹里的‘sensor 传感器模块’文件夹，找到‘7.霍尔传感器实验.ipynb’程序，双击用 JupyterLab 打开程序:

图 4.7.4 霍尔传感器实验程序截图

第 3 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 4 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：现在屏幕上将显示“Current intensity of magnetic field:xxx”。将磁铁靠近模拟霍尔传感器，北极磁极朝向传感器，然后显示“Magnet:North”。移开磁铁，然后显示“Magnet:None”。如果磁铁靠近传感器，南磁极朝向它，将在屏幕上显示“Magnet:South”。

注意：只有当磁铁的南极接近时，模拟霍尔传感器的引脚 D0 才会输出“0”，否则输出“1”。

```
Current intensity of magnetic field :  182

*****************
* Magnet North *
*****************

Current intensity of magnetic field :  177
Current intensity of magnetic field :  177
Current intensity of magnetic field :  1

*****************
* Magnet South *
*****************

Current intensity of magnetic field :  177

*****************
* Magnet North *
*****************

Current intensity of magnetic field :  1
```

图 4.7.5 霍尔传感器实验效果图 1

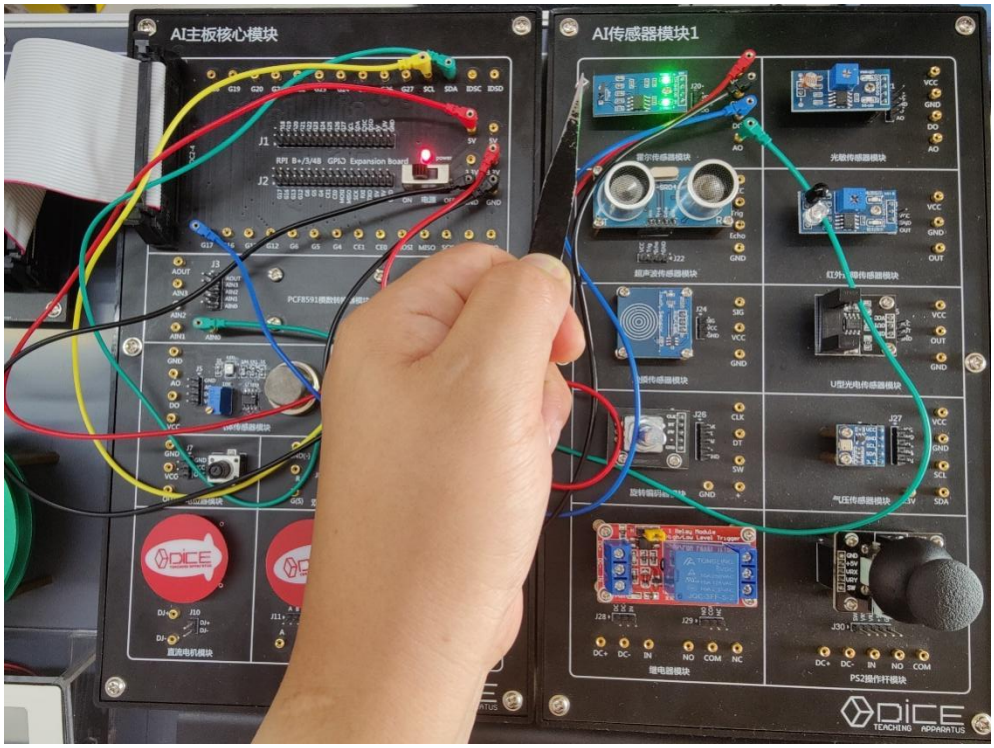


图 4.7.6 霍尔传感器实验效果图 2

4.7.4、python 代码

7.霍尔传感器实验

```
@ 文件名：7.霍尔传感器实验.py
@ 版本：V2.0
@ 说明：模拟霍尔传感器实验 我们一起来学习一下模拟霍尔传感器实验吧！

import RPi.GPIO as GPIO
import PCF8591 as ADC
import time

# 模块地址设置
def makerobo_setup():
    ADC.setup(0x48) # 设置 PCF8591 模块地址

# 打印出磁场的信息
def makerobo_Print(x):
    if x == 0:      #   没有磁场
        print ('')
        print ('**************')
        print ('* No Magnet *')
        print ('**************')
        print ('')
```

```
    if x == 1:      #  磁场为南
        print ('')
        print ('*****************')
        print ('* Magnet North *')
        print ('*****************')
        print ('')
    if x == -1:     # 磁场为北
        print ('')
        print ('*****************')
        print ('* Magnet South *')
        print ('*****************')
        print ('')


# 无限循环
def makerobo_loop():
    makerobo_status = 0 # 给状态变量赋 0 值
    while True:  # 无限循环
        makerobo_res = ADC.read(0) # 获取 AIN0 的模拟量值；
        print ('Current intensity of magnetic field : ', makerobo_res) # 打印出磁场的电流强度值
        if makerobo_res - 133 < 5 and makerobo_res - 133 > -5: # 判断磁场强度范围
            makerobo_tmp = 0      #  没有磁场
        if makerobo_res < 128:  # 磁场强度为北
            makerobo_tmp = -1
        if makerobo_res > 138:  # 磁场强度为南
            makerobo_tmp = 1
        if makerobo_tmp != makerobo_status: # 磁场发生改变
            makerobo_Print(makerobo_tmp) # 调用打印出磁场的信息
            makerobo_status = makerobo_tmp # 把当前状态值设置为比较状态值，避免重复打印；
        time.sleep(1.0) # 延时 200ms


# 程序入口
if __name__ == '__main__':
    makerobo_setup() # 调用初始化程序
    makerobo_loop()  # 调用循环程序
```

4.8、 光敏传感器实验

4.8.1、硬件介绍

光敏传感器是对外界光信号或光辐射有响应或转换功能的敏感装置。

光敏传感器是利用光敏元件将光信号转换为电信号的传感器，它的敏感波长在可见光波长附近，包括红外线波长和紫外线波长。光传感器不只局限于对光的探测，它还可以作为探测元件组成其他传感器，对许多非电量进行检测，只要将这些非电量转换为光信号的变化即可。

光敏传感器中最简单的电子器件是光敏电阻，它能感应光线的明暗变化，输出微弱的电信号，通过简单电子线路放大处理，可以控制 LED 灯具的自动开关。因此在自动控制、家用电器中得到广泛的应用，对于远程的照明灯具，例如：在电视机中作亮度自动调节，照相机中作自动曝光；另外，在路灯、航标等自动控制电路、卷带自停装置及防盗报警装置中等。

图 4.8.1 光敏传感器

4.8.2、实验原理

随着光强度的增加，光敏电阻的电阻将降低。因此输出电压改变。由光敏电阻收集的模拟信号通过 PCF8591 转换为数字信号。然后将这些数字信号传输到 Jetson Nano 并打印在屏幕上。模块原理图如下图所示:

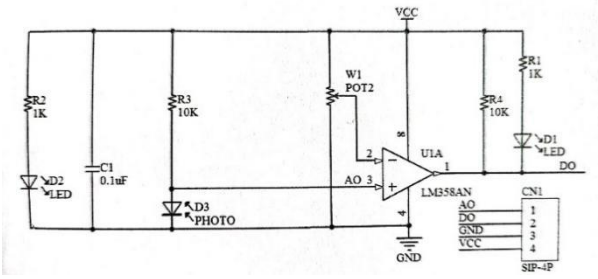


图 4.8.2 光敏传感器原理图

4.8.3、实验步骤

第 1 步：搭建实验电路

GPIO Expansion Board	PCF8591 模数转换器模块	光敏传感器
SDA	SDA	
SCL	SCL	
5V	VCC	VCC
GND	GND	GND
	AIN0	A0

第 2 步：进入‘Notebook’文件夹里的‘sensor 传感器模块’文件夹，找到‘8.光敏传感器实验.ipynb’程序，双击用 JupyterLab 打开程序:

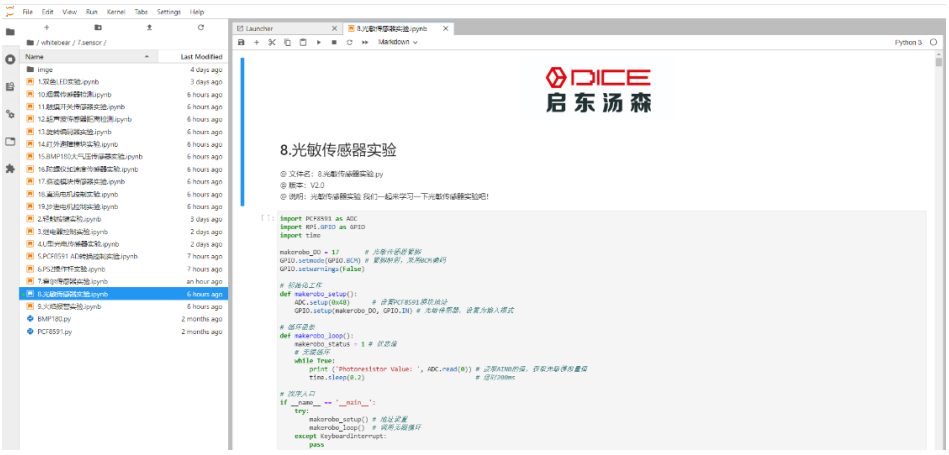


图 4.8.3 光敏传感器实验程序截图

第 3 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 4 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：改变光强度（例如用不透明纸遮挡模块或用手电筒照射模块），屏幕上打印的值将对应改变。

```
Photoresistor Value:  28
Photoresistor Value:  28
Photoresistor Value:  28
Photoresistor Value:  28
Photoresistor Value:  28
Photoresistor Value:  28
Photoresistor Value:  34
Photoresistor Value:  49
Photoresistor Value:  52
Photoresistor Value:  78
Photoresistor Value:  112
Photoresistor Value:  124
Photoresistor Value:  131
Photoresistor Value:  130
Photoresistor Value:  127
Photoresistor Value:  113
Photoresistor Value:  55
Photoresistor Value:  59
Photoresistor Value:  67
Photoresistor Value:  68
```

图 4.8.4 光敏传感器实验效果图 1

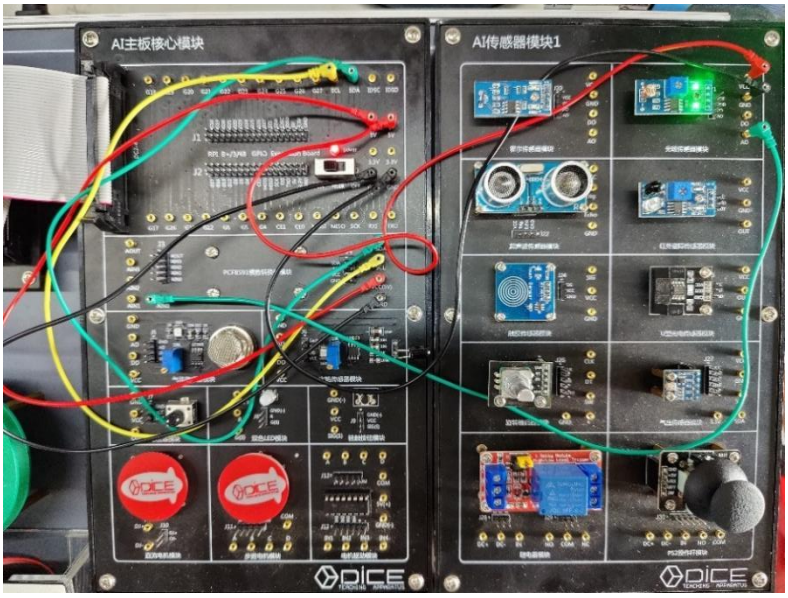


图 4.8.5 光敏传感器实验效果图 2

4.8.4、python 代码

8.光敏传感器实验

```
@ 文件名：8.光敏传感器实验.py
@ 版本：V2.0
@ 说明：光敏传感器实验 我们一起来学习一下光敏传感器实验吧！

import PCF8591 as ADC
import RPi.GPIO as GPIO
import time

makerobo_DO = 17          # 光敏传感器管脚
GPIO.setmode(GPIO.BCM) # 管脚映射，采用 BCM 编码
GPIO.setwarnings(False)

# 初始化工作
def makerobo_setup():
    ADC.setup(0x48)       # 设置 PCF8591 模块地址
    GPIO.setup(makerobo_DO, GPIO.IN) # 光敏传感器，设置为输入模式

# 循环函数
def makerobo_loop():
    makerobo_status = 1 # 状态值
    # 无限循环
    while True:
```

```
        print ('Photoresistor Value: ', ADC.read(0)) #  读取 AIN0 的值，获取光敏模拟量值
        time.sleep(0.2)                                  #  延时 200ms


#  程序入口
if __name__ == '__main__':
    try:
        makerobo_setup() #  地址设置
        makerobo_loop()   #  调用无限循环
    except KeyboardInterrupt:
        pass
```

4.9、 火焰报警实验

4.9.1、硬件介绍

火焰传感器，flame transducer 火焰是由各种燃烧生成物、中间物、高温气体、碳氢物质以及无机物质为主体的高温固体微粒构成的。火焰的热辐射具有离散光谱的气体辐射和连续光谱的固体辐射。不同燃烧物的火焰辐射强度、波长分布有所差异，但总体来说，其对应火焰温度的近红外波长域及紫外光域具有很大的辐射强度,根据这种特性可制成火焰传感器。

图 4.9.1 火焰传感器

4.9.2、实验原理

远红外火焰传感器能够探测到波长在 700 纳米～1000 纳米范围内的红外光，探测角度为 60，其中红外光波长在 880 纳米附近时，其灵敏度达到最大。远红外火焰探头将外界红外光的强弱变化转化为电流的变化，通过 A/D 转换器反映为 0～255 范围内数值的变化。外界红外光越强，数值越小；红外光越弱，数值越大。模块原理图如下所示：

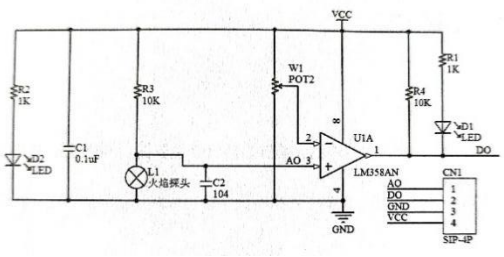


图 4.9.2 火焰传感器原理图

4.9.3、实验步骤

第 1 步：搭建实验电路

GPIO Expansion Board	PCF8591 模数转换器模块	火焰传感器
SDA	SDA	
SCL	SCL	
5V	VCC	VCC
GND	GND	GND
	AIN0	A0
GPIO23		D0

第 2 步：进入‘Notebook’文件夹里的‘sensor 传感器模块’文件夹，找到‘9.火焰报警实验.ipynb’程序，双击用 JupyterLab 打开程序:

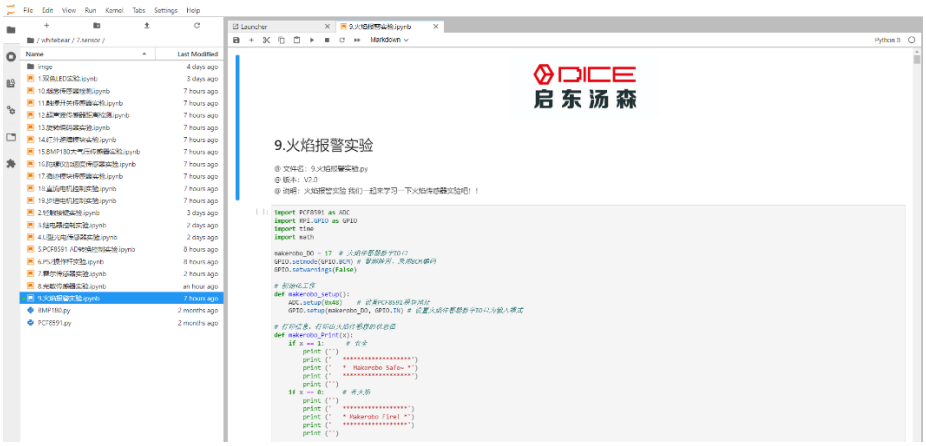


图 4.9.3 火焰传感器实验程序截图

第 3 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 4 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：现在，在传感器附近打开打火机，在 80cm 的范围内，将在屏幕上显示“whitebear Fire!”。

```
10
10
17
7


   ******************
   * whitebear Fire! *
   ******************

98
13


   ******************
   * whitebear Fire! *
   ******************

12


   ******************
   * whitebear Fire! *
   ******************

12


   ******************
   * whitebear Fire! *
   ******************
```

图 4.9.4 火焰传感器实验效果图 1

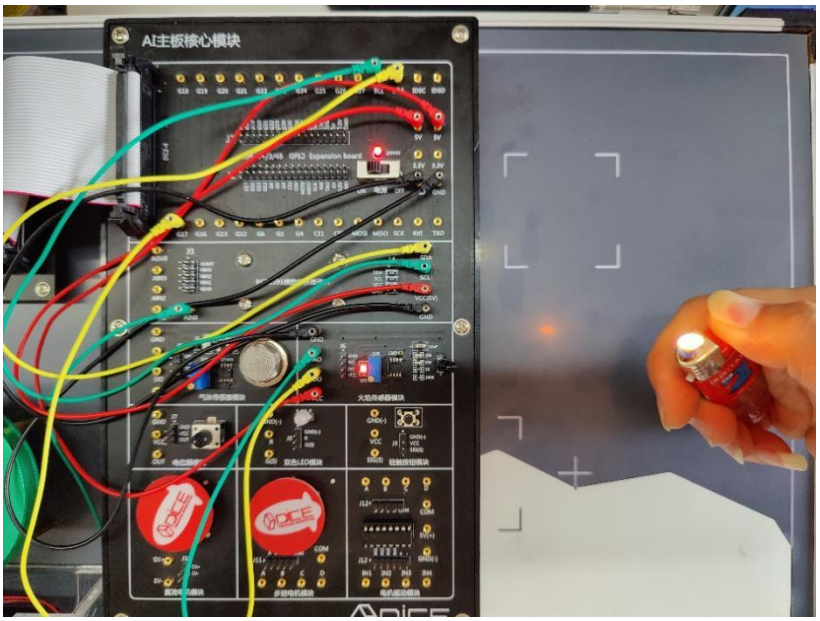


图 4.9.5 火焰传感器实验效果图 2

4.9.4、python 代码

9.火警报警实验

```
@ 文件名：9.火焰报警实验.py
@ 版本：V2.0
@ 说明：火焰报警实验 我们一起来学习一下火焰传感器实验吧！！

import PCF8591 as ADC
import RPi.GPIO as GPIO
import time
import math

whitebear_DO = 11   # 火焰传感器数字 IO 口
GPIO.setmode(GPIO.BOARD)          # Numbers GPIOs by physical location
GPIO.setwarnings(False)           #   忽略 GPIO 警告

# 初始化工作
def whitebear_setup():
    ADC.setup(0x48)      # 设置 PCF8591 模块地址
    GPIO.setup(whitebear_DO, GPIO.IN) # 设置火焰传感器数字 IO 口为输入模式

# 打印信息，打印出火焰传感器的状态值
def whitebear_Print(x):
    if x == 1:          # 安全
        print ('')
```

```
            print ('     ********************')
            print ('     * whitebear Safe~ *')
            print ('     ********************')
            print ('')
        if x == 0:        # 有火焰
            print ('')
            print ('     ********************')
            print ('     * whitebear Fire! *')
            print ('     ********************')
            print ('')


# 循环函数
def whitebear_loop():
    whitebear_status = 1          # 状态值
    # 无限循环
    while True:
        print (ADC.read(0))   # 读取 AIN0 的值，打印出火焰传感器值
        # 读取火焰传感器数字 IO 口
        whitebear_tmp = GPIO.input(whitebear_DO)
        if whitebear_tmp != whitebear_status:       # 判断状态发生改变
            whitebear_Print(whitebear_tmp)           # 打印出火焰传感器的提示信息
            status = whitebear_tmp                   # 当前状态值作为下次状态值进行比较，避免重复
打印


        time.sleep(1)                               # 延时 1s


# 程序入口
if __name__ == '__main__':
    try:
        whitebear_setup() # 初始化
        whitebear_loop()   # 循环函数
    except KeyboardInterrupt:
        pass
```

4.10、 气体传感器检测

4.10.1、硬件介绍

气体传感器适用于感测 LPG,烟雾，酒精，丙烷，氢气，甲烷和碳一氧化碳在空气中的浓度。如果您打算创建一个室内空气质量监测系统；呼吸检查器或早期火灾探测系统，MQ2 气体传感器模块是一个不错的选择。

图 4.10.1 MQ-2 气体检测传感器

4.10.2、实验原理

MQ2 是 MQ 传感器系列中最常用的气体传感器之一。这是一种金属氧化物半导体（MOS)型气体传感器，也称为化学传感器，因为检测是基于当气体与材料接触时感测材料的电阻变化。使用简单的分压器网络，可以检测气体浓度。MQ2 气体传感器可在 5VDC 上工作，功耗约 800mW.它可以检测 LPG、烟、酒、丙烷、氢气、甲烷和碳一氧化碳浓度的任何地方从 200 至 10000PPM.在这个实验中，如果有害气体达到一定浓度，红色指示灯闪烁报警。

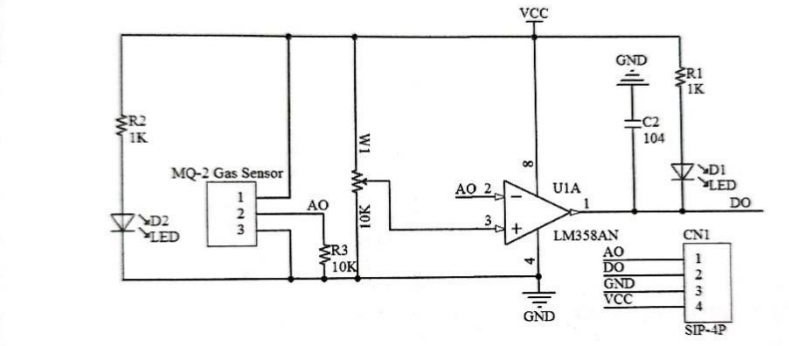


图 4.10.2 气体传感器模块原理图

4.10.3、实验步骤

第 1 步：搭建实验电路

GPIO Expansion Board	PCF8591 模数转换器模块	气体传感器
SDA	SDA	
SCL	SCL	
5V	VCC	VCC

GND	GND	GND
	AIN0	A0
GPIO23		D0

第 2 步：进入‘Notebook’文件夹里的‘sensor 传感器模块’文件夹，找到‘10.气体传感器检测.ipynb’程序，双击用 JupyterLab 打开程序:

图 4.10.3 气体检测传感器程序截图

第 3 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 4 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：我们轻轻按动打火机释放可燃气体的方式产生可燃性气体靠近 MQ-2 气体传感器的位置，屏幕上将显示 0 到 255 之间的值。如果有害气体达到一定浓度，蜂鸣器会发出蜂鸣声，并且屏幕上会印有 " whitebear Danger Gas!”。您也可以转动模块上电位器的轴来提高或降低浓度阈值。MQ-2 气体传感器需要加热一段时间。等到屏幕上打印的值保持稳定并且传感器变热，这意味着它可以正常且敏感地工作。

注意：气体传感器发热是正常的。实际上，温度越高，传感器就越敏感。

```
8
8
8
58
109

   ************************
   * Makerobo Danger Gas! *
   ************************

119
118
123
111
98
84
70
```

图 4.10.4 气体检测传感器实验效果图 1

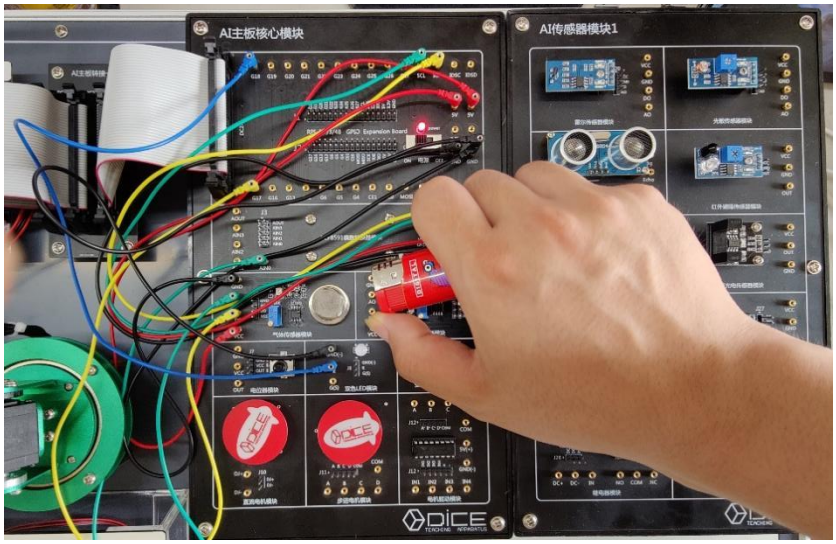


图 4.10.5 气体检测传感器实验效果图 2

4.10.4、python 代码

10. 气体传感器检测

```
@ 文件名：10.气体传感器检测.py
@ 版本：V2.0
@ 说明：气体传感器检测 我们一起来学习一下气体传感器实验吧！！

import PCF8591 as ADC
import RPi.GPIO as GPIO
import time
import math

whitebear_DO = 17                    # 烟雾传感器数字 IO 口
whitebear_Buzz = 18                  # led 数字 IO 口
GPIO.setmode(GPIO.BCM)               # 管脚映射，采用 BCM 编码
GPIO.setwarnings(False)              # 忽略 GPIO 警告

# 初始化工作
def whitebear_setup():
    ADC.setup(0x48)                          # 设置 PCF8591 模块地址
    GPIO.setup(whitebear_DO,GPIO.IN)         # 烟雾传感器数字 IO 口,设置为输入模式
    GPIO.setup(whitebear_Buzz,GPIO.OUT)      # led 数字 IO 口,设置为输出模式
    GPIO.output(whitebear_Buzz,0)            # 设置 led 为低电平，初始状态关闭 led

# 打印信息，打印出是否检测到烟雾信息
def whitebear_Print(x):
```

```
    if x == 1:        # 安全
        print ('')
        print ('     ******************')
        print ('     * whitebear Safe~ *')
        print ('     ******************')
        print ('')
    if x == 0:       # 检测到烟雾
        print ('')
        print ('     *************************')
        print ('     * whitebear Danger Gas! *')
        print ('     *************************')
        print ('')

# 循环函数
def whitebear_loop():
    whitebear_status = 1     # 定义状态值变量
    whitebear_count = 0      # 定义计数器变量值
    while True:      # 无限循环
        print (ADC.read(0)) # 获取 AIN0 的模拟量值；

        whitebear_tmp = GPIO.input(whitebear_DO)   # 读取 GAS 烟雾传感器数字 IO 口值
        if whitebear_tmp != whitebear_status:       # 判断状态发生改变
            whitebear_Print(whitebear_tmp)           # 打印函数，打印出烟雾传感器信息
            whitebear_status = whitebear_tmp         # 把当前状态值设置为比较状态值，避免重复打
印；
        if whitebear_status == 0:                    # 当检测到烟雾
            whitebear_count += 1                     # 计数器值累计
            # 高低电平交替变化，led 闪烁
            if whitebear_count % 2 == 0:             # 进行求余处理，一半为 1，一半为 0
                GPIO.output(whitebear_Buzz, 1)
            else:
                GPIO.output(whitebear_Buzz, 0)
        else:
            GPIO.output(whitebear_Buzz, 0)           # 设置 led 为低电平，初始状态关闭 led
            whitebear_count = 0                      # 计数器赋 0

        time.sleep(1)                               # 延时 1s

# 资源释放函数
```

```
def destroy():
    GPIO.output(whitebear_Buzz, 0)    # 设置 led 为低电平，初始状态关闭 led
    GPIO.cleanup()                # 释放资源


# 程序入口
if __name__ == '__main__':
    try:
        whitebear_setup()     # 初始化函数
        whitebear_loop()      # 循环函数
    except KeyboardInterrupt: # 当按下 Ctrl+C 时，将执行 destroy()子程序。
        destroy()   #资源释放
```

4.11、 触摸开关传感器实验

4.11.1、硬件介绍

触摸开关传感器是一种复制单个触觉按钮的触摸板检测器 IC.该触摸检测 IC 旨在用各种尺寸的键盘代替传统的直接按钮键。

图 4.11.1 触摸开关传感器

4.11.2、实验原理

数字电容式触摸传感器模块基于 TTP223B.通常，它输出“LOW”并保持较低状态。如果在圆形标记区域检测到触摸，则输出“HIGH”并切换到快速响应状态。如果将模块安装在非金属表面上，如果未触摸 12 秒钟，则会再次切换到低功耗状态。例如玻璃，塑料和丙烯酸等材料，您可以将钥匙隐藏在墙壁和桌子上。

TTP223 是触摸板检测器集成电路，提供一个触摸键。TTP223 触摸检测 IC 旨在替代传统的直接按键。容式触摸使电子设备可以感应到手指何时位于表面几毫米内，从而像按按钮一样刺激按钮的按下。电容式感应可以在任何需要低至无力感应的地方使用。

该模块的原理图如下所示:

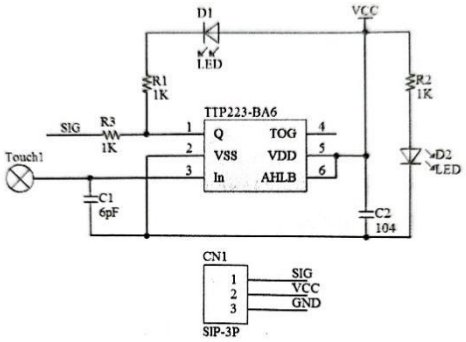


图 4.11.2 触摸传感器原理图

4.11.3、实验步骤

第 1 步：搭建实验电路

Jetson Nano pin	GPIO Expansion Board	触摸传感器	双色 LED
16	GPIO23	SIG	
5V	5V	VCC	
GND	GND	GND	GND
12	GPIO18		R
13	GPIO27		G(S)

第 2 步：进入‘Notebook’文件夹里的‘sensor 传感器模块’文件夹，找到‘11.触摸开关传感器实验.ipynb’程序，双击用 JupyterLab 打开程序:

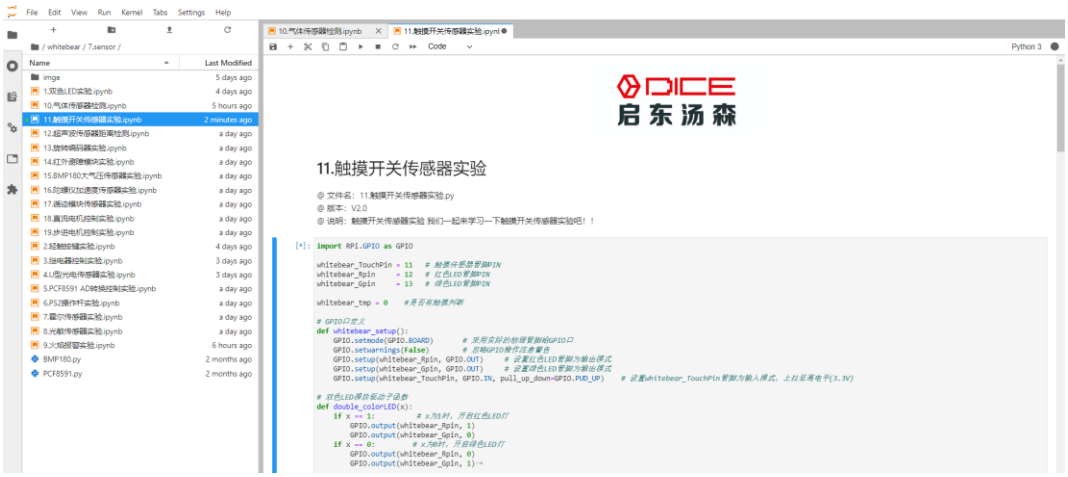


图 4.11.3 触摸开关传感器实验程序截图

第 3 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 4 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：现在，触摸金属盘，您可以看到 LED 改变其颜色，并在屏幕上打印“whitebear Have a touch"和“whitebear No touch occurred”。

```
*************************
* whitebear Have a touch *
*************************
*****************************
* whitebear No touch occurred*
*****************************
*************************
* whitebear Have a touch *
*************************
*****************************
* whitebear No touch occurred*
*****************************
*************************
* whitebear Have a touch *
*************************
*****************************
* whitebear No touch occurred*
*****************************
```

图 4.11.4 触摸开关传感器实验效果图 1

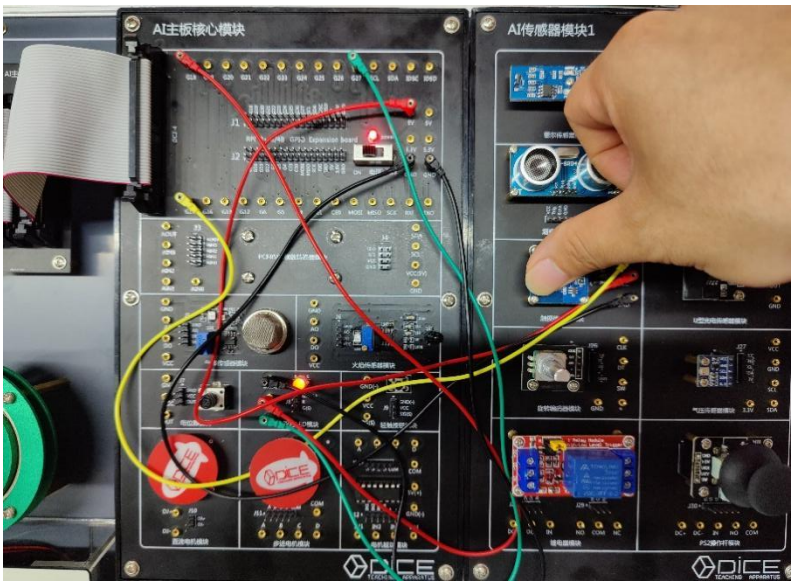


图 4.11.5 触摸开关传感器实验效果图 2

4.11.4、python 代码

11.触摸开关传感器实验

@ 文件名：11.触摸开关传感器实验.py

@ 版本：V2.0

@ 说明：触摸开关传感器实验 我们一起来学习一下触摸开关传感器实验吧！！

```
import RPi.GPIO as GPIO

whitebear_TouchPin = 11     # 触摸传感器管脚 PIN
whitebear_Rpin     = 12     # 红色 LED 管脚 PIN
whitebear_Gpin     = 13     # 绿色 LED 管脚 PIN

whitebear_tmp = 0       #是否有触摸判断

# GPIO 口定义
def whitebear_setup():
```

```
    GPIO.setmode(GPIO.BOARD)          # 采用实际的物理管脚给 GPIO 口
    GPIO.setwarnings(False)           # 忽略 GPIO 操作注意警告
    GPIO.setup(whitebear_Rpin, GPIO.OUT)      # 设置红色 LED 管脚为输出模式
    GPIO.setup(whitebear_Gpin, GPIO.OUT)      # 设置绿色 LED 管脚为输出模式
    GPIO.setup(whitebear_TouchPin,  GPIO.IN,  pull_up_down=GPIO.PUD_UP)          #  设  置
whitebear_TouchPin 管脚为输入模式，上拉至高电平(3.3V)

# 双色 LED 模块驱动子函数
def double_colorLED(x):
    if x == 1:              # x 为 1 时，开启红色 LED 灯
        GPIO.output(whitebear_Rpin, 1)
        GPIO.output(whitebear_Gpin, 0)
    if x == 0:              # x 为 0 时，开启绿色 LED 灯
        GPIO.output(whitebear_Rpin, 0)
        GPIO.output(whitebear_Gpin, 1)

# 打印函数，显示出是否有触摸
def whitebear_Print(x):
    global whitebear_tmp
    if x != whitebear_tmp:
        if x == 1:     #  发生触摸
            print ('**************************')
            print ('* whitebear Have a touch *')
            print ('**************************')

        if x == 0:     # 没有发生触摸
            print ('********************************')
            print ('* whitebear No touch occurred*')
            print ('********************************')
        whitebear_tmp = x   # 触摸状态值保存和下次做比较，避免重复打印

# 循环函数
def whitebear_loop():
    while True:   # 无限循环
        double_colorLED(GPIO.input(whitebear_TouchPin)) # 调用双色 LED 模块驱动子函数
        whitebear_Print(GPIO.input(whitebear_TouchPin))   # 调用打印函数，显示出是否有触摸

# 资源释放函数
def destroy():
```

```
    GPIO.output(whitebear_Gpin, GPIO.LOW)          # 关闭绿色 LED
    GPIO.output(whitebear_Rpin, GPIO.LOW)          # 关闭红色 LED
    GPIO.cleanup()                                 # 释放资源


# 程序入口
if __name__ == '__main__':
    whitebear_setup()   # GPIO 口定义
    try:
        whitebear_loop() # 循环函数
    except KeyboardInterrupt:     # 当按下 Ctrl+C 时，将执行 destroy()子程序。
        destroy() # 资源释放函数
```

4.12、超声波传感器距离检测

4.12.1 硬件介绍

超声波传感器是将超声波信号转换成其它能量信号（通常是电信号）的传感器。超声波是振动频率高于 20kHz 的机械波。它具有频率高、波长短、绕射现象小，特别是方向性好、能够成为射线而定向传播等特点。超声波对液体、固体的穿透本领很大，尤其是在阳光不透明的固体中。超声波碰到杂质或分界面会产生显著反射形成反射回波，碰到活动物体能产生多普勒效应。因此超声波测距对环境有较好的适应能力，此外超声波测量在实时、精度、价格也能得到很好的折衷。超声波传感器广泛应用在工业、国防、生物医学等方面。

HC-SR04 超声波测距模块可提供 2cm-400cm 的非接触式距离感测功能，测距精度可达高到 3mm；模块包括超声波发射器、接收器与控制电路。像智能小车的测距以及转向，或是一些项目中，常常会用到。智能小车测距可以及时发现前方的障碍物，使智能小车可以及时转向，避开障碍物。

图 4.12.1 超声波模块

4.12.2、实验原理

当持续时间至少为 10μS(10 微秒）的脉冲施加到触发引脚时，一切就开始了。响应于此，传感器以 40 KHz 发射八个脉冲的声音脉冲。这种 8 脉冲模式使设备的“超声特征”变得独一无二，从而使接收器能够将发射的模式与周围的超声噪声区分开。八个超声波脉冲通过空气传播，同时，回声引脚变为高电平，开始形成回声信号的开始。如果这些脉冲没有被反射回来，那么回波信号将在 38 毫秒（38 毫秒）后超时并返回低电平。因此，38ms 的脉冲表示在传感器范围内没有障碍物。

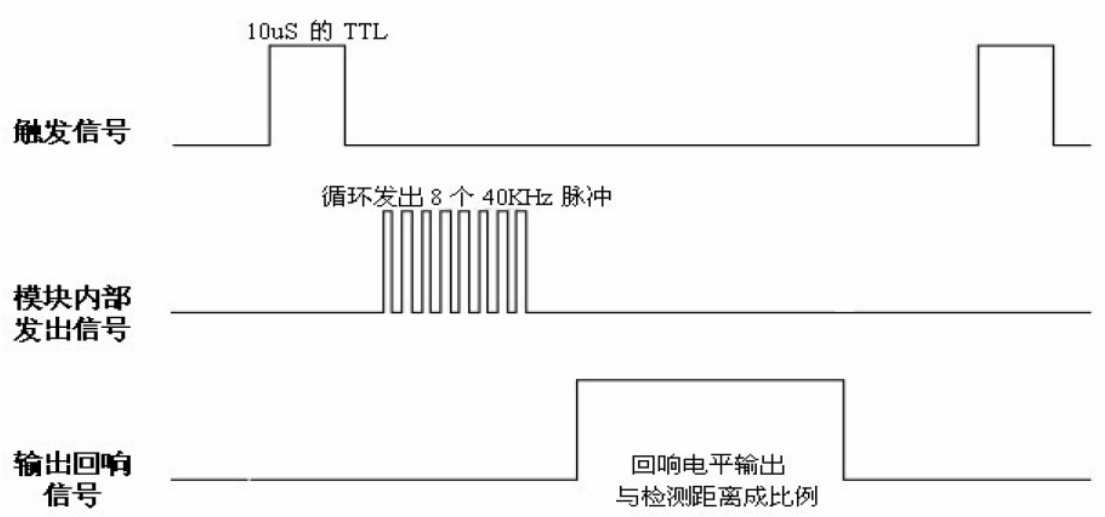


图 4.12.2 超声波时序图

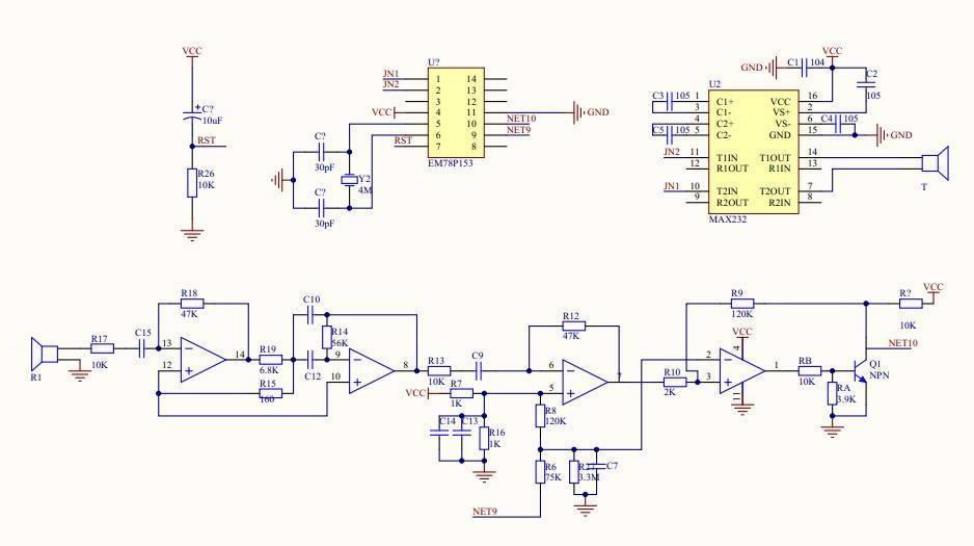


图 4.12.3 HC-SR04 超声波传感器模块原理图

4.12.3、实验步骤

第 1 步：搭建实验电路

GPIO Expansion Board	超声波传感器模块
GPIO23	Trig
GPIO18	Echo
GND	GND
5V	VCC

第 2 步：进入‘Notebook’文件夹里的‘sensor 传感器模块’文件夹，找到‘12.超声波传感器距离检测.ipynb’程序，双击用 JupyterLab 打开程序：

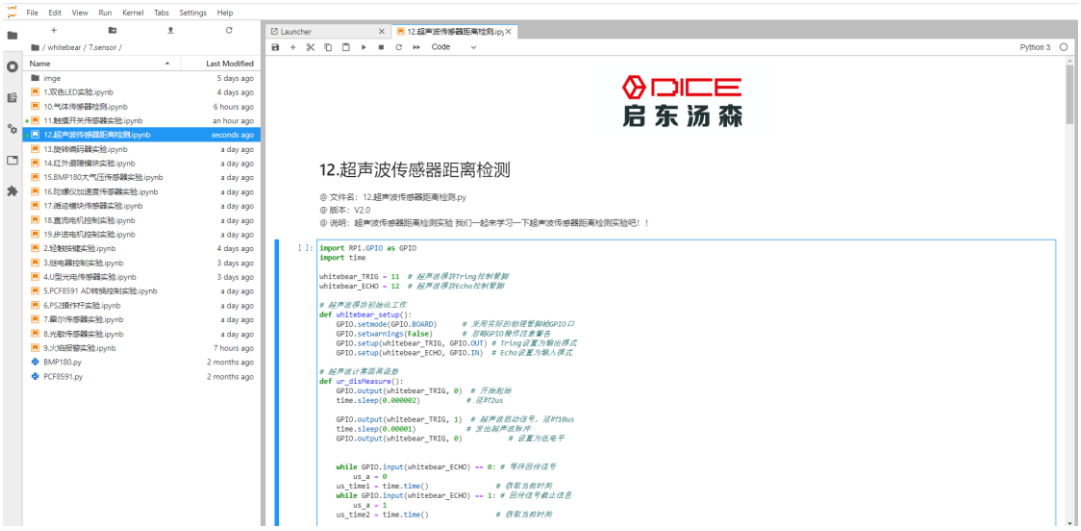


图 4.12.4 超声波传感器距离检测程序截图

第 3 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 4 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：现在您可以看到超声波测距模块与屏幕前方障碍物之间的距离。慢慢地将遮挡物放在超声测距模块上方，观察屏幕上打印的距离。

```
33.75434875488281 cm
15.738248825073242 cm
16.581296920776367 cm
15.580177307128906 cm
15.300512313842773 cm
19.122600555419922 cm
15.462636947631836 cm
32.17363357543945 cm
31.666994094848633 cm
198.74858856201172 cm
125.78034400939941 cm
```

图 4.12.5 超声波距离检测实验效果图 1

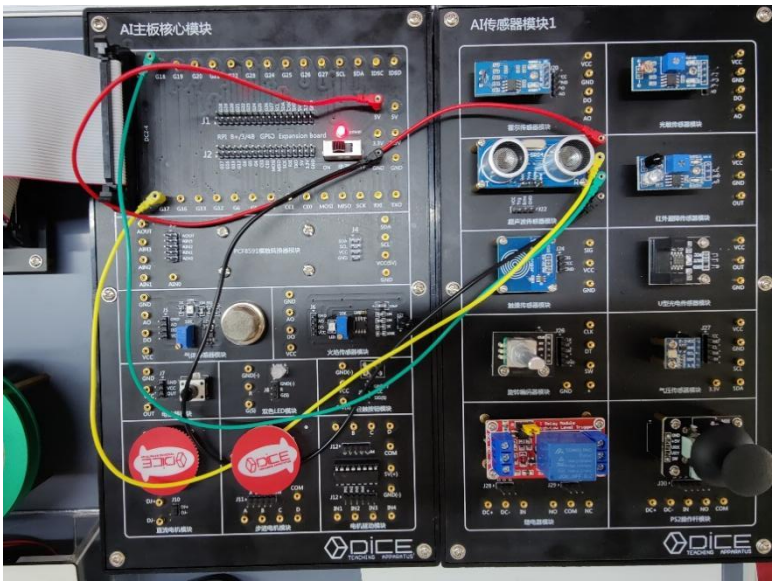


图 4.12.6 超声波距离检测实验效果图 2

4.12.4、python 代码

12.超声波传感器距检测

@ 文件名：12.超声波传感器距离检测.py

@ 版本：V2.0

@ 说明：超声波传感器距离检测实验 我们一起来学习一下超声波传感器距离检测实验吧！！

```
import RPi.GPIO as GPIO
import time

whitebear_TRIG = 11   # 超声波模块 Tring 控制管脚
whitebear_ECHO = 12   # 超声波模块 Echo 控制管脚

# 超声波模块初始化工作
def whitebear_setup():
    GPIO.setmode(GPIO.BOARD)        # 采用实际的物理管脚给 GPIO 口
    GPIO.setwarnings(False)         # 忽略 GPIO 操作注意警告
    GPIO.setup(whitebear_TRIG, GPIO.OUT) # Tring 设置为输出模式
    GPIO.setup(whitebear_ECHO, GPIO.IN)   # Echo 设置为输入模式

# 超声波计算距离函数
def ur_disMeasure():
    GPIO.output(whitebear_TRIG, 0)   # 开始起始
    time.sleep(0.000002)             # 延时 2us

    GPIO.output(whitebear_TRIG, 1)   # 超声波启动信号，延时 10us
```

```
    time.sleep(0.00001)                 # 发出超声波脉冲
    GPIO.output(whitebear_TRIG, 0)              # 设置为低电平



    while GPIO.input(whitebear_ECHO) == 0: # 等待回传信号
        us_a = 0
    us_time1 = time.time()                      # 获取当前时间
    while GPIO.input(whitebear_ECHO) == 1: # 回传信号截止信息
        us_a = 1
    us_time2 = time.time()                      # 获取当前时间

    us_during = us_time2 - us_time1             # 转换微秒级的时间

    # 声速在空气中的传播速度为 340m/s, 超声波要经历一个发送信号和一个回波信息，
    # 计算公式如下所示：
    return us_during * 340 / 2 * 100            # 求出距离

# 循环函数
def whitebear_loop():
    while True:
        us_dis = ur_disMeasure()    # 获取超声波计算距离
        print (us_dis, 'cm')            # 打印超声波距离值
        print ('')
        time.sleep(1)                   # 延时 300ms

# 资源释放函数
def destroy():
    GPIO.cleanup() # 释放资源

# 程序入口
if __name__ == "__main__":
    whitebear_setup() # 调用初始化函数
    try:
        whitebear_loop() # 调用循环函数
    except KeyboardInterrupt: # 当按下 Ctrl+C 时，将执行 destroy()子程序。
        destroy() # 释放资源
```

4.13、 旋转编码器实验

4.13.1、硬件介绍

旋转编码器是一种位置传感器，用于确定旋转轴的角位置。它根据旋转运动生成模拟或数字电信号，旋转编码器有许多不同类型，它们根据输出信号或传感技术进行分类，常见类型有增量式编码器，绝对值编码器，光电式编码器，磁式编码器，电容式编码器等。。我们在本教程中将使用的特定旋转编码器是增量式旋转编码器，该旋转编码器也称为正交编码器或相对旋转编码器，其输出是一系列方波脉冲。它是测量旋转的最简单的位置传感器。

图 4.13.1 增量式旋转编码器

4.13.2、实验原理

增量式旋转编码器通过内部两个光敏接受管转化其角度码盘的时序和相位关系，得到其角度码盘角度位移量增加（正方向）或减少（负方向）。在接合数字电路特别是单片机后，增量式旋转编码器在角度测量和角速度测量较绝对式旋转编码器更具有廉价和简易的优势。

下面对增量式旋转编码器的内部工作原理:

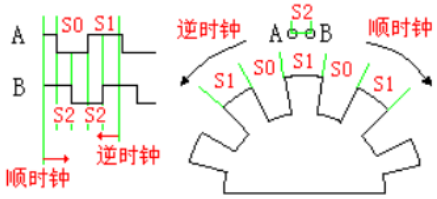


图 4.13.2 增量式旋转编码器内部工作原理

A,B 两点对应两个光敏接受管，A,B 两点间距为 S2 ,角度码盘的光栅间距分别为 S0 和 S1。当角度码盘以某个速度匀速转动时，那么可知输出波形图中的 S0：S1：S2 比值与实际图的 S0：S1：S2 比值相同，同理角度码盘以其他的速度匀速转动时，输出波形图中的 S0：S1：S2 比值与实际图的 S0：S1：S2 比值仍相同。如果角度码盘做变速运动，把它看成为多个运动周期（在下面定义）的组合，那么每个运动周期中输出波形图中的 S0：S1：S2 比值与实际图的 S0：S1：S2 比值仍相同。

通过输出波形图可知每个运动周期的时序为：

顺时针运动	逆时针运动
AB	AB
11	11
01	10
00	00
10	01

图 4.13.3 运动周期时序

我们把当前的 A、B 输出值保存起来，与下一个到来的 A、B 输出值做比较，就可以得出角度码盘转动的方向。

如果光栅格 S0 等于 S1 时，也就是 S0 和 S1 弧度夹角相同，且 S2 等于 S0 的 1/2，那么可得到此次角度码盘运动位移角度为 S0 弧度夹角的 1/2，再除以所用的时间，就得到此次角度码盘运动的角速度。S0 等于 S1 时，且 S2 等于 S0 的 1/2 时，1/4 个运动周期就可以得到运动方向位和位移角度，如果 S0 不等于 S1，S2 不等于 S0 的 1/2，那么要 1 个运动周期才可以得到运动方向位和位移角度了。

我们常用的鼠标的滚轮也是这个原理。

实际使用的增量式编码器输出三组方波脉冲 A、B 和 Z（有的叫 C 相）相。A、B 两组脉冲相位差 90°，可以判断出旋转方向和旋转速度。而 Z 相脉冲又叫做零位脉冲（有时也叫索引脉冲），为每转一周输出一个脉冲，Z 相脉冲代表零位参考位，通过零位脉冲，可获得编码器的零位参考位，专门用于基准点定位，如下图所示。

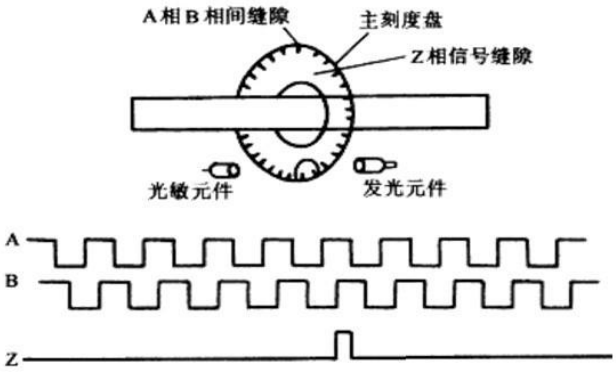


图 4.13.4 增量式旋转编码器工作原理详细图解 1

增量式编码器转轴旋转时，有相应的脉冲输出，其计数起点可以任意设定，可实现多圈无限累加和测量。编码器轴转动一圈会输出固定的脉冲数，脉冲数由编码器码盘上面的光栅的线数所决定，编码器以

每旋转 360 度提供多少通或暗的刻线称为分辨率，也称解析分度、或称作多少线，一般在每转 5~10000 线，当需要提高分辩率时，可利用 90 度相位差的 A、B 两路信号进行倍频或者更换高分辩率编码器。

增量型编码器精度取决于机械和电气的因素，这些因素有：光栅分度误差、光盘偏心、轴承偏心、电子读数装置引入的误差以及光学部分的不精确性，误差存在于任何编码器中。

编码器的信号输出有正弦波（电流或电压）、方波（TTL、HTL）等多种形式。并且都可以用差分驱动方式，含有对称的 A+/A-、B+/B-、Z+/Z-三相信号，由于带有对称负信号的连接，电流对于电缆贡献的电磁场为 0，信号稳定衰减最小，抗干扰最佳，可传输较远的距离，例如：对于 TTL 的带有对称负信号输出的编码器，信号传输距离可达 150 米。对于 HTL 的带有对称负信号输出的编码器，信号传输距离可达 300 米。

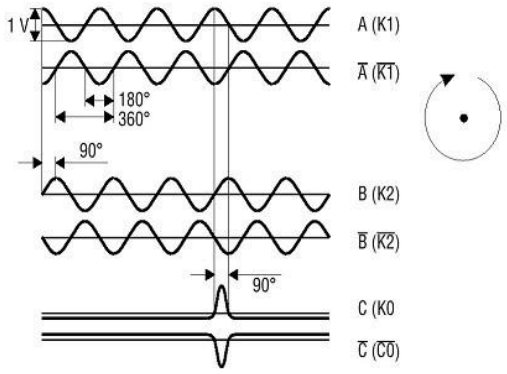


图 4.13.5 增量式旋转编码器工作原理详细图解 2

模块原理图如下：

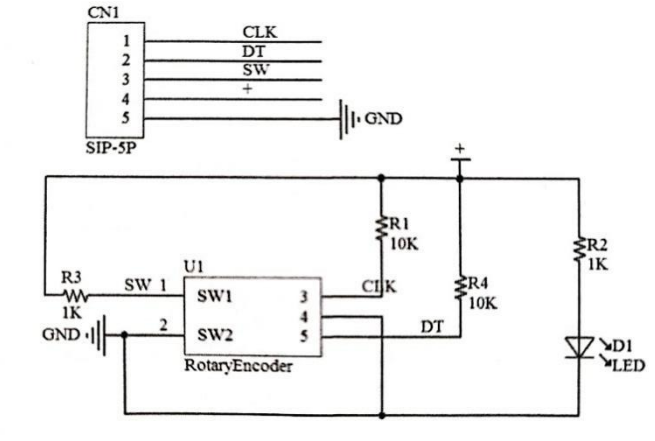


图 4.13.6 旋转编码器模块原理图

4.13.3、实验步骤

第 1 步：搭建实验电路

GPIO Expansion Board	旋转编码器模块
GPIO23	CLK

GPIO18	DT
GPIO27	SW
GND	GND
5V	+

第 2 步：进入‘Notebook’文件夹里的‘sensor 传感器模块’文件夹，找到‘13.旋转编码器实验.ipynb’程序，双击用 JupyterLab 打开程序:

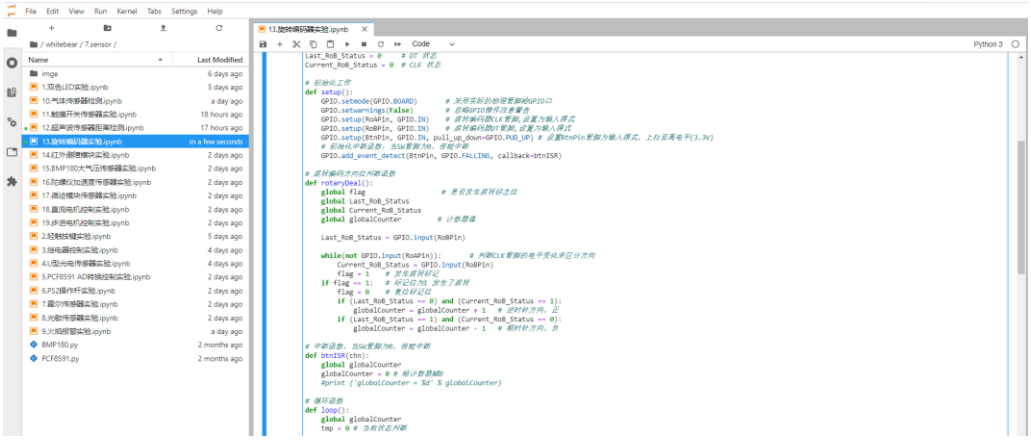


图 4.13.7 旋转编码器实验程序截图

第 3 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 4 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：现在，您可以看到屏幕上打印的旋转编码器的角位移。当旋转编码器顺时针旋转时，角位移增大；当它逆时针旋转时，数值会减小。按下旋转编码器上的开关，该值将返回到零。

```
globalCounter = -1
globalCounter = -2
globalCounter = -3
globalCounter = -4
globalCounter = -5
globalCounter = -6
globalCounter = -7
globalCounter = -8
globalCounter = -9
globalCounter = -10
globalCounter = -11
globalCounter = -12
globalCounter = -13
globalCounter = 0
```

图 4.13.8 旋转编码器实验效果图 1

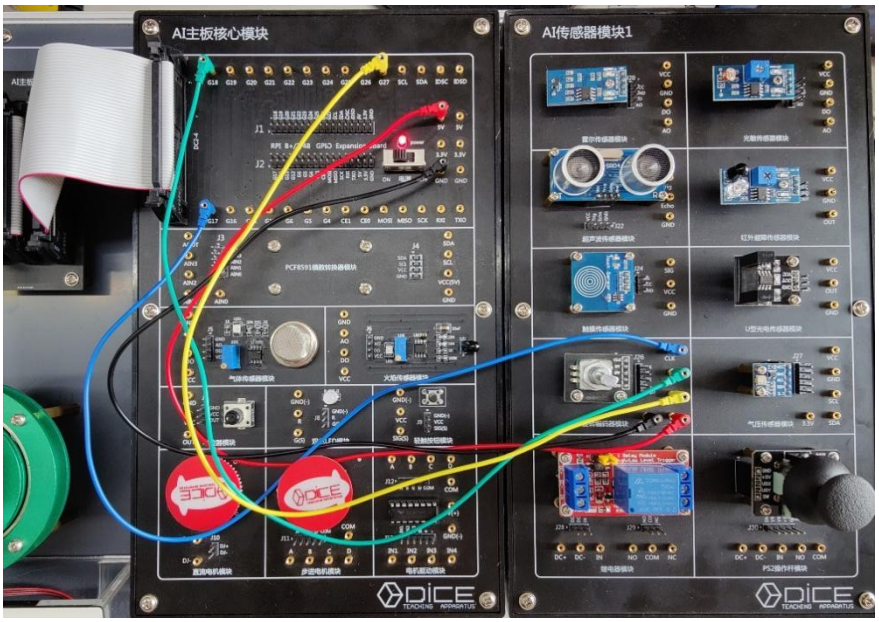


图 4.13.9 旋转编码器实验效果图 2

4.13.4、python 代码

13.旋转编码器实验

```
@ 文件名：13.旋转编码器实验.py
@ 版本：V2.0
@ 说明：旋转编码器实验 我们一起来学习一下旋转编码器传感器实验吧！！

import RPi.GPIO as GPIO
import time

RoAPin = 11      # 旋转编码器 CLK 管脚
RoBPin = 12      # 旋转编码器 DT 管脚
BtnPin = 13      # 旋转编码器 SW 管脚

globalCounter = 0   # 计数器值

flag = 0                    # 是否发生旋转标志位
Last_RoB_Status = 0         # DT 状态
Current_RoB_Status = 0    # CLK 状态

# 初始化工作
def setup():
    GPIO.setmode(GPIO.BOARD)          # 采用实际的物理管脚给 GPIO 口
    GPIO.setwarnings(False)           # 忽略 GPIO 操作注意警告
    GPIO.setup(RoAPin, GPIO.IN)       # 旋转编码器 CLK 管脚,设置为输入模式
```

```
    GPIO.setup(RoBPin, GPIO.IN)      # 旋转编码器 DT 管脚,设置为输入模式
    GPIO.setup(BtnPin, GPIO.IN, pull_up_down=GPIO.PUD_UP) # 设置 BtnPin 管脚为输入模式，上拉至高电平(3.3V)
    # 初始化中断函数，当 SW 管脚为 0，使能中断
    GPIO.add_event_detect(BtnPin, GPIO.FALLING, callback=btnISR)

# 旋转编码方向位判断函数
def rotaryDeal():
    global flag                         # 是否发生旋转标志位
    global Last_RoB_Status
    global Current_RoB_Status
    global globalCounter              # 计数器值

    Last_RoB_Status = GPIO.input(RoBPin)

    while(not GPIO.input(RoAPin)):          # 判断 CLK 管脚的电平变化来区分方向
        Current_RoB_Status = GPIO.input(RoBPin)
        flag = 1      # 发生旋转标记
    if flag == 1:    # 标记位为 1 发生了旋转
        flag = 0      # 复位标记位
        if (Last_RoB_Status == 0) and (Current_RoB_Status == 1):
            globalCounter = globalCounter + 1     # 逆时针方向，正
        if (Last_RoB_Status == 1) and (Current_RoB_Status == 0):
            globalCounter = globalCounter - 1     # 顺时针方向，负

# 中断函数，当 SW 管脚为 0，使能中断
def btnISR(chn):
    global globalCounter
    globalCounter = 0 # 给计数器赋 0
    #print ('globalCounter = %d' % globalCounter)

# 循环函数
def loop():
    global globalCounter
    tmp = 0 # 当前状态判断

    while True:
        rotaryDeal()          # 旋转编码方向位判断函数
        if tmp != globalCounter: # 判断状态值发生改变
```

```
            print ('globalCounter = %d' % globalCounter) #  打印出状态信息
            tmp = globalCounter       #    把当前状态赋值到下一个状态，避免重复打印


#  释放资源
def destroy():
    GPIO.cleanup()                    #  释放资源


#  程序入口
if __name__ == '__main__':
    setup()      #  调用初始化工作
    try:
        loop() #  调用循环函数
    except KeyboardInterrupt:   #  当按下 Ctrl+C 时，将执行 destroy()子程序。
        destroy()
```

4.14、 红外避障传感器实验

4.14.1、硬件介绍

红外避障传感器具有一对红外线发射与接收管，发射管发射出一定频率的红外线，当检测方向遇到障碍物（反射面）时，红外线反射回来被接收管接收。它常用于安装在小车上，判断前方是否有障碍物。可通过电位器设置阀值。正前方有障碍时绿灯亮起，OUT 引脚为低电平，反之为高电平。

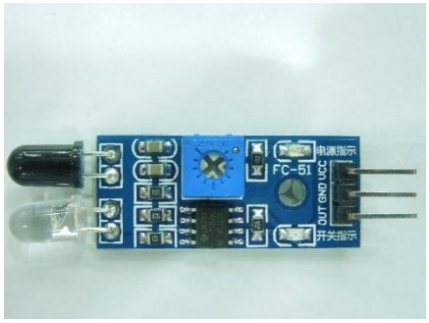


图 4.14.1 红外避障传感器

4.14.2、实验原理

该传感器模块对环境光线适应能力强，其具有一对红外线发射与接收管，发射管发射出频率的红外线，当检测方向遇到障碍物（反射面）时，红外线反射回来被接收管接收，经过比较器电路处理之后，绿色指示灯会亮起，同时信号输出接口输出数字信号（一个低电平信号），可通过电位器旋钮调节检测距离，有效距离范围 2～30cm，检测角度 35°。该传感器的探测距离可以通过电位器调节、具有干扰小、便于装配、使用方便等特点，可以广泛应用于机器人避障、避障小车、流水线计数及黑白线循迹等众多场合。

模块原理图如下：

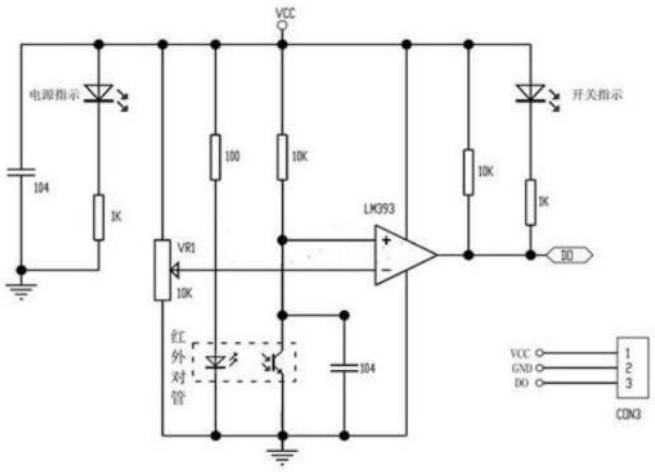


图 4.14.2 红外避障传感器模块原理图

4.14.3、实验步骤

第 1 步：搭建实验电路

GPIO Expansion Board	红外避障传感器
GPIO23	OUT
GND	GND
5V	VCC

第 2 步：进入‘Notebook’文件夹里的‘sensor 传感器模块’文件夹，找到‘14.红外避障传感器实验.ipynb’程序，双击用 JupyterLab 打开程序:

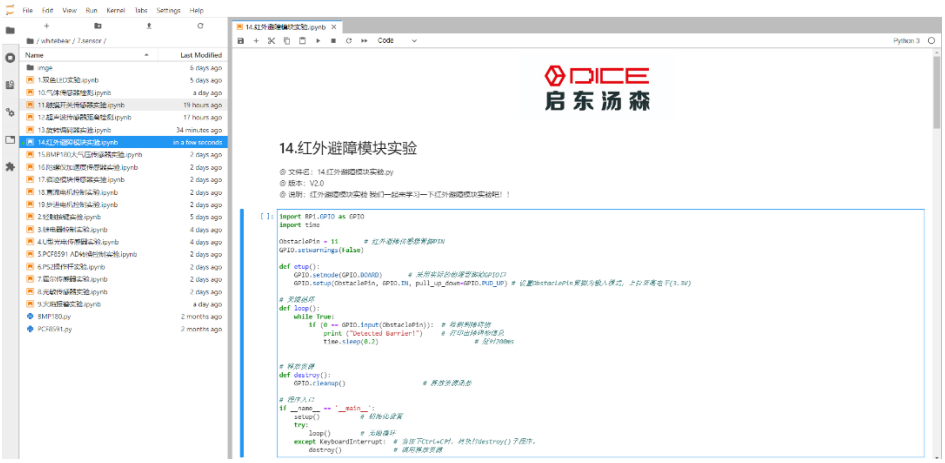


图 4.14.3 红外避障传感器实验程序截图

第 3 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 4 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：现在，如果前面有障碍物，屏幕上会打印一个字符串“Detected Barrier!”。

```
Detected Barrier!
Detected Barrier!
Detected Barrier!
Detected Barrier!
Detected Barrier!
Detected Barrier!
Detected Barrier!
Detected Barrier!
Detected Barrier!
```

图 4.14.4 红外避障传感器实验效果图 1

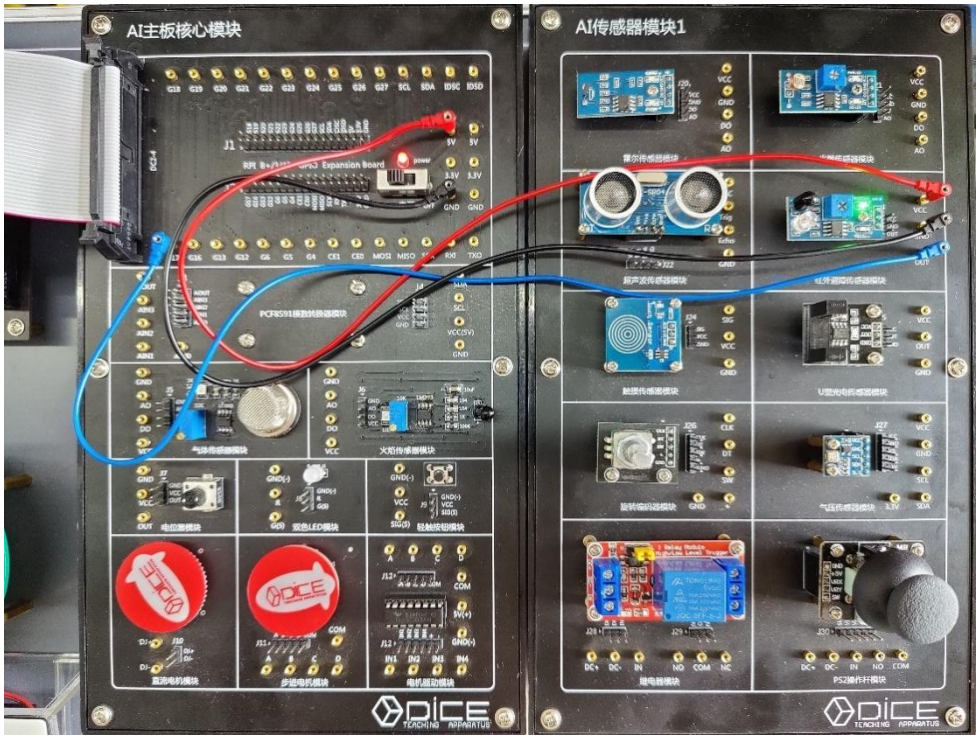


图 4.14.5 红外避障传感器实验效果图 2

4.14.4、python 代码

14.红外避障传感器实验

@ 文件名：14.红外避障传感器实验.py

@ 版本：V2.0

@ 说明：红外避障传感器实验 我们一起来学习一下红外避障传感器实验吧！！

```
import RPi.GPIO as GPIO
import time

ObstaclePin = 11            # 红外避障传感器管脚 PIN
GPIO.setwarnings(False)

def setup():
    GPIO.setmode(GPIO.BOARD)          # 采用实际的物理管脚给 GPIO 口
    GPIO.setup(ObstaclePin, GPIO.IN, pull_up_down=GPIO.PUD_UP) # 设置 ObstaclePin 管脚为输入模式，上拉至高电平(3.3V)

# 无限循环
def loop():
    while True:
        if (0 == GPIO.input(ObstaclePin)):   # 检测到障碍物
            print ("Detected Barrier!")      # 打印出障碍物信息
```

```
            time.sleep(0.2)                         # 延时 200ms



# 释放资源
def destroy():
    GPIO.cleanup()                      # 释放资源函数


# 程序入口
if __name__ == '__main__':
    setup()             # 初始化设置
    try:
        loop()          # 无限循环
    except KeyboardInterrupt:   # 当按下 Ctrl+C 时，将执行 destroy()子程序。
        destroy()               # 调用释放资源
```

4.15、 BMP180 气压传感器实验

4.15.1 硬件介绍

在测量海拔高度时，传统的做法是通过测量某一高度的大气压力，再经过换算才能得到高度数据。为了测量大气压力，就得用上气压传感器。BMP180 不仅可以实时的测量 300 至 1100hpa（海拔 9000m 至-500m）的 大气压力，还能测量-40℃至 85℃的温度。同时它还具有 IIC 总线的接口，便于单片机进行访问。另外它的使用也很方便，不需要太多的操作就可读取到气压及测量数据。

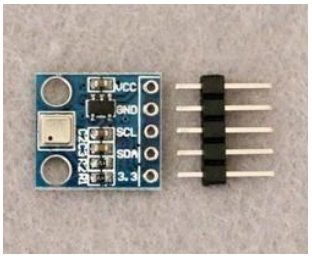


图 4.15.1 BMP180 气压传感器模块

4.15.2、实验原理

BMP180 内包含有电阻式压力传感器、AD 转换器和控制单元，其中控制单元包括了 EEPROM 和 IIC 接口。

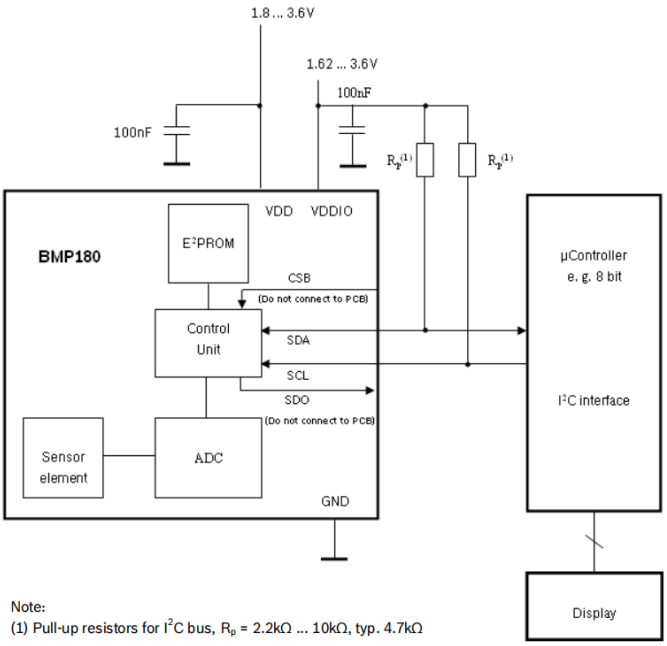


图 4.15.2 BMP180 气压传感器典型电路

读取 BMP180 时会直接传送没有经过补偿的温度值和压力值。而在 EEPROM 中则储存了 176 位单独的校准数据，这些数据将对读取的温度压力值进行补偿。176 位的 EEPROM 被划分为 11 个字，每个字 16

位，这样就包含有 11 个校准系数。每个器件模块都有自己单独的校准系数，在每一次计算温度压力数据之前，单片机就应该先读出 EEPROM 中这些校准数据，然后再开始采集数据温度和压力数据。和所有的 IIC 总线器件一样，BMP180 也有一个器件的固定地址，根据其数据手册，出厂时默认 BMP180 的从机地址为 0xEE（写入方向），或 0xEF（读出方向）。温度数据 UT 和压力数据 UP 都存储在寄存器的第 0 到 15 位之中，压力数据 UP 的精度还可扩展至 16~19 位。

下面是 11 个校准参数及地址：

	BMP180 reg adr	
Parameter	MSB	LSB
AC1	0xAA	0xAB
AC2	0xAC	0xAD
AC3	0xAE	0xAF
AC4	0xB0	0xB1
AC5	0xB2	0xB3
AC6	0xB4	0xB5
B1	0xB6	0xB7
B2	0xB8	0xB9
MB	0xBA	0xBB
MC	0xBC	0xBD
MD	0xBE	0xBF

图 4.15.3 校准参数及地址

读取温度和气压的寄存器地址及等待时间（osrs：采样分辨率）：

Measurement	Control register value (register address 0xF4)	Max. conversion time [ms]
Temperature	0x2E	4.5
Pressure (osrs = 0)	0x34	4.5
Pressure (osrs = 1)	0x74	7.5
Pressure (osrs = 2)	0xB4	13.5
Pressure (osrs = 3)	0xF4	25.5

图 4.15.4 寄存器地址及等待时间

从图中可以看出，要获得温度数据，必须先向控制寄存器（地址 0xF4）写 0x2E，然后等待至少 4.5ms，才可以从地址 0xF6 和 0xF7 读取十六位的温度数据。同样，要获得气压数据，必须先向控制寄存器（地址 0xF4）写 0x34（0x74、0xB4、0xF4），然后等待至少 4.5ms，才可以从地址 0xF6 和 0xF7 读取 16 位的气压数据。

读取气压大致流程为：开始 →读取 EEPROM 校准值 → 读取温度值 → 读取气压值 → 校准。查看传感器手册可知，流程如下：

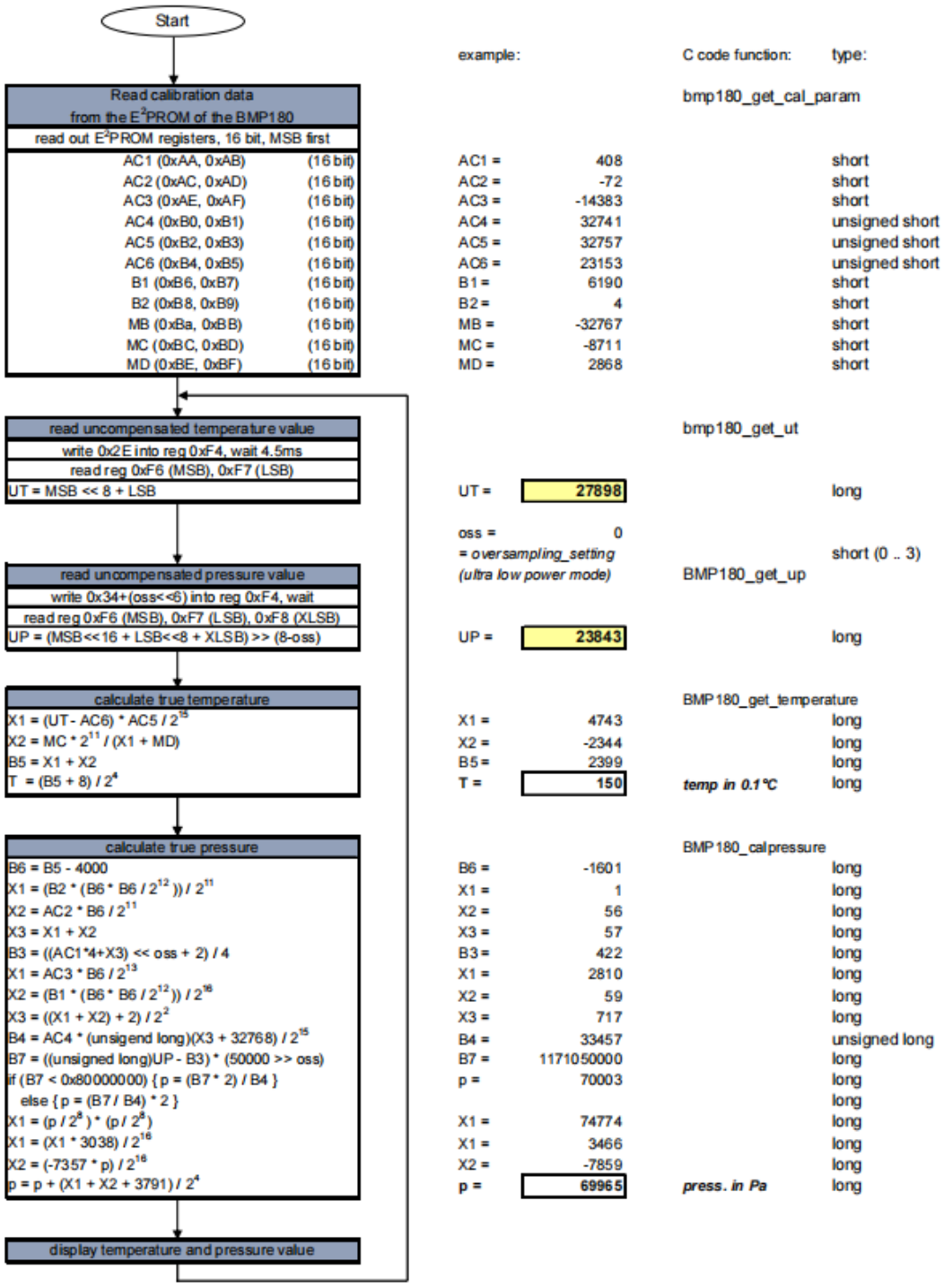


图 4.15.5 传感器开发流程

海拔计算方法：

$$altitude = 44330 * \left(1 - \left(\frac{p}{p_0}\right)^{\frac{1}{5.255}}\right)$$

图 4.15.6 海拔计算方式

注：P：大气压强　　P0：水平面参考压强（例：1013.25hPa）

模块原理图如下：

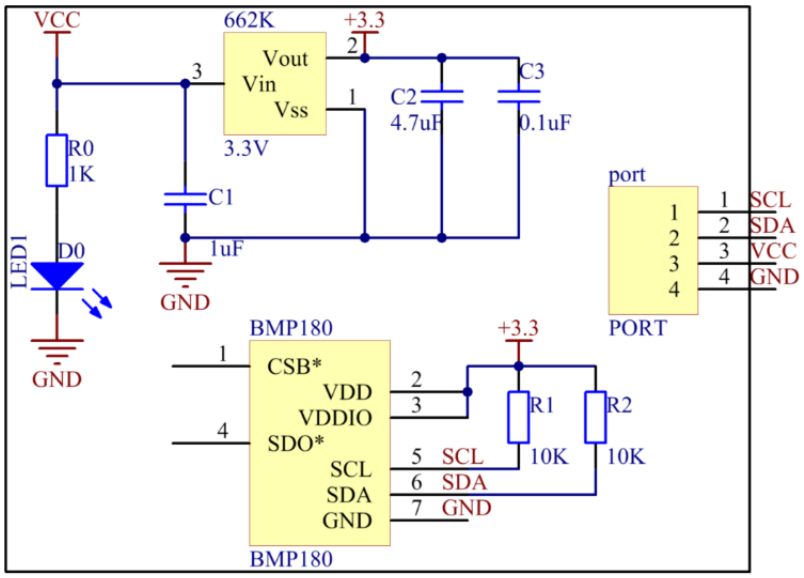


图 4.15.7 模块原理图

4.15.3、实验步骤

第 1 步：搭建实验电路

GPIO Expansion Board	BMP180 气压传感器
SCL	SCL
SDA	SDA
5V	VCC
GND	GND
3.3V	3.3V

第 2 步：进入‘Notebook’文件夹里的‘sensor 传感器模块’文件夹，找到‘15.BMP180 大气压传感器实验.ipynb’程序，双击用 JupyterLab 打开程序：

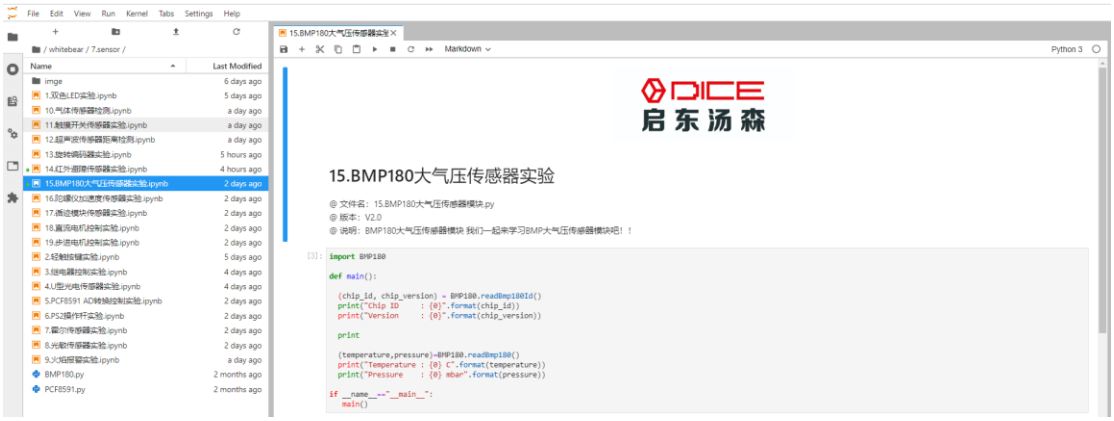


图 4.15.8 BMP180 大气压传感器实验程序截图

第 3 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 4 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：现在就可以看到屏幕上会打印出温度和压力值。

```
Chip ID     : 85
Version     : 2
Temperature : 30.0 C
Pressure    : 1003.76 mbar
```

图 4.15.9 BMP180 大气压传感器实验效果图 1

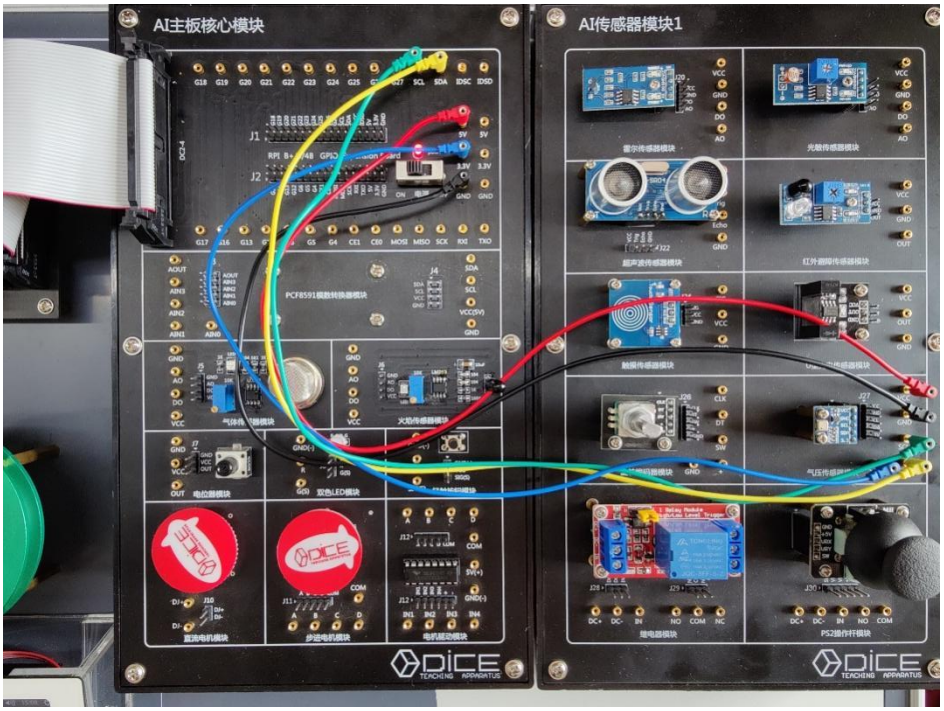


图 4.15.10 BMP180 大气压传感器实验效果图 2

4.15.4、python 代码

15.BMP180 大气压传感器实验

```
@ 文件名：15.BMP180 大气压传感器模块.py
@ 版本：V2.0
@ 说明：BMP180 大气压传感器模块 我们一起来学习 BMP 大气压传感器模块吧！！

import BMP180

def main():

    (chip_id, chip_version) = BMP180.readBmp180Id()
    print("Chip ID         : {0}".format(chip_id))
    print("Version         : {0}".format(chip_version))

    print

    (temperature,pressure)=BMP180.readBmp180()
    print("Temperature : {0} C".format(temperature))
    print("Pressure        : {0} mbar".format(pressure))

if __name__=="__main__":
    main()
```

4.16、 直流电机模块实验

4.16.1、硬件介绍

直流电机（direct current machine）是指能将直流电能转换成机械能（直流电动机）或将机械能转换成直流电能（直流发电机）的旋转电机。它是能实现直流电能和机械能互相转换的电机。常用作风扇、小型水泵、电动自行车等。

图 4.16.1 直流电机

4.16.2、实验原理

由于电机的驱动电流较大，Jetson Nano 主板或其他 MCU 不能直接驱动，一般都是使用 ULN2003 达林顿阵列驱动。ULN2003 是最常见的电机驱动器 IC 之一，由 7 对达林顿晶体管对的阵列组成，每对能够驱动高达 500mA 和 50V 的负载。其与开发板及电机连接。

图 4.16.2 ULN2003 驱动芯片

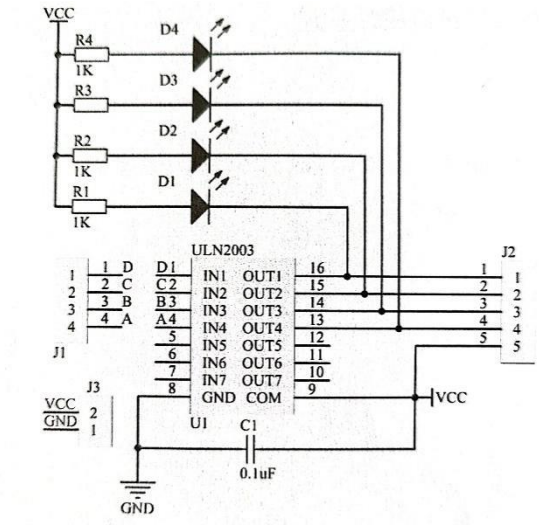


图 4.16.3 电机驱动模块原理图

4.16.3、实验步骤

第 1 步：搭建实验电路

GPIO Expansion Board	电机驱动模块	直流电机模块
GPIO23	IN4	
5V	VCC	
GND	GND	
	D	DJ+
	COM	DJ-

第 2 步：进入‘Notebook’文件夹里的‘sensor 传感器模块’文件夹，找到‘16.直流电机控制实验.ipynb’程序，双击用 JupyterLab 打开程序：

图 4.16.4 直流电机控制实验程序截图

第 3 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 4 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：现在我们可以看到屏幕上轮流开始打印“DJ START”和“DJ STOP”,当打印“DJ START”时电机开始转动，当打印“DJ STOP”时电机停止转动。

```
DJ START
DJ STOP
DJ START
DJ STOP
DJ START
DJ STOP
DJ START
DJ STOP
DJ START
DJ STOP
```

图 4.16.5 直流电机控制实验效果图 1

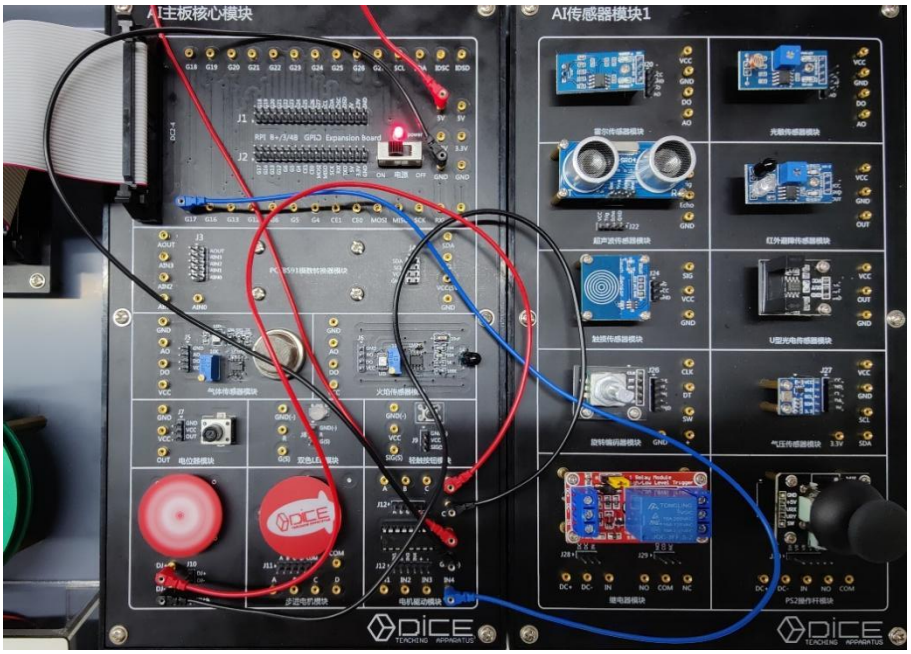


图 4.16.6 直流电机控制实验效果图 2

4.16.4、python 代码

16.直流电机控制实验

@ 文件名：16.直流电机控制实验.py

@ 版本：V2.0

@ 说明：直流电机控制实验 我们一起来学习直流电机的驱动方式吧！！

```
import RPi.GPIO as GPIO
import time
```

```
# 设置风扇管脚 PIN
MotorPin1     = 17



def setup():
    # 将 GPIO 模式设置为 BCM 编号
    GPIO.setmode(GPIO.BCM)
    GPIO.setwarnings(False)                 # 忽略警告
    GPIO.setup(MotorPin1, GPIO.OUT)     # 风扇设置为输出模式

# 风扇电机控制函数
def motor(direction):
    # 开启电机
    if direction == 1:
        GPIO.output(MotorPin1, GPIO.HIGH)
        print ("DJ START")

    # 关闭电机
    if direction == 0:
        # 关闭电机
        GPIO.output(MotorPin1, GPIO.LOW)
        print ("DJ STOP")

def main():
    fs_directions = {'Open': 1, 'STOP': 0} # 定义开和关
    while True:
        # 电机开
        motor(fs_directions['Open'])
        time.sleep(5)
        # 电机关
        motor(fs_directions['STOP'])
        time.sleep(5)

# 释放资源
def destroy():
    # 关闭电机
    GPIO.output(MotorPin1, GPIO.LOW)
    # 释放资源
    GPIO.cleanup()
```

```
# 程序入口
if __name__ == '__main__':
    setup()
    try:
        main()
    except KeyboardInterrupt:    # 当按下 Ctrl+C 时，将执行 destroy()子程序。
        destroy()    # 释放资源
```

4.17、 步进电机驱动模块实验

4.17.1、硬件介绍

步进电机是用于位置控制的出色电机。它们是一种特殊类型的无刷电机，它将完整的旋转分为多个相等的“步长”。它们通常在台式打印机，3D 打印机，CNC 铣床以及其他需要精确定位控制的设备中找到。

图 4.17.1 四相五线步进电机

有两种类型的步进电机，单极性步进电机和双极性步进电机，并且知道您正在使用哪种类型非常重要。在这个实验中，我们将使用单极步进电机。

Jetson Nano 主板或其他 MCU 不能直接驱动步进电机。驱动电路是必要的，所以我们使用上个实验中使用过的 ULN2003 达林顿阵列驱动来驱动步进电机。

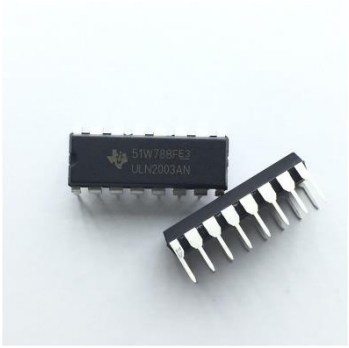


图 4.17.2 ULN2003 驱动芯片

4.17.2、实验原理

这次我们使用的步进电机时四相五线步进电机，采用单极性直流电源供电。只要给步进电机的各项绕组合适的脉冲，就能使步进电机转动。下图就是四相五线步进电机的内部结构示意图：

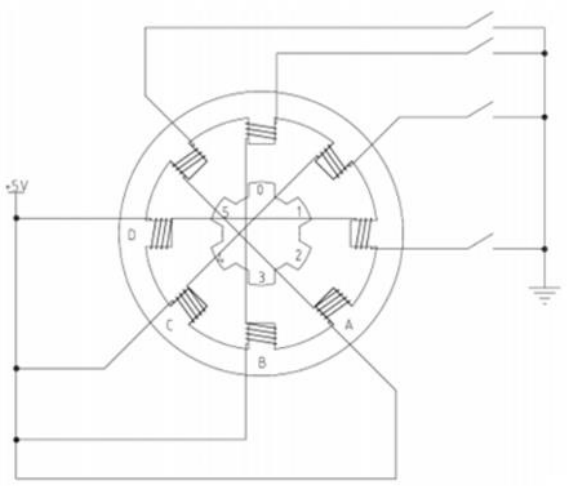


图 4.17.3 四相五线步进电机内部结构示意图

四相步进电机的工作方式：

- 单相四拍工作方式，其电机控制绕组 A、B、C、D 相的正转通电顺序为：A→B→C→D→A；反转通电顺序为：A→D→C→B→A。

- 四相八拍工作方式，正转的绕组通电顺序为：A→AB→B→BC→C→CD→D→DA→A；反向的通电顺序为：A→AD→D→DC→C→CB→B→BA→A。

- 双四拍工作方式，正转的绕组通电顺序为：AB→BC→CD→DA→AB；反向的通电顺序为：AB→AD→DC→CB→BA。

每给步进电机一个脉冲，步进电机都会运行一步，可以让他们准确定位，给不同的脉冲将极大地影响电机的性能，脉冲序列会影响电机的旋转方向，脉冲的频率决定电机运转速度，脉冲数决定电机旋转圈数。步进电机在要求快速启停，精确定位的场合做为执行部件，得到了广泛采用。

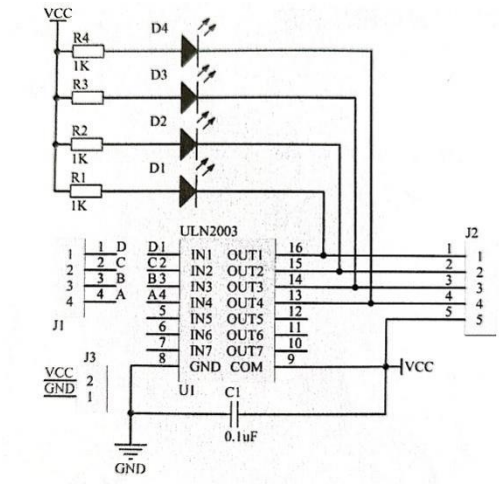


图 4.17.4 电机驱动模块原理图

4.17.3、实验步骤

第 1 步：搭建实验电路

GPIO Expansion Board	电机驱动模块	步进电机模块
GPIO18	IN1	
GPIO23	IN2	
GPIO24	IN3	
GPIO25	IN4	
5V	VCC	
GND	GND	
	COM	COM
	A	A
	B	B
	C	C
	D	D

第 2 步：进入‘Notebook’文件夹里的‘sensor 传感器模块’文件夹，找到‘17.步进电机控制实验.ipynb’程序，双击用 JupyterLab 打开程序:

图 4.17.5 步进电机控制实验程序截图

第 3 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 4 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：输入‘a’后回车，步进电机顺时针旋转，输入‘c’回车后，步进电机逆时针旋转！

```
select motor direction a=anticlockwise, c=clockwise:  a
motor running anti-clockwise
```

图 4.17.6 步进电机控制实验效果图 1

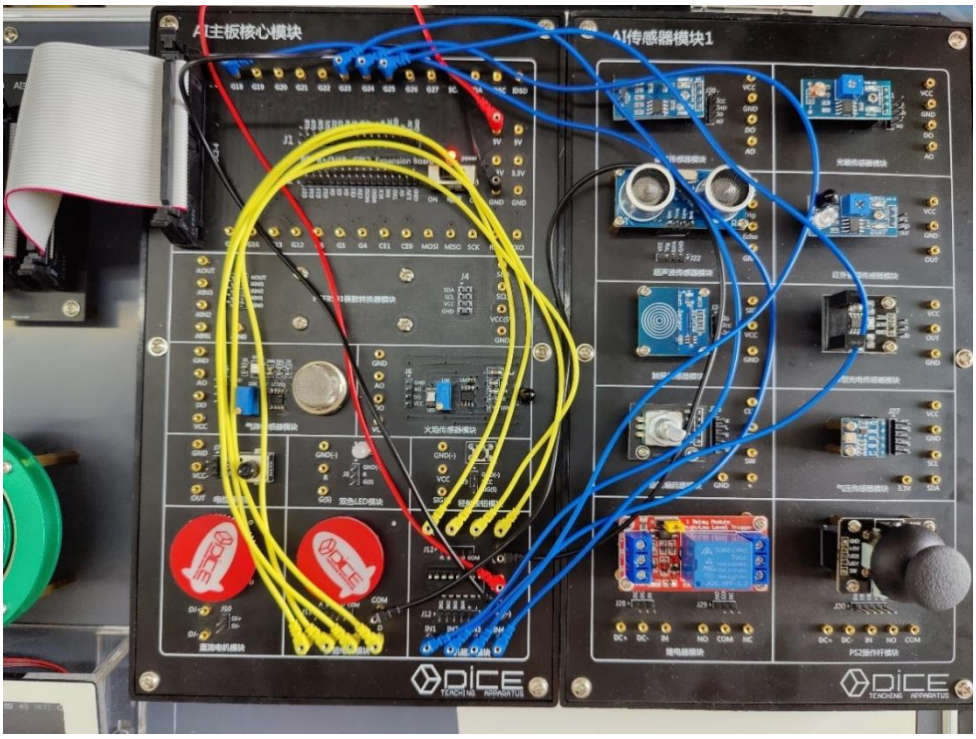


图 4.17.7 步进电机控制实验效果图 2

4.17.4、python 代码

```
17.步进电机控制实验
@ 文件名：17.步进电机控制实验.py
@ 版本：V2.0
@ 说明：步进电机控制实验 我们一起来学习步进电机的驱动方式吧！！
import RPi.GPIO as GPIO
from time import sleep


motorPin = (18,23,24,25)        # 步进电机管脚 PIN
rolePerMinute =15                 # 每分钟转数
```

```
stepsPerRevolution = 2048       # 每转一圈的步数
stepSpeed = (60/rolePerMinute)/stepsPerRevolution    # 每一步所用的时间

# 初始化设置
def setup():
    GPIO.setmode(GPIO.BCM)    # 将 GPIO 模式设置为 BCM 编号
    GPIO.setwarnings(False) # 忽略警告
    for i in motorPin:
        GPIO.setup(i, GPIO.OUT) # 设置步进电机的所有管脚为输出模式

# 步进电机旋转
def rotary(clb_direction):
    if(clb_direction == 'a'):         # 逆时针旋转
        for j in range(4):
            for i in range(4):
                GPIO.output(motorPin[i],0x99>>j & (0x08>>i))
            sleep(stepSpeed)

    elif(clb_direction == 'c'):       # 顺时针旋转
        for j in range(4):
            for i in range(4):
                GPIO.output(motorPin[i],0x99<<j & (0x80>>i))
            sleep(stepSpeed)

# 循环函数
def loop():
    while True:
        clb_direction = input('select motor direction a=anticlockwise, c=clockwise: ')
        if(clb_direction == 'c'):
            print('motor running clockwise\n')            # 顺时针旋转
            break
        elif(clb_direction == 'a'):
            print('motor running anti-clockwise\n')    # 逆时针旋转
            break
        else:
            print('input error, please try again!') # 输入错误，再次输入
    while True:
        rotary(clb_direction)            # 让步进电机旋转
```

```
# 释放资源
def destroy():
    for i in motorPin:
        GPIO.output(i, GPIO.LOW) # 设置步进电机的所有管脚为输出模式
    GPIO.cleanup() # 释放资源


# 程序入口
if __name__ == '__main__':
    setup()    # 初始化设置函数
    try:
        loop()  # 循环函数
    except KeyboardInterrupt:    # 当按下 Ctrl+C 时，将执行 destroy()子程序。
        destroy()  # 资源释放
```

4.18、 循迹传感器实验

4.18.1、硬件介绍

红外循迹传感器本质是一个红外 LED 和一个红外检测器。具体的检测原理是跑道黑线吸收光，红外传感器接收不到返回的数据，经比较器后输出高电平。场地区域为白色反射所有光，传感器发射的红外光遇场地反射，接收管接收到反射光，经比较器后输出低电平。

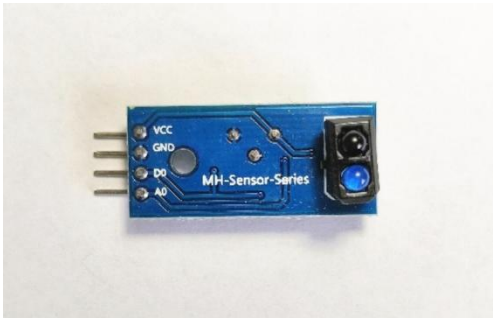


图 4.18.1 循迹传感器

4.18.2、实验原理

红外循迹传感器本质上是一个红外 LED 和一个红外检测器。它通过向表面向下发射一束红外光来工作。当红外线发射器向纸上发射光线时，如果光线照射在白色表面上，它们将被接收器反射并接收，并且引脚 DO 将输出低电平；如果光线遇到黑线，它们将被吸收，因此接收器将一无所获，并且引脚 DO 将输出高电平。当它接近白色 / 黑色物体时，它就像一个简单的开关。您可以使用电位计调节灵敏度。它使用 LM393 比较器芯片和跟踪传感器来获得干净的输出。灵敏度：距白皮书约 1 英寸（2.5 厘米），距人的手指约 1 厘米。

该模块的原理图如下所示：

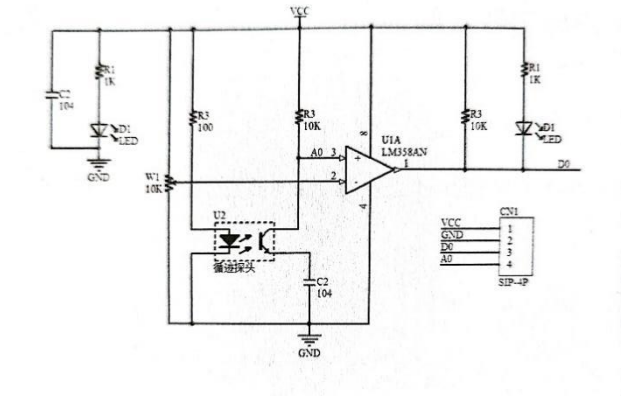


图 4.18.2 循迹传感器模块原理图

4.18.3、实验步骤

第 1 步：搭建实验电路

GPIO Expansion Board	循迹传感器
GPIO23	D0
5V	VCC
GND	GND

第 2 步：进入'Notebook'文件夹里的'sensor 传感器模块'文件夹，找到'18.循迹模块传感器实验.ipynb'程序，双击用 JupyterLab 打开程序：

图 4.18.3 循迹模块传感器实验程序截图

第 3 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 4 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：当循迹传感器遇到白线时，将在屏幕上打印字符串“White line is detected”。当循迹传感器遇黑线时，将在屏幕上打印字符串“Black line is detected”。

```
Black line is detected
Black line is detected
Black line is detected
Black line is detected
Black line is detected
Black line is detected
Black line is detected
White line is detected
White line is detected
White line is detected
White line is detected
White line is detected
White line is detected
White line is detected
```

图 4.18.4 循迹传感器实验效果图 1

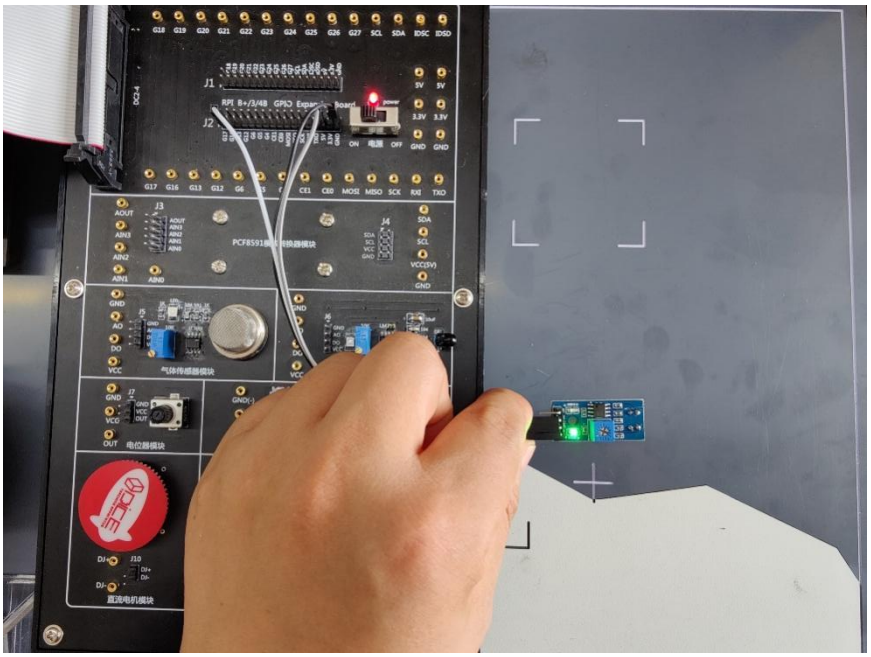


图 4.18.5 循迹传感器实验效果图 2

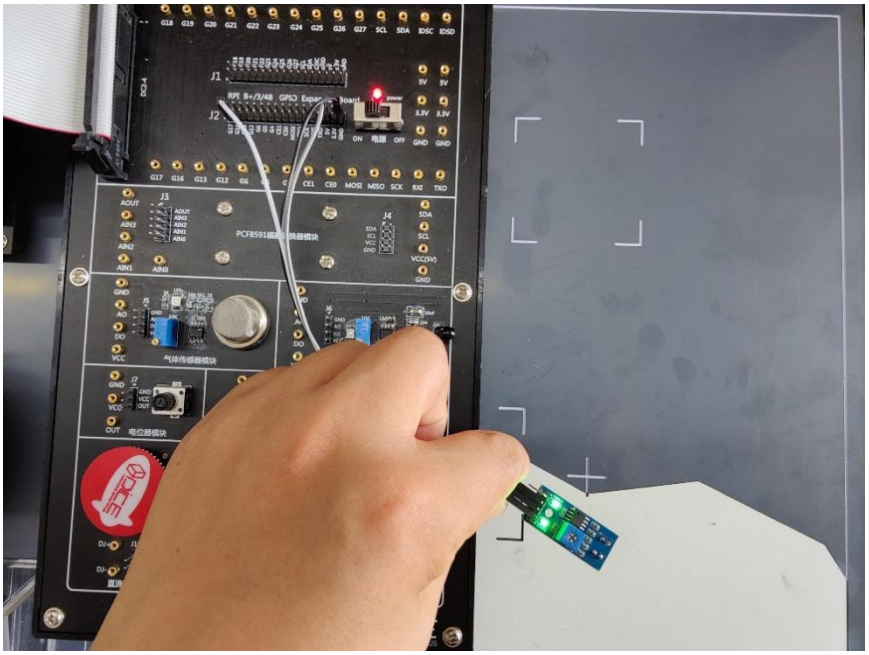


图 4.18.6 循迹传感器实验效果图 3

4.18.4、python 代码

18.循迹模块传感器实验

@ 文件名：18.循迹模块传感器实验.py

@ 版本：V2.0

@ 说明：循迹模块传感器实验 我们一起来学习循迹模块的驱动方式吧！！

```
import RPi.GPIO as GPIO
import time
```

```
TrackPin = 11 #  循迹传感器 PIN 管脚


#    初始化设置函数
def setup():
    GPIO.setmode(GPIO.BOARD)          #  采用实际的物理管脚给 GPIO 口
    GPIO.setwarnings(False)           #  忽略警告
     #  设置 TrackPin 管脚为输入模式，上拉至高电平(3.3V)
    GPIO.setup(TrackPin, GPIO.IN, pull_up_down=GPIO.PUD_UP)


#  循环函数
def loop():
    while True:
        if GPIO.input(TrackPin) == GPIO.LOW:    #  检测到白色线
            print ('White line is detected')
        else:
            print ('Black line is detected') #  检测到黑色线


        time.sleep(0.2) #  延时 200ms

#  释放资源
def destroy():
    GPIO.cleanup()                          #  释放资源

#  程序入口
if __name__ == '__main__':
    setup()          #  调用初始化程序
    try:
        loop()      #  调用循环函数
    except KeyboardInterrupt:    #    当按下 Ctrl+C 时，将执行 destroy()子程序。
        destroy()
```

4.19、 MPU6050 陀螺仪加速度传感器

4.19.1、硬件介绍

MPU6050 是世界上第一款也是唯一一款专为智能手机、平板电脑和可穿戴传感器的低功耗、低成本和高性能要求而设计的 6 轴运动跟踪设备。它集成了 3 轴 MEMS 陀螺仪，3 轴 MEMS 加速度计，以及一个可扩展的数字运动处理器 DMP（ DigitalMotion Processor），可用 I2C 接口连接一个第三方的数字传感器，比如磁力计。扩展之后就可以通过其 I2C 或 SPI 接口输出一个 9 轴的信号（ SPI 接口仅在 MPU-6000 可用）。 MPU-60X0 也可以通过其 I2C 接口连接非惯性的数字传感器，比如压力传感器。

图 4.19.1 MPU6050 陀螺仪加速度传感器

4.19.2 实验原理

陀螺仪的原理就是，一个旋转物体的旋转轴所指的方向在不受外力影响时，是不会改变的。人们根据这个道理，用它来保持方向。然后用多种方法读取轴所指示的方向，并自动将数据信号传给控制系统。我们骑自行车其实也是利用了这个原理。轮子转得越快越不容易倒，因为车轴有一股保持水平的力量。现代陀螺仪可以精确地确定运动物体的方位的仪器，它在现代航空，航海，航天和国防工业中广泛使用的一种惯性导航仪器。传统的惯性陀螺仪主要部分有机械式的陀螺仪，而机械式的陀螺仪对工艺结构的要求很高。70 年代提出了现代光纤陀螺仪的基本设想，到八十年代以后，光纤陀螺仪就得到了非常迅速的发展，激光谐振陀螺仪也有了很大的发展。光纤陀螺仪具有结构紧凑，灵敏度高，工作可靠。光纤陀螺仪在很多的领域已经完全取代了机械式的传统的陀螺仪，成为现代导航仪器中的关键部件。光纤陀螺仪同时发展的除了环式激光陀螺仪外。

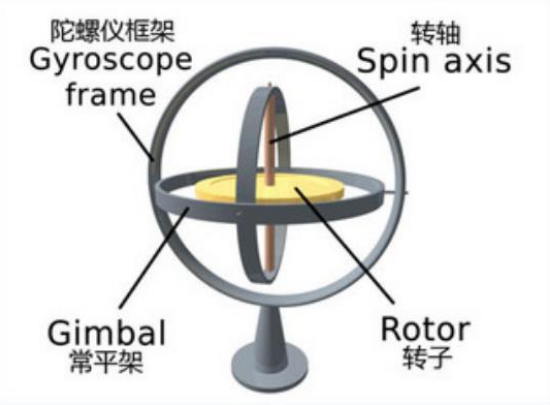


图 4.19.2 陀螺仪

MPU-60X0 由三个独立的振动 MEMS 速率陀螺仪组成，可检测旋转角度 X 轴，Y 轴和 Z 轴。 当陀螺仪围绕任何感应轴旋转时，科里奥利效应就会产生电容式传感器检测到的振动。 所得到的信号被放大，解调和滤波产生与角速度成比例的电压。 该电压使用单独的片内数字化 16 位模数转换器（ADC）对每个轴进行采样。 陀螺仪传感器可以全面范围的被数字编程为每秒±250，±500，±1000 或±2000 度（dps）。 ADC 样本速率可以从每秒 8,000 个采样点编程到每秒 3.9 个采样点，并且可由用户选择低通滤波器可实现广泛的截止频率。

MPU6050 由具有微机电系统（MEMS)技术的三轴陀螺仪组成。如下图所示，用于检测沿 X,Y,Z 轴的旋转速度。旋转的正向可用右手螺旋定则判断:

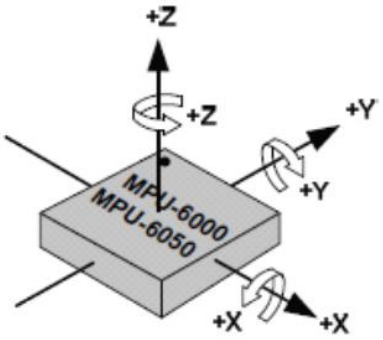


图 4.19.3 三轴陀螺仪

加速度传感器是一种能够测量加速度的传感器。通常由质量块、阻尼器、弹性元件、敏感元件和适调电路等部分组成。传感器在加速过程中，通过对质量块所受惯性力的测量，利用牛顿第二定律获得加速度值。根据传感器敏感元件的不同，常见的加速度传感器包括电容式、电感式、应变式、压阻式、压电式等。

MPU-60X0 的 3 轴加速度计为每个轴使用单独的检测质量。 加速沿着一条特定轴在相应的检测质量上引起位移，并且电容式传感器检测到该位移位移有差别。 MPU-60X0 的架构降低了加速度计的敏感度制造变化以及热漂移。 当设备放置在平坦的表面上时，将进行测量在 X 和 Y 轴上为 0g，在 Z 轴上为+1g。 加速度计的比例因子在工厂进行校准并且在名义上与电源电压无关。 每个传感器都有一个专用的 sigma-delta ADC 来提供数字输出。 数字输出的满量程范围可以调整到±2g，±4g，±8g 或±16g。

其实说简单点，在 mpu6050 中我们用陀螺仪传感器测角度，用加速度传感器测加速度。

嵌入式数字运动处理器（DMP）位于 MPU-60X0 内部，可从主机处理器中卸载运动处理算法的运算。 DMP 从加速度计，陀螺仪以及其他第三方传感器（如磁力计）获取数据，并处理数据。结果数据可以从 DMP 的寄存器中读取，或者可以在 FIFO 中缓冲。 DMP 可以访问其中的一个 MPU 的外部引脚，可用于产生中断。

DMP 的目的是卸载主机处理器的时序要求和处理能力。通常，运动处理算法应该以高速运行，通常在 200Hz 左右，以提供低延迟的精确结果。即使应用程序以更低的速率更新，这也是必需的。例如，一个低功率的用户界面可能会以 5Hz 的速度更新，但运动处理仍然应该以 200Hz 运行。 DMP 可以作为一种工具使用，以最大限度地降低功耗，简化定时，简化软件架构，并在主机处理器上节省宝贵的 MIPS，以便在应用中使用。

4.19.3、实验步骤

第 1 步：搭建实验电路

GPIO Expansion Board	MPU6050 陀螺仪模块
SCL	SCL
SDA	SDA
GND	GND
5V	VCC

第 2 步：进入‘Notebook’文件夹里的‘sensor 传感器模块’文件夹，找到‘19.陀螺仪加速度传感器实验.ipynb’程序，双击用 JupyterLab 打开程序：

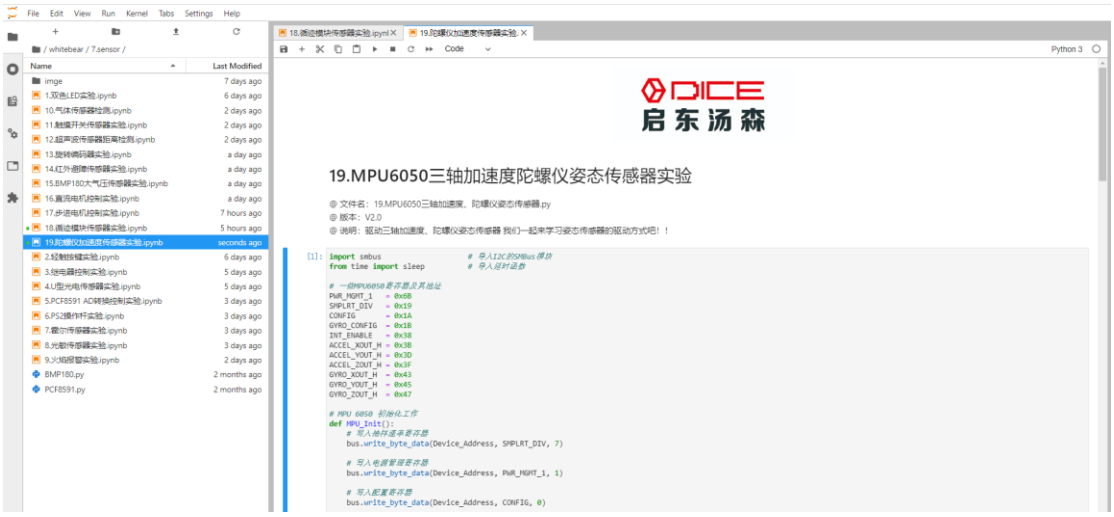


图 4.18.5 陀螺仪加速度传感器程序截图

第 3 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 4 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：现在您可以看到不断在屏幕上打印的 MPU6050 读取的加速度传感器，陀螺仪和 XY 轴旋转的值。

```
 Reading Data of Gyroscope and Accelerometer
Gx=0.00 °/s     Gy=0.92 °/s     Gz=-3.73 °/s    Ax=0.00 g       Ay=0.00 g       Az=0.00 g
Gx=0.15 °/s     Gy=-0.35 °/s    Gz=0.34 °/s     Ax=-0.19 g      Ay=-0.09 g      Az=1.00 g
Gx=2.45 °/s     Gy=-1.58 °/s    Gz=-0.29 °/s    Ax=0.42 g       Ay=0.14 g       Az=0.91 g
Gx=3.27 °/s     Gy=-0.24 °/s    Gz=-1.47 °/s    Ax=0.04 g       Ay=0.96 g       Az=0.12 g
Gx=-14.88 °/s   Gy=2.20 °/s     Gz=3.24 °/s     Ax=0.06 g       Ay=0.25 g       Az=0.96 g
Gx=-4.35 °/s    Gy=1.05 °/s     Gz=0.85 °/s     Ax=0.04 g       Ay=-0.53 g      Az=0.83 g
Gx=-0.53 °/s    Gy=-1.14 °/s    Gz=-0.60 °/s    Ax=0.19 g       Ay=0.08 g       Az=0.94 g
Gx=-0.68 °/s    Gy=0.03 °/s     Gz=0.05 °/s     Ax=0.13 g       Ay=0.01 g       Az=1.00 g
```

图 4.19.6 陀螺仪加速度传感器实验效果图 1

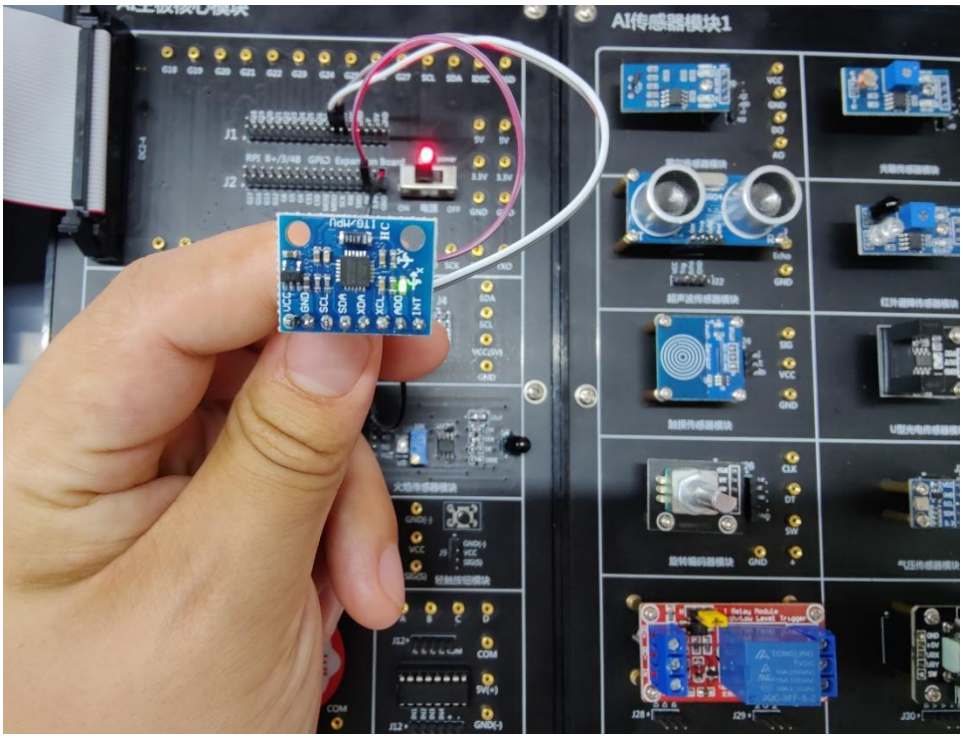


图 4.19.7 陀螺仪加速度传感器实验效果图 2

4.19.4、python 代码

19.MPU6050 三轴加速度陀螺仪姿态传感器实验

@ 文件名：19.MPU6050 三轴加速度、陀螺仪姿态传感器.py

@ 版本：V2.0

@ 说明：驱动三轴加速度、陀螺仪姿态传感器 我们一起来学习姿态传感器的驱动方式吧！！

```
import smbus                    # 导入 I2C 的 SMBus 模块
from time import sleep          # 导入延时函数

# 一些 MPU6050 寄存器及其地址
PWR_MGMT_1   = 0x6B
SMPLRT_DIV   = 0x19
CONFIG       = 0x1A
GYRO_CONFIG  = 0x1B
INT_ENABLE   = 0x38
ACCEL_XOUT_H = 0x3B
ACCEL_YOUT_H = 0x3D
ACCEL_ZOUT_H = 0x3F
GYRO_XOUT_H  = 0x43
GYRO_YOUT_H  = 0x45
GYRO_ZOUT_H  = 0x47
```

```
# MPU 6050  初始化工作
def MPU_Init():
    #  写入抽样速率寄存器
    bus.write_byte_data(Device_Address, SMPLRT_DIV, 7)

    #  写入电源管理寄存器
    bus.write_byte_data(Device_Address, PWR_MGMT_1, 1)

    #  写入配置寄存器
    bus.write_byte_data(Device_Address, CONFIG, 0)

    #  写入陀螺配置寄存器
    bus.write_byte_data(Device_Address, GYRO_CONFIG, 24)

    #  写中断使能寄存器
    bus.write_byte_data(Device_Address, INT_ENABLE, 1)

    #  读取 MPU6050 数据寄存器
def read_raw_data(addr):
    #  加速度值和陀螺值为 16 位
        high = bus.read_byte_data(Device_Address, addr)
        low =    bus.read_byte_data(Device_Address, addr+1)

        #  连接更高和更低的值
        value = ((high << 8) | low)

        #  从 mpu6050 获取有符号值
        if(value > 32768):
                value = value - 65536
        return value


bus = smbus.SMBus(1)       #  或 bus = smbus.SMBus(0)用于较老的版本板
Device_Address = 0x68      # MPU6050 设备地址

MPU_Init()                 #  初始化 MPU6050

#  打印出提示信息
print (" Reading Data of Gyroscope and Accelerometer")
```

```
# 无限循环
while True:
    # 读取加速度计原始值
    acc_x = read_raw_data(ACCEL_XOUT_H)
    acc_y = read_raw_data(ACCEL_YOUT_H)
    acc_z = read_raw_data(ACCEL_ZOUT_H)

    # 读陀螺仪原始值
    gyro_x = read_raw_data(GYRO_XOUT_H)
    gyro_y = read_raw_data(GYRO_YOUT_H)
    gyro_z = read_raw_data(GYRO_ZOUT_H)

    # 全刻度范围+/- 250 度/℃，根据灵敏度刻度系数
    Ax = acc_x/16384.0
    Ay = acc_y/16384.0
    Az = acc_z/16384.0

    Gx = gyro_x/131.0
    Gy = gyro_y/131.0
    Gz = gyro_z/131.0

    # 打印出 MPU 相关信息
    print ("Gx=%.2f" %Gx, u'\u00b0'+ "/s", "\tGy=%.2f" %Gy, u'\u00b0'+ "/s", "\tGz=%.2f" %Gz, u'\u00b0'+
"/s", "\tAx=%.2f g" %Ax, "\tAy=%.2f g" %Ay, "\tAz=%.2f g" %Az)
    sleep(1)        # 延时 1s
```

5、机械臂基础控制和实践

机械臂基础控制实验操作说明:

本章节的机械臂实验由 Jetson Nano 的 GPIO 直接控制，主要由 6 自由度机械臂和机械臂主板完成。进行实验前需把机械臂主板的电源开至“ON”的位置。机械臂数码管显示的是供电电压，给机械臂舵机供电。

5.1、蜂鸣器控制实验

5.1.1、硬件说明

蜂鸣器是一种一体化结构的电子讯响器，采用直流电压供电，广泛应用于计算机、打印机、复印机、报警器、电子玩具、汽车电子设备、电话机、定时器等电子产品中作发声器件。蜂鸣器主要分为压电式蜂鸣器和电磁式蜂鸣器两种类型。蜂鸣器在电路中用字母“H”或“HA”（旧标准用“FM”、“ZZG”、“LB”、“JD”等）表示。

在这个实验箱上把蜂鸣器在机械臂主板上，可以作为机械臂运行时的报警装置，这样使用起来比较方便。由于已经集成在机械臂主板上，所以不需要搭建电路，直接使用即可。

图 5.2.1 蜂鸣器图片

5.1.2、实验原理

有源蜂鸣器，也称压电式压蜂鸣器，主要由多谐振荡器、压电蜂鸣片、阻抗匹配器及共鸣箱、外壳等组成。其中最重要的就是它的压电蜂鸣片，通常这是一种压电陶瓷，能进行电能与机械能的转换。发声过程为：外部提供恒定直流电压，提供能量给多谐振荡器，多指振荡器起振，并提供变化的电压和频率给压电陶瓷，压电陶瓷将电能转化为机械能，也就发出了声音。

无源蜂鸣器，也称电磁式蜂鸣器，主要由永磁体，线圈，振荡片构成。发声过程为：外部按一定频率提供驱动一个振荡信号（一定占空比的方波，该信号作用于线圈，产生的磁声与永磁体共同作用，使一片金属片（振荡片）发生振动，从而发出声音。

5.1.3、实验步骤

首次使用时，我们需要按照前面所学 JupyterLab 的内容，在电脑端通过网页登陆，Jetson Nano 的 IP 地址加上 8888（IP:8888）如下界面：

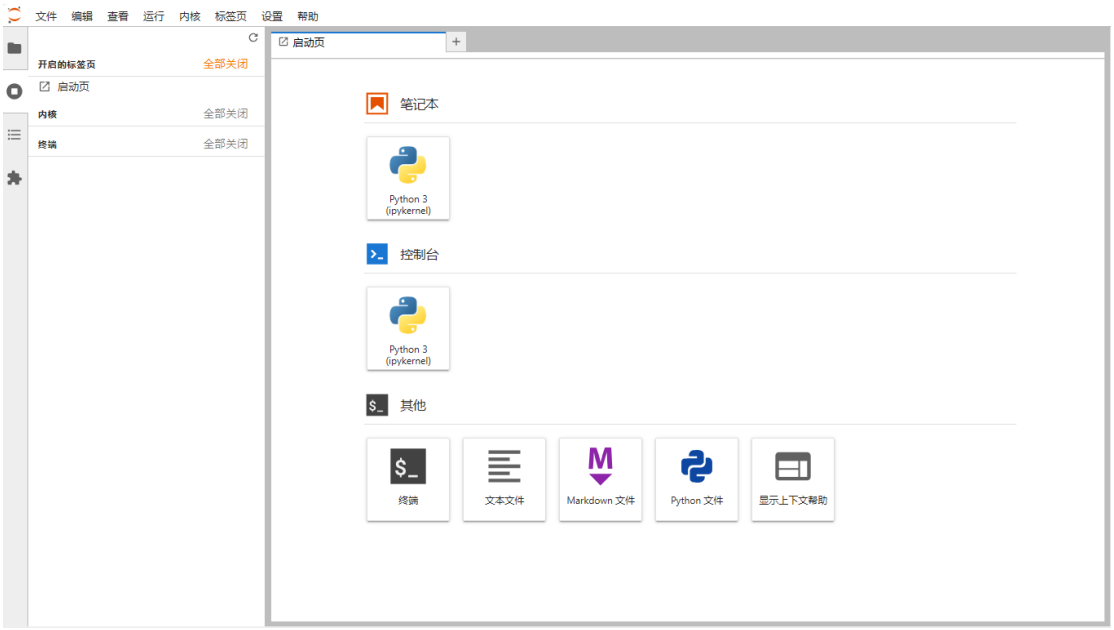


图 5.1.1 JupyterLab 界面截图

第 1 步：进入‘Notebook 文件夹里的‘基础实验’文件夹，找到‘1.蜂鸣器实验.ipynb’程序，双击用 JupyterLab 打开程序:

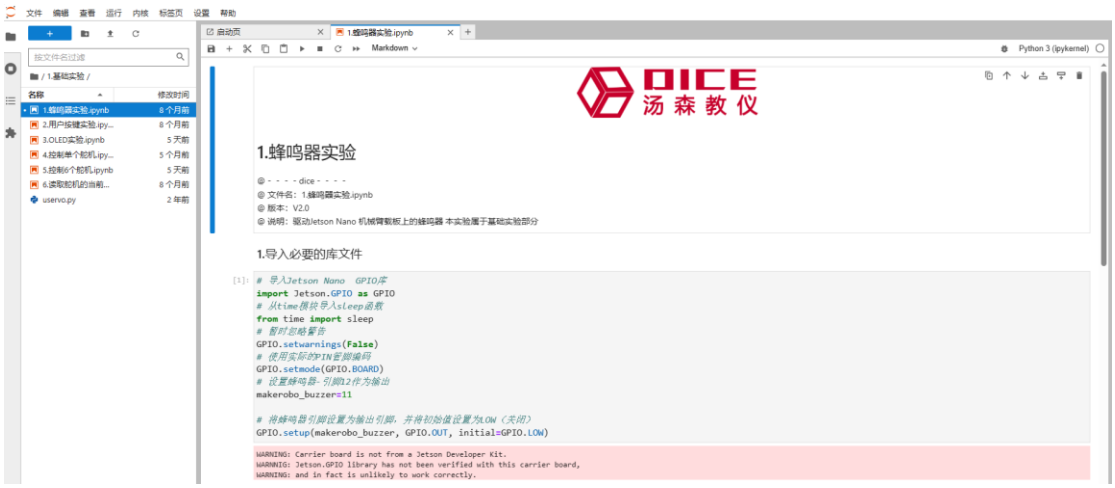


图 5.1.2 蜂鸣器控制实验程序截图

程序说明：

在驱动板的电路上蜂鸣器十连接到 Jetson Nano 的 GPIO 的 11 接口上，对 Jetson Nano 进行编程控制，如高电平打开蜂鸣器，低电平关闭蜂鸣器。

whitebear_buzzer=11

原理图如下图所示：

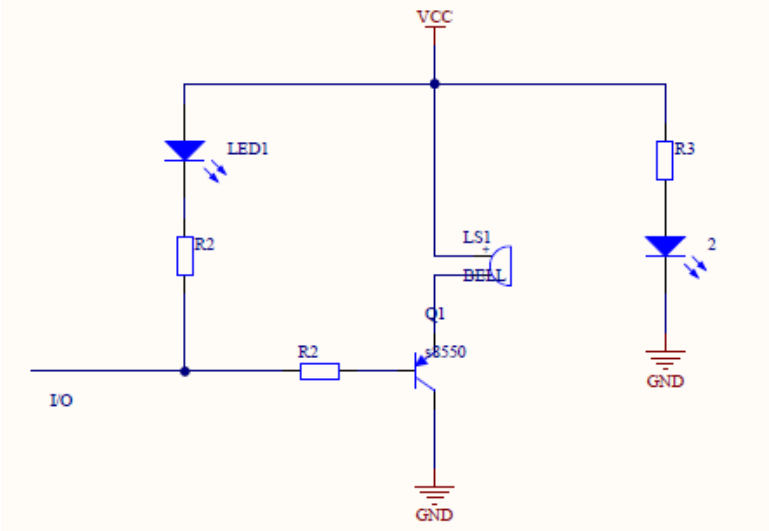


图 5.1.3 蜂鸣器实验原理图

第 2 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 3 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：可以听到蜂鸣器连续响十声。

5.1.4、python 代码

2.蜂鸣器实验

@ 文件名：2.蜂鸣器控制实验.ipynb
@ 版本：V2.0
@ 说明：蜂鸣器
从最简单的 GPIO 口的输出控制为例，让蜂鸣器自动响!

1.导入必要的库文件

```
# 导入 Jetson Nano  GPIO 库
import Jetson.GPIO as GPIO
# 从 time 模块导入 sleep 函数
from time import sleep
# 暂时忽略警告
GPIO.setwarnings(False)
# 使用实际的 PIN 管脚编码
GPIO.setmode(GPIO.BOARD)
# 设置蜂鸣器-引脚 12 作为输出
whitebear_buzzer=11

# 将蜂鸣器引脚设置为输出引脚，并将初始值设置为 LOW（关闭）
GPIO.setup(whitebear_buzzer, GPIO.OUT, initial=GPIO.LOW)
```

2.运行 10 次

```
for i in range(0,10):
    GPIO.output(whitebear_buzzer,GPIO.HIGH)
    print ("Beep")
    sleep(0.5) # 延时 0.5s
    GPIO.output(whitebear_buzzer,GPIO.LOW)
    print ("No Beep")
    sleep(0.5)
```

5.2、用户按键实验

5.2.1、硬件说明

在机械臂转接板上有两个用户自定义按键，使用这两个按键我们可以选择需要运行的功能或者切换不同的应用，作为用户交互使用。

轻触按键模块是最为常见的开关模块，内部有一个轻触拨盘，当按下输出低电平时，松开以保持高电平。

5.1.2、实验原理

使用轻触开关作为 Jetson Nano 的输入设备，当按下开关时，通过 Jetson Nano 的 GPIO 口（key1 为 GPI037，key2 为 GPI040）设置为输入模式检测到低电平（0V），表示按键被按键，当松开按键时，Jetson Nano 检测到高电平（3.3V），在本项目实验中我们将通过前面介绍的蜂鸣器配合使用按键，当按键按下蜂蜜器鸣叫一声作为提示，该模块的原理图如下所示：

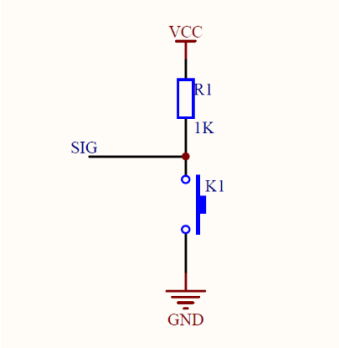


图 5.2.1 用户按键实验原理图

5.1.3、实验步骤

首次使用时，我们需要按照前面所学 JupyterLab 的内容，在电脑端通过网页登陆，Jetson Nano 的 IP 地址加上 8888（IP:8888）如下界面：

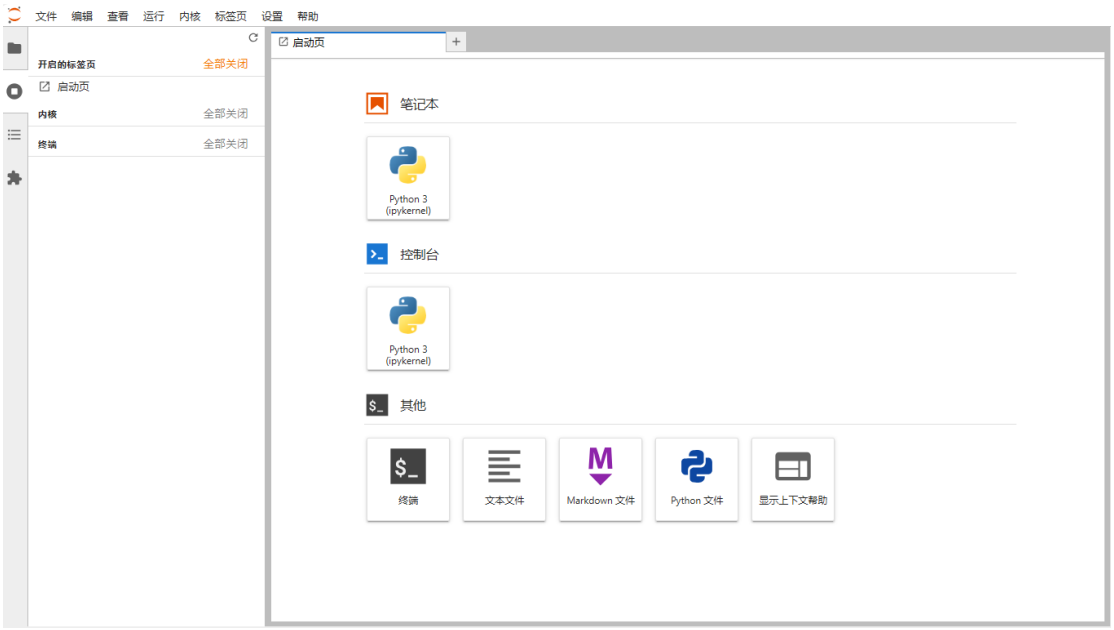


图 5.2.2 JupyterLab 界面截图

第 1 步：进入‘Notebook’文件夹里的‘基础实验’文件夹，找到‘2.用户按键实验.ipynb’程序，双击用 JupyterLab 打开程序:

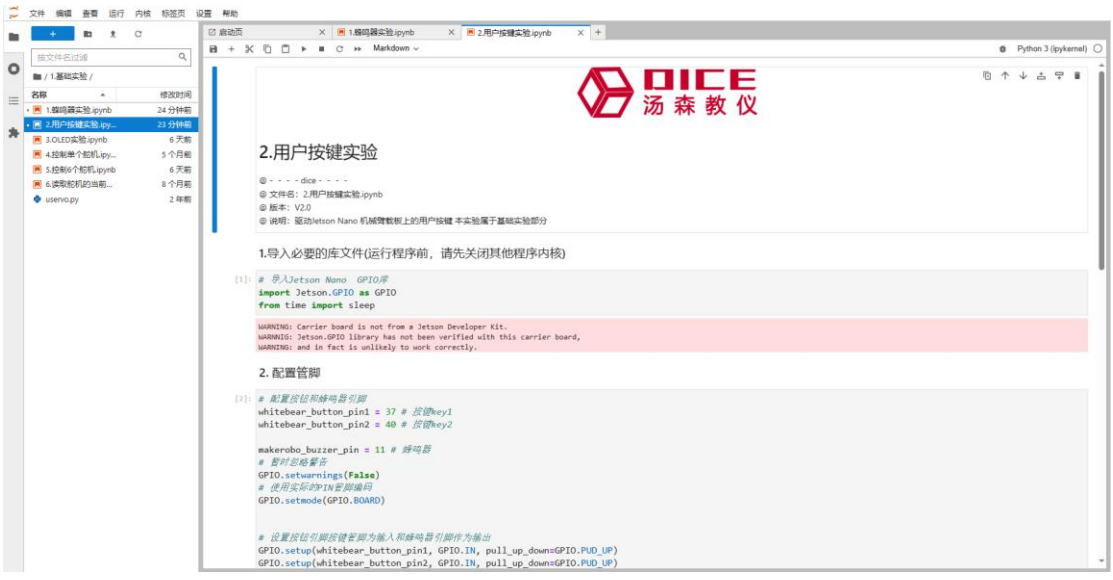


图 5.2.3 用户按键实验程序截图

第 2 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 3 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：运行程序时，当按键按下时蜂鸣器鸣叫一声作为提示。

5.2.4、python 代码

2.用户按键实验实验

@ 文件名：2.用户按键实验.ipynb
@ 版本：V2.0
@ 说明：蜂鸣器
驱动 Jetson Nano 机械臂载板上的用户按键 本实验属于基础实验部分

1.导入必要的库文件

```
# 导入 Jetson Nano  GPIO 库
import Jetson.GPIO as GPIO
from time import sleep
```

2.配置管脚

```
while True:
    #GPIO.setup(whitebear_button_pin1, GPIO.IN)
    ver1 = GPIO.input(whitebear_button_pin1)
    ver2 = GPIO.input(whitebear_button_pin2)
    #GPIO.setup(whitebear_button_pin, GPIO.OUT,initial=GPIO.HIGH)
    # 检查按键是否按下
    if(ver1 ==0 or ver2 ==0):
        GPIO.output(whitebear_buzzer_pin, GPIO.HIGH)
    else:
        # 没有按键按下，关闭蜂鸣器
        GPIO.output(whitebear_buzzer_pin, GPIO.LOW)
```

5.3、OLED 实验

5.3.1、硬件说明

在机械臂转接板上有一块 0.96 寸的 OLED 显示屏，实时显示我们重要的信息，如 IP 地址、内存使用情况及闪存显示情况，作为我们和硬件交互的重要显示信息。

OLED 显示屏属于固态显示和常见的液晶显示屏相比，具有可视角度大，使用寿命长，响应速度快等优点。

图 5.3.1 OLED 显示屏实物图

5.3.2、实验原理

OLED 显示屏通过 Jetson Nano 的 I2 C 总线连接，所以接线相对容易。

Jetson Nano 板载有两路 I2C 接口，白熊主板上的 OLED 就是通过 I2 C 接口来操作的。这里我们可以使用 i2ctool 工具来查看 i2 C 设备。这里指令最后的参数为 1， 表示查看的是 i2c 总线 1 的设备，如果参数为 0，则查看的是总线 0 的设备。

```
i2cdetect -y -r 1
```

Jetson Nano J41 Header

BCM	Name	BOARD	BOARD	Name	BCM
3V3	3.3VDC (Power)	1	2	5.0VDC (Power)	5V
2	I2C_2_SDA (12C Bus 1)	3	4	5.0VDC (Power)	5V
3	I2C_2_SCL (12C Bus 1)	5	6	GND	GND
4	AUDIO_MCLK	7	8	UART_2_TX (dev/ttyTHS1)	14
GND	GND	9	10	UART_2_RX (dev/ttyTHS1)	15

图 5.3.2 Jetson Nano I2C 引脚

硬件连接：

1. VCC ---接 Nano 引脚 3.3V;
2. GND ---接 Nano 引脚 GND;
3. SCL ---接 Nano 引脚第 5 脚(I2C_SCL),电平已转换成 3.3V 电平;
4. SDA ---接 Nano 引脚第 3 脚(I2C_SDA),电平已转换成 3.3V 电平;

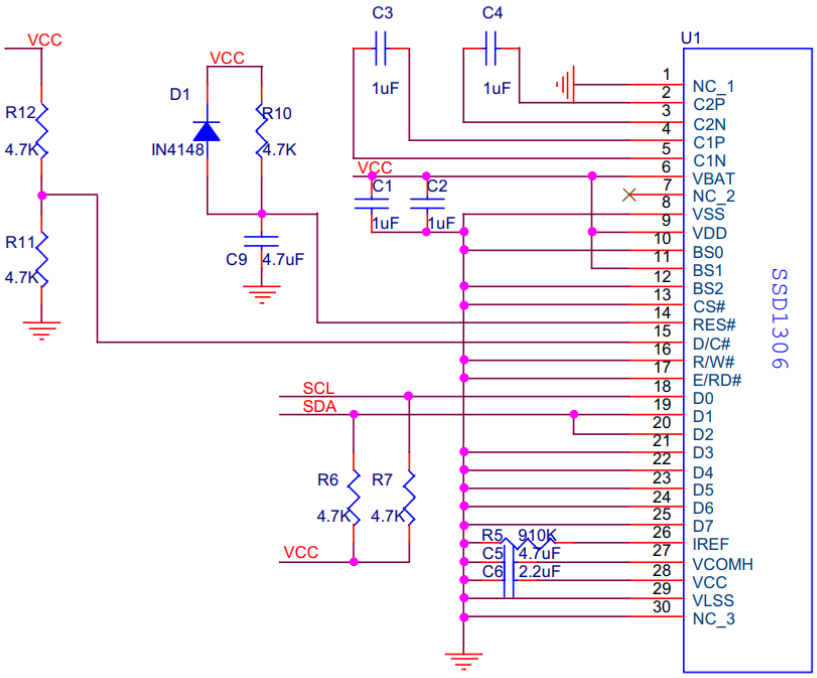


图 5.3.3 SSD1306 屏幕原理图

5.3.3、实验步骤

第 1 步：我们在终端中输入“i2cdetect -y -r 1”查看挂载的 I2C 设备

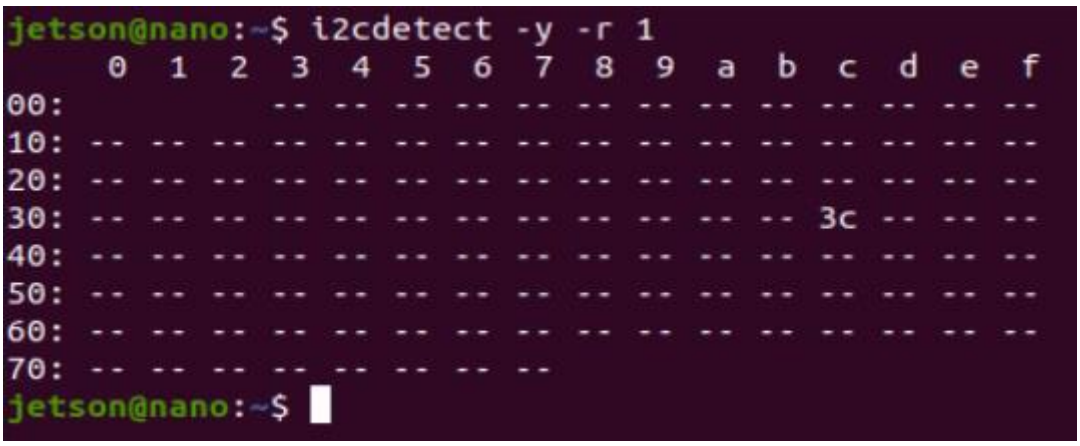


图 5.3.4 OLED 屏幕已挂载到 I2C

第 2 步：在终端中运行以下指令，查看正在运行的 OLED 后台进程：（显示主板信息的后台程序）

```
ps aux | grep oled
```

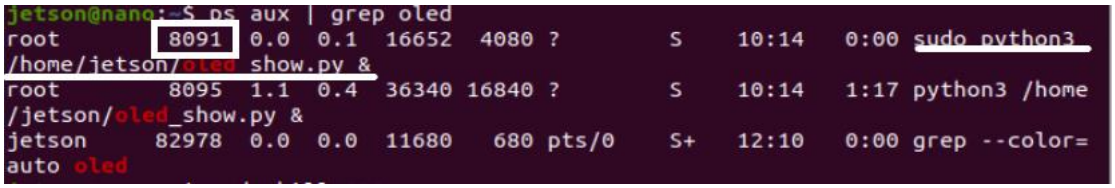


图 5.3.5 查找”OLED”相关的后台进程

找到相对应的后台进程 ID，上图白色框内为 ID 号，上图中画白线的后台执行命令，运行以下指令关闭

```
sudo kill 8091
```

第 3 步：进入‘Notebook’文件夹里的‘1.基础实验’文件夹，找到‘3.OLED 实验.ipynb’程序，双击用 JupyterLab 打开程序:

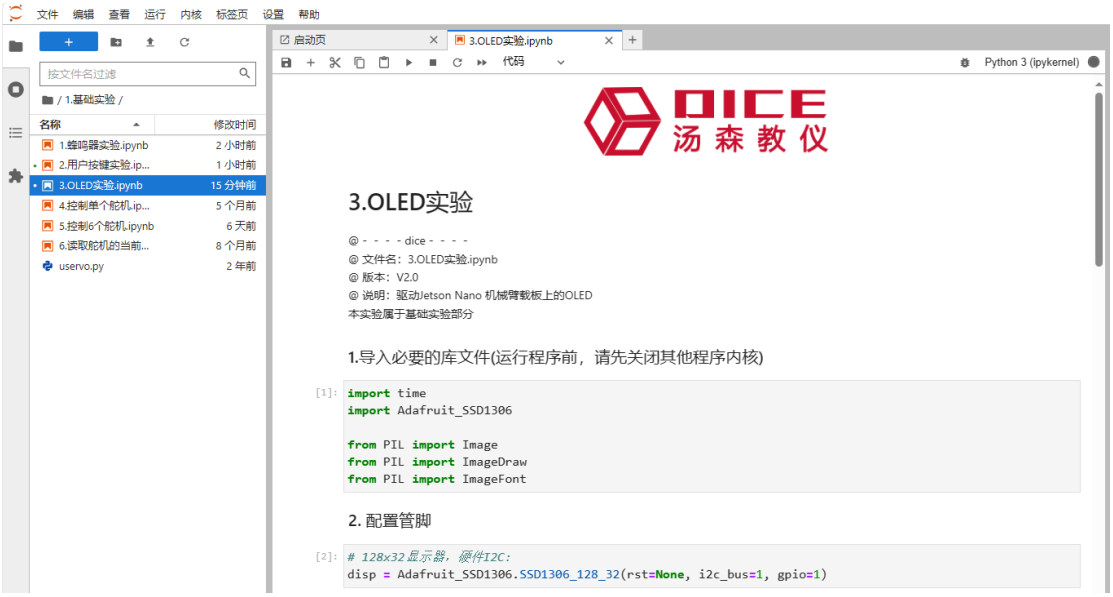


图 5.3.6 OLED 实验程序截图

Adafruit_SS1306 是 Adafruit Electronics 提供的驱动小尺寸 OLED 屏幕的驱动。需要提前安装在设备上，否则无法驱动屏幕。

如有需求可以通过以下指令安装：

```
sudo git clone https://github.com/adafruit/Adafruit_Python_SSD1306.git

cd Adafruit_Python_SSD1306

sudo python3 setup.py install
```

第 4 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 5 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：OLED 屏幕显示“**DICE AI arm5dof**”文字。

5.3.4、python 代码

```
3.控制单个舵机

@ 文件名：3.OLED 实验.ipynb
@ 版本：V2.0
@ 说明：驱动 Jetson Nano 机械臂载板上的 OLED
通过 I2C 控制 OLED 屏幕！

1.导入必要的库文件
```

```
import time
import Adafruit_SSD1306

from PIL import Image
from PIL import ImageDraw
from PIL import ImageFont
```

2.配置管脚

```
# 128x32 显示器，硬件 I2C:
disp = Adafruit_SSD1306.SSD1306_128_32(rst=None, i2c_bus=1, gpio=1)
```

3.执行检测

```
#  初始化库。
disp.begin()
#  清除显示内容
disp.clear()
disp.display()

#为绘图创建空白图像。
#确保为 1 位颜色创建模式为“1”的图像（单色）
width = disp.width
height = disp.height
image = Image.new('1', (width, height))

#  载入默认字体
font = ImageFont.load_default()

#  获取要在图像上绘制的绘图对象。
draw = ImageDraw.Draw(image)

while True:
    #  画一个黑色填充框清除图像。
    draw.rectangle((0,0,width,height), outline=0, fill=0)
    draw.text((0,0),"DICE AI arm5dof",font=font,fill=255)

    #  显示图像。
    disp.image(image)
    disp.display()
    time.sleep(3)
```

5.4、控制单个舵机

5.4.1 总线舵机介绍

对于总线舵机来说，我们的项目采用的是 Python 编程实现，为了简化整个操作流程，我们有提供操作总线舵机的操作库，我们只需要把我们提供的操作库复制到我们控制的工程目录下即可，对于舵机的每个功能，我们都有提供对应的函数进行操作，大大降低了操作难度。

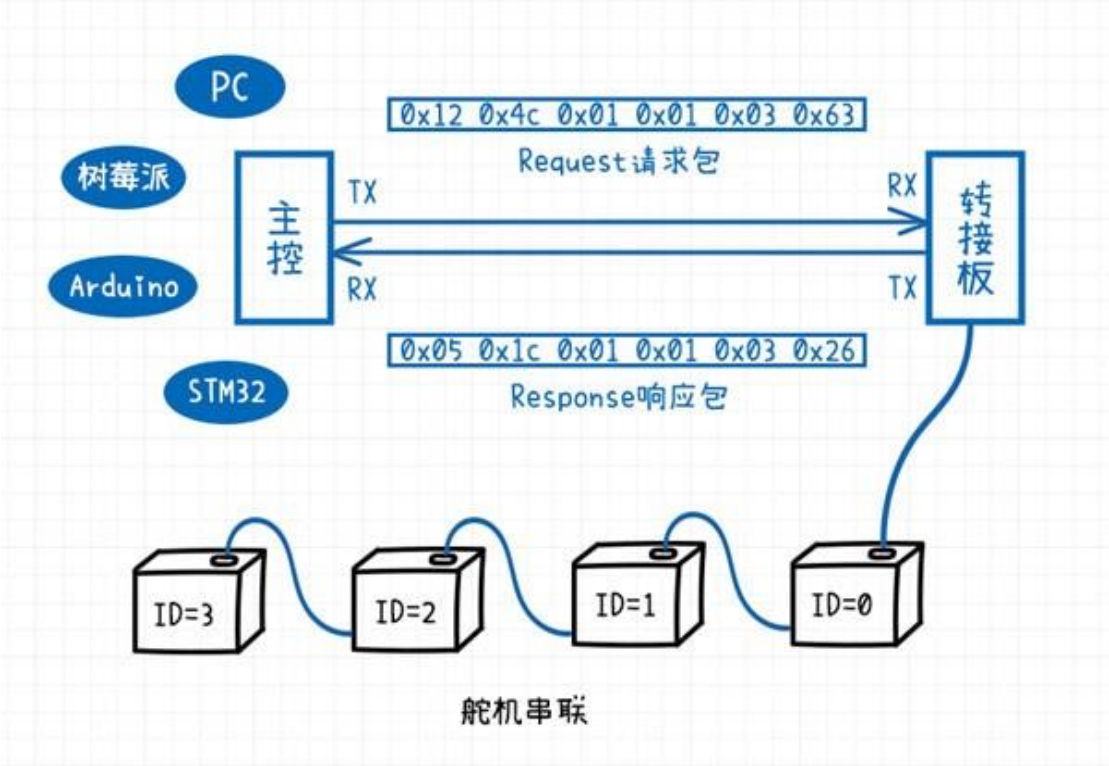


图 5.4.1 总线舵机控制方式

5.4.2 总线舵机驱动库说明

串口总线舵机的文件路径程序案例/Notebook/1.基础实验/uservo.py，使用的时候可以将 uservo.py 拷贝至你当前工程文件夹里面。

或者使用的时候，将 uservo.py 所在的文件夹添加到系统路径里面，相对路径/绝对路径都可以。

```
# 添加 uservo.py 的系统路径
import sys
sys. path. append ("./../src")
```

然后使用的过程中一般需要导入如下这两个依赖

```
# PySerial 负责串口总线通信
import serial
# UartServoManager 串口总线舵机管理器
```

```
from uservo import UartServoManager
```

接下来要创建串口对象，指定相关的参数

```
#角度定义
DEVICE_PORT _DEFAULT = ' /dev/ttyTHS1'  # Jetson Nano
SERVO_BAUDRATE = 115200 #  舵机的波特率
SERVO_ID = 0 #舵机的 ID 号
#  初始化串口
uart = serial.Serial(port=DEVICE_PORT_DEFAULT,
baudrate=SERVO_BAUDRATE,parity=serial.PARITY_NONE, stopbits=1,bytesize=8,timeout=0)
```

创建舵机管理器，将串口对象传入到构造器 UartServolanager 里面。

```
#初始化舵机管理器
uservo = UartServoManager(uart, is_debug=True)
```

UART 串行总线舵机 Python 函数说明：

函数名	功能	函数原型	参数说明
ping	调用舵机的 ping()函数用于舵机的通信检测，判断舵机是否在线	def ping(self,servo_id:int):	输入参数: servo_id:舵机 ID 输出参数: is_online:舵机是否在线
set_damping	设置舵机为阻尼模式	def set_damping(self,servo_id, power=0):	输入参数: servo_id:舵机 ID power:舵机功率，单位 Mw 输出参数：无
query_servo_angle	舵机角度查询	def query_servo_angle(self,servo_id):	输入参数: servo_id:舵机 ID 输出参数: angle:舵机角度（单圈/多圈）
set_servo_angle	设置舵机角度，这个 API 包含了 6 种舵机角度控制模式，通过传入不同的参数调用不同的指令。	defset-servo-angle （self, servo_id:int, angle:f1oat, is_mturn:bool=False,interval:float=None,velocity:float=None,t_acc:int=20,t_dec:int=20,power:int=0,mean_ps:float=100.0) :	输入参数 servo_id：舵机的 ID 号 angle:目标角度 is_mturn：是否是多圈模式 interval：中间间隔 单位 ms velocity：舵机的目标转速，单位 dps t_acc:加速时间，在指定目标转速时有效.单位 ms t_dec：减速时间，在指定减速时间时有效.单位 ms power：功率限制，单位 mW

			mean_dps：平均转速，单位 dps，用于估计 interval 输出参数：无
wait	等待所有舵机到达目标角度	def wait(self,timeout=None):	输入参数 timeout:阻塞式等待的超时判断阈值，单位 ms 输出参数：无
wheel_stop	轮转模式停止转动	def wheel_stop(self,servo_id):	输入参数 servo_id：舵机的 ID 号 输出参数：无
set_wheel_norm	设置轮转普通模式，转速单位°/s	def set_wheel_norm(self,servo_id.is_cw=Ture,mean_dps=None):	输入参数 servo_id：舵机的 ID 号 is_cw:是否是顺时针 Ture：顺时针 False:逆时针 mean_dps：平均转速 输出参数：无
reset_user_data	重置用户数据表，恢复默认值	def reset_user_data(self,servo_id)	输入参数 servo_id：舵机的 ID 号 输出参数：无
read_data	读取数据	def read_data(self,servo_id,address)	输入参数 servo_id：舵机的 ID 号 address:内存表 输出参数 content:数值的二进制数据流
write_data	写入数据	def write_data(self,servo_id,address,content)	输入参数 servo_id：舵机的 ID 号 address:内存表 content:数值的二进制数据流 输出参数：无
query_voltage	查询当前的电压	def query_voltage (self,servo_id)	输入参数 servo_id：舵机的 ID 号 输出参数 voltage:电压，单位 V
query_current	查询当前的电流	def query_current (self,servo_id)	输入参数 servo_id：舵机的 ID 号 输出参数 current:舵机电流，单位 A

query_power	查询当前的功率	def query_power (self,servo_id)	输入参数 servo_id：舵机的 ID 号 输出参数 power:舵机功率，单位 W
query_temperature	查询舵机当前温度	def query_temperature (self,servo_id)	输入参数 servo_id：舵机的 ID 号 输出参数 temperature:温度，ADC 值

注：ADC 转换为摄氏度值公式：

$$R1 = 10000; (NTC_{分压电阻值}，R1_{靠近电源}，NTC_{靠近}GND)$$
$$Rt25 = 10000; (NTC_{在25^\circ室温时的电阻值})$$
$$B = 3435; (NTC_{的材料常数})$$
$$K = 273.15; (_{绝对零度})$$
$$T = 1/(\ln(R1 \cdot ADC \div Rt25 \div (4096 - ADC)) \div B + 1 \div (K + 25)) - K$$

图 5.4.2 ADC 转换为摄氏度值公式

50-79℃温度/ADC 参考表:

温度(℃)	ADC	温度(℃)	ADC	温度(℃)	ADC
50	1191	60	941	70	741
51	1164	61	918	71	723
52	1137	62	897	72	706
53	1110	63	876	73	689
54	1085	64	855	74	673
55	1059	65	835	75	657
56	1034	66	815	76	642
57	1010	67	796	77	627
58	986	68	777	78	612
59	963	69	759	79	598

5.4.3 实验步骤

第 1 步：进入‘Notebook’文件夹里的‘1.基础实验’文件夹，找到‘4. 控制单个舵机.ipynb’程序，双击用 JupyterLab 打开程序:

图 5.4.3 控制单个舵机程序截图

第 2 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 3 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：舵机根据程序中设置的舵机号、平移角度、转速、加速时间、减速时间转动。

5.4.4、python 代码

4.控制单个舵机

@－－－－DICE－－－－

@ 文件名：4.控制单个舵机实验.ipynb

@ 版本：V2.0

@ 说明：驱动 Jetson Nano 机械臂单个舵机

本实验属于基础实验部分

1.导入必要的库文件

```
# 导入依赖
import time
import struct
import serial
from uservo import UartServoManager
```

2.设置串口总线参数

```
DEVICE_PORT_DEFAULT = '/dev/ttyTHS1'     # Jetson Nano
```

```
SERVO_BAUDRATE = 115200                 # 舵机的波特率
JOINT1_SERVO_ID = 0                     # 关节 1
JOINT2_SERVO_ID = 1                     # 关节 2
JOINT3_SERVO_ID = 2                     # 关节 3
JOINT4_SERVO_ID = 3                     # 关节 4
JOINT5_SERVO_ID = 4                     # 关节 5
JOINT6_SERVO_ID = 5                     # 关节 6
```

3.初始化舵机管理器

```
uart = serial.Serial(port=DEVICE_PORT_DEFAULT,
baudrate=SERVO_BAUDRATE,parity=serial.PARITY_NONE, stopbits=1,bytesize=8,timeout=0)
uservo = UartServoManager(uart, is_debug=True)
```

4.控制舵机旋转

```
print("设置平移舵机角度为 0.0°,设置转速为 200 °/s, 加速时间 100ms, 减速时间 100ms")
uservo.set_servo_angle(JOINT6_SERVO_ID, 30.0, velocity=200.0, t_acc=100, t_dec=100) # 设置舵机角度
uservo.wait() # 等待舵机静止
```

5.5、同时控制 6 个总线舵机动作

5.5.1、机械臂总线舵机介绍

在上一个实验中，我们介绍了单个的总线舵机，在我们的机械臂中一共是用到了 6 个总线舵机，采用串联的方式进行连接，大大的降低了布线密度。

在这个实验中我们直接驱动 6 个舵机，熟练掌握多个舵机的驱动方式。

图 5.5.1 6 舵机机械臂实物图

5.5.2、实验步骤

第 1 步：进入‘Notebook’文件夹里的‘1.基础实验’文件夹，找到‘5. 同时控制 6 个舵机动作.ipynb’程序，双击用 JupyterLab 打开程序:

图 5.5.2 控制 6 个舵机实验程序截图

第 2 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 3 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：运行程序后可以看到机械臂逐渐运行至直立状态。

5.5.2、python 代码

5.控制 6 个舵机动作

```
@----DICE-----
@ 文件名：5.控制 6 个舵机实验.ipynb
@ 版本：V2.0
@ 说明：驱动 Jetson Nano 机械臂 6 个舵机
本实验属于基础实验部分
```

1.导入必要的库文件

```
# 导入依赖
import time
import struct
import serial
from uservo import UartServoManager
```

2.设置串口总线参数

```
DEVICE_PORT_DEFAULT = '/dev/ttyTHS1'    # Jetson Nano
SERVO_BAUDRATE = 115200                 # 舵机的波特率
JOINT1_SERVO_ID = 0                     # 关节 1
JOINT2_SERVO_ID = 1                     # 关节 2
JOINT3_SERVO_ID = 2                     # 关节 3
JOINT4_SERVO_ID = 3                     # 关节 4
JOINT5_SERVO_ID = 4                     # 关节 5
JOINT6_SERVO_ID = 5                     # 关节 6
```

3.初始化舵机管理器

```
uart = serial.Serial(port=DEVICE_PORT_DEFAULT,
baudrate=SERVO_BAUDRATE,parity=serial.PARITY_NONE, stopbits=1,bytesize=8,timeout=0)
uservo = UartServoManager(uart, is_debug=True)
```

4.控制舵机旋转

```
print("设置关节 1 舵机角度为 0.0°,设置转速为 200 °/s, 加速时间 100ms, 减速时间 100ms")
uservo.set_servo_angle(JOINT1_SERVO_ID, 0.0, velocity=200.0, t_acc=100, t_dec=100) # 设置舵机角度
uservo.wait() # 等待舵机静止

print("设置关节 2 舵机角度为 0.0°,设置转速为 200 °/s, 加速时间 100ms, 减速时间 100ms")
uservo.set_servo_angle(JOINT2_SERVO_ID, 0.0, velocity=200.0, t_acc=100, t_dec=100) # 设置舵机角度
uservo.wait() # 等待舵机静止
```

```
print("设置关节 3 舵机角度为 0.0°,设置转速为 200 °/s, 加速时间 100ms, 减速时间 100ms")
uservo.set_servo_angle(JOINT3_SERVO_ID, 0.0, velocity=200.0, t_acc=100, t_dec=100) # 设置舵机角度
uservo.wait() # 等待舵机静止

print("设置关节 4 舵机角度为 0.0°,设置转速为 200 °/s, 加速时间 100ms, 减速时间 100ms")
uservo.set_servo_angle(JOINT4_SERVO_ID, 0.0, velocity=200.0, t_acc=100, t_dec=100) # 设置舵机角度
uservo.wait() # 等待舵机静止

print("设置关节 5 舵机角度为 0.0°,设置转速为 200 °/s, 加速时间 100ms, 减速时间 100ms")
uservo.set_servo_angle(JOINT5_SERVO_ID, 0.0, velocity=200.0, t_acc=100, t_dec=100) # 设置舵机角度
uservo.wait() # 等待舵机静止

print("设置关节 6 舵机角度为 0.0°,设置转速为 200 °/s, 加速时间 100ms, 减速时间 100ms")
uservo.set_servo_angle(JOINT6_SERVO_ID, 0.0, velocity=200.0, t_acc=100, t_dec=100) # 设置舵机角度
uservo.wait() # 等待舵机静止
```

5.6、读取舵机当前的位置

5.6.1、机械臂舵机位置反馈介绍

在对机械臂的控制的时候，如果要进行机械臂的运动学处理和需要有一定的精度的话，每个关节的位置反馈是必要的，它可以让我们控制机械臂到底指定位置并作出反馈是否到达，这里我们一起来操作一下总线舵机的位置反馈。

5.6.2、实验步骤

第 1 步：进入‘Notebook’文件夹里的‘1.基础实验’文件夹，找到‘6. 读取舵机的当前位置’程序，双击用 JupyterLab 打开程序：

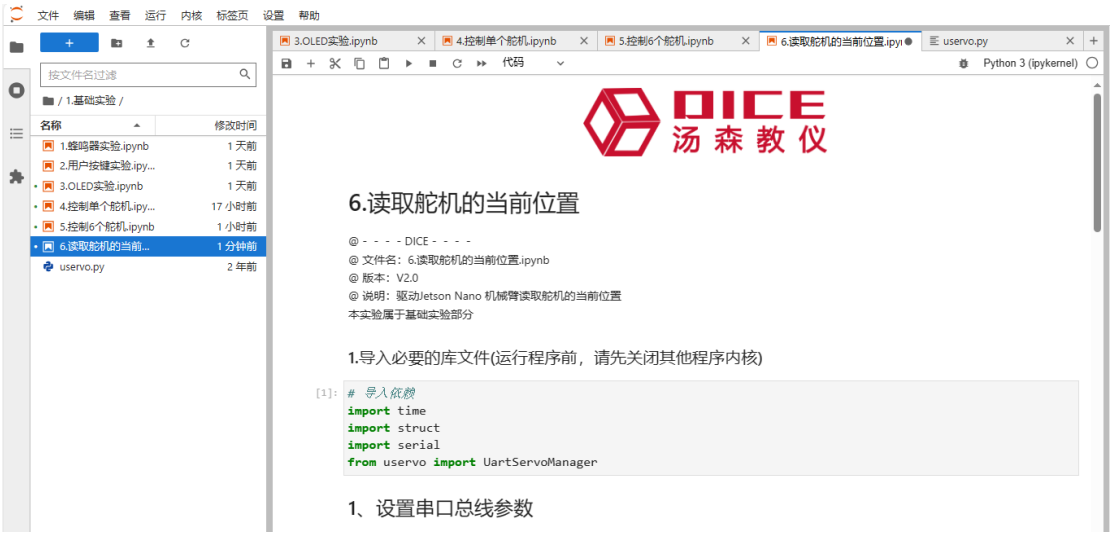


图 5.6.1 读取舵机当前位置实验程序截图

第 2 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 3 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：程序运行后会打印出舵机当前所在的角度。

5.6.2、python 代码

```
6.读取舵机当前的位置
@－－－－DICE－－－－
@ 文件名：6.读取舵机的当前位置.ipynb
@ 版本：V2.0
@ 说明：驱动 Jetson Nano 机械臂读取舵机的当前位置
本实验属于基础实验部分
```

1.导入必要的库文件

```
# 导入依赖
import time
import struct
import serial
from uservo import UartServoManager
```

2.设置串口总线参数

```
DEVICE_PORT_DEFAULT = '/dev/ttyTHS1'     # Jetson Nano
SERVO_BAUDRATE = 115200                    # 舵机的波特率
JOINT1_SERVO_ID = 0                        # 关节 1
JOINT2_SERVO_ID = 1                        # 关节 2
JOINT3_SERVO_ID = 2                        # 关节 3
JOINT4_SERVO_ID = 3                        # 关节 4
JOINT5_SERVO_ID = 4                        # 关节 5
JOINT6_SERVO_ID = 5                        # 关节 6
```

3.初始化舵机管理器

```
uart = serial.Serial(port=DEVICE_PORT_DEFAULT,
baudrate=SERVO_BAUDRATE,parity=serial.PARITY_NONE, stopbits=1,bytesize=8,timeout=0)
uservo = UartServoManager(uart, is_debug=True)
```

4.打印舵机角度

```
print("设置关节 1 舵机角度为 0.0°,设置转速为 200 °/s, 加速时间 100ms, 减速时间 100ms")
uservo.set_servo_angle(JOINT1_SERVO_ID, 0.0, velocity=200.0, t_acc=100, t_dec=100) # 设置舵机角度
uservo.wait() # 等待舵机静止
print("-> {}".format(uservo.query_servo_angle(JOINT1_SERVO_ID)))

print("设置关节 2 舵机角度为 0.0°,设置转速为 200 °/s, 加速时间 100ms, 减速时间 100ms")
uservo.set_servo_angle(JOINT2_SERVO_ID, 0.0, velocity=200.0, t_acc=100, t_dec=100) # 设置舵机角度
uservo.wait() # 等待舵机静止
print("-> {}".format(uservo.query_servo_angle(JOINT2_SERVO_ID)))

print("设置关节 3 舵机角度为 0.0°,设置转速为 200 °/s, 加速时间 100ms, 减速时间 100ms")
uservo.set_servo_angle(JOINT3_SERVO_ID, 0.0, velocity=200.0, t_acc=100, t_dec=100) # 设置舵机角度
uservo.wait() # 等待舵机静止
print("-> {}".format(uservo.query_servo_angle(JOINT3_SERVO_ID)))

print("设置关节 4 舵机角度为 0.0°,设置转速为 200 °/s, 加速时间 100ms, 减速时间 100ms")
uservo.set_servo_angle(JOINT4_SERVO_ID, 0.0, velocity=200.0, t_acc=100, t_dec=100) # 设置舵机角度
uservo.wait() # 等待舵机静止
print("-> {}".format(uservo.query_servo_angle(JOINT4_SERVO_ID)))

print("设置关节 5 舵机角度为 0.0°,设置转速为 200 °/s, 加速时间 100ms, 减速时间 100ms")
```

```
uservo.set_servo_angle(JOINT5_SERVO_ID, 0.0, velocity=200.0, t_acc=100, t_dec=100) # 设置舵机角度
uservo.wait() # 等待舵机静止
print("-> {}".format(uservo.query_servo_angle(JOINT5_SERVO_ID)))

print("设置关节 6 舵机角度为 0.0°,设置转速为 200 °/s, 加速时间 100ms, 减速时间 100ms")
uservo.set_servo_angle(JOINT6_SERVO_ID, 0.0, velocity=200.0, t_acc=100, t_dec=100) # 设置舵机角度
uservo.wait() # 等待舵机静止
print("-> {}".format(uservo.query_servo_angle(JOINT6_SERVO_ID)))
```

5.7、机械臂关节标定实践

5.7.1、机械臂关节标定介绍

总线舵机属于绝对值舵机，也就是说旋转的角度是固定的，而机械臂的安装属于手工安装，必然会带来绝对误差，在这里我们将学习如何消除误差!

5.7.2、机械臂关节标定计算表达

消除误差的方式其实是比较简单的，我们采用采集两组机械臂的固定姿态，分别计算出每种姿态下，每个舵机的角度，计算出两组每个舵机角度的比例系数（Ki）值和偏移量（bi）值。

1.样本采集

采集两组固定姿态的数据:

第一种姿态为将机械臂掰直，水平于桌面，夹爪张开。采集舵机原始角度。

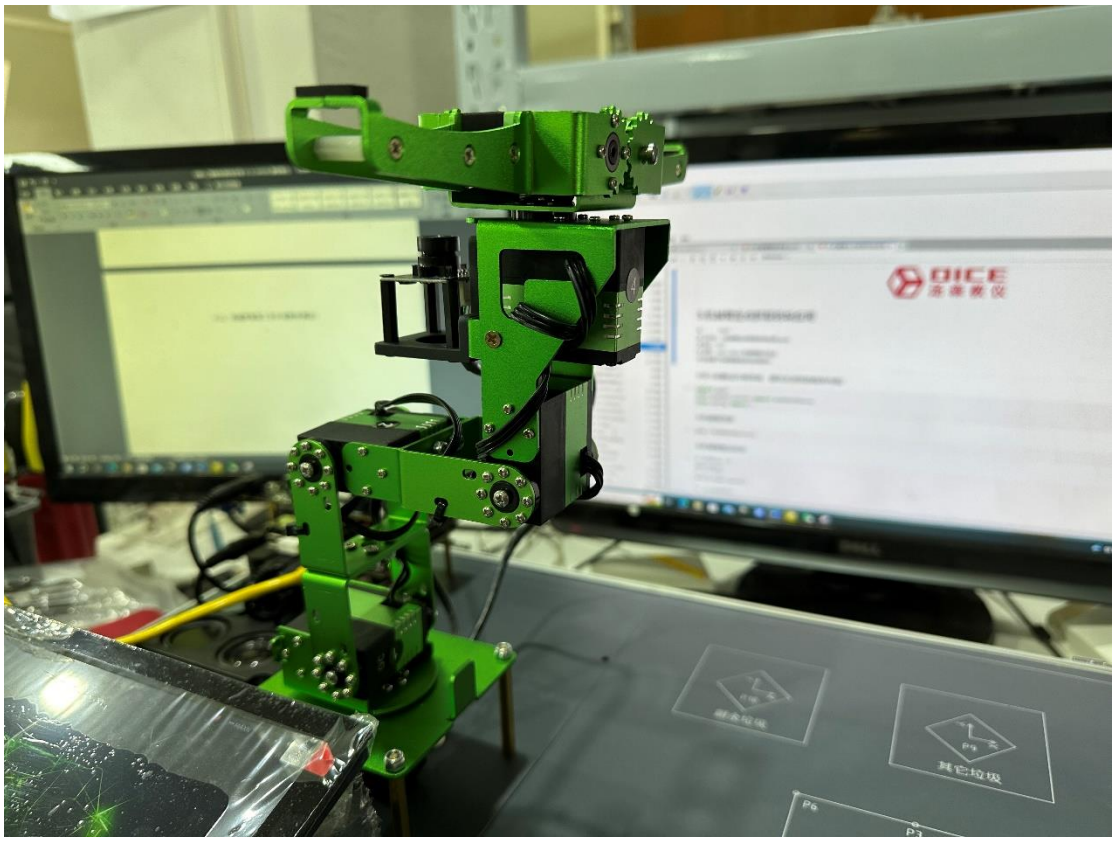


图 5.7.1 pose1

第二个姿态为夹爪闭合，令夹爪运动到机械臂左侧，夹爪未端垂直于桌面，且连杆之间均为 90 度直角。

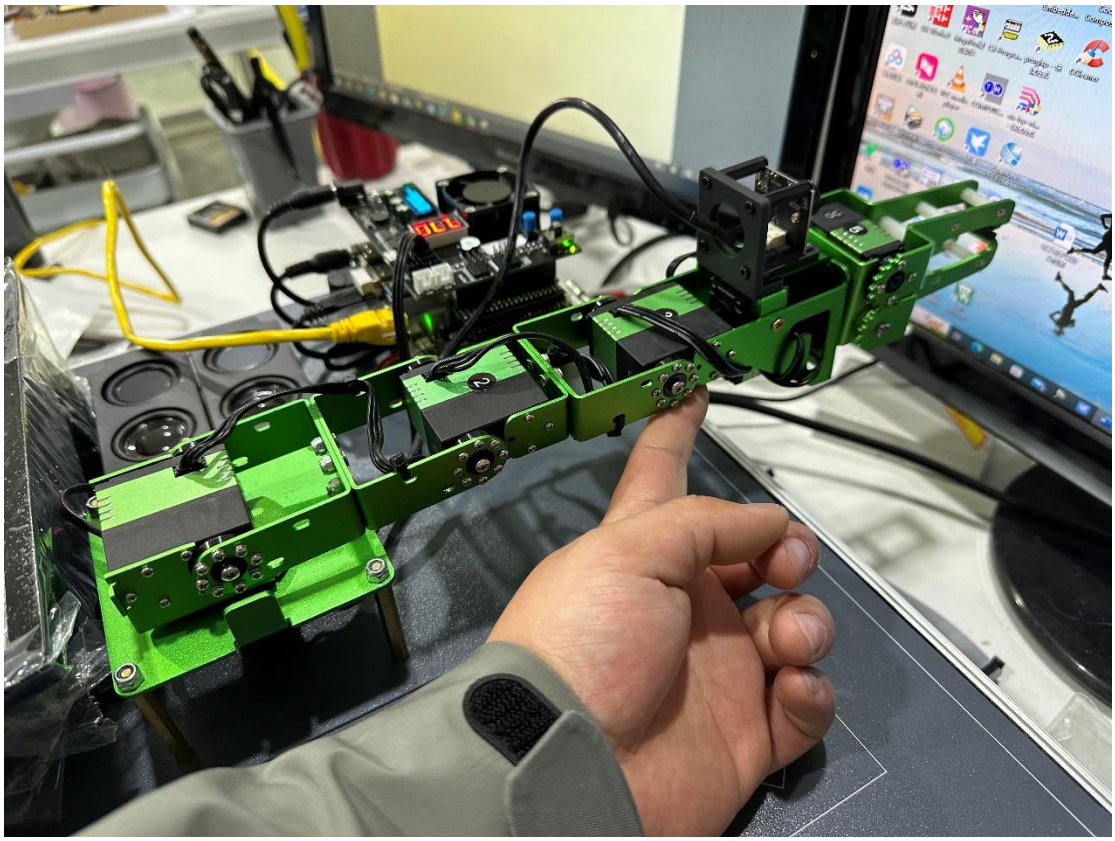


图 5.7.2 pose2

计算各个关节的比例系数与角度偏移量。用一个简单的一次函数来表示舵机原始角度与机械臂关节弧度之间的关系。

$$angle_i = k_i * \theta_i + b_i$$

- θ_i关节弧度
- $angle_i$ 舵机原始角度
- k_i 比例系数
- b_i 偏移量

5.7.3、实验步骤

第 1 步：进入‘Notebook’文件夹里的‘机械臂实验’文件夹，找到‘1.机械臂关节标定.ipynb’程序，双击用 JupyterLab 打开程序:

图 5.7.3 机械臂关节标定程序截图

第 2 步：运行代码，点击运行一步一步运行代码，操作至第三步（设置点击为阻尼模式），将机械臂掰直，水平与桌面，夹爪张开。采集舵机原始角度。见图 5.7.1

第 3 步：继续运行代码，待程序打印出舵机原始角度后，将机械臂夹爪闭合，令夹爪运动到机械臂上侧，夹爪末端垂直于桌面，且连杆之间均为 90 度直角。见图 5.7.2

第 4 步：继续运行代码，待程序打印出机械臂标定数据后，复制数据，将打印出来的字符串替换掉 config.py 里面的 JOINT2SERVO_K 与 JOINT2SERVO_B。

```
k1, b1 = calc_kb(j1_0, j1_90, 0, math.pi/2)
k2, b2 = calc_kb(j2_n90, j2_0, -math.pi/2, 0)
k3, b3 = calc_kb(j3_90, j3_0, math.pi/2, 0)
k4, b4 = calc_kb(j4_n90, j4_0, -math.pi/2, 0)
k5, b5 = calc_kb(j5_0, j5_90, 0, math.pi/2)
k6, b6 = calc_kb(j6_90, j6_0, math.pi/2, 0)

print(f'JOINT2SERVO_K=[{k1:.3f}, {k2:.3f}, {k3:.3f}, {k4:.3f}, {k5:.3f}, {k6:.3f}]')
print(f"JOINT2SERVO_B=[{b1:.3f}, {b2:.3f}, {b3:.3f}, {b4:.3f}, {b5:.3f}, {b6:.3f}]")
```

```
JOINT2SERVO_K=[-56.850, 59.842, -56.341, -57.678, 0.064, 56.659]
JOINT2SERVO_B=[-0.100, 90.700, -0.400, 0.000, -0.100, 6.100]
```

将打印出来的字符串替换掉 config.py 里面的 JOINT2SERVO_K 与 JOINT2SERVO_B

图 5.7.4 程序打印出数据

```
#JOINT2SERVO_K=[-57.614, 56.850, -57.805, -56.850, -58.951, 58.951]
#JOINT2SERVO_B=[-1.500, 78.400, -3.600, 1.800, 0.000, -0.500]
JOINT2SERVO_K=[-56.850, 59.842, -56.341, -57.678, 0.064, 56.659]
JOINT2SERVO_B=[-0.100, 90.700, -0.400, 0.000, -0.100, 6.100]
```

图 5.7.5 将打印出的数据复制到 config.py

第 5 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

5.7.4、python 代码

01.机械臂关节标定

```
@----DICE----
@ 文件名：1.机械臂关节标定.ipynb
@ 版本：V2.0
@ 说明：Jetson Nano 机械臂基本运动
本实验属于机械臂基础运动实验部分
```

1.导入依赖

```
import math
from arm5dof_uservo import Arm5DoFUServo
from config import *
```

2.机械臂初始化

```
arm = Arm5DoFUServo(is_init_pose=False)
```

```
3.设置舵机为阻尼模式
# 设置为阻尼模式
# 数值越大，阻尼越大
arm.set_damping(1000)
```

3.样本采集

```
pose1_servo_raw_angle = arm.get_servo_angle_list()
# 打印舵机原始角度
j1_0, j2_n90, j3_90, j4_n90, j5_0, j6_90 = pose1_servo_raw_angle
print(f"J1: {j1_0:.1f} J2: {j2_n90:.1f} J3: {j3_90:.1f} J4: {j4_n90:.1f} J5: {j5_0:.1f} J6: {j6_90:.1f}")

pose2_servo_raw_angle = arm.get_servo_angle_list()
# 打印舵机原始角度
j1_90, j2_0, j3_0, j4_0, j5_90, j6_0 = pose2_servo_raw_angle
print(f"J1: {j1_90:.1f} J2: {j2_0:.1f} J3: {j3_0:.1f} J4: {j4_0:.1f} J5: {j5_90:.1f} J6: {j6_0:.1f}")
```

6.标定舵机

```
def calc_kb(angle_a, angle_b, theta_a, theta_b):
    k = (angle_a-angle_b) / (theta_a-theta_b)
    b = angle_a - k*theta_a
    return k, b
k1, b1 = calc_kb(j1_0, j1_90, 0, math.pi/2)
k2, b2 = calc_kb(j2_n90, j2_0, -math.pi/2, 0)
k3, b3 = calc_kb(j3_90, j3_0, math.pi/2, 0)
k4, b4 = calc_kb(j4_n90, j4_0, -math.pi/2, 0)
```

```
k5, b5 = calc_kb(j5_0, j5_90, 0, math.pi/2)
k6, b6 = calc_kb(j6_90, j6_0, math.pi/2, 0)

print(f'JOINT2SERVO_K=[{k1:.3f}, {k2:.3f}, {k3:.3f}, {k4:.3f}, {k5:.3f}, {k6:.3f}]')
print(f"JOINT2SERVO_B=[{b1:.3f}, {b2:.3f}, {b3:.3f}, {b4:.3f}, {b5:.3f}, {b6:.3f}]")
```

5.8、机械臂运动学正逆解算

5.8.1、机械臂运动学正逆解算介绍

最近，随着应用场景和机械臂数量的增加，实现高精度、稳定性的机械手运动控制系统备受关注。作为决定机械手运动精度和稳定性的重要因素，机械手的运动学建模和求解已成为研究热点。在功能上，运动学可以分为正向和反向运动学。随着每个关节角度的变化，利用正向运动学可以获得机械手末端执行器的位置和姿态。这个过程代表了从关节空间到笛卡尔空间的转变。逆运动学根据末端执行器的位置和姿态求解机械手关节角度的变化。由于机械手的运动学方程是一组非线性方程，因此这些方程的解必须考虑单解和多解的解法的存在。与转发运动学相比，逆运动学更难建模和求解。

逆运动学的研究在机器人研究领域至关重要。这不仅是因逆向运动学是机器人轨迹规划运动控制和工作空间分析的基础，而且也是机器人技术中的一个技术问题。随着逆运动学的不断研究，机械手的逆运动学可以通过几何法、解析法、数值法、人工智能法四种方法求解。

几何方法首先被提出来求解逆运动学，但是，求解多自由度机械手的反运动学的过程非常复杂，几何方法不起作用。解决这个问题的最有用的方法是分析和数值方法。在求解逆运动学之前，分析方法必须分析机械手关节的数量和结构，由于雅可比迭代方法原理简单，对机械手的结构和关节数量没有特殊要求，因此被用作解决大多数 IK 问题的方法。同时，该数值方法还具有能够获得包含从初始姿态到目标姿态的路径规划的逆解的优点，因此适用于路径规划。然而，从理论上讲，不可能验证逆解是最优还是次优。

随着人工智能和智能算法的不断发展，一些人工智能方法已被用于解决机械手的反运动学，并取得了良好的效果。并且与几何，数值和分析方法求解机械手的反运动学相比，人工智能算法不需要进行推导运算即可获得雅可比矩阵，并且在常规迭代方法中不存在奇点问题。

5.8.2、机械臂的运动学分析

为了进一步研究机械手的运动学，本文选择 6-DOF 机械手作为研究对象。首先，需要建立机械手的运动学模型。在运动学模型方面，使用指数旋转矩阵直接描述操纵器的物理机制，但矩阵的多次变换产生大量的计算。在本文中，根据 Danavit 和 Hartenberg 模型（Danavit 和 Hartenberg,1964）建立了 6-DOF 机械手的坐标系。如图所示：

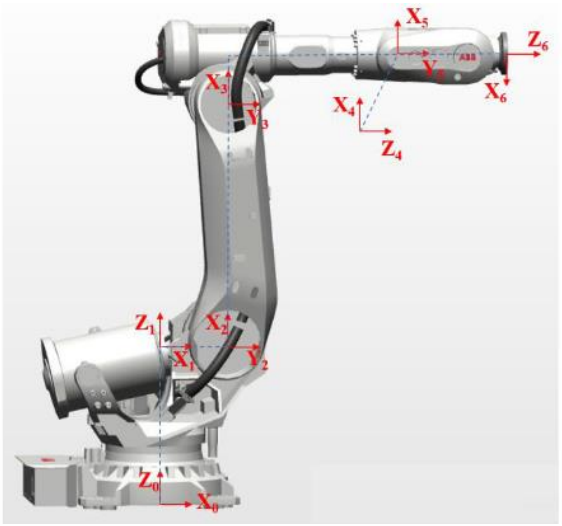


图 5.8.1 机械手坐标系

就机械手 i 和 i-1 的相邻坐标系而言，齐次坐标变换矩阵 ${}^{i}_{i-1}\mathrm{T}$ 可以用来表示位置和姿态的相对关系。

$$
{}^{i}_{i-1}\mathrm{T} = \begin{bmatrix} c_i & -s_i & 0 & a_{i-1} \\ s_i c_{\alpha_{i-1}} & c_i c_{\alpha_{i-1}} & -s_{\alpha_{i-1}} & -s_{\alpha_{i-1}} d_i \\ s_i s_{\alpha_{i-1}} & c_i s_{\alpha_{i-1}} & c_{\alpha_{i-1}} & s_{\alpha_{i-1}} d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}
$$

图 5.8.2 运动学变换矩阵公式

上式中的s_i为$sin(\theta_i)$,s_{α_i}为$sin(\alpha_i)$，c为cos

根据 DH 模型，很容易获得 6-DOF 机械手前向运动学的模型，模型可以表示如下：

$$
{}^{0}_{6}\mathrm{T} = \begin{bmatrix} a_{11} & a_{12} & a_{13} & p_x \\ a_{21} & a_{22} & a_{23} & p_y \\ a_{31} & a_{32} & a_{33} & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix}
$$

$$
{}^{0}_{6}\mathrm{T} = {}^{0}_{1}\mathrm{T}\,{}^{1}_{2}\mathrm{T}\,{}^{2}_{3}\mathrm{T}\,{}^{3}_{4}\mathrm{T}\,{}^{4}_{5}\mathrm{T}\,{}^{5}_{6}\mathrm{T}
$$

${}^{0}_{6}\mathrm{T}$ 表示机械臂末端执行器相对于惯性坐标系的位置矩阵；${}^{i}_{i-1}\mathrm{T}$ 表示相邻坐标系的齐次变换矩阵。

机械手的 IK 可以通过获得机械手末端执行器的姿态矩阵并求解机械手关节角度的变化来求解。本文所用 6-DOF 机械手的 DH 参数列于表中。

连杆 i	d_i(mm)	α(Deg)	a_i(mm)	θ_i(Deg)	θ_i(Degree)
1	450	-90	150	θ_1	-170~170
2	0	0	570	θ_2	-110~155
3	0	-90	130	θ_3	-80~210
4	640	90	0	θ_4	-190~190
5	0	-90	0	θ_5	-50~230
6	95	0	0	θ_6	-360~360

图 5.8.1 DOF 机械手的 DH 参数

5.8.3、python 代码

五自由度运动学求解器

```
-------------------------------------------
@https://www.dice.com.cn/
'''
import math
import numpy as np
from transform import *
from pose import Pose


# 设置 Numpy 打印精度
np.set_printoptions(precision=3, suppress=True)


class Arm5DoFKinematic:
    '''Arm5DoF 协作机械臂运动学'''
    # 关节角度下限，单位：弧度
    JOINT_ANGLE_LOWERB = np.float64([-1.5708, -2.65, -0.7854, -2.0944, -1.5708])
    # 关节角度上限制，单位：弧度
    JOINT_ANGLE_UPPERB = np.float64([1.5708, -0.2269, 2.18166, 2.0944, 1.5708])
```

```
# 默认关节角度 单位: 弧度
# 注: 角度转弧度
JOINT_ANGLE_DEFAULT = np.radians([0, -90.0, 90.0, 0, 0.0, 0.0])
# 关节编号
JOINT1 = 0
JOINT2 = 1
JOINT3 = 2
JOINT4 = 3
JOINT6 = 4
# DH 表 连杆长度定义，单位 mm
# 注: 参数仅供参考，根据自己机械臂的连杆参数进行调整
D1 = 70.00
A1 = 0.00
A2 = 80.00
A3 = 76.15
A4 = 185.40
# 预定义常量
PI = math.pi
MINUS_PI = - math.pi
PI_M_2 = math.pi * 2.0 # pi*2
PI_D_2 = math.pi / 2.0 # pi/2
MIMUS_PI_D_2 = -1.0 * PI_D_2 # -pi/2


def __init__(self, is_debug=False):
    '''初始化'''
    self.is_debug = is_debug
    # 当前关节角度赋值为默认角度
    self.cur_joint_angle = np.copy(self.JOINT_ANGLE_DEFAULT)
```

```
    def forward_kinematic_v1(self, joint_angle):
        '''正向运动学-工具坐标系'''
        # 提取关节角度
        theta1, theta2, theta3, theta4, theta6 = joint_angle
        # 关节与关节之间的变换
        T01 = Transform.dhmat(0, 0, theta1, self.D1)
        T12 = Transform.dhmat(self.MIMUS_PI_D_2, self.A1, theta2, 0)
        T23 = Transform.dhmat(0, self.A2, theta3, 0)
        T34 = Transform.dhmat(0, self.A3, theta4, 0)
        T45 = Transform.dhmat(0, self.A4, self.MIMUS_PI_D_2, 0)
        T56 = Transform.dhmat(self.MIMUS_PI_D_2, 0, theta6, 0)
        # 计算机械臂基坐标系到腕关节坐标系的变换
        T02 = T01.dot(T12)
        T03 = T02.dot(T23)
        T04 = T03.dot(T34)
        T05 = T04.dot(T45)
        T06 = T05.dot(T56)
        # 创建位姿对象
        pose = Pose()
        pose.set_transform_matrix(T06)
        return pose, [T01, T02, T03, T04, T05, T06]

    def forward_kinematic_v2(self, joint_angle, return_type="XYZ_Pitch_Roll"):
        '''正向运动学-工具坐标系
        @joint_angle: 关节角度
        @return_type: 返回类型
            - "Pose": 返回 Pose 对象
            - "XYZ_Pitch_Roll": 返回[x6, y6, z6, pitch, roll]
    '''
```

```
# 提取关节角度
theta1, theta2, theta3, theta4, theta6 = joint_angle
# 复合角度
theta23 = theta2 + theta3
theta234 = theta23 + theta4
# 预先计算三角函数
s1, s2, s3, s4, s6 = np.sin(joint_angle)
c1, c2, c3, c4, c6 = np.cos(joint_angle)
s23, c23 = np.sin(theta23), np.cos(theta23)
s234, c234 = np.sin(theta234), np.cos(theta234)
# 三维坐标
x60 = self.A1 + self.A2*c2 + self.A3*c23 + self.A4*c234
x6 = x60 * c1
y6 = x60 * s1
z6 = self.D1 - self.A2*s2 - self.A3*s23 - self.A4*s234
if return_type == "Pose":
    # 求解旋转矩阵
    r11 = s1*s6 + c1*s234*c6
    r12 = s1*c6 - c1*s234*s6
    r13 = c1*c234
    r21 = s1*s234*c6 - c1*s6
    r22 = - s1*s234*s6 - c1*c6
    r23 = s1*c234
    r31 = c234*c6
    r32 = - c234*s6
    r33 = - s234
    # 创建位姿对象
    pose = Pose()
    # 构造旋转矩阵
```

```
                rmat = np.float32([
                    [r11, r12, r13],
                    [r21, r22, r23],
                    [r31, r32, r33]])
                pose.set_rotation_matrix(rmat)
                pose.set_position(x6, y6, z6)
                return pose
            else:
                #  返回[x6, y6, z6, pitch]
                #  计算俯仰角 Pitch
                # beta = \theta_{234} + \pi/2
                pitch = theta234 + self.PI_D_2
                if pitch > self.PI_M_2:
                    pitch -= self.PI_M_2
                #  计算横滚角 Roll
                # gamma = \theta_6 + \pi
                roll = theta6 + self.PI
                if roll > self.PI:
                    roll -= self.PI_M_2
                return [x6, y6, z6, pitch, roll]


    def angle_unique(self, angle_list):
        '''得到不重复的角度列表'''
        return np.unique(np.round(np.float32(angle_list), 8))


    def is_joint_angle_legal(self, theta, joint_index, is_debug=None):
        '''判断关节角度是否合法'''
        if is_debug is None:
            is_debug = self.is_debug
```

```
        lowerb = self.JOINT_ANGLE_LOWERB[joint_index]
        upperb = self.JOINT_ANGLE_UPPERB[joint_index]
        is_legal = theta >= lowerb and theta <= upperb
        if is_debug:
            print(f"关节索引:  {joint_index}")
            print(f"关节角度: {theta}  是否合法: {is_legal}")
            print(f"下限: {lowerb},  上限: {upperb}")
        return is_legal

    def joint_angle_filter(self, theta_list, joint_index, is_debug=None):
        '''关节角度过滤器'''
        if is_debug is None:
            is_debug = self.is_debug
        # 过滤唯一的候选角度
        theta_list = np.unique(np.float64(theta_list))
        # 筛选后的候选关节角度
        theta_list_filter = []
        for theta in theta_list:
            if self.is_joint_angle_legal(theta, joint_index, is_debug):
                theta_list_filter.append(theta)
        if is_debug:
            print(f"关节索引:  {joint_index}")
            print(f"过滤前的关节角度: {theta_list}")
            print(f"过滤后的关节角度:  {theta_list_filter}")
        return theta_list_filter

    def inverse_kinematic(self, x6, y6, z6, pitch, roll,\
            last_joint_angle = None, is_debug=None):
        '''逆向运动学'''
```

```
        if is_debug is None:
            is_debug = self.is_debug
        # 上一次的关节角度
        if last_joint_angle is None:
            last_joint_angle = self.JOINT_ANGLE_DEFAULT
        # 候选关节角度
        candi_joint_angle_list = []
        # 求解 theta6
        theta6 = roll - self.PI
        if theta6 < self.MINUS_PI:
            theta6 += self.PI_M_2
        if not self.is_joint_angle_legal(theta6, self.JOINT6, is_debug):
            return []
        # 求解 theta234
        theta234 = pitch - self.PI_D_2
        c234 = math.cos(theta234)
        s234 = math.sin(theta234)
        # 求解关节 1 的角度
        theta1_list = None
        if x6 == 0.0 and y6 == 0.0:
            theta1_1 = last_joint_angle[self.JOINT1]
            theta1_list = [theta1_1]
        else:
            theta1_1 = math.atan2(y6, x6)
            theta1_2 = theta1_1 + np.pi
            if theta1_2 > np.pi:
                theta1_2 -= self.PI_M_2
            theta1_list = [theta1_1, theta1_2]
        for theta1 in self.joint_angle_filter(\
```

```
        theta1_list, self.JOINT1, is_debug):
    c1 = math.cos(theta1)
    s1 = math.sin(theta1)
    # 求解关节 6 在关节 1 坐标系下的坐标
    x_16 = c1*x6 + s1*y6
    z_16 = z6 - self.D1
    # 定义 b1, b2
    b1 = x_16 - self.A1 - self.A4 * c234
    b2 = -z_16   - self.A4 * s234
    c3 = (b1**2 + b2**2 - self.A2**2 - self.A3**2) / (2*self.A2*self.A3)
    # 数值原因 cos(theta3)可能会再 1.0 附近
    # 为了被判定为非法，因此做下特殊处理
    c3 = np.round(c3, 10)
    if is_debug:
        print(f"cos(theta3) = {c3}")
    s3_pw2 = np.round(1.0 - c3**2, 10)
    if np.abs(c3) > 1.0 or np.abs(s3_pw2) > 1.0:
        if is_debug:
            # 非法 cos 值，工作区达不到
            print("非法 cos 值 工作区达不到")
        continue

    s3_abs = math.sqrt(s3_pw2)
    theta3_1 = math.atan2(s3_abs, c3)
    theta3_2 = math.atan2(-s3_abs, c3)
    # 求解 theta2
    for theta3 in self.joint_angle_filter(\
        [theta3_1, theta3_2], self.JOINT3, is_debug):
        if is_debug:
```

```
                print(f"theta3 = {theta3}")
            K1 = self.A2 + self.A3 * c3
            K2 = self.A3 * math.sin(theta3)
            theta2 = math.atan2(b2, b1) - math.atan2(K2, K1)
            if not self.is_joint_angle_legal(theta2, self.JOINT2, is_debug):
                continue
            # 求解 theta4
            theta4 = theta234 - theta2 - theta3
            if not self.is_joint_angle_legal(theta4, self.JOINT4, is_debug):
                continue
            candi_joint_angle_list.append([theta1, theta2, theta3, theta4, theta6])
    return candi_joint_angle_list
```

5.9、机械臂关节弧度及末端姿态控制

5.9.1、机械臂关节弧度及末端姿态控制介绍

此实验的目的是把积木从中间方形区域移动到四周不同的方形区域。首先把黄色的方块放到中间的方形区域中，再依次运行代码单元到第六个单元（从中间方形区域位置抓取一块积木放到右下角方形积木块的位置上），此时机械臂会自动抓取放在中间方形区域的方块，然后放到右下角方形区域内，然后返回准备位置。运行第七个代码单元前，需要把红色方块放到中间的方形区域内，再运行第七个单元（从中间方形区域位置抓取一块积木放到右上角方形区域的位置上），这样红色的方块也会被抓取到右上角的方形区域，其他的方块操作方式也是一样。

图 5.9.1 不同颜色方块对应的位置

5.9.2、实验步骤

第 1 步：进入'whitebear'文件夹里的'jetson 基础控制项目'文件夹，找到'9.机械臂夹方块.ipynb'程序，双击用 JupyterLab 打开程序:

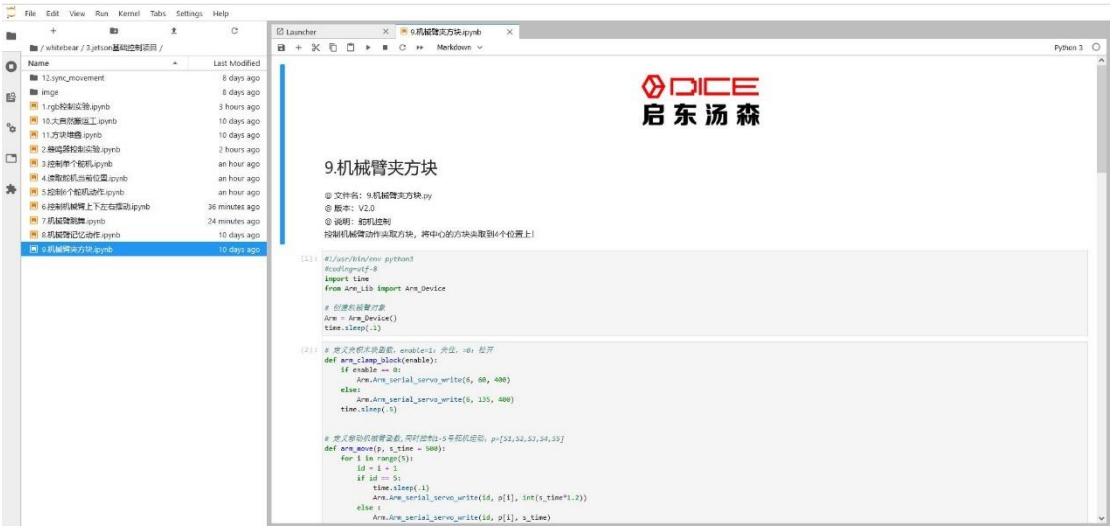


图 5.9.1 机械臂夹方块程序截图

第 2 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 3 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：运行程序后机械臂会把中间方形区域内的方块夹取到四周的位置，期间需要手动依次把木块放置到中间方形区域。

5.9.3、python 代码

```
9.机械臂夹方块

@ 文件名：9.机械臂夹方块.py
@ 版本：V2.0
@ 说明：舵机控制
控制机械臂动作夹取方块，将中心的方块夹取到 4 个位置上！

#!/usr/bin/env python3
#coding=utf-8
import time
from Arm_Lib import Arm_Device

# 创建机械臂对象
Arm = Arm_Device()
time.sleep(.1)

# 定义夹积木块函数，enable=1：夹住，=0：松开
def arm_clamp_block(enable):
    if enable == 0:
```

```
            Arm.Arm_serial_servo_write(6, 60, 400)
        else:
            Arm.Arm_serial_servo_write(6, 135, 400)
        time.sleep(.5)


# 定义移动机械臂函数,同时控制 1-5 号舵机运动，p=[S1,S2,S3,S4,S5]
def arm_move(p, s_time = 500):
    for i in range(5):
        id = i + 1
        if id == 5:
            time.sleep(.1)
            Arm.Arm_serial_servo_write(id, p[i], int(s_time*1.2))
        else :
            Arm.Arm_serial_servo_write(id, p[i], s_time)
        time.sleep(.01)
    time.sleep(s_time/1000)

# 机械臂向上移动
def arm_move_up():
    Arm.Arm_serial_servo_write(2, 90, 1500)
    Arm.Arm_serial_servo_write(3, 90, 1500)
    Arm.Arm_serial_servo_write(4, 90, 1500)
    time.sleep(.1)

# 定义不同位置的变量参数
p_mould = [90, 130, 0, 0, 90]
p_top = [90, 80, 50, 50, 270]
p_Brown = [90, 68, 28, 28, 270]

p_Yellow = [62, 35, 54, 56, 270]
p_Red = [116, 35, 54, 56, 270]

p_Green = [132, 86, 16, 26, 270]
p_Blue = [44, 86, 16, 26, 270]

# 让机械臂移动到一个准备抓取的位置
arm_clamp_block(0)
arm_move(p_mould, 1000)
time.sleep(1)

# 从中间方形区域位置抓取一块积木放到黄色积木块的位置上。
arm_move(p_top, 1000)
arm_move(p_Brown, 1000)
```

```
arm_clamp_block(1)

arm_move(p_top, 1000)
arm_move(p_Yellow, 1000)
arm_clamp_block(0)



arm_move(p_mould, 1000)

time.sleep(1)

# 从中间方形区域位置抓取一块积木放到红色积木块的位置上。
arm_move(p_top, 1000)
arm_move(p_Brown, 1000)
arm_clamp_block(1)

arm_move(p_top, 1000)
arm_move(p_Red, 1000)
arm_clamp_block(0)

arm_move_up()
arm_move(p_mould, 1100)

time.sleep(1)

# 从中间方形区域位置抓取一块积木放到绿色积木块的位置上。
arm_move(p_top, 1000)
arm_move(p_Brown, 1000)
arm_clamp_block(1)

arm_move(p_top, 1000)
arm_move(p_Green, 1000)
arm_clamp_block(0)

arm_move_up()
arm_move(p_mould, 1100)

time.sleep(1)

# 从中间方形区域位置抓取一块积木放到蓝色积木块的位置上。
arm_move(p_top, 1000)
arm_move(p_Brown, 1000)
arm_clamp_block(1)
```

```
arm_move(p_top, 1000)
arm_move(p_Blue, 1000)
arm_clamp_block(0)

arm_move_up()
arm_move(p_mould, 1100)

time.sleep(1)

del Arm    # 释放掉 Arm 对象
```

5.10、大自然搬运工（需更新）

5.10.1、实验简介

实验开始前，把不同颜色的四个积木从下到上按照蓝绿红黄的顺序叠起来放到中间方形区域上，然后运行代码，机械臂会按照夹取第四层的方块放到黄色区域，夹取第三层的方块放到红色区域，夹取第二层的方块放到绿色区域，夹取最底层的方块放到蓝色区域的顺序依次执行。

摆放积木块的方式如下图所示：

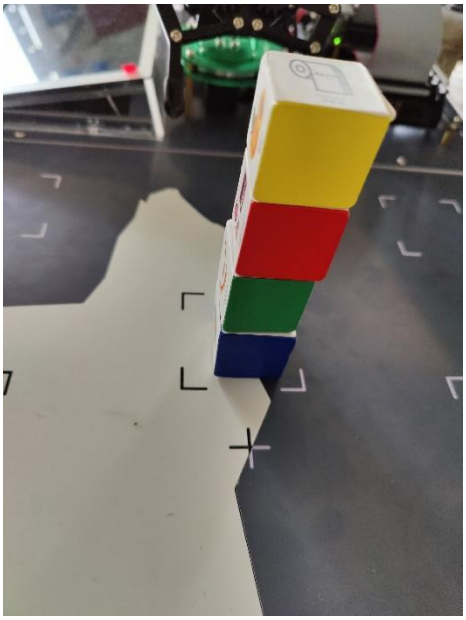


图 5.10.1 方块堆放

第 1 步：进入‘whitebear’文件夹里的‘jetson 基础控制项目’文件夹，找到‘10.大自然搬运工.ipynb’程序，双击用 JupyterLab 打开程序：

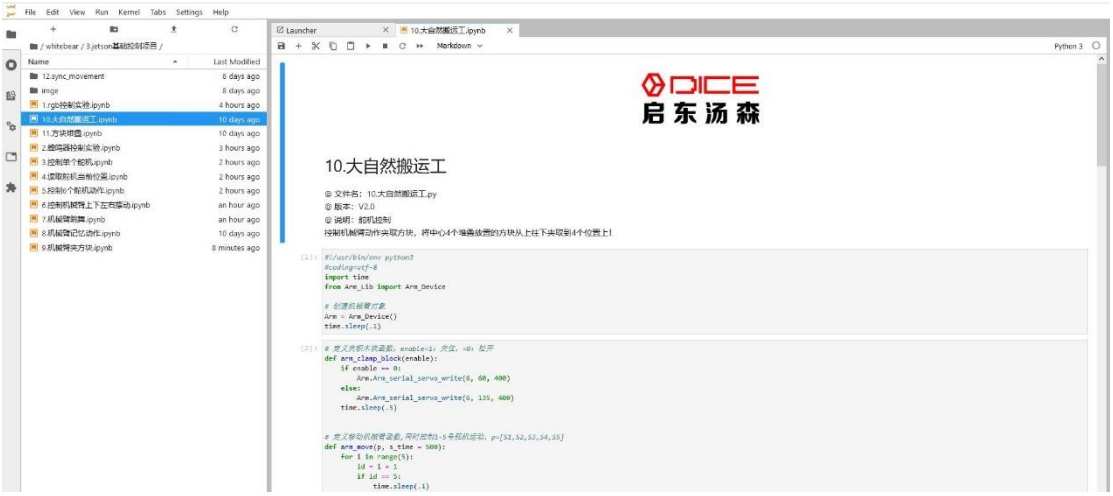


图 5.10.2 大自然搬运工程序截图

第 2 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 3 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：运行代码后，机械臂会从上至下依次夹取方块，放到四周的 4 个位置上。

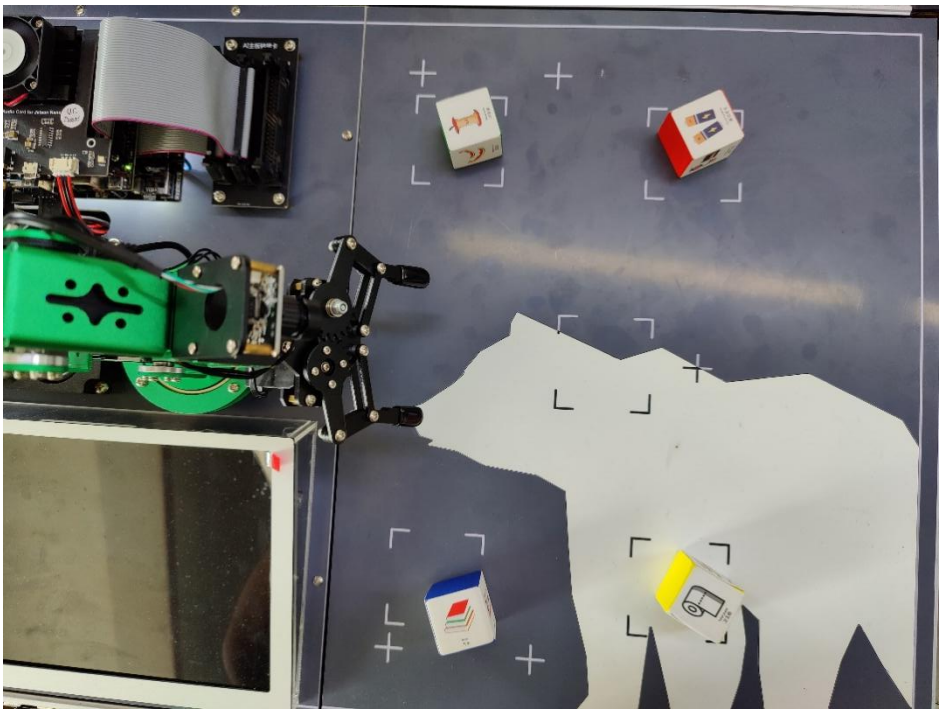


图 5.10.2 大自然搬运工实验效果

5.10.2、python 代码

```
10.大自然搬运工

@ 文件名：10.大自然搬运工.py
@ 版本：V2.0
@ 说明：舵机控制
控制机械臂动作夹取方块，将中心 4 个堆叠放置的方块从上往下夹取到 4 个位置上！

#!/usr/bin/env python3
#coding=utf-8
import time
from Arm_Lib import Arm_Device

# 创建机械臂对象
Arm = Arm_Device()
time.sleep(.1)
```

```
# 定义夹积木块函数，enable=1：夹住，=0：松开
def arm_clamp_block(enable):
    if enable == 0:
        Arm.Arm_serial_servo_write(6, 60, 400)
    else:
        Arm.Arm_serial_servo_write(6, 135, 400)
    time.sleep(.5)


# 定义移动机械臂函数,同时控制 1-5 号舵机运动，p=[S1,S2,S3,S4,S5]
def arm_move(p, s_time = 500):
    for i in range(5):
        id = i + 1
        if id == 5:
            time.sleep(.1)
            Arm.Arm_serial_servo_write(id, p[i], int(s_time*1.2))
        else :
            Arm.Arm_serial_servo_write(id, p[i], s_time)
        time.sleep(.01)
    time.sleep(s_time/1000)

# 机械臂向上移动
def arm_move_up():
    Arm.Arm_serial_servo_write(2, 90, 1500)
    Arm.Arm_serial_servo_write(3, 90, 1500)
    Arm.Arm_serial_servo_write(4, 90, 1500)
    time.sleep(.1)

# 定义不同位置的变量参数
p_mould = [90, 130, 0, 0, 90]
p_top = [90, 80, 50, 50, 270]

p_Yellow = [62, 35, 54, 56, 270]
p_Red = [116, 35, 54, 56, 270]

p_Green = [132, 86, 16, 26, 270]
p_Blue = [44, 86, 16, 26, 270]


p_layer_4 = [90, 80, 54, 13, 270]
p_layer_3 = [90, 74, 43, 19, 270]
p_layer_2 = [90, 70, 38, 23, 270]
p_layer_1 = [90, 68, 28, 28, 270]
```

```
# 让机械臂移动到一个准备抓取的位置
arm_clamp_block(0)
arm_move(p_mould, 1000)
time.sleep(1)

# 搬运第四层的积木块到黄色区域
arm_move(p_top, 1000)
arm_move(p_layer_4, 1000)
arm_clamp_block(1)

arm_move(p_top, 1000)
arm_move(p_Yellow, 1000)
arm_clamp_block(0)

time.sleep(.1)

arm_move_up()
arm_move(p_mould, 1100)

# time.sleep(1)

# 搬运第三层的积木块到红色区域
arm_move(p_top, 1000)
arm_move(p_layer_3, 1000)
arm_clamp_block(1)

arm_move(p_top, 1000)
arm_move(p_Red, 1000)
arm_clamp_block(0)

time.sleep(.1)

arm_move_up()
arm_move(p_mould, 1100)

# time.sleep(1)

# 搬运第二层的积木块到绿色区域
arm_move(p_top, 1000)
arm_move(p_layer_2, 1000)
arm_clamp_block(1)

arm_move(p_top, 1000)
```

```
arm_move(p_Green, 1000)
arm_clamp_block(0)

time.sleep(.1)

arm_move_up()
arm_move(p_mould, 1100)

# time.sleep(1)

#  搬运第一层的积木块到蓝色区域
arm_move(p_top, 1000)
arm_move(p_layer_1, 1000)
arm_clamp_block(1)

arm_move(p_top, 1000)
arm_move(p_Blue, 1000)
arm_clamp_block(0)

time.sleep(.1)

arm_move_up()
arm_move(p_mould, 1100)

# time.sleep(1)

del Arm    #  释放掉 Arm 对象
```

5.11、方块堆叠（需更新）

5.11.1、实验简介

此实验的目的与上个实验‘大自然搬运工’正好相反，是把不同四周的积木块按顺序夹取后堆叠到中间的方形区域中。

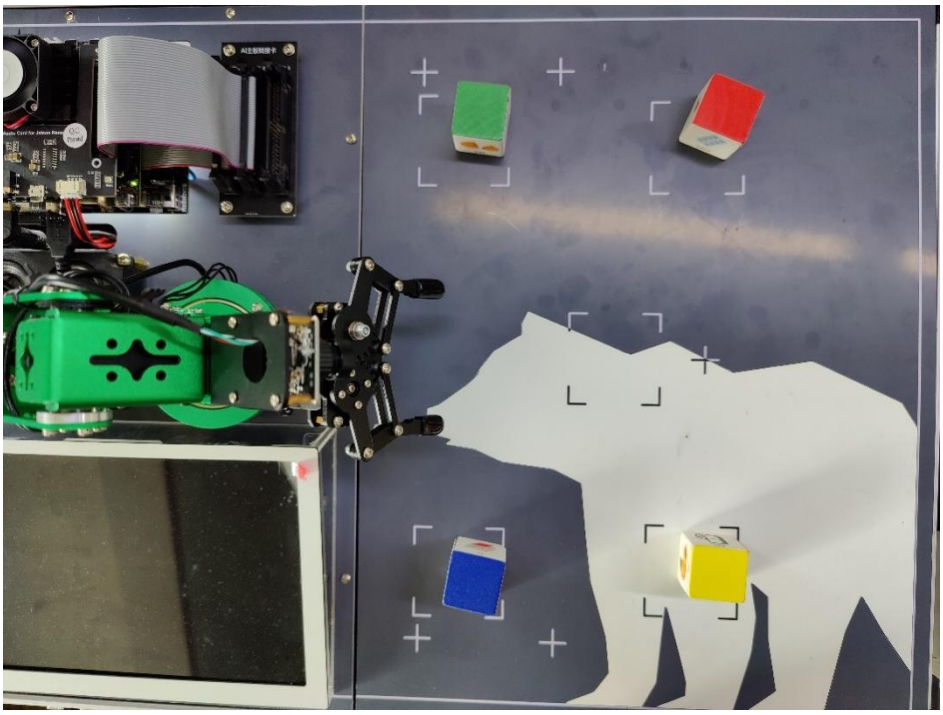


图 5.11.1 实验前方块摆放

5.11.2、实验步骤

第 1 步：进入‘whitebear’文件夹里的‘jetson 基础控制项目’文件夹，找到‘11.方块堆叠.ipynb’程序，双击用 JupyterLab 打开程序:

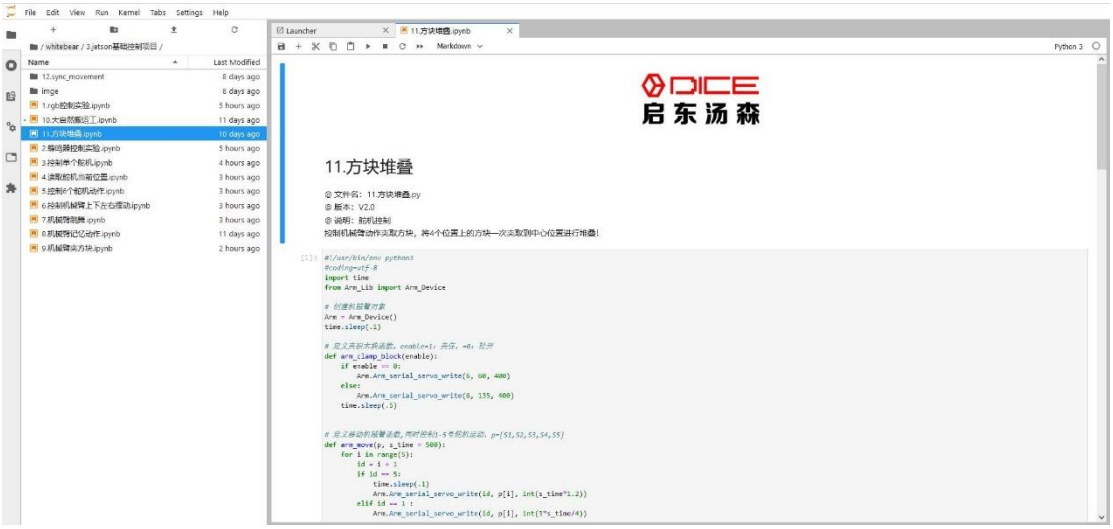


图 5.11.2 方块堆叠实验程序截图

第 2 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 3 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：运行程序后，机械臂会依次将四周的方块夹取到中间的方形区域内，堆叠起来。

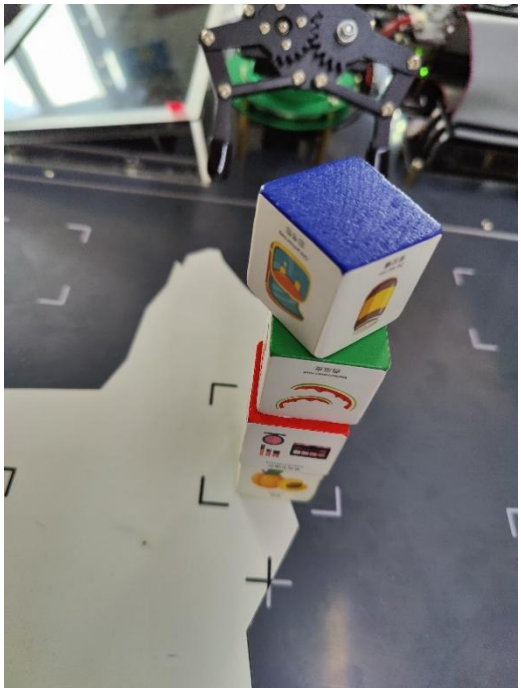


图 5.11.3 堆叠实验效果

5.11.3、python 代码

11.方块堆叠

@ 文件名：11.方块堆叠.py
@ 版本：V2.0
@ 说明：舵机控制
控制机械臂动作夹取方块，将 4 个位置上的方块一次夹取到中心位置进行堆叠!

```
#!/usr/bin/env python3
#coding=utf-8
import time
from Arm_Lib import Arm_Device

# 创建机械臂对象
Arm = Arm_Device()
time.sleep(.1)

# 定义夹积木块函数，enable=1：夹住，=0：松开
def arm_clamp_block(enable):
    if enable == 0:
        Arm.Arm_serial_servo_write(6, 60, 400)
    else:
        Arm.Arm_serial_servo_write(6, 135, 400)
    time.sleep(.5)


# 定义移动机械臂函数,同时控制 1-5 号舵机运动，p=[S1,S2,S3,S4,S5]
def arm_move(p, s_time = 500):
    for i in range(5):
        id = i + 1
        if id == 5:
            time.sleep(.1)
            Arm.Arm_serial_servo_write(id, p[i], int(s_time*1.2))
        elif id == 1 :
            Arm.Arm_serial_servo_write(id, p[i], int(3*s_time/4))

        else:
            Arm.Arm_serial_servo_write(id, p[i], int(s_time))
        time.sleep(.01)
    time.sleep(s_time/1000)

# 定义不同位置的变量参数
p_mould = [90, 130, 0, 0, 90]
p_top = [90, 80, 50, 50, 270]
```

```
p_Yellow = [62, 35, 54, 56, 270]
p_Red = [116, 35, 54, 56, 270]

p_Green = [132, 86, 16, 26, 270]
p_Blue = [44, 86, 16, 26, 270]



p_layer_4 = [90, 80, 54, 13, 270]
p_layer_3 = [90, 74, 43, 19, 270]
p_layer_2 = [90, 70, 38, 23, 270]
p_layer_1 = [90, 68, 28, 28, 270]

# 让机械臂移动到一个准备抓取的位置
arm_clamp_block(0)
arm_move(p_mould, 1000)
time.sleep(1)

# 夹取黄色区域的方块堆叠到中间最底层的位置。
arm_move(p_top, 1000)
arm_move(p_Yellow, 1000)
arm_clamp_block(1)

arm_move(p_top, 1000)
arm_move(p_layer_1, 1000)
arm_clamp_block(0)

time.sleep(.1)

arm_move(p_mould, 1100)

# time.sleep(1)

# 夹取红色区域的方块堆叠到中间第二层的位置。
arm_move(p_top, 1000)
arm_move(p_Red, 1000)
arm_clamp_block(1)

arm_move(p_top, 1000)
arm_move(p_layer_2, 1000)
arm_clamp_block(0)

time.sleep(.1)

arm_move(p_mould, 1100)
```

```
# time.sleep(1)

#  夹取绿色区域的方块堆叠到中间第三层的位置。
arm_move(p_top, 1000)
arm_move(p_Green, 1000)
arm_clamp_block(1)

arm_move(p_top, 1000)
arm_move(p_layer_3, 1000)
arm_clamp_block(0)

time.sleep(.1)

arm_move(p_mould, 1100)

# time.sleep(1)

#  夹取蓝色区域的方块堆叠到中间第四层的位置。
arm_move(p_top, 1000)
arm_move(p_Blue, 1000)

arm_clamp_block(1)

arm_move(p_top, 1000)
arm_move(p_layer_4, 1000)
arm_clamp_block(0)

time.sleep(.1)

arm_move(p_mould, 1100)

# time.sleep(1)

del Arm    #  释放掉 Arm 对象
```

5.12、两台机械臂同步运作（需更新）

5.12.1、通讯原理

本次实验的原理是把其中一台机械臂作为服务器，开启 TCP 网络服务，接收数据，执行操作；而另一台机械臂作为客户端，也就是操作器，读取依次读取所有舵机的角度值，然后立即通过 TCP 服务把所有舵机的角度发送给服务器；此时，客户端一直快速读取舵机的角度值，并且发送给服务器，服务器接收并设置立即更新所有舵机的角度，以这样的方式来实现两个机械臂远程同步动作的功能。

5.12.2、实验前准备

1.本次实验涉及到两台硬件相同的机械臂，除了运行的程序不同，其他都是可以是相同的。如果机械臂使用的是出厂自带的系统镜像，请先结束掉开机自启动的出厂大程序，再进行此实验。

2.保证两台机械臂都连接到同一个局域网，即连接到同一个路由器（有线连接或者无线连接都可以），并且在局域网内能够通讯。

3.由于这里涉及到两台机械臂，所以为了容易区分它们，我们把作为服务器的那台机械臂叫做 A 机械臂（被控制方），把作为控制器的那台机械臂叫做 B 机械臂（自由掰动方）。由于 A 机械臂的动作依赖于 B 机械臂的运动，为了避免刚启动程序时，A 机械臂运动范围过大，所以在启动程序前，请将 A 机械臂和 B 机械臂最好都调节到竖直向上的状态。

5.12.3、实验步骤

第 1 步：A 机械臂用终端进入‘whitebear’文件夹里的‘jetson 基础控制项目’文件夹中的‘12.sync_movement’文件夹。

```
cd /whitebear/ jetson 基础控制项目/12.sync_movement
```

A 机械臂运行代码‘A_arm_follow.py’

```
python3 A_arm_follow.py
```

并记录下打印出来的 IP 地址

```
jetson@jetson-desktop:~$ cd /home/jetson/Dofbot/3.ctrl_Arm/12.sync_movement/
jetson@jetson-desktop:~/Dofbot/3.ctrl_Arm/12.sync_movement$ ls
A_arm_follow.py  B_arm_ctrl.py
jetson@jetson-desktop:~/Dofbot/3.ctrl_Arm/12.sync_movement$ python3 A_arm_follow.py
192.168.2.101:6100
start_tcp_server
```

图 5.12.1 打印出的 IP 地址

第 2 步：打开 **B 机械臂**的终端，把上一步记下的 IP 地址替换到 B_arm_ ctrl.py 中的 ip 地址位置

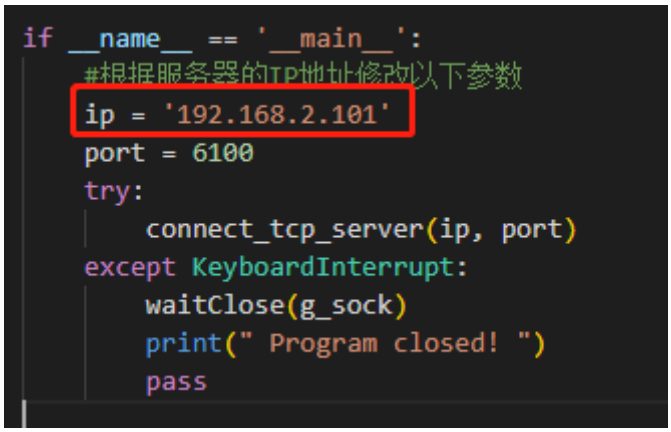


图 5.12.2 修改 B 机械臂程序中的 IP 地址

第 3 步：修改完成之后保存退出，并运行'B_arm_ ctrl.py 程序'

```
python3 B_arm_ctrl.py
```

注意一定要先运行 A 机械臂的 A_arm_follow.py 程序，再运行 B 机械臂的 B_arm_ctrl.py 程序。B 机械臂终端会打印连接信息并且打印发送的数据，同时两台机械臂的通讯正式开始。

实验结果：此时可以看到 A 机械臂的状态已经和 B 机械臂的状态一样了，并且可以通过掰动 B 机械臂的姿态，从而修改 A 机械臂的姿态。通过这样的 TCP 传输命令，就可以达到两台机械臂同步动作的效果。A 机械臂被控制是有角度限制的，而 B 机械臂自动活动的没有角度限制，加入 B 机械臂的 1 号舵机掰动超过 180 度，则 A 会一直停留在 180 度直到 B 机械臂的角度调节回合理范围才可以继续运动。

5.12.4、python 代码

A_arm_follow.py

```
#coding=utf-8
import socket
import os
import time
from Arm_Lib import Arm_Device



global g_init
g_init = False
Arm = Arm_Device()
```

```
# 获取本机 IP
def getLocalip():
    ip = os.popen("/sbin/ifconfig eth0 | grep 'inet' | awk '{print $2}'").read()
    ip = ip[0 : ip.find('\n')]
    if(ip == ''):
        #读取 WLAN 的 IP
        ip = os.popen("/sbin/ifconfig wlan0 | grep 'inet' | awk '{print $2}'").read()
        ip = ip[0 : ip.find('\n')]
        if(ip == ''):
            ip = 'x.x.x.x'
    return ip


#协议解析部分
def Analysis(socket, cmd):
    print(cmd)
    #       $20111222333444555666#
    try :
        check = cmd[1:3]
        if check == '20' and len(cmd) == 22:
            s1_angle = int(cmd[3:6])
            s2_angle = int(cmd[6:9])
            s3_angle = int(cmd[9:12])
            s4_angle = int(cmd[12:15])
            s5_angle = int(cmd[15:18])
            s6_angle = int(cmd[18:21])

            Arm.Arm_serial_servo_write6(s1_angle, s2_angle, s3_angle, s4_angle, s5_angle, s6_angle,
500)
        else:
            print("cmd data error! continue...")
    except:
        print("cmd data error! continue...")

#socket TCP 通信建立
def start_tcp_server(ip, port):
    global g_init, g_socket
    g_init = True
    print('start_tcp_server')
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    sock.bind((ip, port))
```

```
    sock.listen(5)
    while True:
        conn, address = sock.accept()
        g_socket = conn
        print("client connected:" + str(address))
        while True:
            cmd = g_socket.recv(1024).decode(encoding="utf-8")
            if not cmd:
                print("Client Disconnected!")
                break
            # print('     [-]cmd:', cmd, len(cmd))
            index1 = cmd.find("$")
            index2 = cmd.find("#")
            if index1 >= 0 and index2 > index1:
                Analysis(g_socket, cmd[index1:index2+1])



#关闭 socket
def waitClose():
    global g_init, g_socket
    g_init = False
    g_socket.close()


if __name__ == '__main__':
    try:
        port = 6100
        if g_init == False:
            while(True):
                ip = getLocalip()
                print("%s:%d" % (ip, port))
                if(ip == "x.x.x.x"):
                    continue
                if(ip != "x.x.x.x"):
                    break
        start_tcp_server(ip, port)
    except KeyboardInterrupt:
        waitClose()
        print(" Program closed! ")
        pass
```

B_arm_ ctrl.py

```
#coding=utf-8
```

```
import socket
import os
import time
from Arm_Lib import Arm_Device



global g_sock
Arm = Arm_Device()


# socket 客户端
def connect_tcp_server(ip, port):
    global g_sock
    print("Connecting server...")
    g_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    g_sock.connect((ip, port))
    print("Connected!")
    time.sleep(1)
    Arm.Arm_serial_set_torque(0)
    last_angle = [0, 0, 0, 0, 0, 0]
    while True:
        angle = [0, 0, 0, 0, 0, 0]
        for i in range(6):
            id = i + 1
            angle[i] = Arm.Arm_serial_servo_read(id)
            if angle[i] == None:
                time.sleep(.001)
                angle[i] = Arm.Arm_serial_servo_read(id)
                if angle[i] == None:
                    angle[i] = last_angle[i]

            last_angle[i] = angle[i]
            time.sleep(.001)

        pos1str = "%03d" % angle[0]
        pos2str = "%03d" % angle[1]
        pos3str = "%03d" % angle[2]
        pos4str = "%03d" % angle[3]
        pos5str = "%03d" % angle[4]
        pos6str = "%03d" % angle[5]
        data = "$20"+pos1str+pos2str+pos3str+pos4str+pos5str+pos6str+"#"
        print(data)
        b_data = bytes(data, encoding = "utf8")
```

```
            g_sock.send(b_data)
            time.sleep(.1)



#关闭 socket
def waitClose(sock):
    sock.close()
    Arm.Arm_serial_set_torque(1)


if __name__ == '__main__':
    #根据服务器的 IP 地址修改以下参数
    ip = '192.168.2.100'
    port = 6100
    try:
        connect_tcp_server(ip, port)
    except KeyboardInterrupt:
        waitClose(g_sock)
        print(" Program closed! ")
        pass
```

6、AI基础图像处理

6.1、安装和使用 Matplotlib、Pyplot 和 Numpy

6.1.1、介绍

在学习人工智能课程之前，我们来一起学习一下 Matplotlib、Pyplot 和 Numpy 库，因为我们进入人工智能之后，会发现人工智能与数据的关系非常密切，而数据则需要通过一些简单实用的库来进行庞大的数据处理，而且庞大的数据基本上以数组或者矩阵的形式，所以我们需要学习 Numpy、Matplotlib 等 Pyplot 库。Matplotlib 是 Python 的绘图库。它可与 NumPy 一起使用，提供了一种有效的 MatLab 开源替代方案。它也可以和图形工具包一起使用，如 PyQt 和 wxPython.numpy(Numerical Python extensions)是一个第三方的 Python 包，用于科学计算。这个库的前身是 1995 年就开始开发的一个用于数组运算的库。经过了长时间的发展，基本上成了绝大部分 Python 科学计算的基础包，当然也包括所有提供 Python 接口的深度学习框架。

6.1.2、安装 Matplotlib、Pyplot 和 Numpy 库

在 Jetson Nano 联网的情况下，我们采用 apt-get 来进行安装：

```
sudo apt-get install python3-matplotlib
```

在中途会弹出提示是否继续执行，我们输入‘Y’，确定继续执行！

安装完成后，我们可以测试一下，通过 Xshell 软件或 JupyterLab 登录到 Jetson Nano 系统上或者直接在 Jetson Nano 上运行终端，运行 python3，然后输入如下代码，确保安装成功。

```
import matplotlib.pyplot as plt
import numpy as np
np.__version__
```

```
jetson@jetson-desktop:~$ python3
Python 3.6.9 (default, Jan 26 2021, 15:33:00)
[GCC 8.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import matplotlib.pyplot as plt
Matplotlib created a temporary config/cache directory at /tmp/matplotlib-06hl0dme because the default path
(/home/jetson/.config/matplotlib) is not a writable directory; it is highly recommended to set the MPLCONFI
GDIR environment variable to a writable directory, in particular to speed up the import of Matplotlib and t
o better support multiprocessing.
>>> import numpy as np
>>> np._version_
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: module 'numpy' has no attribute '_version_'
>>> np.__version__
'1.18.5'
>>>
```

图 6.1.1 运行代码

相关教程，我们可以去官网 API 去进行学习：https://matplotlib.org/api/pyplot_api.html

6.1.3 使用 Matplotlib、Pyplot 和 Numpy 库

下面我们通过一个简单的案例进行说明，我们这里是在 JupyterLab 上绘制 sin 函数和 cos 函数，然后进行一些相关操作。

第 1 步：进入‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘1.安装和使用 Matplotlib,Pyplot 和 Numpy.ipynb’程序，双击用 JupyterLab 打开程序。

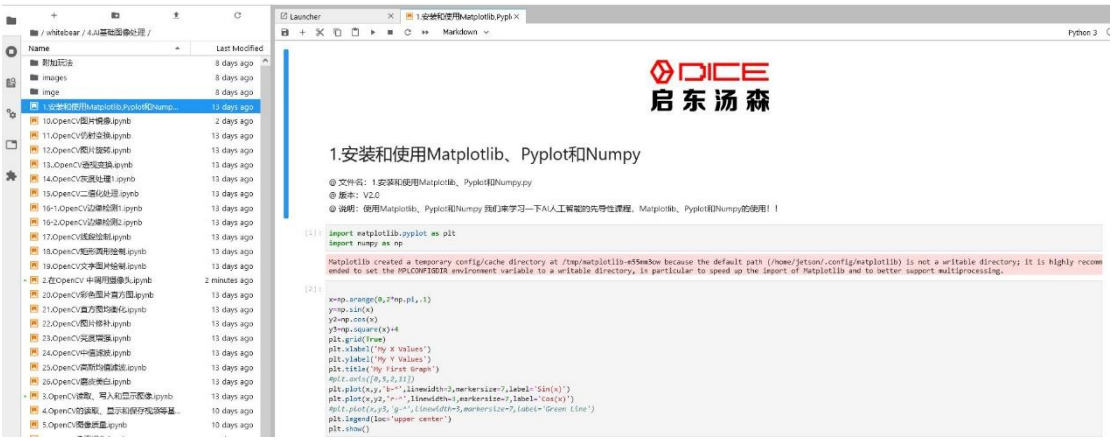


图 6.1.2 程序截图

第 2 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 4 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：屏幕上绘制出 sin 函数和 cos 函数。

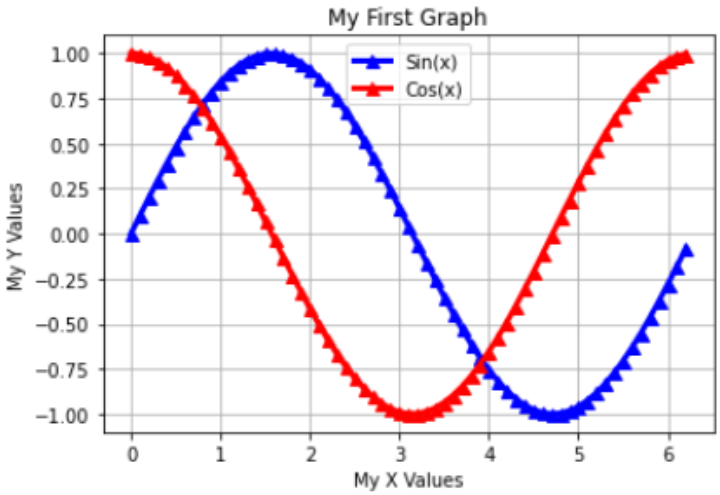


图 6.1.3 安装和使用 Matplotlib,Pyplot 和 Numpy.ipynb 实验效果图

6.1.4、python 代码

1. 安装和使用 Matplotlib、Pyplot 和 Numpy

@ 文件名：1.安装和使用 Matplotlib、Pyplot 和 Numpy.py

@ 版本：V2.0

@ 说明：使用 Matplotlib、Pyplot 和 Numpy 我们来学习一下 AI 人工智能的先导性课程，Matplotlib、Pyplot 和 Numpy 的使用！！

[1]:

```
import matplotlib.pyplot as plt
import numpy as np
```

[2]:

```
x=np.arange(0,2*np.pi,.1)
y=np.sin(x)
y2=np.cos(x)
y3=np.square(x)+4
plt.grid(True)
plt.xlabel('My X Values')
plt.ylabel('My Y Values')
plt.title('My First Graph')
#plt.axis([0,5,2,11])
plt.plot(x,y,'b-^',linewidth=3,markersize=7,label='Sin(x)')
plt.plot(x,y2,'r-^',linewidth=3,markersize=7,label='Cos(x)')
#plt.plot(x,y3,'g-^',linewidth=3,markersize=7,label='Green Line')
plt.legend(loc='upper center')
plt.show()
```

6.2、在 OpenCV 中运行摄像头

6.2.1、介绍

Jetson Nano 可以支持多种摄像头，包括 USB 摄像头、Stereolabs ZED 摄像头、IMX219 摄像头以及 IP 摄像头。在白熊 AI 平台上我们使用的是 USB 摄像头，把摄像头的 USB 线插上主机的 USB 口即可使用。设备出厂时均已安装。摄像头固定在机械臂上。

图 6.2.1 USB 摄像头

6.2.2、摄像头的使用

使用前我们可以测试以下摄像头是否存在，我们可以在终端中输入如下命令查询：

```
ls -l /dev/video*
```

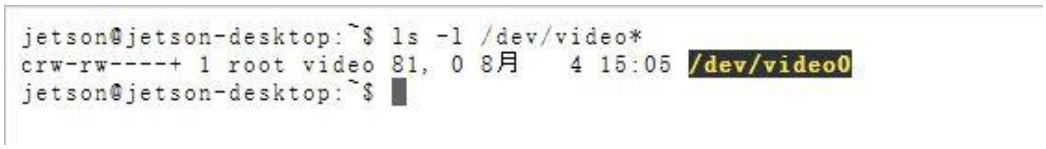


图 6.2.2 查询摄像头

如果能看到/dev/video0 设备的话，说明摄像头已经连接。

出厂程序会占用摄像头，摄像头使用前请先关闭出厂程序

```
sh ~/whitebear/kill_YahboomArm.sh
```

第 1 步：进入‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘2.在 OpenCV 中调用摄像头.ipynb’程序，双击用 JupyterLab 打开程序：

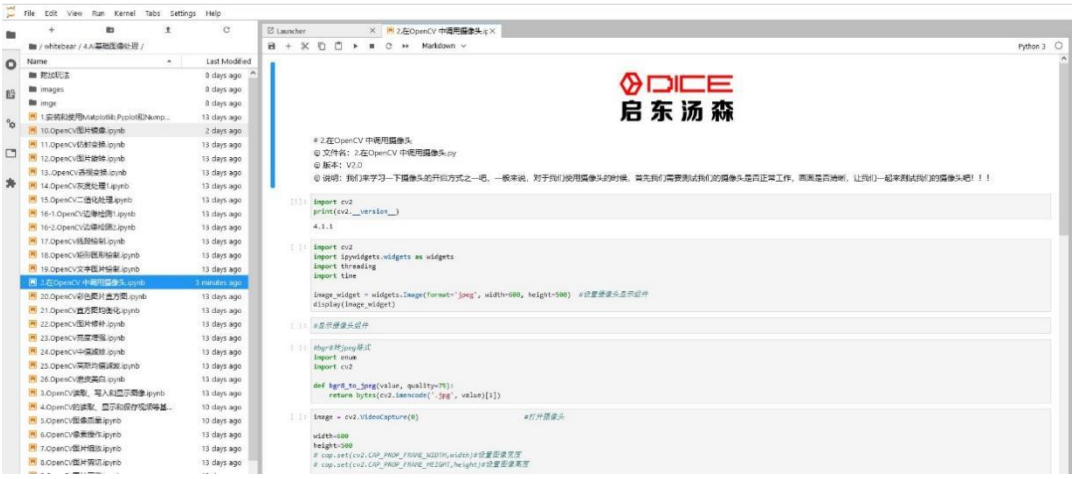


图 6.2.3 运行摄像头程序截图

第 2 步：运行代码，点击运行一步一步运行代码，具体操作见前面所学 JupyterLab 操作。

第 3 步：运行完成之后，我们可以清除内核的运行，否则会一直在后台运行，从而干扰其他程序运行！具体操作请看实验 4.1 第 4 步。

实验结果：运行程序就能看到，第一个单元下方会打印 OpenCV 版本，并且程序窗口会出现一个显示窗口，并显示摄像头拍摄到的内容。

```
[1]: import cv2
     print(cv2.__version__)

     4.1.1
```

图 6.2.4 显示 OpenCV 版本

#显示摄像头组件

图 6.2.5 摄像头画面

6.2.3、python 代码

2.在 OpenCV 中调用摄像头

@ 文件名：2.在 OpenCV 中调用摄像头.py

@ 版本：V2.0

@ 说明：我们来学习一下摄像头的开启方式之一吧，一般来说，对于我们使用摄像头的时候，首先我们需要测试我们的摄像头是否正常工作，画面是否清晰，让我们一起来测试我们的摄像头吧！！！

```
import cv2
print(cv2.__version__)

import cv2
import ipywidgets.widgets as widgets
import threading
import time

image_widget = widgets.Image(format='jpeg', width=600, height=500)  #设置摄像头显示组件
display(image_widget)

#显示摄像头组件
#bgr8 转 jpeg 格式
import enum
import cv2
```

```
def bgr8_to_jpeg(value, quality=75):
    return bytes(cv2.imencode('.jpg', value)[1])

image = cv2.VideoCapture(0)                                    #打开摄像头
width=600
height=500
ret, frame = image.read()       #读取摄像头数据
image_widget.value = bgr8_to_jpeg(frame)

try:
    while 1:
        ret, frame = image.read()
        image_widget.value = bgr8_to_jpeg(frame)
        time.sleep(0.010)
except KeyboardInterrupt:
    print(" Program closed! ")
    pass

image.release()                 #使用完成对象记住释放掉对象，不然下一个程序使用这个对象模块会被占用，导致无法使用
```

6.3、OpenCV 读取、写入和显示图像

6.3.1、介绍

在这里你将学会怎样读入一幅图像，怎样显示一幅图像，以及如何保存一幅图像

- 你将使用到如下函数：cv2.imread(),cv2.imwrite()及 ipywidgets 的显示控件
- 如果你愿意的话，可以使用 Matplotlib 显示一幅图片

6.3.2、程序位置

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘3. OpenCV 读取、写入和显示图像.ipynb’程序，双击用 JupyterLab 打开程序。

6.3.3、读入图像

使用函数 cv2.imread()读入图像。这幅图像应该在此程序的工作路径，或者给函数提供完整路径，第二个参数是要告诉函数应该如何读取这幅图片。

- cv2.IMREAD_COLOR:读入一副彩色图像。图像的透明度会被忽略，这是默认参数。
- cv2.IMREAD_GRAYSCALE:以灰度模式读入图像
- cv2.IMREAD_UNCHANGED:读入一幅图像，并且包括图像的 alpha 通道

```
import numpy as np
import cv2
# 在灰度上加载彩色图像
input=cv2.imread('./images/whitebear.jpg')
```

6.3.4、显示图像

显示图像需要调用到 ipywidgets 组件和 IPython.display 的显示组件，具体使用方式如下所示：

```
# 载入显示库
from Jetson Nano import bgr8_to_jpeg
```

```
import ipywidgets.widgets as widgets
from IPython.display import display
image=widgets.Image(format='jpeg',width=640,height=480)
display(image)
# 显示图像
image.value=bgr8_to_jpeg(input)
```

```
[4]: # 载入显示库
from jetcam.utils import bgr8_to_jpeg
import ipywidgets.widgets as widgets
from IPython.display import display
image = widgets.Image(format='jpeg', width=640, height=480)
display(image)

# 显示图像
image.value = bgr8_to_jpeg(input)
```

图 6.3.1 在程序中显示图像

6.3.5、保存图像

使用函数 cv2.imwrite()来保存一个图像。首先需要一个文件名，之后才是你要保存的图像。

```
cv2. imwrite('output. png', input)
```

6.3.6、使用 Matplotlib 显示图像

Matplotlib 是 Python 的绘图库，可为你提供多种绘图方法。你将在接下来的文章中看到它们。在这里，你将学习如何使用 Matplotlib 显示图像。你可以使用 Matplotlib 缩放图像，保存图像等。

```
#显示图像
import matplotlib.pyplot as plt
img = cv2.cvtColor(input,cv2.COLOR_BGR2RGB)
plt.imshow(img),plt.title('whitebearTest image',color='blue')
plt.xticks(I),plt.yticks(I)#隐藏 x 轴和 y 轴上的刻度值
plt.show()
```

图 6.3.2 Matplotlib 显示图像

注意：OpenCV 加载的彩色图像处于 BGR 模式。但是 Matplotlib 以 RGB 模式显示。所以需要转换为 RGB 的模式，在这个程序中就调用了 cv2.cvtColor(img,cv2.COLOR_BGR2RGB)进行转换。

6.4、OpenCV 读取、显示和保存视频

6.4.1、介绍

学会读取视频文件，显示视频，保存视频文件

- 学会从摄像头获取并显示视频
- 使用到这些函数：cv2.VideoCapture(),cv2.VideoWrite()

6.4.2、程序位置

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘4.OpenCV 读取、显示和保存视频.ipynb’程序，双击用 JupyterLab 打开程序。

6.4.3、用摄像头捕获视频

我们经常需要使用摄像头捕获实时图像。OpenCV 为这中应用提供了一个非常简单的接口。让我们使用摄像头来捕获一段视频，并把它转换成灰度视频显示出来。从这个简单的任务开始吧。为了获取视频，你应该创建一个 VideoCapture 对象。他的参数可以是设备的索引号，或者是一个视频文件。设备索引号就是在指定要使用的摄像头。之后，你就可以一帧一帧的捕获视频了。但是最后，别忘了停止捕获视频。

1、用摄像头捕获视频

```
# 载入库
import cv2
#from jetcam.utils import bgr8_to_jpeg
# 线程函数操作库
import threading # 线程
import ctypes
import inspect
#bgr8 转 jpeg 格式
import enum
#import cv2


def bgr8_to_jpeg(value, quality=75):
    return bytes(cv2.imencode('.jpg', value)[1])
```

1.1、 创建显示控件

```
import cv2
import ipywidgets.widgets as widgets
import threading
import time
import traitlets
from IPython.display import display

image_widget = widgets.Image(format='jpeg', width=600, height=500)   #设置摄像头显示组件
display(image_widget)
```

1.2、 线程的结束代码

```
def _async_raise(tid, exctype):
    tid = ctypes.c_long(tid)
    if not inspect.isclass(exctype):
        exctype = type(exctype)
    res = ctypes.pythonapi.PyThreadState_SetAsyncExc(tid, ctypes.py_object(exctype))
    if res == 0:
        raise ValueError("invalid thread id")
    elif res != 1:
        ctypes.pythonapi.PyThreadState_SetAsyncExc(tid, None)
        raise SystemError("PyThreadState_SetAsyncExc failed")

def stop_thread(thread):
    _async_raise(thread.ident, SystemExit)
```

1.3、 动态显示摄像头视频

```
image = cv2.VideoCapture(0)                                    #打开摄像头

width=600
height=500

ret, frame = image.read()       #读取摄像头数据
image_widget.value = bgr8_to_jpeg(frame)

def Video_display():
    while True:
        ret, frame = image.read()
        image_widget.value = bgr8_to_jpeg(frame)
    #
    cap.release()
```

```
t = threading.Thread(target=Video_display)
t.setDaemon(True)
t.start()
```

1.4、 结束线程

```
stop_thread(t)
```

cap.read0 返回一个布尔值（True/False).如果帧读取的是正确的，就是 True.所以最后你可以通过检查他的返回值来查看视频文件是否已经到了结尾。有时 cap 可能不能成功的初始化摄像头设备。这种情况下上面的代码会报错。你可以使用 cap.isOpened0,来检查是否成功初始化了。如果返回值是 True,那就没有问题。否则就要使用函数 cap.open0.你可以使用函数 cap.get(propId)来获得视频的一些参数信息。这里 propId 可以是 0 到 18 之间的任何整数。每一个数代表视频的一个属性。

其中的一些值可以使用 cap.set(propId,value)来修改，value 就是你想要设置成的新值。例如，我可以使用 cap.get(3)和 cap.get(4)来查看每一帧的宽和高。默认情况下得到的值是 640X480.但是我可以使用 ret=cap.set(3,320)和 ret=cap.set(4,240)来把宽和高改成 320X240。

6.4.4、从文件播放视频

它与从相机捕获相同，只是用视频文件名更改摄像机索引。另外，在显示框架时，请使用适当的时间延时。如果太小，则视频将非常快，而如果太大，则视频将变得很慢（这就是显示慢动作的方式）。正常情况下 25 毫秒就可以了。

2、从文件中播放视频

```
# -*- coding: utf-8 -*-
from jetcam.utils import bgr8_to_jpeg
import numpy as np
import cv2

import traitlets
import ipywidgets.widgets as widgets
from IPython.display import display
face_image = widgets.Image(format='jpeg', width=640, height=480)
display(face_image)

cap = cv2.VideoCapture('./images/walking.avi')
```

```
while cap.isOpened():
    ret, frame = cap.read()
    # 如果正确读取帧，ret 为 True
    if not ret:
        print("Can't receive frame (stream end?). Exiting ...")
        break
    frame = cv2.flip(frame,4)
    face_image.value = bgr8_to_jpeg(frame)

cap = cv2.VideoCapture('./images/walking.avi')

while cap.isOpened():
    ret, frame = cap.read()
    # 如果正确读取帧，ret 为 True
    if not ret:
        print("Can't receive frame (stream end?). Exiting ...")
        break
    frame = cv2.flip(frame,4)
    face_image.value = bgr8_to_jpeg(frame)
```

6.4.5、保存视频

所以我们捕捉一个视频，一帧一帧地处理，我们想要保存这个视频。对于图像，它非常简单，只需使用 cv.imwrite().这里还需要做一些工作。这次我们创建一个 VideoWriter 对象。我们应该指定输出文件名（例如：output.avi).然后我们应该 指定 FourCC 代码（详见下一段）。然后传递帧率的数量和帧大小。最后一个是颜色标志。如果为 True, 编码器期望颜色帧，否则它与灰度帧一起工作。FourCC:http://en.wikipedia.org/wiki/FourCC 是用于指定视频编解码器的 4 字节代码。可用代码 列表可在 fourcc.org 中：http://www.fourcc.org/codecs.php 找到。它取决于平合。遵循编解码器对 我来说效果很好。在 Fedora 中：DIVX,XVID,MJPG,X264,WMV1,WMV2.(最好使用 XVID.MJPG 会生 成大尺寸的视频。X264 会生成非常小的尺寸的视频）在 Windows 中：DIVX(尚待测试和添加）在 OSX 中：MJPG(.mp4),DIVX (.avi),X264（.mkv)。FourCC 代码作为 MJPG 的 cv.VideoWriter_fourcc ('M', 'J', 'P', 'G') orcv.VideoWriter_fourcc(*'MJPG') 传递。在从摄像机捕获的代码下面，沿垂直方向翻转每一帧并保存。

3.保存视频

```
# 载入库
```

```
import cv2
from jetcam.utils import bgr8_to_jpeg
# 线程函数操作库
import threading # 线程
import ctypes
import inspect
#bgr8 转 jpeg 格式
import enum
#import cv2

def bgr8_to_jpeg(value, quality=75):
    return bytes(cv2.imencode('.jpg', value)[1])



import cv2
import ipywidgets.widgets as widgets
import threading
import time
import traitlets
from IPython.display import display

image_widget = widgets.Image(format='jpeg', width=600, height=500)  #设置摄像头显示组件
display(image_widget)



image = cv2.VideoCapture(0)                                    #打开摄像头

width=600
height=500

ret, frame = image.read()       #读取摄像头数据
image_widget.value = bgr8_to_jpeg(frame)


# 定义编解码器并创建 VideoWriter 对象
fourcc = cv2.VideoWriter_fourcc(*'XVID')
```

```
out = cv2.VideoWriter('output.avi', fourcc, 20.0, (640, 480))
while image.isOpened():
    ret, frame = image.read()
    if not ret:
        print("Can't receive frame (stream end?). Exiting ...")
        break
    frame = cv2.flip(frame, 4)
    # 写翻转的框架
    out.write(frame)
    image_widget.value = bgr8_to_jpeg(frame)

# 完成工作后释放所有内容
cap.release()
out.release()
```

6.5、OpenCV 图像质量和像素操作

6.5.1、OpenCV 压缩图像

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘5.OpenCV 图像质量.ipynb’程序，双击用 JupyterLab 打开程序。

在这个实验里，我们可以通过这个程序改变图像的质量，一共提供了两种压缩方式，一种是有损压缩还有无损压缩，下面是压缩方式：

- cv2.imwrite('whitebearTest.jpg', img, [cv2.IMWRITE_JPEG_QUALITY, 50])
- cv2.CV_IMWRITE_JPEG_QUALITY： 设置图片格式为.jpeg 或者.jpg 的图片质量，其值为 0---100（数值越大质量越高），默认 95
- cv2.CV_IMWRITE_WEBP_QUALITY： 设置图片的格式为.webp 格式的图片质量，值为 0--100
- cv2.CV_IMWRITE_PNG_COMPRESSION： 设置.png 格式的压缩比，其值为 0--9（数值越大，压缩比越大），默认为 3

Python 代码：

```
import cv2
img = cv2.imread('./images/whitebear.jpg',1)
cv2.imwrite('./images/whitebearTest.jpg', img, [cv2.IMWRITE_JPEG_QUALITY, 10])
#1M 100k 10k 0-100  有损压缩
```

```
# 1  无损  2  透明度属性
import cv2
img = cv2.imread('./images/whitebear.jpg',1)
cv2.imwrite('./images/whitebearTest.png', img, [cv2.IMWRITE_PNG_COMPRESSION,2])
# jpg 0  压缩比高 0-100          png 0  压缩比低 0-9
```

```
#bgr8 转 jpeg 格式
import enum
import cv2
```

```
def bgr8_to_jpeg(value, quality=75):
    return bytes(cv2.imencode('.jpg', value)[1])
```

```
import ipywidgets.widgets as widgets

image_widget1 = widgets.Image(format='jpg', )
image_widget2 = widgets.Image(format='jpg', )
# create a horizontal box container to place the image widget next to eachother
image_container = widgets.HBox([image_widget1, image_widget2])

# display the container in this cell's output
display(image_container)

img1 = cv2.imread('./images/whitebearTest.jpg',1)
img2 = cv2.imread('./images/whitebearTest.png',1)
image_widget1.value = bgr8_to_jpeg(img1)
image_widget2.value = bgr8_to_jpeg(img2)
```

实验结果如下：

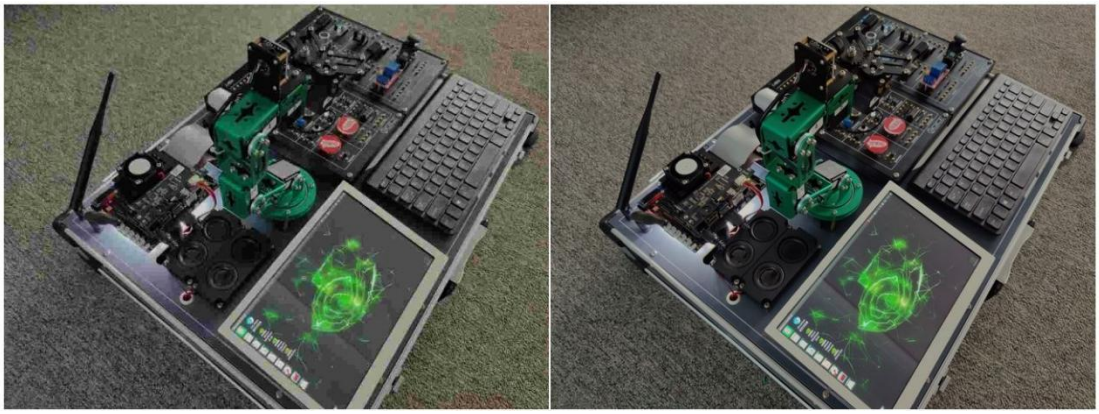


图 6.5.1 两种压缩方式压缩结果

6.5.2、OpenCV 像素操作

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘6.OpenCV 像素操作.ipynb’程序，双击用 JupyterLab 打开程序。

在这个实验里，我们可以给任意位置改成新的像素颜色，这里我们先读取图像，然后赋值一个区域为白色。

Python 代码:

```
import cv2

img = cv2.imread('./images/whitebear.jpg',1)
(b,g,r) = img[100,100]
print(b,g,r)# bgr
#10 100 --- 110 100
i=j=0
for j in range(1,500):
    img[i,j] = (255,255,255)
    for i in range(1,500):
        img[i,j] = (255,255,255)

# cv2.imshow('image',img)
# cv2.waitKey(0) #1000 ms
```

```
#bgr8 转 jpeg 格式
import enum
import cv2

def bgr8_to_jpeg(value, quality=75):
    return bytes(cv2.imencode('.jpg', value)[1])
```

```
import ipywidgets.widgets as widgets

image_widget1 = widgets.Image(format='jpg', )
image_widget2 = widgets.Image(format='jpg', )
# create a horizontal box container to place the image widget next to eachother
image_container = widgets.HBox([image_widget1, image_widget2])

# display the container in this cell's output
display(image_container)

img1 = cv2.imread('./images/whitebear.jpg',1)

image_widget1.value = bgr8_to_jpeg(img1)   #原始的
image_widget2.value = bgr8_to_jpeg(img)    #经过像素操作的
```

实验结果如下：

图 6.5.2 像素操作实验结果

6.6、OpenCV 几何变换基础

6.6.1、OpenCV 图片缩放

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘7.OpenCV 图片缩放.ipynb’程序，双击用 JupyterLab 打开程序。

在 OpenCV 中，实现图像缩放的函数是：cv2.resize(InputArray src, OutputArray dst, Size, fx, fy, interpolation)。

参数解释：

InputArray src	**输入图片**
OutputArray dst	输出图片
Size	输出图片尺寸
fx, fy	沿 x 轴，y 轴的缩放系数
interpolation	插入方式

选项所用的插值方法：

INTER_NEAREST	**最近邻插值**
INTER_LINEAR	双线性插值（默认设置）
INTER_AREA	使用像素区域关系进行重采样。
INTER_CUBIC	4x4 像素邻域的双三次插值
INTER_LANCZOS4	8x8 像素邻域的 Lanczos 插值

注意：

1.输出尺寸格式为（宽，高）

2.默认的插值方法为：双线性插值

Python 代码：

```
import cv2
import matplotlib.pyplot as plt  # Python 的 2D 绘图库


# 读入原图片
img = cv2.imread('./images/whitebear.jpg')
# 打印出图片尺寸
print(img.shape)
# 将图片高和宽分别赋值给 x，y
x, y = img.shape[0:2]
```

```
# 显示原图
#cv.imshow('OriginalPicture', img)

# 缩放到原来的二分之一，输出尺寸格式为（宽，高）
img_test1 = cv2.resize(img, (int(y / 2), int(x / 2)))
# cv2.imshow('resize0', img_test1)
# cv2.waitKey()

# 最近邻插值法缩放
# 缩放到原来的四分之一
img_test2 = cv2.resize(img, (0, 0), fx=0.25, fy=0.25, interpolation=cv2.INTER_NEAREST)
# cv.imshow('resize1', img_test2)
# cv.waitKey()
# cv.destroyAllWindows()
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
dst1 = cv2.cvtColor(img_test1, cv2.COLOR_BGR2RGB)
dst2 = cv2.cvtColor(img_test2, cv2.COLOR_BGR2RGB)

# 显示原始图像
plt.imshow(img)
plt.show()
```

原始图像为 800*1080：

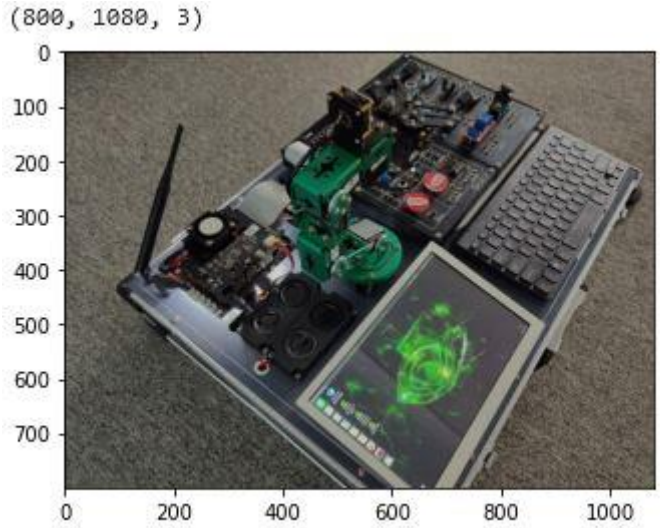


图 6.6.1 原始图像大小

```
# 显示缩放 1/2
plt.imshow(dst1)
plt.show()
```

实验结果如下： 图像尺寸为 400*540，缩放了二分之一

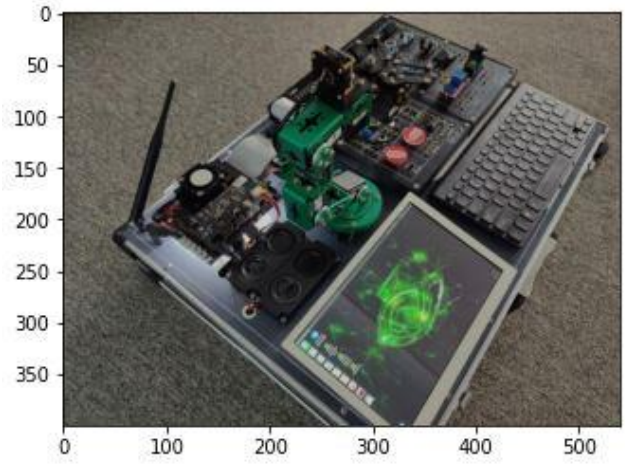
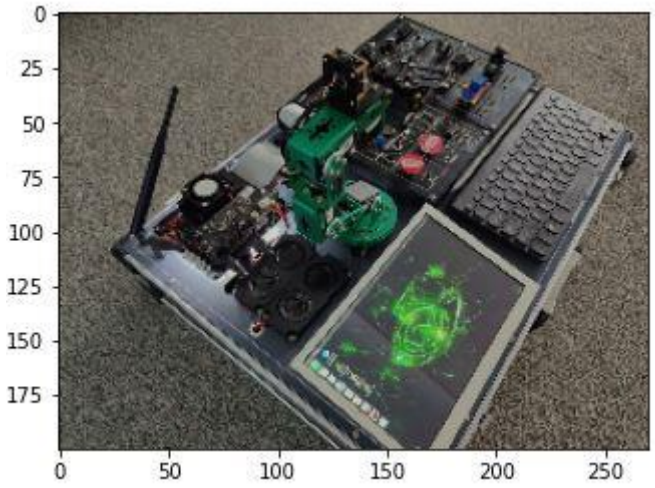


图 6.6.2 图像缩放 1/2

```
# 显示缩放 1/4 邻插值法缩放
plt.imshow(dst2)
plt.show()
```

实验结果如下：图像尺寸为 200*270，缩放了四分之一



6.6.3 图像缩放 1/4

下面提供了一个 matplotlib 的一个小例子，参考教程：https://www.runoob.com/numpy/numpy-matplotlib.html

```
import numpy as np
from matplotlib import pyplot as plt
```

```
x = np.arange(1,11)
y =  2 * x +  5
plt.title("Matplotlib demo")
plt.xlabel("x axis caption")
plt.ylabel("y axis caption")
plt.plot(x,y)
plt.show()
```

实验结果如下：

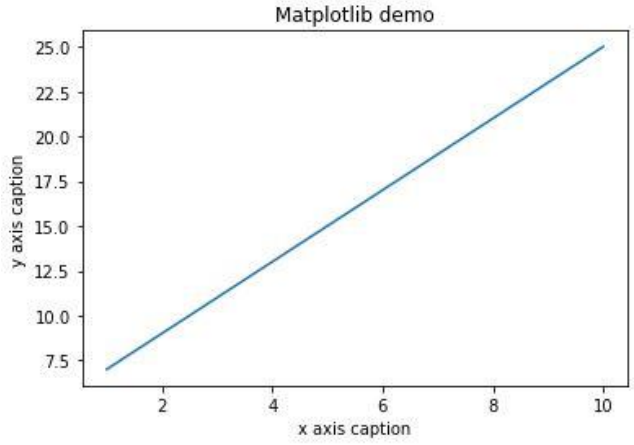


图 6.6.4 matplotlib demo

6.6.2、OpenCV 图片剪切

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘8.OpenCV 图片剪切.ipynb’程序，双击用 JupyterLab 打开程序。

图片剪切首先读取图像，然后再数组中获取像素点区域。下面代码中选取形区域 X：300-500 Y：500-700，注意图像尺寸是 800*1080，所以选择的区域不要超过此分辨率。

Python 代码：

```
import cv2
img = cv2.imread('./images/whitebear.jpg', 1)

dst = img[1000:1400,1200:1650]  #这里选取矩形区域 X：1200-1650  Y：1000-1400

#cv2.imshow('image',dst)
#cv2.waitKey(0)
```

```
#bgr8 转 jpeg 格式
import enum
import cv2

def bgr8_to_jpeg(value, quality=75):
    return bytes(cv2.imencode('.jpg', value)[1])
```

```
import ipywidgets.widgets as widgets

image_widget1 = widgets.Image(format='jpg', )
image_widget2 = widgets.Image(format='jpg', )

# display the container in this cell's output
display(image_widget1)
display(image_widget2)

img1 = cv2.imread('./images/whitebear.jpg',1)

image_widget1.value = bgr8_to_jpeg(img1)   #原始图像
image_widget2.value = bgr8_to_jpeg(dst)    #剪切的图像
```

实验结果如下：

图 6.6.5 实验结果

6.6.3、OpenCV 图片平移

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘9.OpenCV 图片平移.ipynb’程序，双击用 JupyterLab 打开程序。

通过转换矩阵 M 实现将原始图像 src 转换为目标图像 dst：

dst(x, y) = src(M11x + M12y+M13, M21x+M22y+M23)

将原始图像 src 向右侧移动 200、向下移动 100 个像素，则其对应关系为：

dst(x, y) = src(x+200, y+100)

将上述表达式补充完整，即：

dst(x, y) = src(1 • x + 0 • y + 200, 0 • x + 1 • y + 100)

根据上述表达式，可以确定对应的转换矩阵 M 中各个元素的值为：

- M11=1
- M12=0
- M13=200
- M21=0
- M22=1
- M23=100

将上述值代入转换矩阵 M，得到：

$$M = \begin{bmatrix} 1 & 0 & 200 \\ 0 & 1 & 100 \end{bmatrix}$$

下面直接使用转换矩阵 M 调用函数 cv2.warpAffine()完成图像的平移

Python 代码：

```
# 1 API 2  算法原理  3  源代码
import cv2
import numpy as np
img = cv2.imread('./images/whitebear.jpg',1)
#cv2.imshow('src',img)
```

```
imgInfo = img.shape

height = imgInfo[0]

width = imgInfo[1]

####

matShift = np.float32([[1,0,200],[0,1,100]])# 2*3

dst = cv2.warpAffine(img,matShift,(height,width))#1 data 2 mat 3 info
```

```
#bgr8 转 jpeg 格式

import enum

import cv2


def bgr8_to_jpeg(value, quality=75):

    return bytes(cv2.imencode('.jpg', value)[1])
```

```
import ipywidgets.widgets as widgets


image_widget1 = widgets.Image(format='jpg', )

image_widget2 = widgets.Image(format='jpg', )

# create a horizontal box container to place the image widget next to eachother

image_container = widgets.HBox([image_widget1, image_widget2])


# display the container in this cell's output

display(image_container)

#display(image_widget2)


img1 = cv2.imread('./images/whitebear.jpg',1)


image_widget1.value = bgr8_to_jpeg(img1)

image_widget2.value = bgr8_to_jpeg(dst)
```

实验结果如下：

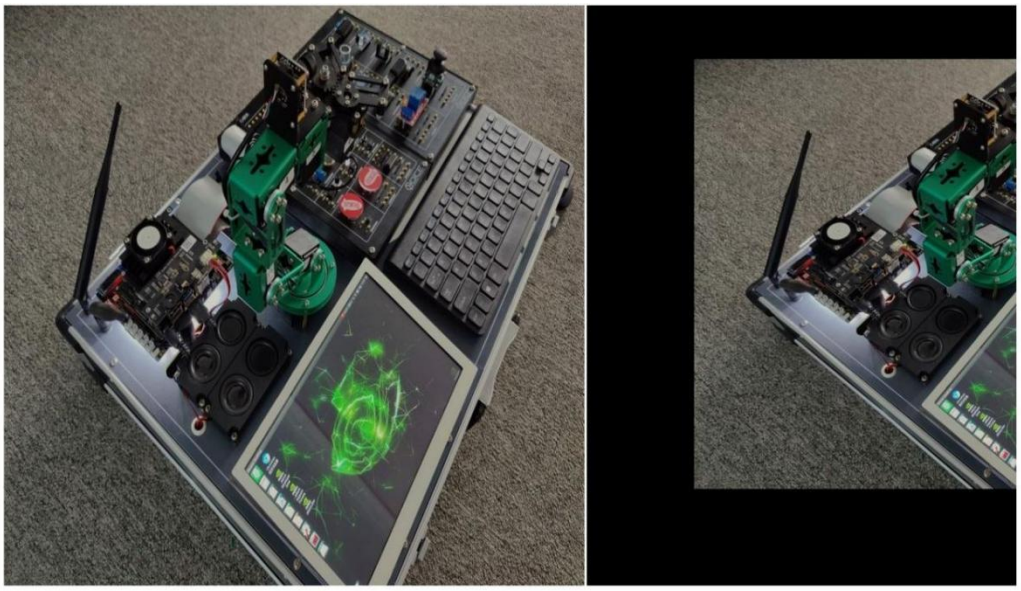


图 6.6.6 平移效果

6.6.4、OpenCV 图片镜像

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘10.OpenCV 图片镜像.ipynb’程序，双击用 JupyterLab 打开程序。

图像的镜像变换分为两种：水平镜像和垂直镜像。水平镜像以图像垂直中线为轴，将图像的像素进行对换，也就是将图像的左半部和右半部对调。垂直镜像则是以图像的水平中线为轴，将图像的上半部分和下班部分对调。

变换原理：

设图像的宽度为 width，长度为 height。(x,y)为变换后的坐标，(x0,y0)为原图像的坐标

水平镜像变换

$$x = width - x_0 - 1$$
$$y = y_0$$

向前映射

其逆变换为

$$x_0 = width - x - 1$$
$$y_0 = y$$

向后映射

垂直镜像变换

$$x = x_0$$
$$y = height - y_0 - 1$$

其逆变换为

$$x_0 = x$$
$$y_0 = height - y - 1$$

总结：

在水平镜像变换时，遍历了整个图像，然后根据映射关系对每个像素都做了处理。实际上，水平镜像变换就是将图像坐标的列换到右边，右边的列换到左边，是可以以列为单位做变换的。同样垂直镜像变换也如此，可以以行为单位进行变换。

Python 代码：

```
import cv2
import numpy as np
img = cv2.imread('./images/whitebear.jpg',1)
#cv2.imshow('src',img)
imgInfo = img.shape
height = imgInfo[0]
width = imgInfo[1]
deep = imgInfo[2]
newImgInfo = (height*2,width,deep)
dst = np.zeros(newImgInfo,np.uint8)#uint8
for i in range(0,height):
    for j in range(0,width):
```

```
            dst[i,j] = img[i,j]
            #x y = 2*h - y -1
            dst[height*2-i-1,j] = img[i,j]
for i in range(0,width):
    dst[height,i] = (0,0,255)#BGR
```

```
#bgr8 转 jpeg 格式
import enum
import cv2

def bgr8_to_jpeg(value, quality=75):
    return bytes(cv2.imencode('.jpg', value)[1])
```

```
import ipywidgets.widgets as widgets

image_widget1 = widgets.Image(format='jpg', )
# image_widget2 = widgets.Image(format='jpg', )
# create a horizontal box container to place the image widget next to eachother
# image_container = widgets.HBox([image_widget1, image_widget2])

# display the container in this cell's output
display(image_widget1)
#display(image_widget2)

image_widget1.value = bgr8_to_jpeg(dst)
```

实验结果如下：

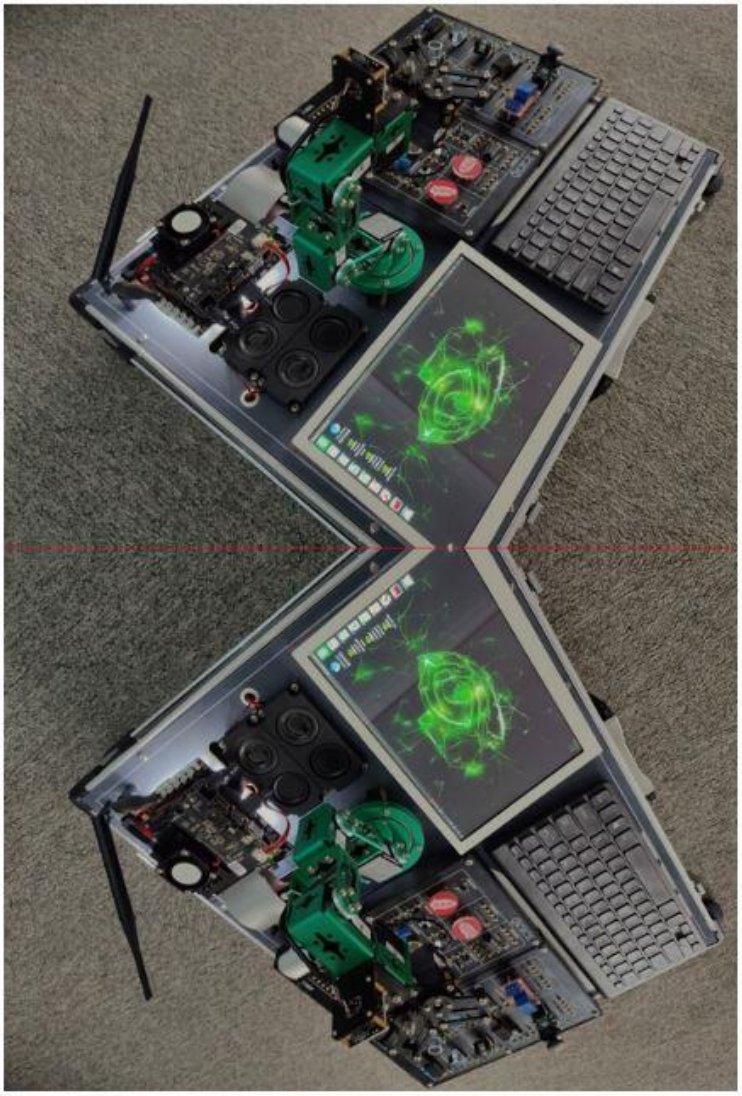


图 6.6.7 图片镜像效果

6.7、OpenCV 几何变换进阶

6.7.1、OpenCV 仿射变换

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘11.OpenCV 仿射变换.ipynb’程序，双击用 JupyterLab 打开程序。

仿射变换（Affine Transformation 或 Affine Map）是一种二维坐标（x, y）到二维坐标（u, v）的线性变换，其数学表达式形式如下：

$$\begin{cases} u = a_1 x + b_1 y + c_1 \\ v = a_2 x + b_2 y + c_2 \end{cases}$$

对应的齐次坐标矩阵表示形式为：

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

仿射变换保持了二维图形的“平直性”（直线经仿射变换后依然为直线）和“平行性”（直线之间的相对位置关系保持不变，平行线经仿射变换后依然为平行线，且直线上点的位置顺序不会发生变化）。非共线的三对对应点确定一个唯一的仿射变换。

图像的旋转加上拉升就是图像仿射变换，仿射变化也是需要一个 M 矩阵就可以，但是由于仿射变换比较复杂，一般直接找很难找到这个矩阵，OpenCV 提供了根据变换前后三个点的对应关系来自动求解 M。这个函数是 M=cv2.getAffineTransform(pos1,pos2),其中两个位置就是变换前后的对应位置关系。输出的就是仿射矩阵 M。然后在使用函数 cv2.warpAffine()。

我们看下整个仿射变换和透视变换的使用框图：图像变换的两种方法 cv2.warpAffine 和 cv2.warpPerspective

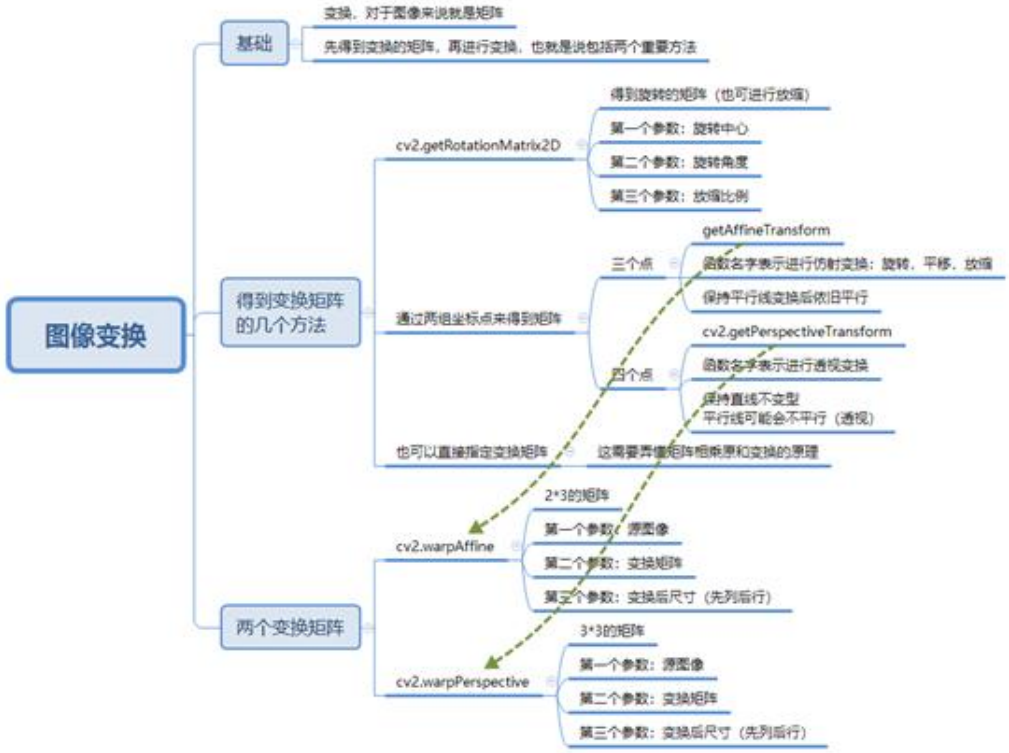


图 6.7.1 图像变换框图

Python 代码：

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('./images/whitebear.jpg',1)

img_bgr2rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img_bgr2rgb)
plt.show()
# cv2.waitKey(0)
```

```
imgInfo = img.shape
height = imgInfo[0]
width = imgInfo[1]
#src 3->dst 3 (左上角 左下角 右上角)
matSrc = np.float32([[0,0],[0,height-1],[width-1,0]])
matDst = np.float32([[50,50],[300,height-200],[width-300,100]])
#组合
matAffine = cv2.getAffineTransform(matSrc,matDst)# mat 1 src 2 dst
```

```
dst = cv2.warpAffine(img,matAffine,(width,height))
img_bgr2rgb = cv2.cvtColor(dst, cv2.COLOR_BGR2RGB)
plt.imshow(img_bgr2rgb)
```

实验结果如下：

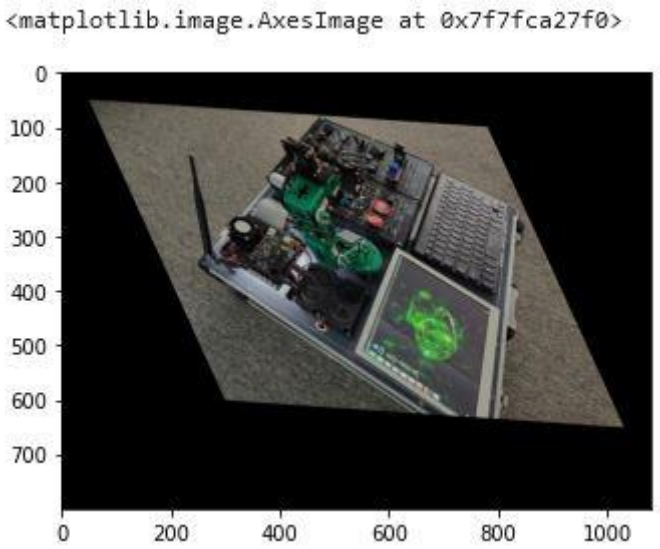


图 6.7.2 仿射变换结果

6.7.2、OpenCV 图片旋转

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘12.OpenCV 图片旋转.ipynb’程序，双击用 JupyterLab 打开程序。

图像旋转是指图像按照某个位置转动一定角度的过程，旋转中图像仍保持这原始尺寸。图像旋转后图像的水平对称轴、垂直对称轴及中心坐标原点都可能会发生变换，因此需要对图像旋转中的坐标进行相应转换。

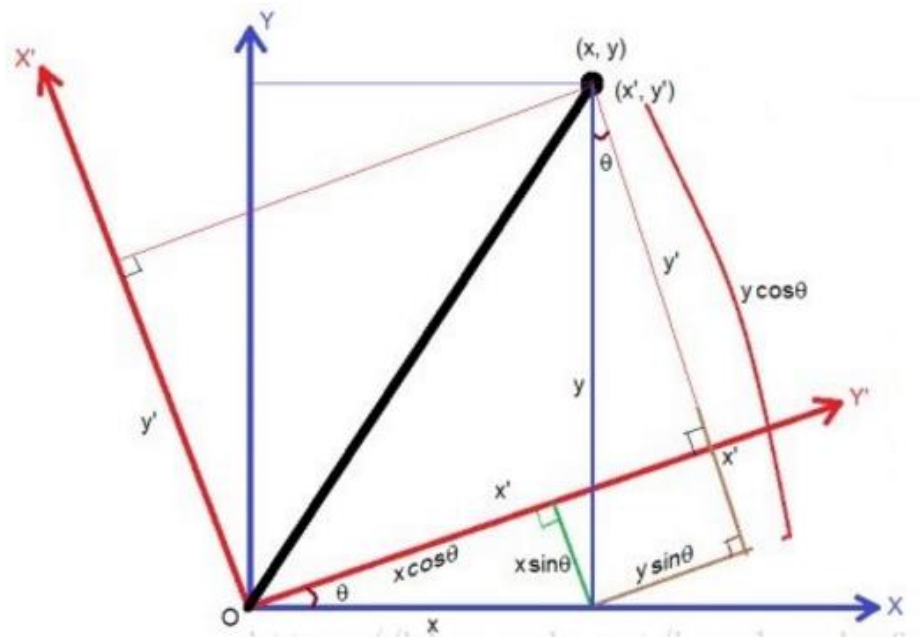


图 6.7.3 图像坐标转换

假设图像逆时针旋转 θ，则根据坐标转换可得旋转转换为：

$$\begin{cases} x' = r\cos(\alpha - \theta) \\ y' = r\sin(\alpha - \theta) \end{cases} \quad (1)$$

而

$$r = \sqrt{x^2 + y^2}, \sin\alpha = \frac{y}{\sqrt{x^2 + y^2}}, \cos\alpha = \frac{x}{\sqrt{x^2 + y^2}} \quad (2)$$

（2）带入（1）可得：

$$\begin{cases} x' = x\cos\theta + y\sin\theta \\ y' = -x\sin\theta + y\cos\theta \end{cases} \quad (3)$$

即如下：

$$[x' \quad y' \quad 1] = [x \quad y \quad 1] \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

而旋转后的图片的灰度值等于原图中相应位置的灰度值如下：

f(x′ ,y′)=f(x,y)

以上是旋转的原理，但是 OpenCV 提供的 API 可以直接通过函数获取转换矩阵。该函数的语法格式为：

matRotate = cv2.getRotationMatrix2D(center, angle, scale)

- center：旋转的中心点
- angle：旋转的角度。正数为逆时针；负数为顺时针。
- scale：变换尺度（缩放大小）。1 为不改变，小于 1 缩小，大于 1 是放大。

Python 代码：

```
import cv2
import numpy as np
img = cv2.imread('./images/whitebear.jpg',1)
#cv2.imshow('src',img)
imgInfo = img.shape
height = imgInfo[0]
width = imgInfo[1]
# 2*3
# cv2.getRotationMatrix2D(center, angle, scale)
# center: 旋转中心点
# angle：旋转角度，正数表示逆时针，负数表示顺时针
# scale：变换尺度
matRotate = cv2.getRotationMatrix2D((height*0.5, width*0.5), 45, 0.5)# mat rotate 1 center 2 angle 3 scale
#100*100 25
dst = cv2.warpAffine(img, matRotate, (height,width))
```

```
#bgr8 转 jpeg 格式
import enum
```

```
import cv2

def bgr8_to_jpeg(value, quality=75):
    return bytes(cv2.imencode('.jpg', value)[1])
```

```
import ipywidgets.widgets as widgets

image_widget1 = widgets.Image(format='jpg', )
image_widget2 = widgets.Image(format='jpg', )
# create a horizontal box container to place the image widget next to eachother
image_container = widgets.HBox([image_widget1, image_widget2])

# display the container in this cell's output
display(image_container)
#display(image_widget2)

img1 = cv2.imread('./images/whitebear.jpg',1)

image_widget1.value = bgr8_to_jpeg(img1)
image_widget2.value = bgr8_to_jpeg(dst)
```

实验结果如下：

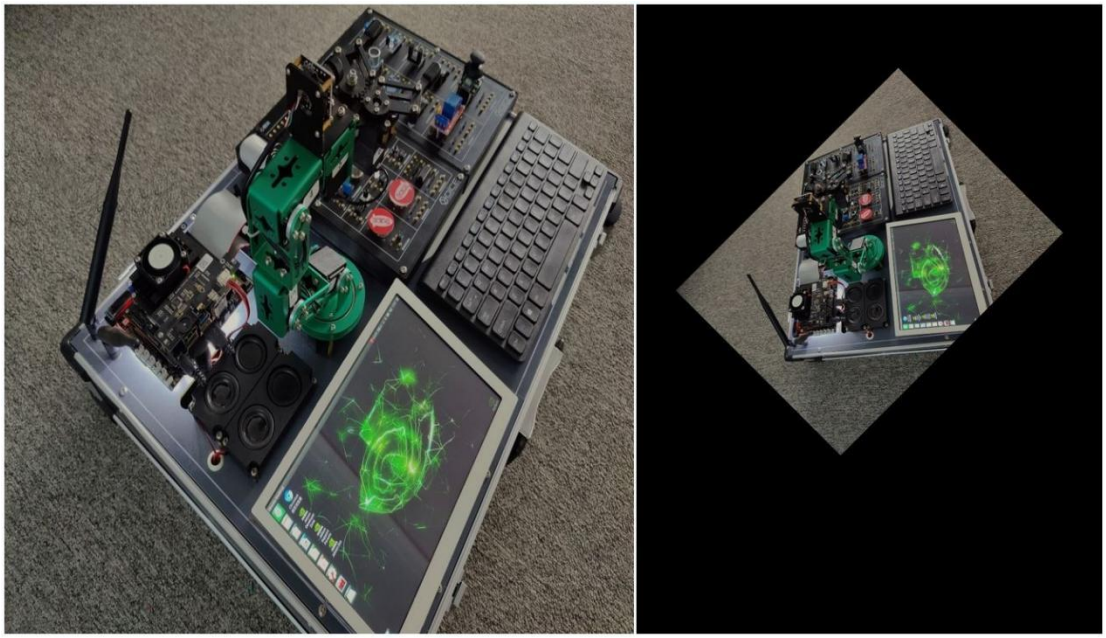


图 6.7.4 图像旋转实验结果

6.7.3、OpenCV 透视变换

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘13.OpenCV 透视变换.ipynb’程序，双击用 JupyterLab 打开程序。

透视变换又叫投影变换，我们常说的仿射变换是透视变换的一个特例。透视变换的目的就是把现实中为直线的物体，在图片上可能呈现为斜线，通过透视变换转换成直线的变换。透视变换则可以将矩形映射为任意四边形。在后面我们机器人自动驾驶的时候会用到此技术。

透视变换通过函数:

dst = cv2. warpPerspective(src, M, dsize[,flag, [,borderMode[,borderValue]]])

- dst : 透视变换后的输出图像，dsize 决定了输出的实际大小。
- src：源图像
- M：3X3 的变换矩阵
- dsize: 输出图像尺寸。
- flags：插值方法，默认 INTER_LINEAR(双线性插值)，当为 WARP_INVERSE_MAP 时，意味着 M 是逆变换，能实现从目标 dst 到 src 的逆变换。
- borderMode：边类型。默认为 BORDER_CONSTANT。当该值为 BORDER_TRANSPARENT 时，目标图像内的值不做改变，这些值对应原始图像内的异常值。
- borderValue：边界值，默认为 0。

与仿射变换一样，OpenCV 还是会提供一个函数 cv2.getPerspectiveTransform()为上面提供转换矩阵。

函数如下：

matAffine = cv2.getPerspectiveTransform(matSrc, matDst)

- matSrc：输入图像的四个顶点坐标。
- matDst：输出图像的四个顶点坐标。

Python 代码：

```
import cv2

import numpy as np

import matplotlib.pyplot as plt



img = cv2.imread('./images/whitebear.jpg',1)



imgInfo = img.shape

height = imgInfo[0]

width = imgInfo[1]

#src 4->dst 4 (左上角 左下角 右上角 右下角)

matSrc = np.float32([[200,100],[200,400],[600,100],[width-1,height-1]])

matDst = np.float32([[200,200],[200,300],[500,200],[500,400]])

#组合

matAffine = cv2.getPerspectiveTransform(matSrc,matDst)# mat 1 src 2 dst

dst = cv2.warpPerspective(img,matAffine,(width,height))

img_bgr2rgb = cv2.cvtColor(dst, cv2.COLOR_BGR2RGB)

plt.imshow(img_bgr2rgb)

plt.show()
```

实验结果如下：

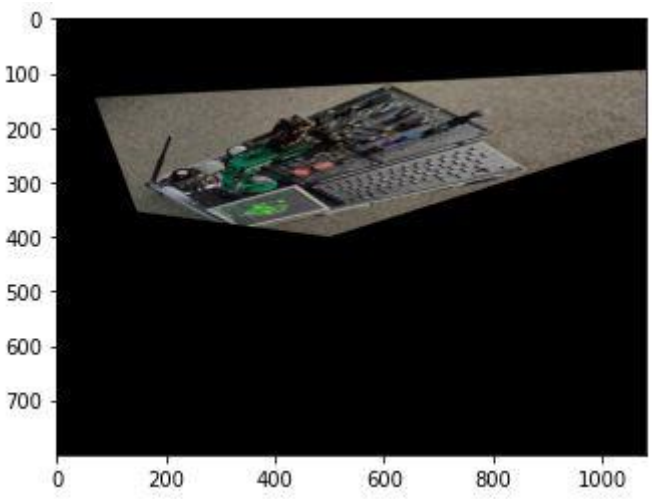


图 6.7.5 透视变换实验结果

6.8、OpenCV 图片处理

6.8.1、OpenCV 灰度处理

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘14.OpenCV 灰度处理.ipynb’程序，双击用 JupyterLab 打开程序。

将彩色图像转化为灰度图像的过程是图像的灰度化处理。彩色图像中的每个像素的颜色由 R，G，B 三个分量决定，而每个分量中可取值 0-255，这样一个像素点可以有 1600 多万（256*256*256=1677256）的颜色的变化范围。而灰度图像是 R，G，B 三个分量相同的一种特殊的彩色图像，其中一个像素点的变化范围为 256 种，所以在数字图像处理中一般将各种格式的图像转化为灰度图像以使后续的图像的计算量少一些。灰度图像的描述与彩色图像一样仍然反映了整副图像的整体和局部的色度和高亮等级的分布和特征。

图像灰度化处理

灰度化处理就是将一幅色彩图像转化为灰度图像的过程。彩色图像分为 R，G，B 三个分量，分别显示出红绿蓝等各种颜色，灰度化就是使彩色的 R，G，B 分量相等的过程。灰度值大的像素点比较亮（像素值最大为 255，为白色），反之比较暗（像素最下为 0，为黑色）。

图像灰度化核心思想是 R = G = B ，这个值也叫灰度值。

图像灰度化的算法：

1. 最大值法：使转化后的 R，G，B 得值等于转化前 3 个值中最大的一个，即：R=G=B=max（R，G，B）。这种方法转换的灰度图亮度很高。

2. 平均值法：是转化后 R，G，B 的值为转化前 R,G,B 的平均值。即：R=G=B=(R+G+B)/3。这种方法产生的灰度图像比较柔和。

3. 加权平均值法：按照一定权值，对 R，G，B 的值加权平均，即：$R=G=B=(\omega_R R+\omega_G G+\omega_B B)/3$ 分别为 R，G，B 的权值，取不同的值形成不同的灰度图像。由于人眼对绿色最为敏感，红色次之，对蓝色的敏感性最低，因此使将得到较易识别的灰度图像。一般时，得到的灰度图像效果最好。

实验中我们会使用 4 种方法实现灰度化：

方法 1： imread

Python 代码：

```
#方法 1

#注意：有时第一次运行图片不会显示出来，第二次才会显示出来

#方法 1 imread


import cv2

import matplotlib.pyplot as plt


img0 = cv2.imread('./images/whitebear.jpg',0)

img1 = cv2.imread('./images/whitebear.jpg',1)

# print(img0.shape)

# print(img1.shape)

# cv2.imshow('src',img0)

# cv2.waitKey(0)


#原始图像

# img_bgr2rgb1 = cv2.cvtColor(img1, cv2.COLOR_BGR2RGB)

# plt.imshow(img_bgr2rgb1)


#灰色图像

img_bgr2rgb0 = cv2.cvtColor(img0, cv2.COLOR_BGR2RGB)

plt.imshow(img_bgr2rgb0)

plt.show()
```

实验结果如下：

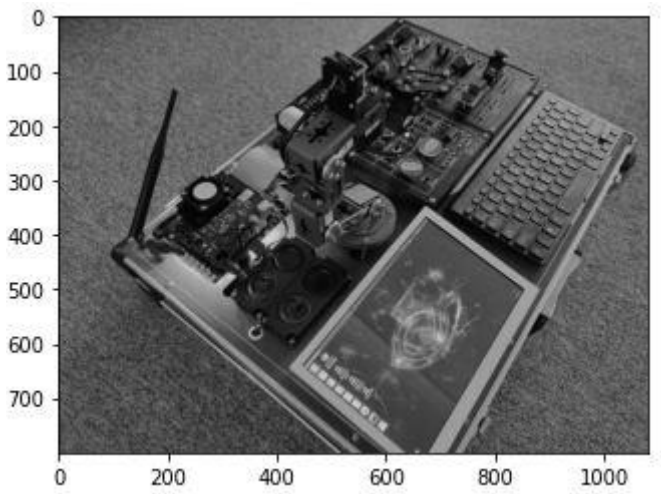


图 6.8.1 方法 1 处理结果

方法 2： cvtColor

Python 代码：

```
#方法 2 cvtColor

#注意：有时第一次运行图片不会显示出来，第二次才会显示出来

import cv2

import matplotlib.pyplot as plt


img = cv2.imread('./images/whitebear.jpg',1)

dst = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)#  颜色空间转换  1 data 2 BGR gray

#cv2.imshow('dst',dst)

#cv2.waitKey(0)


#原始图像

# img_bgr2rgb1 = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# plt.imshow(img_bgr2rgb1)


#灰色图像

img_bgr2rgb0 = cv2.cvtColor(dst, cv2.COLOR_BGR2RGB)

plt.imshow(img_bgr2rgb0)

plt.show()
```

实验结果如下：

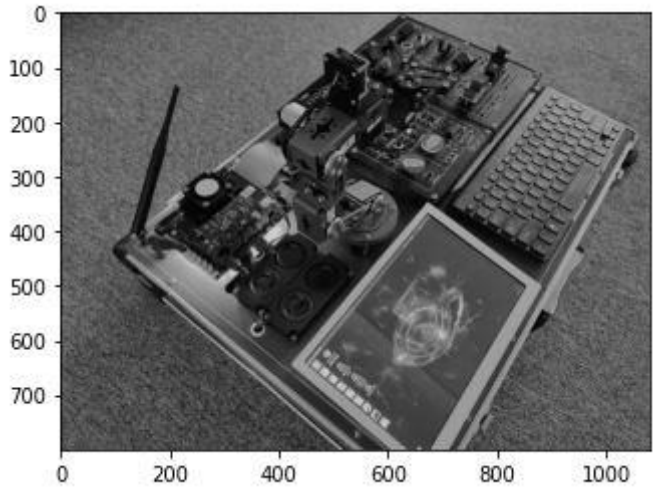


图 6.8.2 方法 2 处理结果

方法 3： 平均值法

Python 代码：

```
#方法 3 平均值法
import cv2
import numpy as np
import matplotlib.pyplot as plt


img = cv2.imread('./images/whitebear.jpg',1)
imgInfo = img.shape
height = imgInfo[0]
width = imgInfo[1]
# RGB R=G=B = gray  (R+G+B)/3
dst = np.zeros((height,width,3),np.uint8)
for i in range(0,height):
    for j in range(0,width):
        (b,g,r) = img[i,j]
        gray = (int(b)+int(g)+int(r))/3
        dst[i,j] = np.uint8(gray)
#cv2.imshow('dst',dst)
```

```
#cv2.waitKey(0)


#原始图像
# img_bgr2rgb1 = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
# plt.imshow(img_bgr2rgb1)


#灰色图像
img_bgr2rgb0 = cv2.cvtColor(dst, cv2.COLOR_BGR2RGB)
plt.imshow(img_bgr2rgb0)
plt.show()
```

实验结果如下：

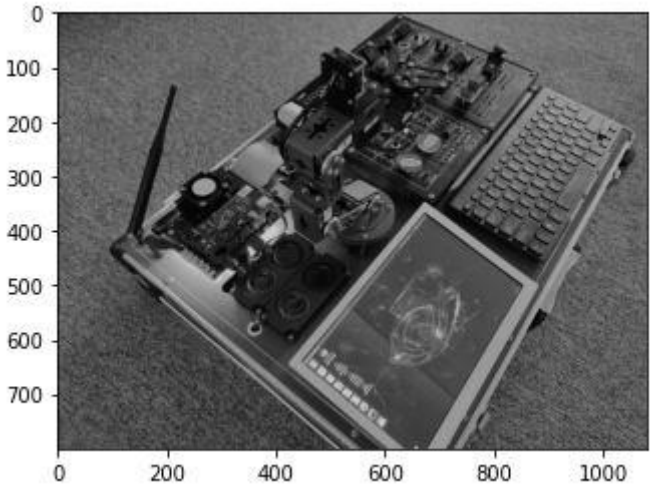


图 6.8.3 方法 3 处理结果

方法 4：加权平均值方法

Python 代码：

```
#方法 4 加权平均值方法
# gray = r*0.299+g*0.587+b*0.114


import cv2
import numpy as np
img = cv2.imread('./images/whitebear.jpg',1)
imgInfo = img.shape
```

```
height = imgInfo[0]
width = imgInfo[1]
dst = np.zeros((height,width,3),np.uint8)
for i in range(0,height):
    for j in range(0,width):
        (b,g,r) = img[i,j]
        b = int(b)
        g = int(g)
        r = int(r)
        gray = r*0.299+g*0.587+b*0.114
        dst[i,j] = np.uint8(gray)
#cv2.imshow('dst',dst)
#cv2.waitKey(0)

#原始图像
# img_bgr2rgb1 = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
# plt.imshow(img_bgr2rgb1)

#灰色图像
img_bgr2rgb0 = cv2.cvtColor(dst, cv2.COLOR_BGR2RGB)
plt.imshow(img_bgr2rgb0)
plt.show()
```

实验结果如下：

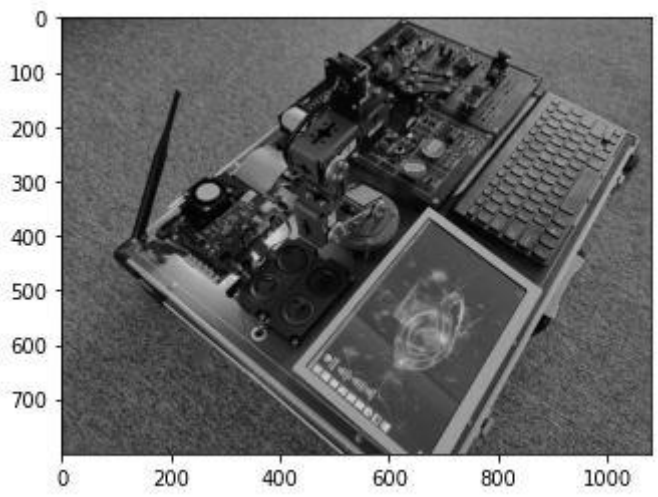


图 6.8.4 方法 4 处理结果

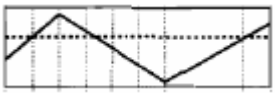
6.8.2、OpenCV 图像二值化

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘15.OpenCV 图像二值化.ipynb’程序，双击用 JupyterLab 打开程序。

二值化核心思想，设阈值，大于阈值的为 0（黑色）或 255（白色），使图像称为黑白图。阈值可固定，也可以自适应阈值。自适应阈值一般为一点像素与这点为中序的区域像素平均值或者高斯分布加权和的比较，其中可以设置一个差值也可以不设置。

全局阈值：

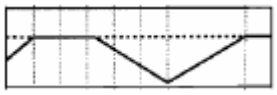
Python-OpenCV 中提供了阈值（threshold）函数：cv2.threshold（src, threshold, maxValue, method）



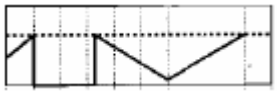
src 原图：破折线为将被阈值化的值；虚线为阈值

cv2.THRESH_BINARY：大于阈值的像素点的灰度值设定为 maxValue(如 8 位灰度值最大为 255)，灰度值小于阈值的像素点的灰度值设定为 0。

cv2.THRESH_BINARY_INV：大于阈值的像素点的灰度值设定为 0，而小于该阈值的设定为 maxValue。



cv2.THRESH_TRUNC：像素点的灰度值小于阈值不改变，大于阈值的灰度值的像素点就设定为该阈值。



cv2.THRESH_TOZERO：像素点的灰度值小于该阈值的不进行任何改变，而大于该阈值的部分，其灰度值全部变为 0。

cv2.THRESH_TOZERO_INV：像素点的灰度值大于该阈值的不进行任何改变，像素点的灰度值小于该阈值的，其灰度值全部变为 0。

Python 代码：

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread('./images/whitebear.jpg',1)

GrayImage = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
#GrayImage = np.array(dst).reshape(800,800).astype(np.uint8)

ret,thresh1=cv2.threshold(GrayImage,10,255,cv2.THRESH_BINARY)
ret,thresh2=cv2.threshold(GrayImage,10,255,cv2.THRESH_BINARY_INV)
ret,thresh3=cv2.threshold(GrayImage,10,255,cv2.THRESH_TRUNC)
ret,thresh4=cv2.threshold(GrayImage,10,255,cv2.THRESH_TOZERO)
ret,thresh5=cv2.threshold(GrayImage,10,255,cv2.THRESH_TOZERO_INV)
titles = ['Gray Image','BINARY','BINARY_INV','TRUNC','TOZERO','TOZERO_INV']
images = [GrayImage, thresh1, thresh2, thresh3, thresh4, thresh5]
```

```
for i in range(6):
    plt.subplot(2,3,i+1),plt.imshow(images[i],'gray')
    plt.title(titles[i])
    plt.xticks([]),plt.yticks([])
plt.show()
```

实验结果如下：

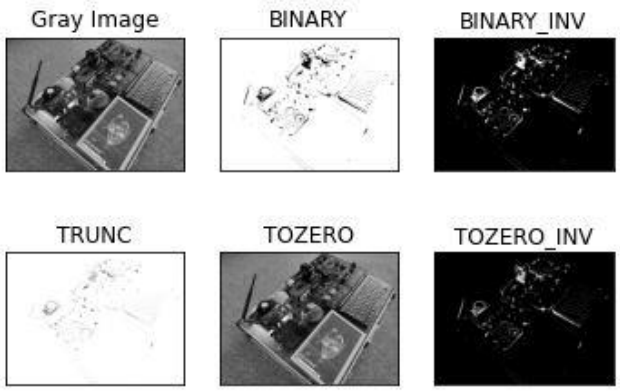


图 6.8.5 图片二值化效果

6.8.3、OpenCV 边缘检测

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘16-1.OpenCV 边缘检测.ipynb’和‘16-2.OpenCV 边缘检测.ipynb’程序，双击用 JupyterLab 打开程序。

边缘检测的目的是在保留原有图像属性的情况下，显著减少图像的数据规模。目前有多种算法可以进行边缘检测，虽然 Canny 算法年代久远，但可以说它是边缘检测的一种标准算法，而且仍在研究中广泛使用。

Canny 边缘检测是从不同视觉对象中提取有用的结构信息并大大减少要处理的数据量的一种技术，目前已广泛应用于各种计算机视觉系统。Canny 发现，在不同视觉系统上对边缘检测的要求较为类似，因此，可以实现一种具有广泛应用意义的边缘检测技术。边缘检测的一般标准包括：

- 以低的错误率检测边缘，也即意味着需要尽可能准确的捕获图像中尽可能多的边缘。
- 检测到的边缘应精确定位在真实边缘的中心。
- 图像中给定的边缘应只被标记一次，并且在可能的情况下，图像的噪声不应产生假的边缘。

为了满足这些要求，Canny 使用了变分法。Canny 检测器中的最优函数使用四个指数项的和来描述，它可以由高斯函数的一阶导数来近似。

在目前常用的边缘检测方法中，Canny 边缘检测算法是具有严格定义的，可以提供良好可靠检测的方法之一。由于它具有满足边缘检测的三个标准和实现过程简单的优势，成为边缘检测最流行的算法之一。

Canny 边缘检测算法可以分为以下 5 个步骤：

1、使用高斯滤波器，以平滑图像，滤除噪声。

2、计算图像中每个像素点的梯度强度和方向。

3、应用非极大值（Non-Maximum Suppression）抑制，以消除边缘检测带来的杂散响应。

4、应用双阈值（Double-Threshold）检测来确定真实的和潜在的边缘。

5、通过抑制孤立的弱边缘最终完成边缘检测。

方法 1python 代码：

```
import cv2
import numpy as np
import random
import matplotlib.pyplot as plt


img = cv2.imread('./images/whitebear.jpg',1)
imgInfo = img.shape
height = imgInfo[0]
width = imgInfo[1]
#cv2.imshow('src',img)
#canny 1 gray 2  高斯  3 canny
gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
imgG = cv2.GaussianBlur(gray,(3,3),0)
dst = cv2.Canny(img,50,50) #图片卷积——》th
```

```
img_bgr2rgb1 = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(img_bgr2rgb1)

plt.show()
```

边缘检测图像

```
img_bgr2rgb1 = cv2.cvtColor(dst, cv2.COLOR_BGR2RGB)

plt.imshow(img_bgr2rgb1)

plt.show()
```

实验结果如下：

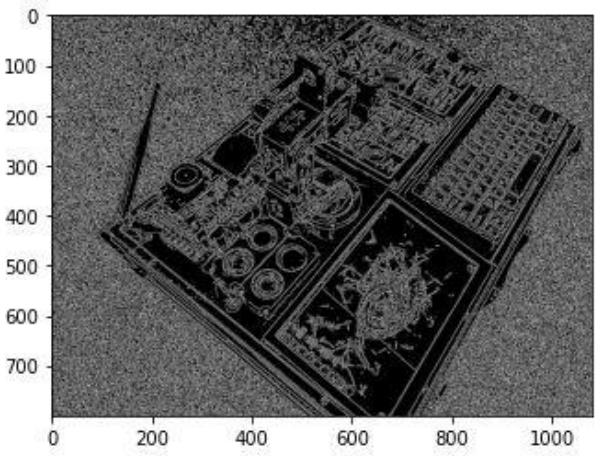


图 6.8.6 边缘计算方法 1 效果

方法 1python 代码：

```
import cv2

import numpy as np

import random

import math

img = cv2.imread('./images/dice007.jpg',1)

imgInfo = img.shape

height = imgInfo[0]

width = imgInfo[1]

# cv2.imshow('src',img)

# sobel 1 算子模版 2 图片卷积 3 阈值判决
```

```
# [1 2 1              [ 1 0 -1
#   0 0 0               2 0 -2
# -1 -2 -1 ]          1 0 -1 ]
# [1 2 3 4] [a b c d] a*1+b*2+c*3+d*4 = dst
# sqrt(a*a+b*b) = f>th
gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
dst = np.zeros((height,width,1),np.uint8)
for i in range(0,height-2):
    for j in range(0,width-2):
        gy = gray[i,j]*1+gray[i,j+1]*2+gray[i,j+2]*1-gray[i+2,j]*1-gray[i+2,j+1]*2-gray[i+2,j+2]*1
        gx = gray[i,j]+gray[i+1,j]*2+gray[i+2,j]-gray[i,j+2]-gray[i+1,j+2]*2-gray[i+2,j+2]
        grad = math.sqrt(gx*gx+gy*gy)
        if grad>50:
            dst[i,j] = 255
        else:
            dst[i,j] = 0
```

```
import matplotlib.pyplot as plt
img_bgr2rgb1 = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img_bgr2rgb1)
plt.show()
```

边缘检测

```
img_bgr2rgb1 = cv2.cvtColor(dst, cv2.COLOR_BGR2RGB)
plt.imshow(img_bgr2rgb1)
plt.show()
```

实验结果如下：

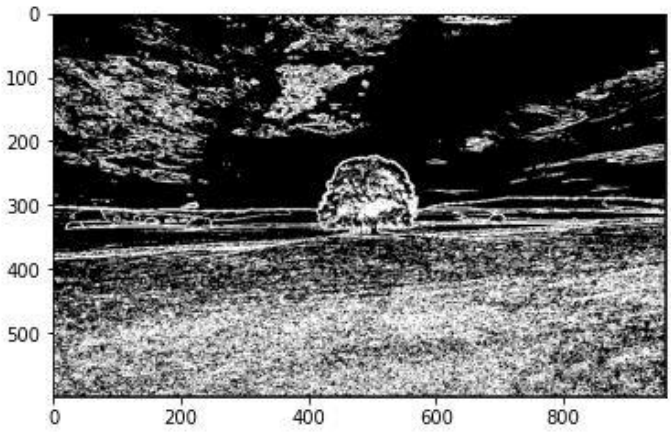


图 6.8.7 边缘计算方法 2 效果

6.9、OpenCV 绘制文字线段

6.9.1、OpenCV 线段绘制

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘17.OpenCV 线段绘制.ipynb’程序，双击用 JupyterLab 打开程序。

在使用 OpenCV 处理图像时，我们有时候会需要在图像上画线段、矩形等。OpenCV 中使用 line（dst，pt1，pt2，color，thickness=None，lineType=None，shift=None）函数进行线段的绘制。

参数说明：

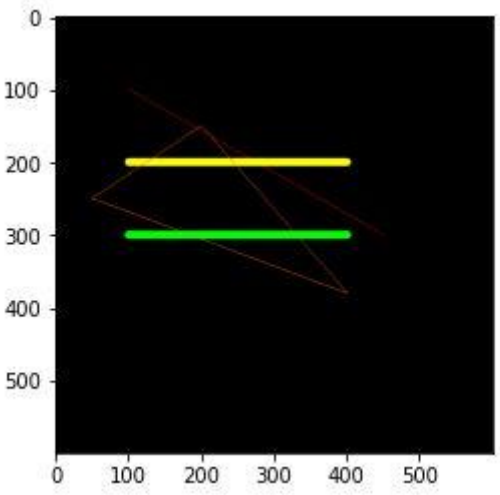
- dst：输出图像。
- pt1，pt2：必选参数。线段的坐标点，分别表示起始点和终止点
- color：必选参数。用于设置线段的颜色
- thickness：可选参数。用于设置线段的宽度
- lineType：可选参数。用于设置线段的类型，可选 8（8 邻接连接线-默认）、4（4 邻接连接线）和 cv2.LINE_AA 为抗锯齿

Python 代码：

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

newImageInfo = (600, 600, 3)
dst = np.zeros(newImageInfo,np.uint8)

# line
#  绘制线段  1 dst 2 begin 3 end 4 color
cv2.line(dst, (100,100), (450,300), (0,0,255))
# 5 line w
cv2.line(dst, (100,200), (400,200), (0,255,255), 10)
```

```
# 6 line type
cv2.line(dst, (100,300), (400,300), (0,255,0), 10, cv2.LINE_AA)

cv2.line(dst, (200,150), (50,250), (25,100,255))
cv2.line(dst, (50,250), (400,380), (25,100,255))
cv2.line(dst, (400,380), (200,150), (25,100,255))

# cv2.imshow('dst',dst)
# cv2.waitKey(0)
dst = cv2.cvtColor(dst, cv2.COLOR_BGR2RGB)
plt.imshow(dst)
plt.show()
```

实验结果如下：



6.9.1 线段绘制效果

6.9.2、OpenCV 矩形圆形绘制

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘17.OpenCV 矩形圆形绘制.ipynb’程序，双击用 JupyterLab 打开程序。

1、矩形的绘制

rectangle（img，pt1，pt2，color，thickness=None，lineType=None，shift=None）

参数说明：

- img：画布或者载体图像。
- pt1，pt2：必选参数。矩形的顶点，分别表示顶点与对角顶点，即矩形的左上角与右下角（这两个顶点可以确定一个唯一的矩形）
- color：必选参数。用于设置矩形的颜色
- thickness：可选参数。用于设置矩形边的宽度，当值为负数时，表示对矩形进行填充
- lineType：可选参数。用于设置线段的类型，可选 8（8 邻接连接线-默认）、4（4 邻接连接线）和 cv2.LINE_AA 为抗锯齿

2、圆的绘制

cv2.circle(img, center, radius, color[,thickness[,lineType]])

参数说明：

- img:画布或者载体图像
- center：为圆心坐标，格式： （50,50）
- radius：半径
- color：颜色
- thickness： 线条粗细。默认为 1.如果-1 则为填充实心。
- lineType：线条类型。默认是 8，连接类型。如下表格说明

参数	说明
cv2.FILLED	填充
cv2.LINE_4	4 连接类型
cv2.LINE_8	8 连接类型
cv2.LINE_AA	抗锯齿，该参数会让线条更平滑

3、绘制椭圆

cv2.ellipse(img, center, axes, angle, StartAngle, endAngle, color[,thickness[,lineType]])

- center：椭圆的中心点，（x，x）
- axes：指的是短半径和长半径，（x，x）
- angle：指的是逆时针旋转的角度
- StartAngle：圆弧起始角的角度
- endAngle：圆弧终结角的角度
- img、color 可以参考圆的说明。

第五个参数指的是逆时针开始画图的角度，第六个指的是逆时针结束画图的角度

四五六参数若加上符号，表示的反方向，即顺时针方向

4、绘制多边形

cv2.polylines(img,[pts],isClosed, color[,thickness[,lineType]])

- pts：多边形的顶点
- isClosed：是否闭合。（True/False）

其他参数参照圆的绘制参数

Python 代码：

```
import cv2
import numpy as np
newImageInfo = (500,500,3)
dst = np.zeros(newImageInfo,np.uint8)
#  1 2 左上角 3 右下角 4 5 fill -1 >0 line w
```

```
cv2.rectangle(dst,(350,100),(400,270),(0,255,0),3)
# 2 center 3 r
cv2.circle(dst,(250,250),(50),(255,0,0),2)
# 2 center 3  轴  4 angle 5 begin 6 end 7
cv2.ellipse(dst, (256,256), (150,100), 0, 0, 180, (0,255,255), -1)

points = np.array([[150,50], [140,140], [200,170], [250,250], [150,50]], np.int32)
#print(points.shape)
points = points.reshape((-1,1,2))
#print(points.shape)
cv2.polylines(dst,[points],True,(255,255,0))
```

```
import matplotlib.pyplot as plt

dst = cv2.cvtColor(dst, cv2.COLOR_BGR2RGB)
plt.imshow(dst)
plt.show()
```

实验结果如下：

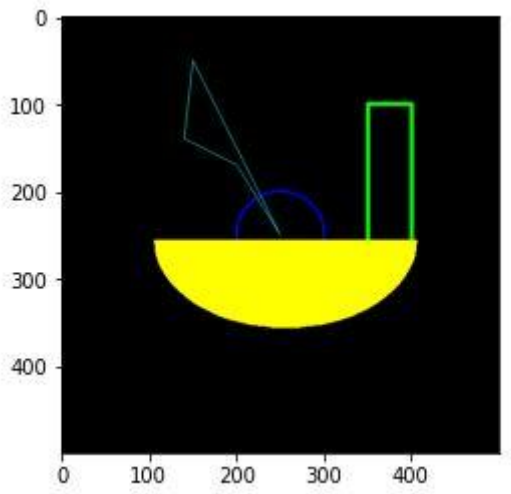


图 6.9.2 圆形矩形绘制效果

6.9.3、OpenCV 文字图片绘制

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘19.OpenCV 文字图片绘制.ipynb’程序，双击用 JupyterLab 打开程序。

函数调用：**cv2.putText(img, str, origin, font, size,color,thickness)**

各参数依次是：图片，添加的文字，左上角坐标(整数)，字体，字体大小，颜色，字体粗细。

字体类型如下：

枚举	
FONT_HERSHEY_SIMPLEX Python: cv.FONT_HERSHEY_SIMPLEX	正常大小sans-serif字体
FONT_HERSHEY_PLAIN Python: cv.FONT_HERSHEY_PLAIN	小尺寸sans-serif字体
FONT_HERSHEY_DUPLEX Python: cv.FONT_HERSHEY_DUPLEX	正常大小的sans-serif字体（比FONT_HERSHEY_SIMPLEX更复杂）
FONT_HERSHEY_COMPLEX Python: cv.FONT_HERSHEY_COMPLEX	正常大小的衬线字体
FONT_HERSHEY_TRIPLEX Python: cv.FONT_HERSHEY_TRIPLEX	正常大小的serif字体（比FONT_HERSHEY_COMPLEX更复杂）
FONT_HERSHEY_COMPLEX_SMALL Python: cv.FONT_HERSHEY_COMPLEX_SMALL	较小版本的FONT_HERSHEY_COMPLEX
FONT_HERSHEY_SCRIPT_SIMPLEX Python: cv.FONT_HERSHEY_SCRIPT_SIMPLEX	手写风格的字体
FONT_HERSHEY_SCRIPT_COMPLEX Python: cv.FONT_HERSHEY_SCRIPT_COMPLEX	更复杂的FONT_HERSHEY_SCRIPT_SIMPLEX变体
FONT_ITALIC Python: cv.FONT_ITALIC	标志为斜体字体

图 6.9.3 字体类型

文字绘制 Python 代码：

```
import cv2
import numpy as np

img = cv2.imread('./images/whitebear.jpg',1)
font = cv2.FONT_HERSHEY_SIMPLEX

cv2.rectangle(img,(200,100),(750,400),(0,255,0),3)
# 1 dst 2 文字内容 3 坐标 4 5 字体大小 6 color 7 粗细 8 line type
cv2.putText(img,'whitebear',(250,50),font,3,(200,200,0),2,cv2.LINE_AA)
```

```
import matplotlib.pyplot as plt

dst = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(dst)
plt.show()
```

实验结果如下：

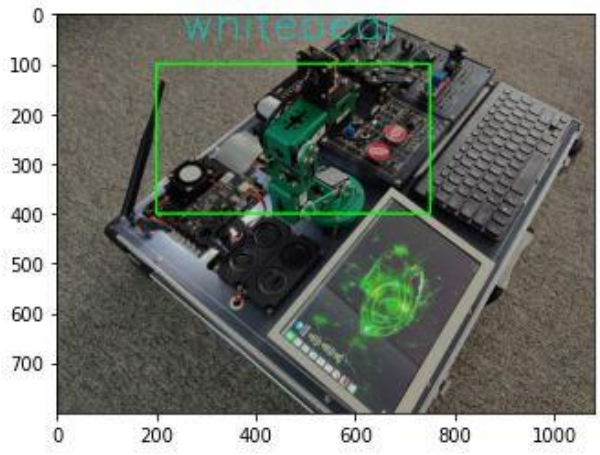


图 6.9.4 文字绘制效果

图片绘制 Python 代码：

```
import cv2
img = cv2.imread('./images/whitebear.jpg',1)
height = int(img.shape[0]*0.2)
width = int(img.shape[1]*0.2)
imgResize = cv2.resize(img,(width,height))
for i in range(0,height):
    for j in range(0,width):
        img[i+200,j+350] = imgResize[i,j]
```

```
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img)
plt.show()
```

实验结果如下：

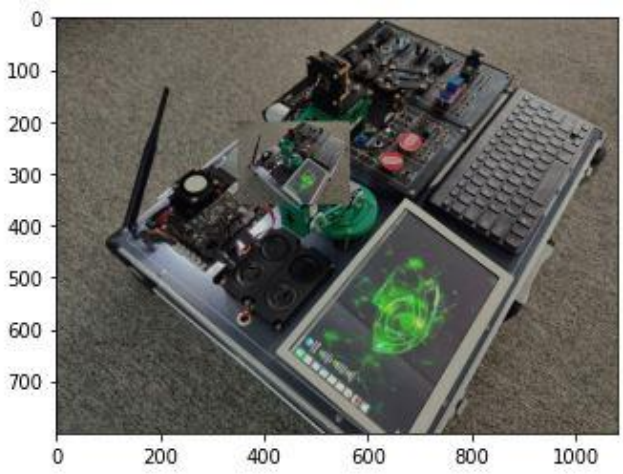


图 6.9.5 图片绘制效果

6.10、OpenCV 图像美化 1

6.10.1、OpenCV 彩色图片直方图

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘20.OpenCV 彩色图片直方图.ipynb’程序，双击用 JupyterLab 打开程序。

颜色直方图，指的是一幅图像中的颜色分布，与图像中的特定的物体无关，只是用来表示人的眼睛观察到的图像中的颜色分布情况，例如说，一幅图中红色占了多少比例，绿色占了多少比例等。

我们知道，计算机色彩显示器采用 R、G、B 相加混色的原理，通过发射出三种不同强度的电子束，使屏幕内侧覆盖的红、绿、蓝磷光材料发光而产生色彩。在 RGB 颜色空间中，任意色光 F 都可以用 RGB 三色不同分量的相加混合而成。也就是说，图像中每个像素的颜色值都可以用一个三元值（R,G,B）来表示，例如(0,0,0)表示黑色，(0,0,255)表示蓝色，……每个分量的大小从 0~255。

那么，我们可以知道一张彩色图像可以由 R、G、B 三个通道堆叠而成。可以想象成将三张大小相同的纸叠放，然后人眼从上往下看到的图像就是原来的彩色图像。这里，假设图像的分辨率为 320*240，那么每个通道的长为 320，宽为 240，单位是像素，总的像素个数就是 3*320*240.。

方法 1：

使用 matplotlib 的子库 pyplot 实现，它提供了类似于 Matlab 的绘图框架，matplotlib 是非常强大基础的一个 Python 绘图包。其中绘制直方图主要调用 hist() 函数实现，它根据数据源和像素级绘制直方图。

hist()函数形式如下：

hist(数据源，像素级)

其中，参数：

- 数据源：必须是一维数组，通常需要通过函数 ravel() 拉直图像
- 像素级：一般是 256，表示[0, 255]

函数 ravel() 将多维数组降为一维数组，其格式为：

一维数组 = 多维数组.ravel()

eg: plt.hist(img.ravel(), 256) #ravel() 二维降一维 256 灰度级的分组情况

方法 2：

OpenCV 给我们提供的函数是：

cv2.calcHist(image,channels,mask,histSize,ranges[,hist[,accumulate]])

该函数有 5 个参数：

- image：输入图像，传入时应该用中括号[]括起来
- channels：传入图像的通道，如果是灰度图像，那就不用说了，只有一个通道，值为 0，如果是彩色图像（有 3 个通道），那么值为 0,1,2,中选择一个，对应着 BGR 各个通道。这个值也得用[]传入。
- mask：掩膜图像。如果统计整幅图，那么为 none。主要是如果要统计部分图的直方图，就得构造相应的炎掩膜来计算。
- histSize：灰度级的个数，需要中括号，比如[256]
- ranges:像素值的范围，通常[0,256]，有的图像如果不是 0-256，比如说你来回各种变换导致像素值负值、很大，则需要调整后才可以。

Python 代码：

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('./images/dice007.jpg', 1)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img)
plt.show()
```

Plot 绘制直方图

```
plt.hist(img.ravel(), 256)     #ravel() 二维降一维   256 灰度级的分组情况
plt.show()
```

OpenCV 绘制直方图

```
histb = cv2.calcHist([img], [0], None, [256], [0, 255])
plt.plot(histb, color='b')
plt.show()
```

实验结果如下：

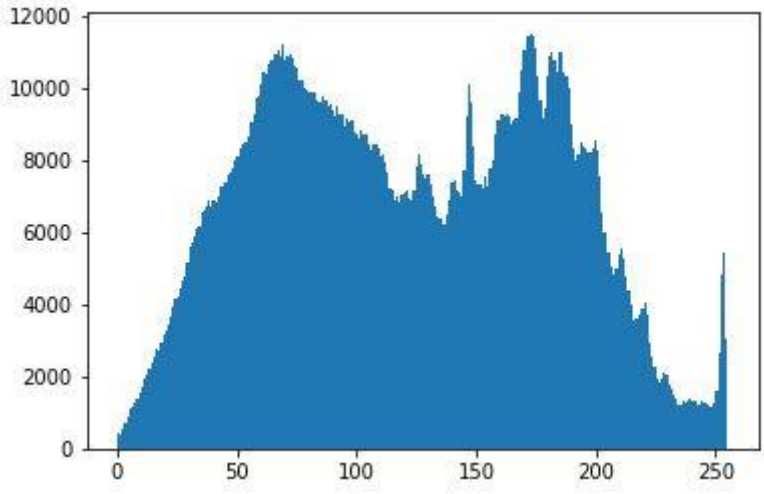


图 6.10.1 Plot 绘制直方图效果

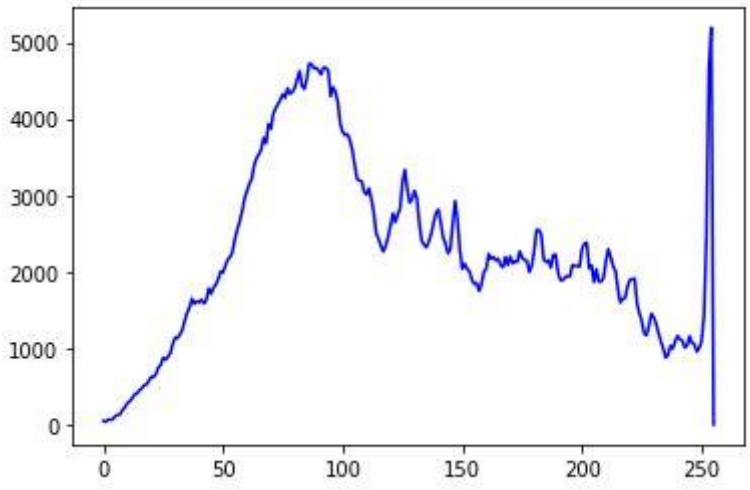


图 6.10.2 OpenCV 绘制直方图效果

6.10.2、OpenCV 直方图均衡化

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘21.OpenCV 直方图均衡化.ipynb’程序，双击用 JupyterLab 打开程序。

图像的空域处理是一种重要的图像处理技术，这类方法直接以图像的像素操作为基础，主要分为灰度变换和空域滤波两大类，直方图均衡化（Histogram equalization）就是一种常用的灰度变换方法。通常，暗图像直方图的分量集中在灰度较低的一端，而亮图像直方图分量偏向于灰度较高的一端。

如果一幅图像的灰度直方图几乎覆盖了整个灰度的取值范围，并且除了个别灰度值的个数较为突出，整个灰度值分布近似于均匀分布，那么这幅图像就具有较大的灰度动态范围和较高的对比度，同时图像的细节更为丰富。已经证明，仅仅依靠输入图像的直方图信息，就可以得到一个变换函数，利用该变换函数可以将输入图像达到上述效果，该过程就是直方图均衡化。

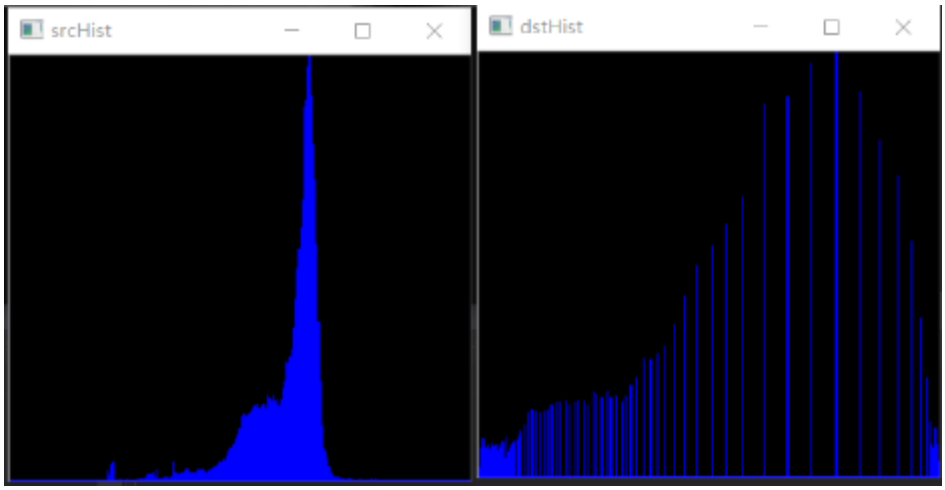
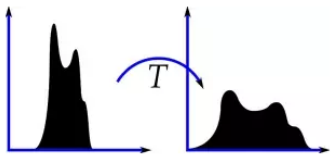


图 6.10.3 左边为直方图，右边为直方图均衡化效果。

直方图均衡化就是将原始的直方图拉伸，使之均匀分布在全部灰度范围内，从而增强图像的对比度。

直方图均衡化的中心思想是把原始图像的的灰度直方图从比较集中的某个区域变成在全部灰度范围内的均匀分布。旨在使得图像整体效果均匀，黑与白之间的各个像素级之间的点更均匀一点。

函数：**cv2.equalizeHist()**



灰度-直方图均衡化 Python 代码：

```
import cv2
import numpy as np
```

```
import matplotlib.pyplot as plt



img = cv2.imread('./images/dice007.jpg',1)
gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
#cv2.imshow('src',gray)
dst = cv2.equalizeHist(gray)
#cv2.imshow('dst',dst)
#cv2.waitKey(0)



gray = cv2.cvtColor(gray, cv2.COLOR_BGR2RGB)
dst = cv2.cvtColor(dst, cv2.COLOR_BGR2RGB)
#plt 绘制前后两张图片显示效果
#源图显示
plt.figure(figsize=(14, 9), dpi=100)#设置绘图区域的大小和像素
plt.subplot(121)  # 一行两列第一个
plt.imshow(gray)

#灰度 直方图均衡化
plt.subplot(122)  # 一行两列第二个
plt.imshow(dst)


plt.show()
```

实验效果如下：

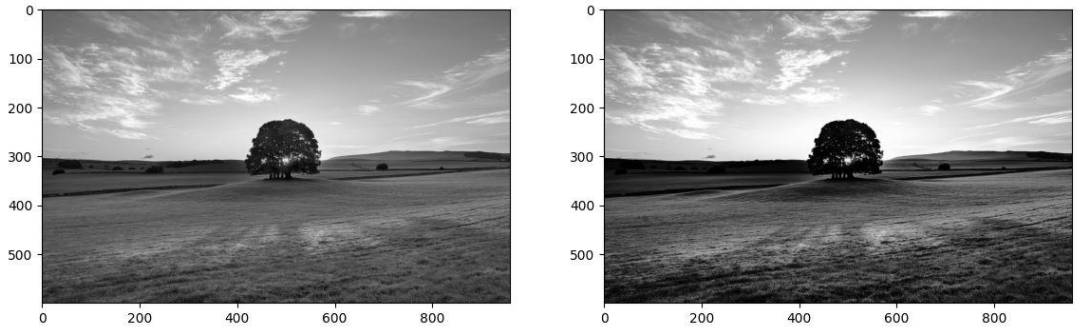


图 6.10.4 灰度-直方图均衡化效果

彩色-直方图均衡化 Python 代码：

```
import cv2
import numpy as np
```

```
img = cv2.imread('./images/dice007.jpg',1)
# cv2.imshow('src',img)
(b,g,r) = cv2.split(img)#通道分解
bH = cv2.equalizeHist(b)
gH = cv2.equalizeHist(g)
rH = cv2.equalizeHist(r)
result = cv2.merge((bH,gH,rH))#  通道合成
# cv2.imshow('dst',result)
# cv2.waitKey(0)

img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
dst = cv2.cvtColor(dst, cv2.COLOR_BGR2RGB)
#plt 绘制前后两张图片显示效果
plt.figure(figsize=(14, 9), dpi=100)#设置绘图区域的大小和像素
plt.subplot(121)   #  一行两列第一个
plt.imshow(img)
plt.subplot(122)   #  一行两列第二个
#彩色  直方图均衡化
plt.imshow(dst)
plt.show()
```

实验效果如下：

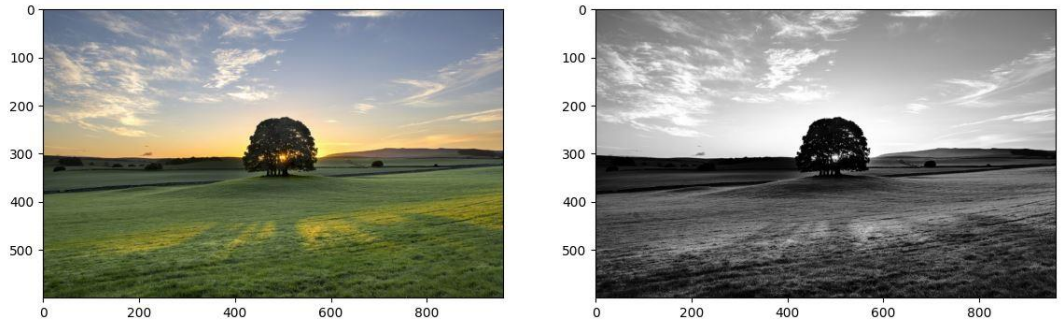


图 6.10.5 彩色-直方图均衡化效果

彩色-直方图均衡化 Python 代码：

```
import cv2
import numpy as np
img = cv2.imread('./images/dice007.jpg',1)
imgYUV = cv2.cvtColor(img,cv2.COLOR_BGR2YCrCb)
# cv2.imshow('src',img)
channelYUV = cv2.split(imgYUV)
```

```
channelYUV[0] = cv2.equalizeHist(channelYUV[0])
channels = cv2.merge(channelYUV)
result = cv2.cvtColor(channels,cv2.COLOR_YCrCb2BGR)
# cv2.imshow('dst',result)
# cv2.waitKey(0)



imgYUV = cv2.cvtColor(imgYUV, cv2.COLOR_BGR2RGB)
result = cv2.cvtColor(result, cv2.COLOR_BGR2RGB)
#plt 绘制前后两张图片显示效果
plt.figure(figsize=(14, 9), dpi=100)#设置绘图区域的大小和像素
plt.subplot(121)  #  一行两列第一个
plt.imshow(imgYUV)
plt.subplot(122)  #  一行两列第二个
#彩色  直方图均衡化
plt.imshow(result)
plt.show()
```

实验效果如下：

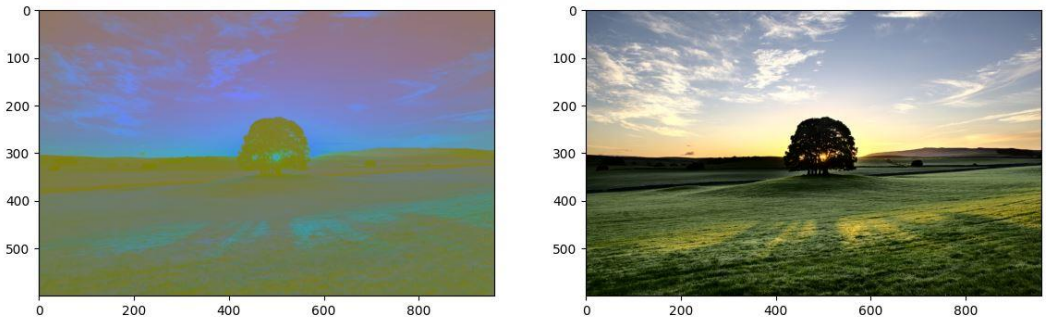


图 6.10.6 YUV-直方图均衡化效果

6.11、OpenCV 图像美化 2

6.11.1、OpenCV 图片修补

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘22.OpenCV 图片修补.ipynb’程序，双击用 JupyterLab 打开程序。

图像修复是计算机视觉中的一类算法，其目标是填充图像或视频内的区域。该区域使用二进制掩模进行标识，填充通常根据需要填充的区域边界信息来完成。图像修复的最常见应用是恢复旧的扫描照片。它还用于删除图像中的小的不需要的对象。

dst = cv2.inpaint(src, inpaintMask, inpaintRadius, flags)

参数定义：

- src：源图像
- inpaintMask：二进制掩码，指示要修复的像素。
- dst：结果图像
- inpaintRadius：表示修复的半径
- flags：修复算法，主要有 INPAINT_NS (Navier-Stokes based method) or INPAINT_TELEA (Fast marching based method)

基于 Navier-Stokes 的修复应该更慢，并且倾向于产生比 fast marching method 的方法更模糊的结果。在实践中，我们没有发现这种情况。INPAINT_NS 在我们的测试中产生了更好的结果，速度也略高于 INPAINT_TELEA。

Python 代码：

向原图添加破损

```
#尝试向原图添加破损
import cv2
import numpy as np
import matplotlib.pyplot as plt



img = cv2.imread('./images/dice007.jpg',1)
for i in range(200,300):
    img[i,200] = (255,255,255)
```

```
        img[i,200+1] = (255,255,255)
        img[i,200-1] = (255,255,255)
for i in range(150,250):
        img[250,i] = (255,255,255)
        img[250+1,i] = (255,255,255)
        img[250-1,i] = (255,255,255)
cv2.imwrite('damaged.jpg',img)
# cv2.imshow('image',img)
# cv2.waitKey(0)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img)
plt.show()
```

修补破损

```
#1 坏图 2 掩膜 3 inpaint
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('damaged.jpg',1)
#cv2.imshow('src',img)
imgInfo = img.shape
height = imgInfo[0]
width = imgInfo[1]
paint = np.zeros((height,width,1),np.uint8)

for i in range(200,300):
        paint[i,200] = 255
        paint[i,200+1] = 255
        paint[i,200-1] = 255
for i in range(150,250):
        paint[250,i] = 255
        paint[250+1,i] = 255
        paint[250-1,i] = 255
#cv2.imshow('paint',paint)
#1 src 2 mask
imgDst = cv2.inpaint(img,paint,3,cv2.INPAINT_TELEA)

# cv2.imshow('image',imgDst)
# cv2.waitKey(0)
```

```
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
paint = cv2.cvtColor(paint, cv2.COLOR_BGR2RGB)
imgDst = cv2.cvtColor(imgDst, cv2.COLOR_BGR2RGB)



plt.figure(figsize=(14, 9), dpi=100)#设置绘图区域的大小和像素
plt.subplot(131)  # 一行三列第一个
plt.imshow(img)
plt.subplot(132)  # 一行三列第二个
plt.imshow(paint)
plt.subplot(133)  # 一行三列第三个
plt.imshow(imgDst)

plt.show()
```

实验结果如下：

图 6.11.1 图片添加破损效果

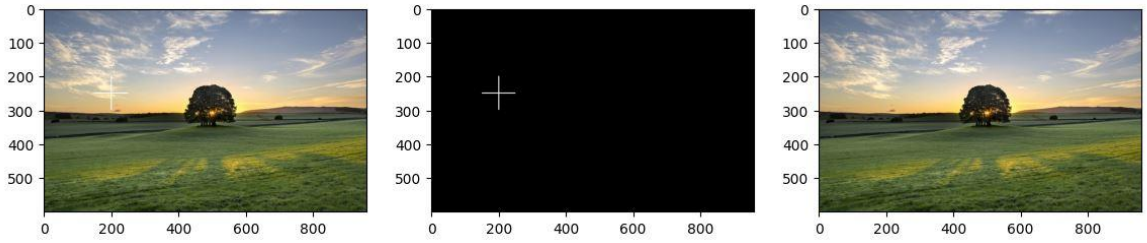


图 6.11.2 图片修补效果

6.11.2、OpenCV 亮度增强

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘23.OpenCV 亮度增强.ipynb’程序，双击用 JupyterLab 打开程序。

实现过程：对每个像素点的三通道值进行同步放大，同时保持通道值在 0-255 之间

- map(f,list)，将函数 f 作用于整个列表，返回新列表
- np.clip(a, a_min, a_max, out=None)，将 a 中的元素限制在最小值和最大值之间，超过此区间的值赋值为最小值或最大值

Python 代码：

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('./images/dice007.jpg',1)
imgInfo = img.shape
height = imgInfo[0]
width = imgInfo[1]
#cv2.imshow('src',img)
dst = np.zeros((height, width, 3),np.uint8)
for i in range(0, height):
    for j in range(0, width):
        (b,g,r) = img[i,j]
        bb = int(b) + 40
        gg = int(g) + 40
        rr = int(r) + 40
        if bb>255:
            bb = 255
        if gg>255:
            gg = 255
        if rr>255:
            rr = 255
        dst[i,j] = (bb,gg,rr)
# cv2.imshow('dst',dst)
# cv2.waitKey(0)
```

```
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
dst = cv2.cvtColor(dst, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(14, 6), dpi=100) #设置绘图区域的大小和像素
plt.subplot(121)  # 一行二列第一个
plt.imshow(img)
plt.subplot(122)  # 一行二列第二个
plt.imshow(dst)
plt.show()
```

实验效果如下：

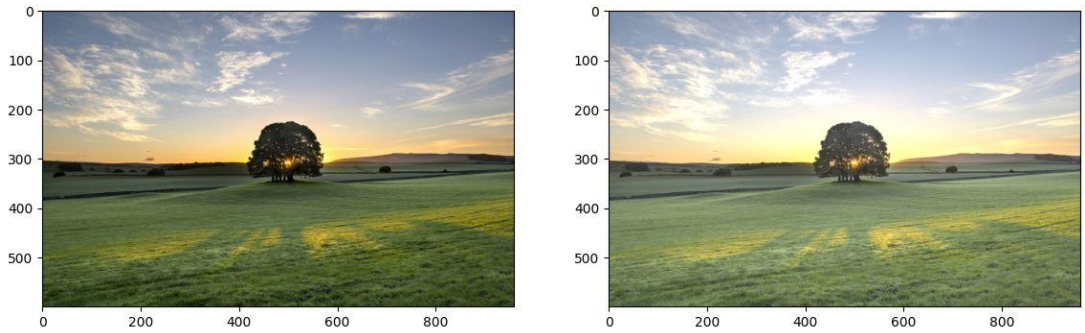


图 6.11.3 亮度增强效果

6.12、OpenCV 图像美化 3

6.12.1、OpenCV 磨皮美白与双边滤波

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘24.OpenCV 磨皮美白.ipynb’程序，双击用 JupyterLab 打开程序。

图片的美白公式：**p = P*1.4(a)+ b**;

双边滤波是一种非线性的滤波方法，是结合图像的空间邻近度和像素值相似度的一种折衷处理，同时考虑空间与信息和灰度相似性，达到保边去噪的目的，具有简单、非迭代、局部处理的特点。之所以能够达到保边去噪的滤波效果是因为滤波器由两个函数构成：一个函数是由几何空间距离决定滤波器系数，另一个是由像素差值决定滤波器系数。

通俗来讲就是双边滤波模板主要有两个模板生成，第一个是高斯模板，第二个是以灰度级的差值作为函数系数生成的模板，然后这两个模板点乘就得到了最终的双边滤波模板，第一个模板是全局模板，所以只需要生成以西，第二个模板需要对每个像素都计算一次。双边滤波器比高斯滤波器多了一个高斯方差 sigma-d,它是基于空间分布的高斯滤波函数，所以在边缘附近，离的较远的像素不会太多影响到边缘上的像素，这样就能对边缘附近的像素值予以保存，但是由于保存过多的高频信息，对于彩色图像里的高频噪声，双边滤波器不能够干净的滤掉，只能够对于低频信息进行较好的滤除。

磨皮美白 Python 代码：

```
#图片的美白公式：p = P*1.4(a)+ b;
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('./images/dicetlp.jpg',1)
imgInfo = img.shape
height = imgInfo[0]
width = imgInfo[1]
#cv2.imshow('src',img)
dst = np.zeros((height,width,3),np.uint8)
for i in range(0,height):
    for j in range(0,width):
```

```
            (b,g,r) = img[i,j]
            bb = int(b*1.3) + 10
            gg = int(g*1.2) + 15

            if bb>255:
                bb = 255
            if gg>255:
                gg = 255

            dst[i,j] = (bb,gg,r)
# cv2.imshow('dst',dst)
# cv2.waitKey(0)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
dst = cv2.cvtColor(dst, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(14, 6), dpi=100) #设置绘图区域的大小和像素
plt.subplot(121)   #  一行二列第一个
plt.imshow(img)
plt.subplot(122)   #  一行二列第二个
plt.imshow(dst)
plt.show()
```

实验结果如下：

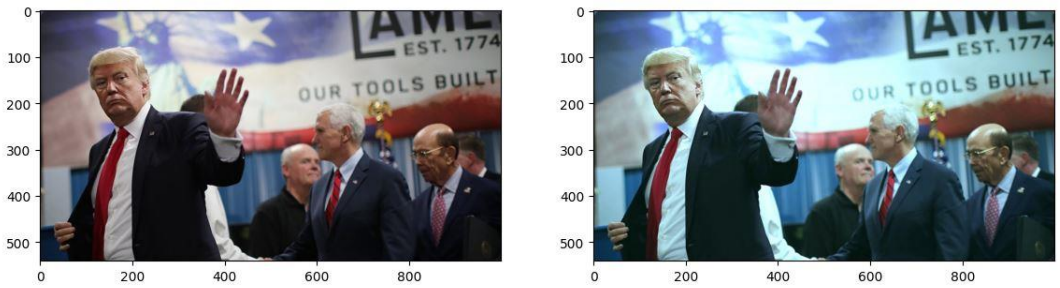


图 6.12.1 磨皮美白实验效果

双边滤波 Python 代码：

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('./images/dicetlp.jpg',1)
#cv2.imshow('src',img)
dst = cv2.bilateralFilter(img,15,35,35)
# cv2.imshow('dst',dst)
# cv2.waitKey(0)
```

```
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
dst = cv2.cvtColor(dst, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(14, 6), dpi=100) #设置绘图区域的大小和像素
plt.subplot(121)  # 一行二列第一个
plt.imshow(img)
plt.subplot(122)  # 一行二列第二个
plt.imshow(dst)
plt.show()
```

实验结果如下：

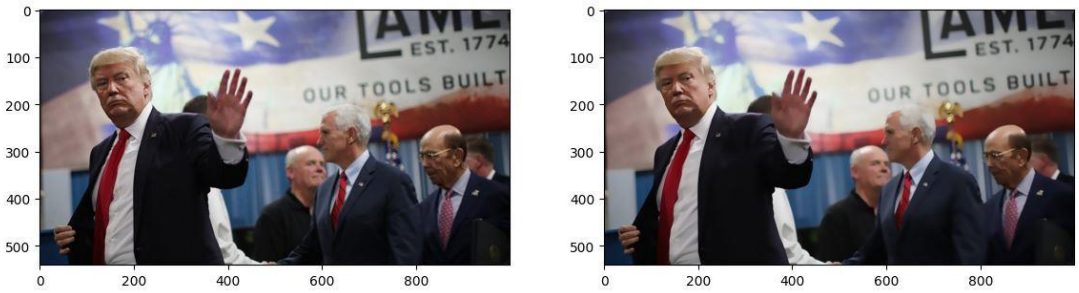


图 6.12.2 双边滤波实验效果

6.12.2、OpenCV 高斯均值滤波

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘25.OpenCV 高斯均值滤波.ipynb’程序，双击用 JupyterLab 打开程序。

- 过滤 ：是信号和图像处理中基本的任务。其目的是根据应用环境的不同，选择性的提取图像中某些认为是重要的信息。过滤可以移除图像中的噪音、提取感兴趣的可视特征、允许图像重采样等等。

- 频域分析 ：将图像分成从低频到高频的不同部分。低频对应图像强度变化小的区域，而高频是图像强度变化非常大的区域。

在频率分析领域的框架中，滤波器是一个用来增强图像中某个波段或频率并阻塞（或降低）其他频率波段的操作。低通滤波器是消除图像中高频部分，但保留低频部分。高通滤波器消除低频部分。

滤波(高通、低通、带通、带阻) 、模糊、去噪、平滑等。

高斯模糊实质上就是一种均值模糊，只是高斯模糊是按照加权平均的，距离越近的点权重越大，距离越远的点权重越小。

通俗的讲，高斯滤波就是对整幅图像进行加权平均的过程，每一个像素点的值，都由其本身和邻域内的其他像素值经过加权平均后得到。

Python 代码：

高斯模糊方法

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('./images/dice007.jpg',1)
# cv2.imshow('src',img)
dst = cv2.GaussianBlur(img,(5,5),1.5)
# cv2.imshow('dst',dst)
# cv2.waitKey(0)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
dst = cv2.cvtColor(dst, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(14, 6), dpi=100) #设置绘图区域的大小和像素
plt.subplot(121)  # 一行二列第一个
plt.imshow(img)
plt.subplot(122)  # 一行二列第二个
plt.imshow(dst)
plt.show()
```

均值

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('./images/dice007.jpg',1)
# cv2.imshow('src',img)
imgInfo = img.shape
height = imgInfo[0]
width = imgInfo[1]
dst = np.zeros((height,width,3),np.uint8)
for i in range(3,height-3):
    for j in range(3,width-3):
        sum_b = int(0)
        sum_g = int(0)
        sum_r = int(0)
```

```
            for m in range(-3,3):#-3 -2 -1 0 1 2
                for n in range(-3,3):
                    (b,g,r) = img[i+m,j+n]
                    sum_b = sum_b+int(b)
                    sum_g = sum_g+int(g)
                    sum_r = sum_r+int(r)


            b = np.uint8(sum_b/36)
            g = np.uint8(sum_g/36)
            r = np.uint8(sum_r/36)
            dst[i,j] = (b,g,r)
# cv2.imshow('dst',dst)
# cv2.waitKey(0)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
dst = cv2.cvtColor(dst, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(14, 6), dpi=100) #设置绘图区域的大小和像素
plt.subplot(121)   #  一行二列第一个
plt.imshow(img)
plt.subplot(122)   #  一行二列第二个
plt.imshow(dst)
plt.show()
```

实验结果如下：

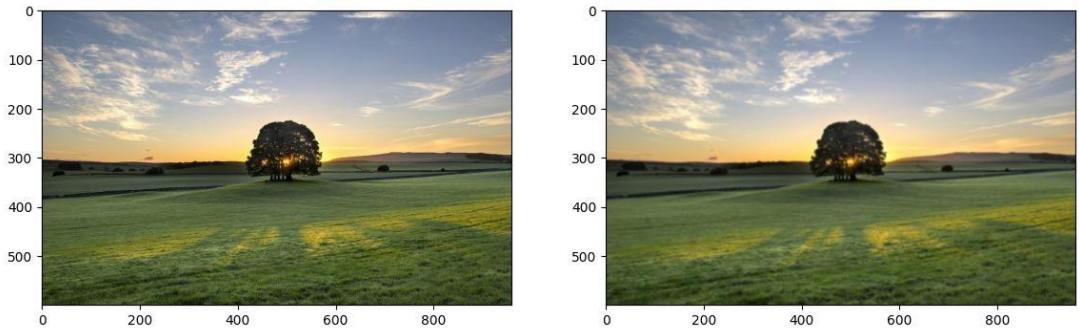


图 6.12.3 高斯模糊方法效果

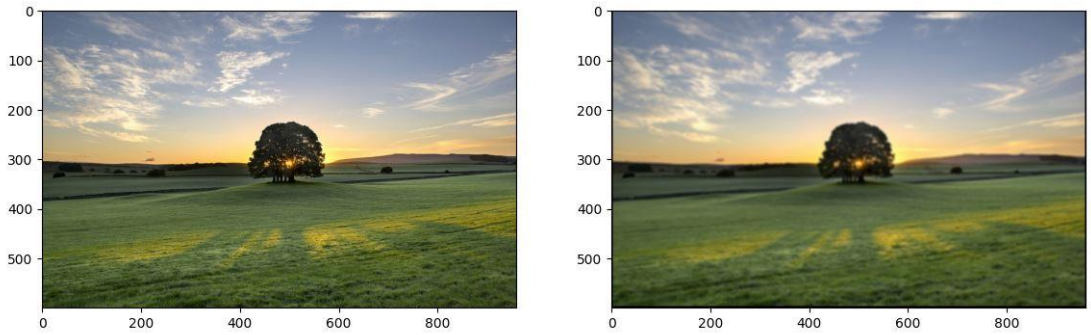


图 6.12.4 均值滤波效果

6.12.3、OpenCV 中值滤波

程序位于‘whitebear’文件夹里的‘AI 基础图像处理’文件夹，找到‘26.OpenCV 中值滤波.ipynb’程序，双击用 JupyterLab 打开程序。

中值滤波：图像平滑处理，可消除椒盐噪声，其基本思路是通过滤波器遍历图像，取每个滤波器区域像素值中值为新的像素值。

算法思路如下：

- 输入图像并转灰；
- 为灰度图添加椒盐噪声
- 遍历像素点，将滤波器区域中的像素值放入一维数组中；
- 对一维数组进行选择排序，并将中间值赋给滤波器中心，即将遍历到的原图像像素点改为滤波器区域中值；
- 输出中值滤波后的图像。

Python 代码：

```
import cv2
import numpy as np
import matplotlib.pyplot as plt


img = cv2.imread('./images/dice007.jpg',1)
imgInfo = img.shape
height = imgInfo[0]
width = imgInfo[1]
img = cv2.cvtColor(img,cv2.COLOR_RGB2GRAY)
# cv2.imshow('src',img)
dst = np.zeros((height,width,3),np.uint8)
collect = np.zeros(9,np.uint8)
```

```
for i in range(1,height-1):
    for j in range(1,width-1):
        k = 0
        for m in range(-1,2):
            for n in range(-1,2):
                gray = img[i+m,j+n]
                collect[k] = gray
                k = k+1
        # 0 1 2 3 4 5 6 7 8
        #     1
        for k in range(0,9):
            p1 = collect[k]
            for t in range(k+1,9):
                if p1<collect[t]:
                    mid = collect[t]
                    collect[t] = p1
                    p1 = mid
        dst[i,j] = collect[4]
# cv2.imshow('dst',dst)
# cv2.waitKey(0)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
dst = cv2.cvtColor(dst, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(14, 6), dpi=100) #设置绘图区域的大小和像素
plt.subplot(121)  # 一行二列第一个
plt.imshow(img)
plt.subplot(122)  # 一行二列第二个
plt.imshow(dst)
plt.show()
```

实验效果如下：

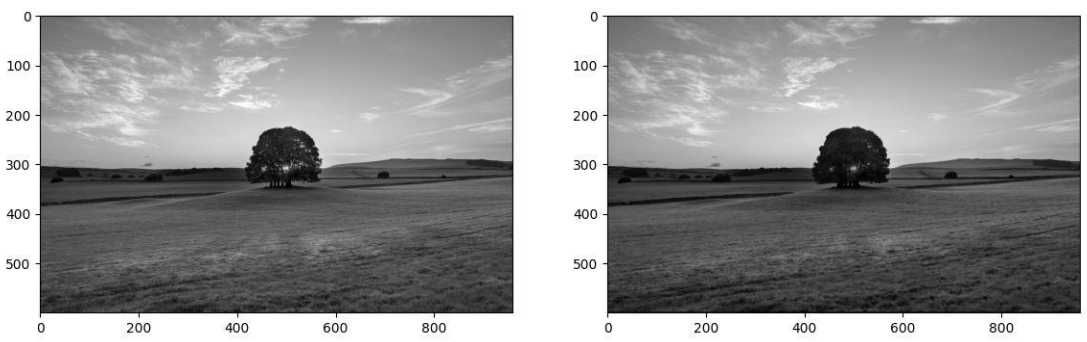


图 6.12.5 中值滤波效果

7、AI视觉综合项目（部分实验需更新）

7.1、使用 OpenCV 进行颜色检测

7.1.1、原理介绍

色彩空间介绍

通常，我们的相机将使用 RGB 颜色模式，可以通过将其视为可以由红色，绿色和蓝色的三种彩色灯制成的所有可能颜色来理解。我们将在这里使用 BGR（蓝色，绿色，红色）代替。

如上所述，对于 BGR，像素由 3 个参数表示，蓝色，绿色和红色。每个参数的值通常为 0-255（或以十六进制表示的 O 到 FF）。例如，计算机屏幕上的纯蓝色像素的 B 值为 255，G 值为 0，R 值为 0。

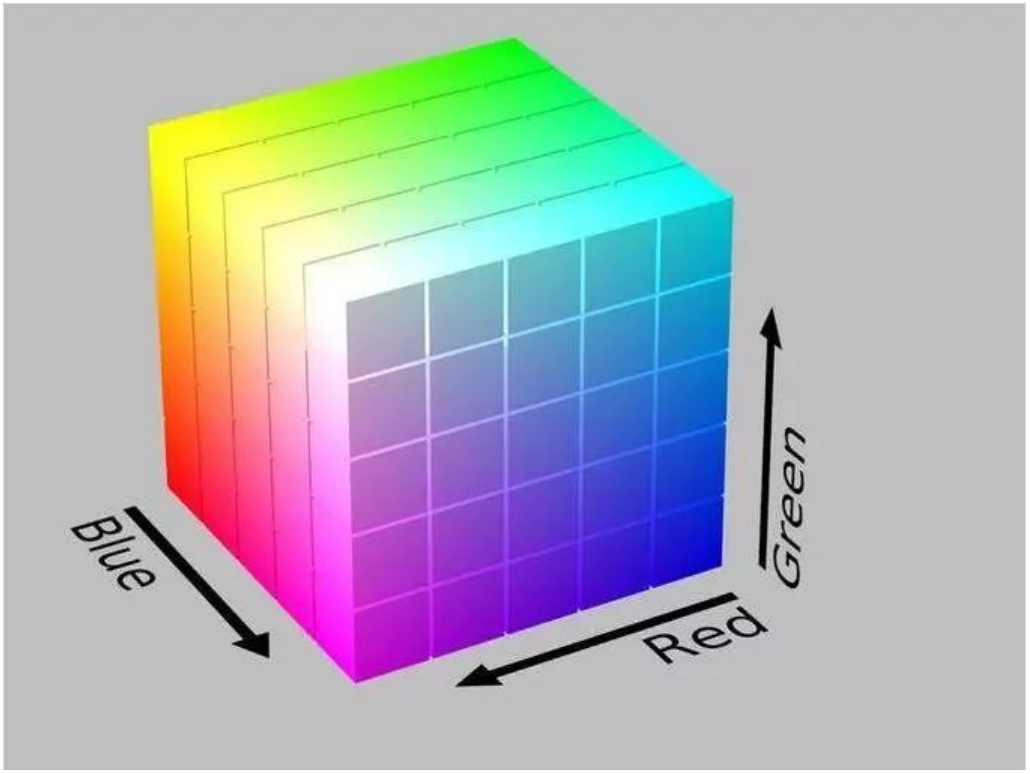


图 7.1.1 BGR

OpenCV 与 HSV（色调，饱和度，值）颜色模型一起使用，它是 RGB 颜色模型的替代表示，由计算机图形学研究人员在 20 世纪 70 年代设计，以更加贴近人类视觉感知颜色制作属性的方式保持一致：

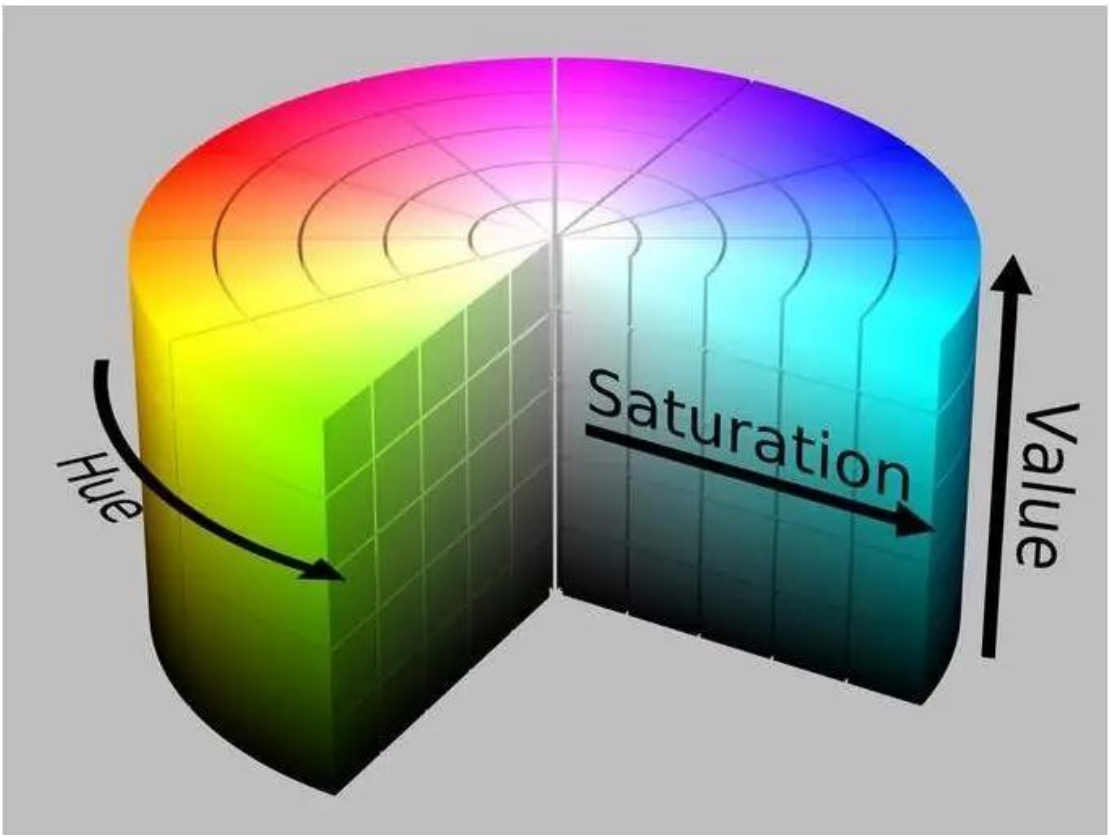


图 7.1.2 HSV

因此，如果要使用 OpenCV 跟踪某种颜色，则必须使用 HSV 模型对其进行定义。

HSV 基本颜色分量范围

一般对颜色空间的图像进行有效处理都是在 HSV 空间进行的，然后对于基本色中对应的 HSV 分量需要给定一个严格的范围，下面是通过实验计算的模糊范围（准确的范围在网上都没有给出）。

- H: 0 — 180
- S: 0 — 255
- V: 0 — 255

此处把部分红色归为紫色范围：

	黑	灰	白	红		橙	黄	绿	青	蓝	紫
hmin	0	0	0	0	156	11	26	35	78	100	125
hmax	180	180	180	10	180	25	34	77	99	124	155
smin	0	0	0	43		43	43	43	43	43	43
smax	255	43	30	255		255	255	255	255	255	255

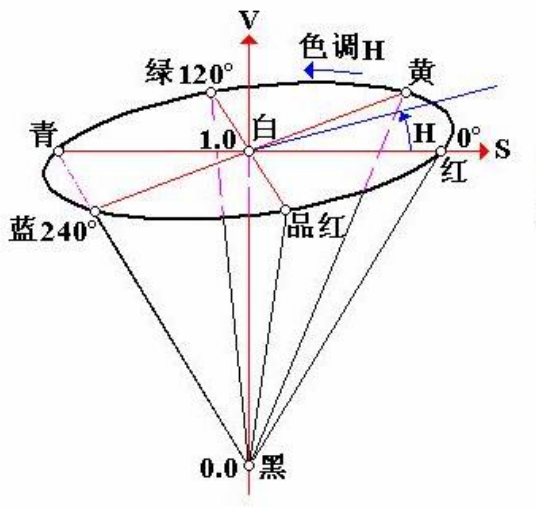
vmin	0	46	221	46	46	46	46	46	46	46			
vamx	46	220	255	255	255	255	255	255	255	255			

Color	R	G	B	H	H_2	C	C_2	V	L	I	Y'_{601}	S_{HSV}	S_{HSL}	S_{HSI}
	1.000	1.000	1.000	n/a	n/a	0.000	0.000	1.000	1.000	1.000	1.000	0.000	0.000	0.000
	0.500	0.500	0.500	n/a	n/a	0.000	0.000	0.500	0.500	0.500	0.500	0.000	0.000	0.000
	0.000	0.000	0.000	n/a	n/a	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
	1.000	0.000	0.000	0.0°	0.0°	1.000	1.000	1.000	0.500	0.333	0.299	1.000	1.000	1.000
	0.750	0.750	0.000	60.0°	60.0°	0.750	0.750	0.750	0.375	0.500	0.664	1.000	1.000	1.000
	0.000	0.500	0.000	120.0°	120.0°	0.500	0.500	0.500	0.250	0.167	0.293	1.000	1.000	1.000
	0.500	1.000	1.000	180.0°	180.0°	0.500	0.500	1.000	0.750	0.833	0.850	0.500	1.000	0.400
	0.500	0.500	1.000	240.0°	240.0°	0.500	0.500	1.000	0.750	0.667	0.557	0.500	1.000	0.250
	0.750	0.250	0.750	300.0°	300.0°	0.500	0.500	0.750	0.500	0.583	0.457	0.667	0.500	0.571
	0.628	0.643	0.142	61.8°	61.5°	0.501	0.494	0.643	0.393	0.471	0.581	0.779	0.638	0.699
	0.255	0.104	0.918	251.1°	250.0°	0.814	0.750	0.918	0.511	0.426	0.242	0.887	0.832	0.756
	0.116	0.675	0.255	134.9°	133.8°	0.559	0.504	0.675	0.396	0.349	0.460	0.828	0.707	0.667
	0.941	0.785	0.053	49.5°	50.5°	0.888	0.821	0.941	0.497	0.593	0.748	0.944	0.893	0.911
	0.704	0.187	0.897	283.7°	284.8°	0.710	0.636	0.897	0.542	0.596	0.423	0.792	0.775	0.686
	0.931	0.463	0.316	14.3°	13.2°	0.615	0.556	0.931	0.624	0.570	0.586	0.661	0.817	0.446
	0.998	0.974	0.532	56.9°	57.4°	0.466	0.454	0.998	0.765	0.835	0.931	0.467	0.991	0.363
	0.099	0.795	0.591	162.4°	163.4°	0.696	0.620	0.795	0.447	0.495	0.564	0.875	0.779	0.800
	0.211	0.149	0.597	248.3°	247.3°	0.448	0.420	0.597	0.373	0.319	0.219	0.750	0.601	0.533
	0.495	0.493	0.721	240.5°	240.4°	0.228	0.227	0.721	0.607	0.570	0.520	0.316	0.290	0.135

HSV 六棱锥

H 参数表示色彩信息，即所处的光谱颜色的位置。该参数用一角度量来表示，红、绿、蓝分别纯度 S 为一比例值，范围从 0 到 1，它表示成所选颜色的纯度和该颜色最大的纯度之间的比率。S=0 时，只有灰度。相隔 120 度。互补色分别相差 180 度。

V 表示色彩的明亮程度，范围从 0 到 1。有一点要注意：它和光强度之间并没有直接的联系。



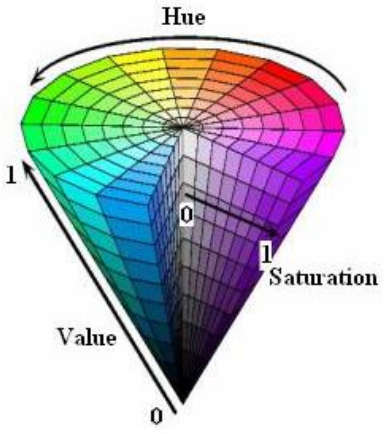


图 7.1.3 HSV 六棱锥

颜色空间转换

在 OpenCV 中有超过 150 中进行颜色空间转换的方法。但是你以后就会发现我们经常用到的也就两种：BGR→Gray 和 BGR→HSV。

我们要用到的函数是：cv2.cvtColor(input_image,flag),其中 flag 就是转换类型。

对于 BGR→Gray 的转换，我们要使用的 flag 就是 cv2.COLOR_BGR2GRAY.同样对于 BGR→HSV 的转换，我们用的 flag 就是 cV2.COLOR_BGR2HSV。

你还可以通过下面的命令得到所有可用的 flag。

```
# -*- coding: utf-8 -*-
import cv2


flags = [i for i in dir(cv2) if i.startswith('COLOR_')]
print(flags)
```

注意：在 OpenCV 的 HSV 格式中，H(色彩 / 色度）的取值范围是［0,179],S(饱和度）的取值范围［0,255],V(亮度）的取值范围［0,255].但是不同的软件使用的值可能不同。所以当你需要拿 OpenCV 的 HSV 值与其他软件的 HSV 值进行对比时，一定要记得归一化。

7.1.2、OpenCV 颜色识别

程序位于‘whitebear’文件夹里的‘AI 视觉综合项目’文件夹，找到‘1.使用 OpenCV 进行颜色检测.ipynb’程序，双击用 JupyterLab 打开程序。

假设我必须跟踪一个黄色物体，如下图所示的塑料盒。容易的部分是找到其 BGR 元素。您可以使用其他软件获取到 RGB 值（比如使用 Inkscape 工具）；

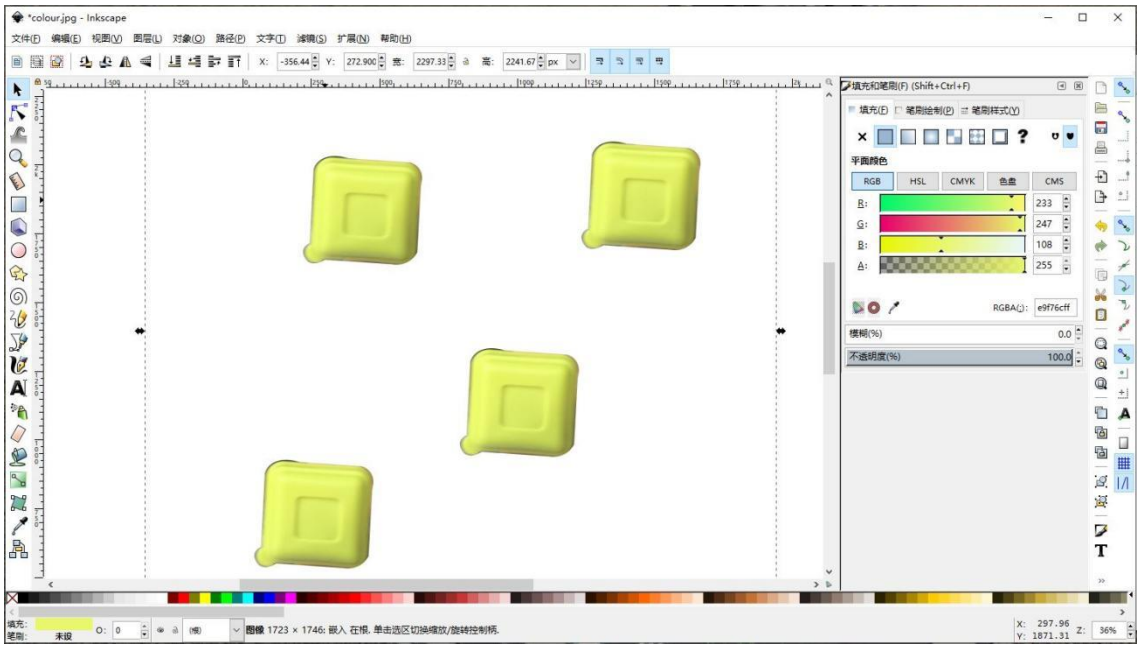


图 7.1.4 Inkscape 软件

从软件中拾取颜色可以发现：

蓝色：108

绿色：247

红色：233

接下来，我们必须将 BGR(108、247、233)模型转换为 HSV 模型，该模型将使用上下范围边界进行定义。

把 BGR 转换为 HSV 颜色空间

```
import sys

import numpy as np

import cv2


print("Please enter blue:")

blue = input()

print("Please enter green:")

green = input()
```

```
print("Please enter red:")

red = input()


color = np.uint8([[[blue, green, red]]])
hsv_color = cv2.cvtColor(color, cv2.COLOR_BGR2HSV)


hue = hsv_color[0][0][0]


print("Lower bound is :")
print("[" + str(hue-10) + ", 100, 100]\n")


print("Upper bound is :"),
print("[" + str(hue + 10) + ", 255, 255]")
```

```
Please enter blue:
 108
Please enter green:
 247
Please enter red:
 233
Lower bound is :
[23, 100, 100]

Upper bound is :
[43, 255, 255]
```

图 7.1.5 填入 BGR 值转换 HSV 数据

得到该组数据之后，我们替换掉程序里面的值：

```
lower_range = np.array([23, 100, 100], dtype=np.uint8)
upper_range = np.array([43, 255, 255], dtype=np.uint8)
```

图 7.1.6 替换程序里面的值

修改完程序之后运行，可以看到如下的效果：

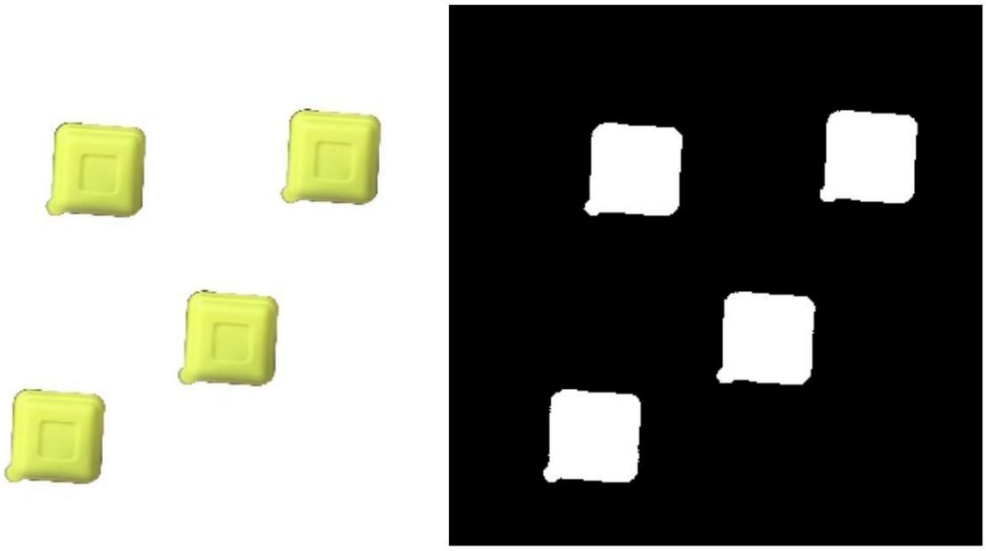


图 7.1.7 颜色检测实验效果

7.1.3、OpenCV 颜色物体追踪

程序位于‘whitebear’文件夹里的‘AI 视觉综合项目’文件夹，找到‘1.使用 OpenCV 进行颜色检测.ipynb’程序，双击用 JupyterLab 打开程序。

这个实验我们将对黄色的圆形物体进行追踪。

OpenCV 中图像 ROI（掩膜）

有时你需要对一幅图像的特定区域进行操作。例如我们要检测一副图像中眼睛的位置，我们首先应该在图像中找到脸，再在脸的区域中找眼睛，而不是直接在一幅图像中搜索。这样会提高程序的准确性和性能。

图像的 ROI 有时候不是很好理解，其实我们只需要知道就是提取我们感兴趣的区域。屏蔽图片的某些区域，在特征提取方面用到的很多！

OpenCV 中颜色物体追踪

现在，我们知道了如何使用（ROI)掩膜“选择”对象，让我们使用摄像头实时跟踪其运动。

首先，确认您是否已安装了 imutils 库。Adrian 收集了 OpenCV 便利功能，使一些基本任务（如调整大小或翻转屏幕）更加容易。如果不是，请使用以下命令输入以在您的 Python 环境中安装该库：

```
sudo pip3 install imutils
```

接下来，我们执行代码：

图 7.1.8 颜色物体追踪实验效果

OpenCV 中颜色物体追踪 Python 代码：

```
# 导入必要的包
from collections import deque
import numpy as np
import argparse
import imutils
import cv2
```

```
# 线程函数操作库
import threading # 线程
import ctypes
import inspect
```

```
# 线程结束代码
def _async_raise(tid, exctype):
    tid = ctypes.c_long(tid)
    if not inspect.isclass(exctype):
        exctype = type(exctype)
    res = ctypes.pythonapi.PyThreadState_SetAsyncExc(tid, ctypes.py_object(exctype))
```

```
        if res == 0:
            raise ValueError("invalid thread id")
        elif res != 1:
            ctypes.pythonapi.PyThreadState_SetAsyncExc(tid, None)
            raise SystemError("PyThreadState_SetAsyncExc failed")

def stop_thread(thread):
    _async_raise(thread.ident, SystemExit)
```

```
# 构造参数解析并解析参数
ap = argparse.ArgumentParser()
ap.add_argument("-v", "--video",help="path to the (optional) video file")
ap.add_argument("-b", "--buffer", type=int, default=64,help="max buffer size")
args = vars(ap.parse_args(args=[]))
```

```
# 定义“黄色对象”的上下边界
# (或“球”)中的 HSV 颜色空间，然后初始化
# 跟踪点列表

colorLower = (24, 100, 100)
colorUpper = (44, 255, 255)
pts = deque(maxlen=args["buffer"])
```

```
# 创建显示控件
from jetcam.utils import bgr8_to_jpeg
#bgr8 转 jpeg 格式
import enum
import cv2

def bgr8_to_jpeg(value, quality=75):
    return bytes(cv2.imencode('.jpg', value)[1])
import traitlets
import threading
import ipywidgets.widgets as widgets
from IPython.display import display
Frame = widgets.Image(format='jpeg', width=640, height=480)
display(Frame)
```

```
dispW=640
```

```
dispH=480
flip=4
#调用 CSI 摄像头的属性参数
camSet='nvarguscamerasrc ！ video/x-raw(memory:NVMM), width=3264, height=2464, format=NV12, framerate=21/1 ! nvvidconv flip-method='+str(flip)+' ! video/x-raw, width='+str(dispW)+', height='+str(dispH)+', format=BGRx ! videoconvert ! video/x-raw, format=BGR ! appsink'

if not args.get("video", False):
    camera = cv2.VideoCapture(0)
else:
    camera = cv2.VideoCapture(args["video"])
```

```
def Video_display():
    while True:
        (grabbed, frame) = camera.read()
        #如果我们在观看视频时没有抓取帧，然后我们就到了视频的结尾了
        if args.get("video") and not grabbed:
            break
        # 调整帧大小，倒转(垂直翻转 180 度)，
        # 模糊它，并转换为 HSV 颜色空间
        frame = imutils.resize(frame, width=600)
        #frame = imutils.rotate(frame, angle=180)
        hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

        # construct a mask for the color "green", then perform
        # a series of dilations and erosions to remove any small
        # blobs left in the mask
        mask = cv2.inRange(hsv, colorLower, colorUpper)
        mask = cv2.erode(mask, None, iterations=2)
        mask = cv2.dilate(mask, None, iterations=2)

        # find contours in the mask and initialize the current
        # (x, y) center of the ball
        cnts = cv2.findContours(mask.copy(), cv2.RETR_EXTERNAL,cv2.CHAIN_APPROX_SIMPLE)[-2]
        center = None

        if len(cnts) > 0:
            c = max(cnts, key=cv2.contourArea)
            ((x, y), radius) = cv2.minEnclosingCircle(c)
```

```
                M = cv2.moments(c)
                center = (int(M["m10"] / M["m00"]), int(M["m01"] / M["m00"]))

                if radius > 10:
                    # draw the circle and centroid on the frame,
                    # then update the list of tracked points
                    cv2.circle(frame, (int(x), int(y)), int(radius),(0, 255, 255), 2)
                    cv2.circle(frame, center, 5, (0, 0, 255), -1)

            pts.appendleft(center)

            for i in range(1, len(pts)):
                if pts[i - 1] is None or pts[i] is None:
                    continue

                thickness = int(np.sqrt(args["buffer"] / float(i + 1)) * 2.5)
                cv2.line(frame, pts[i - 1], pts[i], (0, 0, 255), thickness)

            #cv2.imshow("Frame", frame)
            Frame.value = bgr8_to_jpeg(frame)
        camera.release()

t = threading.Thread(target=Video_display)
t.setDaemon(True)
t.start()
```

```
# 结束线程
stop_thread(t)
```

7.2、OpenCV 中使用 Haar 分类器进行特征检测

7.2.1、OpenCV 脸部和眼睛检测

程序位于‘whitebear’文件夹里的‘AI 视觉综合项目’文件夹，找到‘2.OpenCV 脸部和眼睛检测.ipynb’程序，双击用 JupyterLab 打开程序。

使用 Haar 分类器进行面部及眼睛检测

以 Haar 特征分类器为基础的对象检测技术是一种非常有效的对象检测技术（2001 年 Paul_Viola 和 Michael_Jones 提出）。它是基于机器学习的，通过使用大量的正负样本图像训练得到一个 ascade_function，最后再用它来做对象检测。

现在我们来学习面部检测。开始时，算法需要大量的正样本图像（面部图像）和负样本图像（不含面部的图像）来训练分类器。我们需要从其中提取特征。下图中的 Haar 特征会被使用。它们就像我们的卷积核。每一个特征是一个值，这个值等于黑色矩形中的像素值之后减去白色矩形中的像素值之和。

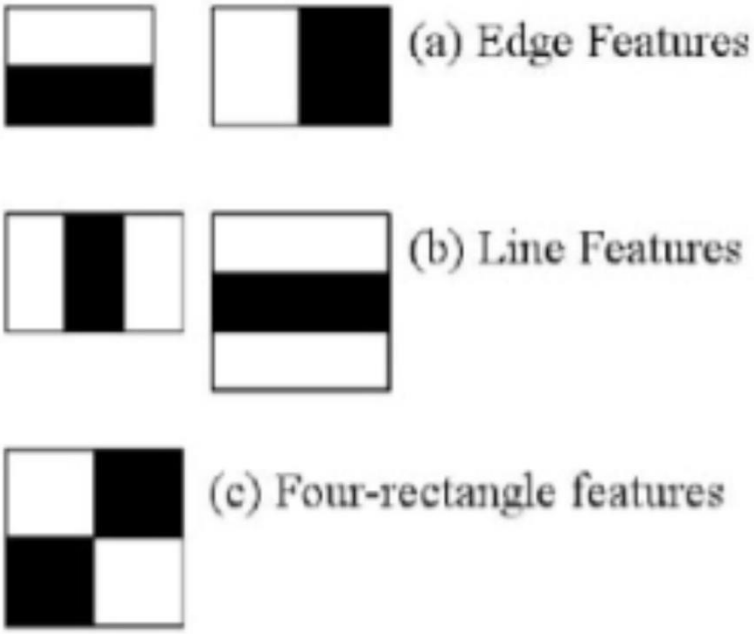


图 7.2.1 Haar 特征

使用所有可能的核来计算足够多的特征。（想象一下这需要多少计算量？仅仅是一个 24x24 的窗口就有 160000 个特征）。对于每一个特征的计算我们好需要计算白色和黑色矩形内的像素和。为了解决这个问题，作者引入了积分图像，这可以大大的简化求和运算，对于任何一个区域的像素和只需要对积分图像上的四个像素操作即可。非常漂亮，它可以使运算速度飞快！

但是在我们计算得到的所有的这些特征中，大多数是不相关的。如下图所示。上边一行显示了两个好的特征，第一个特征看上去是对眼部周围区域的描述，因为眼睛总是比鼻子黑一些。第二个特征是描述的是眼睛比鼻梁要黑一些。

但是如果把这两个窗口放到脸颊的话，就一点都不相关。那么我们怎样从超过 160000+ 个特征中选出最好的特征呢？使用 Adaboost。

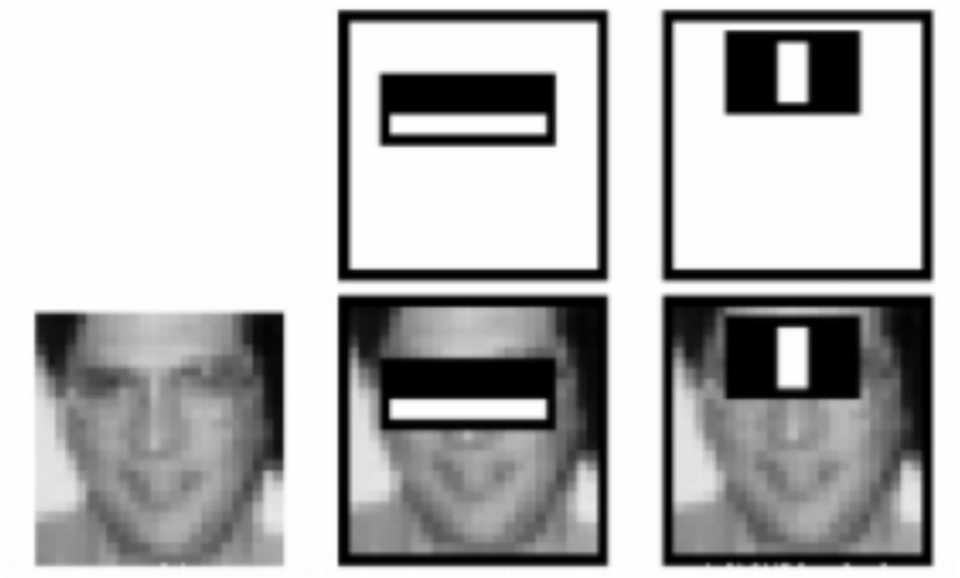


图 7.2.2 眼部区域特征

为了达到这个目的，我们将每一个特征应用于所有的训练图像。对于每一个特征，我们要找到它能够区分出正样本和负样本的最佳阈值。但是很明显，这会产生错误或者错误分类。我们要选取错误率最低的特征，这说明它们是检测面部和非面部图像最好的特征。（这个过程其实不像我们说的这么简单。在开始时每一张图像都具有相同的权重，每一次分类之后，被错分的图像的权重会增大。同样的过程会被再做一遍。然后我们又得到新的错误率和新的权重。重复执行这个过程知道达要求的准确率或者错误率或者要求数目的特征找到）。

最终的分类器是这些弱分类器的加权和。之所以成为弱分类器是应为只是用这些分类器不足以对图像进行分类，但是与其他的分类器联合起来就是一个很强的分类器了。文章中说 200 个特征就能够提供 95% 的准确度了。他们最后使用 6000 个特征。

现在你有一幅图像，对每一个 24x24 的窗口使用这 6000 个特征来做检查，看它是不是面部。这是不是很低效很耗时呢？的确如此，但是有更好的解决方法。

在一副图像中大多数区域是非面部区域。所以最好有一个简单的方法来证明这个窗口不是面部区域，如果不是就直接抛弃，不用对它再做处理。而不是集中在研究这个区域是不是面部。按照这种方法我们可以在可能是面部的区域多花点时间。 为了达到这个目的作者提出了级联分类器的概念。不是在一开始就对窗口进行这 6000 个特征测试，将这些特征分成不同组。在不同的分类阶段逐个使用。（通常前面很少的几个阶段使用较少的特征检测）。如果一个窗口第一阶段的检测都过不了就可以直接放弃后面的测试了，如果它通过了就进入第二阶段的检测。如果一个窗口经过了所有的测试，那么这个窗口就被认为是面部区域。

采用的模型将 6000 多个特征分为 38 个阶段，前五个阶段的特征数分别为 1，10， 25， 25 和 50。（上图中的两个特征其实就是从 Adaboost 获得的最好特征）。

OpenCV 中的 Haar 级联检测

OpenCV 自带了训练器和检测器。如果你想自己训练一个分类器来检测汽车，飞机等的话，可以使用 OpenCV 构建。其中的细节在这里： Cascade Classifier Training 现在我们来学习一下如何使用检测器。 OpenCV 已经包含了很多已经训练好的分类器，其中包括：面部，眼睛，微笑等。这些 XML 文件保存在/whitebear/5.AI 视觉综合项目/images/haarcascades/文件夹中。下面我们将使用 OpenCV 创建一个面部和眼部检测器。

首先我们要加载需要的 XML 分类器。然后以灰度格式加载输入图像或者是视频。

现在我们在图像中检测面部。如果检测到面部，它会返回面部所在的矩形区域 Rect（x,y,w,h）。一旦我们获得这个位置，我们可以创建一个 ROI 并在其中进行眼部检测。

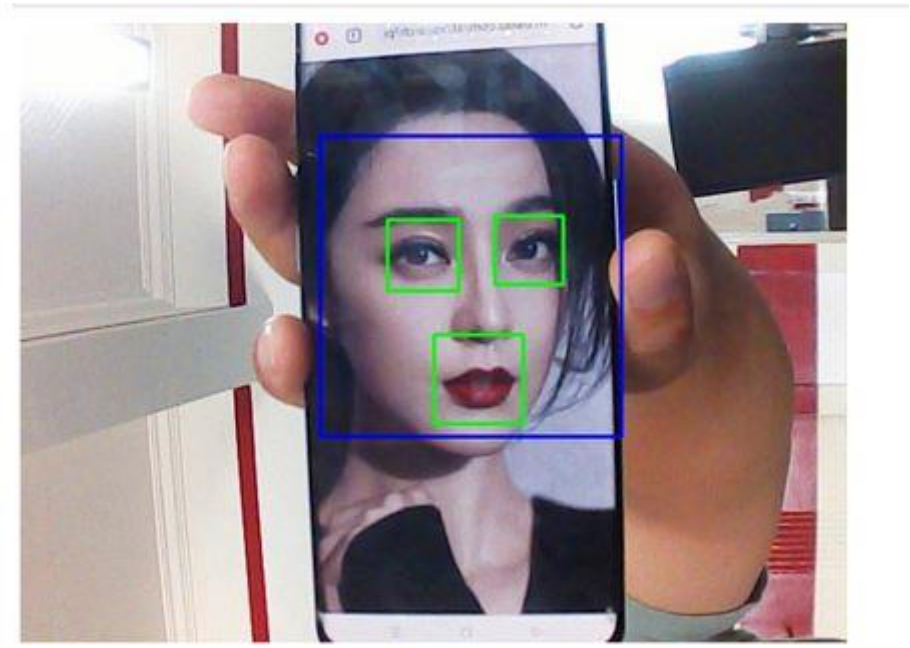


图 7.2.3 脸部和眼睛检测效果

Python 代码：

```
# 导入必要的库
import numpy as np
import cv2
```

```
# 载入 HAAR 模型
face_cascade = cv2.CascadeClassifier('./images/haarcascade_frontalface_default.xml')
eye_cascade = cv2.CascadeClassifier('./images/haarcascade_eye.xml')
```

```
# 线程函数操作库
import threading # 线程
import ctypes
import inspect

# 线程结束代码
def _async_raise(tid, exctype):
    tid = ctypes.c_long(tid)
    if not inspect.isclass(exctype):
        exctype = type(exctype)
    res = ctypes.pythonapi.PyThreadState_SetAsyncExc(tid, ctypes.py_object(exctype))
    if res == 0:
        raise ValueError("invalid thread id")
```

```
        elif res != 1:
            ctypes.pythonapi.PyThreadState_SetAsyncExc(tid, None)
            raise SystemError("PyThreadState_SetAsyncExc failed")

def stop_thread(thread):
    _async_raise(thread.ident, SystemExit)
```

```
# 创建显示控件
#from jetcam.utils import bgr8_to_jpeg
import enum
import cv2

def bgr8_to_jpeg(value, quality=75):
    return bytes(cv2.imencode('.jpg', value)[1])
import traitlets
import threading
import ipywidgets.widgets as widgets
from IPython.display import display
face_imge = widgets.Image(format='jpeg', width=480, height=320)
display(face_imge)
```

```
#dispW=480
#dispH=320
#flip=4
#Uncomment These next Two Line for Pi Camera
#camSet='nvarguscamerasrc !  video/x-raw(memory:NVMM), width=3264, height=2464, format=NV12,
framerate=21/1 ! nvvidconv flip-method='+str(flip)+' ! video/x-raw, width='+str(dispW)+',
height='+str(dispH)+', format=BGRx ! videoconvert ! video/x-raw, format=BGR ! appsink'
#cap= cv2.VideoCapture(0)



dispW=480
dispH=320
flip=4
#Uncomment These next Two Line for Pi Camera
cap = cv2.VideoCapture(0)                                    #打开摄像头
```

```
def Video_display():
    while True:
```

```
            ret, img = cap.read()
            img = cv2.flip(img,1)
            gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
            faces = face_cascade.detectMultiScale(gray, 1.3, 5)
            for (x,y,w,h) in faces:
                cv2.rectangle(img,(x,y),(x+w,y+h),(255,0,0),2)
                roi_gray = gray[y:y+h, x:x+w]
                roi_color = img[y:y+h, x:x+w]
                print(int(x+w/2), int(y+h/2))
                eyes = eye_cascade.detectMultiScale(roi_gray)
                for (ex,ey,ew,eh) in eyes:
                    cv2.rectangle(roi_color,(ex,ey),(ex+ew,ey+eh),(0,255,0),2)
            face_imge.value = bgr8_to_jpeg(img)
        cap.release()
```

```
t = threading.Thread(target=Video_display)
t.setDaemon(True)
t.start()
```

```
# 结束线程
stop_thread(t)
```

7.2.2、OpenCV 实现行人检测

程序位于‘whitebear’文件夹里的‘AI 视觉综合项目’文件夹，找到‘2.OpenCV 脸部和眼睛检测.ipynb’程序，双击用 JupyterLab 打开程序。

OpenCV 实现行人检测

我们可以采用官方自带的人体检测 HAAR 分类器实现对视频的人体检测，我们也可以自己改成摄像头的方式，下面我们来一起实验一下吧！

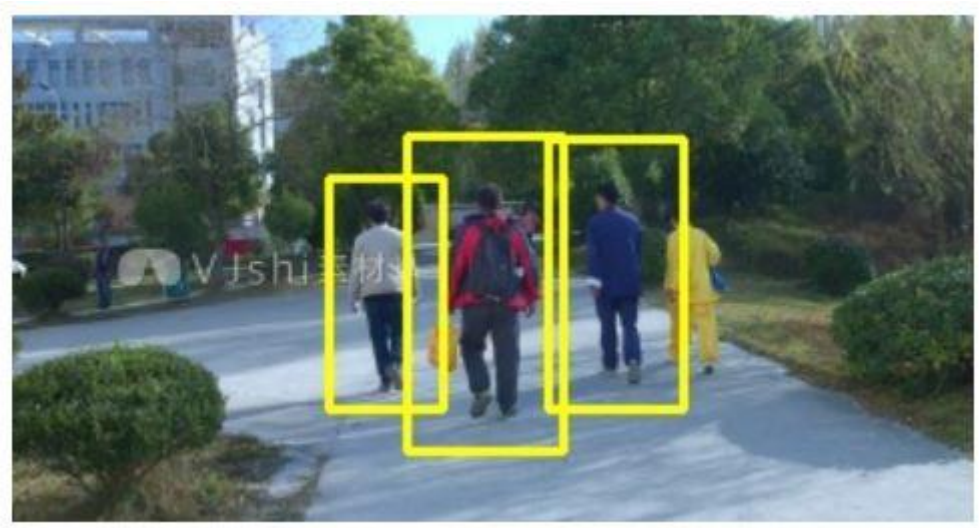


图 7.2.4 行人检测实验效果

Python 代码：

```
# 载入必要的库
import cv2
import numpy as np

# 创建我们的身体分类器
body_classifier = cv2.CascadeClassifier('./images/haarcascade_fullbody.xml')
```

```
# 为视频文件启动视频捕获
cap = cv2.VideoCapture('./images/walking1.mp4')
```

```
# 线程函数操作库
import threading # 线程
import ctypes
import inspect

# 线程结束代码
def _async_raise(tid, exctype):
    tid = ctypes.c_long(tid)
    if not inspect.isclass(exctype):
        exctype = type(exctype)
    res = ctypes.pythonapi.PyThreadState_SetAsyncExc(tid, ctypes.py_object(exctype))
    if res == 0:
        raise ValueError("invalid thread id")
    elif res != 1:
        ctypes.pythonapi.PyThreadState_SetAsyncExc(tid, None)
```

```
            raise SystemError("PyThreadState_SetAsyncExc failed")


def stop_thread(thread):
    _async_raise(thread.ident, SystemExit)
```

```
# 创建显示控件
from jetcam.utils import bgr8_to_jpeg
import traitlets
import ipywidgets.widgets as widgets
from IPython.display import display
Pedestrians_imge = widgets.Image(format='jpeg', width=480, height=320)
display(Pedestrians_imge)
```

```
#一旦视频成功加载，循环播放
def Video_display():
    while cap.isOpened():
        # Read first frame
        ret, frame = cap.read()
        frame = cv2.resize(frame, None,fx=0.5, fy=0.5, interpolation = cv2.INTER_LINEAR)

        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        # Pass frame to our body classifier
        bodies = body_classifier.detectMultiScale(gray, 1.2, 3)

        # Extract bounding boxes for any bodies identified
        for (x,y,w,h) in bodies:
            cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 255), 2)
            Pedestrians_imge.value = bgr8_to_jpeg(frame)
    cap.release()
```

```
t = threading.Thread(target=Video_display)
t.setDaemon(True)
t.start()
```

```
# 结束线程
stop_thread(t)
```

7.2.3、OpenCV 实现汽车检测

程序位于‘whitebear’文件夹里的‘AI 视觉综合项目’文件夹，找到‘4.OpenCV 实现汽车检测.ipynb’程序，双击用 JupyterLab 打开程序。

OpenCV 实现汽车检测

我们可以采用官方自带的汽车检测 HAAR 分类器实现对视频的汽车检测，我们也可以自己改成摄像头的方式，下面我们来一起实验一下吧！

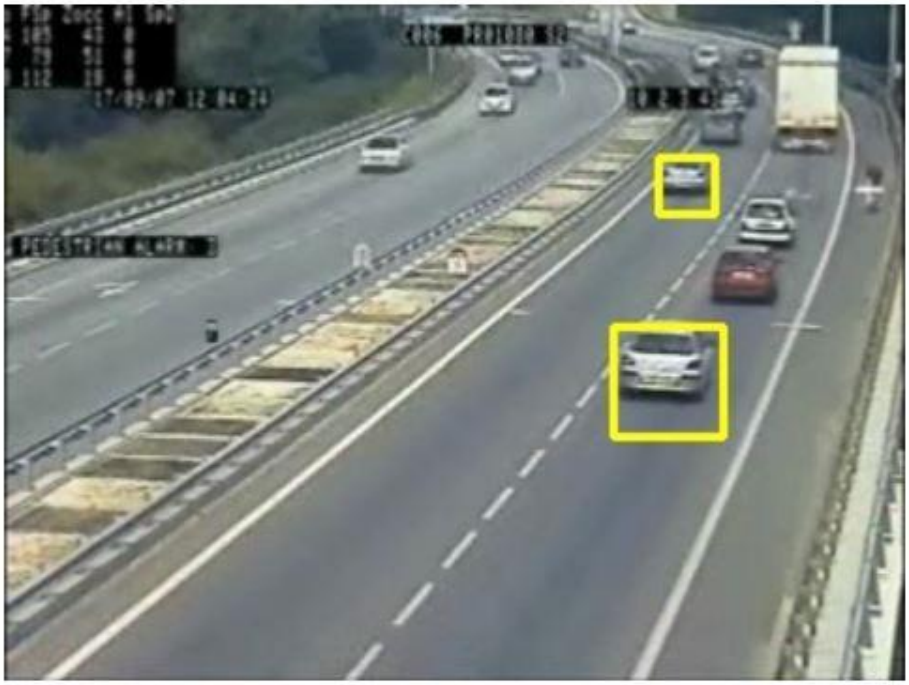


图 7.2.5 汽车检测实验效果

Python 代码：

```
# 载入必要的库
import cv2
import time
import numpy as np
```

```
# 载入 HAAR 分类器
car_classifier = cv2.CascadeClassifier('./images/haarcascade_car.xml')
# 载入视频文件
cap = cv2.VideoCapture('./images/cars.avi')
```

```
# 线程函数操作库
import threading # 线程
import ctypes
import inspect
```

```
# 线程结束代码
def _async_raise(tid, exctype):
    tid = ctypes.c_long(tid)
    if not inspect.isclass(exctype):
        exctype = type(exctype)
    res = ctypes.pythonapi.PyThreadState_SetAsyncExc(tid, ctypes.py_object(exctype))
    if res == 0:
        raise ValueError("invalid thread id")
    elif res != 1:
        ctypes.pythonapi.PyThreadState_SetAsyncExc(tid, None)
        raise SystemError("PyThreadState_SetAsyncExc failed")

def stop_thread(thread):
    _async_raise(thread.ident, SystemExit)
```

```
# 创建显示控件
from jetcam.utils import bgr8_to_jpeg
import traitlets
import ipywidgets.widgets as widgets
from IPython.display import display
car_imge = widgets.Image(format='jpeg', width=480, height=320)
display(car_imge)
```

```
#一旦视频成功加载，循环播放
def car_Video_display():
    while cap.isOpened():
        time.sleep(.05)
        # Read first frame
        ret, frame = cap.read()
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # Pass frame to our car classifier
        cars = car_classifier.detectMultiScale(gray, 1.4, 2)
        # Extract bounding boxes for any bodies identified
        for (x,y,w,h) in cars:
            cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 255), 2)
            car_imge.value = bgr8_to_jpeg(frame)
    cap.release()
```

```
t = threading.Thread(target=car_Video_display)
t.setDaemon(True)
t.start()
```

```
# 结束线程
stop_thread(t)
```

7.3、OpenCV 进行车牌检测

7.3.1、OpenCV 进行车牌检测

程序位于‘whitebear’文件夹里的‘AI 视觉综合项目’文件夹，找到‘5.OpenCV 车牌检测.ipynb’程序，双击用 JupyterLab 打开程序。

Python pytesseract 字符识别 OCR 库介绍

Python 提供了支持 Teseract-OCR 引擎的 Python 版本的库 pytesseract,pytessea 是一欺用于光学字符识别（OCR)的 python 工具，即从图片中识别出和“读取”其中嵌入的文字。pytesseract 是对 Tesseract-OCR 的一层封装，同时也可以单独作为对 Tesseract 引擎的调用脚本，支持使用 PIL 库（Python Imagjing Library)读取各种图片文件类型，包括 jpeg、png、gif、bmp、tif 等其它格式。作为脚本使用时，pytesseract 将打印识别出的文字，而不是将其写入文件。

Pytesseract 字符识别库安装

```
sudo apt-get update
sudo apt-get install libleptonica-dev
sudo apt-get install tesseract-ocr
sudo apt-get install libtesseract-dev
pip3 install pytesseract
pip3 install pillow
```

车牌识别相关操作

1.车牌检测：第一步是从汽车上检测车牌所在位置。我们将使用 OpenCV 中矩形的轮廓检测来寻找车牌。如果我们知道车牌的确切尺寸，颜色和大致位置，则可以提高准确性。通常，也会将根据摄像机的位置和该特定国家 / 地区所使用的车牌类型来训练检测算法。但是图像可能并没有汽车的存在，在这种情况下我们将先进行汽车的，然后是车牌。

2.字符分割：检测到车牌后，我们必须将其裁剪并保存为新图像。同样，这可以使用 OpenCV 来完成。

3.字符识别：现在，我们在上一步中获得的新图像肯定可以写上一些字符（数字 / 字母）。因此，我们可以对其执行 OCR(光学字符识别）以检测数字。

OpenCV 车牌检测

让我们以汽车的样本图像为例，首先检测该汽车上的车牌。然后，我们还将使用相同的图像进行字符分割和字符识别。

使用的测试图像如下所示：

图 7.3.1 待测试的车牌图像

运行程序后的识别效果：

图 7.3.2 车牌检测识别车牌位置

识别结果：

```
programming_fever's License Plate Recognition

Detected license plate Number is: CZ20FSE
```

图 7.3.3 车牌检测识别结果

Python 代码：

```
import cv2
import imutils
import numpy as np
import pytesseract
```

```
img    = cv2.imread('./images/car.jpg',cv2.IMREAD_COLOR)
```

```
img = cv2.resize(img, (600,400) )
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = cv2.bilateralFilter(gray, 13, 15, 15)
```

```
edged = cv2.Canny(gray, 30, 200)
contours = cv2.findContours(edged.copy(), cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
contours = imutils.grab_contours(contours)
contours = sorted(contours, key = cv2.contourArea, reverse = True)[:10]
screenCnt = None
```

```
# 载入显示库
from jetcam.utils import bgr8_to_jpeg
import ipywidgets.widgets as widgets
from IPython.display import display
imagecar = widgets.Image(format='jpeg', width=600, height=400)
imageCropped = widgets.Image(format='jpeg', width=600, height=400)
display(imagecar)
display(imageCropped)
```

```
for c in contours:

    peri = cv2.arcLength(c, True)
    approx = cv2.approxPolyDP(c, 0.018 * peri, True)

    if len(approx) == 4:
```

```
            screenCnt = approx
            break

if screenCnt is None:
    detected = 0
    print ("No contour detected")
else:
     detected = 1

if detected == 1:
    cv2.drawContours(img, [screenCnt], -1, (0, 0, 255), 3)

mask = np.zeros(gray.shape,np.uint8)
new_image = cv2.drawContours(mask,[screenCnt],0,255,-1,)
new_image = cv2.bitwise_and(img,img,mask=mask)

(x, y) = np.where(mask == 255)
(topx, topy) = (np.min(x), np.min(y))
(bottomx, bottomy) = (np.max(x), np.max(y))
Cropped = gray[topx:bottomx+1, topy:bottomy+1]

text = pytesseract.image_to_string(Cropped, config='--psm 11')
print("programming_fever's License Plate Recognition\n")
print("Detected license plate Number is:",text)
img = cv2.resize(img,(500,300))
Cropped = cv2.resize(Cropped,(400,200))
# 显示图像
imagecar.value = bgr8_to_jpeg(img)
imageCropped.value = bgr8_to_jpeg(Cropped)
```

7.4、OpenCV 定位物体的实时位置

7.4.1、OpenCV 定位物体的实时位置

程序位于‘whitebear’文件夹里的‘AI 视觉综合项目’文件夹，找到‘6.OpenCV 定位物体的实时位置.ipynb’程序，双击用 JupyterLab 打开程序。

实验思路

这里的想法是使用平移 / 倾斜舵机机械臂将对象放置在屏幕中间。但是问题是，对于运动的物体，我们必须实时知道对象的位置。所以这里我们一旦有了对象中心的坐标，这将非常容易。

首先，我们之前已经学习过识别颜色，这里我们对其进行修改以打印所建立对象的 x,y 坐标。

代码的“核心”是我们找到对象并在其上画一个圆并在其中心带有一个红点的部分。采用了双色 LED 灯作为检测到物体的显示

实验操作：

搭建电路：

GPIO Expansion Board	双色 LED 模块
GPIO17	R
GND	GND

让我们将中心坐标“导出”到 mapobjectPosition(int(x),int(y))函数以打印其坐标。

```
# 打印物体的实时位置
def mapObjectPosition (x, y):
    print ("[INFO] Object Center coordenates at X0 = {0} and Y0 =  {1}".format(x, y))
```

运行程序，我们将在终端上看到（x,y)位置坐标，如上所示。移动对象并观察坐标。我们将意识到 x 从 0 到 500(从左到右），y 从 0 到 350(从上到下）。

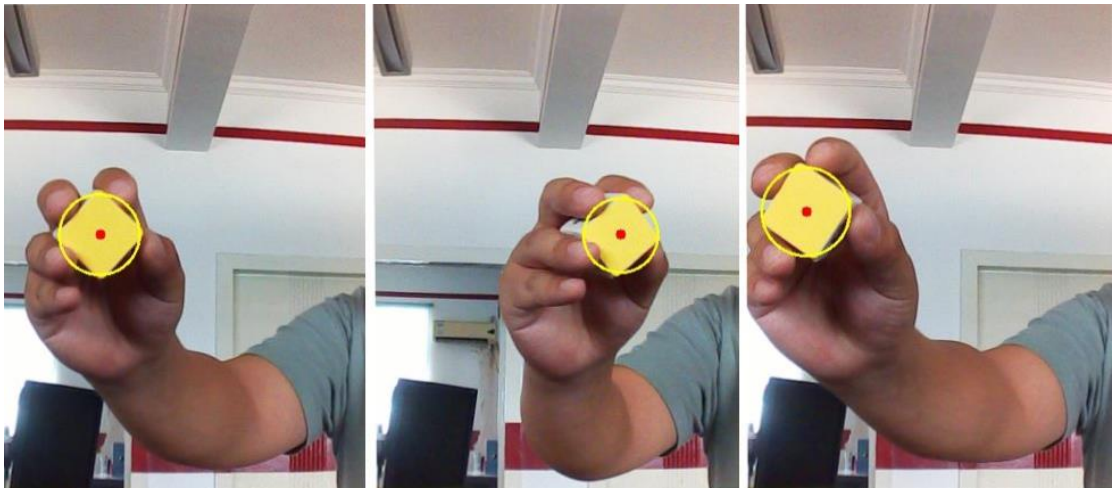


图 7.4.1 实验中物体检测

现在，我们必须将这些坐标用作平移 / 倾斜跟踪系统的起点。

在 JupyterLab 上实时打印出坐标信息：

```
[INFO] Object Center coordenates at X0 = 1 and Y0 =  293
[INFO] Object Center coordenates at X0 = 34 and Y0 =  297
[INFO] Object Center coordenates at X0 = 26 and Y0 =  280
[INFO] Object Center coordenates at X0 = 79 and Y0 =  279
[INFO] Object Center coordenates at X0 = 121 and Y0 =  279
[INFO] Object Center coordenates at X0 = 162 and Y0 =  282
[INFO] Object Center coordenates at X0 = 199 and Y0 =  283
[INFO] Object Center coordenates at X0 = 234 and Y0 =  288
[INFO] Object Center coordenates at X0 = 265 and Y0 =  293
[INFO] Object Center coordenates at X0 = 294 and Y0 =  298
[INFO] Object Center coordenates at X0 = 320 and Y0 =  303
[INFO] Object Center coordenates at X0 = 338 and Y0 =  308
[INFO] Object Center coordenates at X0 = 353 and Y0 =  311
[INFO] Object Center coordenates at X0 = 365 and Y0 =  309
[INFO] Object Center coordenates at X0 = 373 and Y0 =  310
[INFO] Object Center coordenates at X0 = 381 and Y0 =  313
[INFO] Object Center coordenates at X0 = 383 and Y0 =  317
[INFO] Object Center coordenates at X0 = 381 and Y0 =  323
[INFO] Object Center coordenates at X0 = 380 and Y0 =  326
[INFO] Object Center coordenates at X0 = 376 and Y0 =  323
[INFO] Object Center coordenates at X0 = 372 and Y0 =  318
[INFO] Object Center coordenates at X0 = 367 and Y0 =  313
[INFO] Object Center coordenates at X0 = 364 and Y0 =  309
[INFO] Object Center coordenates at X0 = 363 and Y0 =  307
```

图 7.4.2 实验中打印坐标信息

当识别到物体的时候，我们可以观察双色 LED 灯的变化。

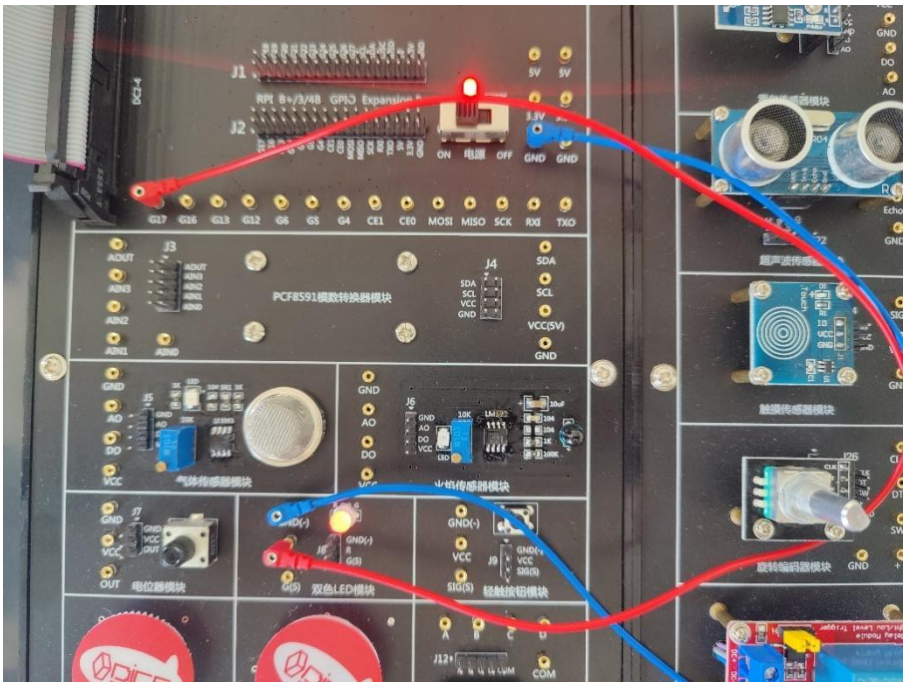


图 7.4.3 检测到物体双色 LED 状态

Python 代码：

```
# 载入必要的库文件
from __future__ import print_function
from imutils.video import VideoStream
import imutils
import time
import cv2
import os
import RPi.GPIO as GPIO
```

```
# LED 初始化
redLed = 17
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)
GPIO.setup(redLed, GPIO.OUT)
```

```
# 打印物体的实时位置
def mapObjectPosition (x, y):
    print ("[INFO] Object Center coordenates at X0 = {0} and Y0 =  {1}".format(x, y))
```

```
dispW=500
dispH=350
```

```
flip=4
#调用 CSI 摄像头的属性参数
# 初始化视频流并允许摄像机传感器预热
#print("[INFO] waiting for camera to warmup...")
vs = cv2.VideoCapture(0)
#time.sleep(2.0)
```

```
# 定义对象的上下边界
# 在 HSV 颜色空间中进行跟踪
colorLower = (9,135,231)
colorUpper = (31,255,255)
```

```
# 创建显示控件

import enum
import cv2
from jetcam.utils import bgr8_to_jpeg

def bgr8_to_jpeg(value, quality=75):
    return bytes(cv2.imencode('.jpg', value)[1])
import traitlets
import threading
import ipywidgets.widgets as widgets
from IPython.display import display
Frame = widgets.Image(format='jpeg', width=500, height=350)
display(Frame)
```

```
# 线程函数操作库
import threading # 线程
import ctypes
import inspect

# 线程结束代码
def _async_raise(tid, exctype):
    tid = ctypes.c_long(tid)
    if not inspect.isclass(exctype):
        exctype = type(exctype)
    res = ctypes.pythonapi.PyThreadState_SetAsyncExc(tid, ctypes.py_object(exctype))
    if res == 0:
```

```
            raise ValueError("invalid thread id")
        elif res != 1:
            ctypes.pythonapi.PyThreadState_SetAsyncExc(tid, None)
            raise SystemError("PyThreadState_SetAsyncExc failed")


def stop_thread(thread):
    _async_raise(thread.ident, SystemExit)
```

```
# 关闭 LED 灯
GPIO.output(redLed, GPIO.LOW)
ledOn = False
```

```
def Video_display():
    global ledOn
    # 循环的帧从视频流
    while True:
        # 从视频流中抓取下一帧，调整大小
        # 帧，并将其转换为 HSV 颜色空间
        ret, frame = vs.read()
        frame = imutils.resize(frame, width=500)
        frame = imutils.rotate(frame, angle=0)
        hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

        # 为对象颜色构造一个遮罩，然后执行
        # 一系列的膨胀和侵蚀，以消除任何小的
        # blobs left in the mask
        mask = cv2.inRange(hsv, colorLower, colorUpper)
        mask = cv2.erode(mask, None, iterations=2)
        mask = cv2.dilate(mask, None, iterations=2)
        # 找到遮罩中的轮廓并初始化
        # (x, y) center of the object
        cnts = cv2.findContours(mask.copy(), cv2.RETR_EXTERNAL,cv2.CHAIN_APPROX_SIMPLE)[-2]
        #cnts = cnts[0] if imutils.is_cv2() else cnts[1]
        center = None

        # 只有在找到至少一条轮廓线时才进行
        if len(cnts) > 0:
            # 在蒙版中找到最大的轮廓，然后使用
            # 它可以计算出最小的围圆
```

```
            # 重心
            c = max(cnts, key=cv2.contourArea)
            ((x, y), radius) = cv2.minEnclosingCircle(c)
            M = cv2.moments(c)
            center = (int(M["m10"] / M["m00"]), int(M["m01"] / M["m00"]))
            # 只有当半径满足最小尺寸时才进行
            if radius > 10:
                # 在框架上画圆和质心，
                # 然后更新跟踪点的列表
                cv2.circle(frame, (int(x), int(y)), int(radius),(0, 255, 255), 2)
                cv2.circle(frame, center, 5, (0, 0, 255), -1)
                # 定位舵机在圆心
                mapObjectPosition(int(x), int(y))
                # 如果 led 还没有打开，打开 led
                if not ledOn:
                    GPIO.output(redLed, GPIO.HIGH)
                    ledOn = True
        # 如果没有检测到球，关闭 LED 灯
        elif ledOn:
            GPIO.output(redLed, GPIO.LOW)
            ledOn = False

        # 向我们的屏幕显示框架
        Frame.value = bgr8_to_jpeg(frame)
    # 做点清理工作
    print("\n [INFO] Exiting Program and cleanup stuff \n")
    GPIO.cleanup()
    vs.stop()
```

```
# 开始线程
t = threading.Thread(target=Video_display)
t.setDaemon(True)
t.start()
```

```
# 结束线程
stop_thread(t)
```

7.5、OpenCV 运动检测与追踪

7.5.1、OpenCV 运动检测与追踪

程序位于‘whitebear’文件夹里的‘AI 视觉综合项目’文件夹，找到‘7.OpenCV 运动检测与追踪.ipynb’程序，双击用 JupyterLab 打开程序。

运动检测说明

如果对于家庭监控来说，如果我需要一直监控着家里情况的变化，是否有小偷闯入，由于摄像头是一直不动的，我们只需要检测是否有运动的物体即可，如何使用计算机视觉技术构建用于家庭监视的基本运动检测和跟踪系统，则是我们下面需要考虑的问题。

OpenCV 运动检测具体方法介绍

最为简单的为二帧法，但是在 OpenCV4 版本移去了该函数，在许多计算机视觉应用中至关重要。我们用它来计算通过收费站的汽车数量。我们用它来计算进出商店的人数。

在实际用途中有很多方法可以在 OpenCV 中执行运动检测，跟踪和分析，有些很简单，还有一些非常复杂。

两种主要方法是基于高斯混合模型的前景和背景分割形式：

- OpenCV 中提供了一种改进的自适应背景混合模型，用于阴影跟踪的实时跟踪，可通过 cv2.BackgroundSubtractorMOG 函数实现，但是在 OpenCV4 版本移去了该函数。

- OpenCV 中也有改进了背景减除的自适应高斯混合模型，并且也提供了针对背景扣除任务的每个图像像素的高效自适应密度估计，可通过 cv2.BackgroundSubtractorMOG2 功能。

所有这些方法都与从前景中分割背景有关（它们甚至为我们提供了辨别实际运动与阴影和较小照明变化之间的机制）！

那么，为什么如此重要？为什么我们要关心哪些像素属于前景，哪些像素属于背景？

好吧，在运动检测中，我们倾向于做出以下假设：

视频流的背景在很大程度上是静态的，并且在视频的连续帧中保持不变。因此，如果我们可以对背景建模，就可以对其进行实质性的更改进行监视。如果有重大变化，我们可以检测到一这种变化通常对应于视频中的运动。

现在，显然在现实世界中，这种假设很容易失败。由于阴影反射，照明条件以及环境中任何其他可能的变化，我们的背景在视频的各个帧中看起来可能完全不同。而且，如果背景看起来不同，则可能会使我们的算法失效。这就是为什么最成功的背景减法 / 前景检测系统使用固定安装的摄像机并在可控制的照明条件下使用的原因。

上面提到的方法虽然功能强大，但是对主板的性能要求比较高，有的只能在计算机上运行。而我们的任务是要部署在 Jetson Nano 主板上，由于 Jetson Nano 主板的性能原因，一般来说，简单的方法运行起来会速度比较理想一些。

使用 Python 和 OpenCV 进行基本的运动检测和跟踪

我们我可以运行程序，看一下效果。

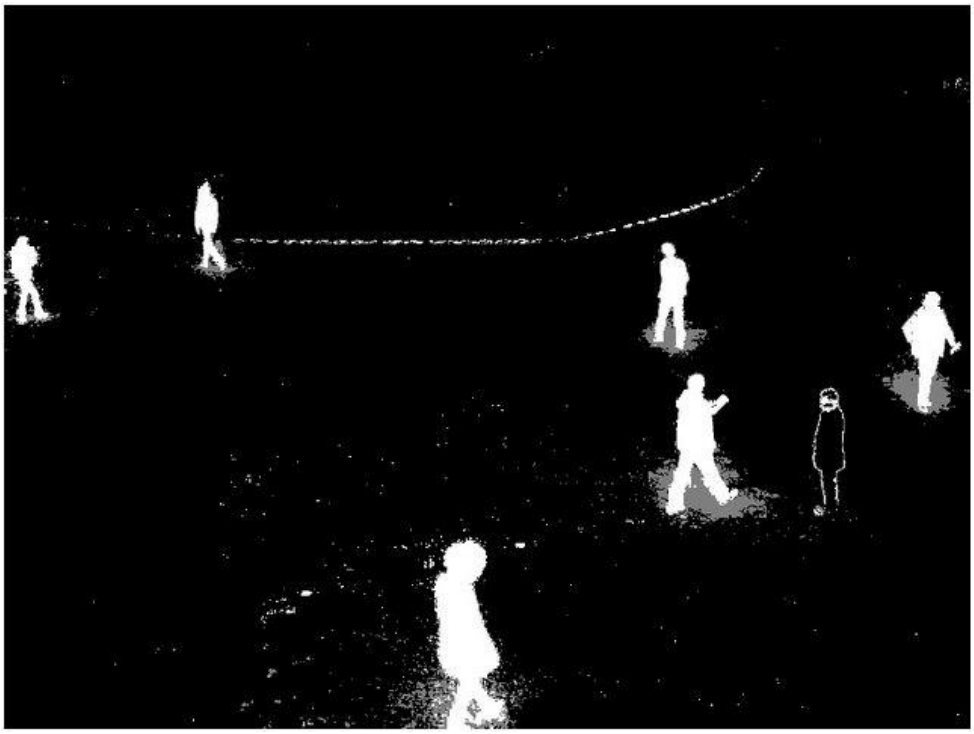


图 7.5.1 运动检测与追踪效果

Python 代码：

```
# 载入必要的库
```

```
import numpy as np
import cv2
```

```
# 载入视频文件
cap = cv2.VideoCapture('./images/walking.avi')
```

```
from jetcam.utils import bgr8_to_jpeg
import traitlets
import ipywidgets.widgets as widgets
from IPython.display import display

foreground_mask_img = widgets.Image(format='jpeg', width=640, height=480)
#dispaly_img = widgets.HBox([foreground_mask_img,testQRCode_img])
display(foreground_mask_img)
```

```
# Initlaize 背景减法器
foreground_background = cv2.createBackgroundSubtractorMOG2()
```

```
while True:


    ret, frame = cap.read()
    # 应用背景减法器获得前景蒙版
    foreground_mask = foreground_background.apply(frame)
    foreground_mask_img.value = bgr8_to_jpeg(foreground_mask)
cap.release()
```

7.6、OpenCV 二维码识别

7.6.1、OpenCV 二维码识别

程序位于‘whitebear’文件夹里的‘AI 视觉综合项目’文件夹，找到‘8.基于 OpenCV4 二维码识别.ipynb’程序，双击用 JupyterLab 打开程序。

二维码介绍

二维码又称二维条码，常见的二维码为 QR Code,QR 全称 Quick Response,是一个近几年来移动设备上超流行的一种编码方式，它比传统的 Bar Code 条形码能存更多的信息，也能表示更多的数据类型。

二维条码 / 二维码（2-dimensional bar code)是用某种特定的几何图形按一定规律在平面（二维方向上）分布的、黑白相间的、记录数据符号信息的图形；在代码编制上巧妙地利用构成计算机内部逻辑基础的“0”、“1”比特流的概念，使用若干个与二进制相对应的几何形体来表示文字数值信息，通过图象输入设备或光电扫描设备自动识读以实现信息自动处理：它具有条码技术的一些共性：每种码制有其特定的字符集；每个字符占有一定的宽度；具有一定的校验功能等。同时还具有对不同行的信息自动识别功能、及处理图形旋转变化点。

OpenCV4 二维码识别库

OpenCV 4.0 发布了许多改进和新功能。其中之一是二维码识别。在网上我们看到的最多的是基于 ZBar 库实现的二维码识别，我们也了解过了 ZBar 编写的程序和文章，因此，我们想检查一下 OpenCV 二维码识别的工作原理以及它是否比 Zbar 二维码更好。本章节中，我们将了解如何使用 OpenCV 自带的二维码识别函数库。

了解条形码 / 二维码码的结构

OpenCV4 中负责二维码检测与解析的类是 QRCodeDetector,它有如下几个方法来实现二维码的检测与解析返回。

1. 负责从图像中找到二维码区域，返回的是二维码四个顶点的坐标。

QRCodeDetector (InputArray img,OutputArray points)const img 参数是输入图像，支持灰度或者彩色 points 是 vector 返回的四个点坐标数组。

2. 负责解析二维码，返回 utf-8 字符串作为解析结果，无法解析返回空。

QRCodeDetector (InputArray img,InputArray points,OutputArray straight_qrcode=noArray)img 表示输入图像 point 表示检测到四个点坐标 straight_grcode 表示解析的二维码 ROl。

3.一步搞定二维码检测与解析。

detectAndDecode(InputArray img,//输入图像 OutputArray points=noArray,//顶点坐标 OutputArray straight_qrcode=noArray//ROI)。

基于 OpenCV 实现二维码识别

程序我们可以通过图片或者摄像头的方式识别二维码，我们我可以运行程序，看一下效果。

图 7.6.1 图片加载和摄像头识别二维码

```
import cv2
image = cv2.imread('./images/testdice.png')
image_qrCodeDetector = cv2.QRCodeDetector()
decodedText, points, _ = image_qrCodeDetector.detectAndDecode(image)
if points is not None:
    nrOfPoints = len(points)
    for i in range(nrOfPoints):
        nextPointIndex = (i+1) % nrOfPoints
        cv2.line(image, tuple(points[i][0]), tuple(points[nextPointIndex][0]), (255,0,0), 5)
    print(decodedText)
    testQRCode_img.value = bgr8_to_jpeg(image)
else:
    print("QR code not detected")
```

启东汤森

图 7.6.2 二维码识别结果

Python 代码：

```
# 载入必要的库
import cv2
import numpy as np
import sys
import time
```

```
# 线程函数操作库
import threading # 线程
import ctypes
import inspect

# 线程结束代码
def _async_raise(tid, exctype):
    tid = ctypes.c_long(tid)
    if not inspect.isclass(exctype):
        exctype = type(exctype)
    res = ctypes.pythonapi.PyThreadState_SetAsyncExc(tid, ctypes.py_object(exctype))
    if res == 0:
        raise ValueError("invalid thread id")
    elif res != 1:
        ctypes.pythonapi.PyThreadState_SetAsyncExc(tid, None)
        raise SystemError("PyThreadState_SetAsyncExc failed")

def stop_thread(thread):
    _async_raise(thread.ident, SystemExit)
```

```
# 创建显示控件


import enum
import cv2


def bgr8_to_jpeg(value, quality=75):
    return bytes(cv2.imencode('.jpg', value)[1])
from jetcam.utils import bgr8_to_jpeg
import threading
import traitlets
import ipywidgets.widgets as widgets
from IPython.display import display


QR_img = widgets.Image(format='jpeg', width=320, height=240)
testQRCode_img = widgets.Image(format='jpeg', width=320, height=240)


dispaly_img = widgets.HBox([testQRCode_img,QR_img])
display(dispaly_img)
```

```
dispW=320
dispH=240
flip=4
#camSet='nvarguscamerasrc ! video/x-raw(memory:NVMM), width=3264, height=2464, format=NV12, framerate=21/1 ! nvvidconv flip-method='+str(flip)+' ! video/x-raw, width='+str(dispW)+', height='+str(dispH)+', format=BGRx ! videoconvert ! video/x-raw, format=BGR ! appsink'
cam= cv2.VideoCapture(0)


width=cam.get(cv2.CAP_PROP_FRAME_WIDTH)
height=cam.get(cv2.CAP_PROP_FRAME_HEIGHT)
print('width:',width,'height:',height)
```

载入图片二维码

```
import cv2
image = cv2.imread('./images/testdice.png')
image_qrCodeDetector = cv2.QRCodeDetector()
decodedText, points, _ = image_qrCodeDetector.detectAndDecode(image)
if points is not None:
    nrOfPoints = len(points)
    for i in range(nrOfPoints):
```

```
            nextPointIndex = (i+1) % nrOfPoints
            cv2.line(image, tuple(points[i][0]), tuple(points[nextPointIndex][0]), (255,0,0), 5)
        print(decodedText)
        testQRCode_img.value = bgr8_to_jpeg(image)
else:
        print("QR code not detected")
```

摄像头二维码实时识别

```
def Video_display():
        qrCodeDetector = cv2.QRCodeDetector()
        while True:
            ret, frame = cam.read()
            gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
            t = time.time()
            data, points, _ = qrCodeDetector.detectAndDecode(gray)
            print("Time Taken for Detect and Decode : {:.3f} seconds".format(time.time() - t))
            if points is not None:
                nrOfPoints = len(points)
                for i in range(nrOfPoints):
                    nextPointIndex = (i+1) % nrOfPoints
                    cv2.line(frame, tuple(points[i][0]), tuple(points[nextPointIndex][0]), (255,0,0), 5)
            QR_img.value = bgr8_to_jpeg(frame)
        cap.release()
```

```
# 开始线程
t = threading.Thread(target=Video_display)
t.setDaemon(True)
t.start()
```

```
# 结束线程
stop_thread(t)
```

7.7、基于 Dlib 玩转人脸身份识别

7.7.1、OpenCV 二维码识别

程序位于‘whitebear’文件夹里的‘AI 视觉综合项目’文件夹，找到‘9.AI 人脸面部身份识别.ipynb’程序，双击用 JupyterLab 打开程序。

安装脸部识别库

1. 准备安装依赖库文件

```
sudo apt-get update
sudo apt-get install cmake libopenblas-dev liblapack-dev libjpeg-dev
```

2. 由于 nano 的内存只有 4G,在编译 dlib 的过程中，需要一个交换文件

```
git clone https://github.com/JetsonHacksNano/installSwapfile
cd installSwapfile
./installSwapfile.sh
```

3. 下载 dlib 安装包，解压之后，需要对源代码中一行代码进行注释

```
wget http://dlib.net/files/dlib-19.17.tar.bz2
tar jxvf dlib-19.17.tar.bz2
cd dlib-19.17
```

4. 修改在 dlib/cuda/目录下的 cudnn_dlibapi.cpp 文件中的内容，使得适用于 Jetson Nano

找到如下目录：

／dlib-19.17/dlib/cuda/cudnn_dlibapi.cpp

用 JupyterLab 打开，修改如下代码

找到对应的一行代码，进行删除（可以采用注释的方式删除）

```
forward_algo = forward_best_algo;
／／forward_algo=forward_best_algo;
```

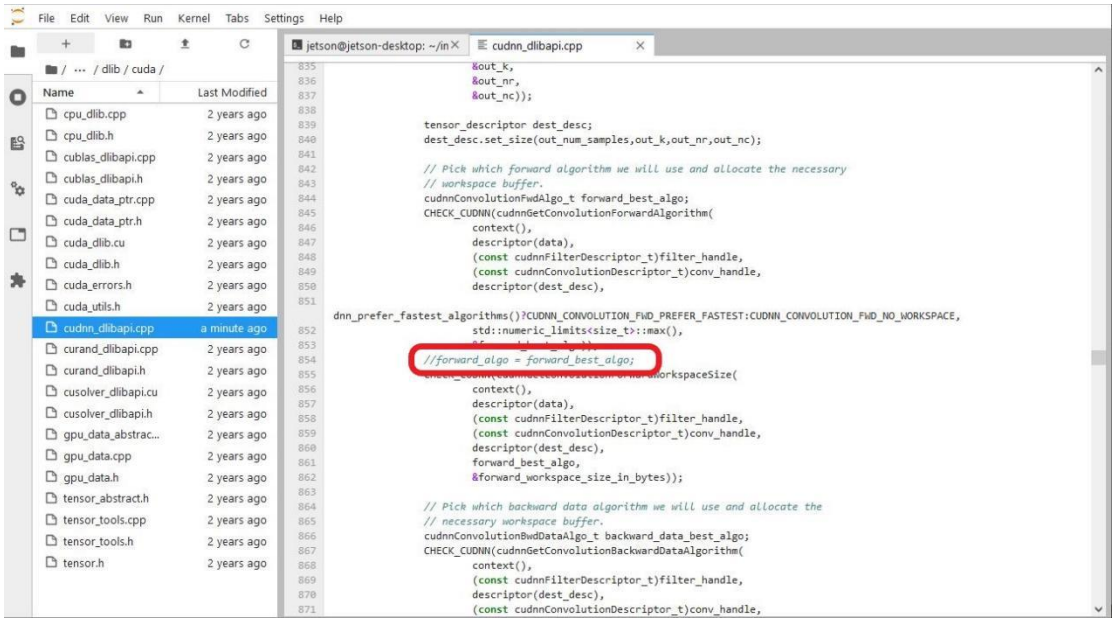


图 7.7.1 代码注释

5. 安装 dlib-19.17 依赖库

```
cd../
cd dlib-19.17
sudo python3 setup.py install
```

执行编译，编译的时间有点长，请耐心等待。

6. 安装 face_recognition 的库

```
sudo pip3 install face_recognition
```

人脸识别库识别人脸 Python 代码：

```
import face_recognition
import cv2
print(cv2.__version__)
```

```
# 线程函数操作库
import threading # 线程
import ctypes
import inspect

# 线程结束代码
def _async_raise(tid, exctype):
    tid = ctypes.c_long(tid)
```

```
    if not inspect.isclass(exctype):
        exctype = type(exctype)
    res = ctypes.pythonapi.PyThreadState_SetAsyncExc(tid, ctypes.py_object(exctype))
    if res == 0:
        raise ValueError("invalid thread id")
    elif res != 1:
        ctypes.pythonapi.PyThreadState_SetAsyncExc(tid, None)
        raise SystemError("PyThreadState_SetAsyncExc failed")


def stop_thread(thread):
    _async_raise(thread.ident, SystemExit)
```

```
# 创建显示控件
from jetcam.utils import bgr8_to_jpeg
import traitlets
import ipywidgets.widgets as widgets
from IPython.display import display
face_imge = widgets.Image(format='jpeg', width=640, height=480)
display(face_imge)
```

```
image=face_recognition.load_image_file('./unknown/u13.jpg')
face_locations=face_recognition.face_locations(image)
print(face_locations)
image=cv2.cvtColor(image,cv2.COLOR_RGB2BGR)
for(row1,col1,row2,col2) in face_locations:
    cv2.rectangle(image,(col1,row1),(col2,row2),(0,0,255),2)
face_imge.value = bgr8_to_jpeg(image)
```

运行程序结果：

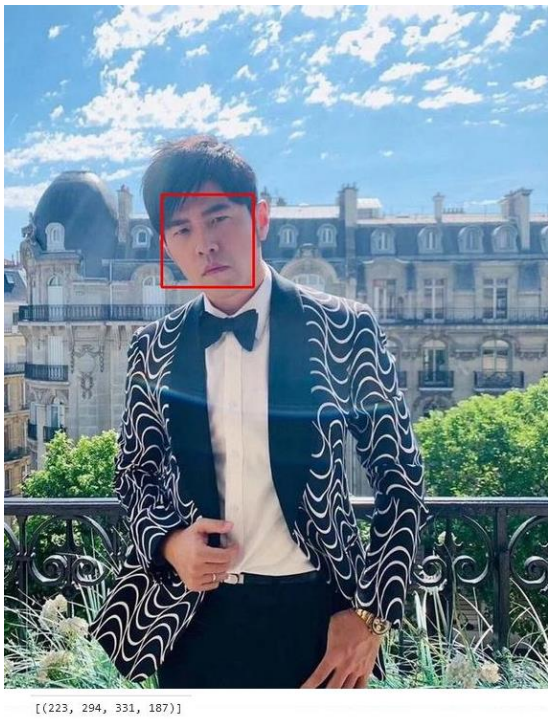


图 7.7.1 识别人脸

人脸身份识别 Python 代码：

```
import face_recognition
import cv2
print(cv2.__version__)
```

```
# 创建显示控件
from jetcam.utils import bgr8_to_jpeg
import traitlets
import ipywidgets.widgets as widgets
from IPython.display import display
face_imge1 = widgets.Image(format='jpeg', width=640, height=480)
display(face_imge1)
```

```
donFace=face_recognition.load_image_file('./known/Donald Trump.jpg')
donEncode = face_recognition.face_encodings(donFace)[0]

nancyFace=face_recognition.load_image_file('./known/Nancy Pelosi.jpg')
nancyEncode = face_recognition.face_encodings(nancyFace)[0]

penceFace=face_recognition.load_image_file('./known/Mike Pence.jpg')
penceEncode = face_recognition.face_encodings(penceFace)[0]
```

```
zhulinFace=face_recognition.load_image_file('./known/zhulin.jpg')
zhulinEncode = face_recognition.face_encodings(zhulinFace)[0]

Encodings=[donEncode,nancyEncode,penceEncode,zhulinEncode]
Names=['The Donald','Nancy Pelosi','Mike Pence','zhulin']

font = cv2.FONT_HERSHEY_SIMPLEX
testImage=face_recognition.load_image_file('./unknown/u13.jpg')
facePositions=face_recognition.face_locations(testImage)
allEncodings=face_recognition.face_encodings(testImage,facePositions)

testImage=cv2.cvtColor(testImage,cv2.COLOR_RGB2BGR)
```

```
for (top,right,bottom,left),face_encoding in zip(facePositions,allEncodings):
    name='Unknown Person'
    matches=face_recognition.compare_faces(Encodings,face_encoding)
    if True in matches:
        first_match_index=matches.index(True)
        name=Names[first_match_index]
    cv2.rectangle(testImage,(left,top),(right,bottom),(0,0,255),2)
    cv2.putText(testImage,name,(left,top-6),font,.75,(0,255,255),2)
face_imge1.value = bgr8_to_jpeg(testImage)
```

运行程序结果：

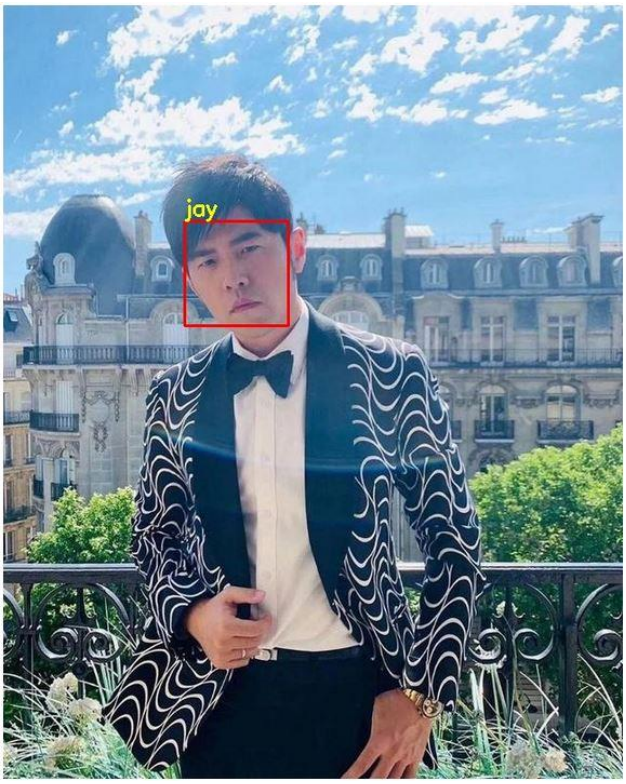


图 7.7.2 人脸身份识别

人脸身份识别 2Python 代码：

```
import face_recognition
import cv2
import os
print(cv2.__version__)
```

```
Encodings=[]
Names=[]
j=0
```

```
# 创建显示控件
from jetcam.utils import bgr8_to_jpeg
import traitlets
import ipywidgets.widgets as widgets
from IPython.display import display
face_imge2 = widgets.Image(format='jpeg', width=640, height=480)
display(face_imge2)
```

```
image_dir='./known'
for root,dirs,files in os.walk(image_dir):
    print(files)
```

```
    for file in files:
        path = os.path.join(root,file)
        print(path)
        name=os.path.splitext(file)[0]
        print(name)
        person=face_recognition.load_image_file(path)
        encoding=face_recognition.face_encodings(person)[0]
        Encodings.append(encoding)
        Names.append(name)
print(Names)

font=cv2.FONT_HERSHEY_SIMPLEX
image_dir='./unknown'
for root,dirs,files in os.walk(image_dir):
    for file in files:
        print(root)
        print(file)
        testImagePath=os.path.join(root,file)
        testImage=face_recognition.load_image_file(testImagePath)
        facePositions=face_recognition.face_locations(testImage)
        allEncodings=face_recognition.face_encodings(testImage,facePositions)
        testImage=cv2.cvtColor(testImage,cv2.COLOR_RGB2BGR)
        for (top,right,bottom,left),face_encoding in zip(facePositions,allEncodings):
            name='Unknown Person'
            matches=face_recognition.compare_faces(Encodings,face_encoding)
            if True in matches:
                first_match_index=matches.index(True)
                name=Names[first_match_index]
            cv2.rectangle(testImage,(left,top),(right,bottom),(0,0,255),2)
            cv2.putText(testImage,name,(left,top-6),font,.75,(0,255,255),2)
        face_imge2.value = bgr8_to_jpeg(testImage)
```

运行程序结果：

依次识别人物头像

图 7.7.3 人脸身份识别 2

图 7.7.4 人脸身份识别 3

在 OpenCV 中训练面部模型（生成训练模型）Python 代码：

```
import face_recognition
import cv2
import os
import pickle
print(cv2.__version__)
```

```
Encodings=[]
Names=[]
j=0
```

```
# 创建显示控件
from jetcam.utils import bgr8_to_jpeg
import traitlets
import ipywidgets.widgets as widgets
from IPython.display import display
face_imge3 = widgets.Image(format='jpeg', width=640, height=480)
display(face_imge3)
```

```
image_dir='./known'
for root,dirs,files in os.walk(image_dir):
    print(files)
    for file in files:
        path = os.path.join(root,file)
        print(path)
        name=os.path.splitext(file)[0]
        print(name)
        person=face_recognition.load_image_file(path)
        encoding=face_recognition.face_encodings(person)[0]
        Encodings.append(encoding)
        Names.append(name)
print(Names)

with open('train.pkl','wb') as f:
    pickle.dump(Names,f)
    pickle.dump(Encodings,f)
Encodings=[]
Names=[]
with open('train.pkl','rb') as f:
    Names=pickle.load(f)
    Encodings=pickle.load(f)

font=cv2.FONT_HERSHEY_SIMPLEX
image_dir='./unknown'
```

```
for root,dirs,files in os.walk(image_dir):
```

```
    for file in files:
        print(root)
        print(file)
        testImagePath=os.path.join(root,file)
        testImage=face_recognition.load_image_file(testImagePath)
        facePositions=face_recognition.face_locations(testImage)
        allEncodings=face_recognition.face_encodings(testImage,facePositions)
        testImage=cv2.cvtColor(testImage,cv2.COLOR_RGB2BGR)
        for (top,right,bottom,left),face_encoding in zip(facePositions,allEncodings):
            name='Unknown Person'
            matches=face_recognition.compare_faces(Encodings,face_encoding)
            if True in matches:
                first_match_index=matches.index(True)
                name=Names[first_match_index]
            cv2.rectangle(testImage,(left,top),(right,bottom),(0,0,255),2)
            cv2.putText(testImage,name,(left,top-6),font,.75,(0,255,255),2)
        face_imge3.value = bgr8_to_jpeg(testImage)
```

运行程序结果：

生成模型

图 7.7.5 生成的模型文件

在 OpenCV 中直接调用面部识别训练模型（直接导入模型进行识别）Python 代码：

```
import face_recognition
import cv2
import os
import pickle
print(cv2.__version__)
```

```
j=0
Encodings=[]
Names=[]
```

```
# 创建显示控件
from jetcam.utils import bgr8_to_jpeg
import traitlets
import ipywidgets.widgets as widgets
from IPython.display import display
face_imge4 = widgets.Image(format='jpeg', width=640, height=480)
```

```
display(face_imge4)
```

```
with open('train.pkl','rb') as f:
    Names=pickle.load(f)
    Encodings=pickle.load(f)

font=cv2.FONT_HERSHEY_SIMPLEX
image_dir='./unknown'
```

```
for root,dirs,files in os.walk(image_dir):
    for file in files:
        print(root)
        print(file)
        testImagePath=os.path.join(root,file)
        testImage=face_recognition.load_image_file(testImagePath)
        facePositions=face_recognition.face_locations(testImage)
        allEncodings=face_recognition.face_encodings(testImage,facePositions)
        testImage=cv2.cvtColor(testImage,cv2.COLOR_RGB2BGR)
        for (top,right,bottom,left),face_encoding in zip(facePositions,allEncodings):
            name='Unknown Person'
            matches=face_recognition.compare_faces(Encodings,face_encoding)
            if True in matches:
                first_match_index=matches.index(True)
                name=Names[first_match_index]
            cv2.rectangle(testImage,(left,top),(right,bottom),(0,0,255),2)
            cv2.putText(testImage,name,(left,top-6),font,.75,(0,255,255),2)
        face_imge4.value = bgr8_to_jpeg(testImage)
```

通过摄像头进行面部身份识别 Python 代码：

```
import face_recognition
import cv2
import os
import pickle
import time
print(cv2.__version__)
fpsReport=0
scaleFactor=.25
```

```
# 创建显示控件
from jetcam.utils import bgr8_to_jpeg
```

```
import traitlets
import ipywidgets.widgets as widgets
from IPython.display import display
face_imge5 = widgets.Image(format='jpeg')#, width=320, height=240)
display(face_imge5)
```

```
Encodings=[]
Names=[]

with open('train.pkl','rb') as f:
    Names=pickle.load(f)
    Encodings=pickle.load(f)

font=cv2.FONT_HERSHEY_SIMPLEX
dispW=640
dispH=480
flip=4
cam= cv2.VideoCapture(0)
timeStamp=time.time()
```

```
while True:
    _,frame = cam.read()
    frameSmall=cv2.resize(frame,(0,0),fx=scaleFactor,fy=scaleFactor)
    frameRGB=cv2.cvtColor(frameSmall,cv2.COLOR_BGR2RGB)
    facePositions=face_recognition.face_locations(frameRGB,model='cnn')
    allEncodings=face_recognition.face_encodings(frameRGB,facePositions)
    for (top,right,bottom,left),face_encoding in zip(facePositions,allEncodings):
        name='Unkown Person'
        matches=face_recognition.compare_faces(Encodings,face_encoding)
        if True in matches:
            first_match_index=matches.index(True)
            name=Names[first_match_index]

        top=int(top/scaleFactor)
        right=int(right/scaleFactor)
        bottom=int(bottom/scaleFactor)
        left=int(left/scaleFactor)

        cv2.rectangle(frame,(left,top),(right,bottom),(0,0,255),2)
```

```
        cv2.putText(frame,name,(left,top-6),font,.75,(0,0,255),2)


dt = time.time()-timeStamp
fps=1/dt
fpsReport=.90*fpsReport + .1*fps
#print('fps is:',round(fpsReport))
timeStamp=time.time()
cv2.rectangle(frame,(0,0),(100,40),(0,0,255),-1)
cv2.putText(frame,str(round(fpsReport,1)) + 'fps',(0,25),font,.75,(0,255,255,2))
face_imge5.value = bgr8_to_jpeg(frame)
```

7.8、安装 NVDIA 对象检测和推理工具

7.8.1、OpenCV 二维码识别

程序位于‘whitebear’文件夹里的‘AI 视觉综合项目’文件夹，找到‘9.AI 人脸面部身份识别.ipynb’程序，双击用 JupyterLab 打开程序。

安装 NVIDIA 对象检测和推理工具库

1. 准备安装依赖库文件

```
sudo apt-get update
sudo apt-get install git cmake libpython3-dev python3-numpy
```

2. 编译 jetson-inference 库

```
git clone --recursive https://github.com/dusty-nv/jetson-inference
cd jetson-inference
mkdir build
cd build
cmake../
```

弹出如下界面，我们在这里不需要进行安装，后面我们统一手动添加即可，因为下载很有可能下载不了，导致我们无法进行到下一步：

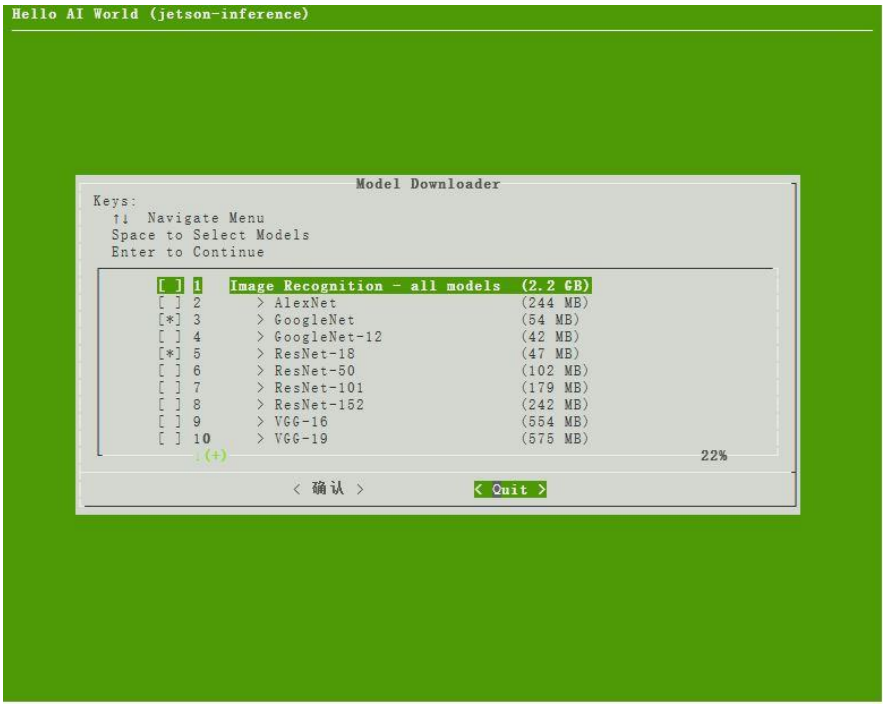


图 7.8.1 jetson-inference

选择＜Quit>退出！

接下来，弹出让我们安装 Pytorch，我们也可以选择<skip>退出，等待我们进行手动装即可！

Python 代码：

```
import cv2
import jetson.inference
import jetson.utils
import numpy as np
import time
```

```
width=1280
height=720

cam=jetson.utils.gstCamera(width,height,'0')
net=jetson.inference.imageNet('googlenet')
#net=jetson.inference.imageNet('alexnet')
font=cv2.FONT_HERSHEY_SIMPLEX
timeMark=time.time()
fpsFilter=0
```

```
from jetcam.utils import bgr8_to_jpeg
import traitlets
import ipywidgets.widgets as widgets
from IPython.display import display

frame_img = widgets.Image(format='jpeg', width=1280, height=720)
display(frame_img)
```

```
while True:
    frame,width,height=cam.CaptureRGBA(zeroCopy=1)
    classID,confidence=net.Classify(frame,width,height)
    item=net.GetClassDesc(classID)
    dt=time.time()-timeMark
    fps=1/dt
    fpsFilter=.95*fpsFilter + .05*fps
    timeMark=time.time()
    frame=jetson.utils.cudaToNumpy(frame,width,height,4)
```

```
frame=cv2.cvtColor(frame,cv2.COLOR_RGBA2BGR).astype(np.uint8)
cv2.putText(frame,str(round(fpsFilter,1))+' fps '+item,(0,30),font,1,(0,0,255),2)
frame_img.value = bgr8_to_jpeg(frame)
```

7.9、初识深度神经网络及卷积神经网络图像识别

7.9.1、人工智能、机器学习与深度学习

1956 年，几个计算机科学家相聚在达特茅斯会议，提出了“人工智能”的概念，梦想着用当时刚刚出现的计算机来构造复杂的、拥有与人类智慧同样本质特性的机器。其后，人工智能就一直萦绕于人们的脑海之中，并在科研实验室中慢慢孵化。之后的几十年，人工智能一直在两极反转，或被称作人类文明耀眼未来的预言，或被当成技术疯子的狂想扔到垃圾堆里。直到 2012 年之前，这两种声音还在同时存在。

2012 年以后，得益于数据量的上涨、运算力的提升和机器学习新算法（深度学习）的出现，人工智能开始大爆发。据领英近日发布的《全球 AI 领域人才报告》显示，截至 2017 年一季度，基于领英平台的全球 AI（人工智能）领域技术人才数量超过 190 万，仅国内人工智能人才缺口达到 500 多万。

人工智能的研究领域也在不断扩大，图二展示了人工智能研究的各个分支，包括专家系统、机器学习、进化计算、模糊逻辑、计算机视觉、自然语言处理、推荐系统等。

但目前的科研工作都集中在弱人工智能这部分，并很有希望在近期取得重大突破，电影里的人工智能多半都是在描绘强人工智能，而这部分在目前的现实世界里难以真正实现（通常将人工智能分为弱人工智能和强人工智能，前者让机器具备观察和感知的能力，可以做到一定程度的理解和推理，而强人工智能让机器获得自适应能力，解决一些之前没有遇到过的问题）。

弱人工智能有希望取得突破，是如何实现的，“智能”又从何而来呢？这主要归功于一种实现人工智能的方法——机器学习。

机器学习：一种实现人工智能的方法

机器学习最基本的做法，是使用算法来解析数据、从中学习，然后对真实世界中的事件做出决策和预测。与传统的为解决特定任务、硬编码的软件程序不同，机器学习是用大量的数据来“训练”，通过各种算法从数据中学习如何完成任务。

举个简单的例子，当我们浏览网上商城时，经常会出现商品推荐的信息。这是商城根据你往期的购物记录和冗长的收藏清单，识别出这其中哪些是你真正感兴趣，并且愿意购买的产品。这样的决策模型，可以帮助商城为客户提供建议并鼓励产品消费。

机器学习直接来源于早期的人工智能领域，传统的算法包括决策树、聚类、贝叶斯分类、支持向量机、EM、Adaboost 等等。从学习方法上来分，机器学习算法可以分为监督学习（如分类问题）、无监督学习（如聚类问题）、半监督学习、集成学习、深度学习和强化学习。

传统的机器学习算法在指纹识别、基于 Haar 的人脸检测、基于 HoG 特征的物体检测等领域的应用基本达到了商业化的要求或者特定场景的商业化水平，但每前进一步都异常艰难，直到深度学习算法的出现。

深度学习：一种实现机器学习的技术

深度学习本来并不是一种独立的学习方法，其本身也会用到有监督和无监督的学习方法来训练深度神经网络。但由于近几年该领域发展迅猛，一些特有的学习手段相继被提出（如残差网络），因此越来越多的人将其单独看作一种学习的方法。

最初的深度学习是利用深度神经网络来解决特征表达的一种学习过程。深度神经网络本身并不是一个全新的概念，可大致理解为包含多个隐含层的神经网络结构。为了提高深层神经网络的训练效果，人们对神经元的连接方法和激活函数等方面做出相应的调整。其实有不少想法早年间也曾有过，但由于当时训练数据量不足、计算能力落后，因此最终的效果不尽如人意。

深度学习摧枯拉朽般地实现了各种任务，使得似乎所有的机器辅助功能都变为可能。无人驾驶汽车，预防性医疗保健，甚至是更好的电影推荐，都近在眼前，或者即将实现。

三者的区别和联系

机器学习是一种实现人工智能的方法，深度学习是一种实现机器学习的技术。我们就用最简单的方法——同心圆，可视化地展现出它们三者的关系。

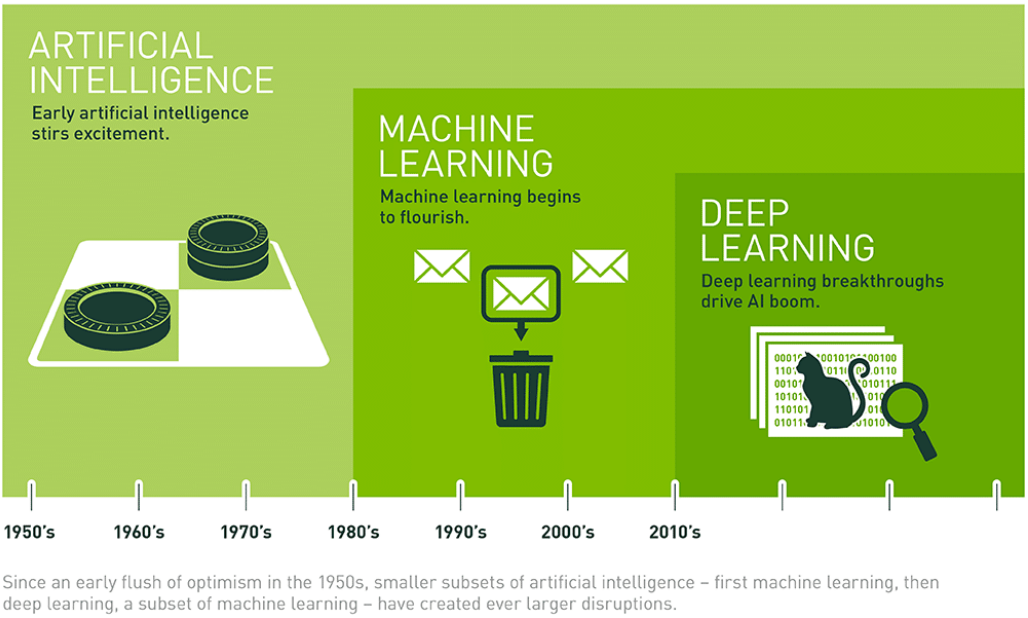


图 7.9.1 三者之间的关系

深度学习

相较而言，深度学习是一个比较新的概念，严格地说是 2006 年提出的。深度学习是用于建立、模拟人脑进行分析学习的神经网络，并模仿人脑的机制来解释数据的一种机器学习技术。它的基本特点，是试图模仿大脑的神经元之间传递，处理信息的模式。最显著的应用是计算机视觉和自然语言处理(NLP)领域。显然，“深度学习”是与机器学习中的“神经网络”是强相关，“神经网络”也是其主要的算法和手段；或者我们可以将“深度学习”称之为“改良版的神经网络”算法。深度学习又分为卷积神经网络（Convolutional neural networks，简称 CNN）和深度置信网（Deep Belief Nets，简称 DBN）。其主要的思想就是模拟人的神经元，每个神经元接受到信息，处理完后传递给与之相邻的所有神经元即可。所以看起来的处理方式有点像下图。

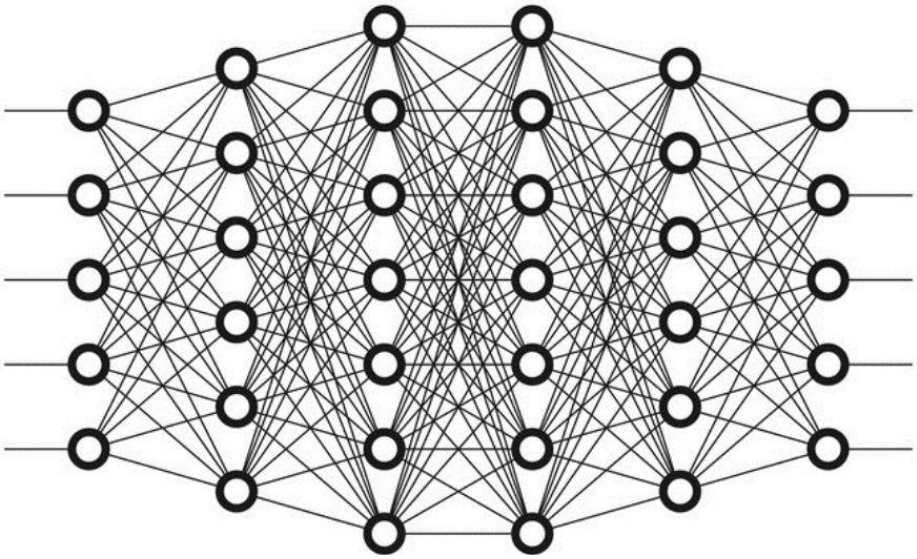


图 7.9.2 神经网络形象图

神经网络的计算量非常大，事实上在很长时间里由于基础设施技术的限制进展并不大。而 GPU 的出现让人看到了曙光，也造就了深度学习的蓬勃发展，“深度学习”才一下子火热起来。击败李世石的 Alpha go 即是深度学习的一个很好的示例。Google 的 TensorFlow 是开源深度学习系统一个比较好的实现，支持 CNN、RNN 和 LSTM 算法，是目前在图像识别、自然语言处理方面最流行的深度神经网络模型。事实上，提出“深度学习”概念的 Hinton 教授加入了 google，而 Alpha go 也是 google 家的。

7.9.2、卷积神经网络（CNN）

神经网络(neual networks)是人工智能研究领域的一部分，当前最流行的神经网络是深度卷积神经网络(deep convolutional neural networks, CNNs)，虽然卷积网络也存在浅层结构，但是因为准确度和表现力等原因很少使用。目前提到 CNNs 和卷积神经网络，学术界和工业界不再进行特意区分，一般都指深层结构的卷积神经网络，层数从”几层“到”几十上百“不定。

CNNs 目前在很多很多研究领域取得了巨大的成功，例如: 语音识别，图像识别，图像分割，自然语言处理等。虽然这些领域中解决的问题并不相同，但是这些应用方法都可以被归纳为:

CNNs 可以自动从(通常是大规模)数据中学习特征，并把结果向同类型未知数据泛化。

网络结构

基础的 CNN 由 卷积(convolution), 激活(activation), and 池化(pooling)三种结构组成。CNN 输出的结果是每幅图像的特定特征空间。当处理图像分类任务时，我们会把 CNN 输出的特征空间作为全连接层或全连接神经网络(fully connected neural network, FCN)的输入，用全连接层来完成从输入图像到标签集的映射，即分类。当然，整个过程最重要的工作就是如何通过训练数据迭代调整网络权重，也就是后向传播算法。目前主流的卷积神经网络(CNNs)，比如 VGG, ResNet 都是由简单的 CNN 调整，组合而来。

CNN

下图显示的是 CNN 的基础结构，现在大型深层的卷积神经网络(CNNs, 请注意这里是复数)通常由多个上述结构前后连接、层内调整组成，根据功能不同，我们称这些前后连接的结构处于不同阶段(stage)。虽然在主流 CNNs 中，不同 stage 里 CNN 会有不同的单元和结构，比如卷积核 (kernel)大小可能不同，激活函数(activition function) 可能不同，pooling 操作可能不存在，但是图 1 的 CNN 结构应当能够包含所有的情况。

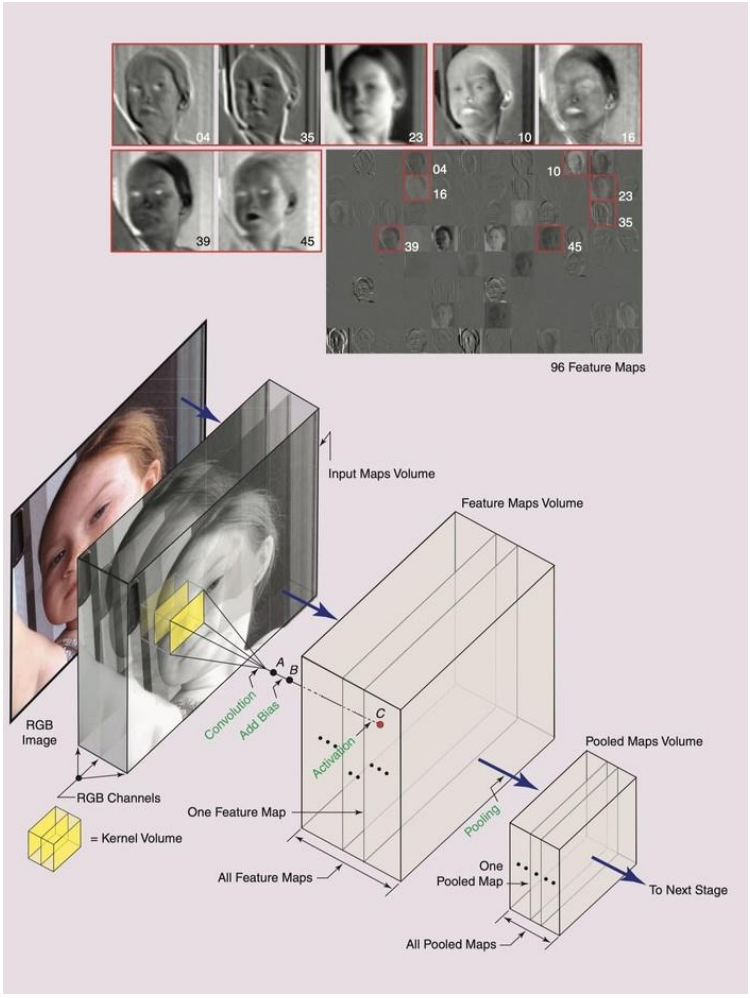


图 7.9.3 CNN 基础结构

我们跟随图 7.9.3 来解释，一个 stage 中的一个 CNN，通常会由三种映射空间组成(Maps Volume, 这里不确定是不是应该翻译为映射空间，或许映射体积会更准确),

- 输入映射空间(input maps volume）
- 特征映射空间(feature maps volume）
- 池化映射空间(pooled maps volume)

例如图 7.9.3 中，输入的是彩色 RGB 图像，那么输入的 maps volume 由红，黄，蓝三通道/三种 map 构成。我们之所以用 input map volume 这个词来形容，是因为对于多通道图像输入图像实际上是由高度，宽度，深度三种信息构成,可以被形象理解为一种"体积"。这里的“深度”，在 RGB 中就是 3，红，黄，蓝三种颜色构成的图像，在灰度图像中，就是 1。

卷积

CNN 中最基础的操作是卷积 convolution，再精确一点，基础 CNN 所用的卷积是一种 2-D 卷积。也就是说，kernel 只能在 x,y 上滑动位移，不能进行深度 (跨通道) 位移。这可以根据图 1 来理解，对于图中的 RGB 图像，采用了三个独立的 2-D kernel，如黄色部分所示，所以这个 kernel 的维度是$X \times Y \times 3$ 。

在基础 CNN 的不同 stage 中，kernel 的深度都应当一致，等于输入图像的通道数。

卷积需要输入两个参数，实质是二维空间滤波，滤波的性质与 kernel 选择有关，CNN 的卷积是在一个 2-D kernel 和输入的 2-D input map 之间，RGB 中各图像通道分别完成。

我们假设单一通道输入图像的空间坐标为(X, Y)

卷积核大小是$p \times q$

kernel 权重为w

图像亮度值是u

卷积过程就是 kernel 所有权重与其在输入图像上对应元素亮度之和，可以表示为：

$$conv_{x,y} = \sum_{i}^{p*q} w_i v_i$$

我们可以用一个例子来说明。

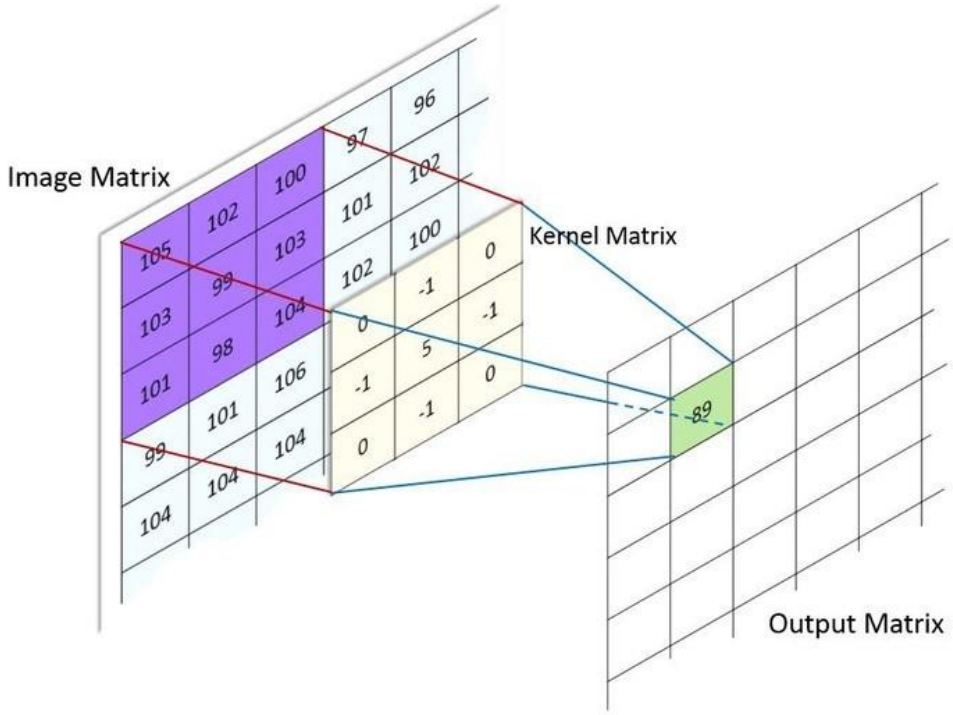


图 7.9.4 卷积示例

如上图所示，这时候输出的单一元素是

$$conv_{x,y} = 105 * 0 + 102 * (-1) + 100 * 0 + 103 * (-1) + 99 * 5 + 103 * (-1) + 101 * 0 + 98 * (-1) + 104 * 0 = 89$$

并将 kernel 随(x,y)平移扫描，可以得到输出空间，这时假设输入图像大小是512×512

卷积核是3×3

在不考虑零填充（zero padding）的情况，输出是$(512 - 3 + 1) = 510 \times 510$

注意卷积层的 kernel 可能不止一个，扫描步长，方向也有不同，这些进阶方式可以归纳一下:

- 可以采用多个卷积核，设为 n 同时扫描，得到的 feature map 会增加 n 个维度，通常认为是多抓取 n 个特征。

- 可以采取不同扫描步长，比如上例子中采用步长为 n, 输出是

- padding，上例里，卷积过后图像维度是缩减的，可以在图像周围填充 0 来保证 feature map 与原始图像大小不变

- 深度升降，例如采用增加一个 1*1 kernel 来增加深度，相当于复制一层当前通道作为 feature map
- 跨层传递 feature map,不再局限于输入即输出，例如 ResNet 跨层传递特征，Faster RCNN 的 POI pooling

经过训练后，CNN 会根据其分类要求自动进行调整以找到最相关的特征。例如，当遇到一般物体识别任务时，CNN 会过滤有关物体形状的信息，但在遇到鸟类识别任务时会提取鸟类的颜色。这是基于 CNN 通过训练认识到的，即不同类别的物体具有不同的形状，但是不同类型的鸟类在颜色上比形状上更有可能不同。使用 GPU 加速 CNN

训练 CNN 模型和通过训练后的 CNN 模型进行推理所需的大量计算可能数量很大，需要大量的计算资源和时间。深度学习框架（例如 Caffe,TensorFlow 和 PyTorch)经过优化，可在 GPU 上更快地运行。这些框架利用了 GPU 的并行处理能力（如果存在），从而加快了训练和推理任务的速度。

Jetson Nano 包含 128 核 NVIDIA Maxwell GPU,由于它可以运行完整的训练框架，因此还可以通过转移学习来重新训练网络，这是您将在本教程中使用的功能。Jetson Nano 使您可以在低成本平台上进行深度学习和 AI 的实验。

7.9.3、卷积神经网络经典模型 AlexNet 及 GoogleNet

AlexNet 模型

2012 年，Alex Krizhevsky、Ilya Sutskever 在多伦多大学 Geoff Hinton 的实验室设计出了一个深层的卷积神经网络 AlexNet,夺得了 2012 年 ImageNet LSVRC 的冠军，且准确率远超第二名（top5 错误率为 15.3%, 第二名为 26.2%),引起了很大的轰动。AlexNet 可以说是具有历史意义的一个网络结构，在此之前，深度学习已经沉寂了很长时间，自 2012 年 AlexNet 诞生之后，后面的 ImageNet 冠军都是用卷积神经网络（CNN)来做的，并且层次越来越深，使得 CNN 成为在图像识别分类的核心算法模型，带来了深度学习的大爆发。

AlexNet 模型的特点

AlexNet 之所以能够成功，跟这个模型设计的特点有关，主要有：

- 使用了非线性激活函数：ReLU
- 防止过拟合的方法：Dropout,数据扩充（Data augmentation)

- 其他：多 GPU 实现，LRN 归一化层的使用

GoogleNet 模型

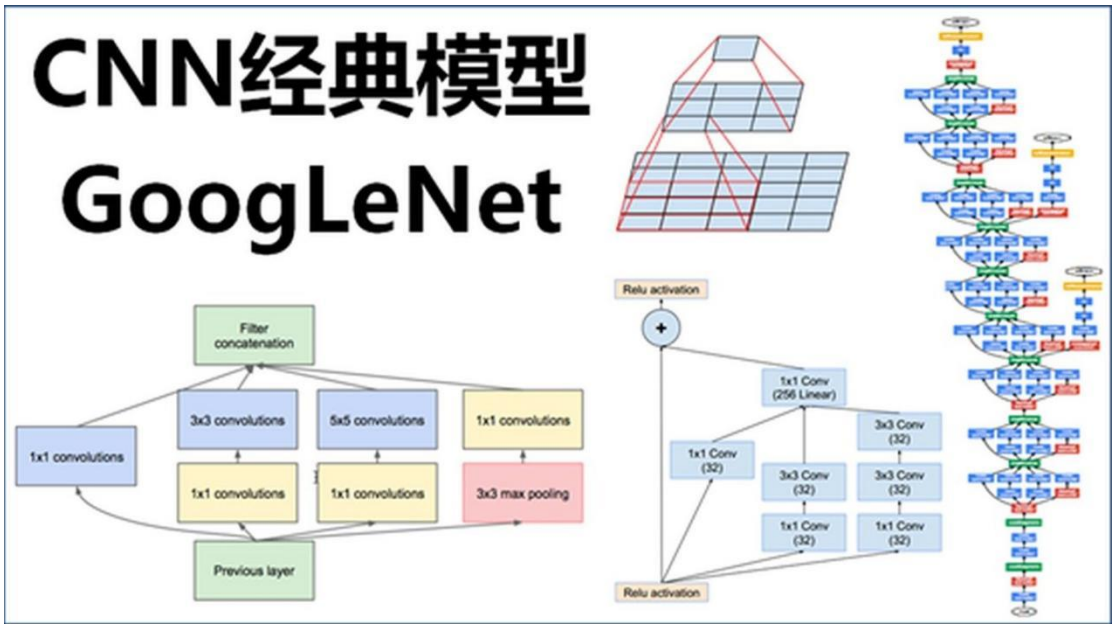


图 7.9.5 GoogleNet 模型

2014 年，GoogleNet 和 VGG 是当年 ImageNet 挑战赛(ILSVRC14)的双雄，GoogleNet 获得了第一名、VGG 获得了第二名，这两类模型结构的共同特点是层次更深了。VGG 继承了 GoogleNet 以及 AlexNet 的一些框架结构，而 GoogleNet 则做了更加大胆的网络结构尝试，虽然深度有 22 层，但大小却比 AlexNet 和 VGG 小很多，GoogleNet 参数为 500 万个，AlexNet 参数个数是 GoogleNet 的 12 倍，VGGNet 参数又是 AlexNet 的 3 倍，因此在内存或计算资源有限时，GoogleNet 是比较好的选择；从模型结果来看，GoogleNet 的性能却更加优越。

那么，GoogleNet 是如何进一步提升性能的呢？

一般来说，提升网络性能最直接的办法就是增加网络深度和宽度，深度指网络层次数量、宽度指神经元数量。但这种方式存在以下问题：

- 参数太多，如果训练数据集有限，很容易产生过拟合；

- 网络越大、参数越多，计算复杂度越大，难以应用；

- 网络越深，容易出现梯度弥散问题（梯度越往后穿越容易消失），难以优化模型。

所以，有人调侃“深度学习”其实是“深度调参”。

解决这些问题的方法当然就是在增加网络深度和宽度的同时减少参数，为了减少参数，自然就想到将全连接变成稀疏连接。但是在实现上，全连接变成稀疏连接后实际计算量并不会有质的提升，因为大部分硬件是针对密集矩阵计算优化的，稀疏矩阵虽然数据量少，但是计算所消耗的时间却很难减少。

那么，有没有一种方法既能保持网络结构的稀疏性，又能利用密集矩阵的高计算性能。大量的文献表明可以将稀疏矩阵聚类为较为密集的子矩阵来提高计算性能，就如人类的大脑是可以看做是神经元的重复堆积，因此，GoogLeNet 团队提出了 Inception 网络结构，就是构造一种“基础神经元”结构，来搭建一个稀疏性、高计算性能的网络结构。

Inception 历经了 V1、V2、V3、V4 等多个版本的发展，不断趋于完善，下面一一进行介绍

7.9.4、基于 GoogleNet 模型图像识别实践

在本案例中重点介绍的深度学习功能是图像识别，它使用已在大型数据集上进行训练的分类网络来识别场景和对象。

所述 imageNet 对象接受一个输入图像，并且输出每一类的概率。经过对 1000 个对象的 ImageNet ILSVRC 数据集的训练后，在构建步骤中自动下载了 GoogleNet 和 ResNet-18 模型。请参见下面的其他分类模型，这些模型也可以下载和使用。

Network	CLI argument	NetworkType enum
AlexNet	alexnet	ALEXNET
GoogleNet	googlenet	GOOGLENET
GoogleNet-12	googlenet-12	GOOGLENET_12
ResNet-18	resnet-18	RESNET_18
ResNet-50	resnet-50	RESNET_50
ResNet-101	resnet-101	RESNET_101
ResNet-152	resnet-152	RESNET_152
VGG-16	vgg-16	VGG-16
VGG-19	vgg-19	VGG-19
Inception-v4	inception-v4	INCEPTION_V4

首先，让我们尝试使用 jetson-inference 库在图像上测试 imageNet 识别。它加载摄像头图像，使用 TensorRT 和 imageNet 该类执行推理，然后覆盖分类结果并保存输出图像。

Python 代码：

```
import cv2
import jetson.inference
import jetson.utils
import numpy as np
import time
```

```
width=1280
height=720

cam=jetson.utils.gstCamera(width,height,'0')
net=jetson.inference.imageNet('googlenet')
#net=jetson.inference.imageNet('alexnet')
font=cv2.FONT_HERSHEY_SIMPLEX
timeMark=time.time()
fpsFilter=0
```

```
from jetcam.utils import bgr8_to_jpeg
import traitlets
import ipywidgets.widgets as widgets
from IPython.display import display

frame_img = widgets.Image(format='jpeg', width=1280, height=720)
display(frame_img)
```

```
while True:
    frame,width,height=cam.CaptureRGBA(zeroCopy=1)
    classID,confidence=net.Classify(frame,width,height)
    item=net.GetClassDesc(classID)
    dt=time.time()-timeMark
    fps=1/dt
    fpsFilter=.95*fpsFilter + .05*fps
    timeMark=time.time()
```

```
frame=jetson.utils.cudaToNumpy(frame,width,height,4)
frame=cv2.cvtColor(frame,cv2.COLOR_RGBA2BGR).astype(np.uint8)
cv2.putText(frame,str(round(fpsFilter,1))+' fps '+item,(0,30),font,1,(0,0,255),2)
frame_img.value = bgr8_to_jpeg(frame)
```

7.10、基于 Jetson Nano 识别和定位图像中感兴趣的对象（需更新）

7.10.1、基于 Jetson Nano 识别和定位图像中感兴趣的对象

在 Jetson-inference 库中使用使用 DetectNet 定位对象

在前面的章节中，我们也演示了物体对象检测的案例，但是我们只是作为一个测试使用，并没有具体说明，这里我们来说明一下！

在 Jetson-inference 库中使用使用 DetectNet 定位对象，是通过提取其边界框来查找各种对象在帧中的位置。与图像分类不同，对象检测网络能够每帧检测许多不同的对象。

图 7.9.1 图片识别

该 detectNet 对象接受图像作为输入， 并输出检测到的边界框的坐标列表及其类别和置信度值。detectNet 可从 Python 和 C++使用。我们可以下载各种预训练的模型进行实验。使用的默认模型是在 MS COCO 数据集上训练的 91 级 SSD-Mobilenet-v2 模型， 该模型在使用 TensorRT 的 Jetson 上实现实时推理性能。

Network	CLI argument	NetworkType enum	Object classes
SSD-Mobilenet-v1	ssd-mobilenet-v1	SSD_MOBILENET_V1	91 (COCO classes)
SSD-Mobilenet-v2	ssd-mobilenet-v2	SSD_MOBILENET_V2	91 (COCO classes)
SSD-Inception-v2	ssd-inception-v2	SSD_INCEPTION_V2	91 (COCO classes)
DetectNet-COCO-Dog	coco-dog	COCO_DOG	dogs
DetectNet-COCO-Bottle	coco-bottle	COCO_BOTTLE	bottles
DetectNet-COCO-Chair	coco-chair	COCO_CHAIR	chairs
DetectNet-COCO-Airplane	coco-airplane	COCO_AIRPLANE	airplanes
ped-100	pednet	PEDNET	pedestrians
multiped-500	multiped	PEDNET_MULTI	pedestrians, luggage
facenet-120	facenet	FACENET	faces

图 1.10.2 物体识别预训练模型

Jetson Nano 物体识别中提取出感兴趣的对象实践

在这个实验中，我们主要是识别出猫，然后用绿色填充的矩形框起来，已和其他识别的对象作为区别，采用的方法是识别出物体的 ID 号，然后把对应的 ID 号所对应的名称做判断，如果是我们需要找的对象则做出区分显示。

Python 代码：

```
import cv2
import jetson.inference
import jetson.utils
import time
import numpy as np
```

```
timeStamp= time.time()
fpsFiltered=0
```

```
from jetcam.utils import bgr8_to_jpeg
import traitlets
import ipywidgets.widgets as widgets
from IPython.display import display

frame_img = widgets.Image(format='jpeg', width=1280, height=720)
display(frame_img)
```

```
net=jetson.inference.detectNet('ssd-mobilenet-v2',threshold=.5)
dispW=1280
```

```
dispH=720
flip=4
font=cv2.FONT_HERSHEY_SIMPLEX

# Gstreamer code for improvded Raspberry Pi Camera Quality
cam=cv2.VideoCapture(0)

while True:
    _,img = cam.read()
    height= img.shape[0]
    width=    img.shape[1]

    frame=cv2.cvtColor(img,cv2.COLOR_BGR2RGBA).astype(np.float32)
    frame=jetson.utils.cudaFromNumpy(frame)
    detections=net.Detect(frame,width,height)

    for detect in detections:
        #print(detect)
        ID=detect.ClassID
        top=int(detect.Top)
        left=int(detect.Left)
        bottom=int(detect.Bottom)
        right=int(detect.Right)
        item=net.GetClassDesc(ID)
        tk=1
        if item == 'cat': #  识别猫
            tk = -1
        cv2.rectangle(img,(left,top),(right,bottom),(0,255,0),tk)
        cv2.putText(img,item,(left,top+20),font,.75,(0,0,255),2)

        #print(item,top,left,bottom,right)

    #print(str(round(fps,1))+' fps ')
    dt=time.time()-timeStamp
    timeStamp=time.time()
    fps=1/dt
    fpsFiltered=.9*fpsFiltered + .1*fps
    cv2.putText(img,str(round(fpsFiltered,1)) + ' fps ',(0,30),font,1,(0,0,255),2)
    frame_img.value = bgr8_to_jpeg(img)
```

7.11、基于 Pytorch 的手势识别、表情识别及脸部特征识别（需更新）

在本课程中可以使用 AI 人工智能解决几个简单视觉问题，如手势识别，表情识别，判断手指、确定鼻子的位置等。尽管这些问题对于任何人类孩子来说都很容易回答，但是使用计算机视觉来解释图像需要复杂的计算机模型，可以对其进行调整以在许多情况下找到答案。例如，竖起大拇指的手势可能与摄像头的角度和距离不同，它可能在各种背景之前被保持，它可能来自各种不同的手，依此类推，但仍然是大拇指手势。有效的 AI 模型必须能够在所有情况下进行概括，甚至可以使用新数据预测正确答案。

7.11.1、ResNet-18 网络

应用程序开发人员可以使用许多世界一流的 CNN 架构进行图像分类和图像回归。PyTorch 和其他框架包括从著名的 Imagenet 大规模视觉识别挑战赛（ILSVRC)的过去获奖者那里访问预训练的模型，研究人员在其中竞争，用计算机视觉算法正确分类和检测物体和场景。2015 年，ResNet 赢得了图像分类，检测和定位方面的奖项。我们将在项目中使用最小版本的 ResNet:ResNet-18。

7.12、手势识别固定动作

7.12.1、手势识别说明

本次例程使用的手势识别是基于百度智能云平台的服务，该服务每天有 50000 次免费使用的机会，仅供学习，勿用商业用途，如果需要长期使用请购买相关服务，本公司概不负责。

百度手势应用创建方法

1. 申请 API 说明：官方 API 地址参考：https://cloud.baidu.com/doc/BODY/s/ajwvxyyn0

用浏览器打开官方 API 地址后，在右上方登录百度账号，如果没有百度账号请先注册。

2. 登录完成后，点击菜单栏上的‘产品’-‘人工智能’-‘人脸与人体识别’-‘人体分析’。

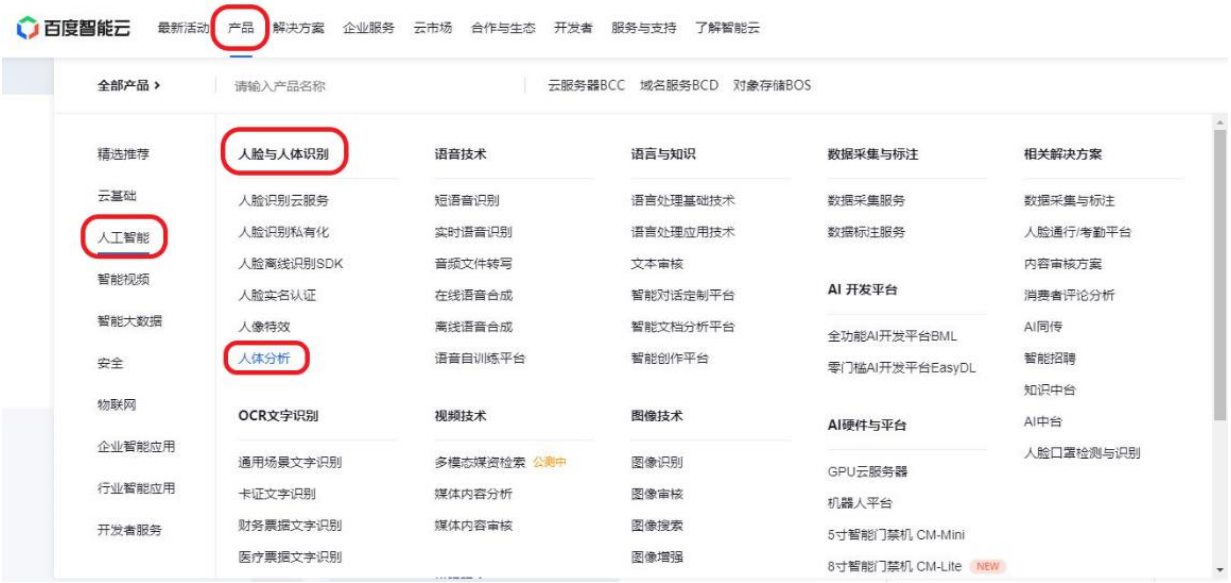


图 7.12.1 人体分析产品

3. 点击产品列表中的‘手势识别’。

图 7.12.2 手势识别

4. 点击立即使用

图 7.12.3 立即使用

5. 接下来会自动进入人体分析的后台管理，点击创建应用。

图 7.12.4 创建应用

6. 填写相关信息后点击立即创建，创建完成后点击应用列表就可以看到刚刚创建的应用的信息，此时把 AppID、API Key 和 Secret Key 下的信息都复制保存起来。

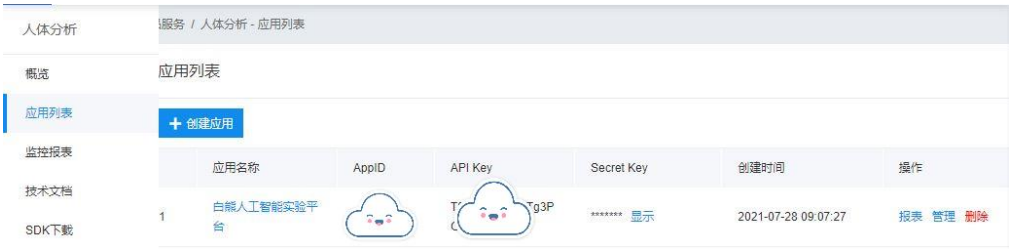


图 7.12.5 应用信息

API 函数

```
from aip import    AipBodyAnalysis

""" 你的 APPID AK SK       """
APP_ID = '你的 App ID'
API_KEY = '你的 Api Key'
SECRET_KEY = '你的 Secret Key'

client =    AipBodyAnalysis(APP_ID, API_KEY, SECRET_KEY)

""" 读取图片    """
def    get_file_content(filePath):
        with    open(filePath,    'rb') as fp:
            return fp.read()
```

```
image =    get_file_content('example.jpg')

      """ 调用手势识别    """
Res    = client.gesture(image);
```

手势识别返回数据参数详情

字段	是否必选	类型	说明
result_num	是	int	结果数量
result	是	object[]	检测到的目标，手势、人脸
+classname	否	string	目标所属类别，24 种手势、other、face
+top	否	int	目标框上坐标
+width	否	int	目标框的宽
+left	否	int	目标框最左坐标
+height	否	int	目标框的高
+probability	否	float	目标属于该类别的概率
log_id	是	int64	唯一的 log id，用于问题定位
image	是	string	图像数据，base64 编码后进行 urlencode，要求 base64 编码和 urlencode 后大小不超过 4M。图片的 base64 编码是不包含图片头的，如（），支持图片格式：jpg、bmp、png，最短边至少 50px，最长边最大 4096px

手势识别返回示例


```
{
  "log_id": 4466502370458351471,
  "result_num": 2,
  "result": [{
      "probability": 0.9844077229499817,
    "top":    20,
    "height":    156,
      "classname": "Face",
    "width":    116,
    "left":    173
  },
  {
      "probability": 0.4679304957389832,
    "top":    157,
    "height":    106,
```

```
        "classname": "Heart_2",
      "width":    177,
      "left":    183
    }]
}
```


手势识别支持的手势及示例图

序号	手势名称	classname	示例图
1	数字 1（原食指）	One	
2	数字 5（原掌心向前）	Five	
3	拳头	Fist	
4	OK	OK	
5	祈祷	Prayer	

6	作揖	Congratulation	
7	作别	Honour	
8	单手比心	Heart_single	
9	点赞	Thumb_up	
10	Diss	Thumb_down	
11	我爱你	ILY	
12	掌心向上	Palm_up	

13	双手比心 1	Heart_1	
14	双手比心 2	Heart_2	
15	双手比心 3	Heart_3	
16	数字 2	two	
17	数字 3	three	
18	数字 4	four	
19	数字 6	six	

20	数字 7	seven	
21	数字 8	eight	
22	数字 9	nine	
23	Rock	Rock	
24	竖中指	Insult	

7.12.2、手势识别固定动作

程序位于‘whitebear’文件夹里的‘AI 视觉综合项目’文件夹，找到‘13.手势识别固定动作.ipynb’程序，双击用 JupyterLab 打开程序。

如果识别到设定的手势动作，则机械臂会做对应的动作。目前手势及动作对应如下：

点赞	机械臂爪子类似于鼓掌的运动
OK	机械臂点点头
祈祷	机械臂祈祷
单手比心	机械臂做出跪下迎接的姿态
数字 5	机械臂返回初始位置
数字 8	机械臂看到枪，做出被吓到的动作

Rock	机械臂扑倒下
拇指向下	机械臂做出马姿态等待主人上车
作揖	主人骑上马儿开始奔跑
数字 7	悬崖勒马，停止奔跑

Python 代码：

```
#bgr8 转 jpeg 格式
import enum
import cv2

def bgr8_to_jpeg(value, quality=75):
    return bytes(cv2.imencode('.jpg', value)[1])
```

```
#导入相关的模块
import threading
import time
from Arm_Lib import Arm_Device



# 创建机械臂对象
Arm = Arm_Device()
time.sleep(.1)
```

定义手势识别函数部分

```
import cv2
import time
import demjson
import pygame
from aip import AipBodyAnalysis
from aip import AipSpeech
from PIL import Image, ImageDraw, ImageFont
import numpy
import ipywidgets.widgets as widgets

# 具体手势请看官方提供 https://ai.baidu.com/ai-doc/BODY/4k3cpywrv
hand={'One':'数字 1','Two':'数字 2','Three':'数字 3','Four':'数字 4',
      'Five':'数字 5', 'Six':'数字 6','Seven':'数字 7',
      'Eight':'数字 8','Nine':'数字 9','Fist':'拳头','Ok':'OK',
      'Prayer':'祈祷','Congratulation':'作揖','Honour':'作别',
      'Heart_single':'比心心','Thumb_up':'点赞','Thumb_down':'Diss',
```

```
        'ILY':'我爱你','Palm_up':'掌心向上','Heart_1':'双手比心 1',
        'Heart_2':'双手比心 2','Heart_3':'双手比心 3','Rock':'Rock',
        'Insult':'竖中指','Face':'脸'}

# 下面的 key 要换成自己的
""" 人体分析 APPID AK SK """
APP_ID = '24615131'
API_KEY = 'T0qrtLO9TgrQTg3PG9N9kLIG'
SECRET_KEY = 'hrfiizKjlAUW8axsz0z3PVxsUdAqnZqf'



client = AipBodyAnalysis(APP_ID, API_KEY, SECRET_KEY)



g_camera = cv2.VideoCapture(0)
g_camera.set(3, 640)
g_camera.set(4, 480)
g_camera.set(5, 30)  #设置帧率
g_camera.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter.fourcc('M', 'J', 'P', 'G'))
g_camera.set(cv2.CAP_PROP_BRIGHTNESS, 40) #设置亮度 -64 - 64  0.0
g_camera.set(cv2.CAP_PROP_CONTRAST, 50) #设置对比度 -64 - 64  2.0
g_camera.set(cv2.CAP_PROP_EXPOSURE, 156) #设置曝光值 1.0 - 5000  156.0

ret, frame = g_camera.read()
```

定义摄像头显示组件

```
image_widget = widgets.Image(format='jpeg', width=600, height=500)  #设置摄像头显示组件
display(image_widget)
image_widget.value = bgr8_to_jpeg(frame)
```

定义转换显示中文函数

```
def cv2ImgAddText(img, text, left, top, textColor=(0, 255, 0), textSize=20):
    if (isinstance(img, numpy.ndarray)):  # 判断是否 OpenCV 图片类型
        img = Image.fromarray(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    # 创建一个可以在给定图像上绘图的对象
    draw = ImageDraw.Draw(img)
    # 字体的格式
    fontStyle = ImageFont.truetype(
        "simhei.ttf", textSize, encoding="utf-8")
    # 绘制文本
    draw.text((left, top), text, textColor, font=fontStyle)
```

```
    # 转换回 OpenCV 格式
    return cv2.cvtColor(numpy.asarray(img), cv2.COLOR_RGB2BGR)
```

```
look_at = [90, 164, 18, 0, 90, 90]
p_Prayer = [90, 90, 0, 180, 90, 180] #祈祷
p_Thumb_up = [90, 90, 90, 90, 90, 180] #点赞
p_Heart_single = [90, 0, 180, 0, 90, 30] #单手比心

p_Eight = [90, 180, 18, 0, 90, 90] #八

p_Congratulation = [90, 131, 52, 0, 90, 180] #作揖
p_Rock = [90, 0, 90, 180, 90, 0] #rock
p_fist = [90, 90, 0, 0, 90, 0] #拳头
p_horse_1 = [90, 7, 153, 19, 0, 126] #
p_horse_2 = [90, 5, 176, 0, 0, 180]
p_horse_3 = [90, 62, 158, 0, 0, 0]

global running
running = 0
```

```
# 定义移动机械臂函数,同时控制 1-6 号舵机运动，p=[S1,S2,S3,S4,S5,S6]
def arm_move_6(p, s_time = 500):
    for i in range(6):
        id = i + 1
        Arm.Arm_serial_servo_write(id, p[i], s_time)
        time.sleep(.01)
    time.sleep(s_time/1000)

# 定义小马运动
def horse_running():
    Arm.Arm_serial_servo_write(6, 150, 300)
    time.sleep(.3)
    Arm.Arm_serial_servo_write(6, 180, 300)
    time.sleep(.3)
```

```
global g_state_arm
g_state_arm = 0
def ctrl_arm_move(index):
    global running
```

```
if index == "Prayer":
        arm_move_6(p_Prayer, 1000)
        time.sleep(1.5)
        arm_move_6(look_at, 1000)
        time.sleep(1)
elif index == "Thumb_up":
        s_time = 500
        Arm.Arm_serial_servo_write(6, 180, s_time)
        time.sleep(s_time/1000)
        Arm.Arm_serial_servo_write(6, 90, s_time)
        time.sleep(s_time/1000)
        Arm.Arm_serial_servo_write(6, 180, s_time)
        time.sleep(s_time/1000)
        Arm.Arm_serial_servo_write(6, 90, s_time)
        time.sleep(s_time/1000)
elif index == "Ok":
        s_time = 300
        Arm.Arm_serial_servo_write(4, 10, s_time)
        time.sleep(s_time/1000)
        Arm.Arm_serial_servo_write(4, 0, s_time)
        time.sleep(s_time/1000)
        Arm.Arm_serial_servo_write(4, 10, s_time)
        time.sleep(s_time/1000)
        Arm.Arm_serial_servo_write(4, 0, s_time)
        time.sleep(s_time/1000)
elif index == "Heart_single":
        arm_move_6([90, 90, 90, 90, 90, 90], 800)
        time.sleep(.1)
        arm_move_6(p_Heart_single, 1000)
        time.sleep(1)
elif index == "Five":
        arm_move_6(look_at, 1000)
        time.sleep(.5)
elif index == "Eight":
        s_time = 300
        arm_move_6(p_Eight, 0)
        time.sleep(1)
        Arm.Arm_serial_servo_write(2, 165, s_time)
        time.sleep(s_time/1000)
```

```
    elif index == "Rock": #rock
        Arm.Arm_serial_servo_write6_array(p_Rock, 1300)
        time.sleep(3)
        Arm.Arm_serial_servo_write6_array(look_at, 1000)
        time.sleep(1)
    elif index == "Thumb_down": #拇指向下
        Arm.Arm_serial_servo_write6_array(p_horse_1, 1300)
        time.sleep(1)
    elif index == "Congratulation": #作揖
        Arm.Arm_serial_servo_write6_array(p_horse_2, 1000)
        time.sleep(1)
        running = 1
        while running == 1:
            horse_running()
    elif index == "Seven": #数字 7
        Arm.Arm_Buzzer_On(8)     #蜂鸣器自动响 0.5 秒
        Arm.Arm_serial_servo_write6_array(p_horse_3, 1000)
        time.sleep(2)
        Arm.Arm_serial_servo_write6_array(look_at, 1000)
        time.sleep(1)



    global g_state_arm
    g_state_arm = 0
```

```
arm_move_6(look_at, 1000)
time.sleep(1)
```

```
def start_move_arm(index):
    # 开启机械臂控制线程
    global g_state_arm
    global running
    if g_state_arm == 0:
        closeTid = threading.Thread(target = ctrl_arm_move, args = [index])
        closeTid.setDaemon(True)
        closeTid.start()
        g_state_arm = 1


    if running == 1 and index == "Seven":
```

```
                running = 0
```

主进程：

```
try:
    Arm.Arm_Buzzer_On(1)
    s_time = 300
    Arm.Arm_serial_servo_write(4, 10, s_time)
    time.sleep(s_time/1000)
    Arm.Arm_serial_servo_write(4, 0, s_time)
    time.sleep(s_time/1000)
    Arm.Arm_serial_servo_write(4, 10, s_time)
    time.sleep(s_time/1000)
    Arm.Arm_serial_servo_write(4, 0, s_time)
    time.sleep(s_time/1000)

    while True:
        """1.拍照  """
        ret, frame = g_camera.read()

        #image = get_file_content('./image.jpg')

        """ 2.调用手势识别  """
        raw = str(client.gesture(image_widget.value))
        text = demjson.decode(raw)
        try:
            res = text['result'][0]['classname']
        except:
#             print('识别结果：什么也没识别到哦~' )
#             img = cv2ImgAddText(frame, "未识别", 250, 30, (0, 0 , 255), 30)
            img = frame
        else:
#             print('识别结果：' + hand[res])
#             img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 , 0), 30)
            if res == 'Prayer': # 1  祈祷
                print('识别结果：' + hand[res])
                img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 , 0), 30)
                start_move_arm(res)
            elif res == 'Thumb_up':# 2  点赞
                print('识别结果：' + hand[res])
                img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 , 0), 30)
```

```
            start_move_arm(res)
        elif res == 'Ok': # 3 OK
            print('识别结果：' + hand[res])
            img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 , 0), 30)
            start_move_arm(res)
        elif res == 'Heart_single': # 4  单手比心
            print('识别结果：' + hand[res])
            img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 , 0), 30)
            start_move_arm(res)
        elif res == 'Five': # 5  数字 5
            print('识别结果：' + hand[res])
            img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 , 0), 30)
            start_move_arm(res)
        elif res == "Eight": #  数字 8
            print('识别结果：' + hand[res])
            img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 , 0), 30)
            start_move_arm(res)

        elif res == "Rock": # rock
            print('识别结果：' + hand[res])
            img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 , 0), 30)
            start_move_arm(res)
        elif res == "Congratulation": #  作揖
            print('识别结果：' + hand[res])
            img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 , 0), 30)
            start_move_arm(res)
        elif res == "Seven": #  数字 7
            print('识别结果：' + hand[res])
            img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 , 0), 30)
            start_move_arm(res)
        elif res == "Thumb_down": #  拇指向下
            print('识别结果：' + hand[res])
            img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 , 0), 30)
            start_move_arm(res)

        else:
            img = frame

    image_widget.value = bgr8_to_jpeg(img)
```

```
except KeyboardInterrupt:
    print(" Program closed! ")
    pass
```

```
g_camera.release()                    #使用完成对象记住释放掉对象，不然下一个程序使用这个对象模块会被占用，导致无法使用
```

7.12.3、手势识别固定动作

程序位于‘whitebear’文件夹里的‘AI 视觉综合项目’文件夹，找到‘14.手势识别堆叠积木.ipynb’程序，双击用 JupyterLab 打开程序。

按照颜色位置依次摆放木块：

图 7.12.6 木块摆放位置

Python 代码：

```
#bgr8 转 jpeg 格式
import enum
```

```
import cv2

def bgr8_to_jpeg(value, quality=75):
    return bytes(cv2.imencode('.jpg', value)[1])
```

```
#导入相关的模块
import threading
from Arm_Lib import Arm_Device

Arm = Arm_Device()
```

定义手势识别函数

```
import cv2
import time
import demjson
import pygame
from aip import AipBodyAnalysis
from aip import AipSpeech
from PIL import Image, ImageDraw, ImageFont
import numpy
import ipywidgets.widgets as widgets

# 具体手势请看官方提供 https://ai.baidu.com/ai-doc/BODY/4k3cpywrv
hand={'One':'数字 1','Two':'数字 2','Three':'数字 3','Four':'数字 4',
      'Five':'数字 5', 'Six':'数字 6','Seven':'数字 7',
      'Eight':'数字 8','Nine':'数字 9','Fist':'拳头','Ok':'OK',
      'Prayer':'祈祷','Congratulation':'作揖','Honour':'作别',
      'Heart_single':'比心心','Thumb_up':'点赞','Thumb_down':'Diss',
      'ILY':'我爱你','Palm_up':'掌心向上','Heart_1':'双手比心 1',
      'Heart_2':'双手比心 2','Heart_3':'双手比心 3','Rock':'Rock',
      'Insult':'竖中指','Face':'脸'}

# 下面的 key 要换成自己的
""" 人体分析 APPID AK SK """
APP_ID = '24615131'
API_KEY = 'T0qrtLO9TgrQTg3PG9N9kLIG'
SECRET_KEY = 'hrfiizKjIAUW8axsz0z3PVxsUdAqnZqf'



client = AipBodyAnalysis(APP_ID, API_KEY, SECRET_KEY)
```

```
g_camera = cv2.VideoCapture(0)
g_camera.set(3, 640)
g_camera.set(4, 480)
g_camera.set(5, 30)  #设置帧率
g_camera.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter.fourcc('M', 'J', 'P', 'G'))
g_camera.set(cv2.CAP_PROP_BRIGHTNESS, 40) #设置亮度 -64 - 64  0.0
g_camera.set(cv2.CAP_PROP_CONTRAST, 50) #设置对比度 -64 - 64  2.0
g_camera.set(cv2.CAP_PROP_EXPOSURE, 156) #设置曝光值 1.0 - 5000  156.0

ret, frame = g_camera.read()
```

定义摄像头显示组件

```
image_widget = widgets.Image(format='jpeg', width=640, height=480)  #设置摄像头显示组件
display(image_widget)
image_widget.value = bgr8_to_jpeg(frame)
```

定义转换显示中文函数

```
def cv2ImgAddText(img, text, left, top, textColor=(0, 255, 0), textSize=20):
    if (isinstance(img, numpy.ndarray)):  # 判断是否 OpenCV 图片类型
        img = Image.fromarray(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    # 创建一个可以在给定图像上绘图的对象
    draw = ImageDraw.Draw(img)
    # 字体的格式
    fontStyle = ImageFont.truetype(
        "simhei.ttf", textSize, encoding="utf-8")
    # 绘制文本
    draw.text((left, top), text, textColor, font=fontStyle)
    # 转换回 OpenCV 格式
    return cv2.cvtColor(numpy.asarray(img), cv2.COLOR_RGB2BGR)
```

```
# 定义移动机械臂函数,同时控制 1-6 号舵机运动，p=[S1,S2,S3,S4,S5,S6]
def arm_move_6(p, s_time = 500):
    for i in range(6):
        id = i + 1
        Arm.Arm_serial_servo_write(id, p[i], s_time)
        time.sleep(.01)
    time.sleep(s_time/1000)

# 定义移动机械臂函数,同时控制 1-5 号舵机运动，p=[S1,S2,S3,S4,S5]
```

```
def arm_move(p, s_time = 500):
    for i in range(5):
        id = i + 1
        if id == 5:
            time.sleep(.1)
            Arm.Arm_serial_servo_write(id, p[i], int(s_time*1.2))
        elif id == 1 :
            Arm.Arm_serial_servo_write(id, p[i], int(3*s_time/4))
        else:
            Arm.Arm_serial_servo_write(id, p[i], int(s_time))
        time.sleep(.01)
    time.sleep(s_time/1000)

# 定义夹积木块函数，enable=1：夹住，=0：松开
def arm_clamp_block(enable):
    if enable == 0:
        Arm.Arm_serial_servo_write(6, 60, 400)
    else:
        Arm.Arm_serial_servo_write(6, 135, 400)
    time.sleep(.5)
```

```
# 定义不同位置的变量参数
look_at = [90, 164, 18, 0, 90, 90]
p_top = [90, 80, 50, 50, 270]

p_Yellow = [65, 22, 64, 56, 270]
p_Red = [118, 19, 66, 56, 270]

p_Green = [136, 66, 20, 29, 270]
p_Blue = [44, 66, 20, 28, 270]

p_layer_4 = [90, 76, 40, 17, 270]
p_layer_3 = [90, 65, 44, 17, 270]
p_layer_2 = [90, 65, 25, 36, 270]
p_layer_1 = [90, 48, 35, 30, 270]



p_push_over_1 = [90, 90, 5, 0, 90, 150]
p_push_over_2 = [90, 90, 0, 50, 90, 150]
```

```
#定义抓取方块的状态
yellow_grabbed = 0
red_grabbed = 0
green_grabbed = 0
blue_grabbed = 0


#定义手势识别次数
Count_One = 0
Count_Two = 0
Count_Three = 0
Count_Four = 0
Count_Fist = 0
```

```
arm_move_6(look_at, 1000)
time.sleep(1)
```

```
#数字功能定义
def number_action(index):
    if index == 1:
        # 抓取黄色的积木块
        arm_move(p_top, 1000)
        arm_move(p_Yellow, 1000)
        arm_clamp_block(1)
#           time.sleep(.5)
        arm_move(p_top, 1000)
    elif index == 2:
        # 抓取红色的积木块
        arm_move(p_top, 1000)
        arm_move(p_Red, 1000)
        arm_clamp_block(1)
        arm_move(p_top, 1000)
    elif index == 3:
        # 抓取绿色的积木块
        arm_move(p_top, 1000)
        arm_move(p_Green, 1000)
        arm_clamp_block(1)
```

```
            arm_move(p_top, 1000)
        elif index == 4:
            # 抓取蓝色的积木块
            arm_move(p_top, 1000)
            arm_move(p_Blue, 1000)
            arm_clamp_block(1)
            arm_move(p_top, 1000)


def put_down_block(layer):

        if layer == 1:
            arm_move(p_layer_1, 1000)
            arm_clamp_block(0)
            arm_move_6(look_at, 1000)
        elif layer == 2:
            arm_move(p_layer_2, 1000)
            arm_clamp_block(0)
            arm_move_6(look_at, 1000)
        elif layer == 3:
            arm_move(p_layer_3, 1000)
            arm_clamp_block(0)
            arm_move_6(look_at, 1000)
        elif layer == 4:
            arm_move(p_layer_4, 1000)
            time.sleep(.1)
            arm_clamp_block(0)
            arm_move_6(look_at, 1000)

# 推倒积木块
def push_over_block():
        arm_move_6(p_push_over_1, 1000)
        time.sleep(.2)
        arm_move_6(p_push_over_2, 1000)
        time.sleep(.1)
        arm_move_6(look_at, 1000)
        time.sleep(1)
        global g_layer
        g_layer = 0
```

```
global g_state_arm
g_state_arm = 0

global g_layer
g_layer = 0

def ctrl_arm_move(index):
    global g_layer
    g_layer = g_layer + 1
    if g_layer >= 5:
        g_layer = 1
    arm_clamp_block(0)
    if index == 1:
        number_action(index)
        put_down_block(g_layer)
    elif index == 2:
        number_action(index)
        put_down_block(g_layer)
    elif index == 3:
        number_action(index)
        put_down_block(g_layer)
    elif index == 4:
        number_action(index)
        put_down_block(g_layer)
    elif index == 5:
        time.sleep(1)
        push_over_block()



    global g_state_arm
    g_state_arm = 0
```

```
def start_move_arm(index):
    # 开启机械臂控制线程
    global g_state_arm
    if g_state_arm == 0:
        closeTid = threading.Thread(target = ctrl_arm_move, args = [index])
        closeTid.setDaemon(True)
```

```
            closeTid.start()


            g_state_arm = 1
```

主进程

```
try:
    Arm.Arm_Buzzer_On(1)
    s_time = 300
    Arm.Arm_serial_servo_write(4, 10, s_time)
    time.sleep(s_time/1000)
    Arm.Arm_serial_servo_write(4, 0, s_time)
    time.sleep(s_time/1000)
    Arm.Arm_serial_servo_write(4, 10, s_time)
    time.sleep(s_time/1000)
    Arm.Arm_serial_servo_write(4, 0, s_time)
    time.sleep(s_time/1000)

    while True:
        """1.拍照 """
        ret, frame = g_camera.read()

        """ 2.调用手势识别 """
        raw = str(client.gesture(image_widget.value))
        text = demjson.decode(raw)
        try:
            res = text['result'][0]['classname']
        except:
#             print('识别结果：什么也没识别到哦~' )
#             img = cv2ImgAddText(frame, "未识别", 250, 30, (0, 0 , 255), 30)
            img = frame
        else:
#             print('识别结果：' + hand[res])
#             img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 , 0), 30)
            if res == 'One':
                Count_One = Count_One + 1
                Count_Two = 0
                Count_Three = 0
                Count_Four = 0
                Count_Fist = 0
                if Count_One >= 3:
```

```
                print('识别结果：' + hand[res])
                img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 , 0), 30)
                if yellow_grabbed == 0:
                    start_move_arm(1)
#                       global yellow_grabbed
                    yellow_grabbed = 1
            elif res == 'Two':
                Count_Two = Count_Two + 1
                Count_Three = 0
                Count_Four = 0
                Count_Fist = 0
                if Count_Two >= 3:
                    print('识别结果：' + hand[res])
                    img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 , 0), 30)
                    if red_grabbed == 0:
                        start_move_arm(2)
#                       global red_grabbed
                        red_grabbed = 1
            elif res == 'Three':
                Count_Three = Count_Three + 1
                Count_Two = 0
                Count_Four = 0
                Count_Fist = 0
                if Count_Three >= 3:
                    print('识别结果：' + hand[res])
                    img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 , 0), 30)
                    if green_grabbed == 0:
                        start_move_arm(3)
#                       global green_grabbed
                        green_grabbed = 1
            elif res == 'Four':
                Count_Four = Count_Four + 1
                Count_Two = 0
                Count_Three = 0
                Count_Fist = 0
                if Count_Four >= 3:
                    print('识别结果：' + hand[res])
                    img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 , 0), 30)
                    if blue_grabbed == 0:
```

```
                                start_move_arm(4)
        #                           global blue_grabbed
                                blue_grabbed = 1
                    elif res == 'Fist': #拳头
                        Count_Fist = Count_Fist + 1
                        Count_One = 0
                        Count_Two = 0
                        Count_Three = 0
                        Count_Four = 0

                        if Count_Fist >= 3:
                            print('识别结果：' + hand[res])
                            img = cv2ImgAddText(frame, hand[res], 250, 30, (0, 255 , 0), 30)
                            start_move_arm(5)
                            yellow_grabbed = 0
                            red_grabbed = 0
                            green_grabbed = 0
                            blue_grabbed = 0
                            Count_Fist = 0
                    else:
                        img = frame

                image_widget.value = bgr8_to_jpeg(img)
    except KeyboardInterrupt:
        print(" Program closed! ")
        pass
```

```
g_camera.release()                  #使用完成对象记住释放掉对象，不然下一个程序使用这个对象模块会被占用，导致无法使用
```

7.13、颜色识别抓取积木

7.13.1、颜色校准

程序位于 dofbot_ws/src/dofbot_color_identify/scripts/HSV 校准.ipynb，双击用 JupyterLab 打开程序。

每个与颜色有关的 AI 玩法，【scripts】文件夹下都有各自的【HSV 校准.ipynb】文件。通过调节 HSV 的高低阈值,过滤掉干扰的颜色,使得方块在复杂的环境中能够非常理想的被识别出来.第一次使用颜色类的玩法时,最好校准一下。

HSV 的介绍在前面‘7.1、使用 OpenCV 进行颜色检测’由介绍。

1. 首先关闭大程序打开终端，依次执行如下命令：

```
cd ~/Arm/
./kill_cam.sh
```

2. 运行程序后界面示例：屏幕中上方显示选择哪种颜色,右侧上方的六个滑动条分别对应着 HSV 的六个数值,滑动滑动条,实时调整每种颜色的 HSV 阈值。

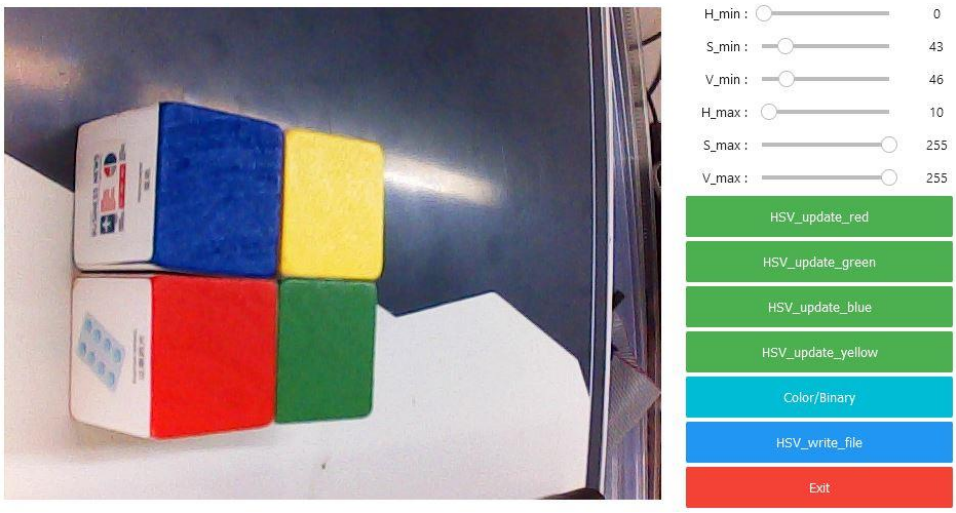


图 7.13.1 程序界面

3. 启动完所有代码块后,在代码的最下方显示如图所示的界面,默认颜色选择为空,故不识别任何颜色。

4. 点击【HSV_update_green】按钮,便开始识别绿色的物体(注:此颜色识别,检测的是轮廓,故物体完全在摄像头范围内才可正常识别),滑动右上方的滑动条进行调节绿色的 HSV 阈值,调节时要注意多方位调

节,在不同的视野环境下调节,直到复杂环境下能够清晰的识别出物体,不被其他物体所干扰为止【其他颜色类似】

图 7.13.2 调节识别颜色

5. 【Color/Binary】按钮是彩色图和二值图的切换,只有在选择完颜色的情况下才有效.切换后只显示选择的颜色二值图像,更便我们调试。

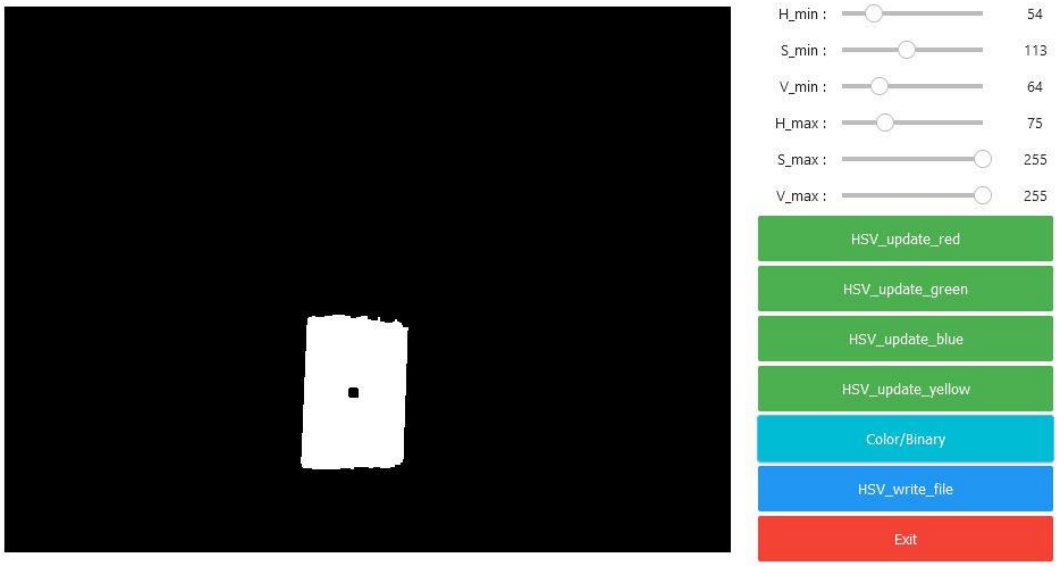


图 7.13.3 二值图

6. 【HSV_write_file】按钮,调试完所有颜色的 HSV 值后,点击此按钮.将调试好的所有参数以【.txt】的格式保存在代码同路径下,下次启动自动读取文件的参数。

7. 【Exit】按钮,关闭摄像头,退出程序。

Python 代码：

导入头文件

```
#!/usr/bin/env python
# coding: utf-8
import threading
import cv2 as cv
from time import sleep
from dofbot_config import *
import ipywidgets as widgets
from IPython.display import display
```

创建实例，初始化参数

```
# 创建更新 HSV 实例
update_hsv = update_hsv()
# 初始化 num 参数
num=0
# 初始化模式
model = "General"
# 初始化 HSV 名称
HSV_name=None
# 初始化 HSV 值
color_hsv    = {"red"     : ((0, 43, 46), (10, 255, 255)),
                "green" : ((35, 43, 46), (77, 255, 255)),
                "blue"    : ((100, 43, 46), (124, 255, 255)),
                "yellow": ((26, 43, 46), (34, 255, 255))}
# 设置随机颜色
color = [[random.randint(0, 255) for _ in range(3)] for _ in range(255)]
# HSV 参数路径
HSV_path="/home/jetson/dofbot_ws/src/dofbot_color_identify/scripts/HSV_config.txt"
# 读取 HSV 配置文件,更新 HSV 值
try: read_HSV(HSV_path,color_hsv)
except Exception: print("Read HSV_config Error!!!")
```

初始化机械臂位置

```
import Arm_Lib
Arm = Arm_Lib.Arm_Device()
joints_0 = [90, 135, 0, 0, 90, 30]
Arm.Arm_serial_servo_write6_array(joints_0, 1000)
```

创建控件

```
# 创建布局
```

```
button_layout          = widgets.Layout(width='260px', height='40px', align_self='center')
# 输出部件
output = widgets.Output()
# 进入颜色更新模式
HSV_update_red        = widgets.Button(description='HSV_update_red',       button_style='success',
layout=button_layout)
HSV_update_green      = widgets.Button(description='HSV_update_green',    button_style='success',
layout=button_layout)
HSV_update_blue       = widgets.Button(description='HSV_update_blue',      button_style='success',
layout=button_layout)
HSV_update_yellow    = widgets.Button(description='HSV_update_yellow', button_style='success',
layout=button_layout)
HSV_write_file        = widgets.Button(description='HSV_write_file',       button_style='primary',
layout=button_layout)
Color_Binary          = widgets.Button(description='Color/Binary',          button_style='info',
layout=button_layout)
# 调整滑杆
H_min_slider          = widgets.IntSlider(description='H_min :', value=0, min=0, max=255, step=1,
orientation='horizontal')
S_min_slider          = widgets.IntSlider(description='S_min :', value=43, min=0, max=255, step=1,
orientation='horizontal')
V_min_slider          = widgets.IntSlider(description='V_min :', value=46, min=0, max=255, step=1,
orientation='horizontal')
H_max_slider          = widgets.IntSlider(description='H_max :', value=10, min=0, max=255, step=1,
orientation='horizontal')
S_max_slider          = widgets.IntSlider(description='S_max :', value=255, min=0, max=255, step=1,
orientation='horizontal')
V_max_slider          = widgets.IntSlider(description='V_max :', value=255, min=0, max=255, step=1,
orientation='horizontal')
# 退出按钮
exit_button = widgets.Button(description='Exit', button_style='danger', layout=button_layout)
# 图像控件
imgbox = widgets.Image(format='jpg', height=480, width=640, layout=widgets.Layout(align_self='center'))
# 调试按钮布局
HSV_slider = widgets.VBox(
    [H_min_slider, S_min_slider, V_min_slider, H_max_slider, S_max_slider, V_max_slider,
HSV_update_red,
     HSV_update_green, HSV_update_blue, HSV_update_yellow, Color_Binary,HSV_write_file,
exit_button],
```

```
    layout=widgets.Layout(align_self='center'))
# 整体布局
controls_box = widgets.HBox([imgbox,HSV_slider], layout=widgets.Layout(align_self='center'))
```

颜色更新回调

```
def update_red_Callback(value):
    # 点击红色按钮,回调函数
    global HSV_name
    HSV_name = "red"
    H_min_slider.value=color_hsv[HSV_name][0][0]
    S_min_slider.value=color_hsv[HSV_name][0][1]
    V_min_slider.value=color_hsv[HSV_name][0][2]
    H_max_slider.value=color_hsv[HSV_name][1][0]
    S_max_slider.value=color_hsv[HSV_name][1][1]
    V_max_slider.value=color_hsv[HSV_name][1][2]
def update_green_Callback(value):
    # 点击绿色按钮,回调函数
    global HSV_name
    HSV_name = "green"
    H_min_slider.value=color_hsv[HSV_name][0][0]
    S_min_slider.value=color_hsv[HSV_name][0][1]
    V_min_slider.value=color_hsv[HSV_name][0][2]
    H_max_slider.value=color_hsv[HSV_name][1][0]
    S_max_slider.value=color_hsv[HSV_name][1][1]
    V_max_slider.value=color_hsv[HSV_name][1][2]
def update_blue_Callback(value):
    # 点击蓝色按钮,回调函数
    global HSV_name
    HSV_name = "blue"
    H_min_slider.value=color_hsv[HSV_name][0][0]
    S_min_slider.value=color_hsv[HSV_name][0][1]
    V_min_slider.value=color_hsv[HSV_name][0][2]
    H_max_slider.value=color_hsv[HSV_name][1][0]
    S_max_slider.value=color_hsv[HSV_name][1][1]
    V_max_slider.value=color_hsv[HSV_name][1][2]
def update_yellow_Callback(value):
    # 点击黄色按钮,回调函数
    global HSV_name
    HSV_name = "yellow"
    H_min_slider.value=color_hsv[HSV_name][0][0]
```

```
        S_min_slider.value=color_hsv[HSV_name][0][1]
        V_min_slider.value=color_hsv[HSV_name][0][2]
        H_max_slider.value=color_hsv[HSV_name][1][0]
        S_max_slider.value=color_hsv[HSV_name][1][1]
        V_max_slider.value=color_hsv[HSV_name][1][2]
# 点击按钮
HSV_update_red.on_click(update_red_Callback)
HSV_update_green.on_click(update_green_Callback)
HSV_update_blue.on_click(update_blue_Callback)
HSV_update_yellow.on_click(update_yellow_Callback)
```

模式切换控件

```
# 模式设置
def write_file_Callback(value):
        global model
        model = 'Write_file'
def Color_Binary_Callback(value):
        global model,num
        if num%2==0: model="Binary"
        if num%2==1: model="General"
        num+=1
def exit_button_Callback(value):
        global model
        model = 'Exit'
        with output: print(model)
# 点击按钮
HSV_write_file.on_click(write_file_Callback)
Color_Binary.on_click(Color_Binary_Callback)
exit_button.on_click(exit_button_Callback)
```

摄像头函数

```
def camera():
        global HSV_name,model,color_hsv,num
        # 打开摄像头
        capture = cv.VideoCapture(0)
        # 当摄像头正常打开的情况下循环执行
        while capture.isOpened():
                try:
                        # 读取相机的每一帧
                        _, img = capture.read()
                        # 统一图像大小
```

```
                img = cv.resize(img, (640, 480))
                if HSV_name!=None:
                    # 添加文字
                    cv.putText(img, HSV_name, (int(img.shape[0] / 2)+40, 50),
cv.FONT_HERSHEY_SIMPLEX, 2, color[random.randint(0, 254)], 2)
                    # 实时修改颜色的 HSV 值
                    color_hsv[HSV_name] = ((H_min_slider.value, S_min_slider.value, V_min_slider.value),
(H_max_slider.value, S_max_slider.value, V_max_slider.value))
                    # 轮廓检测
                    img, binary = update_hsv.get_contours(img, HSV_name,
color_hsv[HSV_name],color_hsv)
                    if model == "Binary":
                        # 二值图显示
                        _, img = update_hsv.get_contours(img, HSV_name,
color_hsv[HSV_name],color_hsv)
                    if model == 'Write_file':
                        # 写入文件
                        try: write_HSV(HSV_path,color_hsv)
                        except Exception: print("File Path Error!!!")
                        # 所有颜色取反
                        cv.bitwise_not(img, img)
                        num=0
                        model="General"
                if model == 'Exit':
                    # 退出程序
                    cv.destroyAllWindows()
                    capture.release()
                    break
                # 图像格式转换
                imgbox.value = cv.imencode('.jpg', img)[1].tobytes()
            except KeyboardInterrupt:
                capture.release()
```

显示

```
# 显示布局
display(controls_box,output)
# 开启图像线程
threading.Thread(target=camera, ).start()
```

7.13.2、颜色识别抓取积木

程序位于‘whitebear’文件夹里的‘AI 视觉综合项目’文件夹，找到‘15.颜色识别抓取积木.ipynb’程序，双击用 JupyterLab 打开程序。

按照颜色位置依次摆放木块：

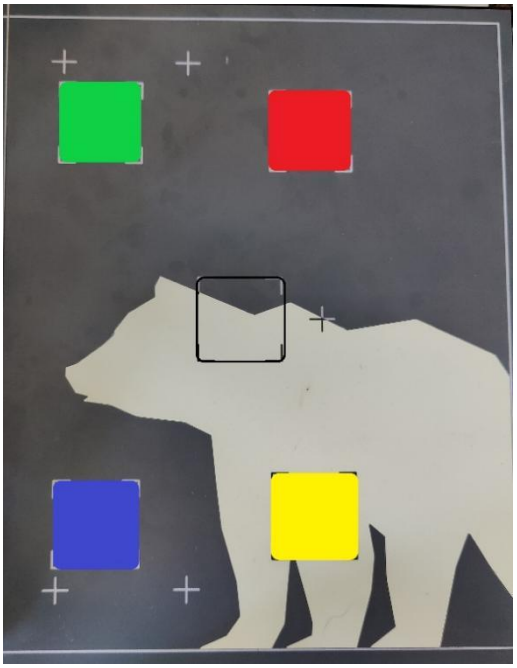


图 7.13.4 木块摆放位置

程序运行后，可以把积木颜色用手机拍下来，块放到摄像头的前面，摄像头会检测到积木块的颜色，然后把积木块放到对应颜色的区域内，机械臂会根据检测到的颜色把对应区域内的积木块夹取到中间的区域。或者先把积木块放到对应颜色区域内，然后找黄色、红色、绿色、蓝色四种颜色的卡片放到摄像头前，机械臂会根据检测到的颜色抓取对应的积木块放到中间的区域。

注意：每次抓一个积木后，需要把中间区域的积木拿开，否则会挡住下一次放积木。

Python 代码：

```
#bgr8 转 jpeg 格式
import enum
import cv2

def bgr8_to_jpeg(value, quality=75):
    return bytes(cv2.imencode('.jpg', value)[1])
```

```
#导入相关的模块
from Arm_Lib import Arm_Device
```

```
Arm = Arm_Device()
```

```
#摄像头组件显示
import traitlets
import ipywidgets.widgets as widgets
import time
# 线程功能操作库
import threading
import inspect
import ctypes

origin_widget = widgets.Image(format='jpeg', width=320, height=240)
mask_widget = widgets.Image(format='jpeg',width=320, height=240)
result_widget = widgets.Image(format='jpeg',width=320, height=240)

# 创建一个水平框容器，将图像小部件相邻放置
image_container = widgets.HBox([origin_widget, mask_widget, result_widget])
# image_container = widgets.Image(format='jpeg', width=600, height=500)
display(image_container)
```

```
#线程相关函数
def _async_raise(tid, exctype):
    """raises the exception, performs cleanup if needed"""
    tid = ctypes.c_long(tid)
    if not inspect.isclass(exctype):
        exctype = type(exctype)
    res = ctypes.pythonapi.PyThreadState_SetAsyncExc(tid, ctypes.py_object(exctype))
    if res == 0:
        raise ValueError("invalid thread id")
    elif res != 1:
        # 如果 res 的值大于 1，则需要异常处理
        ctypes.pythonapi.PyThreadState_SetAsyncExc(tid, None)
def stop_thread(thread):
    _async_raise(thread.ident, SystemExit)
```

```
def get_color(img):
    H = []
    color_name={}
```

```
    img = cv2.resize(img, (640, 480), )
    # 将彩色图转成 HSV
    HSV = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    # 画矩形框
    cv2.rectangle(img, (280, 180), (360, 260), (0, 255, 0), 2)
    # 依次取出每行每列的 H,S,V 值放入容器中
    for i in range(280, 360):
        for j in range(180, 260): H.append(HSV[j, i][0])
    # 分别计算出 H,S,V 的最大最小
    H_min = min(H);H_max = max(H)
#       print(H_min,H_max)
    # 判断颜色
    if H_min >= 0 and H_max <= 10 or H_min >= 156 and H_max <= 180: color_name['name'] = 'red'
    elif H_min >= 26 and H_max <= 34: color_name['name'] = 'yellow'
    elif H_min >= 35 and H_max <= 78: color_name['name'] = 'green'
    elif H_min >= 100 and H_max <= 124: color_name['name'] = 'blue'
    return img, color_name
```

```
# 定义不同位置的变量参数
look_at = [90, 164, 18, 0, 90, 90]
p_top = [90, 80, 50, 50, 270]

p_Yellow =    [62, 35, 54, 56, 270]
p_Red = [116, 35, 54, 56, 270]

p_Green = [132, 86, 16, 26, 270]
p_Blue = [44, 86, 16, 26, 270]

p_gray = [90, 68, 28, 28, 270]
```

```
# 定义移动机械臂函数,同时控制 1-6 号舵机运动，p=[S1,S2,S3,S4,S5,S6]
def arm_move_6(p, s_time = 500):
    for i in range(6):
        id = i + 1
        Arm.Arm_serial_servo_write(id, p[i], s_time)
        time.sleep(.01)
    time.sleep(s_time/1000)

# 定义移动机械臂函数,同时控制 1-5 号舵机运动，p=[S1,S2,S3,S4,S5]
```

```
def arm_move(p, s_time = 500):
    for i in range(5):
        id = i + 1
        if id == 5:
            time.sleep(.1)
            Arm.Arm_serial_servo_write(id, p[i], int(s_time*1.2))
        elif id == 1 :
            Arm.Arm_serial_servo_write(id, p[i], int(3*s_time/4))
        else:
            Arm.Arm_serial_servo_write(id, p[i], int(s_time))
        time.sleep(.01)
    time.sleep(s_time/1000)
# 定义夹积木块函数，enable=1：夹住，=0：松开
def arm_clamp_block(enable):
    if enable == 0:
        Arm.Arm_serial_servo_write(6, 60, 400)
    else:
        Arm.Arm_serial_servo_write(6, 140, 400)
    time.sleep(.5)
```

```
arm_move_6(look_at, 1000)
time.sleep(1)
```

```
global g_state_arm
g_state_arm = 0


def ctrl_arm_move(index):
    arm_clamp_block(0)
    if index == 1:
        print("黄色")
        number_action(index)
        put_down_block()
    elif index == 2:
        print("红色")
        number_action(index)
        put_down_block()
    elif index == 3:
        print("绿色")
        number_action(index)
```

```
            put_down_block()
        elif index == 4:
            print("蓝色")
            number_action(index)
            put_down_block()
        global g_state_arm
        g_state_arm = 0
```

```
#数字功能定义
def number_action(index):
        if index == 1:
            # 抓取黄色的积木块
            arm_move(p_top, 1000)
            arm_move(p_Yellow, 1000)
            arm_clamp_block(1)
#               time.sleep(.5)
            arm_move(p_top, 1000)
        elif index == 2:
            # 抓取红色的积木块
            arm_move(p_top, 1000)
            arm_move(p_Red, 1000)
            arm_clamp_block(1)
            arm_move(p_top, 1000)
        elif index == 3:
            # 抓取绿色的积木块
            arm_move(p_top, 1000)
            arm_move(p_Green, 1000)
            arm_clamp_block(1)
            arm_move(p_top, 1000)
        elif index == 4:
            # 抓取蓝色的积木块
            arm_move(p_top, 1000)
            arm_move(p_Blue, 1000)
            arm_clamp_block(1)
            arm_move(p_top, 1000)



def put_down_block():
        arm_move(p_gray, 1000)
```

```
    arm_clamp_block(0)
    arm_move_6(look_at, 1000)
```

```
def start_move_arm(index):
    # 开启机械臂控制线程
    global g_state_arm
    if g_state_arm == 0:
        closeTid = threading.Thread(target = ctrl_arm_move, args = [index])
        closeTid.setDaemon(True)
        closeTid.start()

        g_state_arm = 1
```

主进程

```
import cv2
import numpy as np
import ipywidgets.widgets as widgets



cap = cv2.VideoCapture(0)
cap.set(3, 640)
cap.set(4, 480)
cap.set(5, 30)  #设置帧率
cap.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter.fourcc('M', 'J', 'P', 'G'))
# image.set(cv2.CAP_PROP_BRIGHTNESS, 40) #设置亮度  -64 - 64   0.0
# image.set(cv2.CAP_PROP_CONTRAST, 50)    #设置对比度  -64 - 64   2.0
# image.set(cv2.CAP_PROP_EXPOSURE, 156)   #设置曝光值  1.0 - 5000   156.0



# 默认选择红色的,程序会自动根据方框中检测到的颜色切换颜色
# 红色区间
color_lower = np.array([0, 43, 46])
color_upper = np.array([10, 255, 255])

# #绿色区间
# color_lower = np.array([35, 43, 46])
# color_upper = np.array([77, 255, 255])

# #蓝色区间
# color_lower=np.array([100, 43, 46])
```

```
# color_upper = np.array([124, 255, 255])

# #黄色区间
# color_lower = np.array([26, 43, 46])
# color_upper = np.array([34, 255, 255])

# #橙色区间
# color_lower = np.array([11, 43, 46])
# color_upper = np.array([25, 255, 255])



def Color_Recongnize():
    Arm.Arm_Buzzer_On(1)
    s_time = 300
    Arm.Arm_serial_servo_write(4, 10, s_time)
    time.sleep(s_time/1000)
    Arm.Arm_serial_servo_write(4, 0, s_time)
    time.sleep(s_time/1000)
    Arm.Arm_serial_servo_write(4, 10, s_time)
    time.sleep(s_time/1000)
    Arm.Arm_serial_servo_write(4, 0, s_time)
    time.sleep(s_time/1000)

    while(1):
        # get a frame and show  获取视频帧并转成 HSV 格式，利用 cvtColor()将 BGR 格式转成 HSV 格式，参数为 cv2.COLOR_BGR2HSV。
        ret, frame = cap.read()
        frame, color_name = get_color(frame)
        if len(color_name)==1:
            global color_lower
            global color_upper
#               print ("color_name :", color_name)
#               print ("name :", color_name['name'])
            if color_name['name'] == 'yellow':
                color_lower = np.array([16, 197, 215])
                color_upper = np.array([38, 255, 255])
                start_move_arm(1)
            elif color_name['name'] == 'red':
                color_lower = np.array([0, 205, 139])
```

```
                color_upper = np.array([76, 255, 224])
                start_move_arm(2)
            elif   color_name['name'] == 'green':
                color_lower = np.array([47, 63, 21])
                color_upper = np.array([67, 255, 253])
                start_move_arm(3)
            elif color_name['name'] == 'blue':
                color_lower=np.array([71, 153, 157])
                color_upper = np.array([113, 255, 255])
                start_move_arm(4)


        origin_widget.value = bgr8_to_jpeg(frame)
        #cv2.imshow('Capture', frame)

        # change to hsv model
        hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

        # get mask  利用 inRange()函数和 HSV 模型中蓝色范围的上下界获取 mask，mask 中原视频中的蓝色部分会被弄成白色，其他部分黑色。
        mask = cv2.inRange(hsv, color_lower, color_upper)
        #cv2.imshow('Mask', mask)
        mask_widget.value = bgr8_to_jpeg(mask)


        # detect blue  将 mask 于原视频帧进行按位与操作，则会把 mask 中的白色用真实的图像替换：
        res = cv2.bitwise_and(frame, frame, mask=mask)
        #cv2.imshow('Result', res)
        result_widget.value = bgr8_to_jpeg(res)

        #         if cv2.waitKey(1) & 0xFF == ord('q'):
        #             break
        time.sleep(0.01)


    cap.release()
    #cv2.destroyAllWindows()
```

```
#启动进程
```

```
thread1 = threading.Thread(target=Color_Recongnize)
thread1.setDaemon(True)
thread1.start()
```

```
#结束进程，只有在结束时才需要执行此段代码
try:
    while True:
        time.sleep(.000001)
except KeyboardInterrupt:
    print(" Program closed! ")
    pass
stop_thread(thread1)
```

7.14、你放我抓（需更新）

7.14.1、你放我抓

程序位于 dofbot_ws/src/dofbot_color_sorting/你放我抓.ipynb，双击用 JupyterLab 打开程序。

HSV 校准方法参见章节‘7.13.1 颜色校准’

1. 首先关闭大程序打开终端，依次执行如下命令：

```
cd ~/Arm/
./kill_cam.sh
```

2. 启动代码块,机械臂自动到达识别位置,将方块放在灰色方格中,机械臂持续正确识别 10 次后,自动抓取,放置对应颜色的方格中。

Python 代码：

```
#!/usr/bin/env python
# coding: utf-8
import cv2 as cv
import threading
from time import sleep
from dofbot_config import *
import ipywidgets as widgets
from IPython.display import display
from color_sorting import color_sorting
```

初始化机械臂位置

```
import Arm_Lib
Arm = Arm_Lib.Arm_Device()
joints_0 = [90, 130, 0, 0, 90, 30]
Arm.Arm_serial_servo_write6_array(joints_0, 1000)
```

创建实例，初始化参数

```
# 创建实例
sorting = color_sorting()
# 初始化模式
model = 'General'
# 颜色 HSV 阈值
color_hsv    = {"red"     : ((0, 43, 46), (10, 255, 255)),
                "green" : ((35, 43, 46), (77, 255, 255)),
```

```
                    "blue"   : ((100, 43, 46), (124, 255, 255)),
                    "yellow": ((26, 43, 46), (34, 255, 255))}
# HSV 参数路径
HSV_path="/home/jetson/dofbot_ws/src/dofbot_color_sorting/HSV_config.txt"
#  读取 HSV 配置文件,更新 HSV 值
try: read_HSV(HSV_path,color_hsv)
except Exception: print("Read HSV_config Error!!!")
```

创建控件

```
#  创建控件布局
button_layout    = widgets.Layout(width='200px', height='70px', align_self='center')
#  输出打印
output = widgets.Output()
#  退出按钮
exit_button = widgets.Button(description='Exit', button_style='danger', layout=button_layout)
#  图像控件
imgbox = widgets.Image(format='jpg', height=480, width=640, layout=widgets.Layout(align_self='center'))
#  垂直放置
controls_box = widgets.VBox([imgbox, exit_button], layout=widgets.Layout(align_self='center'))
# ['auto', 'flex-start', 'flex-end', 'center', 'baseline', 'stretch', 'inherit', 'initial', 'unset']
```

控制按钮

```
def exit_button_Callback(value):
    global model
    model = 'Exit'
#       with output: print(model)
exit_button.on_click(exit_button_Callback)
```

主程序

```
def camera():
    #  打开摄像头
    capture = cv.VideoCapture(0)
    #  当摄像头正常打开的情况下循环执行
    while capture.isOpened():
        try:
            #  读取相机的每一帧
            _, img = capture.read()
            #  统一图像大小
            img = cv.resize(img, (640, 480))
#               cv.line(img, (320, 0), (320, 480), color=(0, 255, 0), thickness=1)
#               cv.line(img, (0, 240), (640, 240), color=(0, 255, 0), thickness=1)
            #  获得运动信息
```

```
                img = sorting.Sorting_grap(img, color_hsv)
                if model == 'Exit':
                    cv.destroyAllWindows()
                    capture.release()
                    break
                # 添加文字
                imgbox.value = cv.imencode('.jpg', img)[1].tobytes()
            except KeyboardInterrupt:capture.release()
```

启动

```
display(controls_box,output)
threading.Thread(target=camera, ).start()
```

7.15、垃圾分拣（需更新）

7.15.1、单个垃圾分拣（需更新）

yolov5 介绍

2020 年 2 月 YOLO 之父 Joseph Redmon 宣布退出计算机视觉研究领域，2020 年 4 月 23 日 YOLOv4 发布，2020 年 6 月 10 日 YOLOv5 发布。 YOLOv5 的开发者声称 YOLOv5 能在 Tesla P100 上实现 140 FPS 的快速检测；而 YOLOv4 的检测速度是 50 FPS .不仅如此，YOLOv5 的大小仅有 27 MB。而 YOLOv4 却是 244 MB。 YOLOv5 比 YOLOv4 小近 90%。在准确度指标上，YOLOv5 与 YOLOv4 相当。

因此，YOLOv5 速度非常快，有非常轻量级的模型大小，同时准确度又与 YOLOv4 基准相当。

Yolov5 作者的算法性能测试图：

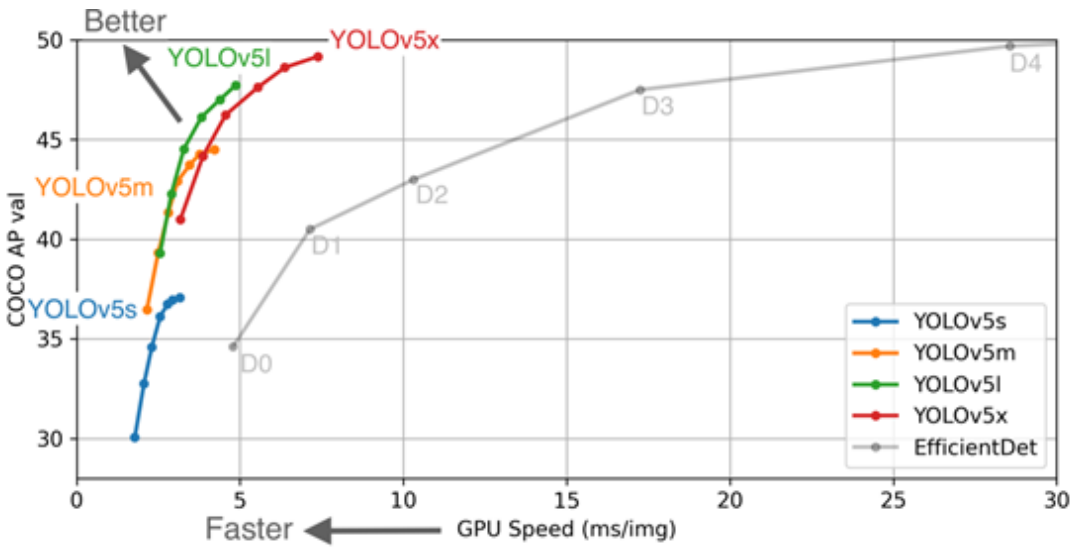


图 7.14.1 算法性能测试图

程序位于 dofbot_ws/src/dofbot_garbage_yolov5/单个垃圾分拣玩法.ipynb，双击用 JupyterLab 打开程序。

首先关闭大程序打开终端，依次执行如下命令：

```
cd ~/Arm/
./kill_cam.sh
```

启动上述代码块时因为要加载模型会比较慢,请耐心等待,将贴有垃圾的方块正对着机械臂放到方框的十字中心,机械臂持续准确识别 10 次,自动分拣。摆放垃圾方块时请正向摆放。

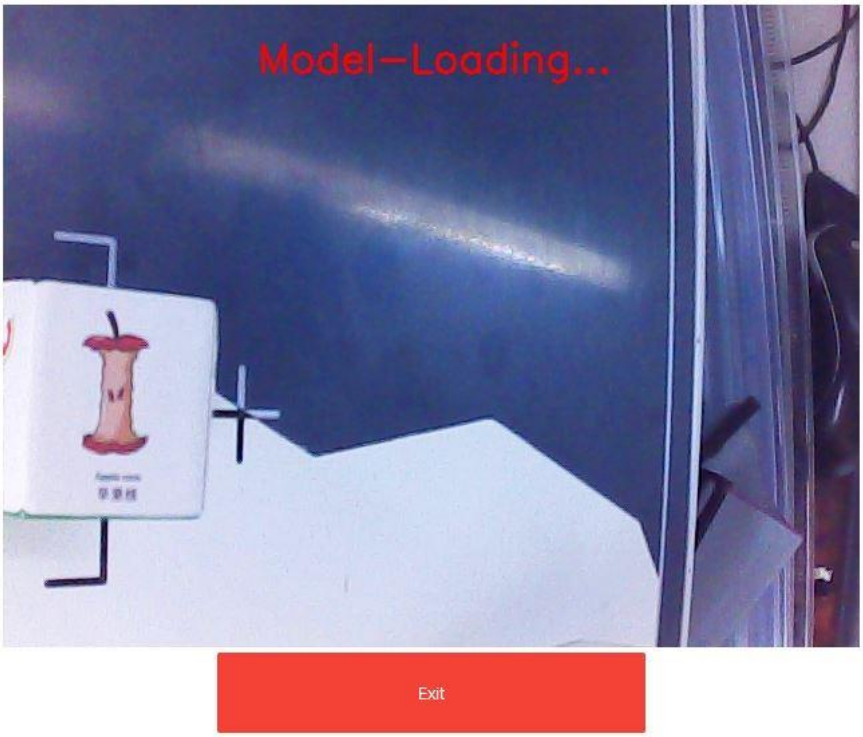


图 7.14.1 加载模型

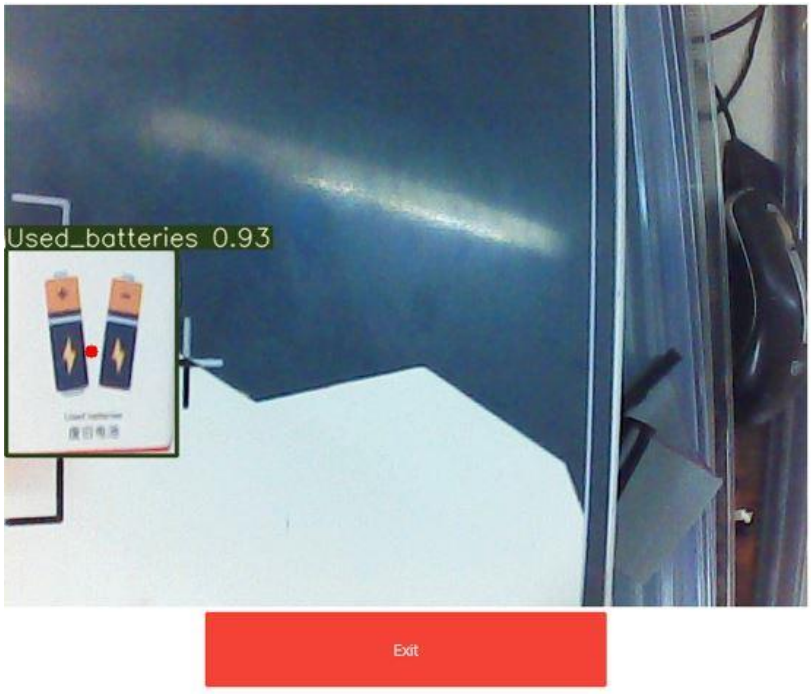


图 7.14.2 识别图案上的垃圾类别

Python 代码：

```
#!/usr/bin/env python
```

```
# coding: utf-8
import Arm_Lib
import cv2 as cv
import threading
from time import sleep
import ipywidgets as widgets
from IPython.display import display
from single_garbage_identify import single_garbage_identify
```

```
# 创建获取目标实例
single_garbage = single_garbage_identify()
# 初始化模式
model = "General"
```

```
import Arm_Lib
arm = Arm_Lib.Arm_Device()
joints_0 = [90, 101, 18, 18, 90, 30]
arm.Arm_serial_servo_write6_array(joints_0, 1000)
```

```
button_layout           = widgets.Layout(width='320px', height='60px', align_self='center')
output = widgets.Output()
# 退出
exit_button = widgets.Button(description='Exit', button_style='danger', layout=button_layout)
imgbox = widgets.Image(format='jpg', height=480, width=640, layout=widgets.Layout(align_self='center'))
controls_box = widgets.VBox([imgbox, exit_button], layout=widgets.Layout(align_self='center'))
```

抓取控制

```
def exit_button_Callback(value):
    global model
    model = 'Exit'
    with output: print(model)
exit_button.on_click(exit_button_Callback)
```

主程序

```
def camera():
    # 打开摄像头
    capture = cv.VideoCapture(0)
    # 当摄像头正常打开的情况下循环执行
    while capture.isOpened():
        try:
            # 读取相机的每一帧
```

```
            _, img = capture.read()
            # 统一图像大小
            img = cv.resize(img, (640, 480))
            img = single_garbage.single_garbage_run(img)
            if model == 'Exit':
                cv.destroyAllWindows()
                capture.release()
                break
            imgbox.value = cv.imencode('.jpg', img)[1].tobytes()
        except KeyboardInterrupt:capture.release()
```

```
# 请将积木块正放在十字中心(图片正对机械臂)
display(controls_box,output)
threading.Thread(target=camera, ).start()
```

7.16、目标追踪（需更新）

目标追踪包含三种玩法颜色追踪、取色追踪(学习追踪)、人脸追踪。其原理是，通过相机对图像做一定的处理，通过特定的方式识别目标，获取到目标在相机下的坐标位置，计算目标中心点距离图像中心点的偏差值，通过 PID 算法调试，驱动机械臂移动。使得目标中心点与图像中心点重合。

7.16.1、颜色追踪与学习追踪

程序位于 dofbot_ws/src/ dofbot_color_follow/颜色追踪与学习追踪.ipynb，双击用 JupyterLab 打开程序。

控制算法 PID 的调节在我们提供的资料中,如需调节,请查看 PID 算法进行学习。

颜色追踪与取色追踪的库代码请见 dofbot_ws/src/dofbot_color_follow/color_follow.py

颜色追踪：

1. 首先关闭大程序打开终端，依次执行如下命令：

```
cd ~/Arm/
./kill_cam.sh
```

2. 运行程序后界面示例，默认不追踪,需选择右侧的功能按钮。

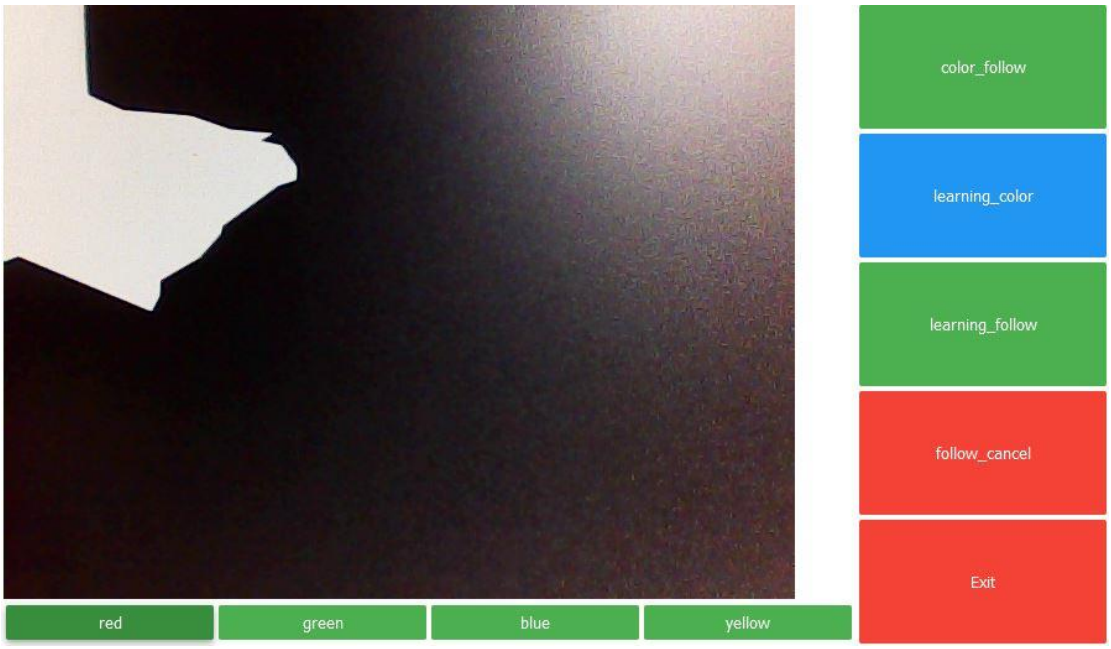


图 7.15.1 界面实例

3. 启动代码块,点击【color_follow】按钮开启颜色追踪,图像下方可以颜色切换.将方块放在视野下,待摄像头识别出来,机械臂追踪方块移动,正对方块中心。

4. 点击【follow_cancel】按钮,取消跟踪,不退出程序.点击 Exit 按钮,退出程序。

取色追踪:

1. 启动代码块,点击【leraning_color】按钮,画面中心出现一个方框,如下图所示。

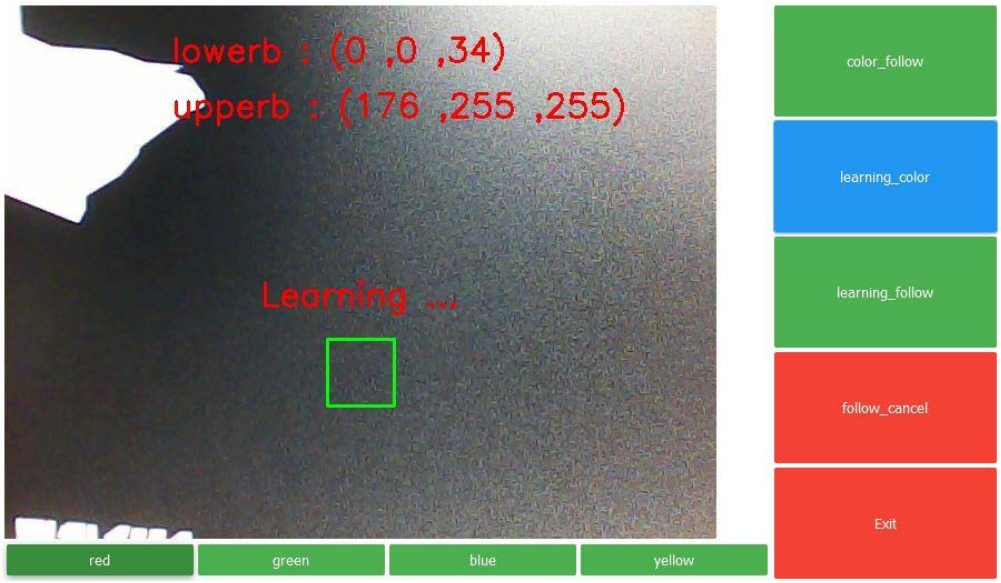


图 7.15.2 取色追踪

2. 在方框内放置物体,实时打印 HSV 的高低阈值,当方框下面出现【OK !!!】字样时,说明识别成功。

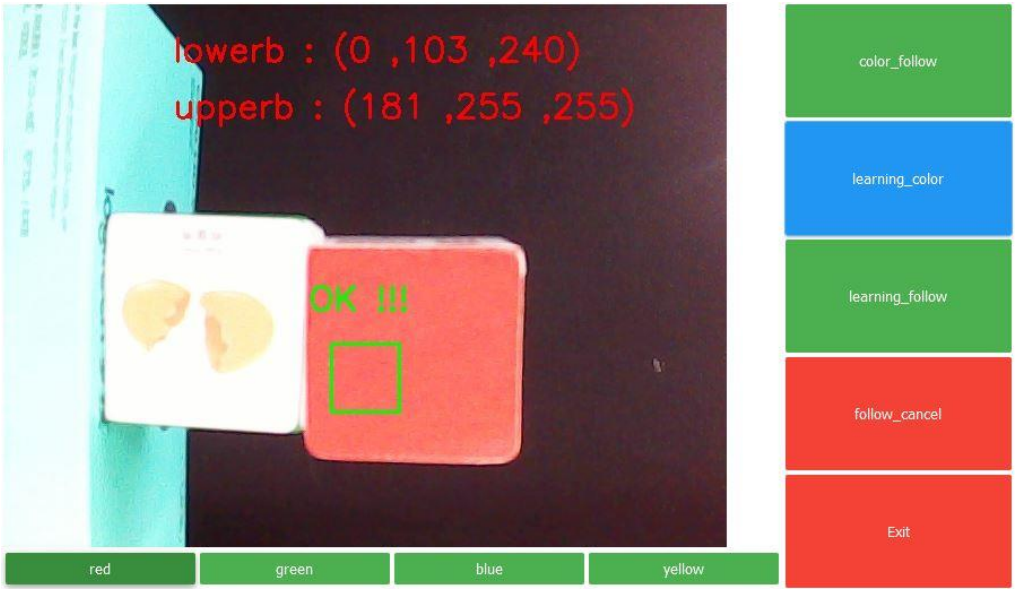


图 7.15.3 打印 HSV 阈值

3. 点击【leraning_follow】按钮,只要能够检测到学习颜色的轮廓,便可以实时跟踪。

4. 点击【follow_cancel】按钮,取消跟踪,不退出程序.点击 Exit 按钮,退出程序。

Python 代码：

```
import cv2 as cv
import threading
import random
from time import sleep
import ipywidgets as widgets
from IPython.display import display
from color_follow import color_follow
```

```
import Arm_Lib
Arm = Arm_Lib.Arm_Device()
joints_0 = [90, 135, 20, 25, 90, 30]
Arm.Arm_serial_servo_write6_array(joints_0, 1000)
```

```
follow = color_follow()
# 初始化模式
model = 'General'
# 初始化 HSV_learning 值
HSV_learning = ()
# 初始化 HSV 值
color_hsv = {"red": ((0, 25, 90), (10, 255, 255)),
             "green": ((53, 36, 40), (80, 255, 255)),
             "blue": ((110, 80, 90), (120, 255, 255)),
             "yellow": ((25, 20, 55), (50, 255, 255))}
# 设置随机颜色
color = [[random.randint(0, 255) for _ in range(3)] for _ in range(255)]
# HSV 参数路径
HSV_path="/home/jetson/dofbot_ws/src/dofbot_color_follow/HSV_config.txt"
try: read_HSV(HSV_path,color_hsv)
except Exception: print("Read HSV_config Error !!!")
```

```
button_layout = widgets.Layout(width='200px', height='100px', align_self='center')
# 输出控件
output = widgets.Output()
# 颜色追踪
```

```
color_follow = widgets.Button(description='color_follow', button_style='success', layout=button_layout)
# 选择颜色
choose_color = widgets.ToggleButtons(options=['red', 'green', 'blue', 'yellow'], button_style='success',
                    tooltips=['Description of slow', 'Description of regular', 'Description of fast'])
# 取消追踪
follow_cancel = widgets.Button(description='follow_cancel', button_style='danger', layout=button_layout)
# 学习颜色
learning_color = widgets.Button(description='learning_color', button_style='primary', layout=button_layout)
# 学习颜色追踪
learning_follow = widgets.Button(description='learning_follow', button_style='success', layout=button_layout)
# 退出
exit_button = widgets.Button(description='Exit', button_style='danger', layout=button_layout)
# 图像控件
imgbox = widgets.Image(format='jpg', height=480, width=640, layout=widgets.Layout(align_self='auto'))
# 垂直布局
img_box = widgets.VBox([imgbox, choose_color], layout=widgets.Layout(align_self='auto'))
# 垂直布局
Slider_box = widgets.VBox([color_follow, learning_color, learning_follow,follow_cancel,exit_button],
                                    layout=widgets.Layout(align_self='auto'))
# 水平布局
controls_box = widgets.HBox([img_box, Slider_box], layout=widgets.Layout(align_self='auto'))
# ['auto', 'flex-start', 'flex-end', 'center', 'baseline', 'stretch', 'inherit', 'initial', 'unset']
```

```
def color_follow_Callback(value):
    global model
    model = 'color_follow'
def learning_color_Callback(value):
    global model
    model = 'learning_color'
def learning_follow_Callback(value):
    global model
    model = 'learning_follow'
def follow_cancel_Callback(value):
    global model
    model = 'General'
def exit_button_Callback(value):
    global model
    model = 'Exit'
color_follow.on_click(color_follow_Callback)
```

```
learning_color.on_click(learning_color_Callback)
learning_follow.on_click(learning_follow_Callback)
follow_cancel.on_click(follow_cancel_Callback)
exit_button.on_click(exit_button_Callback)
```

```
def camera():
    global HSV_learning,model
    # 打开摄像头
    capture = cv.VideoCapture(0)
    capture.set(3, 640)
    capture.set(4, 480)
    capture.set(5, 30)  #设置帧率
    # 当摄像头正常打开的情况下循环执行
    while capture.isOpened():
        try:
            # 读取相机的每一帧
            _, img = capture.read()
            # 统一图像大小
            img = cv.resize(img, (640, 480))
#               cv.line(img, (320, 0), (320, 480), color=(0, 255, 0), thickness=1)
#               cv.line(img, (0, 240), (640, 240), color=(0, 255, 0), thickness=1)
            if model == 'color_follow':
                img = follow.follow_function(img, color_hsv[choose_color.value])
                # 添加文字
                cv.putText(img, choose_color.value, (int(img.shape[0] / 2), 50),
cv.FONT_HERSHEY_SIMPLEX, 2, color[random.randint(0, 254)], 2)
            if model == 'learning_color':
                img,HSV_learning = follow.get_hsv(img)
            if model == 'learning_follow' :
                if len(HSV_learning)!=0:
                    print(HSV_learning)
                    img = follow.learning_follow(img, HSV_learning)
                    # 添加文字
                    cv.putText(img,'LeColor', (240, 50), cv.FONT_HERSHEY_SIMPLEX, 1,
color[random.randint(0, 254)], 1)
            if model == 'Exit':
                cv.destroyAllWindows()
                capture.release()
                break
```

```
                imgbox.value = cv.imencode('.jpg', img)[1].tobytes()
            except KeyboardInterrupt:capture.release()
```

```
display(controls_box,output)
threading.Thread(target=camera, ).start()
```

8.AI听觉综合项目

8.1、声卡设置

设置默认声卡

进入系统图形界面，点击右上角的音频图标进去声音设置界面。Output/Input 设备均选择 USB PnP Audio Device。

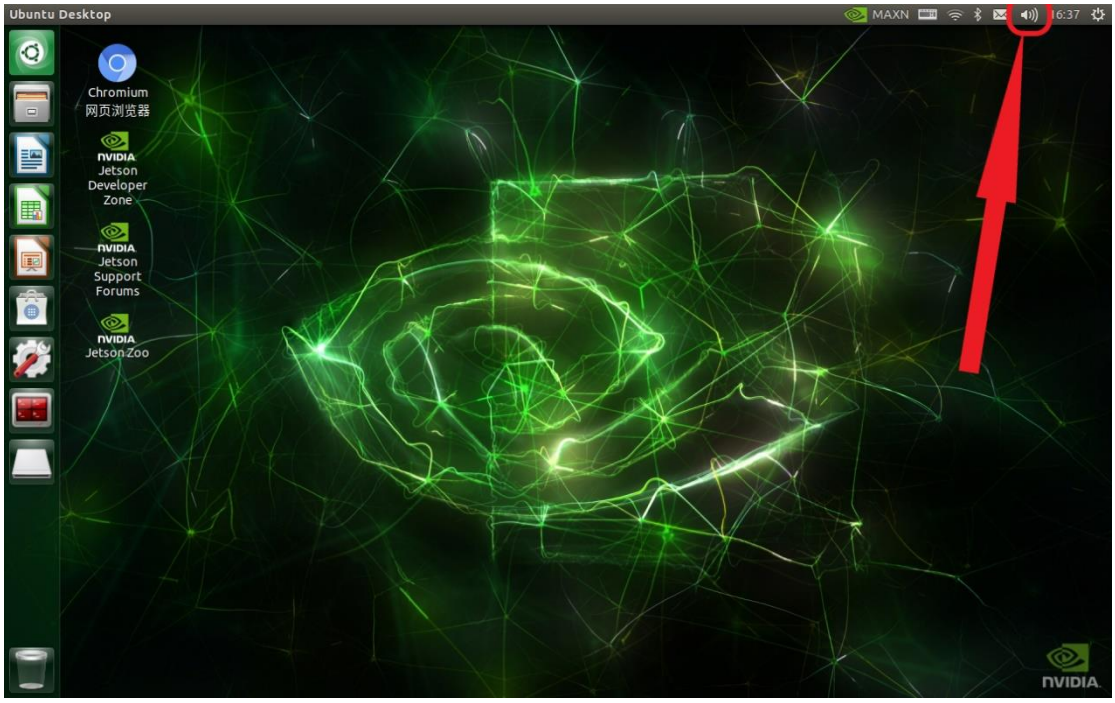


图 8.1.1 声音设置位置

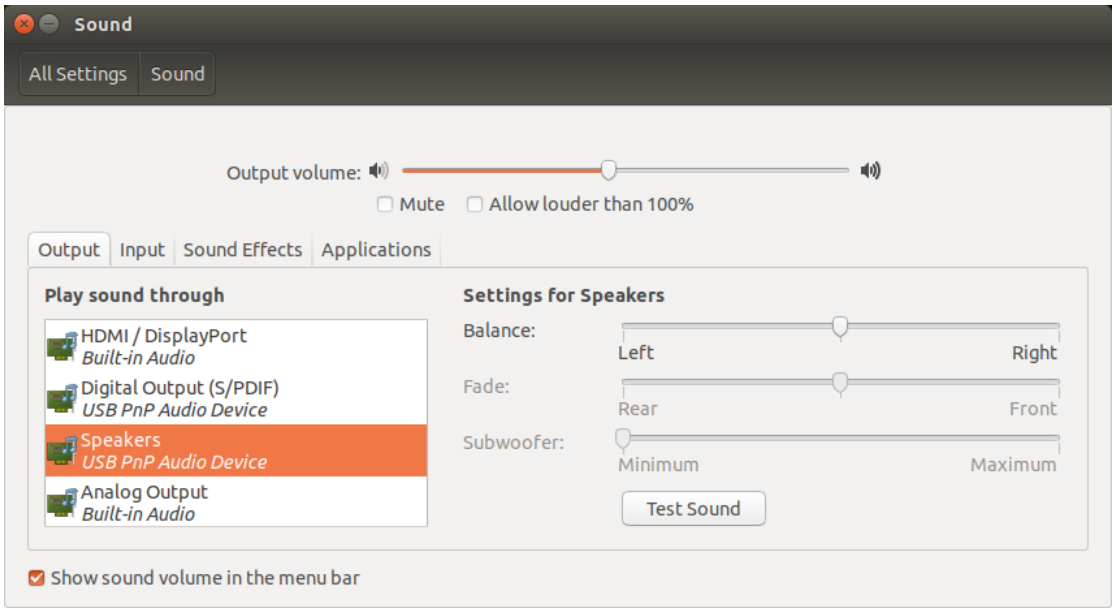


图 8.1.2 output 设备选择

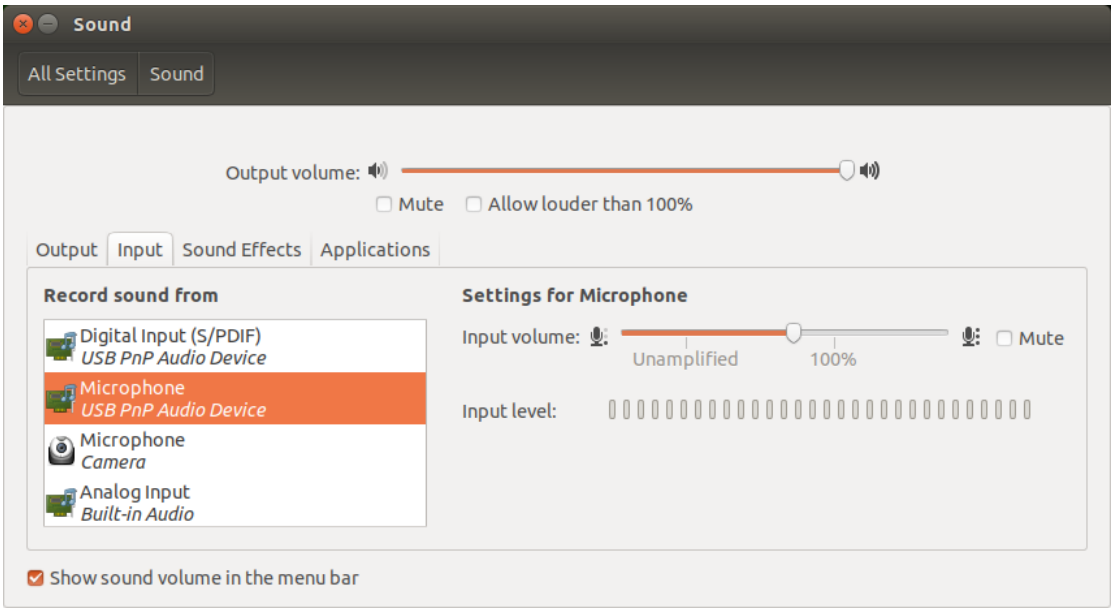


图 8.1.3 input 设备选择

录音测试

```
jetson@linux:~$ arecord -D plughw:2,0 -f S16_LE -r 48000 -c 2 test.wav
Recording WAVE 'test.wav' : Signed 16 bit Little Endian, Rate 48000 Hz, Stereo
```

-D "plughw:1,0"表示 card 1 , device 0，即我们使用的 USB 声卡，-f S16_LE 代表有符号 16 位小端序，-c 2 表示双声道，test.wav 是录制生成的文件名。按 Ctl+C 结束录音。

播放

```
jetson@linux:~$ aplay -D hw:2,0 test.wav
Playing WAVE 'test.wav' : Signed 16 bit Little Endian, Rate 48000 Hz, Stereo
```

播放刚刚录制的音频

安装依赖库

```
sudo apt update
sudo apt install python3-pip
sudo pip3 install websocket
sudo pip3 install websocket-client
sudo apt-get install sox libsox-fmt-all
sudo apt-get install portaudio19-dev
sudo pip3 install pyaudio
sudo pip3 install webrtcvad
sudo apt-get install libatlas-base-dev
```

```
sudo apt-get install flac
sudo pip3 install SpeechRecognition
```

注意：示例程序中使用的是本人的账号，科大讯飞服务每天有申请次数限制，超过将不再响应。用户可以将申请账户，修改程序 APPID 和 APPKEY 的值。详细教程请参考相关网站。

讯飞开放平台:https://www.xfyun.cn/

AIUI 开放平台:https://aiui.xfyun.cn/

图灵机器人:http://www.turingapi.com/

8.2、在线语音合成

程序位于/whitebear/8.AI 听觉综合项目/tts/tts.py

输入如下程序运行程序

```
cd whitebear/8.AI 听觉综合项目/tts
python3 tts.py
```

运行程序后会将文本转换成 MP3 格式的音频文件并播放。如果要修改语音内容，只需替换 Text 文件内容即可。

```
140 if __name__ == "__main__":
141     # 测试时候在此处正确填写相关信息即可运行
142     wsParam = Ws_Param(APPID='bc5841c1', APISecret='ZTAxOTIzNDEwNjU4NWZlNGFiMGJkMjU4',
143                        APIKey='ebb74228f76daa6ec3a1223a60c7062a',
144                        Text=("这时一款由启东汤森教学仪器有限公司和杭电人工智能研究所联合研发的白熊人工智能实验平台"))
145     websocket.enableTrace(False)
146     wsUrl = wsParam.create_url()
147     ws = websocket.WebSocketApp(wsUrl, on_message=on_message, on_error=on_error, on_close=on_close)
148     ws.on_open = on_open
149     ws.run_forever(sslopt={"cert_reqs": ssl.CERT_NONE})
150
```

图 8.2.1 需要修改的内容

Python 代码：

```
#  本 demo 测试时运行的环境为：Windows + Python3.7
#  本 demo 测试成功运行时所安装的第三方库及其版本如下：
#   cffi==1.12.3
#   gevent==1.4.0
#   greenlet==0.4.15
#   pycparser==2.19
#   six==1.12.0
#   websocket==0.2.1
#   websocket-client==0.56.0
#   合成小语种需要传输小语种文本、使用小语种发音人 vcn、tte=unicode 以及修改文本编码方式
#  错误码链接：https://www.xfyun.cn/document/error-code （code 返回错误码时必看）
# # # #
```

```
import json
from urllib.parse import urlencode
import time
import ssl
from wsgiref.handlers import format_date_time
from datetime import datetime
from time import mktime
import _thread as thread
import os


STATUS_FIRST_FRAME = 0  # 第一帧的标识
STATUS_CONTINUE_FRAME = 1  # 中间帧标识
STATUS_LAST_FRAME = 2  # 最后一帧的标识


class Ws_Param(object):
    # 初始化
    def __init__(self, APPID, APIKey, APISecret, Text):
        self.APPID = APPID
        self.APIKey = APIKey
        self.APISecret = APISecret
        self.Text = Text

        # 公共参数(common)
        self.CommonArgs = {"app_id": self.APPID}
        # 业务参数(business)，更多个性化参数可在官网查看
        self.BusinessArgs = {"aue": "lame", "auf": "audio/L16;rate=16000", "vcn": "xiaoyan", "tte": "utf8"}
        self.Data = {"status": 2, "text": str(base64.b64encode(self.Text.encode('utf-8')), "UTF8")}
        #使用小语种须使用以下方式，此处的 unicode 指的是 utf16 小端的编码方式，即"UTF-16LE""
        #self.Data = {"status": 2, "text": str(base64.b64encode(self.Text.encode('utf-16')), "UTF8")}

    # 生成 url
    def create_url(self):
        url = 'wss://tts-api.xfyun.cn/v2/tts'
        # 生成 RFC1123 格式的时间戳
        now = datetime.now()
        date = format_date_time(mktime(now.timetuple()))
```

```
        # 拼接字符串
        signature_origin = "host: " + "ws-api.xfyun.cn" + "\n"
        signature_origin += "date: " + date + "\n"
        signature_origin += "GET " + "/v2/tts " + "HTTP/1.1"
        # 进行 hmac-sha256 进行加密
        signature_sha = hmac.new(self.APISecret.encode('utf-8'), signature_origin.encode('utf-8'),
                                 digestmod=hashlib.sha256).digest()
        signature_sha = base64.b64encode(signature_sha).decode(encoding='utf-8')

        authorization_origin = "api_key=\"%s\", algorithm=\"%s\", headers=\"%s\", signature=\"%s\"" % (
            self.APIKey, "hmac-sha256", "host date request-line", signature_sha)
        authorization = base64.b64encode(authorization_origin.encode('utf-8')).decode(encoding='utf-
8')
        # 将请求的鉴权参数组合为字典
        v = {
            "authorization": authorization,
            "date": date,
            "host": "ws-api.xfyun.cn"
        }
        # 拼接鉴权参数，生成 url
        url = url + '?' + urlencode(v)
        # print("date: ",date)
        # print("v: ",v)
        # 此处打印出建立连接时候的 url,参考本 demo 的时候可取消上方打印的注释，比对相同参数
时生成的 url 与自己代码生成的 url 是否一致
        # print('websocket url :', url)
        return url


def on_message(ws, message):
    try:
        message =json.loads(message)
        code = message["code"]
        sid = message["sid"]
        audio = message["data"]["audio"]
        audio = base64.b64decode(audio)
        status = message["data"]["status"]
        #print(message)
        if status == 2:
            print("ws is closed")
```

```
                ws.close()
            if code != 0:
                errMsg = message["message"]
                print("sid:%s call error:%s code is:%s" % (sid, errMsg, code))
            else:

                with open('./demo.mp3', 'ab') as f:
                    f.write(audio)

                os.system("play -q ./demo.mp3")

    except Exception as e:
        print("receive msg,but parse exception:", e)



# 收到 websocket 错误的处理
def on_error(ws, error):
    print("### error:", error)


# 收到 websocket 关闭的处理
def on_close(ws):
    print("### closed ###")


# 收到 websocket 连接建立的处理
def on_open(ws):
    def run(*args):
        d = {"common": wsParam.CommonArgs,
             "business": wsParam.BusinessArgs,
             "data": wsParam.Data,
             }
        d = json.dumps(d)
        print("------>开始发送文本数据")
        ws.send(d)
        if os.path.exists('./demo.mp3'):
            os.remove('./demo.mp3')
```

```
    thread.start_new_thread(run, ())



if __name__ == "__main__":
    # 测试时候在此处正确填写相关信息即可运行
    wsParam = Ws_Param(APPID='bc5841c1', APISecret='ZTAxOTIzNDEwNjU4NWZlNGFiMGJkMjU4',
                       APIKey='ebb74228f76daa6ec3a1223a60c7062a',
                       Text=("这时一款由启东汤森教学仪器有限公司和杭电人工智能研究所联合研发的白熊人工智能实验平台"))
    websocket.enableTrace(False)
    wsUrl = wsParam.create_url()
    ws = websocket.WebSocketApp(wsUrl, on_message=on_message, on_error=on_error, on_close=on_close)
    ws.on_open = on_open
    ws.run_forever(sslopt={"cert_reqs": ssl.CERT_NONE})
```

8.3、语音听写流式

程序位于/whitebear/8.AI 听觉综合项目/iat/iat.py

输入如下程序运行程序

```
cd whitebear/8.AI 听觉综合项目/iat
python3 iat.py
```

运行程序后会将语音转换成文字文本输出。

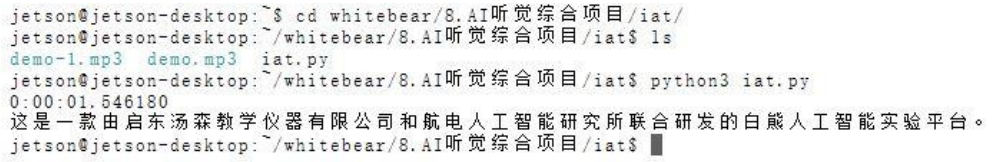


图 8.3.1 运行程序效果

Python 代码：

```
#  本 demo 测试时运行的环境为：Windows + Python3.7
#  本 demo 测试成功运行时所安装的第三方库及其版本如下，您可自行逐一或者复制到一个新的 txt 文件利用 pip 一次性安装：
#   cffi==1.12.3
#   gevent==1.4.0
#   greenlet==0.4.15
#   pycparser==2.19
#   six==1.12.0
#   websocket==0.2.1
#   websocket-client==0.56.0
#
#  语音听写流式 WebAPI 接口调用示例 接口文档（必看）：https://doc.xfyun.cn/rest_api/语音听写（流式版）.html
#  webapi 听写服务参考帖子（必看）：http://bbs.xfyun.cn/forum.php?mod=viewthread&tid=38947&extra=
#  语音听写流式 WebAPI 服务，热词使用方式：登陆开放平台 https://www.xfyun.cn/后，找到控制台--我的应用---语音听写（流式）---服务管理--个性化热词，
#  设置热词
#  注意：热词只能在识别的时候会增加热词的识别权重，需要注意的是增加相应词条的识别率，但并不是绝对的，具体效果以您测试为准。
#  语音听写流式 WebAPI 服务，方言试用方法：登陆开放平台 https://www.xfyun.cn/后，找到控制台--我的应用---语音听写（流式）---服务管理--识别语种列表
#  可添加语种或方言，添加后会显示该方言的参数值
```

```
#  错误码链接：https://www.xfyun.cn/document/error-code （code 返回错误码时必看）
# # # #
```

```
        url = 'wss://ws-api.xfyun.cn/v2/iat'
        # 生成 RFC1123 格式的时间戳
        now = datetime.now()
        date = format_date_time(mktime(now.timetuple()))

        # 拼接字符串
        signature_origin = "host: " + "ws-api.xfyun.cn" + "\n"
        signature_origin += "date: " + date + "\n"
        signature_origin += "GET " + "/v2/iat " + "HTTP/1.1"
        # 进行 hmac-sha256 进行加密
        signature_sha = hmac.new(self.APISecret.encode('utf-8'), signature_origin.encode('utf-8'),
                                 digestmod=hashlib.sha256).digest()
        signature_sha = base64.b64encode(signature_sha).decode(encoding='utf-8')

        authorization_origin = "api_key=\"%s\", algorithm=\"%s\", headers=\"%s\", signature=\"%s\"" % (
            self.APIKey, "hmac-sha256", "host date request-line", signature_sha)
        authorization = base64.b64encode(authorization_origin.encode('utf-8')).decode(encoding='utf-
8')
        # 将请求的鉴权参数组合为字典
        v = {
            "authorization": authorization,
            "date": date,
            "host": "ws-api.xfyun.cn"
        }
        # 拼接鉴权参数，生成 url
        url = url + '?' + urlencode(v)
        # print("date: ",date)
        # print("v: ",v)
        # 此处打印出建立连接时候的 url,参考本 demo 的时候可取消上方打印的注释，比对相同参数
时生成的 url 与自己代码生成的 url 是否一致
        # print('websocket url :', url)
        return url


# 收到 websocket 消息的处理
def on_message(ws, message):
    global text
    try:
        code = json.loads(message)["code"]
```

```
            sid = json.loads(message)["sid"]
            if code != 0:
                errMsg = json.loads(message)["message"]
                print("sid:%s call error:%s code is:%s" % (sid, errMsg, code))

            else:
                data = json.loads(message)["data"]["result"]["ws"]
                # print(json.loads(message))
                result = ""
                for i in data:
                    for w in i["cw"]:
                        result += w["w"]
                #print("sid:%s call success!,data is:%s" % (sid, json.dumps(data, ensure_ascii=False)))
        except Exception as e:
            print("receive msg,but parse exception:", e)
        text += result



# 收到 websocket 错误的处理
def on_error(ws, error):
        print("### error:", error)


# 收到 websocket 关闭的处理
def on_close(ws):
        print("### closed ###")


# 收到 websocket 连接建立的处理
def on_open(ws):
        def run(*args):
            frameSize = 8000  # 每一帧的音频大小
            interval = 0.04  # 发送音频间隔(单位:s)
            status = STATUS_FIRST_FRAME  # 音频的状态信息，标识音频是第一帧，还是中间帧、最后一
帧

            with open(wsParam.AudioFile, "rb") as fp:
                while True:
```

```
                buf = fp.read(frameSize)
                # 文件结束
                if not buf:
                    status = STATUS_LAST_FRAME
                # 第一帧处理
                # 发送第一帧音频，带 business 参数
                # appid 必须带上，只需第一帧发送
                if status == STATUS_FIRST_FRAME:

                    d = {"common": wsParam.CommonArgs,
                         "business": wsParam.BusinessArgs,
                         "data": {"status": 0, "format": "audio/L16;rate=16000",
                                  "audio": str(base64.b64encode(buf), 'utf-8'),
                                  "encoding": "lame"}}
                    d = json.dumps(d)
                    ws.send(d)
                    status = STATUS_CONTINUE_FRAME
                # 中间帧处理
                elif status == STATUS_CONTINUE_FRAME:
                    d = {"data": {"status": 1, "format": "audio/L16;rate=16000",
                                  "audio": str(base64.b64encode(buf), 'utf-8'),
                                  "encoding": "lame"}}
                    ws.send(json.dumps(d))
                # 最后一帧处理
                elif status == STATUS_LAST_FRAME:
                    d = {"data": {"status": 2, "format": "audio/L16;rate=16000",
                                  "audio": str(base64.b64encode(buf), 'utf-8'),
                                  "encoding": "lame"}}
                    ws.send(json.dumps(d))
                    time.sleep(1)
                    break
                # 模拟音频采样间隔
                time.sleep(intervel)
            ws.close()

        thread.start_new_thread(run, ())


if __name__ == "__main__":
```

```
    # 测试时候在此处正确填写相关信息即可运行
    time1 = datetime.now()
    wsParam = Ws_Param(APPID='bc5841c1', APISecret='ZTAxOTIzNDEwNjU4NWZlNGFiMGJkMjU4',
                       APIKey='ebb74228f76daa6ec3a1223a60c7062a',
                       AudioFile=r'demo.mp3')
    websocket.enableTrace(False)
    wsUrl = wsParam.create_url()
    ws = websocket.WebSocketApp(wsUrl, on_message=on_message, on_error=on_error,
on_close=on_close)
    ws.on_open = on_open
    ws.run_forever(sslopt={"cert_reqs": ssl.CERT_NONE})
    time2 = datetime.now()
    print(time2-time1)
    print(text)
```

8.4、图灵机器人

程序位于/whitebear/8.AI 听觉综合项目/turing/ turing.py

或/whitebear/8.AI 听觉综合项目/turing/ turing_tts.py

输入如下程序运行程序

```
cd whitebear/8.AI 听觉综合项目/turing
python3 turing.py
```

运行程序后将显示”我:”,输入对话内容后(例如:你好)，机器人将会回复对话的内容。

如果运行如下程序，机器人的对话的内容将会通过语音播报出来。

```
python3 turing_tts.py
```

```
jetson@jetson-desktop:~$ cd whitebear/8.AI听觉综合项目/turing/
jetson@jetson-desktop:~/whitebear/8.AI听觉综合项目/turing$ ls
demo.mp3  turing.py  turing_tts.py
jetson@jetson-desktop:~/whitebear/8.AI听觉综合项目/turing$ python3 turing.py
我:你好
机器人:我很好，你也要好好的
我:你是谁
机器人:在下江湖人称小微。
我:你是人工智能机器人吗
机器人:你猜呀，有奖励哦。
我:
```

图 8.4.1 图灵机器人程序效果

程序运行过程中可以按‘CTRL’+'C'退出运行。

Python 代码：

```
# -*- coding: utf-8 -*-
import json
import urllib.request
tuling='08c523158a1848f5ae4f2116ae2157a7'
api_url = "http://openapi.tuling123.com/openapi/api/v2"


while True:
    info = input('我:')
    req = {
        "perception":
        {
            "inputText":
            {
                "text": info
```

```
            },
        },
        "userInfo":
        {
            "apiKey": "d91a2724eabd44ff9a3d4261d6ad3c32",
            "userId": "xiaowei"
        }
}
req = json.dumps(req).encode('utf8')
http_post = urllib.request.Request(api_url, data=req, headers={'content-type': 'application/json'})
response = urllib.request.urlopen(http_post)
response_str = response.read().decode('utf8')
response_dic = json.loads(response_str)
results_text = response_dic['results'][0]['values']['text']
print('机器人:' + results_text)
```

8.5、AIUI

程序位于/whitebear/8.AI 听觉综合项目/AIUI/aiui.py

或/whitebear/8.AI 听觉综合项目/aiui/ aiui_tts.py

AIUI 是科大讯飞 2015 年推出的自然语言理解为核心的全链路人机交互语音解决方案，可快速让你的应用和设备能听会说，能理解会思考。简单的说是 AIUI 是科大讯飞的机器人，和图灵机器人的区别是 aiui 是语音输入，输出对话文字，图灵机器输入输出都是文字。

输入如下程序运行程序

```
cd whitebear/8.AI 听觉综合项目/AIUI
python3 aiui.py
```

程序会将 record.wav 文件的语音作为语音输入，运行程序将显示语音对话内容。

如果要替换语音运行如下命令重新录音。

```
arecord -r 16000 -c 1 -t wav -f S16_LE test.wav
```

如果运行如下程序，机器人的对话的内容将会通过语音播报出来

```
python3 aiui_tts.py
```

```
jetson@jetson-desktop:~$ cd whitebear/8.AI听觉综合项目/AIUI/
jetson@jetson-desktop:~/whitebear/8.AI听觉综合项目/AIUI$ python3 aiui.py
我 :喂，你好，你是谁？
机器人 :小朋友你好，我是你的新朋友图灵机器人。
jetson@jetson-desktop:~/whitebear/8.AI听觉综合项目/AIUI$
```

图 8.5.1 AIUI 实验效果图

Python 代码：

```
#-*- coding: utf-8 -*-
import requests
import time
import hashlib
import base64
import re
import struct


URL = "http://openapi.xfyun.cn/v2/aiui"
APPID = "5beaacac"
```

```
API_KEY = "4227d60d150c455db467b623d7ae183d"
AUE = "raw"
AUTH_ID = "2894c985bf8b1111c6728db79d3479ae"
DATA_TYPE = "audio"
SAMPLE_RATE = "16000"
SCENE = "main"
RESULT_LEVEL = "complete"
LAT = "39.938838"
LNG = "116.368624"
#个性化参数，需转义
PERS_PARAM = "{\\\"auth_id\\\":\\\"2894c985bf8b1111c6728db79d3479ae\\\"}"
FILE_PATH = "record.wav"



def buildHeader():
    curTime = str(int(time.time()))
    param =
"{\"result_level\":\""+RESULT_LEVEL+"\",\"auth_id\":\""+AUTH_ID+"\",\"data_type\":\""+DATA_TYPE+"\",\"
sample_rate\":\""+SAMPLE_RATE+"\",\"scene\":\""+SCENE+"\",\"lat\":\""+LAT+"\",\"lng\":\""+LNG+"\"}"
    #使用个性化参数时参数格式如下：
    #param =
"{\"result_level\":\""+RESULT_LEVEL+"\",\"auth_id\":\""+AUTH_ID+"\",\"data_type\":\""+DATA_TYPE+"\",\"
sample_rate\":\""+SAMPLE_RATE+"\",\"scene\":\""+SCENE+"\",\"lat\":\""+LAT+"\",\"lng\":\""+LNG+"\",\"p
ers_param\":\""+PERS_PARAM+"\"}"
    paramBase64 = base64.b64encode(param.encode('utf-8'))

    m2 = hashlib.md5()
    m2.update((API_KEY + curTime + str(paramBase64, 'utf-8')).encode('utf-8'))
    checkSum = m2.hexdigest()

    header = {
        'X-CurTime': curTime,
        'X-Param': paramBase64,
        'X-Appid': APPID,
        'X-CheckSum': checkSum,
    }
    return header

def readFile(filePath):
```

```
        binfile = open(filePath, 'rb')
        data = binfile.read()
        return data


r = requests.post(URL, headers=buildHeader(), data=readFile(FILE_PATH))
#print(r.content)
dic_json = r.json()
for i in dic_json['data']:
        if(i['sub'] == "nlp" and i['intent'] != {}):
                try:
                        text = i['intent']['answer']['text']
                except:
                        text = "不明白你再说什么，你还是说中文吧"
                print('我 :'+i['intent']['text'])
                print('机器人 :'+text)
                break
else:
        print('没有检测到人声'.decode('UTF-8'))
```

8.6、VAD 语音活动检测

程序位于/whitebear/8.AI 听觉综合项目/VAD/vad.py

语音活动检测(Voice Activity Detection,VAD)又称语音端点检测,语音边界检测。简单的说 VAD 就是检测音频流中的有声音部分，消除拟制静音部分。即通过 VAD 录音只录制人说话的部分，其他静音期不录音。

输入如下程序运行程序

```
cd whitebear/8.AI 听觉综合项目/VAD
python3 vad.py
```

程序运行将显示”* recording:”,表示程序在录音检测状态，此时说话将会被检测到并录音。直到显示”* done recording”完成录音并播放录音内容。

Python 代码：

```
'''
Requirements:
+ pyaudio - `pip3 install pyaudio`
+ py-webrtcvad - `pip3 install webrtcvad`
'''
import webrtcvad
import collections
import sys
import signal
import pyaudio

from array import array
from struct import pack
import wave
import time
import os

FORMAT = pyaudio.paInt16
CHANNELS = 1
RATE = 16000
CHUNK_DURATION_MS = 30          # supports 10, 20 and 30 (ms)
PADDING_DURATION_MS = 1500      # 1 sec jugement
```

```
CHUNK_SIZE = int(RATE * CHUNK_DURATION_MS / 1000)    # chunk to read
CHUNK_BYTES = CHUNK_SIZE * 2    # 16bit = 2 bytes, PCM
NUM_PADDING_CHUNKS = int(PADDING_DURATION_MS / CHUNK_DURATION_MS)
NUM_WINDOW_CHUNKS = int(400 / CHUNK_DURATION_MS)    # 400 ms/ 30ms    ge
NUM_WINDOW_CHUNKS_END = NUM_WINDOW_CHUNKS * 2

START_OFFSET = int(NUM_WINDOW_CHUNKS * CHUNK_DURATION_MS * 0.5 * RATE)


def record_to_file(path, data, sample_width):
    "Records from the microphone and outputs the resulting data to 'path'"
    # sample_width, data = record()
    data = pack('<' + ('h' * len(data)), *data)
    wf = wave.open(path, 'wb')
    wf.setnchannels(1)
    wf.setsampwidth(sample_width)
    wf.setframerate(RATE)
    wf.writeframes(data)
    wf.close()


def normalize(snd_data):
    "Average the volume out"
    MAXIMUM = 32767    # 16384
    times = float(MAXIMUM) / max(abs(i) for i in snd_data)
    r = array('h')
    for i in snd_data:
        r.append(int(i * times))
    return r


if __name__ == '__main__':

    while True:
        vad = webrtcvad.Vad(2)

        pa = pyaudio.PyAudio()
        stream = pa.open(format=FORMAT,
                         channels=CHANNELS,
```

```
                    rate=RATE,
                    input=True,
                    start=False,
                    # input_device_index=2,
                    frames_per_buffer=CHUNK_SIZE)


got_a_sentence = False
leave = False

while not leave:
    ring_buffer = collections.deque(maxlen=NUM_PADDING_CHUNKS)
    triggered = False
    voiced_frames = []
    ring_buffer_flags = [0] * NUM_WINDOW_CHUNKS
    ring_buffer_index = 0

    ring_buffer_flags_end = [0] * NUM_WINDOW_CHUNKS_END
    ring_buffer_index_end = 0
    buffer_in = ''
    # WangS
    raw_data = array('h')
    index = 0
    start_point = 0
    StartTime = time.time()
    print("* recording: ")
    stream.start_stream()

    while not got_a_sentence and not leave:
        chunk = stream.read(CHUNK_SIZE,exception_on_overflow = False)
        # add WangS
        raw_data.extend(array('h', chunk))
        index += CHUNK_SIZE
        active = vad.is_speech(chunk, RATE)

        #sys.stdout.write('1' if active else '_')
        ring_buffer_flags[ring_buffer_index] = 1 if active else 0
        ring_buffer_index += 1
```

```
                    ring_buffer_index %= NUM_WINDOW_CHUNKS

                    ring_buffer_flags_end[ring_buffer_index_end] = 1 if active else 0
                    ring_buffer_index_end += 1
                    ring_buffer_index_end %= NUM_WINDOW_CHUNKS_END

                    # start point detection
                    if not triggered:
                        ring_buffer.append(chunk)
                        num_voiced = sum(ring_buffer_flags)
                        if num_voiced > 0.8 * NUM_WINDOW_CHUNKS:
                            sys.stdout.write(' Open ')
                            StartTime = time.time()
                            triggered = True
                            start_point = index - CHUNK_SIZE * 20  # start point
                            # voiced_frames.extend(ring_buffer)
                            ring_buffer.clear()
                    # end point detection
                    else:
                        # voiced_frames.append(chunk)
                        ring_buffer.append(chunk)
                        num_unvoiced = NUM_WINDOW_CHUNKS_END - sum(ring_buffer_flags_end)
                        if num_unvoiced > 0.90 * NUM_WINDOW_CHUNKS_END or (time.time() -
StartTime) > 10:
                            sys.stdout.write(' Close ')
                            triggered = False
                            got_a_sentence = True

                    sys.stdout.flush()

                sys.stdout.write('\n')
                # data = b''.join(voiced_frames)

                stream.stop_stream()
                print("* done recording")
                got_a_sentence = False

                # write to file
                raw_data.reverse()
```

```
            for index in range(start_point):
                raw_data.pop()
            raw_data.reverse()
            raw_data = normalize(raw_data)
            record_to_file("record.wav", raw_data, 2)
            leave = True

    stream.close()
    os.system("play record.wav")
```

8.7、小薇机器人语音对话

程序位于/whitebear/8.AI 听觉综合项目/xiaowei/xiaowei.py

输入如下程序运行程序

```
cd whitebear/8.AI 听觉综合项目/xiaowei
python3 xiaowei.py
```

运行程序后将进入对话状态，例如问启东今天天气怎样？，小微机器人将会通过语音回答启东今天的天气。

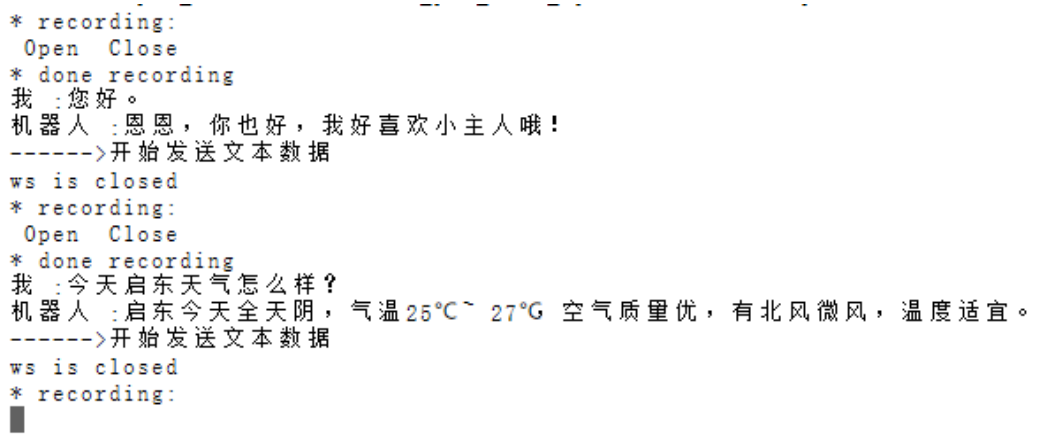


图 8.7.1 小薇程序效果

程序运行过程中可以按‘CTRL’+'C'退出运行。

Python 代码：

```
'''
Requirements:
+ pyaudio - `pip3 install pyaudio`
+ py-webrtcvad - `pip3 install webrtcvad`
'''
import webrtcvad
import collections
import sys
import signal
import pyaudio

from array import array
from struct import pack
import wave
```

```
import time
import os
import aiui



FORMAT = pyaudio.paInt16
CHANNELS = 1
RATE = 16000
CHUNK_DURATION_MS = 30           # supports 10, 20 and 30 (ms)
PADDING_DURATION_MS = 1500     # 1 sec jugement
CHUNK_SIZE = int(RATE * CHUNK_DURATION_MS / 1000)   # chunk to read
CHUNK_BYTES = CHUNK_SIZE * 2   # 16bit = 2 bytes, PCM
NUM_PADDING_CHUNKS = int(PADDING_DURATION_MS / CHUNK_DURATION_MS)
NUM_WINDOW_CHUNKS = int(400 / CHUNK_DURATION_MS)   # 400 ms/ 30ms   ge
NUM_WINDOW_CHUNKS_END = NUM_WINDOW_CHUNKS * 2

START_OFFSET = int(NUM_WINDOW_CHUNKS * CHUNK_DURATION_MS * 0.5 * RATE)



def record_to_file(path, data, sample_width):
    "Records from the microphone and outputs the resulting data to 'path'"
    # sample_width, data = record()
    data = pack('<' + ('h' * len(data)), *data)
    wf = wave.open(path, 'wb')
    wf.setnchannels(1)
    wf.setsampwidth(sample_width)
    wf.setframerate(RATE)
    wf.writeframes(data)
    wf.close()

def normalize(snd_data):
    "Average the volume out"
    MAXIMUM = 32767   # 16384
    times = float(MAXIMUM) / max(abs(i) for i in snd_data)
    r = array('h')
    for i in snd_data:
        r.append(int(i * times))
    return r
```

```
if __name__ == '__main__':

    while True:
        vad = webrtcvad.Vad(2)

        pa = pyaudio.PyAudio()
        stream = pa.open(format=FORMAT,
                         channels=CHANNELS,
                         rate=RATE,
                         input=True,
                         start=False,
                         # input_device_index=2,
                         frames_per_buffer=CHUNK_SIZE)

        got_a_sentence = False
        leave = False

        while not leave:
            ring_buffer = collections.deque(maxlen=NUM_PADDING_CHUNKS)
            triggered = False
            voiced_frames = []
            ring_buffer_flags = [0] * NUM_WINDOW_CHUNKS
            ring_buffer_index = 0

            ring_buffer_flags_end = [0] * NUM_WINDOW_CHUNKS_END
            ring_buffer_index_end = 0
            buffer_in = ''
            # WangS
            raw_data = array('h')
            index = 0
            start_point = 0
            StartTime = time.time()
            print("* recording: ")
            stream.start_stream()

            while not got_a_sentence and not leave:
                chunk = stream.read(CHUNK_SIZE,exception_on_overflow = False)
                # add WangS
                raw_data.extend(array('h', chunk))
```

```
                    index += CHUNK_SIZE
                    active = vad.is_speech(chunk, RATE)

                    #sys.stdout.write('1' if active else '_')
                    ring_buffer_flags[ring_buffer_index] = 1 if active else 0
                    ring_buffer_index += 1

                    ring_buffer_index %= NUM_WINDOW_CHUNKS

                    ring_buffer_flags_end[ring_buffer_index_end] = 1 if active else 0
                    ring_buffer_index_end += 1
                    ring_buffer_index_end %= NUM_WINDOW_CHUNKS_END

                    # start point detection
                    if not triggered:
                        ring_buffer.append(chunk)
                        num_voiced = sum(ring_buffer_flags)
                        if num_voiced > 0.8 * NUM_WINDOW_CHUNKS:
                            sys.stdout.write(' Open ')
                            StartTime = time.time()
                            triggered = True
                            start_point = index - CHUNK_SIZE * 20  # start point
                            # voiced_frames.extend(ring_buffer)
                            ring_buffer.clear()
                    # end point detection
                    else:
                        # voiced_frames.append(chunk)
                        ring_buffer.append(chunk)
                        num_unvoiced = NUM_WINDOW_CHUNKS_END - sum(ring_buffer_flags_end)
                        if num_unvoiced > 0.90 * NUM_WINDOW_CHUNKS_END or (time.time() -
StartTime) > 10:
                            sys.stdout.write(' Close ')
                            triggered = False
                            got_a_sentence = True
                    sys.stdout.flush()

            sys.stdout.write('\n')
            # data = b''.join(voiced_frames)
            stream.stop_stream()
```

```
            print("* done recording")
            got_a_sentence = False

            # write to file
            raw_data.reverse()
            for index in range(start_point):
                raw_data.pop()
            raw_data.reverse()
            raw_data = normalize(raw_data)
            record_to_file("record.wav", raw_data, 2)
            leave = True
    stream.close()
    #os.system("play record.wav")
    aiui.main()
```

8.8、Snowboy 语音唤醒

程序位于/whitebear/8.AI 听觉综合项目/snowboy/snowboy.py

输入如下程序运行程序

```
cd whitebear/8.AI 听觉综合项目/snowboy
python3 demo.py hotword.pmdl
```

运行程序后将显示"Listening..."，表示程序在待机检测状态,说唤醒词唤醒机器人并对话。例如"魔镜魔镜，今天是几号"。机器人识别到唤醒词后会叮的响一声。

注意：示例程序中 hotword.pmdl 文件为我的个人声音唤醒模型，你使用此模型将不能唤醒.你需要训练你自己的模型然后替换示例程序中的模型文件。

目前官网进不去，需更改实验

9、ROS基础和运动学

9.1、ROS 基础课程

9.1.1、ROS 简介

ROS wiki: http://wiki.ros.org/

ROS 的设计目标主要是便于机器人研发过程中的代码复用,因此 ROS 采用了一种分布式架构,各个软件可以各自独立的设计,松散即时的组合起来.

ROS 的主要特点:分布式架构(每一个工作进程都看作一个节点,使用节点管理器统一管理),多语言支持(如 C++,Python 等),良好的伸缩性(既可以写一个节点,也可以通过 roslaunch 将很多节点组织成一个更大的工程),源码开放(ROS 遵循 BSD 协议,对个和商业应用及修改完全免费).

- ROS 的整体架构 ： 开源社区级,文件系统级,计算图级.
- 开源社区级:分布式管理的 ROS 资源.包括一些发行版,软件源,ROS wiki,邮件列表,博客等.
- 文件系统级:程序文件的组织和构建.主要包括功能包,消息,服务,代码等.
- 计算图级:描述程序是如何运行的.主要包括节点,消息,话题,服务,节点管理器.

9.1.2、ROS 小海龟介绍

master 中文名字叫节点管理器，master 是用来管理系统中的很多进程的，每个 node 启动时都要向 master 注册管理 node 之间的通信。master 启动后再去通过 master 注册每一个 node 节点。

启动 ROS master

```
roscore
```

启动完 master，我们就可以启动 node，一个 node 就是一个我们所说的进程，ros 里面有个专门的名字叫 node。当我们运行这些可执行文件，就会加载到进程中。一个节点专门负责一个进程。node 的启动，rosrun+包名+节点名

```
rosrun [--prefix cmd] [--debug] pkg_name node_name [ARGS]
```

rosrun 将会寻找 package 的名为 executable 的可执行程序，将可选参数 ARGS 传入。

启动小海龟仿真器节点

```
rosrun turtlesim turtlesim_node
```

启动小海龟键盘控制器节点

```
rosrun turtlesim turtle_teleop_key
```

启动完成后，可以通过键盘输入操控小海龟移动,键盘操控时，光标一定要在【rosrun turtlesim turtle_teleop_key】这个命令行下,点击键盘上下左右控制小海龟移动。

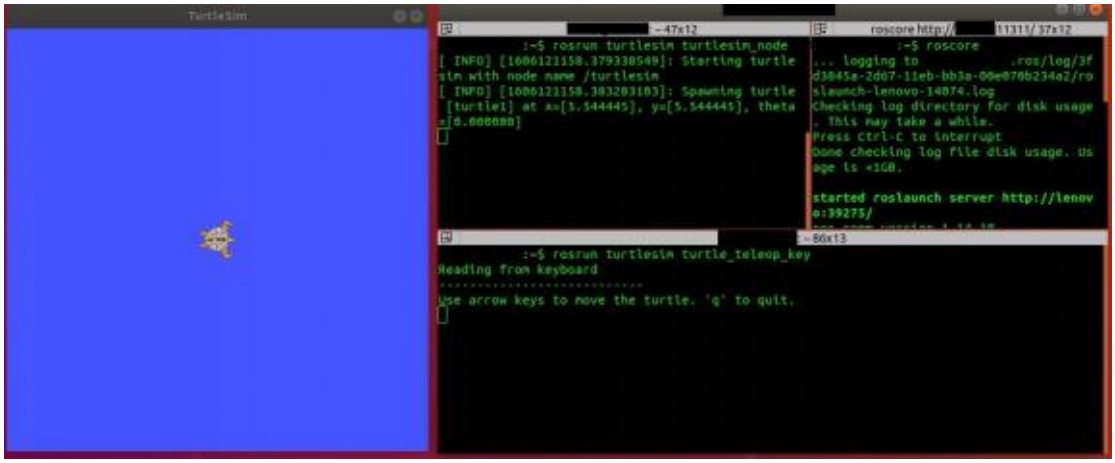


图 9.1.1 小海龟移动

并且在【rosrun turtlesim turtlesim_node】终端会打印一些小海龟的日志信息。

```
[ INFO] [1607648666.226328691]: Starting turtlesim with node name /turtlesim
[ INFO] [1607648666.229275030]: Spawning turtle [turtle1] at x=[5.544445], y=[5.544445],
theta=[0.000000]
```

9.1.3、【rosnode】节点信息

当我们在命令行里输入【rosnode】,然后双击 Tab 键,会发现命令行下方出现这样几个单词.

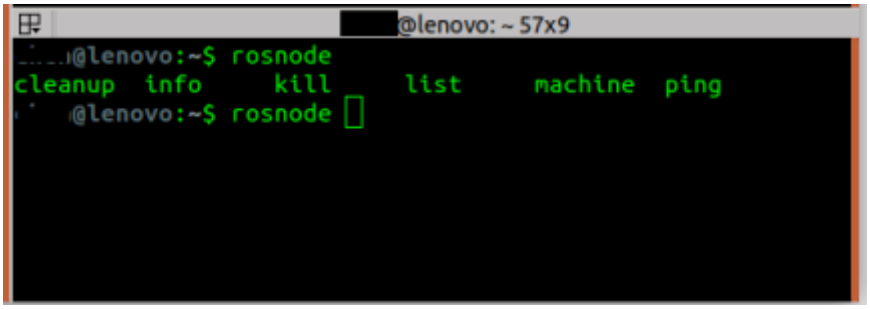


图 9.1.2 命令

【rosnode】命令的详细作用列表如下：

rosnode 命令	作用
rosnode list	查询当前运行的所有节点
rosnode info node_name	显示该节点的详细信息
rosnode kill node_name	结束某个节点
rosnode ping	测试该节点是否存活
rosnode machine	列出在特定机器或列表机器上运行的节点
rosnode cleanup	清除不可运行节点的注册信息

在开发调试时经常会需要查看当前 node 以及 node 信息，所以请记住这些常用命令。如果想不起来，也可以通过【rosnode help】来查看【rosnode】命令的用法。

查询当前运行的所有节点

```
rosnode list
```

小海龟启动过程中，我们启动了两个可执行的程序：【turtlesim_node】和【turtle_teleop_key】。可以通过命令查看当前的启动的节点,在一个新的命令行窗口中输入以上命令后,我们会看到

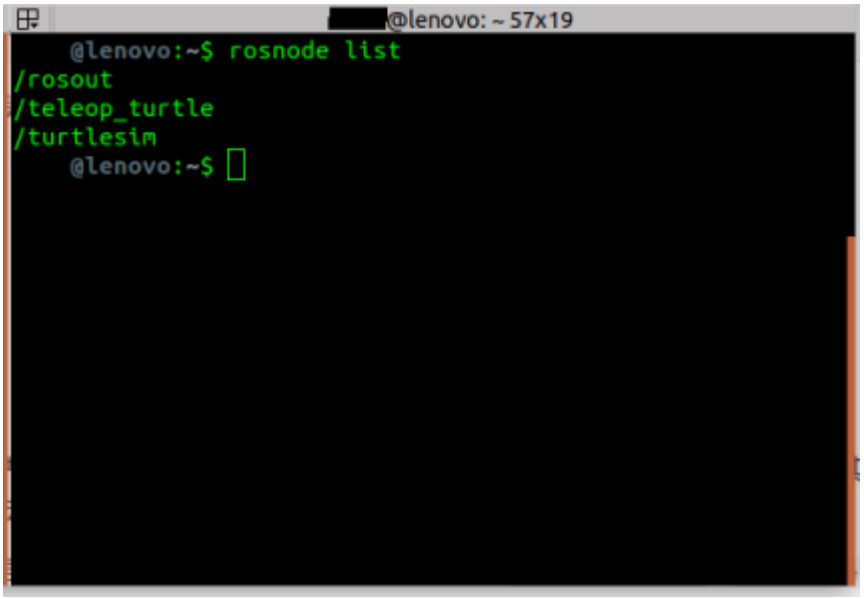


图 9.1.3

我们首先就会发现有一个 rosout 节点，不管运行谁的 ros 程序都会有这样一个节点,其实 rosout 是用来收集和记录节点调试输出信息的，所以它总是运行,大可不必管它,随它去吧。

接下来一一查看节点【/turtlesim】和【/teleop_turtle】的详细信息

查看【/turtlesim】节点

rosnode info /turtlesim

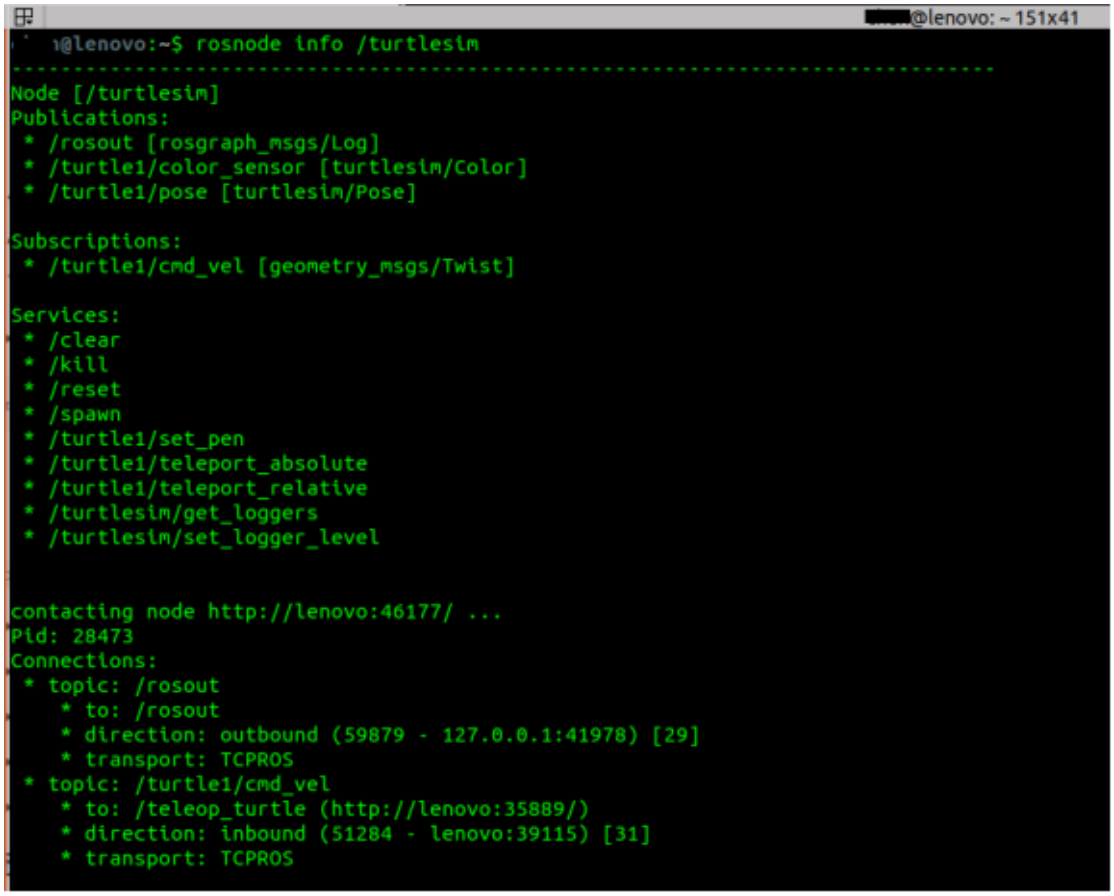


图 9.1.4 turtlesim 节点

简单解释一下当前节点的一些信息:

- Publications：此节点上定义的发布者
- Subscriptions：此节点上定义的订阅者
- Services：此节点上定义的服务
- 进程 id，占用的网络端口
- Connections： 此节点和其他节点间的连接信息

查看【/teleop_turtle】节点

```
rosnode info /teleop_turtle
```

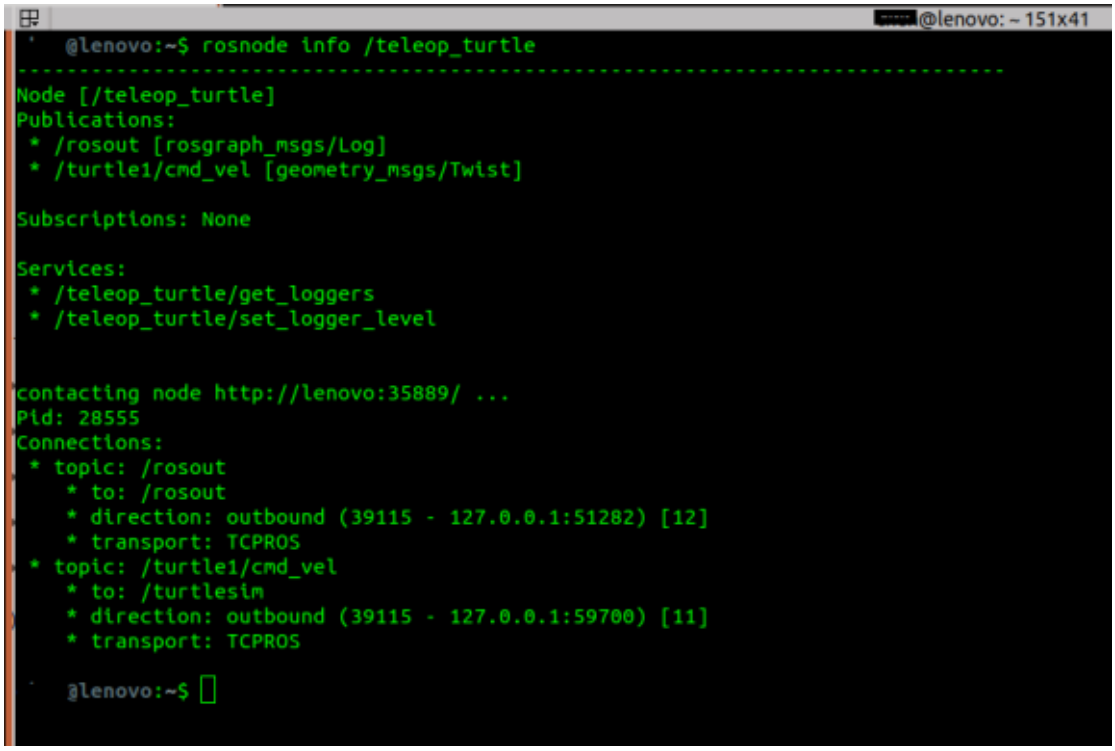


图 9.1.5 teleop_turtle】节点

我们来分析一下节点【/turtlesim】和节点【/teleop_turtle】的通讯关系：

- 在【/turtlesim】节点 Subscribe 中有一个【/turtle1/cmd_vel】话题,该节点在一直的订阅【/turtle1/cmd_vel】话题.

- 在【/teleop_turtle】节点 Publication 中也有一个【/turtle1/cmd_vel】话题，该节点在一直的发布【/turtle1/cmd_vel】话题.

它们之间的通讯由此可以看得出，【/teleop_turtle】节点把信息发布到【/turtle1/cmd_vel】话题中,【/turtlesim】节点不断地从【/turtle1/cmd_vel】话题中获取信息,实现通信的目的。

9.1.4、ROS-rqt(QT 工具)

打开命令行窗口输入【rosrun rqt】然后双击【Tab】键,便可查看 ROS 中 QT 工具包含的内容,如下图所示:

图 9.1.6 查看 ROS 中 QT 工具

接下来我们以小海龟为例简单的介绍一下常用的几种 QT 工具：

1. 【rqt_graph】计算图可视化工具

打开命令行窗口输入以下命令,弹出一个对话窗。

```
rosrun rqt_graph rqt_graph
```

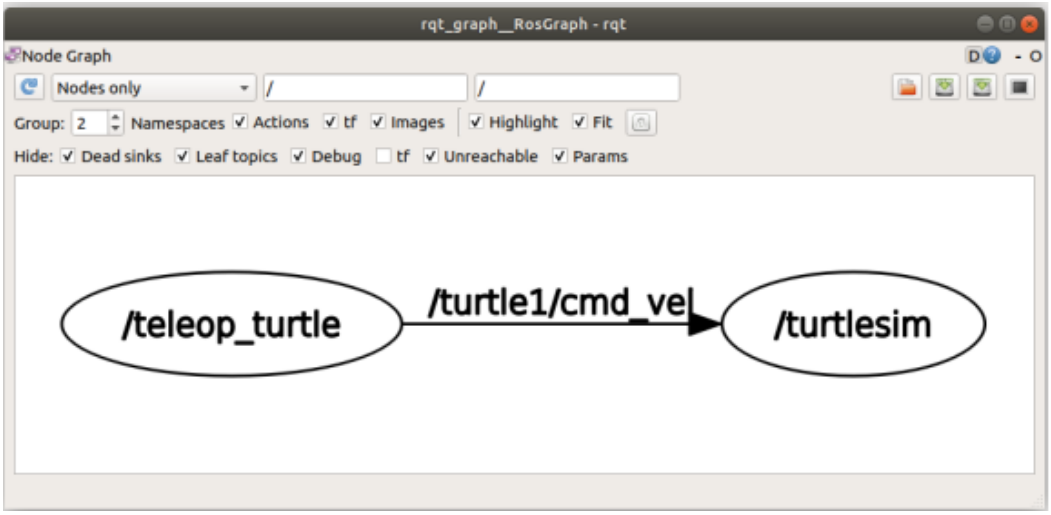


图 9.1.7 rqt_graph

从图像中我们可以很清晰的看得出【/teleop_turtle】节点通过【/turtle1/cmd_vel】话题给【/turtlesim】节点进行数据传递。

- 【/teleop_turtle】为具备 Publisher(发布)功能的节点。
- 【/turtlesim】为具备 Subscriber(订阅)功能的节点。
- 【/turtle1/cmd_vel】为 publisher 和 subscriber 通讯的话题。

2. 【rqt_topic】查看话题工具

```
rosrun rqt_topic rqt_topic
```

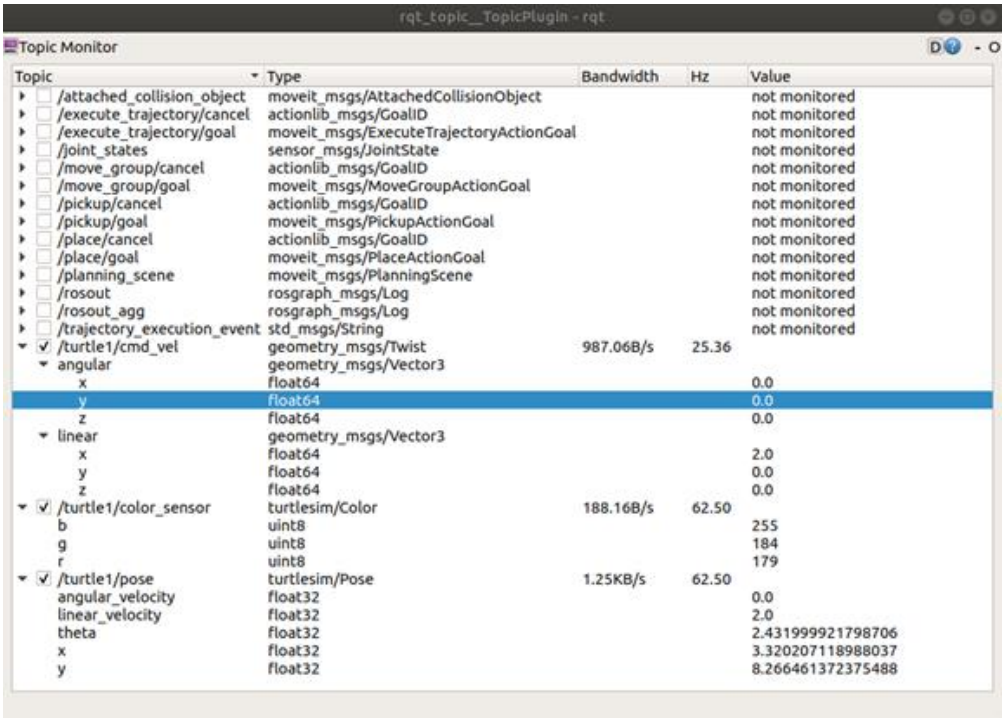


图 9.1.8 查看话题工具

通过该工具,我们可以清楚地看到小海龟的一些实时变化的信息.

3. 【rqt_publisher】工具

【rqt_publisher】提供了一个 GUI 插件，用于发布具有固定或计算字段值的任意消息。

打开命令行窗口输入以下命令,弹出一个对话窗。

```
rosrun rqt_publisher rqt_publisher
```

点击【Topic】右边的选择框找到我们需要的【/turtle1/cmd_vel】话题,点击右侧加号添加,显示如下：

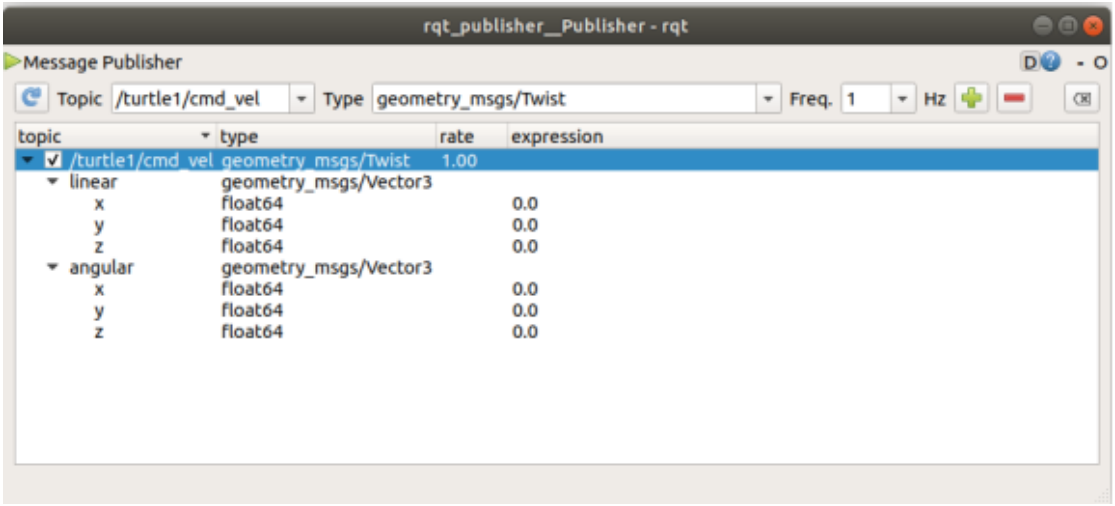


图 9.1.9 rqt_publisher

4. 【rqt_plot】数据绘图工具

```
rosrun rqt_plot rqt_plot
```

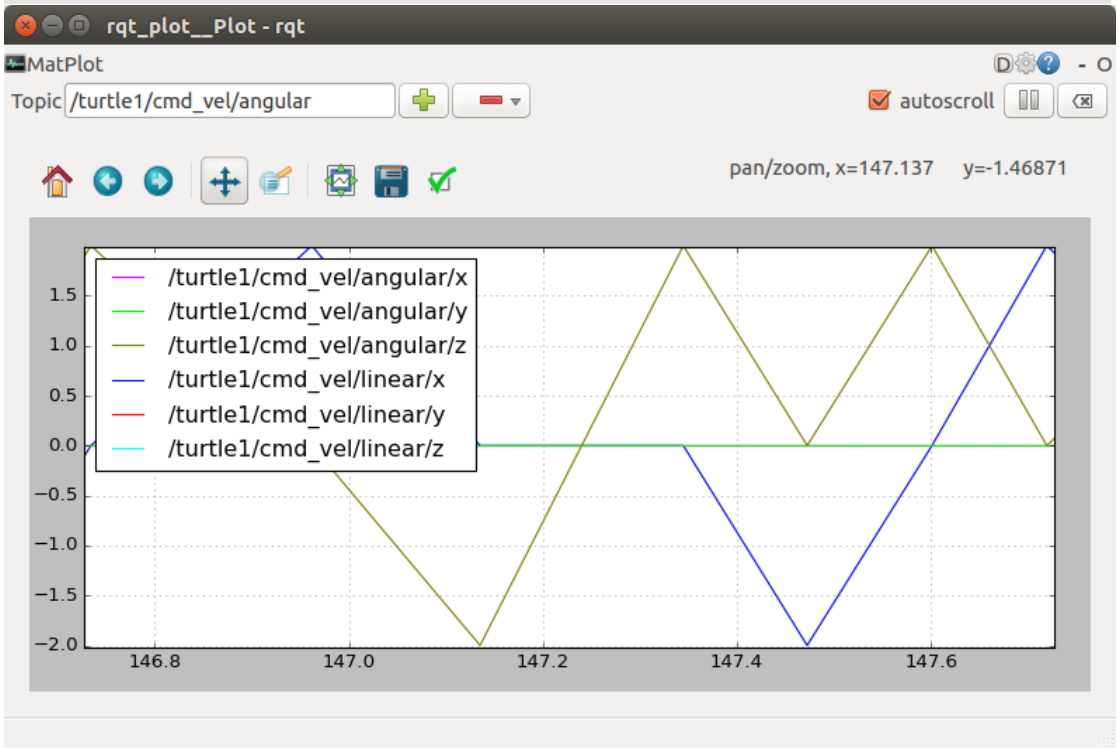


图 9.1.10 rqt_plot

5. 【rqt_console】日志输出工具

```
rosrun rqt_console rqt_console
```

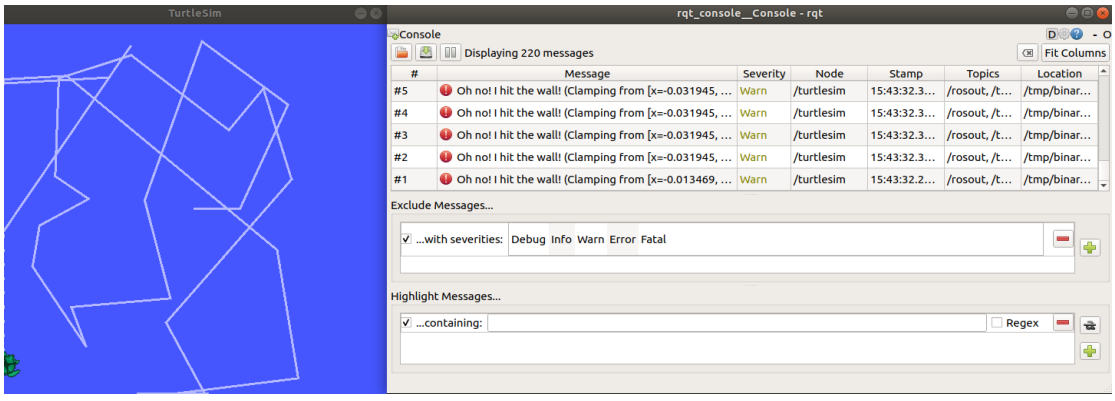


图 9.1.11 rqt_console

日志输出工具属于 ROS 日志框架(logging framework)的一部分，用来显示节点的输出信息.从图上我们可以看出它再提示小海龟已经撞墙啦。

9.1.5、【rostopic】介绍

【rostopic】命令工具能让你获取有关 ROS 话题的信息。

```
rostopic -h
```

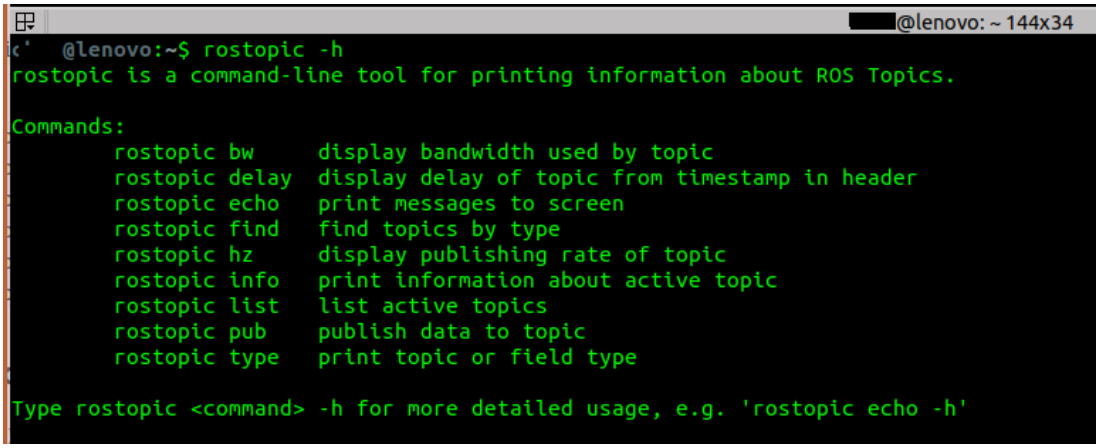


图 9.1.12 rostopic

1. 【rostopic echo】可以显示在某个话题上发布的数据

```
rostopic  echo  /turtle1/cmd_vel
```

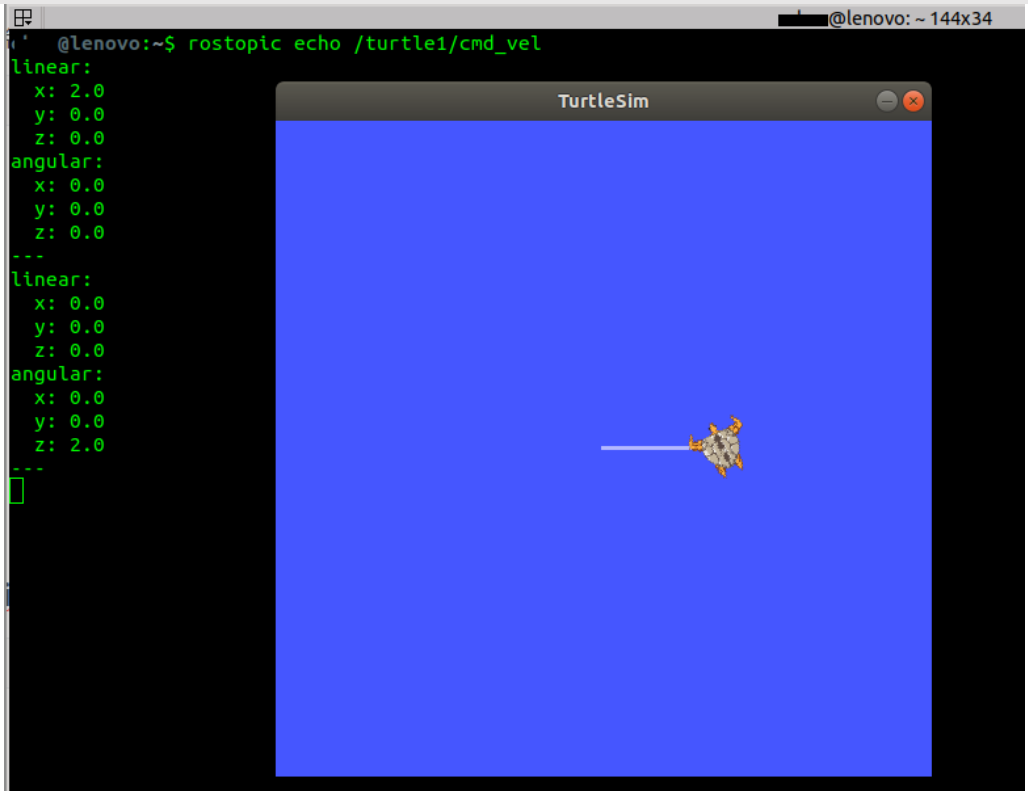


图 9.1.13 rostopic echo

2. 【rostopic type】查看某个话题的类型

```
rostopic type /turtle1/cmd_vel
```

我们通过 rostopic 命令可以获得小海龟的移动数据类型为【geometry_msgs/Twist】。

3. 【rostopic pub】往某个话题中发布数据

```
rostopic pub -r 3 /turtle1/cmd_vel   geometry_msgs/Twist "linear:
  x: 2.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 2.0"
```

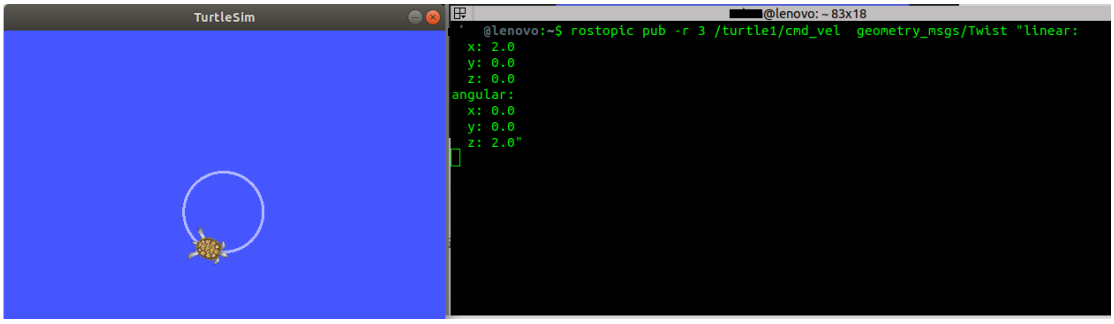


图 9.1.14 rostopic pub

r 代表 rate，后面跟的数字代表一秒发布话题的次数.最后的线速度和角速度双击【Tab】键即可，然后修改数值。

9.1.6、【rostopic list】列出所有当前订阅和发布的话题

让我们查看一下 list 子命令需要的参数，在一个新终端中运行：

```
rostopic list -h
```

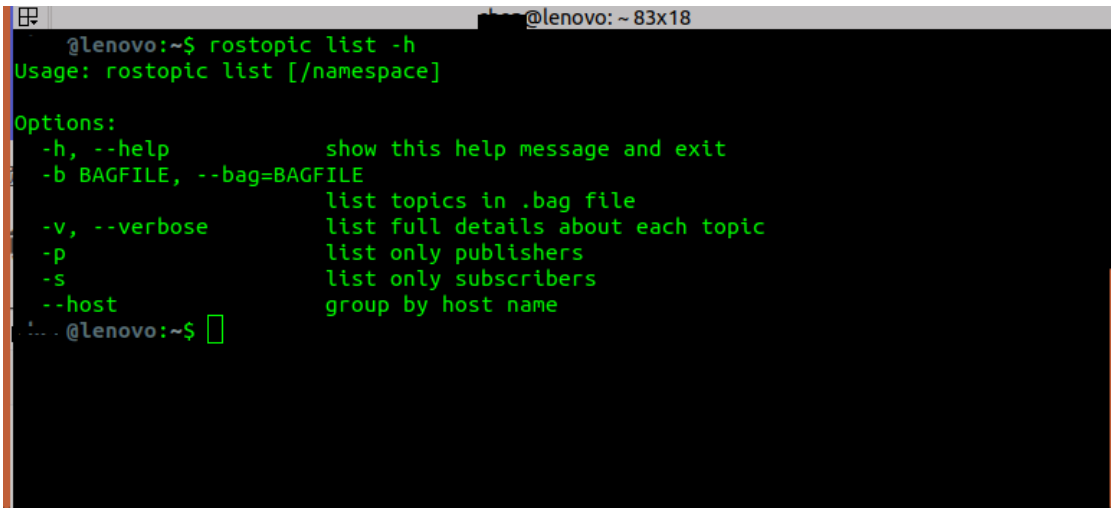


图 9.1.15 rostopic list

在【rostopic list】中使用-v 选项显示出有关所发布和订阅的话题及其类型的详细信息

```
rostopic list -v
```

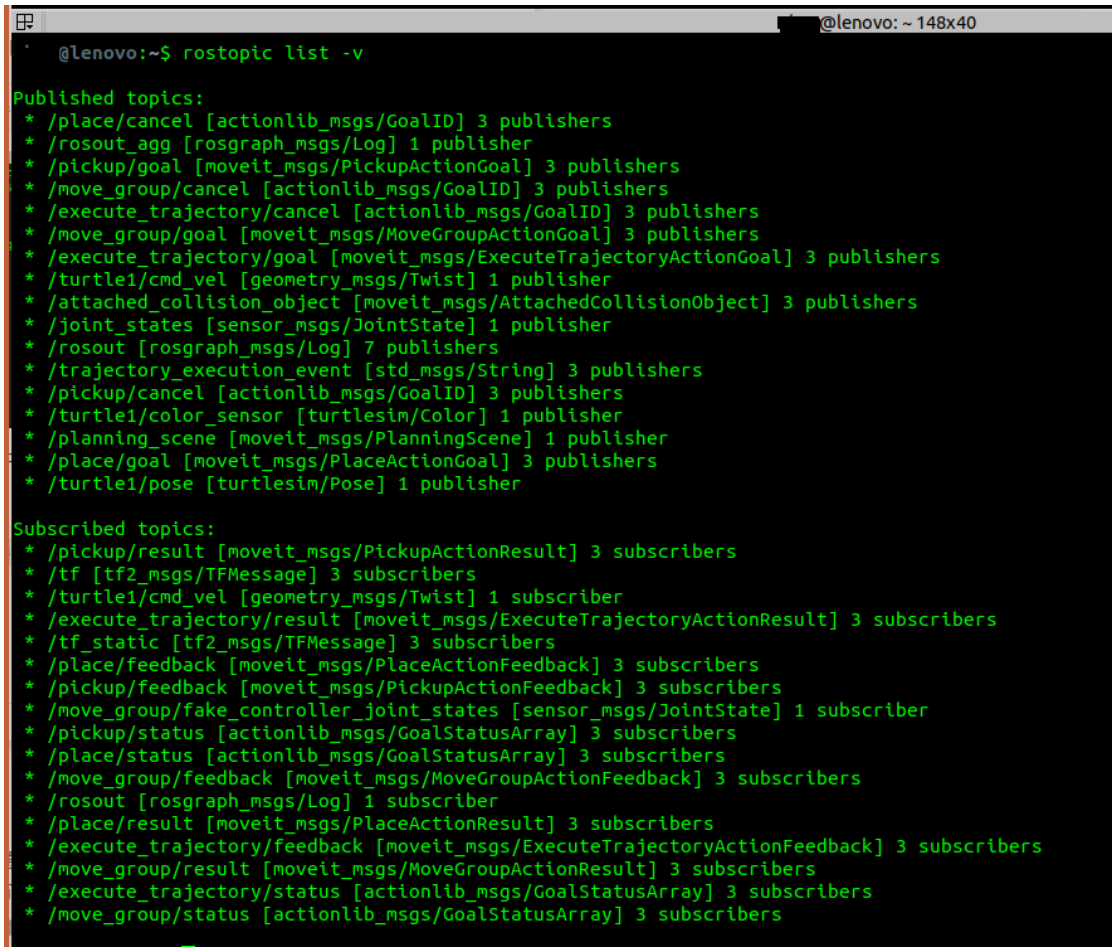


图 9.1.16 详细信息

9.1.7、rosmsg 消息介绍

首先我们先来查看一下消息都包含那些信息

```
rosmsg -h
```

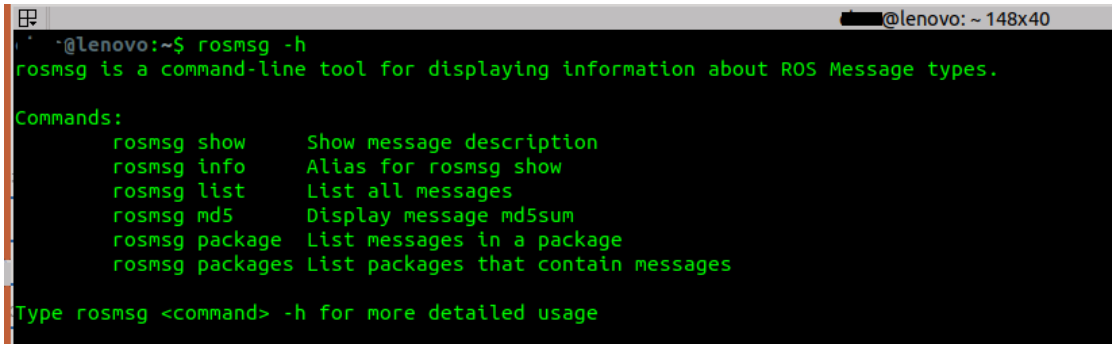


图 9.1.17 rosmsg 消息

查看当前开启的消息

rosmsg list

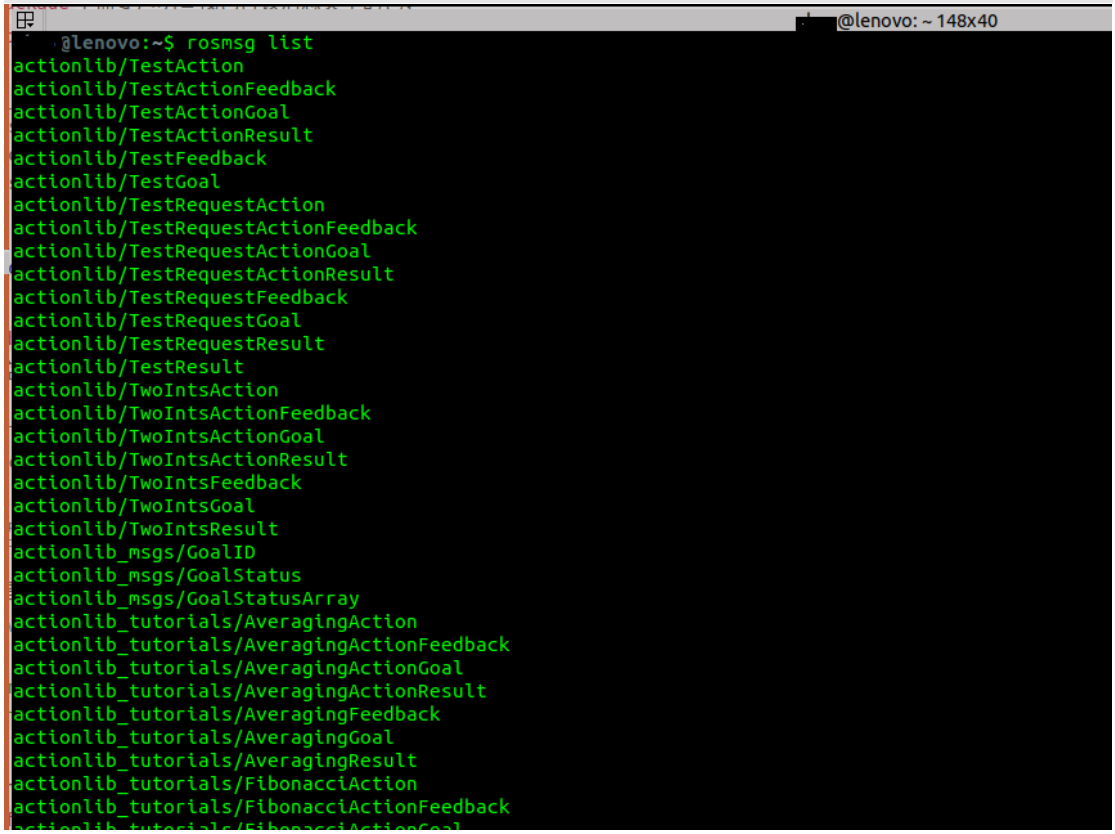


图 9.1.18 当前开启的消息

查看该消息包含的信息

rosmsg info turtlesim/Pose

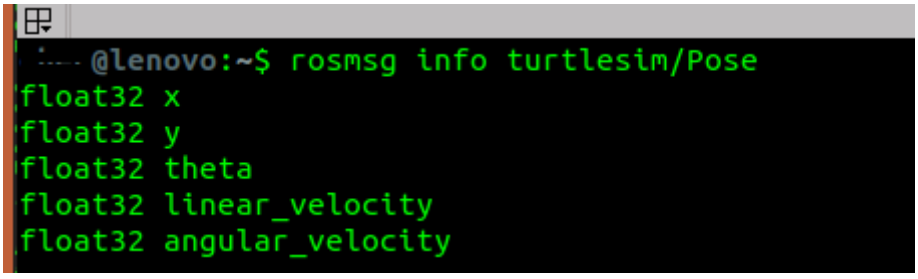


图 9.1.20 包含的信息

自定义消息的数据类型与 python,C++有一定的差别

msg类型	C++对应类型	Python对应类型
bool	uint8_t	bool
int8	int8_t	int
int16	int16_t	int
int32	int32_t	int
int64	int64_t	int , long
uint8	uint8_t	int
uint16	uint16_t	int
uint32	uint32_t	int
uint64	uint64_t	int , long
float32	float	float
float64	float	float
string	std:string	str , bytes
time	ros:Time	rospy.Time
duration	ros::Duration	rospy.Duration

图 9.1.21 数据类型区别

9.2、ROS 创建工程项目

9.2.1、创建工作空间

工作空间，直观的形容就是一个仓库，里面装载着 ROS 的各种项目工程，便于系统组织管理调用。在可视化图形界面里是一个文件夹。我们自己写的 ROS 代码通常就放在工作空间中。

接下来,我们创建一个 dofbot_ws 工作空间,作为我们机械臂工程的一个仓库。

```
mkdir -p ~/dofbot_ws/src  # 创建
cd dofbot_ws/ # 进入工作空间
catkin_make # 编译
```

完成上述步骤,你会看到 dofbot_ws/目录下有三个文件夹 devel/;build/;src/。

- src/: ROS 的 catkin 软件包（源代码包）
- build/: catkin（CMake）的缓存信息和中间文件
- devel/: 生成的目标文件（包括头文件，动态链接库，静态链接库，可执行文件等）、环境变量

9.2.2、创建 package 功能包

创建一个 package 需要在【dofbot_ws/src】下,用到【catkin_create_pkg】命令，用法是：

```
catkin_create_pkg package depends
```

其中 package 是包名，depends 是依赖的包名，可以依赖多个软件包。

```
cd ~/dofbot_ws/src/
catkin_create_pkg dofbot_moveit roscpp rospy rosmsg
cd ~/dofbot_ws/
```

这样就会在当前路径下新建 dofbot_moveit 功能包，包括：

```
├── CMakeLists.txt
├── include
│   └── dofbot_moveit
├── package.xml
└── src
```

【catkin_create_pkg】帮你完成了软件包的初始化，填充好了【CMakeLists.txt】和【package.xml】，并且将依赖项填进了这两个文件中。

- 一般的功能包架构：

```
CMakeLists.txt:  当前 package 的编译脚本。通常需要为 c++代码添加编译时的依赖，执行等操作。
package.xml:     package 相关信息。通常添加一些 ros 库的支持
include 文件夹:    存放 c++ 头文件的
config 文件夹：     存放参数配置文件，格式为 yaml
launch 文件夹：     存放.launch 文件的。
meshes 文件夹：     模型存放文件夹
src 文件夹：        c++源代码
scripts 文件夹：    python 源代码
srv 文件夹：        存放定义的 service
msg 文件夹:         存放自定义的消息协议
action 文件夹：     存放自定义的 action
```

- 统一说明一下命令行启动 ros 节点方式

1. 进入 python 文件所在目录,直接运行 python 文件名.py

有些文件则需用 python3,运行 python3 文件名.py

例如:

```
cd ~/dofbot_ws/src/dofbot_moveit/scripts/
python 02_motion_plan.py
```

2. 进入工作空间下,先编译,再更新环境,最后 rosrun 启动

```
cd ~/dofbot_ws/
catkin_make
source devel/setup.bash
# Python 文件
rosrun dofbot_moveit 02_motion_plan.py
# 对于 C++ 文件来说,此处的 02_motion_plan 不是文件名,是编译之后的目标文件
rosrun dofbot_moveit 02_motion_plan
```

9.2.3、【CMakeLists.txt】介绍

概述：

【CMakeLists.txt】原本是 Cmake 编译系统的规则文件，而 Catkin 编译系统基本沿用了 CMake 的编译风格，只是针对 ROS 工程添加了一些宏定义。所以在写法上，catkin 的【CMakeLists.txt】与 CMake 的基本一致。

这个文件直接规定了这个 package 要依赖哪些 package，要编译生成哪些目标，如何编译等等流程。所以【CMakeLists.txt】非常重要，它指定了由源码到目标文件的规则，catkin 编译系统在工作时首先会找到每个 package 下的【CMakeLists.txt】，然后按照规则来编译构建。

格式：

【CMakeLists.txt】的基本语法都还是按照 CMake，而 Catkin 在其中加入了少量的宏，总体的结构如下：

```
cmake_minimum_required() #CMake 的版本号
project()              #项目名称
find_package()  #找到编译需要的其他 CMake/Catkin  package
catkin_python_setup()  #catkin 新加宏，打开 catkin 的 Python  Module 的支持
add_message_files()  #catkin 新加宏，添加自定义 Message/Service/Action 文件
add_service_files()
add_action_files()
generate_message()  #catkin 新加宏，生成不同语言版本的 msg/srv/action 接口
catkin_package()  #catkin 新加宏，生成当前 package 的 cmake 配置，供依赖本包的其他软件包调用
add_library()          #生成库
add_executable()       #生成可执行二进制文件
add_dependencies()     #定义目标文件依赖于其他目标文件，确保其他目标已被构建
target_link_libraries()  #链接
catkin_add_gtest()     #catkin 新加宏，生成测试
install()          #安装至本机
```

9.2.4、【package.xml】

介绍如果你从未接触过 CMake 的语法，请阅读《CMake 实践》：【https://github.com/Akagi201/learning-cmake/blob/master/docs/cmake-practice.pdf】。掌握 CMake 语法对于理解 ROS 工程很有帮助。

概述：

【package.xml】也是一个 catkin 的 package 必备文件，它是这个软件包的描述文件，在较早的 ROS 版本(rosbuild 编译系统)中，这个文件叫做 manifest.xml，用于描述 pacakge 的基本信息。如果你在网上看到一些 ROS 项目里包含着 manifest.xml，那么它多半是 hydro 版本之前的项目了。

【package.xml】包含了 package 的名称、版本号、内容描述、维护人员、软件许可、编译构建工具、编译依赖、运行依赖等信息。 实际上 rospack find、rosdep 等命令之所以能快速定位和分析出 package 的依赖项信息，就是直接读取了每一个 pacakge 中的【package.xml】文件。它为用户提供了快速了解一个 pacakge 的渠道。

格式：

＜pacakge>	**根标记文件**
＜name>	包名
＜version>	版本号
＜description>	内容描述
＜maintainer>	维护者
＜license>	软件许可证
＜buildtool_depend>	编译构建工具，通常为 catkin
＜depend>	指定依赖项为编译、导出、运行需要的依赖，最常用
＜build_depend>	编译依赖项
＜build_export_depend>	导出依赖项
＜exec_depend>	运行依赖项
＜test_depend>	测试用例依赖项
＜doc_depend>	文档依赖项

9.3、自定义消息

在 Ros 中，如果没有现成的消息类型来描述要去传递的消息时，我们会自定义消息。

我们可以单独建一个 Package 功能包去自定义消息,这个 Package 一般不去写任何的业务逻辑，只是用来声明自定义的消息类型，也可以定义多种消息类型，根据业务需求来定,供其他功能包使用。

当然我们也可以把自定义消息类型定义到一个有业务逻辑的功能包,在创建功能包时,我们要添加 roscpp，rospy，rosmsg 的依赖。接下来,我们就一步步的开始创建自定义消息吧。

9.3.1、新建自定义消息

- 创建功能包

```
cd ~/dofbot_ws/src/
catkin_create_pkg dofbot_info roscpp rosmsg rospy
```

- 新建自定义消息

```
mkdir srv
cd srv/
touch kinemarics.srv
```

建的这个【kinemarics.srv】文件就是自定义消息文件，需要去描述消息的格式。我们可以编辑代码如下:

```
#  request
float64  tar_x
float64  tar_y
float64  tar_z
... ...（此处略去）
---
#  response
float64  joint1
float64  joint2
float64  joint3
... ...（此处略去）
```

这个.srv 文件以---分隔为两部分，在这里，上面部分是【request】的数据，下面部分是【response】的数据。

9.3.2、环境搭建

- 配置【package.xml】文件

```
＜build_depend>message_generation</build_depend>
＜exec_depend>message_runtime</exec_depend>
```

- 配置【CMakeLists.txt】

在【find_package】添加【message_generation】，结果如下:

```
#找到编译需要的其他 CMake/Catkin package
find_package(catkin REQUIRED COMPONENTS
      roscpp
      rosmsg
      rospy
      moveit_core
      moveit_msgs
      message_generation
      )
```

- 添加【add_service_files】，结果如下:

```
#catkin 新加宏，添加自定义 Message/Service/Action 文件
add_service_files(
      FILES
      kinemarics.srv
)
```

这里的【kinemarics.srv】要和你创建的 srv 文件名称一致，且必须时在 srv 目录下，否则编译会出现问题

- 添加【generation_msg】，结果如下:

```
#catkin 新加宏，生成不同语言版本的 msg/srv/action 接口
 generate_messages(
   DEPENDENCIES
   std_msgs   # Or other packages containing msgs
 )
```

这个配置的作用是添加生成消息的依赖，默认的时候要添加【std_msgs】。

- 修改 catkin_package，结果如下:

```
#catkin 新加宏，生成当前 package 的 cmake 配置，供依赖本包的其他软件包调用
catkin_package(
    #  INCLUDE_DIRS include
    #  LIBRARIES dofbot_info
      CATKIN_DEPENDS roscpp rosmsg rospy message_runtime
    #  DEPENDS system_lib
)
```

为 catkin 编译提供了依赖【message_runtime】。

9.3.3、编译

来到工作空间目录下，运行编译

```
cd ~/dofbot_ws/
catkin_make
source devel/setup.bash
```

9.3.4、查看验证

- C++的头文件

来到 devel 的 include 目录下，如果生成了头文件说明，自定义消息创建成功。

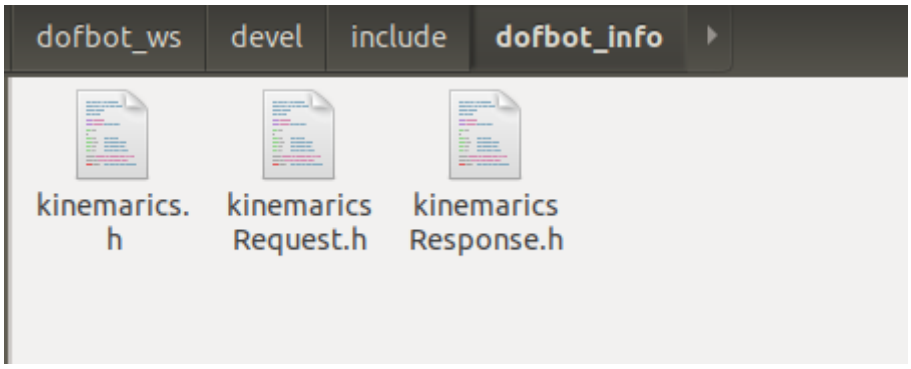


图 9.3.1 头文件

- python 的 py 文件

来到 devel 的【lib/python2.7/dist-package】目录下，查看是否生成和 package 名称相同的目录，以及目录内是否生成对应的 py 文件。

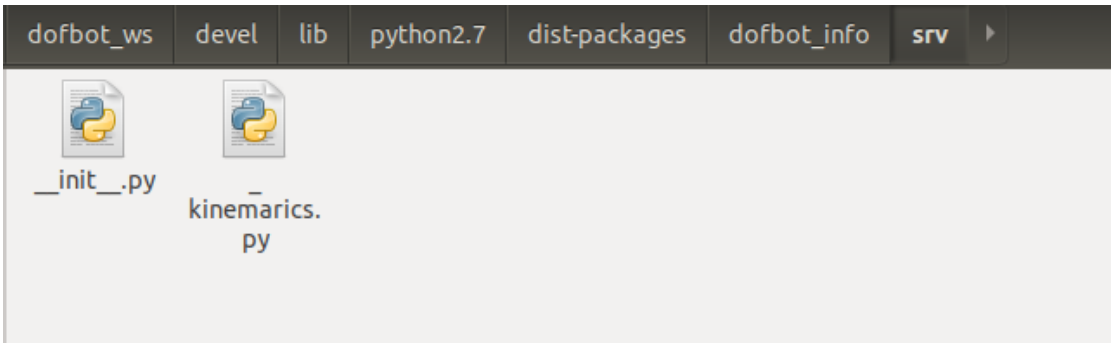


图 9.3.2 对应的 py 文件

- 通过 rossrv 工具校验

```
rossrv show dofbot_info/kinemarics
```

查看运行结果，运行结果和自己定义的相一致，说明成功。

```
@lenovo:    /dofbot_ws$ rossrv show dofbot_info/kinemarics
float64 tar_x
float64 tar_y
float64 tar_z
float64 Roll
float64 Pitch
float64 Yaw
float64 cur_joint1
float64 cur_joint2
float64 cur_joint3
float64 cur_joint4
float64 cur_joint5
float64 cur_joint6
string kin_name
---
float64 joint1
float64 joint2
float64 joint3
float64 joint4
float64 joint5
float64 joint6
float64 x
float64 y
float64 z
float64 Roll
float64 Pitch
float64 Yaw
```

图 9.3.3 运行结果

9.4、Server 通讯

- C++代码创建 server 端

主函数

```
/*
 * 这是机械臂正反解的 ROS 服务端
 * 注:所说的末端指的是第 5 个电机旋转中心,即不算夹爪
 */
int main(int argc, char **argv) {
    //ROS 节点初始化
    ros::init(argc, argv, "dofbot_server");
    //创建节点句柄
    ros::NodeHandle n;
    cout << "等待接收******" << endl;
    // 创建服务端
    ros::ServiceServer server = n.advertiseService("dofbot_kinemarics", srvicecallback);
    // 阻塞
    ros::spin();
    return 0;
}
```

回调函数

```
bool srvicecallback(dofbot_info::kinemaricsRequest &request, dofbot_info::kinemaricsResponse
&response) {
    if (request.kin_name == "fk") {
        double joints[]{request.cur_joint1, request.cur_joint2, request.cur_joint3, request.cur_joint4,
                        request.cur_joint5};
            （此处略去）
    }
    if (request.kin_name == "ik") {
            （此处略去）
    }
    return true;
}
```

- Python 代码实现客户端

创建节点与和客户端(以颜色分拣为例)

```
# ROS 节点初始化
self.n = rospy.init_node('dofbot_identify', anonymous=True)
```

```
# 创建获取反解结果的客户端
self.client = rospy.ServiceProxy("dofbot_kinemarics", kinemarics)
```

发送请求消息

```
def server_joint(self, posxy):
    '''
    发布位置请求,获取关节旋转角度
    :param posxy: 位置点 x,y 坐标
    :return: 每个关节旋转角度
    '''
    # 等待 server 端启动
    self.client.wait_for_service()
    # 创建消息包
    request = kinemaricsRequest()
    request.tar_x = posxy[0]
    request.tar_y = posxy[1]
    request.kin_name = "ik"
    try:
     response = self.client.call(request)
        if isinstance(response, kinemaricsResponse):
        # 获得反解响应结果
        pass
        return joints
    except Exception:
        rospy.loginfo("arg error")
```

9.5、机械臂 URDF 模型

9.5.1、概述

全称为 Unified Robot Description Format,翻译为中文为统一机器人描述格式，是一种使用 xml 格式描述的机器人模型文件，类似于 D-H 参数。

第一行为 xml 必填项，描述了 xml 的版本信息。

第二行描述了当前的机器人名称；当前机器人所有信息均包含在标签内。

URDF 主要包括：Links 和 Joints 两部分，后缀为。urdf.

- Links:连杆，坐标系与几何关系。
- Joints :关节，Links 之间的连接关系。

9.5.2、Links

简介

在 URDF 描述性语言中，link 是用来描述物理特性的。

- 描述视觉显示，＜visual>标签。
- 描述碰撞属性。＜collision>标签。
- 描述物理惯性，inertial>标签，不常用。

Links 还可以描述连杆尺寸（size)\颜色（color)\形状（shape)\惯性矩阵（inertial matrix)\碰撞参数（collision properties)等，每个 Link 会成为一个坐标系。

示例代码：

```
＜link name="linkl">
  ＜visual>
    ＜origin xyz="0 0 0"rpy="0 0 0" / ＞
```

```
    ＜geometry>
        ＜mesh filename="package://dofbot_moveit/meshes/linkl. STL" />
    ＜ / geometry>
    ＜material name="">
    ＜color rgba="0 0.627450980392157 0.23294117647059 17/>
     ＜ / material>
```

```
  < / visual>
  <collision>
    <origin xy="0 0 0" rpy="0 0. 0" />
    <geometry>
      <mesh filename="package://dofbot_moveit/meshos/}inkl.STL"/>
    < / geometry>
  < / collision>
< / link>
```

在<link>标签中 name 属性是 必填项，描述当前连杆名称，并且是唯一。

标签介绍

<origin>标签描述的是位姿信息；xyz 属性描述的是在大环境中的坐标位置，rpy 属性描述的是自身的姿态。

<geometry>标签描述的是形状；mesh 属性主要的功能是去加载纹理文件的 filename 属性纹理路径的文件地址。

<geometry>标签还包括其他的标签描述：box 箱体<box size="1 2 3"/>通过 size 属性描述盒装的长宽高。

- cylinder 圆柱状<cylinder length="1.6" radius="0.5"/>,通过 length 属
- 性描述圆柱的高度，radius 属性描述圆柱的的半径。
- sphere 球状<sphere radius="1" />, 通过 radius 属性描述球的半径。

<material>标签描述的是材质；name 属性为必填项，可以为空，可以重复。

通过标签中的 rgba 属性来描述红、绿、蓝、透明度，中间用空格分隔。颜色的范围为 [0-1]。

9.5.3、Joints

简介

描述两个关节之间的关系。运动位置和速度限制。运动学和动力学属性。

关节类型：

- fixed:固定关节。不允许运动，起连接作用

- continuous:旋转关节。可以持续旋转，没有旋转角度的限制。
- revolute:旋转关节。类似于 continuous,有旋转角度的限制。
- prismatic:滑动关节。沿某一轴线移动，有位置限制。
- floating:悬浮关节。具备六个自由度，3T3R.
- planar:平面关节。允许在平面正交上方平移或者旋转。

示例代码：

```
＜joint name=*jointl" type="revolute">
    ＜origin xyz="0 0 0. 06605" rpy="-0. 010805 0 0" />
    ＜parent link="base_link"/>
    ＜child link="linkl" / ＞
    ＜axis xyz="0 0 1" / ＞
    ＜limit effort="30° velocity="1.0" lower="-1.5708" upper="1.5708"/>
＜ / joint>
```

在＜joint>标签中 name 属性是 必填项，描述关节的名称，并且是唯一。

在＜joint>标签中 type 属性，对应填写六大关节类型。

标签介绍

＜parent>和＜child>子标签代表的是两个要连接的＜link>,和＜link>中的 name 属性对应；在此处将＜parent>作为参照物，＜child>围绕＜parent>进行旋转。

- ＜origin>子标签，指的是旋转关节于 parent 所在坐标系的相对位置。
- ＜axis>子标签表示＜child>对应的＜link>围绕哪一个轴旋转。
- ＜limit>子标签主要是限制＜child>的。lower 属性和 upper 属性限制了旋转的弧度范围，effort 属性限制的是转动过程中的受力范围。（正负 value 值，单位为牛或 N),velocity 属性限制了转动时的速度，单位为米 / 秒或 m/s。

9.5.4、launch 文件介绍

launch 文件是一个 xml 文件，用来描述节点启动的，可以同时启动多个节点。

示例代码：

```
＜launch>
    ＜arg name="model" default="dofbot. urdf"/>
    ＜arg name="gui" default="false"/>
    ＜param name="robot_description" command="$(find xacro)/xacro -inorder $(find
dofbot_moveit)/urdf/$(arg model)"/>
        ＜param name="use_gui" value="$(arg gui)^/>
        ＜node name="joint_state_publisher_gui^ pkg="joint_state_pu5lisher_gui"
type="joint_state_publisher_gui"/>
        ＜node name="robot_state_publisher^ pkg="robot_state_publisher"
type="robot_state_publisher"/>
    ＜node name="rviz" pkg="rviz" type="ryiz"args="-d. $(find
dofbot_moveit)/rviz/arm_urdf. rviz^ required="true"/>
＜ / launch>
```

- pkg 属性要启动的 package 名称。
- type 属性可执行文件名称。
- name 属性可执行文件启动后节点名称。
- ＜param>子标签，是 xacro 执行了 dofbot.urdf 之后输出的结果作为值赋给 robot_description.
- ＜arg>子标签，name 属性为参数名称，default 属性是默认参数。

9.5.5、URDF 可视化

打开命令行，依次执行以下命令：

```
cd~/dofbot_ws/ #进入工作空间
catkin_make  #编译
source devel/setup.bash #  更新系统环境
roslaunch dofbot_moveit dofbot_moveit. launch #  启动 ROS 节点
```

示例图片：

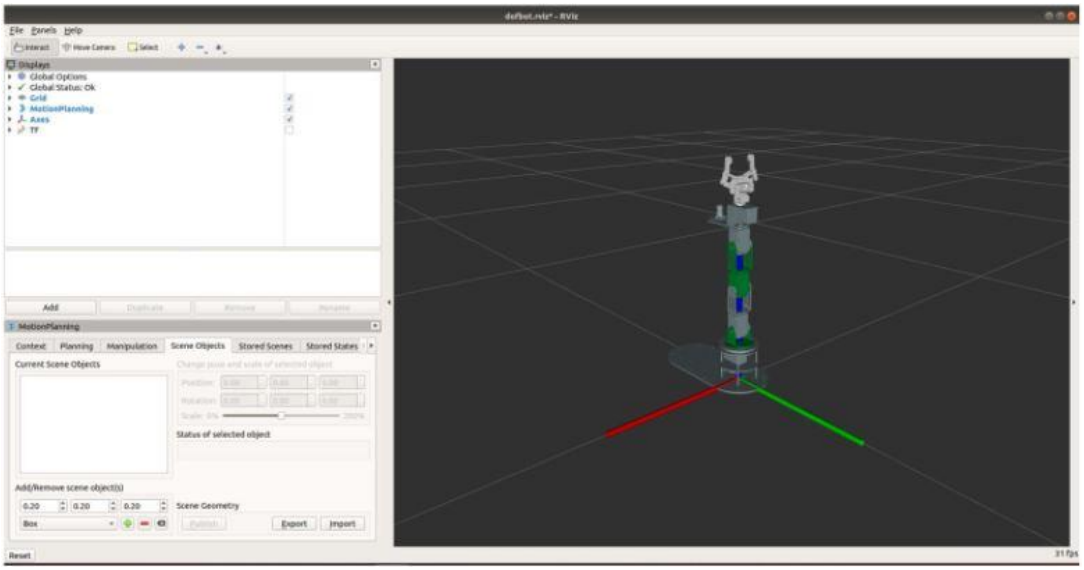


图 9.5.1 示例图片

红色轴线为 X 轴；绿色轴线为 Y 轴；蓝色轴线为 Z 轴；三条轴线构成的坐标系，我们认为是基坐标系。

9.6、机械臂运动学正反解

正解代码路径：dofbot_ws/src/dofbot_moveit/src/dofbot_kinematics_fk.cpp

正解代码测试：

```
cd ~/dofbot_ws/
source devel/setup.bash
rosrun dofbot_moveit dofbot_kinematics_fk
```

反解代码路径：dofbot_ws/src/dofbot_moveit/src/dofbot_kinematics_ik.cpp

反解代码测试：

```
cd ~/dofbot_ws/
source devel/setup.bash
rosrun dofbot_moveit dofbot_kinematics_ik
```

9.6.1、概述

机械臂运动学研究的是机械臂运动,而不考虑产生运动的力.本节仅考虑机械臂在静止的状态下的位姿关系。

典型的机械臂是由一些串行连接的关节和连杆组成.每个关节具有一个自由度,平移或旋转.对于具有 n 个关节的机械臂,关节的编号从 1 到 n,有 n+1 个连杆,编号从 0 到 n.连杆 0 是机械臂的基础,一般是固定的,连杆 n 上带有末端执行器.关节 i 连接连杆 i 和连杆 i-1.一个连杆可以被视为一个刚体,确定与它相邻的两个关节轴之间的相对位置.一个连杆可以用两个参数描述,连杆长度和连杆扭矩,这两个量定义了与它相关的两个坐标轴在空间的相对位置.而第一连杆和最后一个连杆的参数没有意义,一般选择为 0.一个关节用两个参数描述,一是连杆的偏移,是指从一个连杆到下一个连杆沿的关节轴线的距离.二是关节角度,指一个关节相对于下一个关节轴的旋转角度。

9.6.2、三维空间的位姿描述

首先规定一个坐标系，相对于该坐标系，点的位置可以用 3 维列向量表示;刚体的方位可用 3×3 的旋转矩阵来表示。而 4×4 的齐次变换矩阵则可将刚体位置和姿态（位姿）的描述统一起来，它具有以下优点：

(1) 它可描述刚体的位姿，描述坐标系的相对位姿（描述）。

(2) 它可表示点从一个坐标系的描述转换到另一坐标系的描述（映射）。

(3)它可表示刚体运动前、后位姿描述的变换（算子）。

- **位置描述——位置矢量**

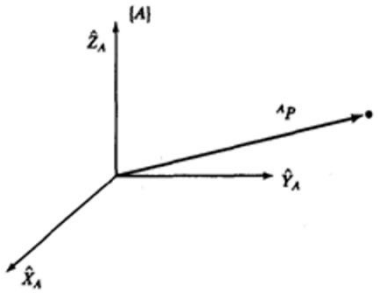


图 9.6.1 相对于坐标系的矢量（示例）

用三个相互正交的带有箭头的单位矢量来表示一个坐标系{A}.那么点 p 在坐标系{A}下空间位置的表示为：

$$^AP = \begin{bmatrix} P_x \\ P_y \\ P_z \end{bmatrix}$$

- **方位的描述——旋转矩阵**

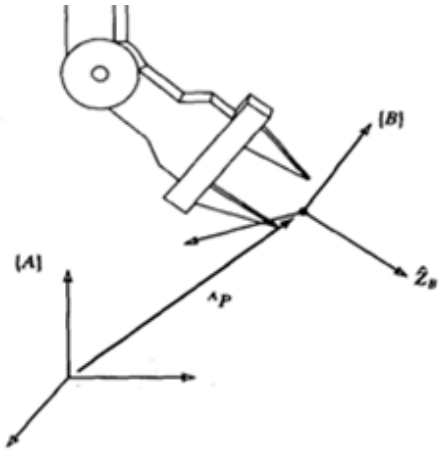


图 9.6.2 物体位置和姿态的确定

常用的旋转变换矩阵是绕 X 轴、绕 Y 轴或绕 Z 轴转一角度θ。它们分别是

$$R(X,\theta)=\begin{bmatrix}1 & 0 & 0\\ 0 & cos(\theta) & -sin(\theta)\\ 0 & sin(\theta) & cos(\theta)\end{bmatrix} R(Y,\theta)=\begin{bmatrix}cos(\theta) & 0 & sin(\theta)\\ 0 & 1 & 0\\ -sin(\theta) & 0 & cos(\theta)\end{bmatrix} R(Z,\theta)=\begin{bmatrix}cos(\theta) & -sin(\theta) & 0\\ sin(\theta) & cos(\theta) & 0\\ 0 & 0 & 1\end{bmatrix}$$

- **非齐次表述**

向量 a 经过一次旋转 R 和一次平移 t 后，得到 a/ : a/=R*a+t

- **齐次表述**

$$\begin{bmatrix}a\prime\\ 1\end{bmatrix}=\begin{bmatrix}R & t\\ 0^T & 1\end{bmatrix}\begin{bmatrix}a\\ 1\end{bmatrix}=T\begin{bmatrix}a\\ 1\end{bmatrix}$$

通用旋转矩阵 R 为：

$$R=R_z(\beta)R_y(\alpha)R_x(\theta)$$

这里的旋转矩阵旋转过程，先绕 X 轴旋转 θ 角，然后绕 Y 轴旋转 α 角，最后绕 Z 轴旋转 β 角。

- **右手法则**

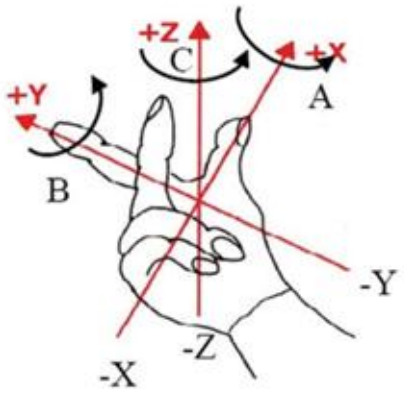


图 9.6.3 右手法则

大拇指指向 x，食指伸向 y，中指的方向是 z。

- **RPY 与欧拉角**

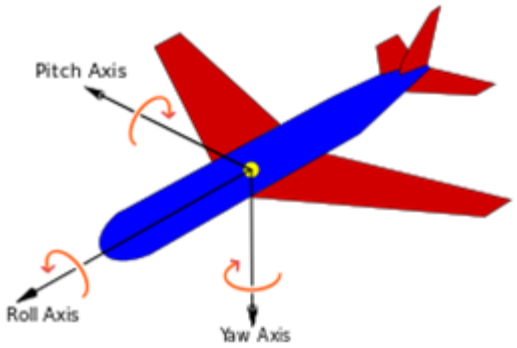


图 9.6.4 RPY 与欧拉角

按照自身坐标系旋转顺序是 Z-->Y-->X，叫做 eulerYPR（Yaw Pitch Roll）；

按照外界坐标系（参考坐标系）的旋转顺序是 x-->y-->z，叫做 RPY（Roll Pitch Yaw）；

描述同一姿态时，以上两者的表示是等价的，即 Yaw Pitch Roll 的角度值是一样的。

定义： Yaw 偏航角 ϒ ; Pitch 俯仰角 β ; Roll 翻滚角 α

- **四元数法**

大概的定义是，用标量和三个虚轴表示向量，叫四元数。

9.6.3、映射变换

映射：从坐标系到坐标系的变换也就是说表达同一个量在不同坐标系的表示，这个在机器人学中经常用到。

- **平移坐标的映射**

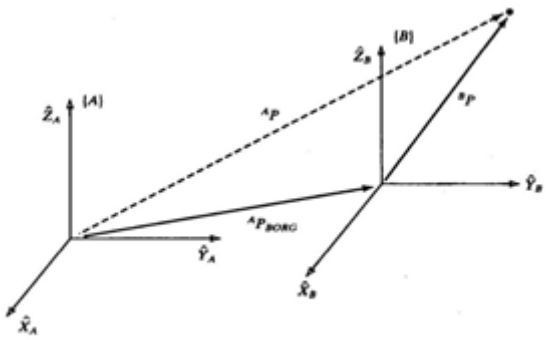


图 9.6.5 平移映射

{A} 和 {B}的姿态相同，{B}不同于{A}的只是平移，即{B}相对于{A}不存在旋转。P 相对于{A}的位置可以用矢量相加表示,公式如下：

$$^{A}P = ^{B}P + ^{A}P_{BORG}$$

- **旋转坐标系的映射（原点重合只有旋转）**

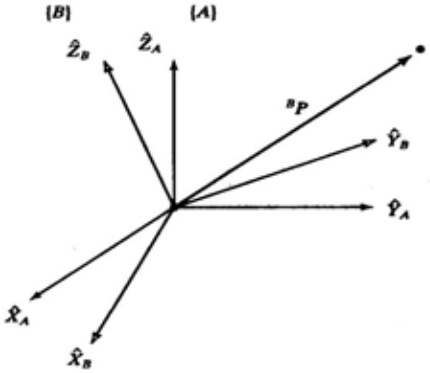


图 9.6.6 矢量的旋转

任意矢量的分量就是该矢量在参考系上的单位矢量方向的投影，投影是由矢量点积计算的。

$$^{A}P_x = ^{B}\hat{X}_A^T * ^{B}P \quad 、 \quad ^{A}P_y = ^{B}\hat{X}_A^T * ^{B}P \quad 、 \quad ^{A}P_z = ^{B}\hat{X}_A^T * ^{B}P$$

可以将其简化为：

$$^{A}P = ^{A}_{B}R\,^{B}P$$

这个映射将空间中的某个点 P 相对于{B}的描述转换成了该点相对于{A}的描述。

- **一般坐标系的映射**

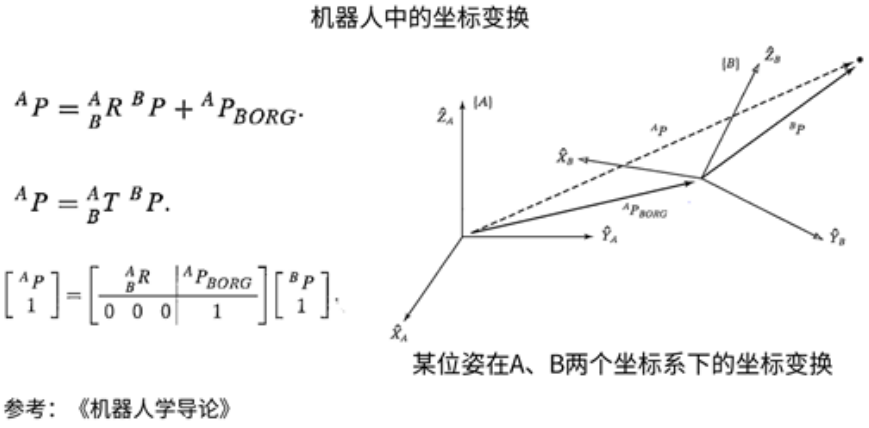


图 9.6.7 一般坐标系的映射

应用场景：两个坐标系 {A} 和 {B}，已知 {B} 相对 {A} 的平移和旋转分别为 APBORG 和 ${}^{A}_{B}R$,BP，求 AP。

结合前面只有平移和只有旋转的变换情况，设定一个中间坐标系{C}，它的原点与{B}重合，姿态与{A}一致，就可以求得下面这个公式：

$$^{A}P = {}^{A}_{B}R\,{}^{B}P + {}^{A}P_{BORG} = {}^{A}_{B}T\,{}^{B}P$$

这就引出了矩阵形式的算子 的概念，表示一个坐标系到另一个坐标系的映射。用分块矩阵的形式写出来更好理解

$$\begin{bmatrix} {}^{A}P \\ 1 \end{bmatrix} = \begin{bmatrix} {}^{A}_{B}R & {}^{A}P_{BORG} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} {}^{B}P \\ 1 \end{bmatrix}$$

上式中间矩阵被称为齐次变换矩阵，尽管简洁形式的齐次变换很有用，但计算机程序中一般不用它来进行矢量变换，因为这样把时间消耗在 0、1 乘法运算上了。

9.6.4、正解反解

正解 ：在一个机械臂 dofbot 中，已知每个关节角度，求解末端位姿。

反解 ：在一个机械臂 dofbot 中，已知末端(除夹爪)位姿，求每个关节角度。

在已知机械臂关节角时，求解机械臂末端位置和姿态，求取末端坐标系相对于基座坐标系的变换方程即可。

求取机械臂正运动学可以分为以下几步：

- 为机械臂个关节建立坐标系。
- 根据坐标系关系及连杆参数，写出机械臂 DH table。
- 由下式及 DH table 求解出各连杆的变换矩阵。
- 依据连续连杆变换矩阵，求解末端相对于基座的变换矩阵。

一个机器人的关节结构可以用一串字符来描述，例如 Puma560 为“RRRRRR”，斯坦福臂为“RRPRRR”，其中每个字符代表相应关节的类型，R 是转动副，P 为移动副。迪拉维特（Denavit）和哈滕贝格（Hartenberg）于 1955 年提出了一种用于描述这种串联式链路上连杆和关节几何关系的系统方法，这就是如今熟知的 D-H 参数法。

Dofbot 的坐标系建立如下图：

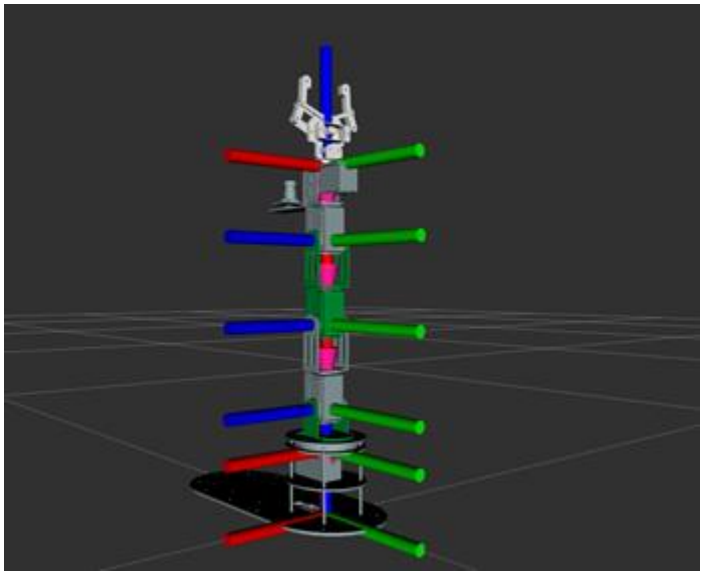


图 9.6.8 坐标系建立

红色轴线为 X 轴;绿色轴线为 Y 轴;蓝色轴线为 Z 轴;最下方的坐标系,我们称作基坐标系。

α_{i-1}：以$\hat{X}_{i-1}$方向看，$\hat{Z}_{i-1}$和$\hat{X}_i$间的夹角

a_{i-1}：以$\hat{X}_{i-1}$方向看，$\hat{Z}_{i-1}$和$\hat{X}_i$间的距离$(a_i > 0)$

θ_i：以$\hat{Z}_i$方向看，$\hat{X}_{i-1}$和$\hat{X}_i$间的夹角

d_i：以$\hat{Z}_i$方向看，$\hat{X}_{i-1}$和$\hat{X}_i$间的距离

Dofbot 的 D-H 参数 ：

i	d_i(沿 Z 轴)	a_i(沿 X 轴)	β_i(绕 Y 轴)	θ_i(绕 Z 轴)
1	0.066	0	0	θ_1
2	0.04145	0	π/2	θ_2
3	0	-0.08285	0	θ_3
4	0	-0.08285	0	θ_4
5	0	-0.07385	-π/2	θ_5

三角函数公式补充:

cos(a+b)=cos(a)cos(b)-sin(a)sin(b) 简写成 ： c(a+b)=c(a)c(b)-s(a)s(b)

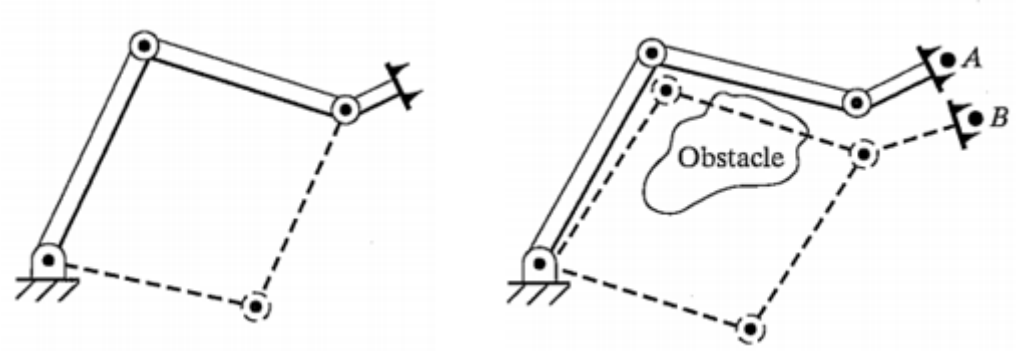
cos(a-b)=cos(a)cos(b)+sin(a)sin(b) 简写成 ： c(a-b)=c(a)c(b)+s(a)s(b)

sin(a±b)=sin(a)cos(b)±cos(a)sin(b)简写成: s(a±b)=s(a)c(b)±c(a)s(b)

sin(0°)=0 ； sin(90°)=1 ； cos(0°)=1 ； cos(90°)=0

机械臂的逆运动学问题

机械臂的逆运动学是指已知机械臂的末端位姿， 即已知齐次变换矩阵，求解各转动关节的角度。因此机械臂的逆运动学问题，可以理解为通过正运动学方程求解关节的 θ1、θ2、θ3、θ4、θ5。计算机器人运动学逆解首先要考虑可解性(solvability)，即考虑无解、多解等情况。在机器人工作空间外的目标点显然是无解的。对于多解的情况从下面的例子可以看出平面二杆机械臂（两个关节可以360°旋转）在工作空间内存在两个解：



如果逆运动学有多个解，那么控制程序在运行时就必须选择其中一个解，然后发给驱动器驱动机器人关节旋转或平移。如何选择合适的解有许多不同的准则，其中一种比较合理的方法就是选择“最近”的解（the closest solution）。如图所示，如果机器人在 A 点，并期望运动到 B 点，合理的解是关节运动量最小的那一个（A good choice would be the solution that minimizes the amount that each joint is required to move）。因此在不存在障碍物的情况下，上面的虚线构型会被选为逆解。在计算逆解时我们可以考虑将当前位置作为输入参数，这样我们就可以选择关节空间中离当前位置最近的解。

这个“最近”有多种定义方式。而当存在障碍物时，“最近”的解的运动路径会与其发生碰撞，这时就要选择另一个运动距离较远的解（"farther" solution）。因此在考虑碰撞、路径规划等问题时我们需要计算出可能存在的全部解。逆解个数取决于机器人关节数目（the number of joints）、机器人的构型（link parameters）以及关节运动范围(the allowable ranges of motion of the joints)。决定机器人构型的 D-H 参数表中的非零值越多，就有越多的解存在。对于通用型 6 轴转动关节的机械臂来说，最多可能存在 16 个不同的解。

另外机器人逆运动学求解也有多种方法，一般分为两类：封闭解（closed-form solutions）和数值解（numerical solutions）。不同学者对同一机器人的运动学逆解也提出不同的解法。应该从计算方法的计算效率、计算精度等要求出发，选择较好的解法。通常来说数值迭代解法比计算封闭解的解析表达式更慢、更耗时，因此在设计机器人的构型时就要考虑封闭解的存在性。

解析解与数值解的区别：数值解(numerical solution)是采用某种计算方法,如有限元的方法，数值逼近,插值的方法，得到的解。别人只能利用数值计算的结果，而不能随意给出自变量并求出计算值。比如上面给出了一元二次方程的解析解，在求一个已知系数的一元二次方程时，将系数的具体取值代入则可以得到其数值解。可以这样来理解二者的区别，解析解是一个求解公式，它适用于所有这类方程的求解，而数值解是某个特定方程的具体的解。

例如 ：方程 2y=x 的解：
y=0.5x —— 解析解
x=1 时，y=0.5 —— 数值解

9.6.5、代码求解

- 环境配置

将头文件 dofbot_kinemarics.h 和动态库 libdofbot_kinemarics.so 放到 dofbot_ws/src/dofbot_moveit/include/dofbot_moveit 目录下,如果出现找不到动态库的情况,将动态库 libdofbot_kinemarics.so 放到/usr/lib 目录下.系统已经默认放在/usr/lib 目录下。

打开功能包中的 CMakeLists.txt 文件,指定头文件 dofbot_kinemarics.h 所在的位置。

```
include_directories(
    include/dofbot_kinemarics
    ${catkin_INCLUDE_DIRS}
)
```

- 指定库以链接库或可执行目标

```
add_executable(dofbot_kinematics_fk src/dofbot_kinematics_fk.cpp)
add_executable(dofbot_kinematics_ik src/dofbot_kinematics_ik.cpp)
target_link_libraries(dofbot_kinematics_fk ${catkin_LIBRARIES}
```

- 导入头文件

```
#include "dofbot_kinemarics.h"
#include
```

- 创建机械臂实例

```
Dofbot dofbot = Dofbot();
```

- 求正解

```
/**
* 正向运动学计算 根据旋转关节角获取当前位姿
* @param urdf_file 模型文件路径
* @param joints 当前关节角度
* @param cartpos 当前末端位姿
*/
bool dofbot_getFK(const char *urdf_file, vector
```

- 示例代码

```
// 当前各关节角速度
double joints[]{90, 135, 0, 0, 90};
// 定义目标关节角容器
vector
// 定义位姿容器
vector
// 目标关节角度单位转换,由角度[0,180]转换成弧度[-1.57,1.57]
for (int i = 0; i < 5; ++i) initjoints.push_back((joints[i] - 90) * DE2RA);
```

```
// 调取正解函数,获取当前位姿
dofbot.dofbot_getFK(urdf_file, initjoints, initpos);
cout <<fixed<< "fk="" kinematics="" result="" :="" "="" <<="" =""  endl;<="" span="" style="pa
dding: 0px; margin: 0px; box-sizing: border-box;"></fixed<<>
cout << "X 坐标 (cm)： " << initpos.at(0) * 100 << "t"
    << "Y 坐标 (cm)： " << initpos.at(1) * 100 << "t"
    << "Z 坐标 (cm)： " << initpos.at(2) * 100 << endl;
cout << "Roll (°)： " << initpos.at(3) * RA2DE << "t"
    << "Pitch (°)： " << initpos.at(4) * RA2DE << "t"
    << "Yaw (°)： " << initpos.at(5) * RA2DE << endl;
```

- 求反解

```
/**
* 逆运动学计算 获取到到目标点各关节需要转动的角度
* @param urdf_file 模型文件路径
* @param targetXYZ 目标位置
* @param targetRPY 目标姿态
* @param outjoints 目标点关节角度
*/
bool dofbot_getIK(const char *urdf_file, vector
```

- 示例代码（须注意的是，这里求得是末端位姿，不包含末端执行器）

```
// 末端的位姿
double Roll = -135;
double Pitch = 0;
double Yaw = 0;
// 求末端位置
double x = 0;
double y = 5.5;
double z = 17.375;
// 末端位置(单位: m)
double xyz[]{x, y, z};
// 末端姿态(单位: 弧度)
double rpy[]{Roll , Pitch, Yaw };
// 创建输出角度容器
vector
// 创建末端位置容器
vector
// 创建末端姿态容器
vector
```

```
// 末端位置单位转换,由 cm 转换成 m
for (int k = 0; k < 3; ++k) targetXYZ.push_back(xyz[k] / 100);
// 末端姿态单位转换,由角度转换成弧度
for (int g = 0; g < 3; ++g) targetRPY.push_back(rpy[g] * DE2RA);
// 反解求到达目标点的各关节角度
dofbot.dofbot_getIK(urdf_file, targetXYZ, targetRPY, outjoints);
// 打印反解结果
cout << fixed << "IK kinematics result : " <<     endl;
// 将弧度[-1.57,1.57]转换成角度[0,180]
for (int i = 0; i < 5; i++) cout << outjoints.at(i) * RA2DE + 90     << "t";
```

详细代码请查看 dofbot_ws/src/dofbot_moveit/src/目录下的 dofbot_kinematics_fk.cpp 和 dofbot_kinematics_ik.cpp。

9.7、MoveIt 配置

9.7.1、配置流程

启动配置程序

启动 roscore

```
roscore
```

另开一个终端,启动 MoveIt

```
rosrun moveit_setup_assistant moveit_setup_assistant
```

配置流程

- 加载 URDF 模型

如果加载模型时,报错【找不到模型】,退出 MoveIt-->进入工作空间-->更新环境(source devel/setup.bash)-->再次启动 MoveIt 配置。

若是第一次加载模型生成配置,则选择左侧,若是修改已生成的配置文件,则选择右侧。

图 9.7.1 URDF 模型

点击 Browse 按钮,找到 URDF 模型文件,点击右下角加载。

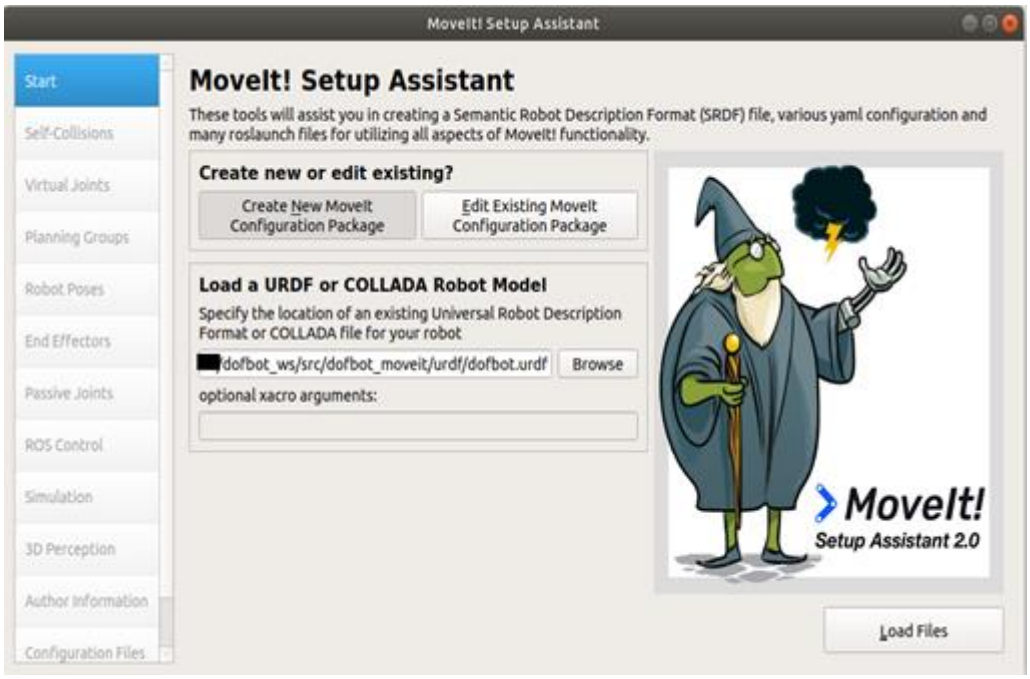


图 9.7.2 点击加载

- 创建碰撞免检矩阵(Avoid Collision Matrix, ACM),点击 Generate Collision Matrix 按钮即可。

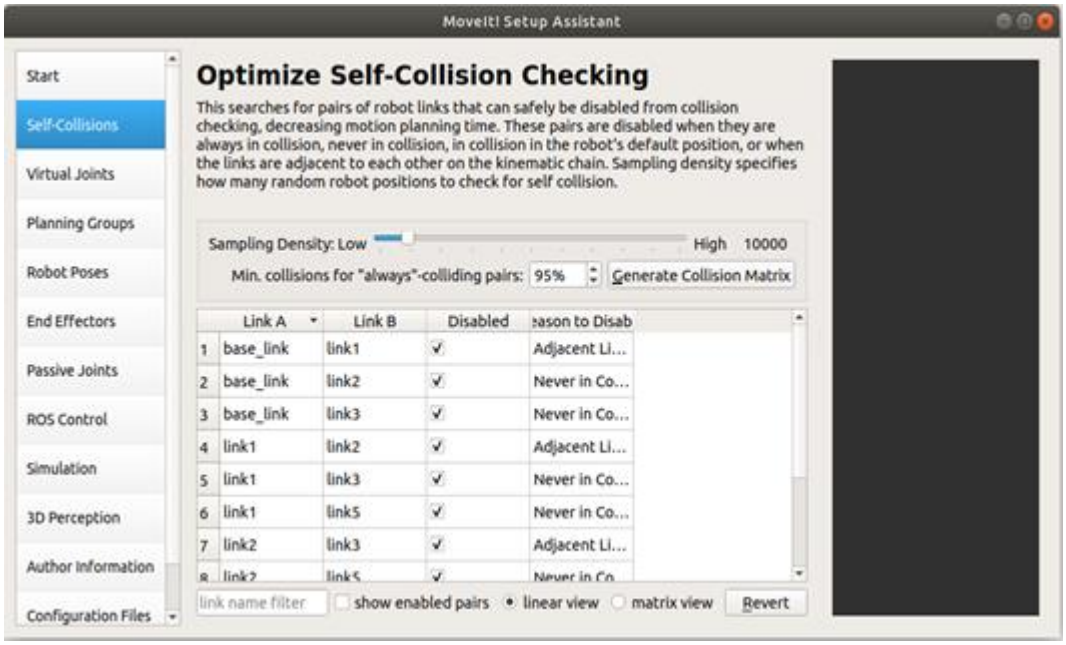


图 9.7.3 创建碰撞免检矩阵

碰撞检测是非常复杂的运算过程。对于多关节机械臂或者类人机器人来说，机械结构复杂，关节多，碰撞检测需要涉及很多的空间几何计算。但是对于刚体机器人来说，有些关节之间是不可能发生碰撞的，比如原本就相邻的肢体。这里生成 ACM 的目的就是告诉我们，哪些关节之间是不会发生碰撞的。在之后的碰撞检测算法中，就可以直接略过对这些关节之间的检测，以提高检测效率。

- 添加虚拟关节，可以理解为一个连接机器人和世界的关节。

Virtual Joint Name 我们命名为 virtual_joint.

Child Link 指的是我们要把‘世界’和机器人的那个部位连接，我们选择基座 base_link。

Parent Frame Name，是世界坐标的名字，在 ROS 中一般叫 world。

Joint Type 关节类型 ,选择固定 Fixed. 代表机器人相对于世界是固定的。而另外两种, Planar 指的是平面移动底座(xy 平面+角度)，用于移动机器人比如 PR2；还有一种 Floating, 指的是浮动基座(xyz position+orientation)，比如类人机器人。

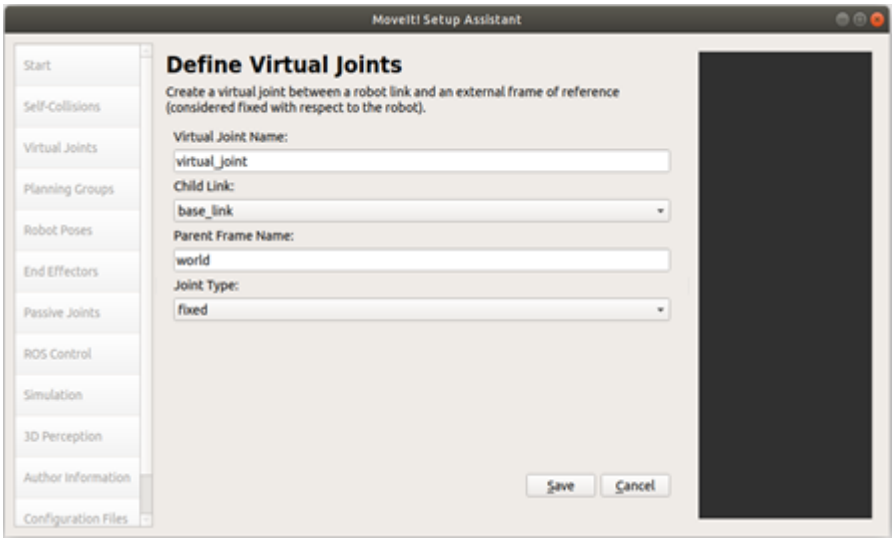


图 9.7.4 添加虚拟关节

- 创建运动规划组 Planning Group 是 MoveIt 的核心之一。

Group Name: 创建组名,我们就叫 dofbot。

Kinematic Solver: 运动学求解工具，这个就是负责求解正运动学(Forward Kinematics)和逆运动学(IK)的。 一般我们选用 KDL， The Kinematics and Dynamics Library。这是一个运动学与动力学的库，可以很好的解决 6 自由度以上的单链机械结构的正逆运动学问题。当然你也可以用其他 IK Solver, 比如 SRV 或者 IK_FAST，甚至你可以自己开发新的 Solver 然后插入进来。

Kin. Search Resolution: 关节空间的采样密度

Kin. Search TImeout: 求解时间

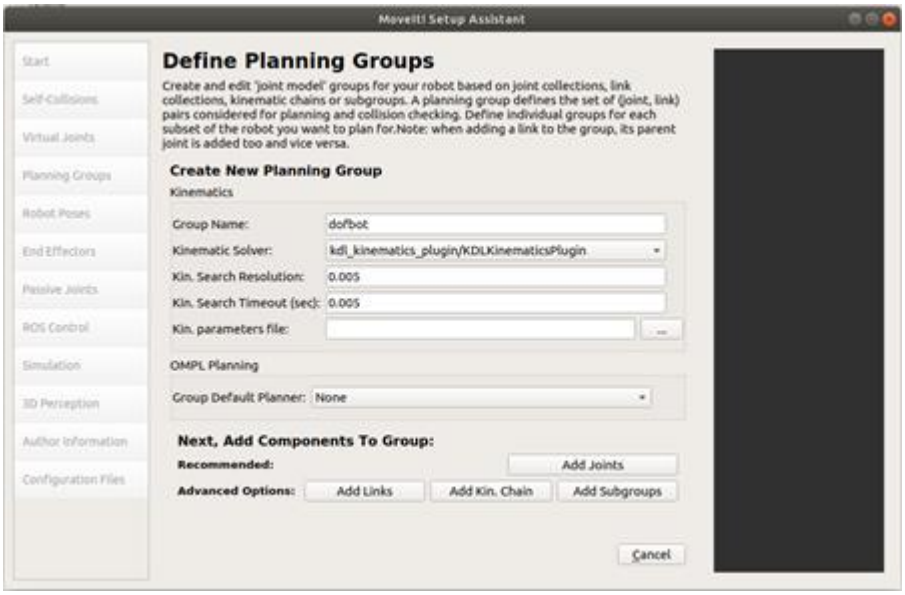


图 9.7.5 创建运动规划组

- 定义规划组

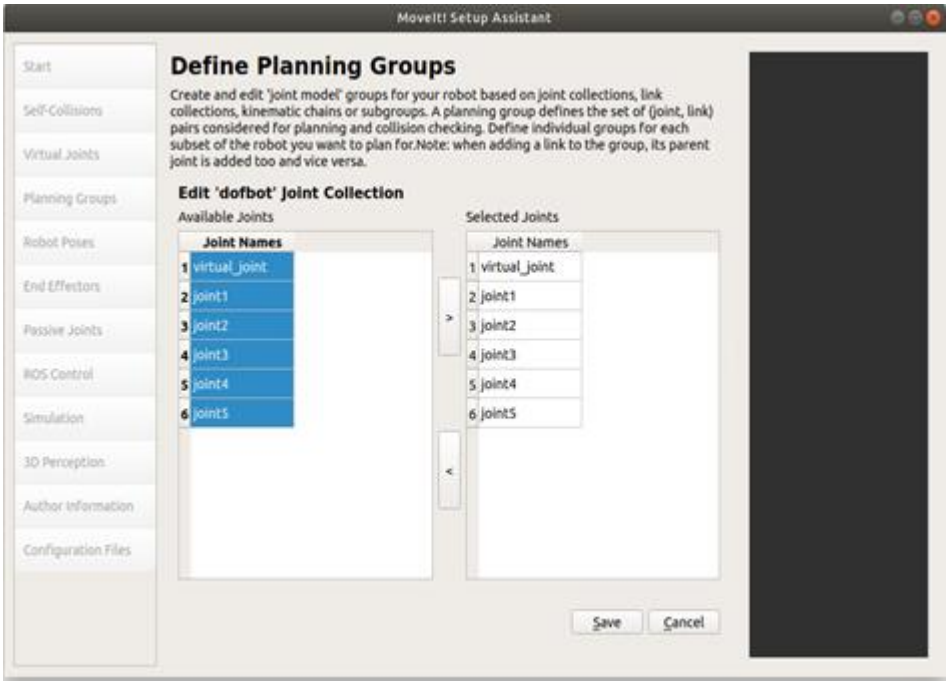


图 9.7.6 定义规划组

- 创建机器人预设位姿 (Robot Poses)

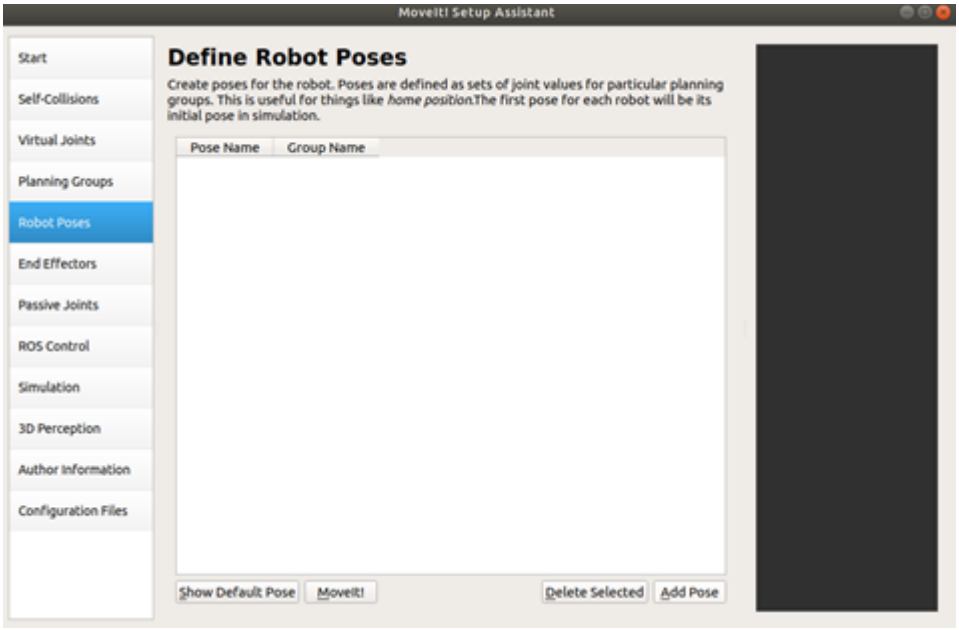


图 9.7.7 创建机器人预设位姿

点击 Add Pose，我们为机械臂创建一个向上直立的位姿 up，选择 Planning Group 为 dofbot。左侧可以看到很多滚动条，全设为 0 就是垂直向上的位姿。然后点击保存。

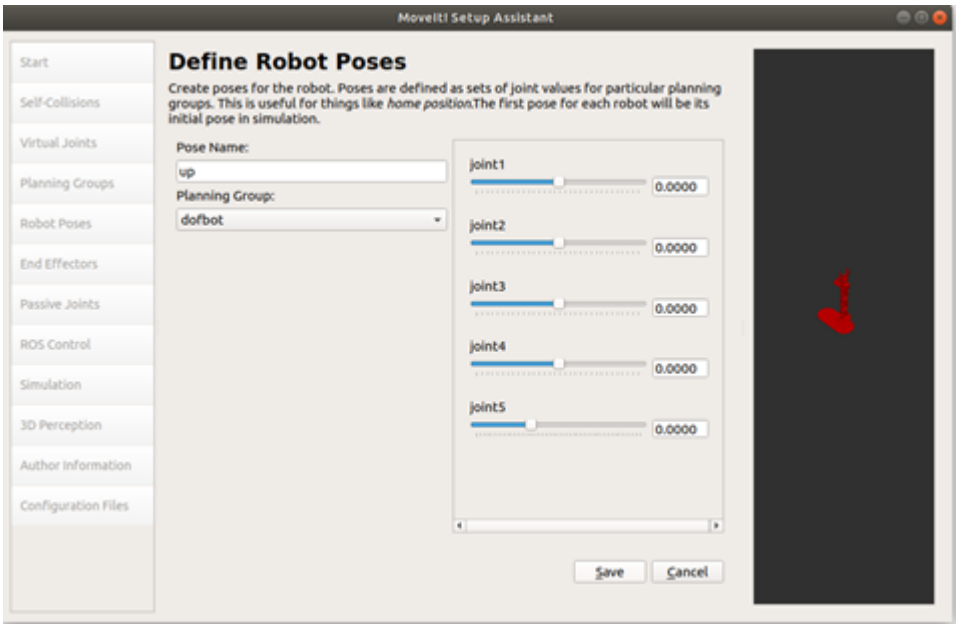


图 9.7.8 创建向上位姿

点击 Add Pose，我们为机械臂创建一个水平的位姿 down，选择 Planning Group 为 dofbot。将 joint2 滑至最左端。

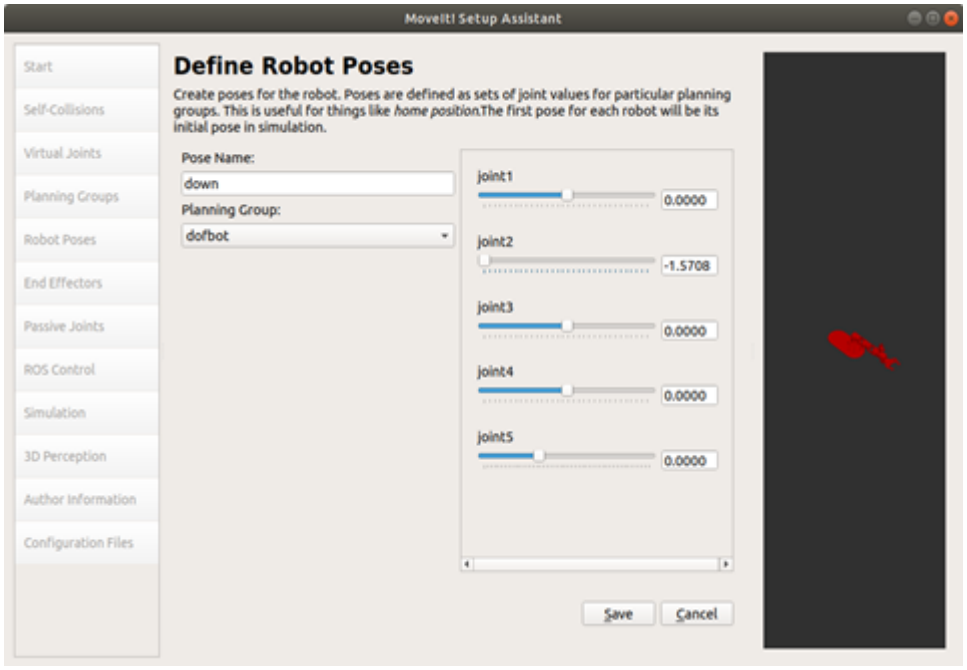


图 9.7.9 创建水平位姿

- 建立 ROS 控制器

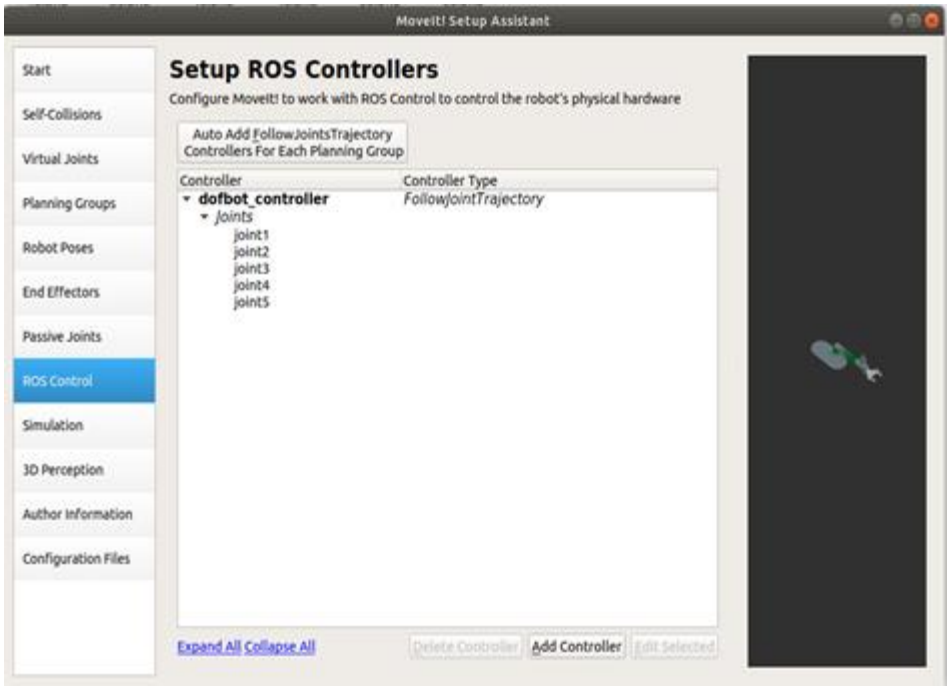


图 9.7.10 建立 ROS 控制器

- 添加可用 gazebo 模拟

图 9.7.11 添加可用 gazebo 模拟

- 建立 3D 信息,可略过

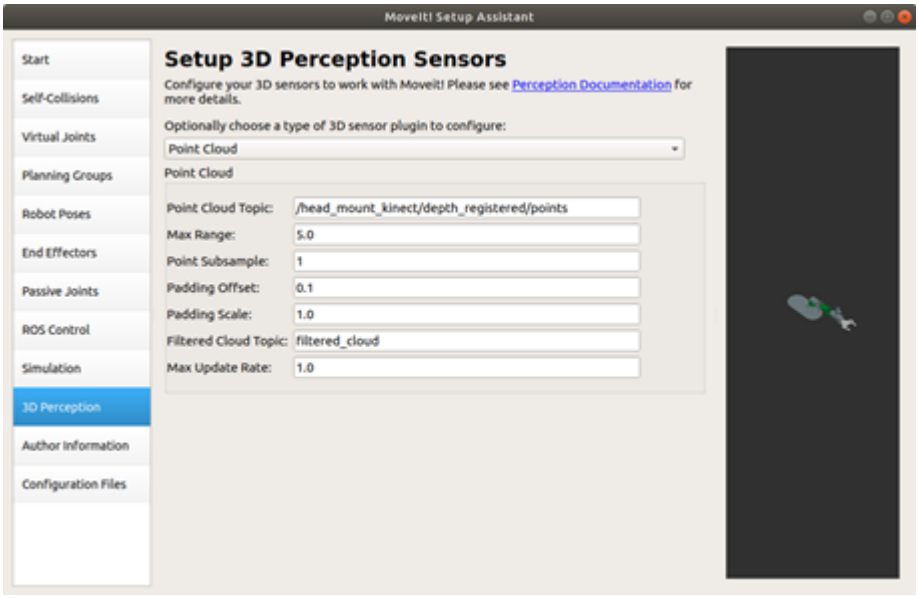


图 9.7.12 建立 3D 信息

- 添加作者信息,不添加,则无法生成

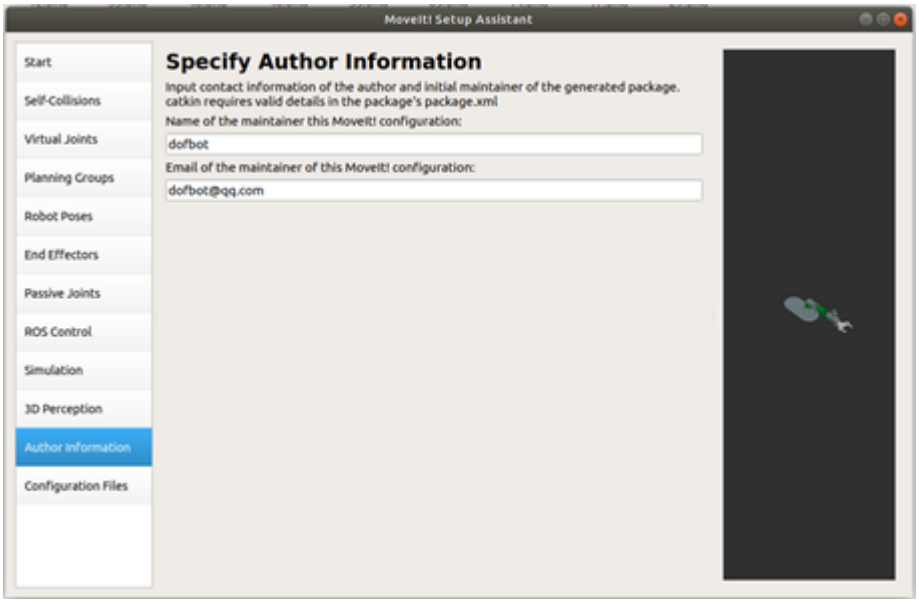


图 9.7.13 添加作者信息

- 生成配置文件

新建 dofbot_config 文件夹,将配置文件放在此文件夹下,点击 Browse 按钮,选中该文件夹,点击 Generate Package 按钮,生成配置文件.完毕后,退出即可。

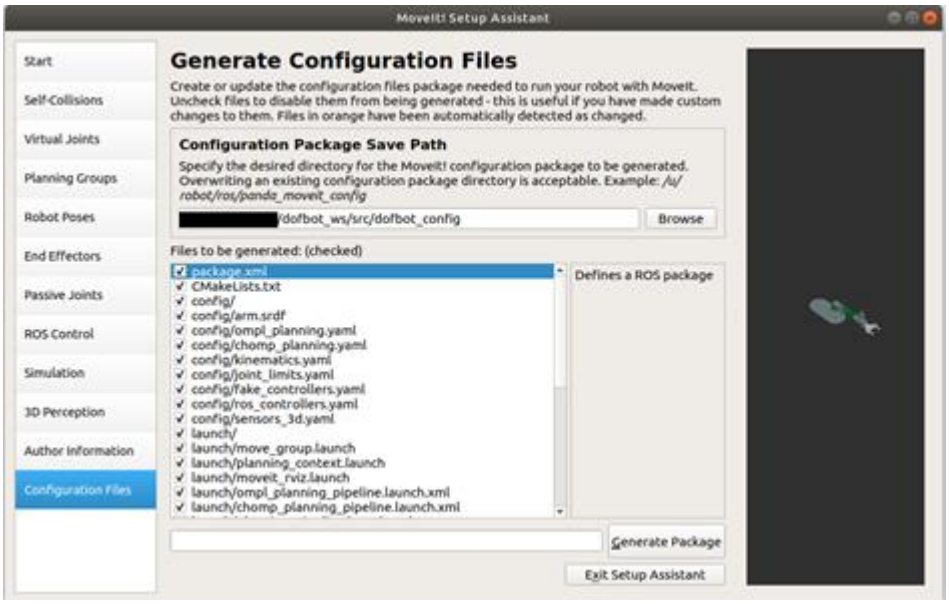


图 9.7.14 生成配置文件

9.7.2、MoveIt 配置包

配置包详解

打开刚刚创建好的 dofbot_config 文件夹，我们发现有 config 和 launch 两个文件夹。

1. config 文件夹

- fake_controllers.yaml：这是虚拟控制器配置文件，方便我们在没有实体机器人，甚至没有任何模拟器（如 gazebo）开启的情况下也能运行 MoveIt。
- joint_limits.yaml：这里记录了机器人各个关节的位置速度加速度的极限，这些都会被用于以后的规划中。
- kinematics.yaml：运动规划组设置的东西，用于初始化运动学求解库
- dofbot.srdf：这个是一个重要的 MoveIt 配置文件。
- ompl_planning.yaml：这里是配置 OMPL 各种算法的各种参数。
- SRDF 文件 :SRDF 是 moveit 的配置文件，配合 URDF 使用。我们可以看到这是一个 xml 格式的配置文件，根是 robot，并有一个属性 name='dofbot'。下面就是刚刚在 Setup Assistant 里面所设置的东西，包含了组群，位姿，终端控制器，虚拟关节，以及碰撞免测矩阵 ACM 的定义。理论上，只要有了 srdf 和 urdf，我们就可以完全定义一个机器人 moveit 信息。

2. launch 文件夹

- demo.launch :demo 是运行的总结点，打开我们可以看到他 include 了其他的 launch 文件。
- move_group.launch :顾名思义，move group 的功能是让一个规划组群动起来。默认的是使用 ompl 运动规划库。其他的都是设置一些基本参数，暂时可以略过。
- planning_context.launch :这里我们可以看到，定义了所使用的 urdf 和 srdf 文件，以及运动学求解库。不建议手动更改这些，但是如果你需要使用不同的 urdf，srdf，可以在这里更改。
- setup_assistant.launch : 如果你需要更改一些配置，那么可以直接运行。

配置验证

进入配置文件所在的工作空间，执行以下命令

```
cd ~/dofbot_ws/  # 进入工作空间
```

```
catkin_make    # 编译
source devel/setup.bash #  更新系统环境
roslaunch dofbot_config demo.launch #     启动 ROS 节点
```

- 示例图片

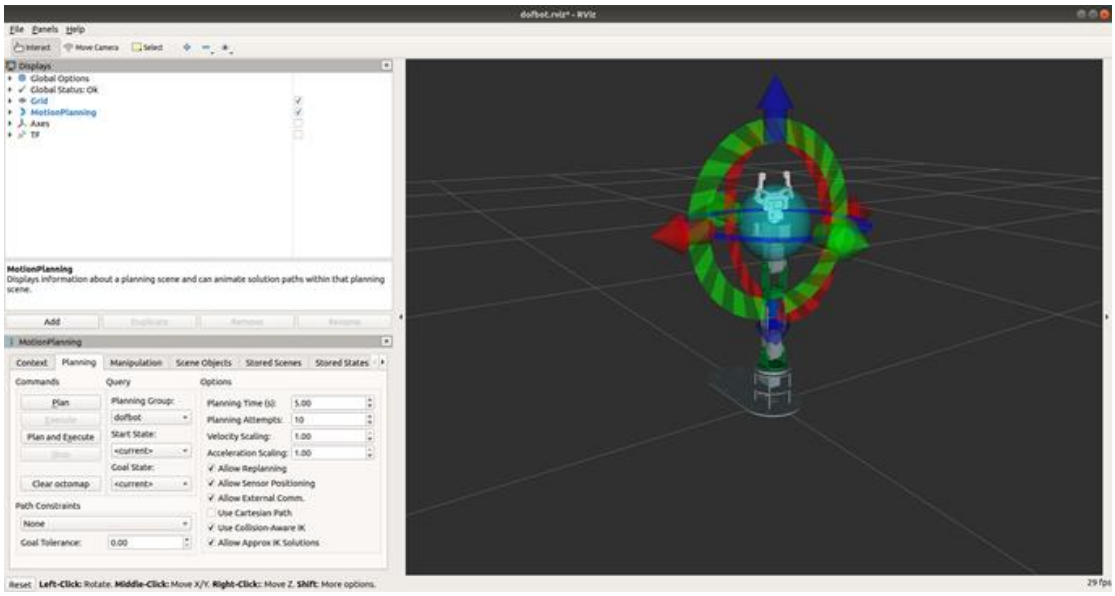


图 9.7.15 示例图片

10、物联网综合实验

待添加